从第一步讲起

数据结构

超级纲要 · 教材式初稿

概念 · 推导 · 原图 · 例题 · 自测

例题保留数据的中间状态与公式的推导步骤。
原始课件图标明出处，补充示意按正文模型解释。

2026-10-25 · sTBook
Wiki 与 Typst 共用正文 · 原图均在图注中注明出处

章节目录

从一段能亲手算完的程序开始 3
1 让一段程序从头到尾都有说得清的理由 5
2 把程序实际做的动作数出来 10
3 递归先回来一次，动态规划再填下一格 16
4 向量：格子不够时，整排搬到哪里 22
5 二分查找：每次删去半段，凭什么没删掉答案 30
6 排序：每次比较以后，数组究竟变成什么 39
7 链表：改一根指针，究竟改了谁通向谁 52
8 栈：把没做完的事留好，先处理最近的一件 59
9 队列：保持先后次序，也学会丢掉已失效的候选 72
10 二叉树：走进一条分支以后，怎样找回没做完的事 82
11 Huffman 编码：为什么总把最轻的两个合在一起 88
12 搜索树：每次比较之后，往哪边走 94
13 伸展树、B 树、红黑树与跳转表 108
14 搜索一段范围，为什么不能只照搬找一个键 126
15 散列表：算出一个位置以后，还要确认是谁 134
16 图：把“谁和谁相连”变成可追踪的状态 141
17 最短路和最小生成树，究竟在最小化什么 149
18 堆：只保证冠军站在最上面 160
19 字符串匹配：已经比对过的字，怎样不再重读 172
20 划分、选择与步长：排序还有哪些做法 191
21 PA1：把题目变成能一步步执行的程序 199
22 PS01：把“会背复杂度”变成“自己能数出来” 208
23 PS02：向量里的每一次搬动，都要知道搬了谁 221
24 PS03：节点身份、接线顺序与排序成本 234
25 PS04：先把栈里的东西写出来，再谈递归、表达式和单调结构 244
26 PS05：树的箭头、遍历顺序和编码成本，一步一步核对 256
27 一次完整的调试：从错误输出，找到第一步走歪的位置 269
28 把证明里省掉的那一步写出来 274
29 题目没有告诉你用什么结构时，先算一个小输入 280
30 PS04 的 21 个表达式：把每一次压栈、弹栈写全 285
31 换掉数字以后，仍然能重做的六种推理 304
32 习题 1-32：把十六段程序逐层数清楚 309
33 教材、习题解析、PS 与 PA，怎样对着解决同一个问题 318
资料覆盖、原图出处与使用说明 329
读图、读程序和读公式时怎样查符号 331

从一段能亲手算完的程序开始

数据结构研究怎样保存数据，以及怎样在上面完成查询、插入、删除和计算。先从一排数开始，不要求你预先记住“摊还”“不变量”“递归树”等名字。第 01 章会把数组编号、变量赋值、条件判断和重复执行都放进一个具体例子中解释。

先试着保留计算过程

给出三个数 4、1、7，求总和。心算得到 12 很快，但程序还需要一份明确的执行顺序。准备一个记录当前总和的变量 s，先存 0；每次读下一项，用“旧 s 加当前项”得到新 s。

当前动作	读到什么	计算	动作后的 s	接下来做什么
初始化	暂不读取	把 0 放进 s	0	读第一项
处理第一项	4	0+4	4	读第二项
处理第二项	1	4+1	5	读第三项
处理第三项	7	5+7	12	没有下一项，返回 12

这里保留的不只是答案，还有“现在走到哪里、已经保存什么”。把后两行遮住，从 s=4 的位置继续，便能检查自己是否理解更新顺序。后面学习栈、树和图时，状态会更多，读法相同：先认出当前数据与控制位置，只执行本步允许的动作，再检查新状态。

“为什么不会漏掉一项”也需要回答。每次都读紧接着的一项，读完向后推进，直到没有项为止；如果没有推进，就会反复读 4；如果允许读过末尾，就会访问不存在的格子。第 01 章把这些具体观察整理成正确性与终止性证明，术语会在证明需要它们时出现。

正文、图和演示怎样配合

章节里的操作图先给具体输入，再画中间状态。数组格内是当前值，格下数字是位置编号。栈会注明哪端是顶部；树会注明父子关系、空孩子及颜色等特殊约定。比较一项而没有改写数据时，图仍会保留这次比较，因为它决定下一步能否移动。

网页中标为“逐步演示”的图可以前后翻步、播放或拖动时间轴。每步下面分别说明做了什么、为什么可以这样做。预测提示可在继续前展开核对；它不记录你是否已经掌握。播放器下方可以展开完整静态过程。PDF 与 Typst 保留同一份步骤，离线阅读也不会少掉中间图。

教师原图保留整页，正文会指出要看哪根箭头、哪一行表、符号怎样对应。网页可以点击原页放大。若原页与本书使用不同实现，例如插入前分裂与溢出后分裂，会说明哪套中间状态属于哪套规则。两个过程不能只看最后都排好，就混着执行。

碰到公式时，先知道它在数什么

例如要数前三轮分别执行 1、2、3 次的总操作数，可以直接写 1+2+3=6。缩写成求和号就是

$$\sum_{i=1}^{3} i = 1 + 2 + 3 = 6.$$

这里 i 依次取 1、2、3，下面的 1 为起点，上面的 3 为终点，符号 Σ 表示把各项相加。公式只是把已经说明白的重复过程压缩成记号；若还不知道每一项代表什么，先回到具体执行表。第 02 章会从这些小计数推到一般 n，不直接要求背结果。

每个例子会说明输入规模和成本模型。例如 n 可表示数组项数；处理有很多位的大整数时，一次加法的成本还随位数改变。“一个算术符号”不总等于“机器一步”。正文第一次引入变量时局部说明，忘记后可回查符号与术语。

按学习问题找到章节

第一次顺读 01—10：从程序正确性和计数，到递归、动态规划、向量、查找与排序，再到链表、栈、队列与二叉树。每一种结构都从实际操作学起，复杂度跟在过程后面解释。

11—20 继续讨论编码、搜索树、范围查询、散列、图、堆、字符串匹配和进一步的排序。第 13 章中的伸展、B 树、红黑树分别保留自己的容量或颜色规则；读一种时可先把这一种的完整例子走完，再比较另一种。高级势能证明放在基本操作之后，不需要先吞下所有数学符号才能看旋转。

PA 五个主题在 21 章，每题先解释题目要维护的状态，再连到相应主干。五份 PS 在 22—26 章，保留原题编号；遇到陌生方法，按题内链接回到完整讲解。32 章逐段拆《习题解析》1-32 中的程序，33 章说明教材、习题、PA 怎样交叉阅读，95 章列来源与实际覆盖边界。

题目中的“为什么”应对应一项能检查的事实。例如二分删半段要依赖有序性，栈弹出某个候选要知道它以后为何不再有用。自己复演时若这一步说不清，就停在这个状态看理由。继续多读几页不会自动填上它。

1 让一段程序从头到尾都有说得清的理由

桌上有四张纸条，依次写着 6、−2、5、1。要把它们相加，可以先算$6-2=4$，再算$4+5=9$，最后加 1 得到 10。程序也要完成这些动作。我们得告诉它每一步读哪里、把结果放哪里、下一步去哪里。

1.1 一排格子与格子之间的边界

数组是一排有编号的格子。下面这一排叫 a，长度是 n=4。编号叫下标，本书从 0 编号，因此“第一个元素”下标 0，“第四个元素”下标 3。

```
下标       0     1     2     3
数组 a   [ 6 ][ -2 ][ 5 ][ 1 ]
          第一项              第四项
```

a[2]是“读取下标 2 格里的值”，读到 5；它不是“寻找数值 2”。长度 4 时，合法读取下标只有 0、1、2、3，a[4]已越过末尾。

区间[lo,hi)的左端包含、右端不包含。例如[1,3)包含下标 1、2 两项，[0,4)包含全部四项，[2,2)没有项。项数统一是 hi−lo。右边界可以等于 4，但不能因此读取 a[4]。边界像格子之间的门缝，最后一格后面也有门缝，门缝本身没有元素。

任务还要说完整：输入是一串整数，允许空串；输出数学意义上的总和，空和定义为 0。实际语言的整数类型必须容纳中间结果，C++固定宽有符号整数溢出不能当作合法数学加法。下面先用可自动增长整数的 Python 观察步骤，大整数越来越长时的单次加法成本另计。

1.2 用手当一次计算机

两个变量就够：i 是“下一项的下标”，s 是“已经加进去的总和”。赋值会用新值替换变量盒子里的旧值。

```python
def total(a):
    n = len(a)
    i = 0
    s = 0
    while i < n:
        s = s + a[i]
        i = i + 1
    return s
```

len(a)取得项数；while 在每次进入缩进主体前判断条件；return 把结果交给调用者。s=s+a[i]先用旧 s 算右边，再写回新值。这是更新指令，不是要求两边恒等的代数方程。

步骤	进入前的判断	读取	写入 s	写入 i	此后已完成什么
初始化	还没判断	不读数组	0	0	尚未处理任何项
第 1 轮	0<4，真	a[0]=6	0+6=6	1	前 1 项
第 2 轮	1<4，真	a[1]=−2	6−2=4	2	前 2 项
第 3 轮	2<4，真	a[2]=5	4+5=9	3	前 3 项
第 4 轮	3<4，真	a[3]=1	9+1=10	4	前 4 项
结束检查	4<4，假	不读 a[4]	保留 10	保留 4	返回 10



图 1　求和的已处理前缀与下一项

绿色格已经计入 s，橙框是本轮将读取的位置。最下行没有橙框，因为 i=4 已经指向尾后边界。逐行把绿色格子的值相加，应与右边 s 相等。盖住上表的“写入 s”一列，再自己算一遍，能检查是否真正跟上了变量更新。

若条件错写成 i<=n，最后还会进入一次并读取 a[4]。若删掉 i=i+1，就永远反复加 6。前一个错误破坏合法下标，后一个错误破坏向结束推进的动作。

1.3 一个例子对了，怎样知道任意长度都对

每次进入循环前都有同一条关系：0<=i<=n，而 s 恰好是下标小于 i 的全部元素之和。这叫循环不变式。“不变”指关系保持，变量数值仍会变。

开始 i=0,s=0，前 0 项的和确为 0。假设本轮开始关系成立，加入紧接着的 a[i]并把 i 加 1，新 s 便是新的前 i 项之和。结束时条件假，得到 i>=n；关系又保证 i<=n，所以 i=n，s 就是全部元素之和。这三步分别检查开始、保持、退出，不能只说“显然一直正确”。

关系成立还不保证会退出。这里再看未处理项数 n-i：它是非负整数，每轮严格减 1，最多减 n 轮就到 0。单说“越来越小”不足以证明终止，$1, \frac{1}{2}, \frac{1}{4}, \ldots$也越来越小，却有无穷多项。

“如果结束，结果对”叫部分正确性；“一定会结束”叫终止性。前缀关系负责前者，未处理项数负责后者。空数组也受同一证明覆盖：初始 i=n=0，不执行主体就返回 0。

1.4 最大值不能照抄求和的初始化

对[-8,-3,-5]求最大值，若设 best=0，结果会是输入中不存在的 0。非空输入应先接受第 0 项为当前答案。

```
def maximum(a):
    if not a:
        raise ValueError("空序列没有最大元素")
    best = a[0]
    i = 1
    while i < len(a):
        if a[i] > best:
            best = a[i]
        i += 1
    return best
```

下一下标	比较前 best	本轮比较	写不写 best	完成后的最大值
1	−8	−3>−8，真	写−3	前 2 项最大−3
2	−3	−5>−3，假	不写	前 3 项最大−3
3	−3	3<3 为假	不读数组	返回−3

从 i=1 开始没有漏项，第 0 项已由初始化处理。新的不变式是“best 等于已处理前缀的最大值”。若接口希望空输入不抛异常，可以返回一个明确的“没有结果”，不能偷换成 0。

对普通无序输入，最大值必须至少读取每个元素一次。若某格没读，把它改成极大的数，执行过程看不到变化，正确答案却变了。这个反例约束所有符合模型的算法，是问题下界；计数眼前程序则只给这一个算法的成本。

1.5 辗转相除丢掉了什么，又保留了什么

48 和 18 的最大公因数是同时整除两者的最大正整数。因为$48 = 2 \times 18 + 12$，能整除 48 和 18 的数一定整除 12；反过来能整除 18 和 12 的数也整除 48。两对数公因数集合相同，可以替换。

当前 a,b	计算余数	下一对	何时结束
48,18	48%18=12	18,12	余数非零，继续
18,12	18%12=6	12,6	继续
12,6	12%6=0	6,0	下一轮判断 b 为 0
6,0	不再做除法	返回 6	最大公因数为 6

```
def gcd(a, b):
    # a,b非负，且不同时为0
    while b != 0:
        a, b = b, a % b
    return a
```

同时赋值先算右边的两个旧值再写回。若在别的语言里先 a=b，再 b=a%b，旧 a 已经丢失，会换掉算法。每轮余数满足 0<=新 b<旧 b，非负整数 b 严格下降，所以终止。对数轮数还需要进一步的减半论证，不能仅凭“变小”推出。

“输入满足的条件”称前置条件，“输出承诺满足的关系”称后置条件。求和允许空数组、最大值拒绝空数组，就是两份不同前置条件。后面二分要求有序，若给乱序数据，证明不能继续沿用。

欧几里得算法的轮数也能具体夹住。若一轮开始 a>=b>0：当 b<=a/2 时，下一对的大数已经不超过旧 a 的一半；当 b>a/2 时，商为 1、余数 a−b<a/2，再下一轮的大数便小于旧 a 的一半。所以至多两轮大数至少减半，才得到对数轮数界。这里每次除法仍按一次基本操作计，精确大整数的位成本另计。

1.6 机器模型：什么算“一步”

分析数组扫描时，我们常把读 `a[i]`、比较和普通整数加法各算常数次。这对应 RAM 模型的一个版本：内存单元能按地址直接访问，每个机器字能容纳下标及通常的数据。该模型适合解释数组与链表等结构的操作成本。

若有一个长度为一百万位的整数，把它和另一个同样长的整数相乘，不能继续说“一条乘法指令，所以 O(1)”。普通处理器一次只能处理有限位数，大整数库需要许多字操作。某些习题明确假设无限字长、移位为单位成本；可以在该模型下求答案，但不要直接把模型结论当作真实 C++的运行时间。

输入大小也要说清。程序枚举 1 到 N 用了 Θ(N)次；可是把 N 以二进制写给程序，只需大约 $\log_2 N$ 位。令输入位数为 `b`，N 可以接近 `2^b`，于是这个算法按输入位数可能是指数时间。一层循环照样可能很慢。

1.7 随机过程与通用停机判定

每轮独立掷公平硬币，正面就停止。恰好第 k 轮停止的概率是 `2^(-k)`，永远不出正面的概率为 0，期望轮数为 2。但对于任意有限的上限 m，连续 m 轮反面仍有正概率 `2^(-m)`。所以“概率 1 终止”与“有一个统一有限最坏轮数”是不同的承诺。

一般程序的终止性没有万能判定器。假设有程序 H 能对任何程序 P 及输入 x 判断会不会停机。构造 D：先问 H“D 对自身输入会停吗”；H 说会停，D 就无限循环；H 说不会停，D 就马上结束。无论 H 回答什么都会错。这说明不能对所有任意程序实现这样的完美判定器，并不妨碍我们对求和、二分和欧几里得算法分别给出证明。

1.8 再看一次机器：二进制加一怎样在纸带上执行

图灵机把数据写成带子上的符号，读写头一次观察一个格子，并根据当前状态决定写什么、向哪移动、换什么状态。它不会一次看见整串数字，所以“加一”也必须拆成小动作。假设二进制串最高位在左，头开始在最右位。加一时，遇 0 就改 1 并结束进位；遇 1 就改 0，向左继续；若一直越过最高位，则在左侧空格写 1。

对 1011 加一：最右 1 改 0，状态变为仍有进位；再遇 1 改 0；再遇 0 改 1，进位消失，剩下最左 1 不动，结果 1100。对 111 加一，会把三个 1 都改 0，左侧空格写 1，得 1000。对 1000 加一，只需把最后 0 改 1，得 1001。如果要求执行结束后读写头回到某个固定起点，还要把移动回去的成本计入。

这个例子说明，数学上一个“+1”与某台机器的一条指令并不总是同一件事。长度 b 的二进制数最坏有 b 个连续末尾 1，进位会走 Θ(b)格；RAM 字内加法常被模型视为 O(1)，是把有限字长硬件操作作为基本单位。两种分析描述不同模型，不矛盾。

减一也可以自己设计：遇 1 改 0 并结束借位，遇 0 改 1 并继续向左。1000 减一，三个末尾 0 依次改 1，再把最高 1 改 0，得到 0111，按表示约定去前导 0 后为 111。输入 0 若仍要求非负整数，则没有合法结果，应由接口拒绝，不能无限向左借位。

把 1011 加一时的读写头状态展开，最高位下标 0、最右位下标 3：

步骤	头在哪	读到	写后纸带	下一动作
开始	3	1	1010	仍进位，向左到 2
下一步	2	1	1000	仍进位，向左到 1
下一步	1	0	1100	进位消失，结束改写
需要复位时	1	不再改值	1100	另按接口移动回指定起点

减一的 1000 则从最右下标 3 开始：读 0 写 1 得 1001；到 2 读 0 写 1 得 1011；到 1 读 0 写 1 得 1111；到 0 读 1 写 0 得 0111，借位消失。中间暂时得到 1111 并不是最终答案，要看借位状态是否结束。输入 0 在非负整数域应拒绝。

1.9 当“验证答案”比“找答案”容易

给出整数 221，想分解成两个大于 1 的因子。找到 13×17 可能要试许多候选，但别人给出 13 与 17 后，乘一下就能核对 221。验证某一份候选只需检查它满足的关系；验证算法对所有输入正确，还要说明搜索不会漏掉任何可能因子并会终止。

朴素试除只需到平方根：若 N=ab 且 a,b 都大于√N，则乘积会大于 N，矛盾，所以合数至少有一个因子不超过√N。对 221 试 2、3、5、7、11、13 时找到 13，再除得 17。若只试奇数，仍必须单独检查 2。素数输入会一直试到√N 附近，成为这种实现的坏情况。

若 N 以 b 位二进制给出，√N 可接近 `2^(b/2)`，所以试除按数值看是平方根级，按输入位数看仍可能指数。这个具体问题把输入编码、基本操作成本与最好最坏情况连在一起：重复除 2 很快的偶数，不能代表难分解的奇数实例。

测试中找到一份因子可以证伪“它是素数”；测试许多候选都没找到，只在你已经覆盖全部必要候选后才能推出素数。算法证明中的覆盖范围，决定“没找到”能不能变成一个可靠答案。

对 221 试除的具体余数为：除 2 余 1，除 3 余 2，除 5 余 1，除 7 余 4，除 11 余 1，除 13 余 0。找到 13 后取商 17，再检查 17 没有不超过其平方根的因子 2、3，于是得到 13×17。完整试除程序需要维护剩余数并允许同一因子反复整除，例如 12 要得到 2、2、3，不能找到一个 2 就跳过它。

1.10 自己做，再核对

练习 1。下面的循环想把 a 原地倒置：`i=0,j=n-1; while i<j: swap(a[i],a[j]); i+=1; j-=1`。给出不变式与下降量，并追踪`[1,2,3,4,5]`。

解答：保存一份只用于证明的原数组 b。在每轮开始，i 左边的位置都已满足 `a[k]=b[n-1-k]`，j 右边的位置也满足同样关系，中间`[i,j]`还待处理。第一次交换后数组为`[5,2,3,4,1]`，i=1,j=3；第二次为`[5,4,3,2,1]`，i=j=2。中间只剩一个元素，它本来就在对称中心。下降量可取尚未确定的元素数 j−i+1，执行一轮就减少 2；只在 i<j 时执行，因此不存在无限下降。n=0、n=1 都不进入循环。

练习 2。程序将正整数 x 反复执行 `x=x//2`，直到 x=0。有人说“每次至少减 1，因此时间 Θ(x)”，哪里不够？

解答：至少减 1 只能给出 O(x)上界，不能证明线性下界。取 x=20，状态依次 20、10、5、2、1、0，共 5 轮。执行 k 轮后为 $\lfloor x/2^k \rfloor$；首次为 0 发生在 $k=\lfloor \log_2 x \rfloor +1$，因此时间 Θ(log x)。终止证明与紧确复杂度证明所需的信息量不同：知道会变小足以证明结束，算多快要知道每次缩小的比例。

2 把程序实际做的动作数出来

先决定数什么：执行一次 mark(i,j)算一次主体操作，暂不换算秒数。固定字长的数组读取、加法、比较通常能当常数成本；一百万位整数乘法不能套同一单价。先数小例子，再把规律写成任意规模的式子。

2.1 两层循环先画实际访问的格子

```
for i in range(n):
    for j in range(i):
        mark(i, j)
```

range(n)给 0 到 n-1；range(i)给 0 到 i-1，不含 i。当 n=5，一次调用画一个点：

```
i=0   没有j
i=1   (1,0)
i=2   (2,0) (2,1)
i=3   (3,0) (3,1) (3,2)
i=4   (4,0) (4,1) (4,2) (4,3)
本行次数 0、1、2、3、4；合计10
```

内层上界随 i 变化，所以不是5×5次。数学用求和号$\sum$缩写逐项相加：$\sum_{i=0}^{n-1} i$的下方规定从 0 开始，上方规定在 n-1 结束，右边规定每次加当前 i。代入 5，展开就是$0+1+2+3+4$。
一般和式可以自己推。正着写一排、倒着写一排：

```
正排   0      1      2     ...     n-1
倒排   n-1    n-2    n-3   ...     0
列和   n-1    n-1    n-1   ...     n-1
```

共有 n 列，两排的总和为$n(n-1)$；只要一排，于是

$$T(n) = \frac{n(n-1)}{2}.$$

T 是总次数函数，n 是输入规模。$T(5) = 5 \times 4/2 = 10$与点图相符。循环条件另有判断成本，会增加低阶项，不改变这里的主要增长。

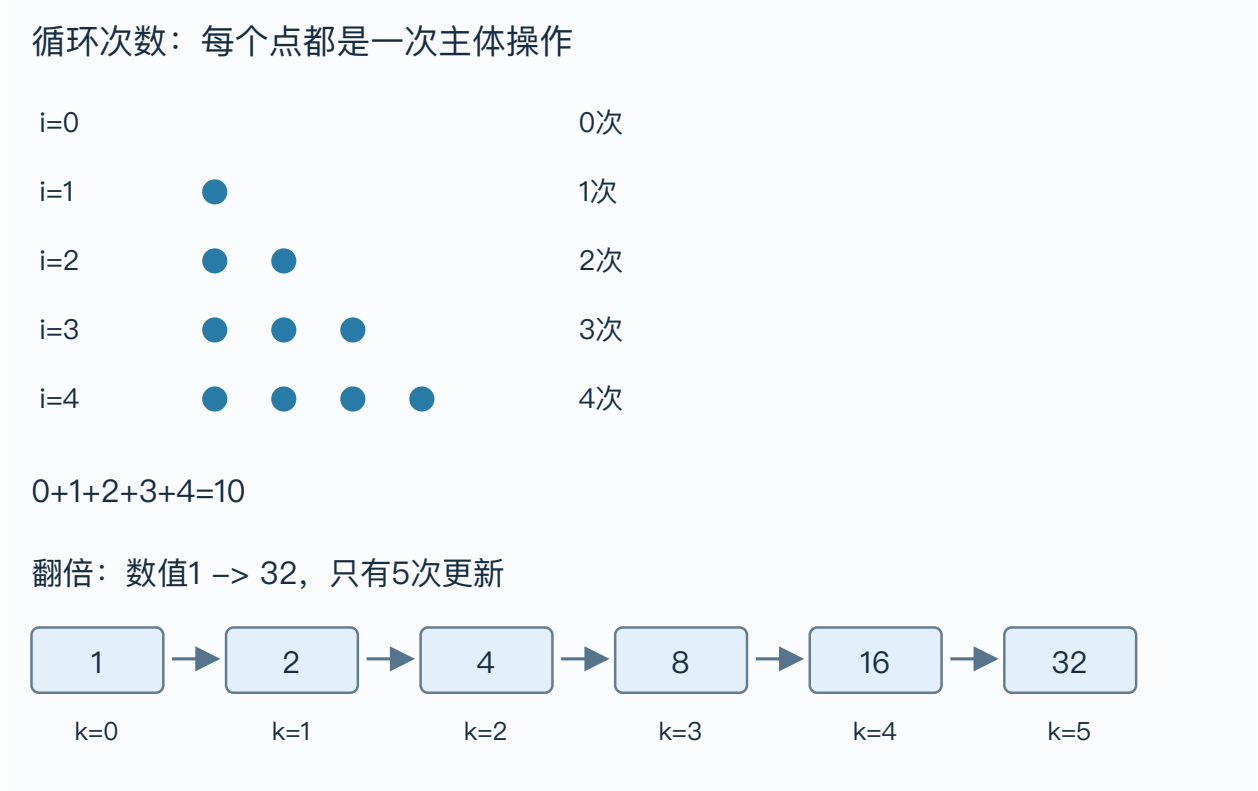


图 2　三角形计数与翻倍状态

左边每点是一次实际调用，不是输入的一项。右边每框是变量状态，不是本轮操作次数。读复杂度图要先认清画的是数据、步骤还是累计成本。

2.2 对数是反过来问“乘了几次”

```
i = 1
while i < n:
    i *= 2
```

已经乘 2 次数 k	当前 i	当 n=20 的判断	动作
0	1	1<20，真	写 2
1	2	2<20，真	写 4
2	4	4<20，真	写 8
3	8	8<20，真	写 16
4	16	16<20，真	写 32
5	32	32<20，假	停止

2^k表示 2 连乘 k 次，$2^0 = 1$。$\log_2 n$反过来问“2 的几次方等于 n”，例如$\log_2 8 = 3$。20 在 16 和 32 之间，对数在 4 和 5 之间，实际只能做整数轮，因此向上取整为 5。$\lceil x \rceil$表示不小于 x 的最小整数。正整数 n 的轮数为$\lceil \log_2 n \rceil$；n=1 恰好 0 轮。

若每轮另外做 i 次动作，同一输入的账单变成$1 + 2 + 4 + 8 + 16 = 31$，不能仍答 5。前面 1 到 8 合计 15，比末项 16 还少，所以几何增长的总和小于末项两倍，总工作随 n 线性增长。轮数与每轮工作是两笔不同的账。

2.3 步长越大，内层次数越少

```
i 从1到n:
    j=0
    当j<n:
        mark(i,j)
        j=j+i
```

n=8 时把 j 的每个实际值列出：

i	j 的值	次数
1	0,1,2,3,4,5,6,7	8
2	0,2,4,6	4
3	0,3,6	3
4	0,4	2
5	0,5	2
6	0,6	2
7	0,7	2
8	0	1

合计 24。一般第 i 行有$\lceil n/i \rceil$次。定义调和数$H_n = 1 + \frac{1}{2} + \cdots + \frac{1}{n}$。每次向上取整比原数多不足 1，n 项累计多不足 n：

$$nH_n \le \sum_{i=1}^{n} \lceil n/i \rceil < nH_n + n.$$

机器仍只执行整数次，分数只是把整数计数夹住的估计。
把H_n按分母分组：1；2 到 3；4 到 7；8 到 15。完整第 k 组有2^k项，每项在$1/2^{k+1}$与$1/2^k$之间，组和在 1/2 与 1 之间。组数随$\log_2 n$增长，所以$H_n = \Theta(\log n)$，总工作为$\Theta(n \log n)$。

2.4 再给增长快慢取名字

$\Theta(n^2)$说成本被 n^2的两个正常数倍夹住；$O(n^2)$只给上界；$\Omega(n^2)$只给下界。常数不许随 n 变，允许先略过有限个小规模。
例如$T(n) = 3n^2 + 7n + 20$。当 n≥1，$7n \le 7n^2$且$20 \le 20n^2$，因此

$$3n^2 \le T(n) \le 30n^2.$$

上下界匹配，所以$\Theta(n^2)$。它也属于$O(n^3)$，但不能据此说“实际恰好立方”。
正式的$T(n) = O(g(n))$表示：存在固定正常数 c 和阈值 n_0，所有 $n \ge n_0$均有$T(n) \le cg(n)$。g 是比较函数，c 吸收固定单价，n_0让有限个小例外不影响增长分类。
大 O 不自带最坏含义。可以研究最好输入、最坏输入、指定分布下的期望成本，再分别给界。摊还对一串操作算总账，不靠随机输入，第 04 章会展开。

2.5 调用树不能只数层数

设长度 8 分成两个 4，再分成四个 2，直到单项，每个非叶任务合并时要处理当前段每一项：

```
第0层    1个长度8任务        本层合并工作8
第1层    2个长度4任务        本层合并工作4+4=8
第2层    4个长度2任务        本层合并工作2+2+2+2=8
第3层    8个单项任务         基础情况合计8
```

8 有 3 层非叶合并；$n = 2^k$有 k 层。每层工作与 n 成比例，总时间$\Theta(n \log n)$。递推式

$$T(n) = 2T(n/2) + \Theta(n)$$

中的 2 是子调用数，n/2 是各子问题规模，最后一项是当前调用自己的工作。

若每个节点自己只做常数动作，层账单改成 1、2、4、8，总数与 8 同阶，时间变成$\Theta(n)$。若只递归一半、自己扫 n 项，账单是 8、4、2、1，也线性。每个符号都要能指向代码中的一项工作。

2.6 常见级数，用状态变化来认

等差和 1+2+…+n 对应每轮多做一点，增长为 Θ(n²)。几何和 1+2+4+…+2^k 由最后一项的常数倍控制，若最后一项约为 n，和就是 Θ(n)。平方和 $1^2+2^2+\dots+n^2$ 为 Θ(n³)：上界 n×n²，下界后 n/2 项每项至少(n/2)²。一般固定 p≥0 时，Σ i^p=Θ(n^(p+1))。

倒数平方和 $\Sigma\ 1/i^2$ 却有常数上界。按 2 的幂分组，第 k 组最多 2^k 项，每项至多 1/2^(2k)，该组不超过 1/2^k；所有组再相加，是收敛的几何级数。因此看到求和号，不能一律乘一个 n。

对数和 Σ log i 等于 log(n!)。上界 n log n；后半 n/2 项每项至少 log(n/2)，所以为 Θ(n log n)。后面比较排序的下界会再次出现这个数量。

2.7 主定理是一个有条件的工具

递推式$T(n)=aT(n/b)+f(n)$规定：每个任务递归出 a 个孩子，孩子规模为原来的 1/b，自己另做 f(n) 工作。a>=1、b>1 都是固定常数，基础小规模成本为正常数。这里先忽略取整，最后再说明整数子问题。令$\alpha=\log_b a$：深度约$\log_b n$，叶子数为$a^{\log_b n}=n^{\alpha}$。

最容易核对的情形是$f(n)=\Theta(n^q)$，q 是固定非负常数。第 k 层有a^k个任务，每个工作约$(n/b^k)^q$，本层为$n^q(a/b^q)^k$。于是只是在加一个几何级数：

条件	层账单向下怎样变	总成本
q<α，即 a/b^q>1	越来越大，由叶侧控制	$\Theta(n^{\alpha})$
q=α，即 a/b^q=1	每层同阶，乘上层数	$\Theta(n^{\alpha}\log n)$
q>α，即 a/b^q<1	越来越小，由根侧控制	$\Theta(n^q)$

更一般的三情况主定理要写清固定正间隔$\varepsilon>0$：若$f(n)=O(n^{\alpha-\varepsilon})$，得$\Theta(n^{\alpha})$；若$f(n)=\Theta(n^{\alpha})$，得$\Theta(n^{\alpha}\log n)$；若$f(n)=\Omega(n^{\alpha+\varepsilon})$且存在固定 c<1 使$af(n/b)\le cf(n)$最终成立，得$\Theta(f(n))$。最后这个不等式叫正则条件，它保证下一层总账能按固定比例下降。只差一个对数因子未必满足前两种多项式间隔，不能把任何“比它小”都套进第一行。

例如 2T(n/2)+n 符合相等情形，得 n log n；4T(n/2)+n，叶子规模 n²大于 n，得 n²；2T(n/2)+n²，本层为平方级，且 2(n/2)²=n²/2 满足正则条件，得 n²。

T(n)=T(n/2)+T(n/4)+1 不符合“所有孩子大小都为 n/b”的形式，不能把两个孩子硬说成 2T(n/2)。32 章会把 n=2^k 后转成 Fibonacci 递推，得 Θ(n^(log₂φ))。更一般的 Akra–Bazzi 方法可处理不同比例，但其应用同样有连续性、增长与扰动范围等条件；不能把“任意递推都可套”当成结论。

Princeton《Algorithms》算法分析一节把“单价”和“执行频率”分开，并用规模翻倍实验检查模型。这里也能做这样的检验：若主体成本恰是 n²，把输入翻倍，主要操作数应约变 4 倍；若是 n 则约 2 倍。实验的机器、输入生成和计费操作需要固定；短输入的计时噪声不能代替增长证明。

2.8 时间之外，再数同时活着的空间

递归调用树可能总共有 n 个节点，但不会所有调用同时占着栈。深度优先执行时，一条从根到当前节点的路径上的调用尚未返回。二分递归若每层只存常数局部变量，调用栈是 O(log n)。线性递归每次 n−1，栈深可达 O(n)。

归并排序通常还需要一个长度 n 的辅助数组，空间为 O(n)，这比 O(log n)调用栈大。若在每层都复制子数组，某些语言的切片还会带来额外分配和复制；复杂度分析必须对应实际实现。

2.9 对数变换会保留哪些界

若 T(n)≤c f(n)，取对数得到 log T≤log c+log f。要把右侧写成 O(log f)，还需要 log f 最终为正且不会趋近 0，例如 f 最终至少为某个大于 1 的常数。若 f 恒等于 1，log f=0，而 T 恒等于 2 仍有 T=O(f)，但 log T=1 不能是 O(0)。边界条件不能在取对数时消失。

反方向也不成立：log T=O(log f)只给出 T≤f^c 乘常数，不能推出 T=O(f)。例如 T=n²、f=n，log T=2log n，确实同阶；T/f=n 却无界。指数函数会把对数域里的常数倍放大成幂次，这正是信息损失的位置。看到复杂度函数先取对数再比较，应把常数移回原空间重新检查。

小 o 记号比大 O 更强。f=o(g)表示对任意 ε>0，充分大时|f|≤ε|g|，常用正函数下相当于比值趋 0。n=o(n log n)，因为比值 1/log n 趋 0；n=O(n)成立，却不是 n=o(n)。小 ω 反过来表示比值趋无穷。符号相似，但严格增长与常数倍同阶不能混为一谈。

2.10 代入法怎样补一个递归树的直觉

递归树给出 T(n)=2T(n/2)+n 约为 n log n 以后，可用归纳验证上界。假设较小规模 T(m)≤c m $\log_2$m，再代入得 T(n)≤2c(n/2)$\log_2$(n/2)+n=c n $\log_2$n−cn+n。取 c≥1 便不超过 c n $\log_2$n，基础小规模再选足够常数覆盖。下界也能按每层至少 cn 工作与层数给出，不能只凭上界就写 Θ。

有时猜 T(n)=cn 不够。例如 T(n)=T(n/2)+n，代入 cn 得到 cn/2+n，要求 c≥2，可成立；但 T(n)=2T(n/2)+n 代入 cn 会变 cn+n，无论 c 多大都无法压回 cn，说明线性猜测不够。代入失败也可能只是猜式缺一个减常数或低阶项，需结合递归树判断。

非二次幂输入可以把数组分成 floor(n/2)与 ceil(n/2)，最长深度不超过 ceil($\log_2$n)，每层元素总数仍 n，所以同阶证明不依赖完美均分。写教材时先用 2 的幂看结构是简化推理，最后仍应交代一般整数规模。

2.11 封底估算要用增长比，而非只背秒数

假设每秒能做一亿次某种简单操作，这是估算假设，不是所有机器与所有指令的承诺。n=10^5时，n^2约 10^{10}，单靠常数优化通常救不了平方扫描；n $\log_2$n 约 1.7×10^6，则在许多场景里有余地。内存访问、字符串比较与分配比一个整数加法贵，估算应留余量。

指数枚举更要早算：2^{20}约一百万，2^{30}约十亿，2^{40}约一万亿。把 n=40 分两半，各 2^{20}项，能从不可行数量降到可处理列表，这是 PA Gift 的来由。它仍然指数，只是可承受规模区间被改变。

2.12 分数次成本从哪里来，机器明明只做整数次

调和和式里出现 n/i，不表示程序真的执行半次操作，而是用连续量夹住整数计数。若一次循环实际 ceil(n/i)次，便有 n/i≤次数<n/i+1。每项误差小于 1，n 项总误差小于 n，主项若为 n log n，这份误差不会改变阶。把取整界写出来，分数估计就与整数执行建立了联系。

若主项本来只有常数或对数，累计 n 个取整误差反而可能成为主导，不能在所有求和里都随手丢掉。分析 floor(n/i)时，也要保留每项至多 1 的误差，再看总和量级决定是否可忽略。渐近简化是证明之后的结论，不是先把麻烦符号全部删掉。

对数底数同样只在固定底数之间差常数倍：log_a n=log_b n/log_b a，a、b 为固定大于 1 常数，因而同 Θ 级。若底数随 n 变化，例如 log_n n=1，就不能再把它与 $\log_2$n 当同阶。32 章迭代对数也必须固定使用的底与停止阈值后才谈精确次数。

2.13 固定多项式间隔，比任何固定对数幂都强

比较 n log n 与 n 的一点零一次方，较小规模可能前者更大，但当 n 无界增长，log n 相对 n 的零点零一次方最终趋于零。因此前者渐近更小。阈值可能极其巨大，渐近优劣并不保证手边规模上的实际快慢。

将 n 写成二的 k 次方后，问题变成比较 k 与某个固定底数大于一的指数增长；指数最终超过线性或任何固定次多项式。这是主定理里“多项式地小或大”的来由：需要固定正指数间隔，而非只知道某项偶尔小一点。若比较 n 与 n 除以 log n，两者只差对数因子，不属于固定多项式间隔，不能直接塞进最简单主定理的第一种情况。

最后还要分清算法与问题。一个算法平方时间，并不证明这个问题必须平方时间；最大区间从立方降到线性就是反例。要给问题下界，必须约束所有符合模型的算法，而不是只分析手中这一段代码。

2.14 练习与完整答案

练习 1。 `i=0,j=1; while i<n: i+=j; j+=1` 执行几轮？列出 n=10 的状态。

解答：每轮结束后 i 依次 1、3、6、10，j 依次 2、3、4、5，共 4 轮。k 轮后 i=k(k+1)/2，所以最小 k 满足 k(k+1)≥2n，k=Θ(√n)。只看到“while i<n”无法决定次数；i 每轮的增量由另一个状态 j 控制。若 j 从 0 开始，则第 1 轮 i 不动，精确轮数不同，渐近阶仍为平方根。

练习 2。 比较 A：`T(n)=3T(n/3)+n`，B：`T(n)=T(n−1)+1`。哪一个调用树节点更多？哪个时间更长？

解答：A 每层总工作 n，约 $\log_3 n$ 层，时间 Θ(n log n)；它的满三叉调用树叶子约 n，节点总数 Θ(n)。B 是一条 n 层长链，每层常数工作，节点数 Θ(n)，时间 Θ(n)。两者节点数同阶，节点本身的工作不同，时间可以不同。A 的调用栈深 Θ(log n)，B 的调用栈深 Θ(n)，也再次说明总调用数与同时占用的栈深是不同指标。

练习 3。 有人对 `1+2+4+…+n` 写出 O(n log n)，理由是“约 log n 项，每项最多 n”。这个答案是否错误？

解答：作为上界，它成立；作为紧确增长级别，它太松。设 `n=2^k`，总和`=2^(k+1)−1=2n−1`，所以 Θ(n)。分析时允许先写一个松上界帮助判断，但最终题目若问渐近复杂度，应继续找匹配下界，或者给精确和式再化简。

3 递归先回来一次，动态规划再填下一格

把“调用自己”写进程序，不会自动让问题变简单。必须先说清：这次调用负责哪一段数据，交给更小调用什么任务，等它返回以后自己还要做什么。我们先用三项求和看一次完整的进入和返回，再去处理最大区间与最长公共子序列。

3.1 调用不是跳走不管，返回地点要留下

数组 a=[4,1,7]。sum_from(i)的任务是求从下标 i 到末尾的和；当 i=3 时那一段为空，返回 0。

```
def recursive_sum(a, i=0):
    if i == len(a):
        return 0
    rest = recursive_sum(a, i + 1)
    return a[i] + rest
```

执行 recursive_sum(a,0)时，还不知道 rest。当前调用先留下“等返回以后再加 4”，再进入参数 1 的新调用。每次调用都有自己的 i 和 rest，不是全体共用一个盒子。

事件	当前调用做什么	尚未完成的待办，从外到内	当前返回值
进入 i=0	请求 i=1 的结果	加 4	无
进入 i=1	请求 i=2 的结果	加 4；加 1	无
进入 i=2	请求 i=3 的结果	加 4；加 1；加 7	无
进入 i=3	空段，不再调用	加 4；加 1；加 7	0
回到 i=2	rest 取得 0，算 7+0	加 4；加 1	7
回到 i=1	rest 取得 7，算 1+7	加 4	8
回到 i=0	rest 取得 8，算 4+8	无	12



图 3　递归求和进入和返回的完整栈状态

图左逐层增长的是尚未返回的调用，不是新复制的数组。图右逐层撤去的是已经完成的调用。最内层先完成，所以待办的顺序与完成顺序相反。用 `i+1` 推进到空段保证终止；若基础条件错写成 `i>len(a)`，`i=3` 还会尝试读取 `a[3]`，先越界，根本到不了正确返回。

同一件事也能每次分两半求和。调用树总节点仍为线性量级，但同时等待的调用只沿一条根叶路径，最长约对数层。一次减 1 的版本栈深线性。分析空间数“此刻同时没返回多少层”，不是把历史调用总数全加起来。

3.2 最大连续区间，先弄清要选哪些东西

对`[-2,3,-1,4,-5,2]`，可选`[3,-1,4]`，和 6；不能把中间`-1`跳过。这里先要求区间非空。`[-4,-2,-7]`应答`-2`，不是 0。

蛮力法可以枚举起点 `l`、终点 `r`，再从 `l` 加到 `r`。例如固定 `l=1`，候选逐个是：

r	实际区间	重新加的式子	和
1	[3]	3	3
2	[3,−1]	3−1	2
3	[3,−1,4]	3−1+4	6
4	[3,−1,4,−5]	3−1+4−5	1
5	[3,−1,4,−5,2]	3−1+4−5+2	3

前一行的部分和被下一行重复计算。保留一个 `s`，每次只加新进入的 `a[r]`，就把每个固定起点的重复加法删掉。6 项共有$6+5+4+3+2+1=21$个非空区间；一般有$n(n+1)/2$个。重新求和最坏立方，固定左端复用和则平方。两种方法仍然检查同一批区间。

再把问题缩窄：只看“必须在当前项结束”的区间。令 `end` 保存这一类中的最大和，`best` 保存目前任意结束点的最大和。这两个数的条件不同：最佳区间可能在很早以前结束，不能拿 `best` 直接接当前项。

所有在 i 结束的非空区间只有两类：只取 a[i]；或者把在 i-1 结束的某段接上 a[i]。后一类当然只需保留和最大的前段。因此

$$end_i = \max(a_i, end_{i-1} + a_i).$$

下标 i 表示当前下标；a_i 就是 a[i]；max 表示取候选中的较大值。

i 与当前值	只取当前项	接上旧 end	选出的 end	best 比较	为什么
0，-2	-2	没有旧段	-2	初始化-2	非空，必须有一项
1，3	3	-2+3=1	3	max(-2,3)=3	抛弃负前段，重起
2，-1	-1	3-1=2	2	max(3,2)=3	接上 3 较好
3，4	4	2+4=6	6	max(3,6)=6	形成[3,-1,4]
4，-5	-5	6-5=1	1	max(6,1)=6	当前段变差，历史最佳不丢
5，2	2	1+2=3	3	max(6,3)=6	最终答案 6

每一行必须先用“旧 end”比较，再更新；不要提前把新 end 写进旧状态的计算。若还需输出端点，就在“只取当前项胜出”时记当前起点；best 更新时把该起点与当前末端一起保存。

```
def max_subarray(a):
    if not a:
        raise ValueError("非空区间要求输入非空")
    end = best = a[0]
    start = left = right = 0
    for i in range(1, len(a)):
        if a[i] > end + a[i]:
            end, start = a[i], i
        else:
            end += a[i]
        if end > best:
            best, left, right = end, start, i
    return best, left, right + 1
```

返回区间是半开[left,right+1)。这份代码并列时保留已有候选，不承诺“最短”规则。老师《01 算法概论》PDF93 页明确提出和相同时短者优先，若按该接口交作业，每次候选相等都必须再比长度，仍并列再按题目约定比位置。若允许空区间，则可把负的当前和丢成 0，但那是另一个输出约定。

3.3 Fibonacci：把同一小题反复做，才是浪费所在

定义$F_0 = 0, F_1 = 1$，其后每项是前两项之和。直接递归求 F(4)的调用如下：

```
F(4)
├─ F(3)
│  ├─ F(2)
│  │  ├─ F(1) → 1
│  │  └─ F(0) → 0
│  │  F(2)返回1
│  └─ F(1) → 1
│  F(3)返回2
└─ F(2)
   ├─ F(1) → 1
   └─ F(0) → 0
   F(2)返回1
F(4)最后返回2+1=3
```

参数 2 的相同问题做了两次，参数 1 做了三次。记忆化是加一张“参数→答案”的表，第一次做完存下，下次直接取。它减少重复计算，却仍可能有线性深度的调用栈。

若从 F_0、F_1开始向后算，就能按顺序填表：$F_2=0+1=1$，$F_3=1+1=2$，$F_4=1+2=3$，$F_5=2+3=5$，$F_6=3+5=8$。每步只用前两项，旧的更前项可以不用保存：

要生成	更新前 p,q	先算 t=p+q	再把 p,q 写成
F_2	0,1	1	1,1
F_3	1,1	2	1,2
F_4	1,2	3	2,3
F_5	2,3	5	3,5
F_6	3,5	8	5,8

一定先算 t，再同时推进两项；若先把 p=q，再算 q=p+q，就会把旧 p 丢掉。固定字成本模型下填表时间线性；精确大整数 F_n 有线性位数，单次相加会越来越贵，位成本不能仍写线性。

记忆化求 F(4)时，初始已知表只有{0:0,1:1}。依次请求 4（缺）、3（缺）、2（缺），再请求 1 命中 1、请求 0 命中 0，存 2:1；回到 3，请求 1 直接命中，存 3:2；回到 4，再请求 2 时直接命中 1，存 4:3。原调用树右边那整棵 F(2)子树就被一次查表替代。这种把小问题答案存下、再按依赖组合的做法属于动态规划；记忆化按需求进入，自底向上则预先安排计算顺序。

3.4 最长公共子序列：九格都亲手填一次

ABC 与 BAC 都能按原顺序读出 AC：第二串跳过开头 B。子序列允许跳字符，子串要求连续。现在求最长公共子序列长度，简称 LCS。

D[i][j]定义为第一串前 i 个字符、第二串前 j 个字符的 LCS 长度。这里 i,j 是长度，不是末字符下标。前 0 个字符是空串，任一方为空时长度都为 0，因此先填第 0 行、第 0 列。

格子	比较两个前缀的末字符	读哪些已知格	写入值
D[1][1]：A/B	不同	上 0，左 0	0
D[1][2]：A/BA	A 相同	左上 D[0][1]+1=0+1	1
D[1][3]：A/BAC	A 与 C 不同	上 D[0][3]=0，左 D[1][2]=1	1
D[2][1]：AB/B	B 相同	左上 D[1][0]+1=0+1	1
D[2][2]：AB/BA	B 与 A 不同	上 1，左 1	1
D[2][3]：AB/BAC	B 与 C 不同	上 1，左 1	1
D[3][1]：ABC/B	C 与 B 不同	上 1，左 0	1
D[3][2]：ABC/BA	C 与 A 不同	上 1，左 1	1
D[3][3]：ABC/BAC	C 相同	左上 D[2][2]+1=1+1	2

完整表如下，表头写的是“实际前缀”，能防止把下标与长度混淆：

第一串前缀 \ 第二串前缀	空	B	BA	BAC
空	0	0	0	0
A	0	0	1	1
AB	0	1	1	1
ABC	0	1	1	2

末字符不同，不能把它们同时作为同一公共序列的最后字符，至少舍弃其中之一，因此取上格、左格最大值；相同则存在最优解用这对末字符，取左上加 1。每格读上、左、左上，所以先填上方行，再从左向右填本行即可。

从右下恢复：在(3,3)记录 C，斜走到(2,2)；这里上左并列，若选上到(1,2)，A 相同，记录 A，斜走到第 0 行停止。记录顺序 C、A 要反转成 AC。若在并列处选左，会得到另一条同样长的 BC。表里只有长度，不能凭“总往上”保证字典序最小答案。

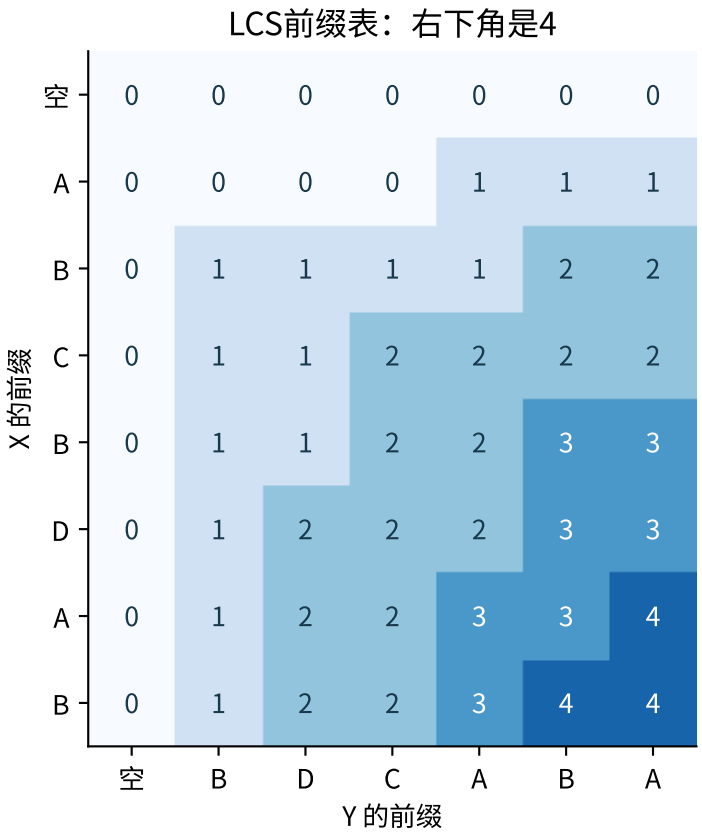


图 4　较大 LCS 实例的完整前缀表

这张原有示意换用了 ABCBDAB 与 BDCABA。每格仍是两个前缀的最长长度，右下 4；用刚才九格的规则可读任一格。若只要长度，保留相邻两行即可；要按格子回走，覆盖掉的旧行会丢失恢复路径，不能一边删表一边假定还能直接回溯。

3.5 同一个问题，分而治之怎样想

把数组切成左右两半，最大区间可能完全在左边、完全在右边，或者跨过中点。前两类交给递归；跨中点的区间一定由“左半的最大非空后缀”和“右半的最大非空前缀”拼成。各从中点向外扫一次，就能求出它。在原例中，切为 [−2,3,−1] 与 [4,−5,2]。左半最大后缀为 [3,−1]，和 2；右半最大前缀为 [4]，和 4，跨中点候选为 6。三类候选比较后得到 6。递推 T(n)=2T(n/2)+Θ(n)，成本 Θ(n log n)。它比一遍扫描慢一些，但训练了“分类覆盖全部可能解”的证明方法。

还能让每个子问题返回四项：总和、最大前缀、最大后缀、最大区间。左右结果合并只需常数时间，得到 Θ(n)的分治；这四项也正是后面线段树维护最大区间时会使用的摘要。选择状态时，要保留未来拼接所需的信息。

跨中点候选也逐项加给自己看。左半 [−2,3,−1] 从中点向左扩后缀：先 −1，和 −1；再加 3 得 2；再加 −2 得 0，所以最大后缀 2 对应 [3,−1]。右半 [4,−5,2] 向右扩前缀：4、−1、1，最大 4 对应 [4]。必须包含中点两侧紧邻项，才能叫“跨中点”；把左半内部最大 3 与右半最大 4 直接相加会漏掉夹在中间的 −1，得到错误 7。

3.6 动态规划的状态必须够用，也不能乱加

动态规划适用的共同结构是：同一个小问题会重复出现，而大问题的答案能由这些小问题按明确规则组合。状态要包含足够决定未来的信息。在最大区间里，“前缀最佳值”不够推下一格，因为最佳区间可能早已结束；所以另设“恰好在此处结束”。在 LCS 里，两个前缀长度足够，因为剩余问题只取决于这两个前缀，不取决于之前怎样跳字符。

状态一旦定义，就逐项检查基础值、转移覆盖、计算顺序与答案位置。若转移读取尚未算好的状态，要么调整顺序，要么明确使用记忆化递归。若依赖形成环，则不是直接按表从左到右就能解决。

3.7 从递归调用看“返回以后还要做什么”

数组求和也可以递归：sum(lo,hi)在空区间返回 0，否则将 a[lo]与 sum(lo+1,hi)相加。求[4,1,7]时，外层保存“返回后还要加 4”，下一层保存“还要加 1”，再一层保存“还要加 7”，到空区间返回 0，依次得到 7、8、12。计算步数线性，等待中的调用深度也是线性。

如果改成分两半，左右求和后相加，叶子仍是每个元素一次，工作总量 $\Theta(n)$，但栈最长路径只约 $\log_2 n$。调用树总节点数与某一时刻尚未返回的节点数不同，所以时间同阶、栈空间可以不同。切片若复制子数组，又会增加额外复制，必须按实际代码分析。

最大区间的四元组也可以从这份“返回什么”出发设计。对子数组保存总和 sum、最大非空前缀 pre、最大非空后缀 suf、最大非空区间 best。左右结果 L、R 合并：sum=L.sum+R.sum；pre=max(L.pre,L.sum+R.pre)；suf=max(R.suf,R.sum+L.suf)；best=max(L.best,R.best,L.suf+R.pre)。每项都对应一个完整分类。

对左段[−2,3,-1]，sum=0、pre=1、suf=2、best=3；右段[4,-5,2]，sum=1、pre=4、suf=2、best=4。合并总和 1，最大前缀 max(1,0+4)=4，最大后缀 max(2,1+2)=3，最大区间 max(3,4,2+4)=6。与线性扫描得到同一答案，而且这四项能继续与别的相邻段组合。

为什么单独保存 best 不够？段[5,-100]与[−100,5]的 best 都为 5，但它们连接右段[6]后的最大区间不同：前者为 6，后者可为 11。是否能跨边界接上，取决于后缀而不是内部最佳值。状态缺信息时，两个看似相同的状态会在同样未来输入下产生不同答案，这是判断状态不充分的直接反例。

3.8 编辑距离与 LCS，名字相近时重新读操作集合

A=ABC、B=ADC。LCS 为 AC，长度 2。只允许插删时，删 B 再插 D 需 2 步，公式 n+m−2L=2。若允许成本 1 替换，直接 B 改 D 就 1 步。两种动态规划都有二维前缀状态，但转移不同；不能因为都叫字符串相似度便共用答案。

对只插删距离 D，边界 D[i][0]=i、D[0][j]=j。字符相同允许对角免费，字符不同的对角替换不在操作集合内，不能写 D[i−1][j−1]+1。把允许操作逐个列出，再将每个动作翻译成一条转移，能避免无意换题。21 章 Filename 会利用预算小进一步只算带状区域。

3.9 练习与答案

练习 1。 对`[-3,4,-1,2,-6,5]`手算非空最大连续区间，同时给出 end 和 best 两列。

解答：end 依次为−3、4、3、5、−1、5；best 依次为−3、4、4、5、5、5。一个最佳区间是`[4,-1,2]`，另一个是末尾单项`[5]`，和都为 5。读到−6 时 end=5−6=−1，但 best 保留 5；下一个 5 应重新起段，因为只取 5 比−1+5=4 更大。若要固定输出规则，例如最短区间或最左区间，必须在相等时一并比较端点，不能只保存和。

练习 2。 对 `AB` 和 `ACB` 写出 LCS 表，并说明为何最长公共子串不是同一个答案。

解答：空行是 0、0、0、0；A 行是 0、1、1、1；AB 行是 0、1、1、2。末格 2 对应子序列 `AB`，第二个串跳过了 C。公共子串必须连续，在 `ACB` 里 A 后面不是 B，最长长度只有 1。两类问题的状态与转移不同：公共子串常用“必须分别在 i、j 结束”的状态，失配时长度归 0，而 LCS 失配时可以从上或左继承。

练习 3。 已经会记忆化，为什么还要学自底向上填表？

解答：记忆化只计算被访问的状态，适合状态稀疏或依赖不易排序的情况；递归会占调用栈并有调用开销。自底向上明确计算顺序，容易滚动压缩与观察每个状态；若可达状态只有少数，却把整个巨大表填满，也会浪费工作。选择取决于状态图，不取决于哪种写法看上去更高级。

4 向量：格子不够时，整排搬到哪里

待办清单会变长，固定数组却只有申请时的那一块空间。向量在数组外面保存三样东西：底层地址、已有项数 size、已申请格数 capacity。格子不够时申请一块更大的连续空间，把旧元素搬过去，再让向量指向新空间。

“向量”在本章指动态数组，不是物理学中有方向的量。按秩访问中的“秩”就是从 0 开始的下标。若底层首地址为 B，每格固定占 w 字节，下标 r 的地址为$B + rw$。例如 B=1000,w=4,r=3，地址是 1012，所以不必从第 0 项挨个找过去。

4.1 三次追加，把新旧两排都画出来

先申请 2 格，不放任何项：size=0,capacity=2。只有[0,size)属于有效元素，后面的格是备用空间，不能把备用格里的残留数值当作清单内容。

动作	底层格 0	格 1	格 2	格 3	size/capacity
初始化	空	空	未申请	未申请	0/2
追加 10，写 a[0]	10	空	未申请	未申请	1/2
追加 20，写 a[1]	10	20	未申请	未申请	2/2
请求追加 30	还没写	还没写	不能越界写	未申请	仍 2/2

此时不是“接着写第 2 格”：旧数组根本没有第 2 格。按满时加倍的规则，第三次追加分成这些动作：

```
① 申请新数组b，容量4
   旧a: [10][20]        新b: [空][空][空][空]
② 复制a[0]到b[0]
   旧a: [10][20]        新b: [10][空][空][空]
③ 复制a[1]到b[1]
   旧a: [10][20]        新b: [10][20][空][空]
④ 迁移成功后，让向量改用b，释放旧存储
⑤ 写入新元素30到b[2]，size由2增至3
   当前: [10][20][30][空]，capacity=4
```

把刚才的迁移逐帧展开。每格下方是它在所属数组内的下标；新旧两排的下标各自从 0 开始。

向量扩容：旧数组与新数组同时出现

1. 容量2，长度0

旧数组

空 空

0 1

size=0，capacity=2。

原因：容量表示分配的格数；长度表示已经存入的元素数。

图 5　逐步演示：向量扩容：旧数组与新数组同时出现

向量扩容：旧数组与新数组同时出现

2. 写入10



old[0]←10，size=1。
原因：尚有空位，因此不必搬动已有元素。

图 6　向量扩容：旧数组与新数组同时出现，第 2 步

向量扩容：旧数组与新数组同时出现

3. 写入20



old[1]←20，size=2。
原因：尚有空位，因此不必搬动已有元素。

图 7　向量扩容：旧数组与新数组同时出现，第 3 步

向量扩容：旧数组与新数组同时出现

4. 再插入30前，先分配4格



旧数组已满，另申请容量4的新数组；size仍为2。
原因：扩容不是让旧数组凭空变长，而是取得一块新空间。

图 8　向量扩容：旧数组与新数组同时出现，第 4 步

向量扩容：旧数组与新数组同时出现

5. 搬移一个已有元素

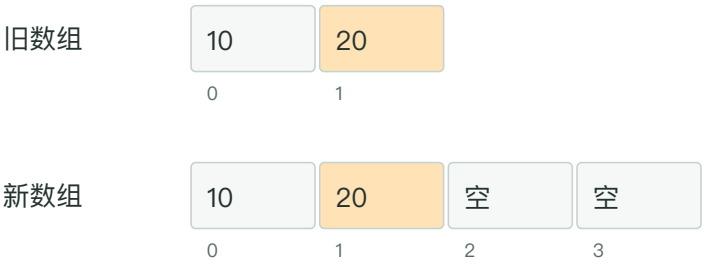


new[0]←old[0]=10。
原因：必须保留已有元素的先后顺序；此时两个数组暂时同时存在。

图 9　向量扩容：旧数组与新数组同时出现，第 5 步

向量扩容：旧数组与新数组同时出现

6. 搬移一个已有元素



new[1]←old[1]=20。
原因：必须保留已有元素的先后顺序；此时两个数组暂时同时存在。

图 10　向量扩容：旧数组与新数组同时出现，第 6 步

向量扩容：旧数组与新数组同时出现

7. 切换存储并写入30



新数组成为有效存储，旧空间释放；new[2]←30，size=3。
原因：有效元素是前3格，第4格只有容量，还不属于序列。

图 11　向量扩容：旧数组与新数组同时出现，第 7 步

旧数据搬完前，新旧数组同时存在。旧容量 2、新容量 4，迁移峰值约 6 个元素槽，不能只算迁移完成后的 4 格。一般旧容量 C 扩到 2C 时，可能暂占 3C 槽。

扩容改变底层地址。外部若保存“旧数组第 1 项的地址”，这个地址不会自动指向新数组；旧存储释放后它失效。C++的 reserve 只改变容量，不改变有效项数；resize 会改变逻辑长度并可能构造/销毁元素。先预留 100 格，不代表可以按向量接口读取第 99 项。

4.2 在中间挤入一个位置，搬的方向不能错

当前[10,20,30,40]，容量 8。要在下标 1 前插 15，最终 20、30、40 都要右移 1 格。先保存最右边尚未搬走的内容：

动作	0	1	2	3	4	为什么先搬它
开始	10	20	30	40	空	有效区[0,4)
a[4]=a[3]	10	20	30	40	40	先保住最右项

动作	0	1	2	3	4	为什么先搬它
a[3]=a[2]	10	20	30	30	40	旧 40 已保存，可覆盖
a[2]=a[1]	10	20	20	30	40	旧 30 已保存，可覆盖
a[1]=15	10	15	20	30	40	空位留好，写新项
size=5	10	15	20	30	40	全部 5 项正式有效



图 12　向量插入每次读写的方向

图中箭头从读的位置指向写的位置。若先 `a[2]=a[1]`，旧 30 就被 20 覆盖；下一次再读 `a[2]`会读到 20，无法恢复旧 30。暂时出现重复项不是错误，它们是搬迁中的副本；结束后有效区才表示完整结果。

教师《02 向量》PDF19 页的完整图将单项推广为一段长度 `d=hi-lo`：先把原后缀`[r,n)`从后向前搬到`[r+d,n+d)`，再填新增区间`[r,r+d)`。我们的单项例子就是 `d=1`。若插入来源与向量自身重叠，还需额外保存来源或采用支持重叠的实现，不能因上述方向正确就忽略别名问题。

删除则相反。删掉`[10,15,20,30,40]`的`[1,3)`，即删 15 和 20：先 `a[1]=a[3]`得到 30，再 `a[2]=a[4]`得到 40，最后 `size=3`，有效内容`[10,30,40]`。左移从前向后，因为写入的都是已经不再需要的左侧格。

4.3 一次很贵，为什么一串追加仍然便宜

只做尾部追加、初始容量 2、满时加倍。把“一次迁移旧项”和“一次写新项”各算 1：

追加后项数	容量	本次迁移数	本次写新项	累计成本
1	2	0	1	1
2	2	0	1	2
3	4	2	1	5
4	4	0	1	6
5	8	4	1	11
6	8	0	1	12
7	8	0	1	13

追加后项数	容量	本次迁移数	本次写新项	累计成本
8	8	0	1	14
9	16	8	1	23

第 9 次成本 9，确实不是最坏常数时间。但前 n 次写新项共 n，迁移规模 2、4、8……构成几何和，最后一次迁移小于 n，总迁移小于 2n，故总成本小于 3n 加常数。把总账均摊到每次为常数，叫摊还常数时间，不需要随机输入假设。

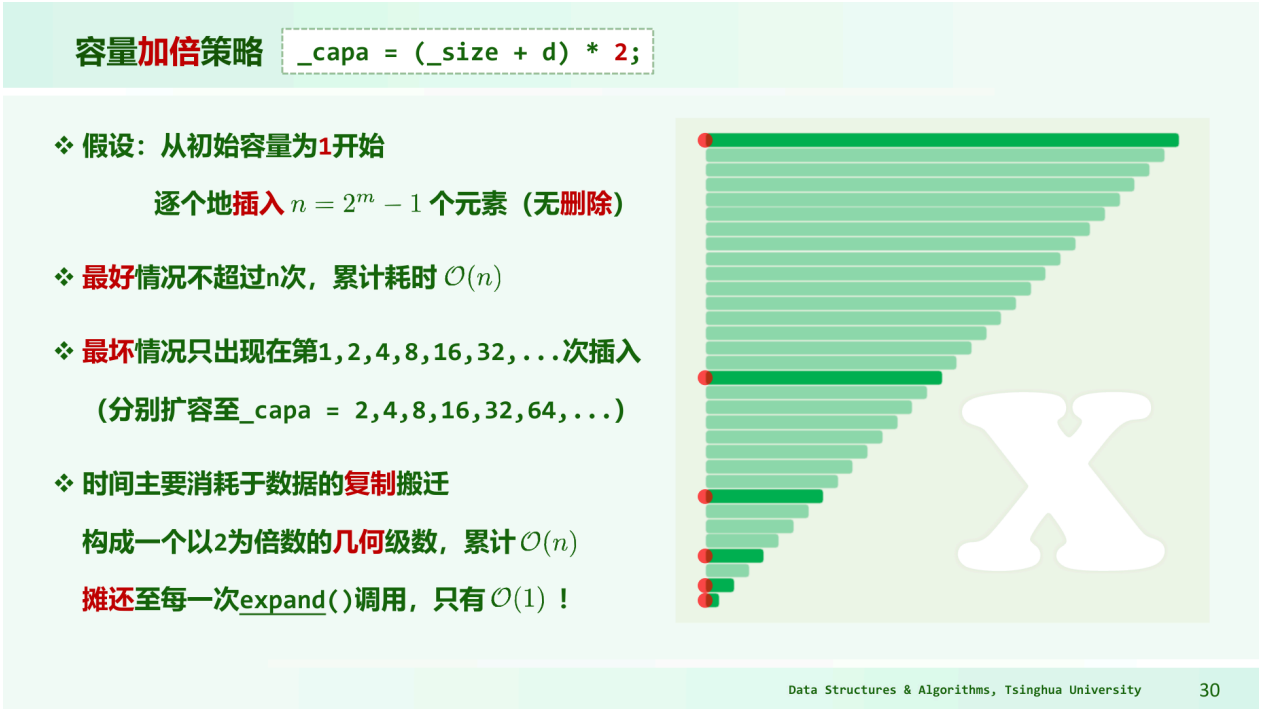


图 13　教师容量增长与累计迁移原图

这张老师原图采用(_size+d)*2，还会保留额外槽位，准确触发点与简化的“容量 2 装两项”不同。读图应认出不断翻倍的迁移规模，而不能把本章表中的容量序列直接当成老师源代码的逐行输出。固定比例增长保证几何总账，具体常数依实现改变。

如果每次只加 2 格，容量 2、4、6、8……，迁移数 2+4+6+8……成为等差和，n 次追加会有平方级总迁移。相反，提前知道要放多少项时一次预留足够空间，可以省去后续这些扩容，但估计过大又会长期占用备用内存。

4.4 缩容别在同一条线反复开关

精确定义一个坏规则：“size 降到 capacity 的一半就立即缩半，满时下一次追加就扩倍”。从容量 8、size5 开始，删除一项变 4，马上缩到 4；追加一项前因满而扩到 8，size 变 5；再删又缩 4。两次很小的操作反复搬动 4 项。把 8 换成很大的容量，这份成本就不能摊成常数。

可以把缩容阈值改为四分之一：满时扩倍，size 降到不超过容量四分之一时才缩半，并保留最低容量。容量 16、size4 缩到 8 后仍半满，离再次扩容或再次缩容都隔着若干普通操作，避免刚才的两步来回。阈值用“低于”还是“不超过”会改变一个边界触发点，分析时应固定一种规则。

4.5 去重也体现了数据布局的成本

无序向量去重，可以逐个查看当前项以前是否出现，发现重复就删除。但每次删除要搬后面的元素，查找本身也可能线性，整体最坏平方时间。若允许改变顺序，可以将待删位置与末项交换后弹出，删除便宜了，但顺序不再保留。

有序向量的相同值相邻，可以用读指针扫描、写指针压紧。例如[1,1,2,2,2,5]：保留第一个 1；第二个 1 跳过；第一次遇 2 写入下标 1；其余 2 跳过；遇 5 写入下标 2，最后有效区间为[1,2,5]。每项读一次，总时间 Θ(n)，没有逐次删除带来的反复搬移。改变维护的信息，算法就能省下一大段工作。

有序去重的每一步可以同样按格子核对。输入[1,1,2,2,2,5]，先保留首项，write=1：

read 与当前值	与最后保留项比较	动作	write 变化	有效去重前缀
1，1	1 等于 a[0]的 1	跳过	仍 1	[1]
2，2	2 不同于 a[0]的 1	a[1]=2	1⟶2	[1,2]
3，2	2 等于 a[1]的 2	跳过	仍 2	[1,2]
4，2	同上	跳过	仍 2	[1,2]
5，5	5 不同于 a[1]的 2	a[2]=5	2⟶3	[1,2,5]

write<=read 保证只覆盖已经读过的位置；最后把逻辑长度改为 3，后面残留格不是新的有效项。空输入应先返回 0，不能先读取 a[0]。筛掉偶数也是同一读写路线，只是保留条件换成“当前是奇数”，不需要输入有序。

4.6 复制、移动与引用，扩容时元素身份怎样变化

设向量中放三条大字符串，底层格子保存字符串对象。扩容会在新数组中构造相应对象，再销毁旧对象；语言可能采用移动以转交内部缓冲区，也可能复制全部字符。若每条字符串长度 L，一次元素迁移不再必然只花与 L 无关的时间。通常分析向量时把“一次元素复制或移动”作为单位，再由元素类型决定单价。

异常也可能在迁移中发生。健壮容器不能复制到一半就丢掉旧数组，应该保证失败时仍能恢复一个合法状态，或明确接口提供的异常保证。教学伪代码省略内存申请失败，并不意味着真实实现可以忽略对象生命周期。

预留空间 reserve 只改变 capacity，不改变 size；调整长度 resize 则可能创建或销毁逻辑元素。预留十格以后写 a[9]，若 size 仍 0，按通常向量接口仍不合法。把“内存已经申请”与“对象已经构造并进入逻辑序列”分开，才能正确使用非平凡类型。

4.7 精确看一个扩容边界

初始容量 1，满时加倍，做 n 次追加。若 n 恰为 2^k，旧元素复制总数为 1+2+...+2^(k−1)=n−1；若 n=2^k+1，刚触发下一次扩容，复制总数变为 2^(k+1)−1，接近 2n。累计曲线会在边界跳跃，却始终被 2n 包住。某次实验看到 n 只多 1 而耗时接近翻倍，不与总账线性矛盾。

不同语言的标准容器未必严格加倍，常数因子可能不同，标准也可能只保证摊还性质而不指定具体增长公式。要分析教师实现，就用教师实际代码；要依赖标准库行为，就依接口保证，别把某次观察出的容量序列当跨版本承诺。

4.8 一次批量删除，怎样避免反复搬相同元素

要删掉向量里所有偶数，逐次调用 remove 可能每次都搬整个后缀，最坏平方。可用 read 从左扫、write 标记下一个保留位置：遇奇数就写到 a[write]并 write 加 1，遇偶数只前进 read。输入[2,7,4,9,6,1]最终前缀为[7,9,1]，逻辑长度改为 3。每项读一次、保留项写一次，总线性，且保留相对顺序。

不变式是已读前缀中的所有应保留项，按原顺序恰好在[0,write)；write≤read 保证写入不会破坏尚未读取的后缀。这个证明同时说明为什么可以原地压紧。它与有序去重的双指针相似，但“是否保留”的判断来自不同条件，不能把两者混成只适合相邻重复的算法。

4.9 练习与答案

练习 1。 容量 4、size4、内容[2,4,6,8]，在秩 2 插入 5。按加倍策略写出扩容、搬移与结果，哪些已有迭代器可能失效？

解答：先分配 8 格，将 4 个旧元素迁移。再从后往前搬 8 到下标 4、6 到下标 3，写 5 到下标 2，得到[2,4,5,6,8]，size5。扩容换了底层地址，指向旧数组元素的指针、引用和迭代器通常全部失效。即使没有扩容，插入点及其后的元素也发生位移，不能继续假设旧迭代器仍代表原逻辑元素；具体失效规则依容器语言规范。

练习 2。 已知接下来会追加 10000 项，提前分配 10000 格和每次加倍有什么差别？

解答：提前预留使后续追加无需扩容迁移，容量申请一次，适合规模可靠且内存足够的情况。每次加倍不要求预先知道总数，仍有线性总搬迁保证，却可能多申请空位和产生数次迁移。提前预留改变的是容量，不应误把 size 直接变成 10000；否则容器中会多出 10000 个逻辑元素。若预估远大于实际需求，会长期浪费空间。

练习 3。 给有序向量`[2,2,3,3,3,4]`原地去重，写指针初值为何可以设 1？空向量又如何？

解答：非空时第 0 项必然保留，令 write=1；从 read=1 开始，若 `a[read] != a[write-1]`，就把当前值写到 a[write]并增加 write。扫描完成得到前 3 项`[2,3,4]`。空向量没有第 0 项，必须先判断 n=0 并返回 0，不能读取 `a[0]`或 `a[write-1]`。不变式是已扫描前缀的不同值，按原顺序恰好保存在 `a[0:write]`。

5 二分查找：每次删去半段，凭什么没删掉答案

在[1,3,3,7,9]中查找3，“找到一个3”与“找到第一个3”答案可能不同。我们先固定后者：返回第一个不小于3的位置。这个接口称lower_bound，即下边界。即使3不存在，也返回插入3后仍有序的最左位置。

5.1 先把每个格子变成一个判断

对目标x=3，逐项判断“这一项是否不小于x”：

```
下标        0    1    2    3    4
数值        1    3    3    7    9
a[i]>=3    假   真   真   真   真
                ↑ 目标：第一个真
```

这条假真序列只转一次，是因为数组非降。非降允许相等，要求左边不大于右边。一旦某项已经不小于3，它右边所有项也不小于3。二分利用的是这项保证。

lo,hi是尚未分类区间的左右边界，写[lo,hi)，左含右不含。所有下标小于lo的项已确认太小；下标不小于hi的项已确认足够大。答案边界的可能位置是闭区间[lo,hi]，允许位于hi，所以“尚未分类的元素”与“可能的边界位置”要分清。

```python
def lower_bound(a, x):
    lo, hi = 0, len(a)
    while lo < hi:
        mid = lo + (hi - lo) // 2
        if a[mid] < x:
            lo = mid + 1
        else:
            hi = mid
    return lo
```

//2取整数除法的下整。例如5//2是2。只在lo<hi时访问，mid必在真实下标范围内。写成lo+(hi-lo)//2也便于在非负固定宽下标模型中避免lo+hi的溢出。

5.2 三次比较，每个被排除的格子都说明理由

比较前 lo,hi	mid 怎么来	读取并比较	可以确认哪些格	更新
0,5	0+(5−0)//2=2	a[2]=3，3<3 是假	2、3、4 都不小于 3；但 2 可能就是第一项	hi=2
0,2	0+(2−0)//2=1	a[1]=3，3<3 是假	1 及其右边都足够大；1 仍可能为答案	hi=1
0,1	0+(1−0)//2=0	a[0]=1，1<3 是真	0 及其左边全太小	lo=1
1,1	区间为空，不算 mid	不再读数组	左边全小于 3，右边全不小于 3	返回 1

下面把比较和边界更新分成两步。橙色标当前比较项，绿色表示已确认小于3，蓝色表示已确认不小于3；空白格仍未分类。

二分：找到第一个不小于3的位置

1. 先保留全部未分类元素

目标值 x=3；尚未分类的元素区间 [0, 5)



lo=0，hi=5

lo=0，hi=5。目标是第一个不小于3的元素位置。
原因：答案边界可能在闭区间[lo,hi]内；右端5是全部元素都小于3时的返回位置。

图 14　逐步演示：二分：找到第一个不小于 3 的位置

二分：找到第一个不小于3的位置

2. 比较区间中点

目标值 x=3；尚未分类的元素区间 [0, 5)



lo=0，hi=5，mid=2，a[mid]=3

mid=⌊(0+5)/2⌋=2，比较3与3。
原因：中点由整数除法向下取整，始终落在非空的未分类元素区间。

图 15　二分：找到第一个不小于 3 的位置，第 2 步

二分：找到第一个不小于3的位置

3. 把中点归入已知右区

目标值 x=3；尚未分类的元素区间 [0, 2)



lo=0，hi=2

hi从5改成2，lo保持0。
原因：中点满足要求，前面还可能有3；a[hi:]均不小于3，hi本身仍可作为最终答案边界。

图 16　二分：找到第一个不小于 3 的位置，第 3 步

二分：找到第一个不小于3的位置

4. 比较区间中点

目标值 x=3；尚未分类的元素区间 [0, 2)



lo=0，hi=2，mid=1，a[mid]=3

mid=⌊(0+2)/2⌋=1，比较3与3。
原因：中点由整数除法向下取整，始终落在非空的未分类元素区间。

图 17　二分：找到第一个不小于 3 的位置，第 4 步

二分：找到第一个不小于3的位置

5. 把中点归入已知右区

目标值 x=3；尚未分类的元素区间 [0, 1)



lo=0，hi=1

hi从2改成1，lo保持0。
原因：中点满足要求，前面还可能有3；a[hi:]均不小于3，hi本身仍可作为最终答案边界。

图 18　二分：找到第一个不小于 3 的位置，第 5 步

二分：找到第一个不小于3的位置

6. 比较区间中点

目标值 x=3；尚未分类的元素区间 [0, 1)



lo=0，hi=1，mid=0，a[mid]=1

mid=⌊(0+1)/2⌋=0，比较1与3。
原因：中点由整数除法向下取整，始终落在非空的未分类元素区间。

图 19　二分：找到第一个不小于 3 的位置，第 6 步

二分：找到第一个不小于3的位置

7. 排除中点及其左侧

目标值 x=3；尚未分类的元素区间 [1, 1)



lo=1，hi=1

lo从0改成1，hi保持1。
原因：有序性保证中点及更左边都小于3，不能是第一个不小于3的位置。

图 20　二分：找到第一个不小于 3 的位置，第 7 步

二分：找到第一个不小于3的位置

8. 边界相遇，返回1

目标值 x=3；尚未分类的元素区间 [1, 1)



lo=1，hi=1

lo=hi=1，a[1]=3；下标0的值1小于3。
原因：循环区间为空，左边全小于3，当前位置及其右边全不小于3。

图 21　二分：找到第一个不小于 3 的位置，第 8 步

绿色表示已确认“小于 3”，蓝色表示已确认“不小于 3”，白色仍未知。蓝色出现并不表示已经可以返回：它左边可能还有更早的蓝色。`a[mid]<x` 时，`mid` 已经排除，所以 `lo=mid+1`；否则保留 `mid` 这个可能边界，让 `hi=mid`。把两个更新都机械写“加 1”或“减 1”会丢答案。

`hi-lo` 是非负整数，每轮变小，只有一个未知格时也会变成 0，因此会结束。结束 `lo=hi`，恰好是一边全假、一边全真的分界。时间随对数增长，额外只用几个变量。

5.3 查不到时，不要把合法边界读成非法元素

同一数组查 `x=10`：

lo,hi	mid	比较	更新
0,5	2	3<10	lo=3
3,5	4	9<10	lo=5
5,5	无	结束	返回 5

5 是“插在末尾”的合法边界，数组却没有 a[5]。要查是否存在，先 p=lower_bound(a,x)，再判断 p<len(a) and a[p]==x。短路条件先检查范围，范围不合法时不执行右边的读取。

查 x=0 会逐步把 hi 降到 0，返回 0；空数组初始 lo=hi=0，一次也不读取。查 x=2 返回 1，但 a[1]=3，说明 2 不存在。返回了一个数组内位置，也不一定就是目标值。

找第一个严格大于目标的位置 upper_bound，仅把分支改为“a[mid]<=x 时 lo=mid+1”。对 3 逐步是：(0,5)读下标 2 的 3，lo 变 3；(3,5)读下标 4 的 9，hi 变 4；(3,4)读下标 3 的 7，hi 变 3；返回 3。因此 3 占据[1,3)，共有 2 项。最后一个不大于 3 的位置是 upper_bound−1=2；若 upper 为 0，前驱为−1，表示不存在，不能用它读数组。

5.4 数字不是数组元素，也能找同一种分界

运输重量[2,3,4,5]，保持顺序，最多分成 2 批，求每批容量最小值。容量至少 5，否则 5 装不下；最多 14，一批全装。给一个容量 C，顺序装，下一件装不下才另开一批：

容量 C	逐件装法	批数	够不够
8	[2,3]；[4]；[5]	3	不够
9	[2,3,4]；[5]	2	够
10	[2,3,4]；[5]	2	够

增加容量不会让原有装法失效，因此判断从“不够”转成“够”。在整数容量[5,15)找第一个真：mid10 够，hi10；mid7 不够，lo8；mid9 够，hi9；mid8 不够，lo9；结束答案 9。这里 15 是尾后边界，不会被当作候选容量检查。

检查函数也需要理由：每批尽可能装向右，在相同批数后不会比其他合法方案停得更早，因此所需批数最少。若货物必须重新排序或成本有复杂限制，这个贪心前提可能失效，不能只看外层二分正确。

5.5 一次比较和两次比较，不要藏进同一个词

版本 A 常先判断相等，再判断大小，某些失败路径会对同一元素做两次比较。lower_bound 式写法每轮只用一个大小关系，结束后若要确认存在再判断一次。它们都为 O(log n)，但比较次数的主项或常数可能不同。

成功与失败的平均查找长度也需要区分。成功在元素节点上停止；失败在相邻元素之间的 n+1 个间隙中停止。不同树形、n 是否为 2 的幂减 1、成功目标分布与失败间隙分布，都会影响精确均值。即使两者都为 Θ(log n)，也不能写成“精确平均次数总相等”。

举 n=3 且平衡地先比中间元素。成功所需大小判定层数为 1、2、2，等概率均值 5/3；失败的 4 个间隙都需要走完 2 层，均值 2。这个例子已经足以否定“总是精确相等”。若教材把等值判断另计，需按同一成本口径重新统计，不能混用。

5.6 位图：用一位回答“有没有”

当键来自较小整数范围[0,U)，只需要保存是否出现，可以给每个可能值分配一位。键 k 对应字节下标 k//8，以及该字节内偏移 k%8。若约定低位在前，掩码为 1 << (k%8)。设置用按位或，测试用按位与，清除用与上反掩码。位次约定必须统一，与机器大端小端是不同层面的概念。

U=16，要标记 2、5、13：第 0 字节置第 2 位与第 5 位，二进制为 00100100；第 1 字节置第 5 位，为 00100000。查询 5 读第 0 字节与 00100000 相与，非 0，存在；查询 6 与 01000000 相与为 0，不存在。标记相同值两次不会计成两份，位图保存的是集合而非出现次数。

空间约 U/8 字节，设置与查询为 O(1)。但清空整个新位图需要 Θ(U)位操作或 Θ(U/w)机器字操作。若 U 巨大而实际只出现少量键，位图可能比散列表浪费得多。某些“快速初始化”技巧用已初始化前缀与互相校验的索引，避免每次全清，但实现需要额外数组与合法性判断，不等于未初始化内存可以随便读取。

5.7 例 4：筛质数，标记从哪里开始

寻找 2 到 30 的质数。开始把所有数设为未合成，0 和 1 另排除。处理 2 时标记 4、6、8.....；处理 3 时标记 9、12、15.....；处理 5 时标记 25、30。处理到 $\lfloor\sqrt{30}\rfloor=5$ 即可，剩余未标记者为 2、3、5、7、11、13、17、19、23、29。

从 p^2开始，是因为小于 p^2的 p 的倍数 $p\times k$ 中 $k<p$，早已被更小的质因子标记。只处理尚未标记的 p，经典埃拉托斯特尼筛的时间 O(U log log U)，空间 O(U)位；证明这个更精细的时间界需要质数倒数和估计，不能把“每个数只被标一次”当理由，合数实际上可能被多个质因子标记。

实现时用 `p <= U//p` 判断 $p^2\le U$，可以避免固定宽度整数中的 p*p 溢出；枚举倍数前也要确保起点可表示。对 0 到 30 的标记写入不必藏在省略号里：

当前 p	为什么处理/跳过	实际尝试置为合数的下标
2	尚未标记，$p^2=4$	4,6,8,10,12,14,16,18,20,22,24,26,28,30
3	尚未标记，$p^2=9$	9,12,15,18,21,24,27,30
4	已经由 2 标记	跳过，不把 4 当质数重复筛
5	尚未标记，$p^2=25$	25,30
6	$6^2>30$	停止检查 p

12、18、30 等会再次被置为真，重复写同一标记不产生重复元素。0、1 在初始化就单独排除，不能因为“没被筛掉”而混入质数输出。扫描最终未标记的 2 到 30，得到正文列出的十个质数。

5.8 在纸上设计二分，不从模板反推题意

任何二分都可以先画一个布尔序列：假、假、假、真、真，目标是第一个真。让 lo、hi 把已经确认的假区和真区夹住；每次看 mid，真就把真区向左扩，假就把假区向右扩。数组查找中谓词是 $a[mid]\ge x$，容量问题中谓词是“这个容量能否完成”。不同题共用的是单调边界，不是同一条数值公式。

如果想找最后一个真，可以转换成找第一个假再减 1，或者另外设计区间约定。不要在半开模板里临时把 hi 改 n−1、把结束条件改≤，却仍沿用原更新式。合法区间、返回意义与更新规则是一套整体。

算 mid 时`(lo+hi)/2` 在固定宽度有溢出风险，常写 `lo+(hi-lo)/2`。但若 lo、hi 可分别为极大正负数，hi−lo 也未必安全；常见数组下标 $0\le lo\le hi\le n$ 的前提才让差值范围受控。边界类型若混有有符号与无符号，−1 前驱也需明确表示，不能让它绕成巨大正数。

5.9 比较查找长度，先固定树和计费方式

对 7 项的理想中点二分，比较层分别有 1 项深 1、2 项深 2、4 项深 3。若成功目标等概率、每层算一次主比较，平均成功层数为(1+4+12)/7=17/7。8 个失败间隙在这棵满分支决策结构里都需要走完 3 层，失败均值 3。它们均为 Θ(log n)，主导增长一致，精确均值不同。

若算法每层先判等再判大小，成功在根只判一次，失败或向下路径可能判两次；此时应该将每条路径的实际判断逐条加起来。所谓平均查找长度有时按“访问节点数”，有时按“关键码比较次数”，两种教材口径不能直接相减。

Fibonacci 查找也在有序数组上排除区间，不过使用相邻 Fibonacci 长度分段，并非随意挑一个偏左中点。可用`[1,3,5,7,9,11,13]`找 11 理解一个常见版本。先用 Fibonacci 数 `1,1,2,3,5,8` 覆盖 7 项；记录覆盖长度 `F=8` 及前两项 `P=5,Q=3`，已排除前缀的末位置 `offset=-1`。每次探测 `min(offset+Q,n-1)`。

F,P,Q,offset	探测位置与值	比较后保留什么	下一状态
8,5,3,−1	位置 2，值 5	5<11，位置 0 到 2 可排除	5,3,2,2
5,3,2,2	位置 4，值 9	9<11，排除到 4	3,2,1,4
3,2,1,4	位置 5，值 11	相等，找到	返回 5

若探测值太大，保留较短左段，三长度按`(F,P,Q)=(Q,P-Q,Q-(P-Q))`的旧值同时更新；太小则按`(P,Q,P-Q)`同时更新并推进 offset。长度降到 1 后还需检查紧接 offset 的最后候选且先检查下标范围。它的接口是找任

意等值项，重复值未承诺最左，与前面 lower_bound 不同。教师版还会按自己的区间接口写成等价形式，具体分支应按其代码核对。

查 3 可以走另一侧：初始探测位置 2 值 5 太大，三长度变成 3、2、1，offset 仍为−1；下一次探测位置 0 值 1 太小，三长度变 2、1、1，offset 变 0；再探测位置 1 值 3，命中。查 13 则依次比较位置 2 的 5、位置 4 的 9、位置 5 的 11，长度降至 1，最后再检查 offset+1 即位置 6 的 13，命中。这个最后候选若不检查，就会漏掉数组末项。

插值查找则额外利用数值间距，猜目标在范围内的比例。例`[10,20,30,40,50]`找 40，按$(40-10)/(50-10)=3/4$猜到下标 3，直接命中；这依赖值分布适合比例估计。若`[1,2,4,8,16]`找 8，初次比例`(8-1)/(16-1)=7/15`只猜到下标 1，还需继续缩区间。重复值可能使分母为 0，目标越过端点也要先拒绝。均匀条件下的快不能推成所有输入保证，最坏仍可线性。

上面的 Fibonacci 三长度满足 `F=P+Q`，每轮把覆盖长度降到旧 `P` 或旧 `Q`，因此会结束。右侧长度与左侧长度来自同一数列，更新时必须读取旧值、一起替换；若逐行覆盖再算差，就可能破坏这条关系。未到数组长度的覆盖区不实际申请虚拟元素，探测用 `min` 限制到最后真实下标。空数组应立即返回失败。这说明它的下标和终止规则比一句“用黄金分割找中点”多得多。

5.10 位图快速清空的一个可实现替代

若每轮只标记少量键，可额外记录本轮真正从 0 变 1 的键列表，结束时仅清这些位。比如 U 很大，本轮只置 2、5、13，记录这三个键，清空只做三次。若重复置 5，不要重复记录，否则列表可能随操作数增长。这个方法的清空成本与本轮不同键数有关，但第一次申请与初始化底层位图仍可能花 Θ(U/w)。

另一做法为每键保存时间戳，当前轮号为 epoch，时间戳等于 epoch 视为已出现。清空只增加 epoch；但每键不再是一位，内存增大，epoch 溢出时要真实重置。将清空从每轮移到偶尔一次，并不等于完全没有初始化或溢出成本。

5.11 把不存在的边界也当作合法答案

对第一个真位置，所有项都假时返回 n；所有项都真时返回 0。两端都不需要读取虚拟元素。若你为了简化证明假设 a[−1]小于所有值、a[n]大于所有值，那是数学哨兵，不等于代码真的可以访问越界位置。程序只在 lo<hi 时计算 mid，因而 mid<n，读取始终位于真实数组内。

许多二分错误来自把“候选答案位置”与“可访问元素下标”混为一谈。返回 p 以后先检查 p<n，再读 a[p]；查前驱返回−1 以后先检查非负。一个函数能用 n 表示尾后边界，另一个接口可能用 null 或可选类型，调用者也要相应转换，不能只因值都是整数就直接当作数组下标。

5.12 谓词只在指定范围内单调也够用

例如判断整数 x 的平方是否至少十六，在非负整数区间上是假后真，可二分第一个真得到四；若范围包含负数，结果会是真、假、真，不再是一个单边界。公式相同，搜索域不同，二分资格也不同。先证明题目实际搜索范围内单调，再设置上下界。

空数组的返回位置为零，但它仍没有可读取的第零项。

5.13 练习与答案

练习 1。 a=`[2,2,4,6,6]`，分别求 lower_bound(6)、upper_bound(6)、lower_bound(5)和≤1 的最后位置。

解答：依次 3、5、3、−1。6 的范围是`[3,5)`，有 2 项；5 不存在，但插入下标 3 前仍有序；≤1 的元素没有，所以前驱为−1。若把 upper_bound(6)的结果 5 拿去读 a[5]就越界；它表示边界，不表示一个可读取元素。

练习 2。 数组`[1,5,3,7]`能否使用同一二分查找 3？写一个失败过程。

解答：按 lower_bound 代码，lo0,hi4,mid2，a[2]=3，hi 变 2；接着 mid1，a[1]=5，hi 变 1；mid0，a[0]=1，lo 变 1，返回 1，a[1]=5，误判 3 不存在。算法依赖的“a[mid]≥x 则右半都不小于 x”在乱序输入中不成立。二分不是对任意数组都有效的机械折半。

练习 3。 $U=10^9$、实际只有 1000 个整数，要做存在查询。位图是否总比散列表好？

解答：位图需要约 125MB，还可能花时间初始化；1000 个键的散列表通常只需远少于这个空间。位图适合范围可控、键较密集或位运算批处理有价值的场景。也可先将出现的键排序，查询用二分，空间与实际键数相关。选择应比较范围 U、实际数 n、更新频率与可用内存。

6 排序：每次比较以后，数组究竟变成什么

给出一排数[4,1,3,2]，我们要把它排成[1,2,3,4]。排序算法需要说明怎样从第一排走到第二排：先比较谁，哪些数需要移动，移动时怎样保住原来的数。下面先用同一组输入，分别试插入排序和冒泡排序。

6.1 插入排序：保住手里的数，再腾出空位

可以先把它们看成四张排在桌上的数字牌。第一张 4 单独放着，暂时不用动。拿起第二张 1，它应该放在 4 左边；为此，先把 4 向右挪一格，再把手里的 1 放进腾出的地方。前两张于是成为[1,4]。

再拿起 3，从已经排好的这一段右端开始比较。3 比 4 小，所以 4 向右挪；3 比 1 大，所以 1 留在原处，把 3 放在 1 后面，得到[1,3,4]。最后拿起 2，依次把 4、3 右移，直到遇见比 2 小的 1，再把 2 放在 1 后，得到[1,2,3,4]。每次只加入一张新牌，左边已经排好的那一段就多一张。这就是“插入”的意思。

换成数组时，数不会真的被“拿走”。程序先把它另存一份，再覆盖原来的格子。处理 1 的实际状态是：

```
原数组              [4,1,3,2]
另存x=1             [4,1,3,2]    原来的1仍在，x里也保存着1
把4右移一格         [4,4,3,2]    第一格等着填入x；第二格已存4，1仍在x里
把x写入第一格       [1,4,3,2]    前两项排好，继续处理3
```

下面把“拿在手里的数”叫 x，把它原来的位置编号叫 i。另用 j 记录最后要填入的位置；每右移一个较大的数，j 就向左退一格。这里说的“空位”是正在等待填入 x 的位置，格子中仍可能残留旧值。代码与后面的表都按这套含义读。

```python
def insertion_sort(a):
    for i in range(1, len(a)):
        x = a[i]
        j = i
        while j > 0 and a[j - 1] > x:
            a[j] = a[j - 1]
            j -= 1
        a[j] = x
```

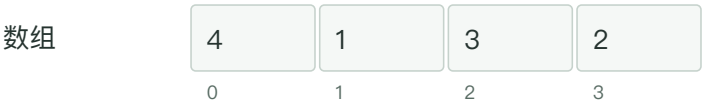
数组格被覆盖前，x 已经保存在局部变量，所以暂时出现两个 4 不会丢掉 1。`while` 先检查 `j > 0`，只有它成立才检查右侧的 `a[j - 1] > x`；j 退到 0 时直接停止，不会去读取下标-1。这是代码中 `and` 从左到右、遇到假就停止判断的规则。

轮与动作	x	j 变化	数组实际格子	为什么停或继续
取下标 1	1	j=1	[4,1,3,2]	左前缀[4]已排序
比 4>1，真，搬 4	1	1⟶0	[4,4,3,2]	旧 1 已在 x 里
j=0，填 x	1	0	[1,4,3,2]	已到最左边
取下标 2	3	j=2	[1,4,3,2]	左前缀[1,4]已排序
比 4>3，真，搬 4	3	2⟶1	[1,4,4,2]	继续看左邻
比 1>3，假，填 x	3	1	[1,3,4,2]	1 在正确一侧，不再动
取下标 3	2	j=3	[1,3,4,2]	左前缀[1,3,4]已排序
比 4>2，搬 4	2	3⟶2	[1,3,4,4]	继续
比 3>2，搬 3	2	2⟶1	[1,3,3,4]	继续
比 1>2，假，填 x	2	1	[1,2,3,4]	全部处理完

逐帧演示沿用代码的 x、i、j：x 暂存新值，i 是本轮原下标，j 是待填位置，比较时读它左边的 a[j-1]。

插入排序：暂存、右移、填入

1. 尚未开始插入



只有a[0]=4构成已排序前缀。
原因：单个元素本来就是有序的。

图 22　逐步演示：插入排序：暂存、右移、填入

插入排序：暂存、右移、填入

2. 先把待插入元素存起来

暂存x=1；本轮i=1；待填位置j=1



x=a[1]=1，j=1。
原因：后续右移会覆盖原位置，暂存可防止丢失这个值。

图 23　插入排序：暂存、右移、填入，第 2 步

插入排序：暂存、右移、填入

3. 与左侧元素比较

暂存x=1；本轮i=1；待填位置j=1



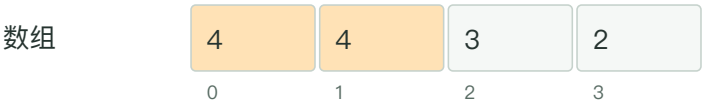
比较a[0]=4是否大于x=1。
原因：前缀已经有序；只有较大的元素需要向右让出位置。

图 24　插入排序：暂存、右移、填入，第 3 步

插入排序：暂存、右移、填入

4. 只执行一次右移

暂存x=1；本轮i=1；待填位置j=0



a[1]←4；j变成0，暂存x仍是1。
原因：重复值是搬移时的临时状态，待填位置由j记录，待填值仍保存在x中。

图 25　插入排序：暂存、右移、填入，第 4 步

插入排序：暂存、右移、填入

5. 填入空出的位置

暂存x=1；本轮i=1；待填位置j=0



a[0]←x=1，前2项已有序。
原因：左侧元素不大于x，右移过的元素均大于x。

图 26　插入排序：暂存、右移、填入，第 5 步

插入排序：暂存、右移、填入

6. 先把待插入元素存起来

暂存x=3；本轮i=2；待填位置j=2



x=a[2]=3，j=2。
原因：后续右移会覆盖原位置，暂存可防止丢失这个值。

图 27　插入排序：暂存、右移、填入，第 6 步

插入排序：暂存、右移、填入

7. 与左侧元素比较

暂存x=3；本轮i=2；待填位置j=2



比较a[1]=4是否大于x=3。
原因：前缀已经有序；只有较大的元素需要向右让出位置。

图 28　插入排序：暂存、右移、填入，第 7 步

插入排序：暂存、右移、填入

8. 只执行一次右移

暂存x=3；本轮i=2；待填位置j=1



a[2]←4；j变成1，暂存x仍是3。
原因：重复值是搬移时的临时状态，待填位置由j记录，待填值仍保存在x中。

图 29　插入排序：暂存、右移、填入，第 8 步

插入排序：暂存、右移、填入

9. 与左侧元素比较

暂存x=3；本轮i=2；待填位置j=1



比较a[0]=1是否大于x=3。
原因：前缀已经有序；只有较大的元素需要向右让出位置。

图 30　插入排序：暂存、右移、填入，第 9 步

插入排序：暂存、右移、填入

10. 填入空出的位置

暂存x=3；本轮i=2；待填位置j=1



a[1]←x=3，前3项已有序。
原因：左侧元素不大于x，右移过的元素均大于x。

图 31　插入排序：暂存、右移、填入，第 10 步

插入排序：暂存、右移、填入

11. 先把待插入元素存起来

暂存x=2；本轮i=3；待填位置j=3



x=a[3]=2，j=3。
原因：后续右移会覆盖原位置，暂存可防止丢失这个值。

图 32　插入排序：暂存、右移、填入，第 11 步

插入排序：暂存、右移、填入

12. 与左侧元素比较

暂存x=2；本轮i=3；待填位置j=3



比较a[2]=4是否大于x=2。
原因：前缀已经有序；只有较大的元素需要向右让出位置。

图 33　插入排序：暂存、右移、填入，第 12 步

插入排序：暂存、右移、填入

13. 只执行一次右移

暂存x=2；本轮i=3；待填位置j=2



a[3]←4；j变成2，暂存x仍是2。
原因：重复值是搬移时的临时状态，待填位置由j记录，待填值仍保存在x中。

图 34　插入排序：暂存、右移、填入，第 13 步

插入排序：暂存、右移、填入

14. 与左侧元素比较

暂存x=2；本轮i=3；待填位置j=2



比较a[1]=3是否大于x=2。
原因：前缀已经有序；只有较大的元素需要向右让出位置。

图 35　插入排序：暂存、右移、填入，第 14 步

插入排序：暂存、右移、填入

15. 只执行一次右移

暂存x=2；本轮i=3；待填位置j=1



a[2]←3；j变成1，暂存x仍是2。
原因：重复值是搬移时的临时状态，待填位置由j记录，待填值仍保存在x中。

图 36　插入排序：暂存、右移、填入，第 15 步

插入排序：暂存、右移、填入

16. 与左侧元素比较

暂存x=2；本轮i=3；待填位置j=1



比较a[0]=1是否大于x=2。
原因：前缀已经有序；只有较大的元素需要向右让出位置。

图 37　插入排序：暂存、右移、填入，第 16 步

插入排序：暂存、右移、填入

17. 填入空出的位置

暂存x=2；本轮i=3；待填位置j=1



a[1]←x=2，前4项已有序。
原因：左侧元素不大于x，右移过的元素均大于x。

图 38　插入排序：暂存、右移、填入，第 17 步

每轮结束，前 i+1 项有序，而且仍是原来的那一批数。开始只有第一项，处理完 1 后有前两项，处理完 3 后有前三项，最后全部有序；暂存和放回保证了移动中没有丢数。

有时相等的数还带着不同记录。例如 3A 和 3B 都是分数 3，字母区分两名学生。只按分数排序时，如果 A 原来在 B 前面，排完相等分数后仍是 A 在 B 前，就说排序是稳定的。插入排序只移动大于 x 的旧项，遇到相等项就停，因此新项留在原相等项后面，保持稳定。若把条件改成>=，输入[2A,2B]在处理 B 时会把 A 右移，变成[2B,2A]，相等记录的先后次序就反了。

本例旧项右移 4 次，分别对应初始的四对数：(4,1)、(4,3)、(4,2)、(3,2)。这些数对中，较大的数站在较小的数前面，次序与升序要求相反，所以叫逆序对。相等的两项不计入逆序。

令 n 为数组项数，I 为最初的逆序对数。每右移一个旧项，它便越过手里的 x，改到 x 后面，恰好消除这一对逆序；其余旧项的相对次序不变。排完后逆序对数变成 0，因此一共需要右移 I 次。除此之外，每项只需取出和放回一次，每轮停止前再做一次边界或大小检查。这些工作合计与 n 成正比，所以总成本为 $\Theta(n+I)$。

已经有序时，I=0，每项比较后就能停，总成本随 n 线性增长。严格倒序时，第一项与后面 n−1 项都构成逆序，第二项与后面 n−2 项构成逆序，直到最后一项没有后继；相加得$I=(n-1)+(n-2)+\cdots+1=n(n-1)/2$。这时移动次数随 n 的平方增长。

6.2 冒泡：先问最大的数会被推到哪里

仍用[4,1,3,2]，每次比较相邻两项，仅在左大于右时交换：

趟	本次比较	动作后数组	本趟结束确认
1	4 与 1，交换	[1,4,3,2]	尚未结束
1	4 与 3，交换	[1,3,4,2]	尚未结束
1	4 与 2，交换	[1,3,2,4]	最大 4 已到末尾
2	1 与 3，不换	[1,3,2,4]	末尾 4 不再参与
2	3 与 2，交换	[1,2,3,4]	后两项 3、4 就位
3	1 与 2，不换	[1,2,3,4]	无交换，可停止

相邻交换让最大项不断右移，已就位的后缀每趟增长一项。相等时不交换，可证明等值记录永远不会交叉，因此稳定。固定趟数版即使有序仍做平方级比较；“一趟无交换就停止”版在有序输入只做一趟，最好线性。记录最后交换位置还能缩短下一趟前缀，最坏仍平方。

6.3 归并：先把小队伍排好，再只比较两队队首

数组[2,4,1,3]。归并排序不是直接让 4 跳到末尾，而是先分成[2,4]与[1,3]，各自继续分成单项，再按返回顺序合并：

```
进入sort([0,4))，等待左右结果
  进入sort([0,2))
    sort([0,1))单项2，直接返回
    sort([1,2))单项4，直接返回
    合并[2]与[4]：比较2<=4，取2；左耗尽，接4
  左结果[2,4]返回
  进入sort([2,4))
    sort([2,3))单项1，直接返回
    sort([3,4))单项3，直接返回
    合并[1]与[3]：取1；接3
  右结果[1,3]返回
  最后合并[2,4]与[1,3]
整个sort返回[1,2,3,4]
```

最后一次合并里，i 指左列尚未取出的首项，j 指右列首项，B 是输出。因为两列已经有序，所有未取出的最小值一定在这两个队首之中。

i,j 与队首	比较	输出写入	指针前进	输出 B
0,0：2 与 1	2<=1 假	写右边 1	j 变 1	[1]
0,1：2 与 3	2<=3 真	写左边 2	i 变 1	[1,2]
1,1：4 与 3	4<=3 假	写右边 3	j 变 2	[1,2,3]
右列已耗尽	不比较越界项	接左边剩下 4	i 变 2	[1,2,3,4]

逐帧图只放大最后一次归并：两段已经分别有序，每次比较剩余首项，再把较小者写到输出。

归并：两个有序段怎样合成一段

1. 先确定两个有序段



左段[2,4]，右段[1,3]；辅助输出初始为空。
原因：本例只演示归并这一步；两段内部已经有序。

图 39　逐步演示：归并：两个有序段怎样合成一段

归并：两个有序段怎样合成一段

2. 取剩余部分中较小的首项



比较左首2和右首1，取右侧1。
原因：两段各自有序，所以未用元素中的最小值必在两个首项之中；现有输出仍有序。

图 40　归并：两个有序段怎样合成一段，第 2 步

归并：两个有序段怎样合成一段

3. 取剩余部分中较小的首项

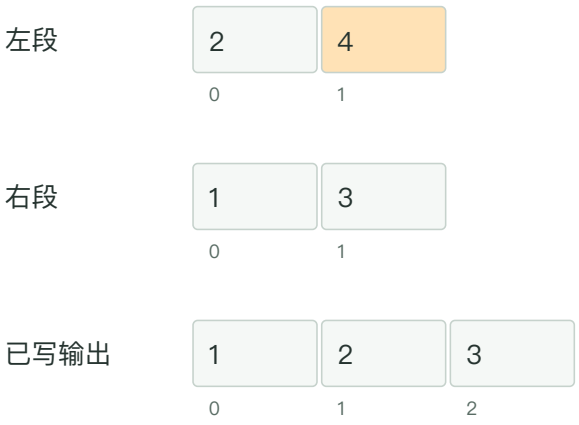


比较左首2和右首3，取左侧2。
原因：两段各自有序，所以未用元素中的最小值必在两个首项之中；现有输出仍有序。

图 41　归并：两个有序段怎样合成一段，第 3 步

归并：两个有序段怎样合成一段

4. 取剩余部分中较小的首项



比较左首4和右首3，取右侧3。
原因：两段各自有序，所以未用元素中的最小值必在两个首项之中；现有输出仍有序。

图 42　归并：两个有序段怎样合成一段，第 4 步

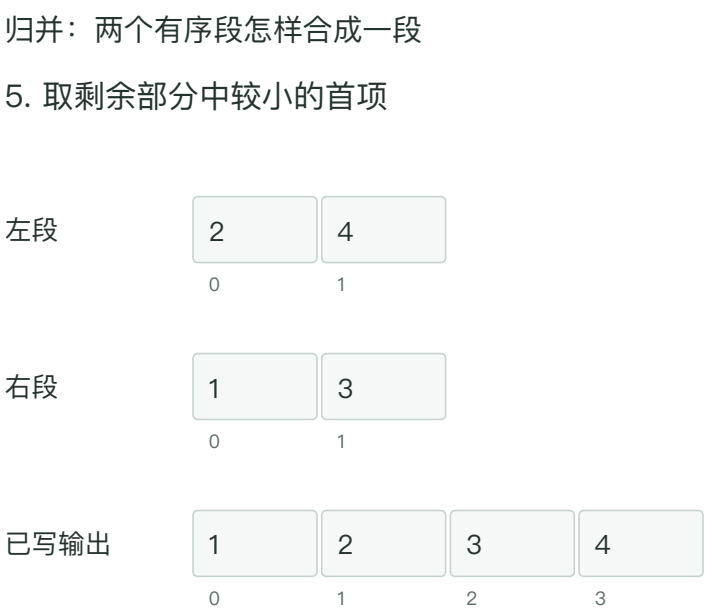


图 43　归并：两个有序段怎样合成一段，第 5 步

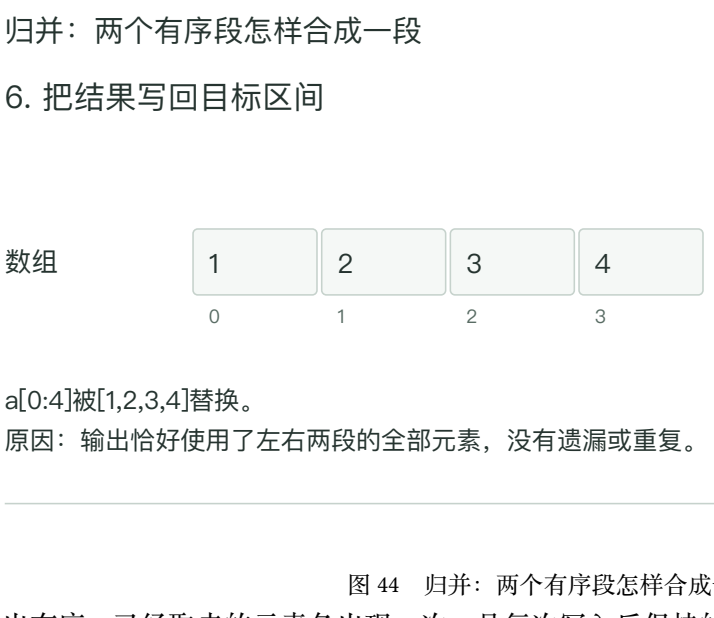


图 44　归并：两个有序段怎样合成一段，第 6 步

输出有序、已经取走的元素各出现一次，是每次写入后保持的关系。一次只推进被取走那侧的指针，另一侧队首还要参加下一次比较。若两边相等先取左，原先位于左半的等值记录不会被右半越过，因此稳定。合并的前提是两边各自已排好；对乱序两队只比较队首并不能保证结果。

可直接运行的简单版本如下，切片和返回新列表都计入空间/复制成本：

```
def merge_sort(a):
    if len(a) <= 1:
        return a[:]
    m = len(a) // 2
    L = merge_sort(a[:m])
    R = merge_sort(a[m:])
    i = j = 0
    B = []
    while i < len(L) and j < len(R):
        if L[i] <= R[j]:
            B.append(L[i])
            i += 1
        else:
            B.append(R[j])
            j += 1
    B.extend(L[i:])
    B.extend(R[j:])
    return B
```

每层合并共处理与 n 成比例的项，约对数层，总时间$\Theta(n \log n)$。这份深度优先实现同时存活的分段长度按几何和控制，峰值额外空间$O(n)$；历史上累计申请的格数可更多，不能与峰值混作同一指标。用一次预分配辅助数组和下标区间可以减少分配，仍保留同一合并逻辑。

Princeton《Algorithms》归并排序一节也从两列有序队首归并推到递归排序，另给自底向上的版本。对本例，自底向上先合并`[2]`与`[4]`、`[1]`与`[3]`，再合并`[2,4]`与`[1,3]`；大例子则按段长 1、2、4、8 逐轮合并。它省去递归调用栈，仍需要合并的辅助存储。该书代码用两端包含的区间，本书代码用切片或半开区间，边界公式须各自配套。

6.4 边归并边数跨界逆序，不再一对对枚举

刚才左`[2,4]`、右`[1,3]`：取右 1 时，左剩 2、4 都大于 1，立刻加 2；取左 2 不加；取右 3 时左剩 4，加 1；最后 4 不加，总跨界逆序 3。原数组的三对恰是(2,1)、(4,1)、(4,3)。

一般取到严格更小的右项时，左列剩余 `len(L)-i` 项全比它大，一次加这个数。相等不计逆序。左内部、右内部、跨界三类互不重叠，递归数前两类、当前合并数第三类，总时间仍$\Theta(n \log n)$。计数上限$n(n-1)/2$可能超 32 位，即使每项值只需 32 位也要留意计数类型。

6.5 例 4：选择排序，“取最后一个最大值”仍可能不稳定

选择排序每轮在未排区找最大项，与未排区末项交换。即使在相等最大项中选最后一个，也不能保证稳定。考虑`[(2,A),(1,B),(2,C),(1,D)]`。

第一轮选最后一个最大项 2C，与末项 1D 交换，得到 `2A,1B,1D,2C`。第二轮在前三项里选最大 2A，与 1D 交换，得到 `1D,1B,2A,2C`。两个关键码 1 的记录已经从 B 在 D 前变成 D 在 B 前。最大项的等值选择规则没有阻止另一个关键码的记录被远距离交换越过。

若改成取出选中项，再把中间项整体平移，最后放到目标位置，可以做稳定的选择过程，但移动成本会改变。链表中也可通过摘下节点再接入目标位置实现不同的稳定版本。判断稳定性要跟着具体动作走，不能只跟算法名字走。

选择排序即使输入已排序，仍需扫描每个未排区找最大，比较次数 $\Theta(n^2)$；交换通常只有 O(n)次。若元素交换特别贵而比较便宜，它可能有自己的使用理由，但大型一般排序通常更偏向其他方法。

6.6 什么时候用哪一种

小数组、接近有序、插入到已有序列：插入排序常易实现且能利用较少逆序。需要稳定且能接受辅助数组：归并排序有可靠的 n log n 最坏界。需要理解相邻交换与逆序：冒泡是清楚的教学模型。若只要最小的 k 项或第 k 项，完整排序可能做了超出需求的工作，18 与 20 章会介绍堆与选择。

比较排序还存在 Ω(n log n)最坏下界，但它不适用于所有输入限制。计数排序对小整数键可用额外范围空间换时间；位图甚至只保存是否出现，不能直接替代带重复计数与记录标签的稳定排序。先看问题允许读取哪些信息，再谈下界。

6.7 相邻交换的逆序变化，为什么恰好只有一对

相邻位置放 x>y，交换为 y,x。二者之间的逆序消失。对左侧任意 z，它与 x、y 的两次比较总贡献只与数值有关，不受 x、y 内部顺序影响；右侧同理。因为中间没有第三项，不会出现其他关系变化，所以逆序总数恰减 1。若交换不相邻，就可能一次消除或新增许多逆序，不能继续把交换数等同初始 I。

例如[3,2,1]有 3 个逆序，直接交换首尾便得到[1,2,3]，只用 1 次任意交换。冒泡需要 3 次相邻逆序交换。两种“交换次数最少”的问题允许动作不同，因此答案不同。28 章的置换环 n−c 下界适用于任意两位置交换，而不是相邻交换。

6.8 多关键字排序，稳定性的用途可以直接看见

学生记录为(班级,成绩,姓名)。先按姓名排序，再稳定地按成绩排序，再稳定地按班级排序，最终以班级为第一关键字、成绩为第二、姓名为第三。后一次稳定排序保留相同新关键字内部已经排好的旧顺序，所以应从最低优先级开始。

若先按班级，再稳定按成绩，最终第一关键字会变成成绩，而非原先班级。稳定性保存的是等值组里的旧次序，不会阻止后一次排序把不同新关键字的记录跨班移动。把三条记录写出来就能检验：甲班 90、乙班 80、甲班 70，最后按成绩全局排序会混开班级。

如果语言排序支持比较器直接比较元组，也可以一次按(班级,成绩,姓名)排序；若比较器在相等关键字上又引入了与原顺序无关的判定，就要重新看它是否满足严格弱序等要求。比较函数自相矛盾，例如同时认为 a<b、b<c、c<a，会让排序的基本假设失效。

6.9 随机排列的平均逆序数，逐对算最清楚

n 个互异元素均匀随机排列，给任意一对数 x<y，x 在 y 前与 y 在 x 前各一半。为这对定义指标 X，逆序取 1，否则 0，期望 1/2。全体逆序数是所有 n(n−1)/2 个指标之和，期望为 n(n−1)/4。各对指标并不相互独立，但线性期望不需要独立。

因此稳定插入排序的平均移动数为二次级，尽管它在有序输入上线性。随机输入假设只用于这个平均结论，最坏平方与近有序 Θ(n+I)是另外两种描述。若现实数据本来由昨天的排序结果局部修改而来，均匀随机排列未必是合理模型。

6.10 每个输出记录也需要被读取

即使输入已经有序，要确认并输出全部记录，通常仍需线性遍历。最好时间可以线性，不可能仅因没有交换就把整个排序检查叫零成本。

6.11 练习与答案

练习 1。 给`[3,1,2,1]`列出所有逆序对，并预测稳定插入排序的元素右移总次数。

解答：3 与后面的 1、2、1 组成 3 对；第一个 1 与后面不构成逆序；2 与最后的 1 组成 1 对，共 4。插入第一个 1 时移 3 一次；插入 2 时移 3 一次；插入最后 1 时移 3、2 两次，共 4 次。两个 1 因严格大于时才移动，原先的 1 仍在后来的 1 前面。比较次数还包括发现无需继续移动的那一次，因此不必恰好等于 4。

练习 2。 归并 L=`[1,3,3]`和 R=`[2,3]`，跨区逆序多少？相等时取右侧会发生什么？

解答：取 1 不增；取 2 时 L 剩 3、3，增 2；以后全是相等 3，不新增逆序，答案 2。若相等时先取右侧，数值排序依然正确，但 R 里的 3 会越过 L 里的 3，失去稳定性。若统计代码把“右侧被取出”一律视为产生逆序，还会把相等项误算，必须把严格大小与相等分开。

练习 3。 对 n 项互异数组，逆序对为 0 能否推出已升序？I 很小能否推出每个元素离最终位置都很近？

解答：I=0 意味着任意 i<j 都有 a[i]<a[j]，因此升序。I 小能约束总的错位，但不保证每一项距离都小。例如`[2,3,4,5,1]`只有 4 个逆序，末项 1 却需要向左走 4 格。说“几乎有序”之前应固定度量，逆序数、最大位移与无序段长度并非同一指标。

7 链表：改一根指针，究竟改了谁通向谁

向量通过下标算地址，链表通过节点里存的链接找邻居。节点可以相隔很远，只要链接正确，逻辑上仍然相邻。因此看链表图时，横向画得近不代表内存地址接近，箭头才决定谁在谁后面。
一个双向节点有三个字段：`data` 存用户数据，`pred` 存前驱节点地址，`succ` 存后继节点地址。`p.succ=q` 只改 `p` 里面的一格地址，不会搬动 `q`，也不会自动替你把 `q.pred` 改成 `p`。

7.1 两个方向是两根独立链接

用 `H`、`T` 作头尾哨兵，它们不保存用户数据。初始表：

```
向右读succ:  H → A → B → C → T
向左读pred:  H ← A ← B ← C ← T
用户序列:       A   B   C
```

写成 `H↔A↔B↔C↔T` 时，每个双箭头只是两根单向链接的简写。空表应是 `H.succ=T,T.pred=H`，不是把哨兵也删掉。H 之前、T 之后不属于用户序列。
“已知节点 B”与“已知第 1 项”成本不同。若已有 B 的地址，可以直接改附近链接；若只拿到秩 1，需先从 H 沿链接找 B。长表的第九千项不能仅凭一个乘法得到地址。下面每个常数时间结论都先假设相关节点地址已知。

7.2 在 B 之前插 X，四次写入逐个检查

原来 `A.succ=B,B.pred=A`。先保存 `q=B,p=q.pred=A`，申请新节点 `x=X`。`p` 必须先存好，不能改了 `q.pred` 再指望它仍代表旧 A。

动作	A.succ	X.pred	X.succ	B.pred	目前能说什么
申请 X 后	B	未设	未设	A	X 还没接入
X.pred=p	B	A	未设	A	X 知道左邻
X.succ=q	B	A	B	A	X 知道右邻
p.succ=X	X	A	B	A	向右能经过 X，反向还未完整
q.pred=X	X	A	B	X	两个方向都成为 A↔X↔B



图 45　插入过程中四个链接字段的逐状态图

图把每个节点的 pred、succ 字段逐行写出，最后一行绿色表示四个接点已经全部就绪。中间状态只保证局部正在施工，不应同时让另一个遍历者使用未完成的表。这是单线程操作推演，不能从四条赋值推断并发安全。

若不保存 `p`，先写 `q.pred=x` 再写 `q.pred.succ=x`，后句实际变成 `x.succ=x`，X 自环，A 仍指 B。错误来自旧地址已经被覆盖，不是“指针很玄学”。

代码写清楚每个接点：

```
p = q.pred
x = 新节点(value)
x.pred = p
x.succ = q
p.succ = x
q.pred = x
size = size + 1
```

在首项前插入时 `p=H`，在尾部插入时 `q=T`，空表时两者都是哨兵，仍用同一套四次更新。不读哨兵 data，不需要再发明空表专用插入公式。

7.3 删除 B 时，先让两边能绕过它

对 A↔B↔C，先保存 p=B.pred=A,q=B.succ=C，若需返回 B 的数据也先保存。之后：

动作	向右路径	向左路径	B 是否仍可读取
初始	A→B→C	C→B→A	可以
A.succ=C	A→C	C→B→A	可以，反向待修
C.pred=A	A→C	C→A	可以，但不再属于表
释放 B，size 减 1	A↔C	双向完整	不能再读取 B 字段

先释放 B 再取 B.succ 是释放后使用。旧内存看起来还保留地址，也不代表对象还活着。外部保存的 B 迭代器同时失效，不会自动改成 C。

删除所有值 2 时，以[2,1,2,2,3]为例，每轮先保存 next=cur.succ，再决定删不删 cur，最后 cur=next。状态依次为[1,2,2,3]、保留 1、[1,2,3]、[1,3]、保留 3。连续的两个 2 都会处理，原因是每次保存的是原来的直接后继。若删除后再读 cur.succ，就读了已经释放的节点。

7.4 单向反转：覆盖旧后继前先拿一份

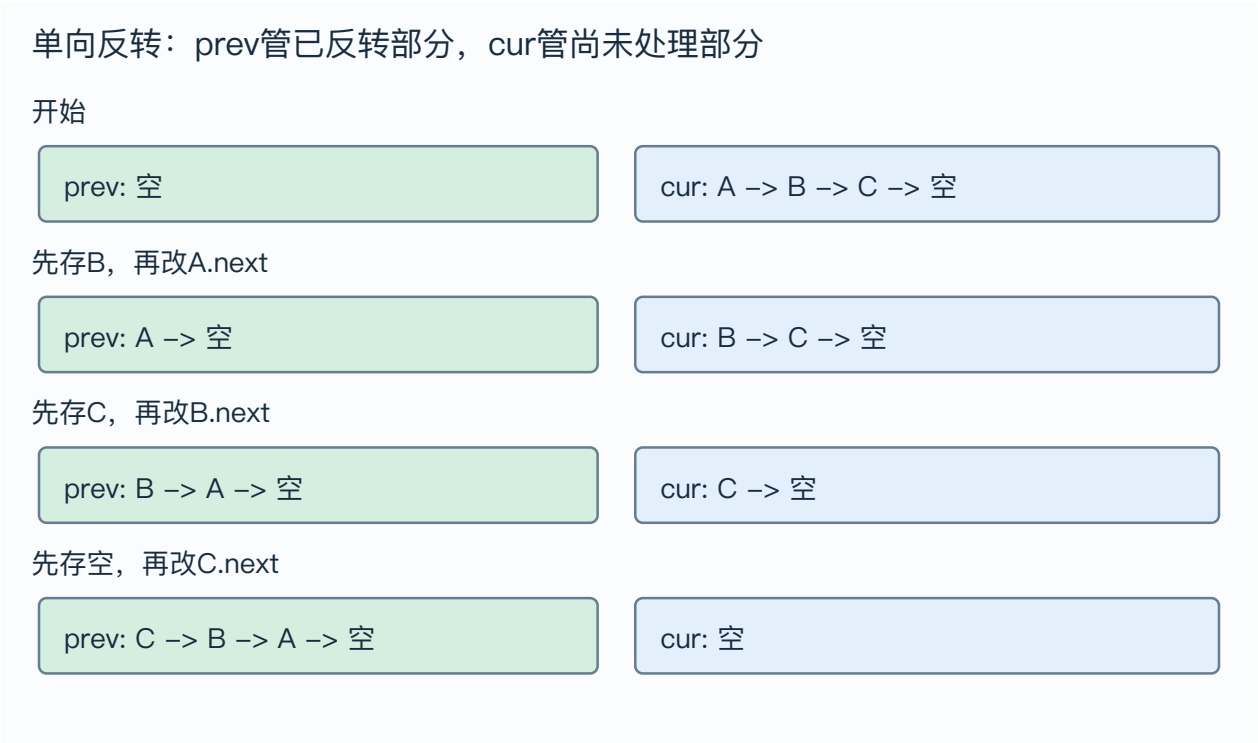


图 46　反转前缀与未处理后缀的四个状态

单向表只有 next，没有 pred。A→B→C→空要变成 C→B→A→空。三个变量分别管已反转部分、当前节点、尚未处理后继：

```
def reverse(head):
    prev = None
    cur = head
    while cur is not None:
        nxt = cur.next
        cur.next = prev
        prev = cur
        cur = nxt
    return prev
```

轮	进入时 prev,cur	先保存 nxt	写 cur.next	再推进 prev,cur	两部分现状
1	空,A	B	A→空	A,B	已反转 A；待处理 B→C
2	A,B	C	B→A	B,C	已反转 B→A；待处理 C
3	B,C	空	C→B	C,空	已反转 C→B→A；待处理空

`cur` 为空时停止，`prev` 是新头。任何一轮，已反转前缀与未处理后缀合起来恰好是原来的全部节点，没有丢节点也没有重复。每轮从后缀取一个，故终止且时间线性、辅助变量数常数。

若写完 `cur.next=prev` 就用 `cur=cur.next` 前进，第一轮会沿刚写的空链接退出，B、C 从新头再也到不了。反转图要同时看“原来朝右的链接”和“现在朝左的链接”，不能只按纸上左右位置猜下一步。

双向表反转可以逐节点交换 `pred,succ`，最后交换头尾哨兵角色；遍历仍需先存旧后继，包括两个哨兵。只有把用户节点倒过来却不改哨兵，会使边界指向错误。

7.5 排序时搬值还是搬节点

链表插入排序可以遍历未排序节点，在已排序前缀中寻找位置，将当前节点摘下并接进去。寻找位置最坏 O(n)，摘接本身 O(1)，总时间仍可能 $\Theta(n^2)$。链表让“挪动一个已经找到的节点”便宜，却没有免除比较。

取`[(3,A),(1,B),(2,C),(1,D)]`。先保留 3A；把 1B 接到 3A 前；把 2C 接到 1B 和 3A 之间；最后将 1D 插在已有 1B 之后、2C 之前，得到 1B、1D、2C、3A。若寻找插入点时越过所有≤当前值的旧节点，等值顺序保持，排序稳定；若停在第一个相等节点前，D 会跑到 B 前面。

链表归并两个已排序子表，可以不断比较头节点，把较小的节点摘下接到输出尾部，避免数组里的大规模搬移。一次归并仍是线性，因为每个节点只取出一次；递归归并排序总时间 $\Theta(n \log n)$。若通过切片复制节点数据再排序，就不再是同一份“只改链接”的空间账。

把上一段的链表插入排序也按比较走完。原节点身份固定为`3A,1B,2C,1D`，每轮先保存当前节点的旧后继，防止摘接后失去下一个待处理节点：

当前节点	从有序前缀尾向左比较	找到的前驱	摘接后用户序列
1B	3A>1B，向左到 H	H	1B,3A,2C,1D
2C	3A>2C；1B<=2C，停止	1B	1B,2C,3A,1D
1D	3A>1D；2C>1D；1B<=1D，停止	1B	1B,1D,2C,3A

每次先把当前节点从旧位置两边接开，再按本章插入步骤接到指定前驱之后。旧 1B 等于新 1D 时停止向左，因此 D 放在 B 后，保持稳定。若只复制 data 而不移动节点，外部指向 B 的地址仍是同一地址，里面的学生身份却可能被换掉；两种接口行为要先约定。

链表归并`[2,4]`与`[1,3]`时，比较头 2/1，摘 1 接输出尾；比较 2/3，摘 2；比较 4/3，摘 3；右表空，接剩余 4。每次取下一节点前先保存旧后继，输出尾的 succ 与新节点 pred 成对更新。比较与数组归并相同，搬迁从“复制格子值”换成“重接已知节点”，不是省略了寻找和比较。

7.6 深复制与析构，数据结构还要负责谁活着

浅复制只复制头指针，两个对象会共同指向同一批节点。若两者都认为自己拥有节点，析构时可能释放两次；一方删节点，另一方也会持有失效指针。深复制会新建每个节点，并按原顺序拷贝数据，两个链表拥有不同节点。

数据字段若本身是指针，还要继续定义它指向的对象归谁所有。复制节点并不自动等于复制所指对象。实际 C++可以用明确所有权的智能指针或遵守拷贝/移动规则的容器设计解决，但不能靠“我只复制三个字段”回避接口含义。

循环链表让尾节点链接回头部，适合轮转。遍历不能再用 `p!=null` 退出，应知道回到哪个哨兵，或准确走 size 步。若遍历条件仍等待 null，就会永远绕圈。判断链表是否意外成环，可以用快慢指针：慢每次走 1 步，快走 2 步；若有环，进入环后相对位置每轮改变 1，最终会相遇。若无环，快指针先到空。

7.7 什么时候链表有优势

已知节点附近频繁插删、需要稳定节点地址、需要把整段节点在容器间摘接时，链表可能合适。大量按下标访问、顺序扫描紧凑数据、对内存占用敏感时，向量常更好。链表每项多存链接，节点可能分散在内存，缓存局部性通常较差；渐近同为 O(n)的遍历，实际耗时仍可能明显不同。

单向与双向也有区别。给定 p，要删 p 的后继在单向表中容易；要删 p 本身，通常还需要知道它的前驱。某些“把后继数据复制到 p 再删后继”的技巧不适用于末节点，也会改变节点身份，不能作为通用删除替代品。

7.8 整段摘接，O(1)的前提要逐一数出来

假设已知一段连续节点首 a、尾 b，以及目标节点 q，想把[a,b]移到 q 之前。先保存旧前驱 p=a.pred、旧后继 r=b.succ，将 p 与 r 相连；再保存 q 的旧前驱 t，将 t 接 a、b 接 q，并修正反向链接。节点数据完全不用复制，已知所有端点且目标不在被移动段内部时，只改常数根链接。

若 a、b 只由秩给出，先找到它们可能要线性遍历；若需要更新两张列表的 size，却不知道这段长度，还要先数节点。若 q 在[a,b]内部，直接按普通摘接操作可能形成环或断链，需要明确拒绝或视为空操作。所谓链表 splice 为常数时间，通常隐含了位置已知、必要计数已知或接口不要求那笔工作的条件。

用 H↔A↔B↔C↔D↔T，把[B,C]移到 A 前：断 A 与 B 之间、C 与 D 之间，接 A↔D；再把 H↔A 之间插入整段，接 H↔B、C↔A，结果 H↔B↔C↔A↔D↔T。B 与 C 内部链接不改，证明可以按原段内部、原段外部与新边界三部分检查。

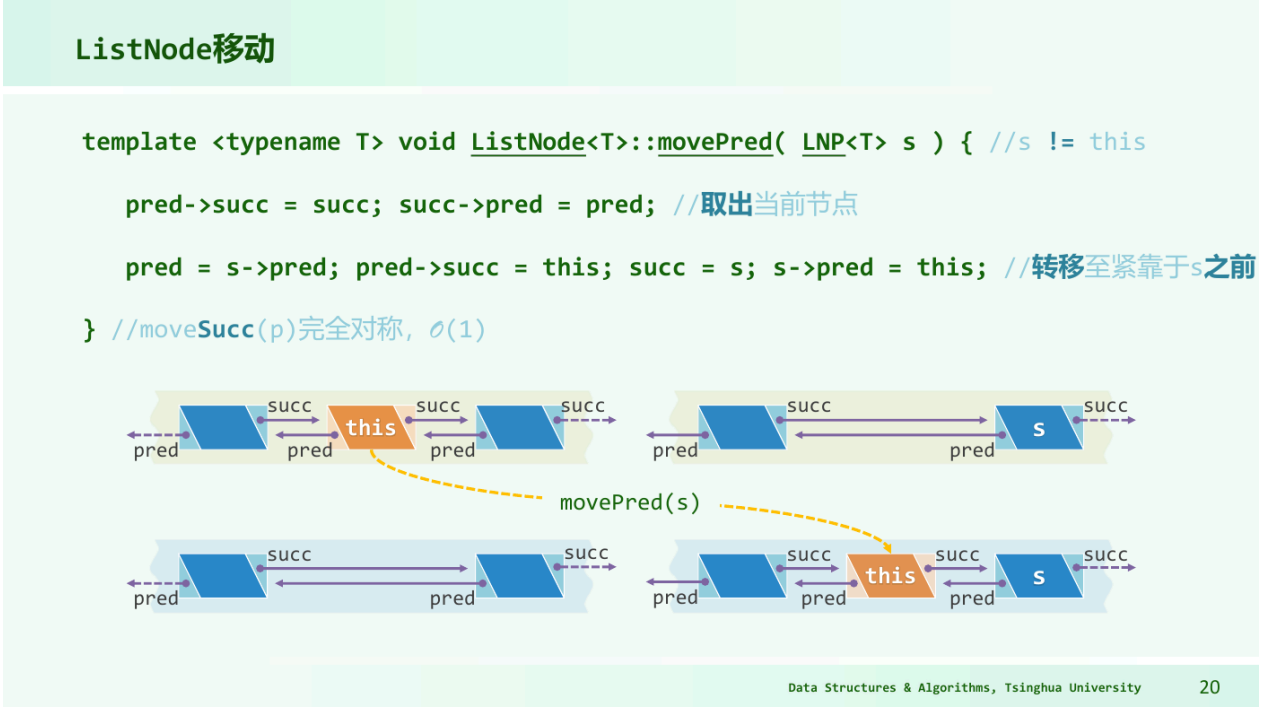


图 47　教师移动一个节点的完整原页

《03 列表》PDF21 页中，this 是被移动的节点，s 是目标位置。上半左图是旧邻居，上半右图是目标附近；下半左图表示旧邻居已经直接接起来，下半右图表示 this 已接到 s 前。代码先执行两条“绕过 this”的赋值，再执行四条“接到 s 前”的赋值，正是本章删除边界与插入边界的组合。这里只移动一个节点，若推广成整段，内部链接保持，仍需核对六个外部边界和目标不在段内的条件。

把[B,C]整体从 H↔A↔B↔C↔D↔T 移到 A 前，六个外部字段可以逐项列出：

赋值	效果
A.succ=D	旧位置向右绕过整段
D.pred=A	旧位置向左也绕过整段
H.succ=B	新位置的左邻接到段首
B.pred=H	段首接回新左邻

赋值	效果
C.succ=A	段尾接到目标 A
A.pred=C	目标接回新段尾

B 与 C 之间两根链接从头到尾不改，结果 H↔B↔C↔A↔D↔T。前面保存的端点不能在过程中靠已经被覆盖的字段重新推算。

7.9 有序链表二分，为什么比较少了却仍要走路

你可以先数出长度，再找中点比较，保留一半，再找下一中点。若每次只在保留区间内顺序走到中点，路径长度可能为 n/2+n/4+...=O(n)，比较只有 O(log n)，总时间仍 O(n)，没有变成向量的 O(log n)。若每次又从全表头重新走中点，甚至可能重复更多路程。

因此“链表不能二分”更准确的意思是：它缺少按秩常数时间定位，普通二分无法得到同样的时间优势。若比较对象特别昂贵，减少比较仍可能有意义，但别把指针移动成本删掉。跳转表给链表增加多层索引，才改变快速到达远处位置的能力。

快慢指针检测环也可以画出精确位置。若 A→B→C→D→B，慢每轮一步、快每轮两步，初始都在 A：第 1 轮慢 B 快 C；第 2 轮慢 C 快 B；第 3 轮慢 D 快 D，相遇，说明有环。无环时快指针或它的后继先变空，应在走两步前检查这两项，不能先解引用空指针。进入环以后，两者的相对位置每轮推进一格，有限环长保证相遇；相遇点未必就是环入口。

7.10 链表调试中的回路检查

双向表从 H 沿 succ 走到 T，数到的用户节点应恰好 size；从 T 沿 pred 走回 H 也应恰好 size，正反序列应互为逆。对每个非边界节点，检查 p.succ.pred==p 与 p.pred.succ==p。这些条件联合起来能抓住漏改一根链接、误自环与 size 没有同步更新。

只检查正向打印可能漏掉反向链接错误；只检查每节点局部互指又可能没发现一批节点从主表脱离成另一个环。结构验证需要同时检查可达性、节点数量与双向一致。修改代码时，先用空表、单节点、两节点和连续删除追踪，通常比一开始就用长随机表更易定位问题。

7.11 环上删除与约瑟夫递推，计数起点仍要明确

若固定每次数到第 k 人删除，剩余者从下一人重新起数，零下标幸存者满足 J(1,k)=0，J(n,k)=(J(n−1,k)+k) mod n。缩小问题中零点对应原环删人之后的下一位，所以恢复原编号要平移 k，而不是随意加 k 减一。

例如 n=5、k=2，依次删除编号 1、3、0、4，幸存 2；递推从一人到五人得到 0、0、2、0、2。链表模拟便于理解删除，递推只求最后幸存者可省去逐次走环。若题目步长每轮变化、恢复的是空座秩，不能不核对规则就套同一递推；21 章 Interview 处理的是更一般的动态空位定位。

只有一个数据节点的循环表，删除后必须让哨兵重新指向自己；若无哨兵，则头指针应变空。不能留下指向已释放节点的头指针，因为下一次入队或遍历会把无效地址当作合法位置。

7.12 练习与答案

练习 1。 空双向链表含 H、T 两个哨兵。在 T 前依次插入 7 和 9，再删除 7，写出所有关键链接。

解答：初始 H.succ=T、T.pred=H。插 7 后 H.succ=7、7.pred=H、7.succ=T、T.pred=7。插 9 后 7.succ=9、9.pred=7、9.succ=T、T.pred=9。删 7 后 H.succ=9、9.pred=H，其余 9.succ=T、T.pred=9 保留。size 依次 0、1、2、1。若删除后 H 仍指 7，就说明只改了反向链接，结构不完整。

练习 2。 已知单向链表头指针，想删掉第 n/2 项并重复 1000 次。为什么不能直接说总成本 O(1000)？

解答：每次先从头找到待删节点的前驱，最坏要走 Θ(n)步，局部改链虽 O(1)，整次仍 Θ(n)。若 size 随删除减少，准确总成本是各次当前长度的和的一部分，通常仍与 1000n 同阶。只有外部已经持有每次需要的前驱节点，且这些指针在操作后仍有效，才可只计算常数次改链。

练习 3。 反转单向表时先写 `cur.next=prev`，随后用 `cur=cur.next` 前进，会出现什么？

解答：第一次若 prev=null，cur.next 被改成 null，随后 cur 也变 null，算法立即停止，原后半段丢失。应在覆盖链接之前保存旧 next，再用它前进。用三节点小图追踪，比记住“三指针口诀”更容易看见错误究竟发生在哪一行。

8 栈：把没做完的事留好，先处理最近的一件

栈是一种限制访问顺序的容器。只能在同一端添加、删除，这一端叫栈顶；先放进去的在下面，后来放进去的在上面。push 压入一项，pop 移除并返回顶项，top 只看不移除。空栈不能弹，程序应明确报错。

```
本章横向写法：[栈底, ..., 栈顶]
push A → [A]
push B → [A,B]
pop    → 返回B，剩[A]
pop    → 返回A，剩[]
```

要记的不是英语缩写，而是“最近放进去的先拿出来”。括号嵌套、函数调用、表达式的待办运算都遵循这个顺序。

8.1 括号匹配，右括号只能关闭最近的左括号

读{[()]}，把尚未关闭的左括号放栈：

字符位置	读到	先查什么	动作后栈
0	{	左括号，无需匹配	大括号
1	[	左括号	大括号，方括号
2	(	左括号	大括号，方括号，圆括号
3	)	非空且顶为(	大括号，方括号
4	]	非空且顶为[	大括号
5	}	非空且顶为{	空栈
结束	无	必须没有未闭合项	合法

表中列的是尚未闭合的左括号类型，从底到顶。每个右括号都必须与最上面的类型核对。

([)]读完前两项，栈底到顶为(,[，第三项)要求顶部为(，但实际为[，当场失败。(()的每个已读右括号都能匹配，却在结束时还剩一个(，也失败。只统计左右数量不能检测交叉或遗留未闭合项。

每个字符至多压入或弹出一次，时间线性；所有字符都是左括号时，栈可长到输入长度，最坏空间线性。

8.2 直接算 3*(2+4)：两个栈分别等什么

数字栈 V 保存已经读到或算出的值，运算符栈 O 保存尚未执行的符号。这里语法只含整数、二元+,-,*,/与圆括号；乘除优先于加减，同级左结合，先用精确数学除法理解。负号若是一元取负，要另外识别，不能直接当二元减法。

另用 # 作为控制标记：开始时只在符号栈底放一个 #，输入末尾也添一个 #。它不参与算术。读到末尾 # 时，先执行尚未完成的真实运算；最终两个 # 相遇并一起消耗，才结束。数栈初始仍为空。

```
记号序列：3  *  (  2  +  4  )  #
V保存数值；O保存待完成运算；每个栈都从底写到顶。
```

当前记号/子步骤	为什么执行这一步	O 变化后	V 变化后
3	数值已知，压入	[#]	[3]
*	没有更早运算需先算，压入	[#, *]	[3]
(	开始一个隔离的子表达式，压入	[#,*,(]	[3]
2	压入数值	[#,*,(]	[3,2]
+	栈顶为(，不跨括号执行*，压入+	[#,*,(,+]	[3,2]

当前记号/子步骤	为什么执行这一步	O 变化后	V 变化后
4	压入数值	[#,*,(,+]	[3,2,4]
遇)	必须先算完括号内部，弹+	[#,*,(]	[3,2,4]
执行+第 1 步	先弹右操作数 4	[#,*,(]	[3,2]
执行+第 2 步	再弹左操作数 2	[#,*,(]	[3]
执行+第 3 步	算 2+4=6 并压回	[#,*,(]	[3,6]
处理完)	弹出配对(，不把括号当运算	[#, *]	[3,6]
输入结束	弹出仍等待的*	[#]	[3,6]
执行*	弹右 6、左 3，算 18 压回	[#]	[18]
两个 # 相遇	消耗输入 # 并弹出栈底 #	[]	[18]
最终检查	运算符栈空，数值恰一个	[]	返回 18

每幅图同时保留数栈、符号栈和弹出后暂存的左右操作数。栈横向从底画到顶；格下的 0、1 等仅标示栈内槽位。

3×(2+4)：两只栈怎样求值

1. 开始，先放哨兵#

输入：3 * (2 + 4) #；当前读取 3

数栈：底→顶 空

符号栈：底→顶 #

0

弹出后：左/右 未取 未取

0 1

数栈为空，符号栈只有#。
原因：哨兵标记栈底，输入末尾#负责触发剩余运算。

图 48 逐步演示：3×(2+4)：两只栈怎样求值

3×(2+4)：两只栈怎样求值

2. 数字3进数栈

输入：3 * (2 + 4) #；当前读取 *

数栈：底→顶 | 3 | (0)

符号栈：底→顶 | # | (0)

弹出后：左/右 | 未取 | 未取 | (0) (1)

读取3后，转到*。
原因：数字是运算对象，直接保留到后续运算。

图 49　3×(2+4)：两只栈怎样求值，第 2 步

3×(2+4)：两只栈怎样求值

3. 乘号进符号栈

输入：3 * (2 + 4) #；当前读取 (

数栈：底→顶 | 3 | (0)

符号栈：底→顶 | # | * | (0) (1)

弹出后：左/右 | 未取 | 未取 | (0) (1)

栈顶#，压入*。
原因：目前没有可先执行的旧运算。

图 50　3×(2+4)：两只栈怎样求值，第 3 步

3×(2+4)：两只栈怎样求值

4. 左括号进符号栈

输入：3 * (2 + 4) #；当前读取 2

数栈：底→顶 [3] (0)

符号栈：底→顶 [#] (0) [*] (1) [(] (2)

弹出后：左/右 [未取] (0) [未取] (1)

压入(，暂时阻止括号外的*参与归约。
原因：括号内应先计算，尚未得到右操作数的*要等待。

图 51　3×(2+4)：两只栈怎样求值，第 4 步

3×(2+4)：两只栈怎样求值

5. 数字2进数栈

输入：3 * (2 + 4) #；当前读取 +

数栈：底→顶 [3] (0) [2] (1)

符号栈：底→顶 [#] (0) [*] (1) [(] (2)

弹出后：左/右 [未取] (0) [未取] (1)

读取2。
原因：括号内第一个运算对象是2。

图 52　3×(2+4)：两只栈怎样求值，第 5 步

3×(2+4)：两只栈怎样求值

6. 加号进符号栈

输入：3 * (2 + 4) #；当前读取 4

数栈：底→顶

0	1
3	2

符号栈：底→顶

0	1	2	3
#	*	(	+

弹出后：左/右

0	1
未取	未取

读取+，栈顶(，压入+。
原因：左括号只划分区域，不进行数值运算。

图 53　3×(2+4)：两只栈怎样求值，第 6 步

3×(2+4)：两只栈怎样求值

7. 数字4进数栈

输入：3 * (2 + 4) #；当前读取)

数栈：底→顶

0	1	2
3	2	4

符号栈：底→顶

0	1	2	3
#	*	(	+

弹出后：左/右

0	1
未取	未取

读取4。
原因：加法现在已有两个操作数，但仍等下一符号确定归约。

图 54　3×(2+4)：两只栈怎样求值，第 7 步

3×(2+4)：两只栈怎样求值

8. 先取出待执行的加号

输入：3 * (2 + 4) #；当前读取)

数栈：底→顶

0	1	2
3	2	4

符号栈：底→顶

0	1	2
#	*	(

弹出后：左/右

0	1
未取	未取

符号栈弹出+，数栈暂时不变。
原因：右括号要求先完成它配对左括号里面的运算。

图 55　3×(2+4)：两只栈怎样求值，第 8 步

3×(2+4)：两只栈怎样求值

9. 先弹出右操作数4

输入：3 * (2 + 4) #；当前读取)

数栈：底→顶

0	1
3	2

符号栈：底→顶

0	1	2
#	*	(

弹出后：左/右

0	1
未取	4

暂存b=4。
原因：后进栈的是右操作数；减法和除法尤其不能交换。

图 56　3×(2+4)：两只栈怎样求值，第 9 步

3×(2+4)：两只栈怎样求值

10. 再弹出左操作数2

输入：3 * (2 + 4) #；当前读取)

数栈：底→顶 | 3 | (0)

符号栈：底→顶 | # | * | (| (0 1 2)

弹出后：左/右 | 2 | 4 | (0 1)

暂存a=2，b=4，执行刚取出的加号。
原因：两个操作数必须按原表达式的左右顺序组合。

图 57　3×(2+4)：两只栈怎样求值，第 10 步

3×(2+4)：两只栈怎样求值

11. 把计算结果6压回

输入：3 * (2 + 4) #；当前读取)

数栈：底→顶 | 3 | 6 | (0 1)

符号栈：底→顶 | # | * | (| (0 1 2)

弹出后：左/右 | 未取 | 未取 | (0 1)

2+4=6，右括号仍未消耗。
原因：一个子表达式归约成一个数，供外层运算使用。

图 58　3×(2+4)：两只栈怎样求值，第 11 步

3×(2+4)：两只栈怎样求值

12. 匹配括号并消耗右括号

输入：3 * (2 + 4) #；当前读取 #

数栈：底→顶 [3] [6]
0 1

符号栈：底→顶 [#] [*]
0 1

弹出后：左/右 [未取] [未取]
0 1

只弹出(；不弹出数值6。
原因：括号完成控制作用，不是算术操作符。

图 59　3×(2+4)：两只栈怎样求值，第 12 步

3×(2+4)：两只栈怎样求值

13. 取出待执行的乘号

输入：3 * (2 + 4) #；当前读取 #

数栈：底→顶 [3] [6]
0 1

符号栈：底→顶 [#]
0

弹出后：左/右 [未取] [未取]
0 1

末尾#触发弹出*，数栈仍为[3,6]。
原因：所有括号已关闭，符号栈中真实运算仍须完成。

图 60　3×(2+4)：两只栈怎样求值，第 13 步

3×(2+4)：两只栈怎样求值

14. 弹出乘法的右操作数6

输入：3 * (2 + 4) #；当前读取 #

数栈：底→顶 3
0

符号栈：底→顶 #
0

弹出后：左/右 未取 6
0 1

暂存b=6。
原因：末尾#优先级低，要求执行尚未完成的乘法。

图 61　3×(2+4)：两只栈怎样求值，第 14 步

3×(2+4)：两只栈怎样求值

15. 弹出左操作数3

输入：3 * (2 + 4) #；当前读取 #

数栈：底→顶　空

符号栈：底→顶 #
0

弹出后：左/右 3 6
0 1

暂存a=3，b=6，计算3×6。
原因：此时括号内已经完成，乘法的两个对象都已确定。

图 62　3×(2+4)：两只栈怎样求值，第 15 步

3×(2+4)：两只栈怎样求值

16. 结果18压回数栈

输入：3 * (2 + 4) #；当前读取 #

数栈：底→顶 18
0

符号栈：底→顶 #
0

弹出后：左/右 未取 未取
0 1

数栈只剩18，符号栈只剩#。
原因：所有真实操作符均已执行。

图 63　3×(2+4)：两只栈怎样求值，第 16 步

3×(2+4)：两只栈怎样求值

17. 哨兵相遇，结束

输入：3 * (2 + 4) #；当前读取 结束

数栈：底→顶 18
0

符号栈：底→顶 空

弹出后：左/右 未取 未取
0 1

输入#与栈底#匹配；返回18。
原因：合法表达式结束时应恰好留下一个数，不能有多余括号或操作符。

图 64　3×(2+4)：两只栈怎样求值，第 17 步

数字 3 先来，却必须等括号内的 6 算好。图中(是边界，不参与数值计算。V 里同时存在 3、2、4 的时刻，数字峰值为 3；O 里是 #,*,(,+时，括号和栈底哨兵也计入，运算符存储峰值为 4；若题目明确不计哨兵，则为 3。换一个表达式，两种峰值未必相同。

先弹的是右操作数。验证减法 `8 3 -`，先拿 3 再拿 8，必须算 8−3=5；顺序反了就得到−5。这条规则与加乘是否交换无关，接口应始终一致。

8.3 把等运算改成输出，就得到逆波兰式

若不立刻算数，而在“应执行运算”时把运算符写到输出末尾，`3*(2+4)`会变成 `3 2 4 + *`。这叫后缀式或逆波兰式：两个操作数出现在运算符前。重读一遍：压 3、压 2、压 4；遇+把 2 与 4 换成 6；遇*把 3 与 6 换成 18。

数值记号必须先分清：`12 3 +`是两个数，不是 1、2、3 三个数。二元运算前栈至少有两项，结束时恰一项。`2 +`前者失败；`2 3` 后者失败。数值足够仍可能除零，算术合法性要单独检查。

下面的后缀求值器可直接运行。输入已按空格分词，支持带符号整数和四个二元运算；不含括号，因为括号结构已由后缀顺序表达。用 `Fraction` 保存有理数，让 `9 2 /`精确得到 `9/2`，避免把整数除法或浮点舍入悄悄带入语义。

```
from fractions import Fraction

def evaluate_rpn(text):
    values = []
    for token in text.split():
        if token.lstrip("-").isdigit():
            values.append(Fraction(int(token)))
            continue
        if token not in {"+", "-", "*", "/"}:
            raise ValueError("不支持的记号")
        if len(values) < 2:
            raise ValueError("二元运算缺少操作数")
        right = values.pop()
        left = values.pop()
        if token == "+":
            values.append(left + right)
        elif token == "-":
            values.append(left - right)
        elif token == "*":
            values.append(left * right)
        else:
            if right == 0:
                raise ZeroDivisionError("除数为0")
            values.append(left / right)
    if len(values) != 1:
        raise ValueError("表达式结束后必须恰有一个值")
    return values[0]
```

用 `3 2 4 + *`得到 18，`8 3 -`得到 5；`2 +`、`2 3` 都会报格式错，`1 0 /`报除零。每次二元运算让栈长度净减 1，因此合法的 m 个数值、m−1 个二元运算最终可剩一项，但光有数量条件仍不足以保证每个前缀都有足够操作数。

8.4 相同优先级还要决定先算哪一个

`8-3-2` 按左结合是(8−3)−2=3。读第二个减号时，旧减号必须先执行：数值 8、3 合为 5，再压新减号，读 2 后得 3。若先把第二个减号也压入，就先算 3−2，结果 8−1=7，换了语法。

教师《04 栈与列表》PDF42 页的优先表里，栈顶^遇输入^先弹栈，规定乘方左结合，所以 `2^3^2` 按原程序得 64。数学中常用的右结合会得 512，必须改等优先级分支。本章^作为表达式记号表示乘方，C++源代码的同字符表示按位异或，不可直接照粘当乘方。

`-2^2` 的一元负号优先级、`3!!`是双阶乘还是连续阶乘，都需要先定词法和语法。双阶乘 `5!!=5×3×1=15`，不同于`(5!)!`。优先表只能处理已识别记号，不能替你猜输入含义。

Princeton 栈与队列一节中的双栈求值示例要求表达式带齐括号，因此能在右括号时执行一个运算。本章还允许省略不必要括号，便需要前面解释的优先级和同级结合性规则。若把前者的“忽略左括号”单独抄到后者，输入 `3*(2+4)`的等待边界就丢了；接口前提必须一起带过来。

8.5 函数调用栈，与自己写的栈有何关系

求 `factorial(4)`，调用会依次等待 factorial(3)、factorial(2)、factorial(1)。每层要保存返回到哪里、自己的参数及还未执行的乘法。最深处返回 1 后，依次乘 2 得到 2、乘 3 得到 6、乘 4 得到 24。栈帧是一次尚未返回的函数调用所需的现场。

把阶乘改成循环，只需一个累计积；递归不是一定必需。尾递归指递归调用之后已没有其他工作，有些编译器或语言会优化复用栈帧，但 C++并不承诺所有尾递归都会被优化。`return n*factorial(n-1)`还要乘 n，本身不是尾调用。

用显式栈改递归时，需保存同样的待办信息。二叉树后序遍历不能只压节点而不记录“左右孩子是否已经处理”；否则弹出时不知道下一步应访问节点还是继续走孩子。10 章会具体演示。

8.6 栈混洗：输入顺序固定，输出能有多少种

输入 1、2、3，按这个顺序进栈，可以随时弹出。输出 231 可行：压 1、压 2、弹 2、压 3、弹 3、弹 1。输出 312 不可行：为了先弹 3，必须先压 1、2、3；弹 3 后栈顶是 2，不能跳过 2 去弹 1。

判断一个给定输出是否可行，可以顺序扫描目标：若栈顶不是所需元素，就继续把尚未输入的项压入；若已经没有可压入项仍不匹配，则失败。每个元素入、出各一次，时间 O(n)，最坏空间 O(n)。这个贪心有必然性：要输出目标 x 之前，若栈顶不是 x，唯一合法动作就是继续压入，不能先弹其他值，否则输出序列就已经错了。

8.7 例 4：Catalan 计数，从第一个元素在哪里弹出拆开

n 个不同元素依序入栈，所有合法压入弹出操作序列共有 C_n 种。把 push 记作左括号，pop 记作右括号：任何前缀里 pop 不能多于 push，最后两者相等。于是问题等价于合法括号串计数。

$C_0=1$，表示空序列也有一种不做操作的安排。看第一个左括号对应的右括号：其内部含 k 对括号，后面含 n−1−k 对，内部与后部可独立选择，因此

$$C_n = \sum_{k=0}^{n-1} C_k C_{n-1-k}.$$

$C_1=1$；$C_2=C_0C_1+C_1C_0=2$；$C_3=C_0C_2+C_1C_1+C_2C_0=2+1+2=5$。三对括号为`((()))`、`(()())`、`(())()`、`()(())`、`()()()`。对应栈混洗 5 种，不等于全部 3!=6 种排列。

还可从全部含 n 个左括号、n 个右括号的串中扣除非法者。总数为 `binom(2n,n)`；将首个出现“右括号比左括号多 1”的前缀左右反转，可与含 n+1 个左括号、n−1 个右括号的串建立双射，非法数为 `binom(2n,n-1)`，差得到 C_n=`binom(2n,n)/(n+1)`。这个反射论证要说明映射可逆，不能只说“减掉一半”。

C_n 增长约为 `4^n/(√π n^(3/2))`，很快超过固定宽整数；精确计算应选足够大的类型或大整数。若只求模数余数，直接除以 n+1 未必合法，要使用递推或符合模数条件的逆元。

8.8 运算符表是语法的一部分，逐格读一次

中缀转换的栈顶运算符 a 与当前输入 b 之间有三类关系：先弹 a 并输出，或先压 b 等待，或括号配对后同时消去。以栈顶、当前`+`为例，乘法应先完成，所以弹；栈顶`+`、当前则应先读乘法右边，所以压；栈顶`(`、当前`)`是配对，不输出括号。

同级结合性就藏在对角格。`-`对`-`先弹，得到左结合；`^`对`^`先弹也为左结合；若要求右结合就改成先压。每修改一格，要重新检查连续链与括号外的关系，不能只改展示的一个例子。后缀阶乘遇下一运算符应先完成，但阶乘操作数必须是合法非负整数；这属于算术域检查，不是优先级表能替代的。

具体转换 `8-3-2`：输出 8，压`-`，输出 3，读第二个`-`时弹旧`-`并输出，然后压新`-`，输出 2，结束弹`-`，得 `8 3 - 2 -`，值 3。若错压第二个`-`不先弹，会得到 `8 3 2 - -`，值 7。一个三数链即可分辨结合性错误。

8.9 栈混洗计数中的空情况不能漏

Catalan 递推里 $C_0=1$，是“没有元素时有一种空操作序列”，并非说有一个空元素。若把 C_0设 0，C_1的递推会变 0，后面全部归零。很多组合递推依赖空对象作为拼接单位，正如加法空和为 0、乘法空积为 1。

把首个左括号配对拆开时，内部与后部各可以为空。n=3、内部 k=0 给()后接 2 对，贡献 2 种；k=1 给(())后接 1 对，贡献 1；k=2 给外括号包 2 对，后部空，贡献 2。逐个列出 5 串后，卷积式中的每项都有具体对应，不会把形式相似的“首弹元素分类”也盲套成同一卷积。

若按第一个弹出的输入编号 k 分类，初始必须先压 1 到 k 再弹 k，栈里已有 k−1 项，后面不再是两个无约束的独立合法括号串。需要考虑初始栈高度与剩余入栈数的 ballot 路径约束。分类变量变了，计数对象的边界条件也会变。

8.10 空间峰值为何要注明算哪个栈

表达式处理至少可能有运算符栈、数值栈以及递归解析器的调用栈。RPN`a b + c +`数值峰值 2，不说明中缀原式需要多少运算符等待；原式加了许多多余括号，运算符栈会更深，但 RPN 数值峰值不变。计数是否包含底部哨兵也会使精确答案差 1，先写约定再比较公式。

8.11 练习与答案

练习 1。 求 RPN `10 2 3 * - 4 /`，并写栈峰值。

解答：10 入栈；2 入栈；3 入栈，峰值 3；*弹 3、2 得 6，栈为 10,6；−弹 6、10 得 4；4 入栈；/得 4/4=1。若使用整数除法，此例仍为 1，但换成`9 2 /`时数学有理数为 9/2，C++整数除法为 4，必须固定语义。

练习 2。 输出 2413 能否由输入 1234 经一个栈产生？

解答：为输出 2，压 1、2 再弹 2；为输出 4，继续压 3、4 再弹 4；此时栈为 1,3，顶是 3，目标却要 1，而且已经没有新输入可压，故不行。失败位置比单说“不符合栈规则”更有信息：是 4 已经输出后，3 挡在 1 上面。

练习 3。 为什么二元 RPN 求值栈峰值不一定等于表达式树高？

解答：计算`(((a+b)+c)+d)`，先算 a+b 留下一个值，再加 c，再加 d，峰值只要 2，树却很高。计算`a+(b+(c+d))`时，a、b、c 需要依次等待，压入 d 时峰值 4。二者运算数与运算符数量相同，树形与既定求值顺序不同，空间就不同；31 章会写出对应递推式。

9 队列：保持先后次序，也学会丢掉已失效的候选

栈先处理最近加入的项，队列则先处理最早加入的项。enqueue 把新项接到尾部，dequeue 从头部取走，空队列不能出队。排队任务、层次遍历适合这种先进先出的规则。

9.1 环形数组的“顺序”要从队头读

数组有 5 格，记容量 C=5，front 指队头物理下标，size 记有效项数。第 k 个逻辑元素在(front+k)%C，这里 % 取非负整数除法余数。例如 6%5=1，表示跨过末格后绕回。

```
物理格子:   0   1   2   3   4
初始内容:  [空][空][空][空][空]    front=0,size=0
```

入队位置是(front+size)%C，先写值再加 size。出队先读 a[front]，再 front=(front+1)%C 并减 size。用下表做完一圈：

操作	算出的读取/写入格	物理数组	front,size	逻辑队列
入 A	写(0+0)%5=0	A,空,空,空,空	0,1	A
入 B	写 1	A,B,空,空,空	0,2	A,B
入 C	写 2	A,B,C,空,空	0,3	A,B,C
入 D	写 3	A,B,C,D,空	0,4	A,B,C,D
出 A	读 0，front 变 1	A,B,C,D,空	1,3	B,C,D
出 B	读 1，front 变 2	A,B,C,D,空	2,2	C,D
入 E	写(2+2)%5=4	A,B,C,D,E	2,3	C,D,E
入 F	写(2+3)%5=0	F,B,C,D,E	2,4	C,D,E,F

格 1 的 B 仍可能留着旧数值，但已不在有效逻辑区间。不能把物理数组从 0 打印到 4 就当队列。空为 size=0，满为 size=C。另一种只存头尾、留一个空槽的设计只能放 C−1 项，满空判断不同，不要把两个版本混用。

9.2 两个栈翻一次，最早入队的就到顶

准备 in、out，每个栈都从底写到顶。入队只向 in 压入。出队先看 out：非空直接弹；空时才把 in 全部倒进 out。入 A、B、C 再出 A 的每一次栈动作如下：

动作	in（底→顶）	out（底→顶）	局部变量/返回
入 A，push in	A	空	无
入 B，push in	A,B	空	无
入 C，push in	A,B,C	空	无
请求出队，out 空	A,B,C	空	开始转移
pop in	A,B	空	暂存 C
push out	A,B	C	C 移完
pop in	A	C	暂存 B
push out	A	C,B	B 移完
pop in	空	C,B	暂存 A
push out	空	C,B,A	A 来到 out 顶
in 空，结束转移；pop out	空	C,B	返回 A

每次弹出后，元素先在局部变量 x 中，随后才压入另一栈。第三排把这个暂存状态也画出来，避免转移途中看起来丢失元素。

双栈队列：整批倒转以后再出队

1. 两个栈均为空

入栈：底→顶 空

出栈：底→顶 空

暂存/返回 x 空

队列初始为空。
原因：入栈接收新元素；出栈的栈顶作为队首。

图 65 逐步演示：双栈队列：整批倒转以后再出队

双栈队列：整批倒转以后再出队

2. 入队A

入栈：底→顶 A

0

出栈：底→顶 空

暂存/返回 x 空

把A压入入栈。
原因：新元素在已有元素之后到达，不应抢先出队。

图 66 双栈队列：整批倒转以后再出队，第 2 步

双栈队列：整批倒转以后再出队

3. 入队B

入栈：底→顶 A B

0 1

出栈：底→顶 空

暂存/返回 x 空

把B压入入栈。
原因：新元素在已有元素之后到达，不应抢先出队。

图 67 双栈队列：整批倒转以后再出队，第 3 步

双栈队列：整批倒转以后再出队

4. 入队C

入栈：底→顶 A B C

0 1 2

出栈：底→顶 空

暂存/返回 x 空

把C压入入栈。
原因：新元素在已有元素之后到达，不应抢先出队。

图 68 双栈队列：整批倒转以后再出队，第 4 步

双栈队列：整批倒转以后再出队

5. 从入栈弹出C

入栈：底→顶 A B

0 1

出栈：底→顶 空

暂存/返回 x C

0

暂存x=C，尚未压入出栈。
原因：出栈为空时才整批倒转；先后次序因此反转一次。

图 69　双栈队列：整批倒转以后再出队，第 5 步

双栈队列：整批倒转以后再出队

6. 把C压入出栈

入栈：底→顶 A B

0 1

出栈：底→顶 C

0

暂存/返回 x 空

出栈现在是['C']，右端为栈顶。
原因：要先完成整批转移，最早进入的A才能来到栈顶。

图 70　双栈队列：整批倒转以后再出队，第 6 步

双栈队列：整批倒转以后再出队

7. 从入栈弹出B

入栈：底→顶 A
0

出栈：底→顶 C
0

暂存/返回 x B
0

暂存x=B，尚未压入出栈。
原因：出栈为空时才整批倒转；先后次序因此反转一次。

图 71　双栈队列：整批倒转以后再出队，第 7 步

双栈队列：整批倒转以后再出队

8. 把B压入出栈

入栈：底→顶 A
0

出栈：底→顶 C B
0 1

暂存/返回 x 空

出栈现在是['C', 'B']，右端为栈顶。
原因：要先完成整批转移，最早进入的A才能来到栈顶。

图 72　双栈队列：整批倒转以后再出队，第 8 步

双栈队列：整批倒转以后再出队

9. 从入栈弹出A

入栈：底→顶 空

出栈：底→顶 C B

0 1

暂存/返回 x A

0

暂存x=A，尚未压入出栈。
原因：出栈为空时才整批倒转；先后次序因此反转一次。

图 73 双栈队列：整批倒转以后再出队，第 9 步

双栈队列：整批倒转以后再出队

10. 把A压入出栈

入栈：底→顶 空

出栈：底→顶 C B A

0 1 2

暂存/返回 x 空

出栈现在是['C', 'B', 'A']，右端为栈顶。
原因：要先完成整批转移，最早进入的A才能来到栈顶。

图 74 双栈队列：整批倒转以后再出队，第 10 步

双栈队列：整批倒转以后再出队

11. 真正出队A

入栈：底→顶　空

出栈：底→顶 C B

0 1

暂存/返回 x A

0

从出栈弹出A，返回给调用者。
原因：A最先入队，所以应当最先离开；B成为下一队首。

图 75　双栈队列：整批倒转以后再出队，第 11 步

此时再入 D，`in=[D],out=[C,B]`。下一次出队必须弹 B，不能把 D 再压到 out 顶，否则新项会抢在旧项前。逻辑队列始终是“out 从顶到底，再接 in 从底到顶”。

单次出队可能一次转移很多项，最坏线性；但每个元素只经历入 in、一次转移、出 out 这条路线，总共常数次栈操作。所以从空队列开始 `m` 次合法操作总成本 `O(m)`，摊还常数。它对每条操作序列都成立，不靠平均运气。

9.3 窗口最大值：丢旧候选必须有可检验的理由

对`[2,1,5,3,4,2]`，每次看连续 3 项，窗口依次为`[2,1,5]`、`[1,5,3]`、`[5,3,4]`、`[3,4,2]`，最大值应是 5、5、5、4。

只保存一个最大值不够：它离开窗口后，需要知道剩下谁能接班。保存所有窗口项虽正确，却还要重新找最大。我们用双端队列保存仍有资格的候选下标：下标递增，值严格递减。头尾都可删除，叫双端队列；“单调”是我们对它施加的维护规则，不是容器自己会做。

每到下标 `i`，当前左边界为 `i-w+1`，`w=3`。先从头删除小于左界的下标，再从尾删除值不大于新值的候选，最后加入 `i`。小于左界才过期，等于左界仍在窗口。

输入步骤	具体动作	候选下标（头→尾）	候选值	是否输出
i=0，值 2	无过期、无尾项，入 0	[0]	[2]	未到 3 项
i=1，值 1	头 0 未过期；2>1 不删；入 1	[0,1]	[2,1]	未到 3 项
i=2，值 5	左界 0，头 0 未过期	[0,1]	[2,1]	先不输出
同一步	尾 1 的值 1<=5，删 1	[0]	[2]	继续比尾
同一步	尾 0 的值 2<=5，删 0	[]	[]	继续到空
同一步	入 2	[2]	[5]	输出 5
i=3，值 3	左界 1，头 2 未过期；5>3；入 3	[2,3]	[5,3]	输出 5

输入步骤	具体动作	候选下标（头→尾）	候选值	是否输出
i=4，值 4	左界 2，头 2 未过期	[2,3]	[5,3]	先不输出
同一步	尾 3 的值 3<=4，删 3	[2]	[5]	5>4 停止删尾
同一步	入 4	[2,4]	[5,4]	输出 5
i=5，值 2	左界 3，头 2<3，删头 2	[4]	[4]	旧最大终于过期
同一步	4>2，不删尾；入 5	[4,5]	[4,2]	输出 4

为什么新 5 可以删除旧 2、旧 1？它更大，而且位置更晚，在未来任何仍含旧项的窗口里，新 5 都还在，旧项都赢不了它。新 5 离开时，旧项早已离开。两个条件必须同时有：只比较大小而不比较新旧到达位置，证明就断了。

相等时也删旧项能正确输出最大值；若还要求最早最大值下标，则应保留旧的等值候选。下标不能随便换成数值，因为相同值可能来自不同位置，要靠下标判断谁过期。

```
from collections import deque

def window_maximum(a, w):
    if not 1 <= w <= len(a):
        raise ValueError("窗口宽度必须在1到数组长度之间")
    q = deque()
    answer = []
    for i in range(len(a)):
        while q and q[0] < i - w + 1:
            q.popleft()
        while q and a[q[-1]] <= a[i]:
            q.pop()
        q.append(i)
        if i >= w - 1:
            answer.append(a[q[0]])
    return answer
```

每个下标进一次，最多从头或尾出一次，两重 while 累计删除也不超过 n 次，总时间$\Theta(n)$，最多保留 w 个候选。某一轮删很多，不等于每轮都删同一批项。宽 1 时应逐项输出自身，宽 n 时只输出全局最大；这些边界可直接检验过期条件有没有差 1。

9.4 最大栈：同时保存当前前缀的最大值

如果需要栈在 push、pop 以外还能 O(1)查询最大值，可以每项保存(值,压入此项后栈内最大值)。依次压 2、5、3，节点记录为(2,2)、(5,5)、(3,5)。弹 3 以后，顶部(5,5)仍给最大值 5；再弹 5，顶部(2,2)给 2。

这需要每项多存一个最大值，总空间仍 O(n)。也可以只在最大值变化时保存“影子记录”，但重复最大值要计数或允许重复压入，避免弹出一个 5 就误以为所有 5 都离开。用两个最大栈组成队列时，队列最大值为两栈最大值的较大者，空栈另行处理。

单调队列更积极地删除被支配的候选，空间在随机输入下可能较小；但最坏递减窗口会保存 w 项。固定一个随机连续分布的窗口，其候选数期望约 H_w，不能据此声称无限长运行过程的峰值也只有 O(log w)。28 章会解释观察时间为何影响峰值。

9.5 例 4：直方图最大矩形，栈里存“还没遇到右边界”的柱子

柱高[2,1,5,6,2,3]，每柱宽 1。最大矩形可以取高度 5、覆盖第 2 与第 3 柱，面积 10。枚举每个区间求最低高度会很慢；单调栈维护高度不降的候选柱，一旦遇到更矮柱，就能确定若干旧柱的最远右边界。

用下标栈，开始压一个虚拟下标-1，结束补高度 0。处理过程：

- i=0，高 2，压 0，栈为-1,0。
- i=1，高 1，比 2 矮，弹 0；弹后左边界-1，宽度 1-(-1)-1=1，面积 2。再压 1。
- i=2，高 5，压 2；i=3，高 6，压 3。

- i=4，高 2，先弹 3，高 6，左侧剩 2，宽 4−2−1=1，面积 6；再弹 2，高 5，左侧剩 1，宽 4−1−1=2，面积 10。高 1 低于 2，停止，压 4。
- i=5，高 3，压 5；i=6 补 0，依次弹 5 得 3，弹 4 得 8，弹 1 得 6，最大仍 10。

宽度为什么是 i−left−1？i 是右边第一个严格更矮柱，不能包含；弹出后栈顶 left 是左边尚留的不高于当前柱的候选位置，本次先不包含它。中间从 left+1 到 i−1，共 i−left−1 根柱子，它们都至少与当前弹出高度一样高。若 left 与当前柱等高，本次结算的只是较窄范围，不是当前高度能向左延伸的最终最大范围；更靠左的等高代表随后弹出时会覆盖完整宽度。例如[2,2,0]先弹右侧 2 只得面积 2，再弹左侧 2 得到面积 4。因此保留等高的版本不能把每次弹后的 left 都称作“严格更矮”。

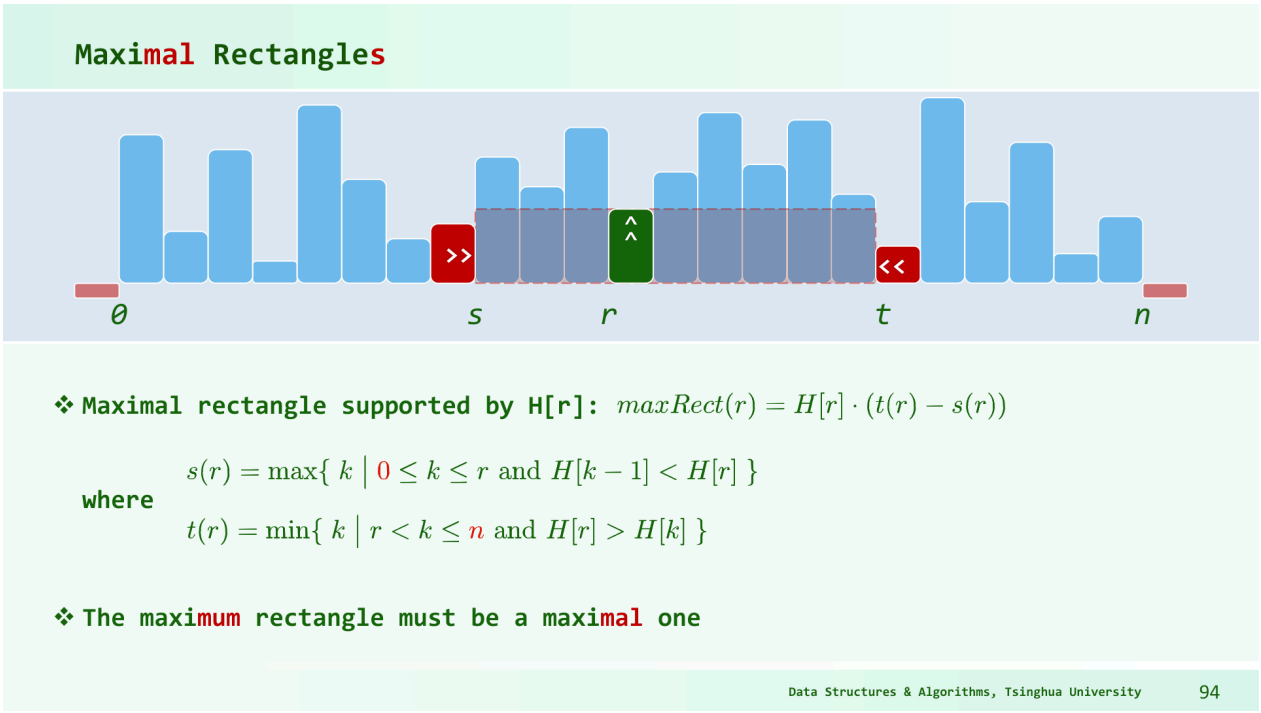


图 76　教师的单调栈示意

图中绿色柱 r 给出矩形高度，左侧红柱比它矮，右侧红柱也比它矮；中间被阴影覆盖的所有柱都不低于绿色柱。原图把左端第一个可用位置记 s，因此宽为 t−s；对图中这组严格更矮边界，若将左侧不可用柱下标记 left、右侧记 i，则同一完整范围宽为 i−left−1。算法保留等高柱时，某次弹出的 left 可能等高，需按上段先结算较窄候选、再由左侧等高代表取得完整宽度的规则理解，不能把每次栈顶都套成图中的严格更矮边界。单调栈的用途是及时找到这些边界，弹出时先结算矩形面积，再移除候选。

相同高度可采用“仅在更高时弹”或“相等也弹”的不同版本，宽度处理与保留代表项的规则需要配套。上面采用严格更高才弹，多个等高项会保留，后面较左者最终获得更宽范围。每项入栈一次、出栈一次，时间 Θ(n)，最坏空间 Θ(n)。

末尾补 0 时的三次结算也要算完整。原柱高 [2,1,5,6,2,3] 处理完下标 5 后，栈为 [−1,1,4,5]；当前 i=6 是补的 0：

弹出下标与高度	弹后栈顶 left	可用柱数 i-left-1	面积
5，高 3	4	6−4−1=1	3
4，高 2	1	6−1−1=4	8
1，高 1	−1	6−(−1)−1=6	6

下标 −1 只作左哨兵，不去读取原数组 a[−1]。遇补 0 弹完所有正高度候选，最大 10 已在之前弹高 5 时记录，不会被末尾较小结果覆盖。若原数据含高度 0，可以保留等高零项，仍不影响正面积答案。

9.6 前缀和是另一种复用，不要见窗口就只想单调队列

若求每个宽 w 窗口的和，维护当前和，右端新值加上，左端旧值减去即可，时间 Θ(n)、额外空间 O(1)，前提能访问即将离开的旧项。若是多次任意区间求和，前缀 P[0]=0，P[i+1]=P[i]+a[i]，则 `sum([l,r))=P[r]-P[l]`。最大值没有一般的“减去离开者”逆运算，所以需要维护候选或额外结构。不同聚合量拥有不同代数性质，直接决定可以怎样压缩状态。

9.7 直方图边界，用一个候选高一次算完

以柱高[3,1,3,2,2]为例。高度 1 可以覆盖所有 5 柱，面积 5；左端 3 只能宽 1，面积 3；下标 2 的 3 右边遇 2，宽 1 面积 3；高度 2 可以覆盖下标 2 到 4 共 3 柱，面积 6。最大为 6。单调栈会在读到第一个 2 时结算高 3，在末尾 0 时结算高 2，让某一个等高代表获得完整宽 3。

若实现弹出条件为>=，新来的等高柱会替换旧代表，则必须让它继承旧代表能向左延伸的起点。例如处理第二个 2 时，不能把可用左界重设到自己，否则漏掉下标 2 的高 3 与前一个 2 构成的宽范围。下标栈的某些版本通过前一个更矮元素自然保留该信息，显式(高度,起点)版本则要手动继承。相等分支可以不同，配套边界不能断。

面积用高度乘宽度，宽度最大 n。即使高度、n 各在 32 位内，乘积可能超过 32 位。末尾哨兵 0 只适用于柱高非负；如果题目允许负高度，“矩形面积”本身的几何语义要重新定义，不能默认同一个补 0 规则覆盖。

9.8 轮转调度，用队列模拟一段真实过程

任务 A 需 3 单位、B 需 1、C 需 2，时间片为 1。初始队列 A3,B1,C2。执行 A 一单位后剩 A2，接到尾，队列 B1,C2,A2；执行 B 完成，队列 C2,A2；执行 C 后剩 C1，队列 A2,C1；执行 A 后剩 A1，队列 C1,A1；执行 C 完成，再执行 A 完成。完成时刻分别 B 为 2、C 为 5、A 为 6。

队列只管理轮到谁，不决定每个任务的总需求。若新任务在执行中到达，应明确入队时刻与当前任务重新入队的先后规则；相同时刻的约定可能改变精确等待时间。用一个时间轴与队列状态同步写，就不会把先进先出误读成每个任务一口气执行完。

9.9 单调结构的证明与普通队列的证明有何不同

普通队列从不删除中途任务，因此只要证明入出顺序一致。单调队列主动从尾部扔掉某些仍在窗口里的元素，必须额外证明这些元素在任何未来窗口都不会胜过留下的新元素。支配关系含两个方向：新值不小于旧值，新下标不早于旧下标。少了后一个条件，只留下最大值会在它过期后失去次大候选；少了前一个条件，就可能扔掉真正最大者。

9.10 练习与答案

练习 1。 宽 3 窗口扫描 `[4,4,2,5]`，相等时删旧候选，写出每步候选下标。

解答：读第 0 个 4，队列[0]；读第 1 个 4，旧 4 被新 4 支配，删 0 后为[1]；读 2，得[1,2]，第一个窗口最大 4；读 5，先检查窗口为下标 1 到 3，1 尚未过期；再从尾删 2、1，得[3]，第二个窗口最大 5。若要最早最大值下标，第一个窗口应答 0，当前删旧规则无法提供它，需要调整相等分支。

练习 2。 柱高 `[2,2,2]` 最大矩形多少？用严格更高才弹的版本追踪结尾。

解答：依次将 0、1、2 压栈，补 i=3 高度 0。弹 2 时左 1，宽 1，面积 2；弹 1 时左 0，宽 2，面积 4；弹 0 时左 −1，宽 3，面积 6。相等高度并未使算法错误，只是最大宽度在最左代表项弹出时才结算。

练习 3。 双栈队列操作为入 A、入 B、出、入 C、出、出。写出出队顺序及每个元素转移次数。

解答：第一次出队把 B、A 从 in 倒到 out，弹 A；入 C 仅进 in；第二次出队直接弹 out 的 B；第三次 out 空，倒 C 后弹 C。顺序 A、B、C，每个元素只从 in 转到 out 一次。若每次出队先无条件倒 in，C 可能越过 B，这正是转移条件不可省的原因。

10 二叉树：走进一条分支以后，怎样找回没做完的事

一棵树从根开始向下分叉。除根外，每个节点有唯一父节点，沿父链接向上最终到根，没有环。二叉树规定每个节点有左、右两个孩子位置，某个位置可以为空；“只有一个孩子”也必须说是左还是右。
我们一直使用这棵六节点树，不在解释遍历的途中更换输入：

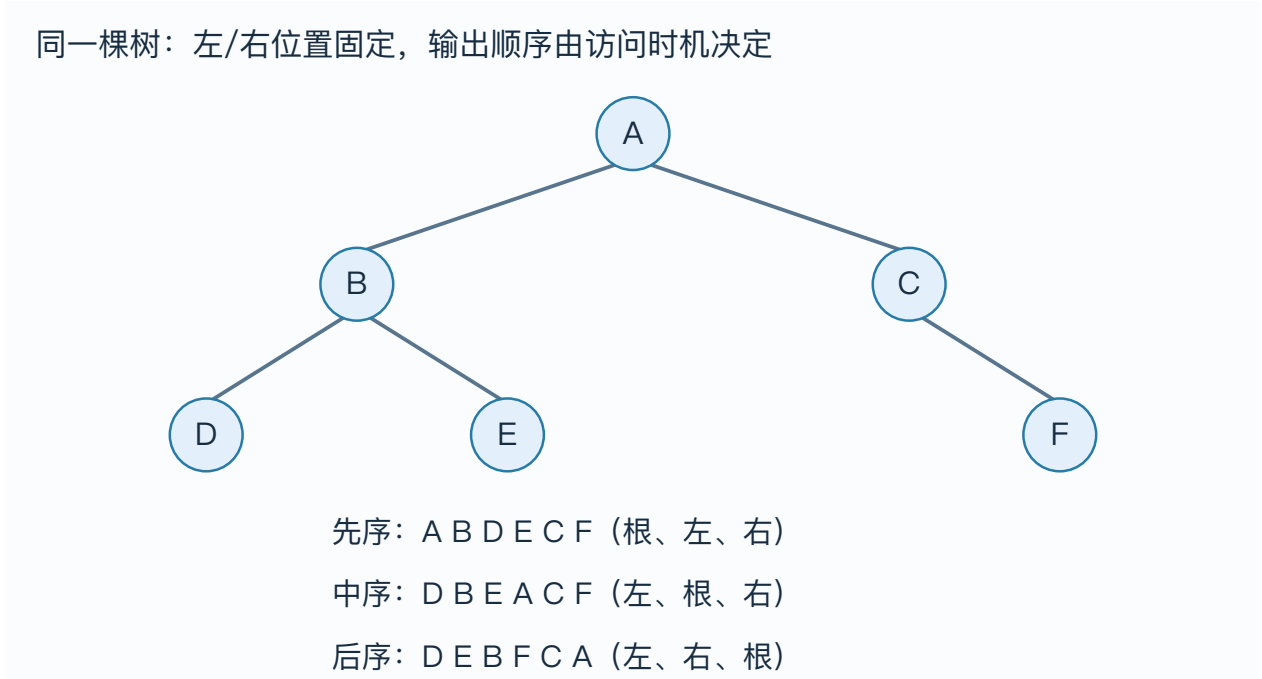


图 77　本章二叉树、遍历编号与重建分界

```
      A
     / \
    B   C
   / \   \
  D   E   F
```

A 是根，D、E、F 没有真实孩子，叫叶节点。B 子树含 B、D、E 三个节点；C 的左孩子是空，右孩子 F。边是相邻父子的一条连接，整树 6 节点 5 边。即使内存中还存了反向 parent 指针，数学上 A 到 B 仍只是一条边。
深度是从根走到节点经过的边数：A 深 0，B/C 深 1，D/E/F 深 2。节点的高度是从它向下到最远叶的边数：叶高 0，B/C 高 1，A 高 2；空树约定高−1。根的深度与整树高度不同，画图时别只背同一个“层数”。

10.1 一次递归调用有三个停靠点

访问一个节点指执行用户动作，比如把它的标签写进输出；“走到这个节点”与“访问输出它”可以发生在不同时间。

```
walk(x):
    如果x为空：返回
    停靠点①
    walk(x.left)
    停靠点②
    walk(x.right)
    停靠点③
    返回
```

在①输出是先序，根在左右孩子之前；在②输出是中序，左子树后、右子树前；在③输出是后序，两个孩子之后。每个调用都有自己的参数 x，递归返回会回到那个调用里被暂停的位置。
拿中序从头走，空孩子也不能凭空跳掉：

事件顺序	具体动作	已经输出	尚待完成的外层
1	进入 A，先调用左 B	空	A 待左完成后访问
2	进入 B，先调用左 D	空	A；B 待左完成
3	进入 D，调用空左，立即返回	空	A；B；D
4	D 左已完，访问 D，再调用空右返回	D	A；B；D
5	D 返回 B，B 访问自身	D,B	A；B 待右完成
6	B 调用右 E，E 调用空左返回	D,B	A；B；E
7	访问 E，空右返回，E 返回 B	D,B,E	A；B
8	B 左右已完，返回 A	D,B,E	A
9	A 访问自身，调用右 C	D,B,E,A	A 待右完成
10	C 调用空左返回，访问 C	D,B,E,A,C	A；C
11	C 调用右 F，F 空左返回后访问 F	D,B,E,A,C,F	A；C；F
12	F 空右返回；F 返回 C；C 返回 A；A 返回	D,B,E,A,C,F	无

中序不是从纸上最左边一路读过去的空间顺序，而是每个子树都执行“左、自己、右”。只有额外满足二叉搜索树关键码规则时，中序输出才按值有序。本章 A/B 等标签不承诺搜索树次序。
同一棵树的三个结果为：先序 `A B D E C F`，中序 `D B E A C F`，后序 `D E B F C A`。可以在图中每个节点旁写①②③，手指沿递归路线走，每次只在指定停靠点记下标签。这样能独立复现三个序列，不必背字母串。

10.2 把“返回以后再做”放进显式栈

中序栈保存“左子树完成后轮到我”的节点。`cur` 表示接下来向下进入的节点，栈从底写到顶。

```python
def inorder(root):
    stack = []
    cur = root
    result = []
    while cur is not None or stack:
        while cur is not None:
            stack.append(cur)
            cur = cur.left
        cur = stack.pop()
        result.append(cur.data)
        cur = cur.right
    return result
```

动作	cur 变成	栈	输出
初始	A	[]	[]
压 A，向左	B	[A]	[]
压 B，向左	D	[A,B]	[]
压 D，向左	空	[A,B,D]	[]
左链到空，弹 D 访问，再转右	空	[A,B]	[D]
弹 B 访问，再转右	E	[A]	[D,B]
压 E，向左	空	[A,E]	[D,B]
弹 E 访问，再转右	空	[A]	[D,B,E]
弹 A 访问，再转右	C	[]	[D,B,E,A]
压 C，向左	空	[C]	[D,B,E,A]
弹 C 访问，再转右	F	[]	[D,B,E,A,C]
压 F，向左	空	[F]	[D,B,E,A,C]
弹 F 访问，再转右	空	[]	[D,B,E,A,C,F]
cur 空且栈空	无	无	退出

外层条件用“或”：cur 空但栈非空时还有祖先待办，不能退出。内层 cur 空后必须弹栈，不能继续访问空节点。每节点压一次、弹一次，总时间线性；同时最多存根叶路径，空间$O(h+1)$，h 为树高。退化链的高为 n−1，不能一概把空间写成对数。

后序要保存“已经展开过孩子了吗”。初见 x 时把“x 待输出”压回，再压右孩子、左孩子；因为后进先出，左先处理。对本树，弹栈事件如下：

```
[A初见]
弹A初见 → 压A待输出、C初见、B初见
弹B初见 → 压B待输出、E初见、D初见
弹D初见 → 压D待输出（没有真实孩子可压）
弹D待输出 → 输出D
弹E初见 → 压E待输出
弹E待输出 → 输出E
弹B待输出 → 输出B
弹C初见 → 压C待输出、F初见
弹F初见 → 压F待输出
弹F待输出 → 输出F
弹C待输出 → 输出C
弹A待输出 → 输出A，栈空
```

不记录初见/待输出就无法区分“刚到 B，该去孩子”和“孩子完了，该输出 B”。这两种情况节点地址相同，缺的是阶段信息。

10.3 层次遍历按先发现先处理

队列存已经发现、尚未访问的节点：

出队访问	再入队哪些孩子（先左后右）	队列头→尾
初始无访问	根 A 入队	[A]
A	B,C	[B,C]
B	D,E	[C,D,E]

出队访问	再入队哪些孩子（先左后右）	队列头→尾
C	F	[D,E,F]
D	无	[E,F]
E	无	[F]
F	无	[]，结束

输出 A B C D E F。新孩子都在队尾，因此浅层待办一定先于更深层。时间线性，队列峰值取决于宽度；链很高但队列只需 1，完美树很宽时队列可占线性空间。

10.4 先序和中序怎样一起确定每一根边

先序 A B D E C F 的首项 A 是根。在中序 D B E A C F 里找到 A，左侧 D B E 三项必属左子树，右侧 C F 两项必属右子树。因先序先走完整左子树，A 后紧跟的三项 B D E 就归左，不能随便从中序挑三项。

当前任务的先序	当前任务的中序	取根并定位	左任务	右任务
A B D E C F	D B E A C F	根 A，中序位置 3	先 B D E / 中 D B E	先 C F / 中 C F
B D E	D B E	根 B，段内位置 1	D/D	E/E
D	D	根 D，两侧空	空，返回空	空，返回空
E	E	根 E，两侧空	空	空
C F	C F	根 C，段内位置 0	空	F/F
F	F	根 F，两侧空	空	空

下层创建的 D、E 先返回给 B，B 连好两边后返回给 A；F 先返回给 C，再由 C 返回给 A。全部左右连接与上方原树一致。先序首项决定根，中序分界决定子树大小，这两个信息缺一不可。

标签互异才可用“标签→中序下标”字典唯一定位。每次线性找根在退化情形会平方；一次建字典、递归传区间而非复制序列，可以做到线性时间。输入还要校验标签集合、重复、根是否落在当前区间、是否恰好用完所有项，不能从“合法输入下唯一”推成“任意两串都能构造”。

后序加中序同样能重建，只是根取后序末项。例如本树后序 D E B F C A，末项 A；中序 A 左边 3 项，所以后序的前 3 项 D E B 归左，接着 2 项 F C 归右。左段末 B、右段末 C，继续按同一规则分。若代码从后序末尾倒着消耗，读完根 A 后应先构造右子树 C，再构造左子树 B；若仍先构造左边，会把 C 错接到左侧。

先序加后序通常不够：A B 与 B A 不能区分 B 在 A 左边还是右边。若额外限定每个内部节点恰有两个孩子且标签互异，则能增加足够约束；不能把这个限定版本的唯一性搬到一般二叉树。

10.5 空孩子也记录，序列才没有丢位置

单靠先序 A B，既可能 A 左为 B，也可能 A 右为 B。把每个空孩子写成记号^，A 左空、右 B 的完整先序变成 A ^ B ^ ^。解析时，普通标签创建节点并递归读左、右；空标记直接返回空。

```
读A：创建A，等待左、右
  读^：A左返回空
  读B：创建B，等待左、右
    读^：B左空
    读^：B右空
  B创建完成，返回给A作为右孩子
A完成，输入恰好用完
```

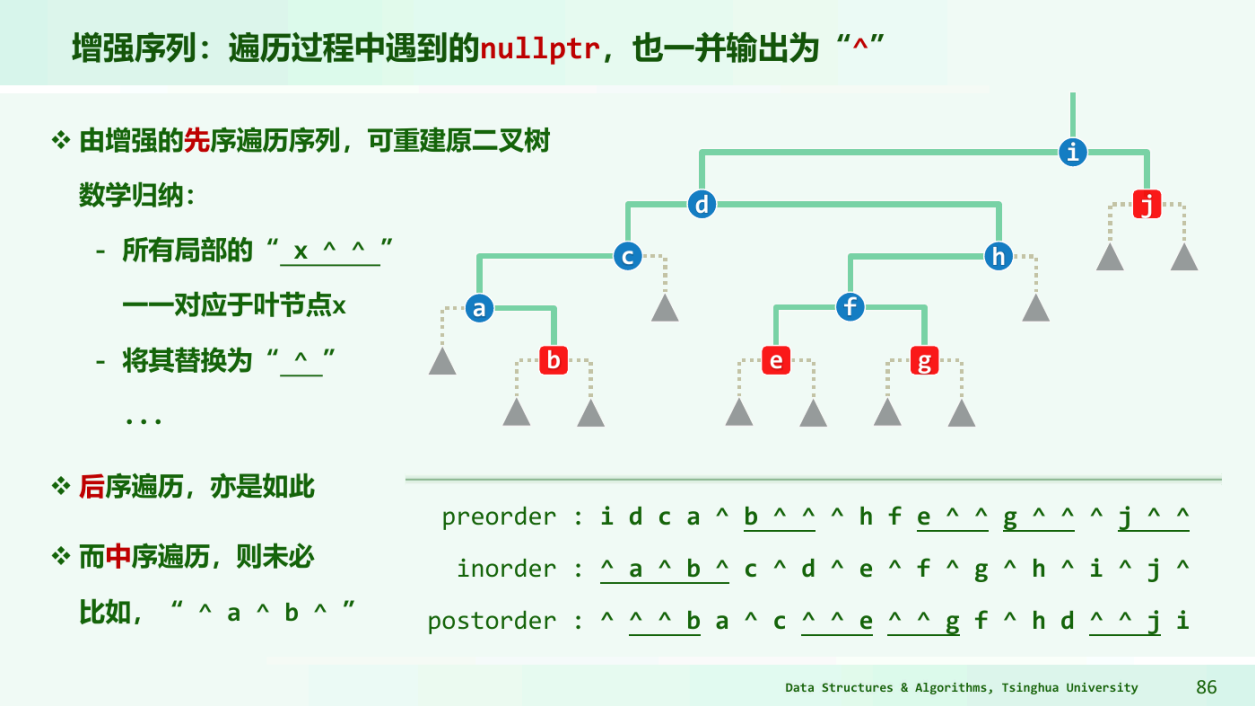


图 78　教师增强遍历的完整一页

老师《05 二叉树》PDF87 页用灰三角画空孩子。红色叶 b、e、g、j 后面各有两个空标记，对应局部 x ^ ^；把这三项缩成一个空位，就能自底向上缩整树。图中的增强中序始终为空标记与标签交替，仍不能定出根分界；不要因为记号数量一样就认为三种序列信息一样多。

初始有一个待填根槽。普通节点占一槽、新增左右两槽，待填槽净加 1；空标记占一槽、净减 1。A ^ B ^ ^的槽数为 1⟶2⟶1⟶2⟶1⟶0，恰好结束。A ^ ^ B ^ ^在前三项后已经归零，后面还来 B，因此是多余输入，不能接受。

10.6 中序后继：不用重走整棵树

若节点 x 有右子树，中序后继是右子树里最左节点。若没有右子树，沿 parent 向上走，直到第一次从某个祖先的左边返回，该祖先就是后继；若一直从右边向上到根，还没有找到，x 就是最后节点，无后继。小树中 E 无右子树，先从 E 到 B，E 是 B 右孩子，继续；从 B 到 A，B 是 A 左孩子，所以后继 A。C 有右子树 F，后继就是 F。单次后继最坏 O(h)，但顺着后继遍历整树，边只会被上下经过常数次，总时间 O(n)，不能把每次最坏 h 机械乘 n 得到唯一结论。

10.7 外部空叶与普通叶子，数的是两类对象

把每个缺失孩子都补成一个外部叶，包括原来叶节点缺少的两个孩子。原树 n 节点有 2n 个孩子槽，已占 n−1，缺 n+1，因此扩充后有 n+1 个外叶，每个原节点都变成恰有两个孩子的内部节点，内部节点数为 n。若原双孩子节点 n_2、单孩子 n_1、原叶 n_0，则原树 $n_0=n_2+1$；$n=2n_2+n_1+1$，所以外叶 $n+1=2n_2+n_1+2$。

例如原树根 A 只有左孩子 B，n=2。A 右边补一个外叶，B 左右各补一个，共 3 个；A 和 B 都变双分支内部节点，共 2 个。只给 A 补右孩子会漏掉 B 的两个空槽，得到的是另一种变换，不能拿来解释增强先序中的空标记。

10.8 后序求子树大小，走一个依赖完整的例子

固定小树中，D、E、F 为叶，size 各 1；B 处理完两个孩子后得 1+1+1=3；C 左空大小 0、右 F 大小 1，得 2；A 最后得 1+3+2=6。若在先序访问 A 时就读孩子尚未更新的 size，可能拿到旧值。遍历顺序的选择来自数据依赖，不是偏好。

同样的后序可以求 height=max(左高,右高)+1，空高−1；求表达式值则先求左右操作数再施运算；释放整树也通常先释放孩子，再释放父，避免父被释放后找不到孩子地址。一个遍历模式能复用，但访问动作必须符合对象生命周期。

10.9 叶数与双孩子节点数，从边数推出

设无孩子节点数为 n 零，一个孩子为 n 一，两个孩子为 n 二。节点总数 n=n 零+n 一+n 二，边数一方面为 n 减一，另一方面每个父贡献孩子数，所以为 n 一加二倍 n 二。两式相减得 n 零=n 二加一。

这个关系对任意非空二叉树成立，单孩子节点多少不影响等式。链有一个叶和零个双孩子节点，成立；完美七节点树有四叶、三个双孩子节点，也成立。空树不属于这一非空计数前提，不能把零等于一当作公式错误。

完全二叉树数组的最后一个非叶下标是长度除二取下整再减一。只有一项时没有非叶节点，建堆下滤循环应直接跳过。

访问空孩子时直接返回。

10.10 练习与答案

练习 1。 先序 `R P Q S`，中序 `P Q R S`，求后序并画出左右关系。

解答：根 R，中序左段 P Q、右段 S；左子树根 P，中序 P Q 说明 P 左空、右 Q。右子树只有 S。后序左子树为 Q P，右子树 S，再根 R，得到 `Q P S R`。写出 P 右 Q 这条边很重要，不能因为只有两个节点就随意画在左侧。

练习 2。 增强先序 `A ^ B ^ ^`代表什么？`A ^ ^ B ^ ^`为何非法？

解答：第一串表示 A 左空、右孩子 B，B 左右空。第二串前三个记号已经形成一棵完整的单节点 A 树，后面又多出 B，无法属于同一棵树的先序编码。只检查每个节点后有没有两个空标记不够，应按待填槽解析：开始 1 槽，普通节点占 1 槽并新增 2 槽，净增 1；空标记占 1 槽，减 1；过程中不得提前归零后继续读，结束必须恰好归零。

练习 3。 一棵 10000 节点的退化树，递归中序与层序的辅助空间各多少？

解答：若每个节点只有一个孩子，树高 9999，递归调用栈 Θ(10000)，有栈溢出风险；层序队列中通常只有 1 个待处理节点，空间 O(1)。二者访问时间都 Θ(n)。算法同样正确，空间却受树形影响。

10.11 用节点数检验形状边界

非空二叉树高为 `h`，最长根叶路径有 `h+1` 个节点，因此$n \geq h+1$。深度 0 最多 1 项，深度 1 最多 2 项，深度 2 最多 4 项，一直到深度 `h` 最多2^h项，全部相加得$n \leq 2^{h+1}-1$。这些是两种不同形状达到的边界：链达到下界，所有层填满的完美树达到上界。

“完全二叉树”要求除末层外都满、末层从左向右连续填。按层序从下标 0 放数组，节点 `i` 的左位置 `2i+1`、右位置 `2i+2`，存在与否再看是否小于 `n`；非根父位置为$\lfloor (i-1)/2 \rfloor$。向下取整$\lfloor x \rfloor$取不大于 x 的最大整数。7 项时根 0 孩子 1、2，节点 1 孩子 3、4，节点 2 孩子 5、6。稀疏一般树用相同编号会留空格，不能保证存储紧凑。

“满二叉树”有的资料指每个内部节点都有两个孩子，有的指所有层都填满。后者本书叫完美树，老师本章原图也使用“同高度规模最大”的含义；做题以题面的定义为准。

11 Huffman 编码：为什么总把最轻的两个合在一起

四个符号 A、B、C、D 的出现次数分别为 5、7、10、15。若每个符号统一用 2 位编码，总长度为 2×37=74 位。频率高的符号若能用更短编码，通常可以减少总位数；但编码必须能被无歧义地解读。
前缀无关编码要求：任何符号的编码都不是另一个符号编码的前缀。比如 `0,10,110,111` 符合；`0,01,11` 不符合，因为读到 0 时不知是否已经完成一个符号，还是后面还有 1。将 0、1 分别当作树的左、右边，所有符号放在叶子，根到叶的路径就是编码，自然具有前缀无关性。

11.1 例 1：完整合并一次，看两种成本怎样相等



图 79　Huffman 构造的完整状态 1

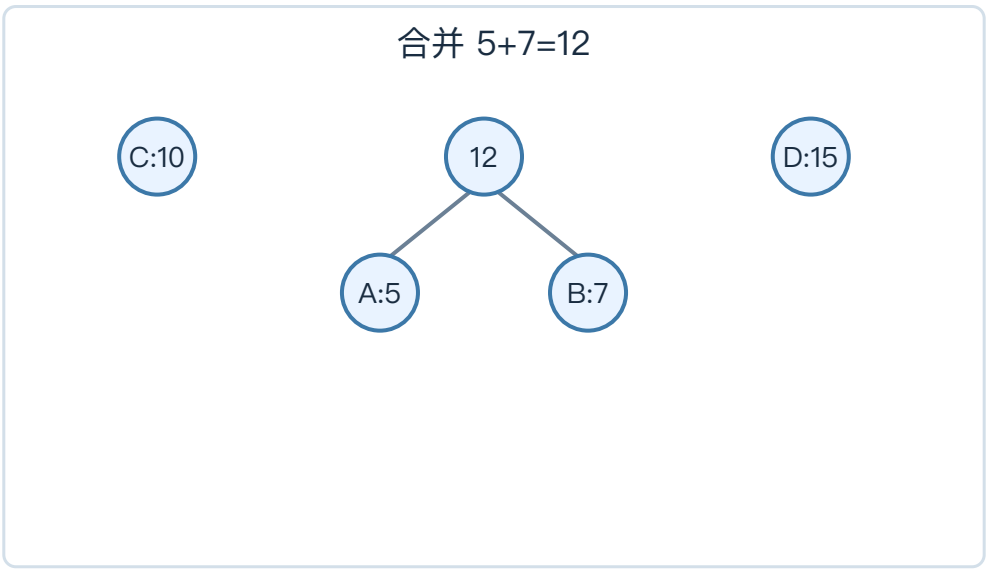


图 80　Huffman 构造的完整状态 2

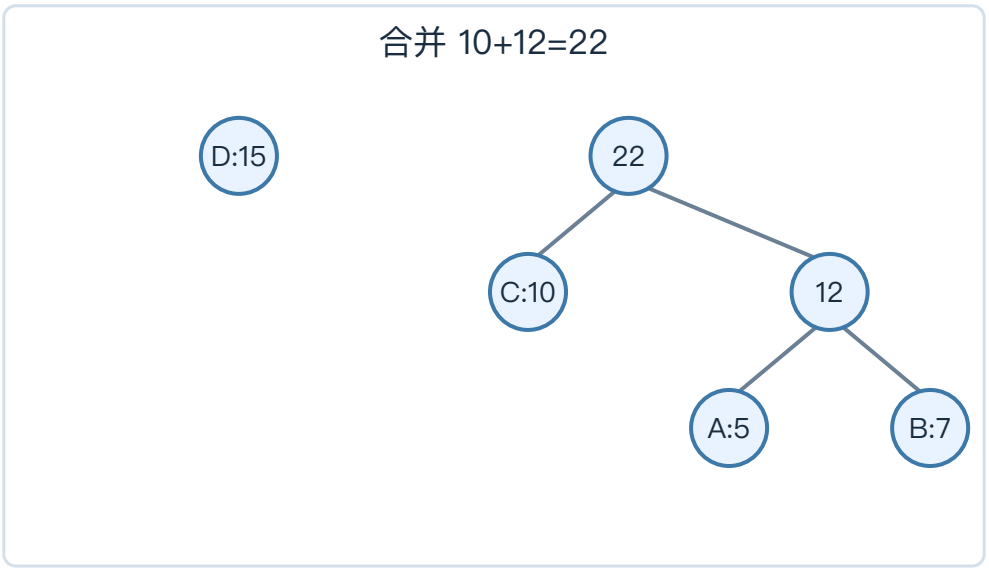


图 81　Huffman 构造的完整状态 3

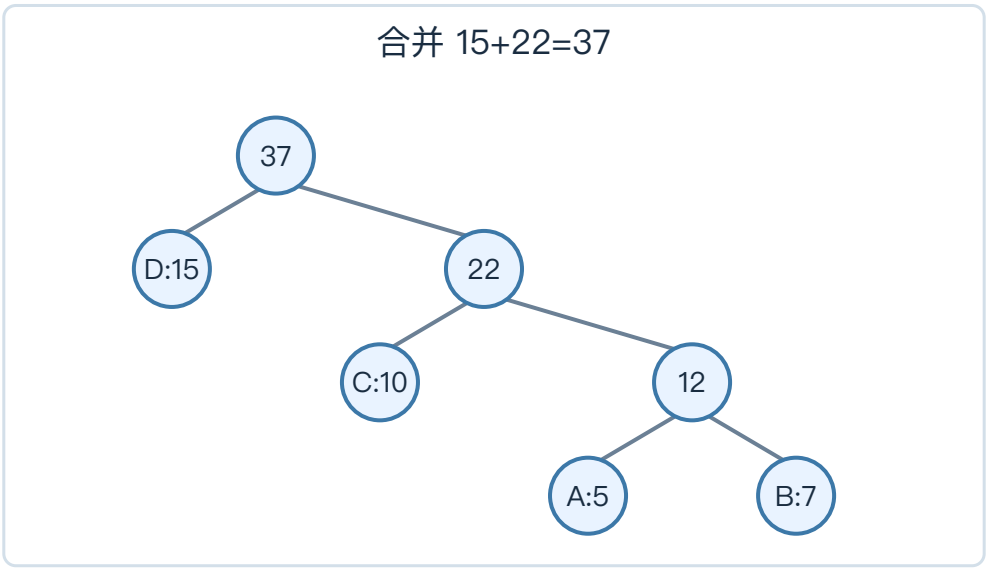


图 82　Huffman 构造的完整状态 4

图中的每个圆是一个树根，A:5 这类标签同时写符号和权重；每一格保存当时整片森林。第一步只把 A 与 B 接在 12 下面，C、D 仍各自独立。第二步新根 22 接 C 与 12，A 和 B 随 12 一起下移一层。第三步 37 接 D 与 22，所有符号才落在同一棵树里。

初始权重为 5、7、10、15。取最小 5 与 7 合并成 12，森林剩 10、12、15；取 10 与 12 合成 22，剩 15、22；再合成 37。若左边记 0、右边记 1，可得到 D=0、C=10、A=110、B=111。

编码长度分别为 A3、B3、C2、D1，带权路径长度为

$$5 \times 3 + 7 \times 3 + 10 \times 2 + 15 \times 1 = 71.$$

也等于每次合并权重之和：12+22+37=71。因为合并两棵子树，相当于让其中所有叶子深度加 1，新增位数恰好是这两棵树的权重总和。每个叶子的权重会在它到根的每次合并中各算一次，所以两种记账一致。

解码 `0110111`：从根读 0 到 D，回根；再读 110 到 A，回根；再读 111 到 B，得到 DAB。每到叶子立即完成一个符号，无需在符号之间额外加分隔符。若只剩一个符号，数学上可用长度 0 编码，但实际文件格式通常还需保存消息长度，或约定用一位码；这一边界需由格式规定。

11.2 贪心选择的理由，要先证明“存在一棵最优树这样做”

对正权重的最优前缀编码树，内部节点不需要只有一个孩子：若存在这种节点，将它绕过，其后所有叶子都少走一条边，总成本严格减小。所以可以考虑每个内部节点恰有两个孩子的树。

在任意这样的最优树中，取最深的一对兄弟叶子。将最轻的两个符号 x、y 交换到这两个叶子上，不会增加成本。为什么？若 w_x≤w_z、深度 d_x≤d_z，交换后成本变化为(w_z−w_x)(d_x−d_z) <= 0。把较轻者放得更深，不会变差。重复交换，可得到一棵仍最优、且最轻两符号为最深兄弟的树。

将这对兄弟收缩成权重 w_x+w_y 的一个伪符号，原树成本减去 w_x+w_y。收缩后的树必须对缩小后的权重集合也最优；否则用一棵更便宜的树替换它，再展开 x、y，便得到比原最优更便宜的树，矛盾。

于是原问题可安全地先合并最小两个，再递归处理。注意证明说的是“至少有一棵最优树具备这个结构”，并没说所有最优树形完全相同。频率相同、左右孩子互换，都可能产生不同编码而总成本相同。

11.3 例 2：不按最小两项合，成本会变多少

权重 1、2、8、9。Huffman 先合 1+2=3，再合 3+8=11，再合 11+9=20，总成本 34。若先合 8+9=17，再合 1+2=3，最后合 17+3=20，总成本 40。

第二种把高频 8、9 一起埋到更深层，每出现一次都多花位数。这个反例说明“任意两棵合起来最后总权相同”不意味着总编码成本相同。根权重总是 20，但叶子的深度不同。



图 83　教师原图中的 Huffman 合并

原图把两棵取出的最小权树接到新根下，新根权重等于两者之和。注意图中接口名可能写 delMax，而注释说取最小，这是通过比较器或权重优先级的定义实现的；读程序应核对比较规则。若直接用默认最大堆取最大的两个，就变成上面的错误策略。

11.4 构造的时间，取决于怎样找最小

用无序表，每次扫一遍找最小两项，剩余项数从 n、n−1 一直降到 2，每轮要看遍这些项，总工作与 n+(n−1)+…+2 同阶。首尾配对或用等差求和得 n(n+1)/2−1，因此 n−1 次合并共 Θ(n²)。用二叉最小堆，初始建堆 O(n)，每轮两次删除最小、一次插入，各最多走树高 O(log n)的一条路径；共 n−1 轮，所以总时间 O(n log n)，树节点 O(n)。每次输出完整编码串还要计入输出位数，不能把所有字符串复制成本都塞进常数。

如果输入权重已经非降排序，可用两个队列。一个放尚未使用的原权重，另一个放新合成权重；每次从两个队头中取较小者两次，合并后放到第二队尾。新合成权重会非降出现，因此第二个队列也有序。所有权重进出常数次，构造为 O(n)。如果原输入无序，先排序通常要 O(n log n)，不能把这一步悄悄删掉。

11.5 例 3：两个有序队列怎样执行

权重[2,3,7,8]已经非降排序。Q1 保存尚未用的原权重，Q2 保存已经合成但尚未用的树根；表中左端都是队头。每取一次都重新比较两队头，取出的项立即出队。

轮次	取第一项之前 Q1 / Q2	第一项	第二项	合并后 Q1 / Q2	本轮增加成本
1	[2,3,7,8] / []	Q1 的 2	Q1 的 3	[7,8] / [5]	5
2	[7,8] / [5]	Q2 的 5	Q1 的 7	[8] / [12]	12
3	[8] / [12]	Q1 的 8	Q2 的 12	[] / [20]	20

最后只剩根 20，停止，总成本 5+12+20=37。第一轮的两个最小项都来自 Q1，所以每轮固定从两个队列各拿一个会在第一轮就出错。

最小两个可能同来自 Q1、同来自 Q2，或者一边一个。不能每轮固定取“一个原符号、一个合成符号”。统一比较两个队头，才与全局取最小两项一致。

11.6 比较排序下界：为什么不能靠比较把所有输入都线性排好

n 个互异元素有 n!种可能的大小次序。这里 n!读作 n 的阶乘，等于 n×(n−1)×…×2×1：第一名可从 n 项里选，第二名剩 n−1 项可选，直到最后一项，所以要连乘。比如 3!=3×2×1=6。只允许两项大小比较的确定性算法，每次比较最多产生两个结果，可画成二叉决策树。不同输入次序必须到达能区分它们的叶子，否则同一输出次序至少会对一种输入错误。

高度 h 的二叉树最多 2^h 个叶子，所以 2^h≥n!，有 h≥$\log_2(n!)$。后半约 n/2 个因子都至少 n/2，因此 $\log_2(n!) \geq (n/2)\log_2(n/2)=\Omega(n \log n)$。这给最坏比较次数下界。

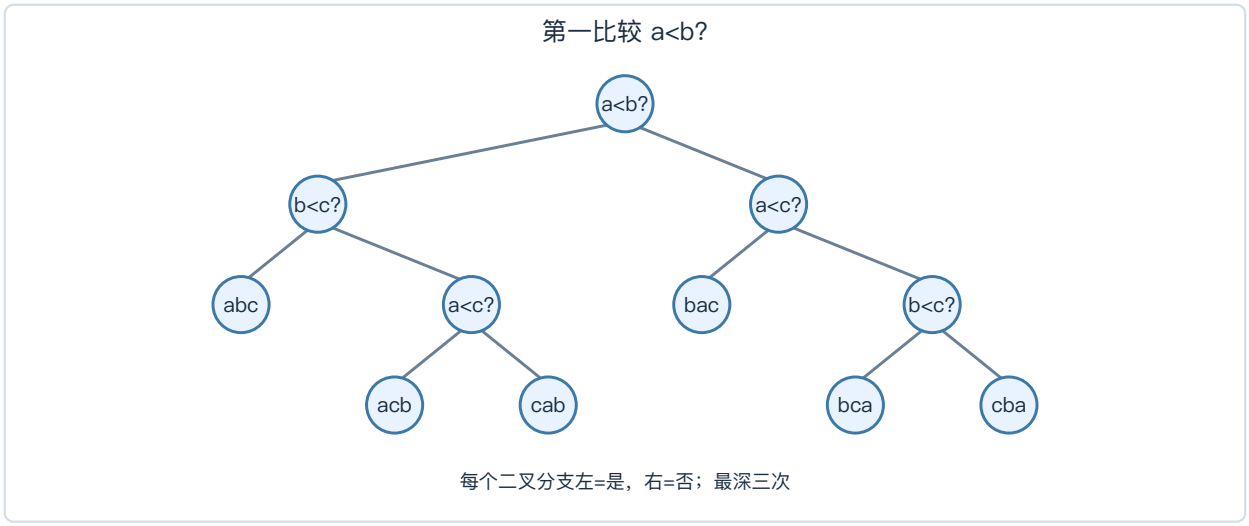


图 84　三个互异元素的一棵比较决策树及六个叶子

图中每条左分支表示“是”，右分支表示“否”。比如先发现 a<b，再发现 b 不小于 c，此时还分不清 acb 和 cab，必须再比较 a<c。走 a<b、b<c 的输入只用了两次；另一些输入会走三次。叶标签 abc 表示三个元素的从小到大顺序，三个比较问句不是在交换数组。

n=3 时有 6 种次序，2 次比较最多区分 4 种结果，故至少某些输入要比较 3 次。它并没说每一个输入都一定 3 次，也没说所有非比较算法都需要 3 次；模型限制是论证的一部分。

若输入是均匀随机排列，平均深度同样有 Ω(n log n)下界，可以用前缀码的 Kraft 不等式证明。若输入分布偏向少数有序情况，平均下界要改用其信息量，不能直接继续套 log(n!)。

11.7 归约时，箭头方向必须读对

若问题 A 能用 O(n)时间转成问题 B，并能从 B 的答案 O(n)恢复 A 的答案，就说 A 可线性归约到 B。若 B 有线性算法，A 便也有线性算法；因此已知 A 有 n log n 下界时，可以将下界传给 B。

要证明 Huffman 构造在任意未排序实权重的比较模型下有相应下界，必须明确你要求输出的对象、从中恢复排序的成本及相关归约细节；不能只因为常用实现先排序，就宣布所有算法必先排序。排序好的权重可线性构造已经说明条件会改变复杂度。本节先给比较树与归约方向；PS05 第 37 题用强分离权重构造至少 n!/2 类可区分带标签输出，进一步展开 Huffman 特定下界。阅读时仍应保持该题的比较模型、输入权重范围与输出对象约定。

11.8 权重相同，树不唯一却仍可严格核对

四个符号权重全 1，先任取两项合 2，再取另两项合 2，最后合 4，总成本 2+2+4=8，每项深 2。若第二步错误地合 1 与刚产生的 2，得到 3，再与剩 1 合 4，总成本 2+3+4=9，比最优差。即使初始权重相同，“每次最小两项”仍有实质约束。

在最优成本固定的情况下，左右交换、相同权重叶子互换会改变码字。自动检查不应只与某一份码字文本完全相等，应验证前缀无关、每个符号都编码、解码可还原、带权长度达到要求。如果题目指定规范 Huffman 码，才额外按长度与符号顺序生成确定的码字。

11.9 交换论证中的成本差，再代一组数

权 1 的 x 在深度 1，权 5 的 z 在深度 3，原贡献 1×1+5×3=16。互换后 1×3+5×1=8，下降 8。一般变化为(w_z−w_x)(d_x−d_z)，这里 4×(−2)=−8。先用数看清符号，再写不等式，就能避免把“轻的放深”证明反了。

把最轻两项放到最深兄弟位置时，可能一次涉及两个已有符号交换，应逐次应用不增成本原则；若它们本来已占其中位置，就少做一次。证明不要求每次严格下降，因为相等权重或深度相同可能使变化 0，仍保留最优。

11.10 两队列线性构造，为何新合并权重不会倒退

本次取出的最小两项为 a≤b，合并 a+b。剩余项都不小于 b，下一轮取出的两项 c≤d 至少满足 c≥b；若其中包含刚生成的 a+b，也仍不小于 b。因为上次 a≤b，下一次合并总值 c+d≥2b≥a+b，因此合成队列非降。

这个不变式保证只看两个队头就够。若允许负权，a+b 可能小于 a，使队列顺序与编码最优化解释都需要重审；普通频率与出现次数为非负，正权证明可将零频符号按格式约定先处理或保留，不要直接把带负权的“成本”套入同一贪心定理。

11.11 输出码字也是工作

如果某个 Huffman 树退化得很深，逐叶输出完整 0/1 字符串的总字符量可达平方级，即使构造树本身用堆只有 n log n。只返回树的节点链接与权重，是 O(n)规模的输出；打印所有根叶路径，是另一种输出要求。复杂度陈述应写清你构造了什么，而不是让“编码”一词同时代指树、长度表与所有完整字符串。

11.12 编码树中的未使用叶子，可以怎样理解

前缀无关编码不一定把整棵二叉空间占满。例如只有编码零与一零，仍可解码，但一一对应的分支空着。若两个符号权重都正，将第二个码缩为一，总成本会下降；所以最优二进制编码可去掉无用单孩子分支，得到真二叉树。

若权重为零，缩短它可能不改变成本，最优树仍可能有多种无成本差异的形状。实际压缩格式还需传送码表、频率表或树结构，这部分头部开销不在单纯带权路径长度中。只有很短的消息时，省下的正文位数可能还不够支付头部；算法优化的目标量要与文件总大小区分。

Huffman 针对已知符号频率的逐符号码最优，不说明它能捕捉任意跨符号相关性。例如两个字符总成对出现，把一对作为一个复合符号建模可能得到另一种压缩效果；模型变了，最优性范围也随之改变。

两个符号频率分别九与一，最优前缀码仍各一位，总十位；不能给常见符号空码、罕见符号一位，因为空串是任何字符串的前缀，解码会丢失符号边界。单符号的空码约定不能推广到两个符号。

11.13 练习与答案

练习 1。 权重 2、2、3、6，写合并序列、总成本和一种编码长度。

解答：2+2=4，3+4=7，6+7=13，总成本 24。权 6 叶深 1，权 3 叶深 2，两项权 2 叶深 3，所以 6×1+3×2+2×3+2×3=24。左右路径可互换，因此具体码字不唯一，长度与成本在该构造下如上。

练习 2。 只给码字长度 1、2、2，能否安排前缀无关二进制编码？长度 1、1、2 呢？

解答：前者可为 0、10、11，成立；后者若两项长度 1，已占 0 和 1，任何更长二进制串都以其中一个为前缀，无法再放第三项。也可用 Kraft 和检查：前者 1/2+1/4+1/4=1，后者 1/2+1/2+1/4>1。

练习 3。 有人拿已经排序权重的 O(n)Huffman 算法，声称推翻比较排序下界，为什么不成立？

解答：算法接收的输入已含顺序信息，省去了原本必须确定的 n!种顺序。比较排序下界讨论任意未排序互异输入。使用更强的输入前提，并没有解决原来同一个问题。任何复杂度冲突，先检查模型、输入范围与输出要求是否一致。

12 搜索树：每次比较之后，往哪边走

有序向量查找快，却可能为一次插入搬动很多项。链表局部插入快，却很难快速找到按键值的位置。二叉搜索树把“有序”放进分叉关系：每个节点左子树的所有键较小，右子树的所有键较大。为了先讲清关系，本章假设键互异；重复键可以用计数，或统一放某一侧，但接口和证明要配套。

12.1 例 1：查找与插入，共用同一条比较路径

依次插入 8、4、12、2、6、10、14。开始 root 为空，插 8 时新节点直接成为根。此后每次从根比较，直到到达空链接，再把新节点接在那里。三张图保留了每次插入后的整棵树；已有节点的键和孩子无需重排。

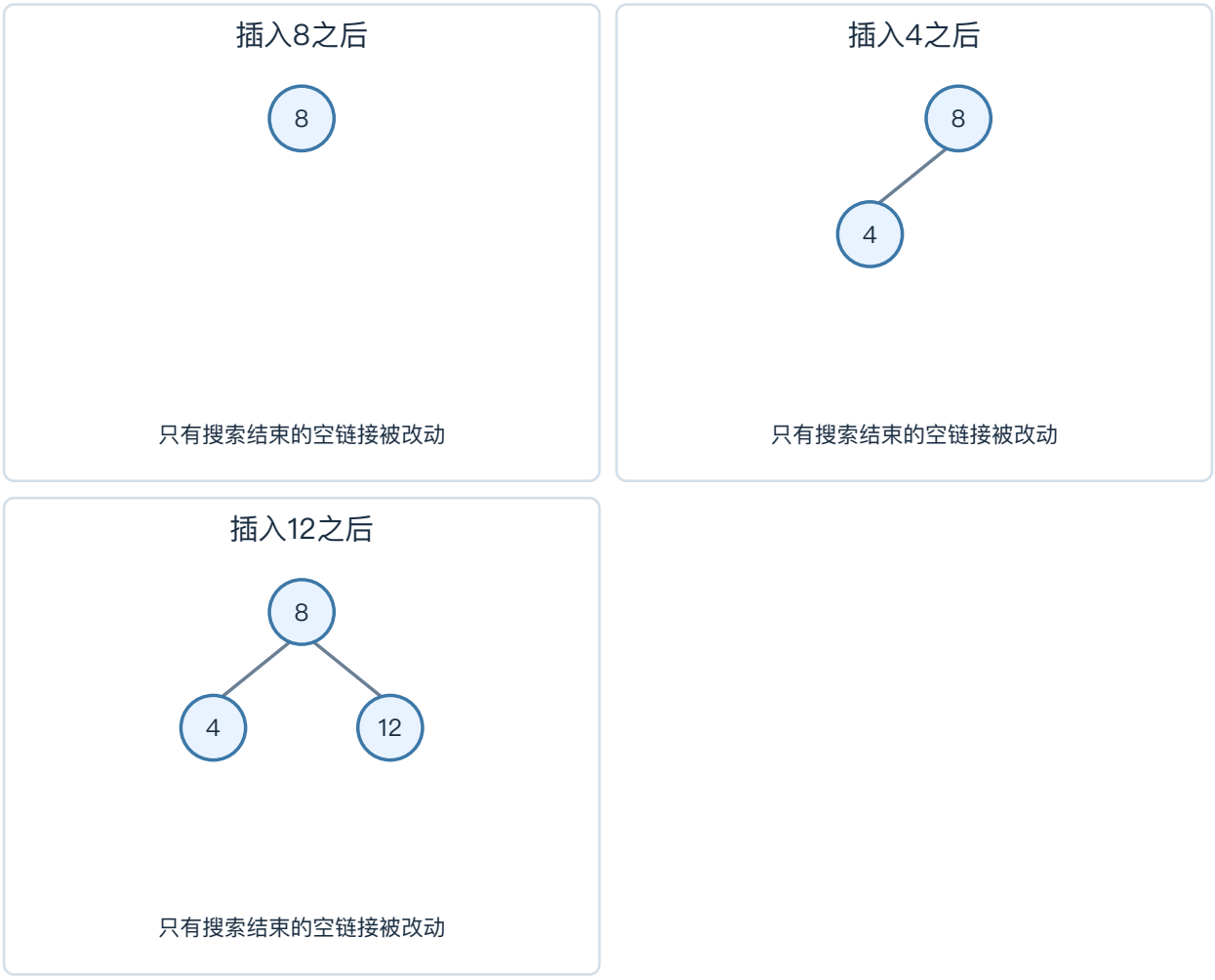


图 85　前三次 BST 插入的完整树

插 2 时沿 8 左、4 左到空；插 6 沿 8 左、4 右到空；插 10 沿 8 右、12 左到空。每个“左”表示目标小于当前键，每个“右”表示目标较大。

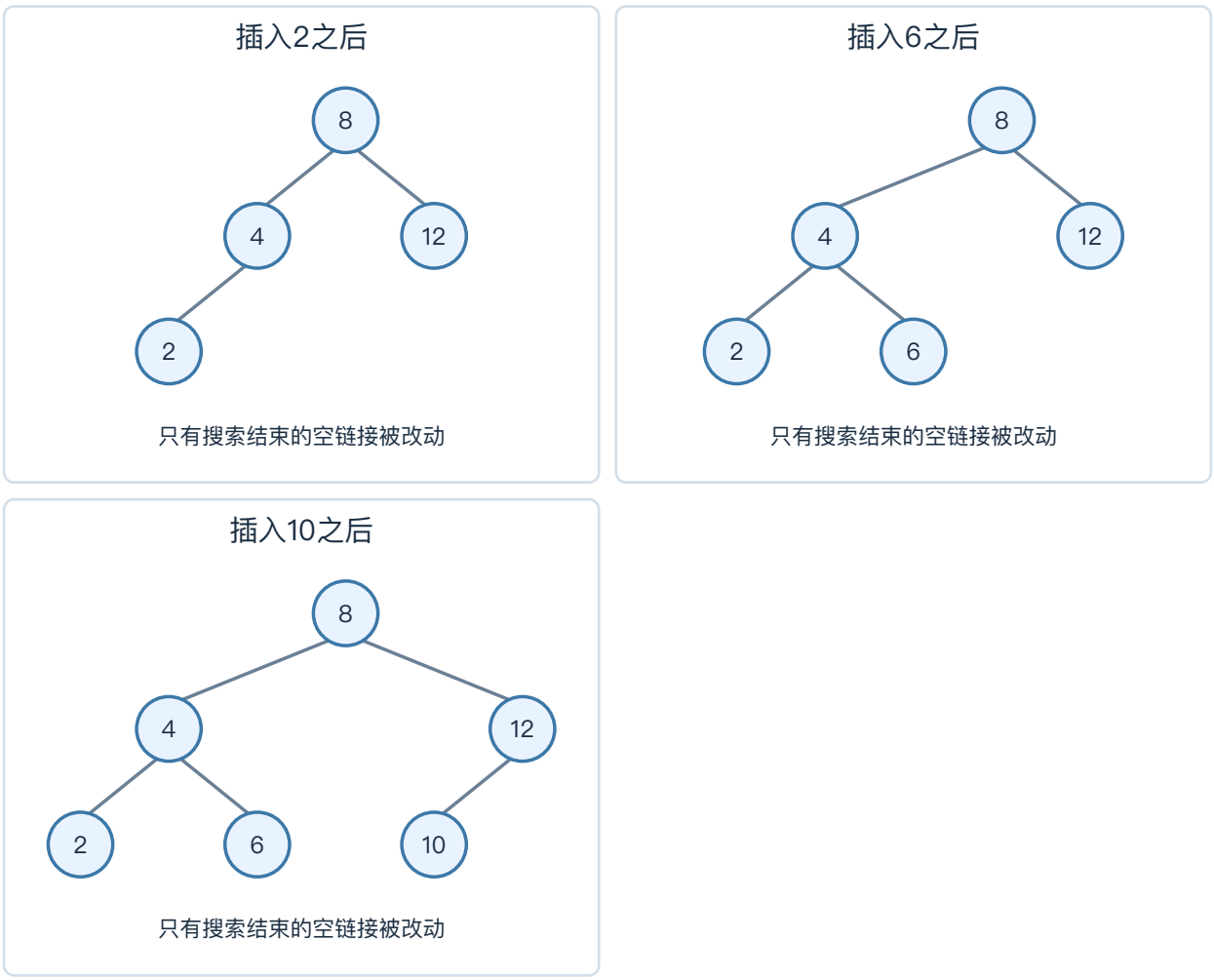


图 86　插 2、6、10 后的完整树

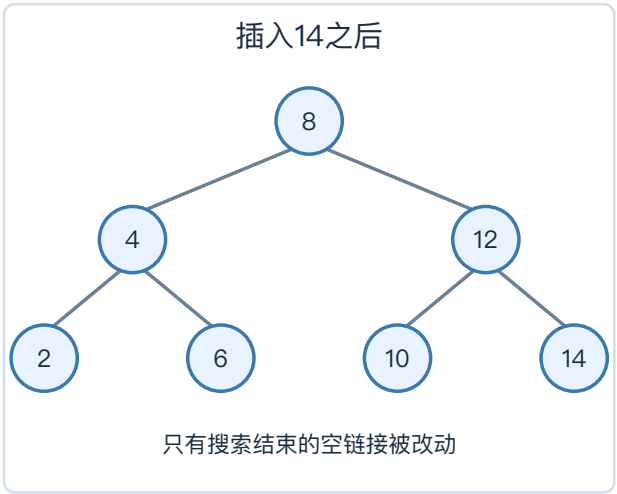


图 87　插 14 后的完整树

查找 6：6<8 向左到 4；6>4 向右到 6，找到。查找 5：向左到 4，向右到 6，再向左到空，失败。这个空槽正是插入 5 的位置。路径上每个比较都排除另一侧的整棵子树，因为那里的所有键都不可能等于目标。不变式是“若目标存在，则它仍在当前子树内”。每步进入一个严格更小的子树，最终到目标或空槽。时间 O(h)，h 为树高。若树退化成链，h=n−1，查找最坏线性；仅仅名字叫“树”不保证对数级。

中序遍历先列左侧小键，再根，再右侧大键，所以 BST 中序严格递增。反过来，若二叉树中所有互异键的中序严格递增，也能推出每个节点左右满足搜索性质。因此旋转是否保序，可以通过中序序列检查。

12.2 例 2：删除节点，要分清它有几个孩子

删叶子 2，直接断开父 4 的左链接。删只有一个孩子的节点，让父直接接到它的唯一孩子。两个孩子都在的节点，不能把左右孩子随便挤在同一个位置。

先独立考虑删叶子 2：把父 4 的左链接置空，其他链接不动。再独立考虑只带右孩子 11 的节点 10：把父 12 的左链接从 10 改到 11，令 11.parent 指向 12，然后释放 10。下面的两孩子例特意保留这个 11，避免把后继误当成必定是叶子。

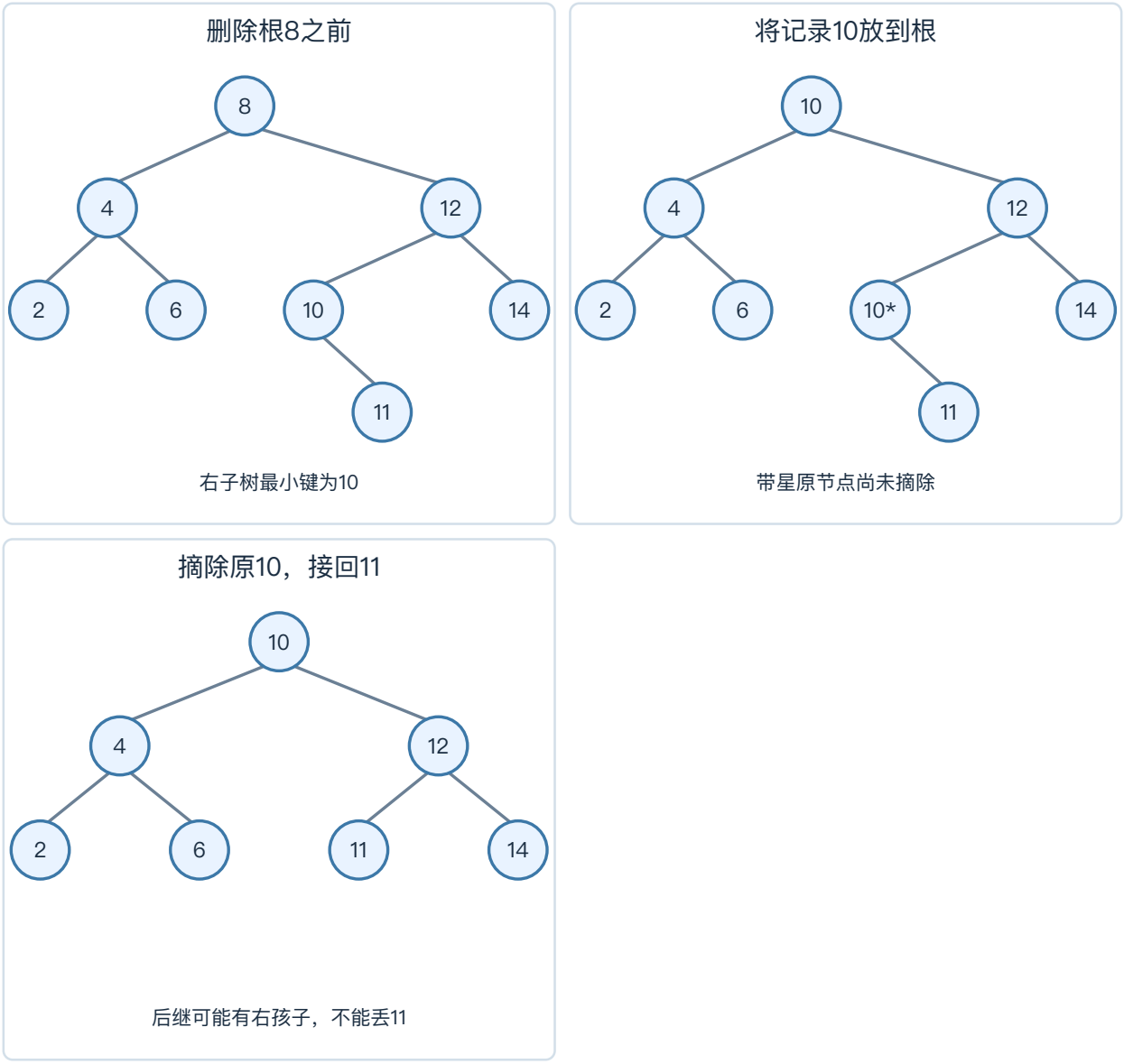


图 88　BST 删除 8 时复制后继记录并接回后继的右孩子

要删根 8，先从其右孩子 12 一路向左，遇 10，再向左为空，故后继为 10。把 10 的完整记录复制到根位置，暂时有两个位置显示 10；然后摘除原 10，用它的右孩子 11 接替原位置。最后根为 10，左边仍 4 带 2/6，右边 12 带 11/14。图中的星号标记的是待摘除的旧位置，不是重复键插入。

若节点存(学号,成绩)记录，替换时必须复制整个对应记录，不能只改学号却留下旧成绩。若外部持有节点身份，复制记录与真正交换节点也有不同语义；接口应说明外部引用会怎样变化。

后继为何没有左孩子？它已是右子树最左节点，有左孩子便还能向左走，与“最左”矛盾。为何替换后仍保序？后继是所有大于原根键的键中最小者；左子树所有键小于原根，也小于它；右子树去掉后继后其余键仍大于它。

删除总路径长度 O(h)，还可能需要沿祖先更新高度或子树大小。若有父指针，删除后孩子的 parent 也必须更新，不能只改向下链接。

12.3 平衡是在控制高度，不要求左右节点数相等

按 1、2、3、4、5 顺序插入普通 BST，会形成一路向右的链。AVL 树要求每个节点的左右子树高度差绝对值不超过 1。平衡因子可定义为 `height(left)-height(right)`，合法值−1、0、1。

现在具体看，这条要求怎样限制高度。用 h 表示树高，仍按边数计：叶子的高度是 0，空树高度是−1。用 `N(h)`表示“高度恰为 h 的合法 AVL 树，最少需要多少个节点”。我们先数出几个小高度，再推一般关系。

一棵高度为 h 的非空树，至少有一棵孩子子树高 h−1，否则整棵树达不到 h。另一棵子树可以矮一些，但 AVL 只允许差 1，因此它至少高 h−2。要用最少节点达到这个高度，就让两棵子树分别采用各自最少节点的形状，再加上根的 1 个节点：

$$N(h) = 1 + N(h-1) + N(h-2), \qquad h \geq 1.$$

这个式子的三项依次是根、较高的孩子子树、较矮的孩子子树。空树没有节点，所以 N(−1)=0；叶子只有自己，所以 N(0)=1。代入时可以一行一行算：

高度 h	最少节点数怎样算	N(h)
−1	空树	0
0	单独一个叶子	1
1	1+N(0)+N(−1)=1+1+0	2
2	1+N(1)+N(0)=1+2+1	4
3	1+N(2)+N(1)=1+4+2	7
4	1+N(3)+N(2)=1+7+4	12

高度不能随意增加：从高度 3 增到 4，即使用最省节点的形状，也要从 7 个增到 12 个。下面给一个不必记住这串数列名称的上界证明。

为消掉递推中的常数 1，另记$M(h) = N(h) + 1$。把两侧都加 1，再将 N 换成 M：

$$M(h) = N(h) + 1 = 2 + N(h-1) + N(h-2) = M(h-1) + M(h-2).$$

M 的初值为正，递推中每项又是两个正数相加，所以每一项都比前一项大；于是$M(h-1) \geq M(h-2)$，从而

$$M(h) \geq 2M(h-2).$$

也就是说，高度每增加两层，所需节点数加 1 后的数值至少翻倍。基础值为 M(−1)=1、M(0)=2。从这两个起点反复使用上式，可得

$$M(h) \geq 2^{(h+1)/2}.$$

逐步代一次就能看出指数怎样增加：若$M(h-2) \geq 2^{(h-1)/2}$，则$M(h) \geq 2 \cdot 2^{(h-1)/2} = 2^{(h+1)/2}$；两个基础高度也都满足这个不等式。

实际的 AVL 树若有 n 个节点、高度 h，节点数至少是 N(h)，所以$n + 1 \geq M(h) \geq 2^{(h+1)/2}$。对两边取以 2 为底的对数，再移项，得到

$$\log_2(n+1) \geq \frac{h+1}{2}, \qquad h \leq 2\log_2(n+1) - 1.$$

因此高度最多按 log n 的速度增长。查找沿一条根到叶的路径向下走，每层只比较一次，时间便受这个高度界约束。这里的常数还可以改进，但已经足以证明 AVL 查找的最坏时间为 O(log n)，不会像那条单链一样增长到线性。

12.4 例 3：一次右旋，为什么不破坏大小关系

```
旋转前          旋转后
      g              p
     / \            / \
    p   C          A   g
   / \                / \
  A   B              B   C
```

旋转前的键关系是 A 里的键<p<B 里的键<g<C 里的键，旋转后这条中序关系完全相同。改动是让 p 升上去，g 成为 p 的右孩子，B 转接成 g 的左孩子。不能丢掉 B，也不能把它接到 g 右边。

若 g 原来不是整树根，g 原父节点指向这棵局部树的链接要改成 p；若 g 是根，要更新 root。再更新 B 的 parent、g 的 parent、p 的 parent。高度必须先更新降下去的 g，再更新升上来的 p，因为 p 的新高度依赖 g。



图 89　教师关于局部旋转的原图

图中左右两幅互为逆操作，X、C、V 等方块代表整棵子树，不是单个值。旋转只改少量链接，整块子树内部不动，所以一次旋转 O(1)。中间子树在两个节点之间重新挂接，是保持中序不变的关键。

12.5 例 4：四种插入失衡，跟着三个节点的方向判断

下面四次实验各从空树开始，空树高度定为−1、叶高度 0。新节点加入后，沿它的父链重算高度，发现平衡因子绝对值达到 2 的最低祖先 g；从 g 到较高孩子 p、再到新节点所在孙子 v 的两次转向决定处理顺序。插 30、20、10：30 左子树高度 1、右空高度−1，相差 2。路径左左，所以在 30 右旋，20 升根，10/30 成为两孩子。

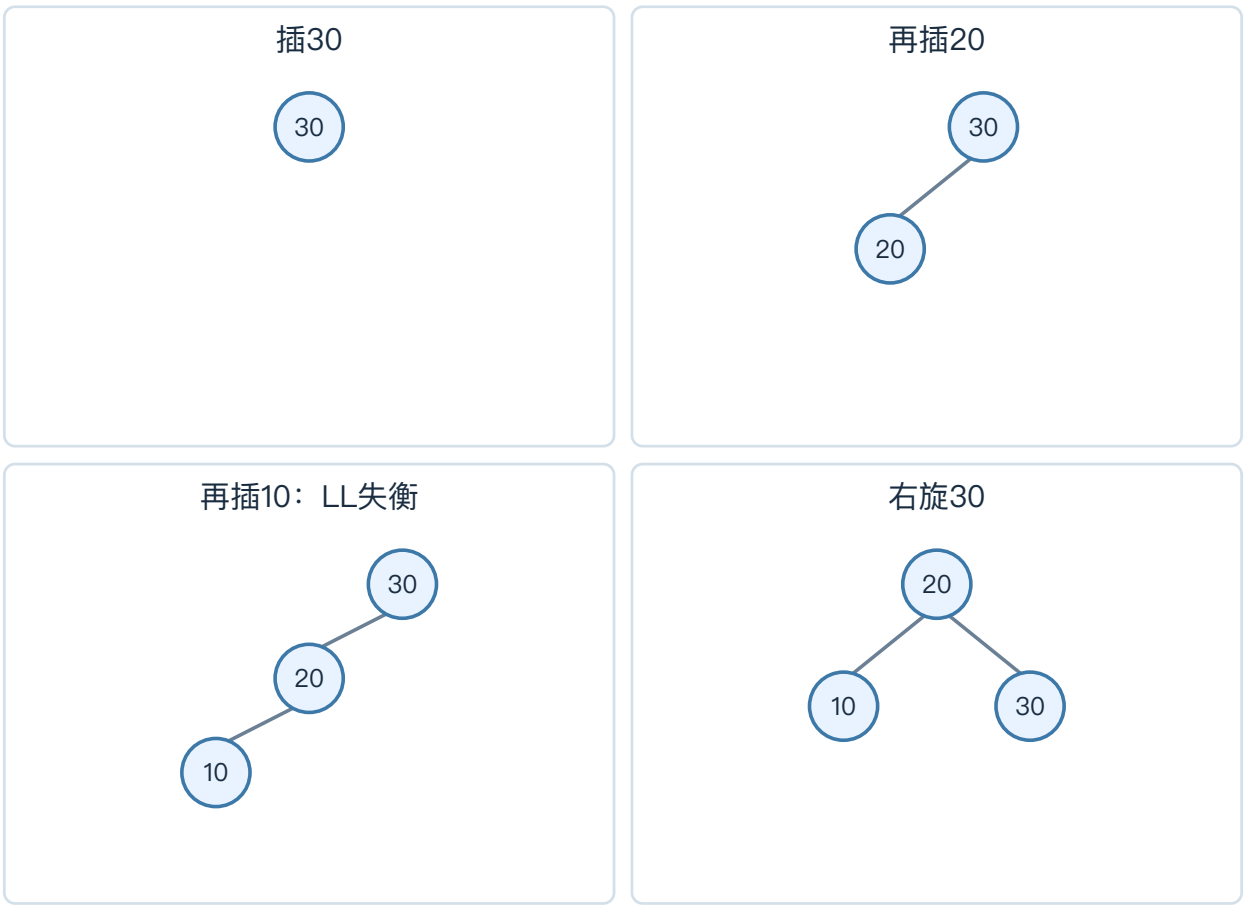


图 90　AVL 左左插入从空树到修复的每次状态

插 10、20、30：10 右侧过高，路径右右，在 10 左旋。这个实验的每次状态与上一例左右镜像。

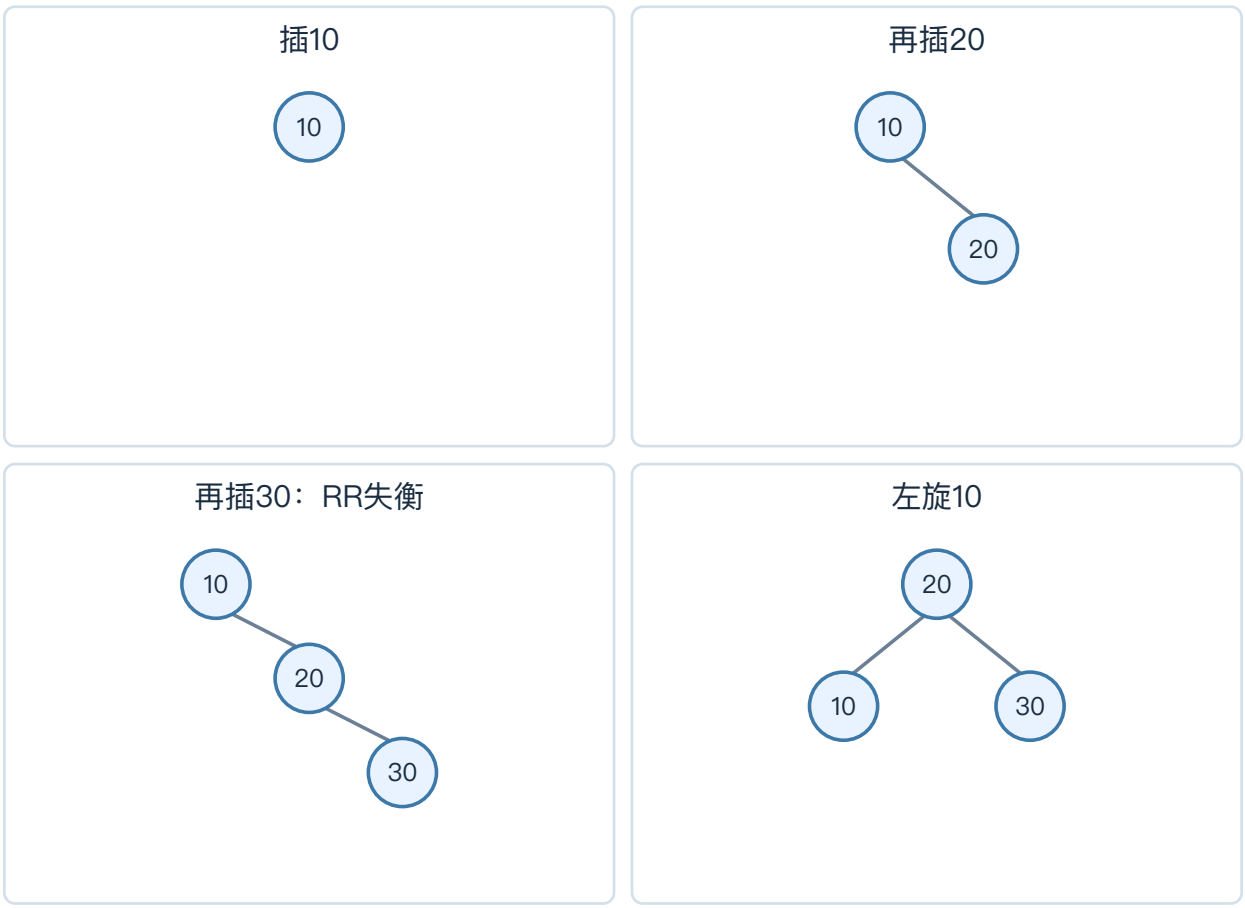


图 91　AVL 右右插入从空树到修复的每次状态

插 30、10、20：30 仍左重，但 10 右重。先左旋 10，20 升到 30 左边，10 成为 20 左孩子；再右旋 30，20 升局部根。

对输入 30、10、20，把两次旋转分别停下来看。图上方的父节点连向下方的孩子；橙色标出本步改变关系的节点。

AVL的LR修复：两次旋转分别做什么

1. 插入30



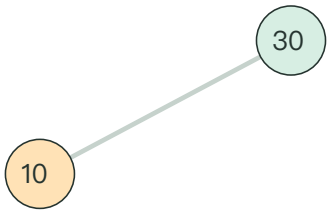
中序次序始终为已插入键的升序；旋转不改变键值。

根指向新节点30。
原因：根只有一个节点，左右空树高度均为−1。

图 92　逐步演示：AVL 的 LR 修复：两次旋转分别做什么

AVL的LR修复：两次旋转分别做什么

2. 插入10



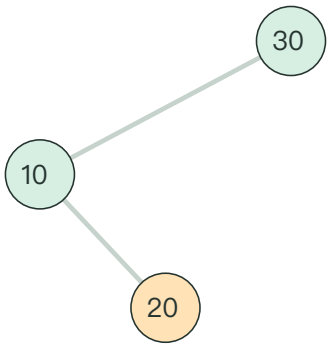
中序次序始终为已插入键的升序；旋转不改变键值。

30的左孩子指向新节点10。
原因：10小于30，成为30的左孩子。

图 93　AVL 的 LR 修复：两次旋转分别做什么，第 2 步

AVL的LR修复：两次旋转分别做什么

3. 插入20



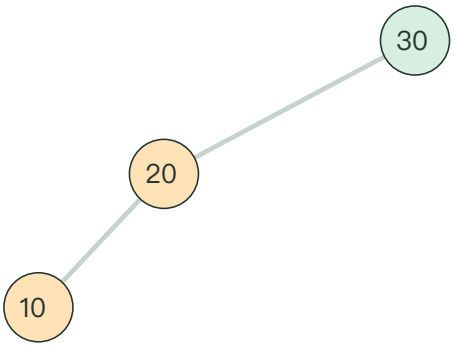
中序次序始终为已插入键的升序；旋转不改变键值。

10的右孩子指向新节点20；失衡点为30。
原因：20小于30但大于10，成为10的右孩子；30左右高度差为2。

图 94　AVL 的 LR 修复：两次旋转分别做什么，第 3 步

AVL的LR修复：两次旋转分别做什么

4. 先左旋10



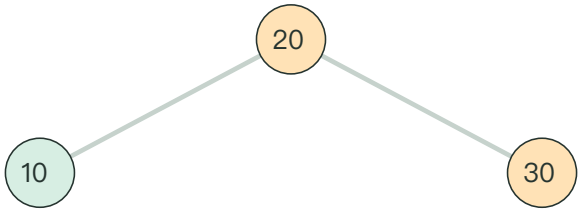
中序次序始终为已插入键的升序；旋转不改变键值。

30的左孩子改成20；10的右孩子改为空；20的左孩子改成10。
原因：20接替10原来的位置，10成为20左孩子；本例20原来的左子树为空。

图 95　AVL 的 LR 修复：两次旋转分别做什么，第 4 步

AVL的LR修复：两次旋转分别做什么

5. 再右旋30



中序次序始终为已插入键的升序；旋转不改变键值。

根改成20；30的左孩子改为空；20的右孩子改成30。
原因：20升为根，30成为20右孩子；20原来的右子树为空，接到30左侧。

图 96　AVL 的 LR 修复：两次旋转分别做什么，第 5 步

插 10、30、20：先右旋 30，使 20 成为 10 右孩子，30 成为 20 右孩子；再左旋 10，20 升根。第一旋仅把内折改成外侧，不能在第一旋后就停止修复。

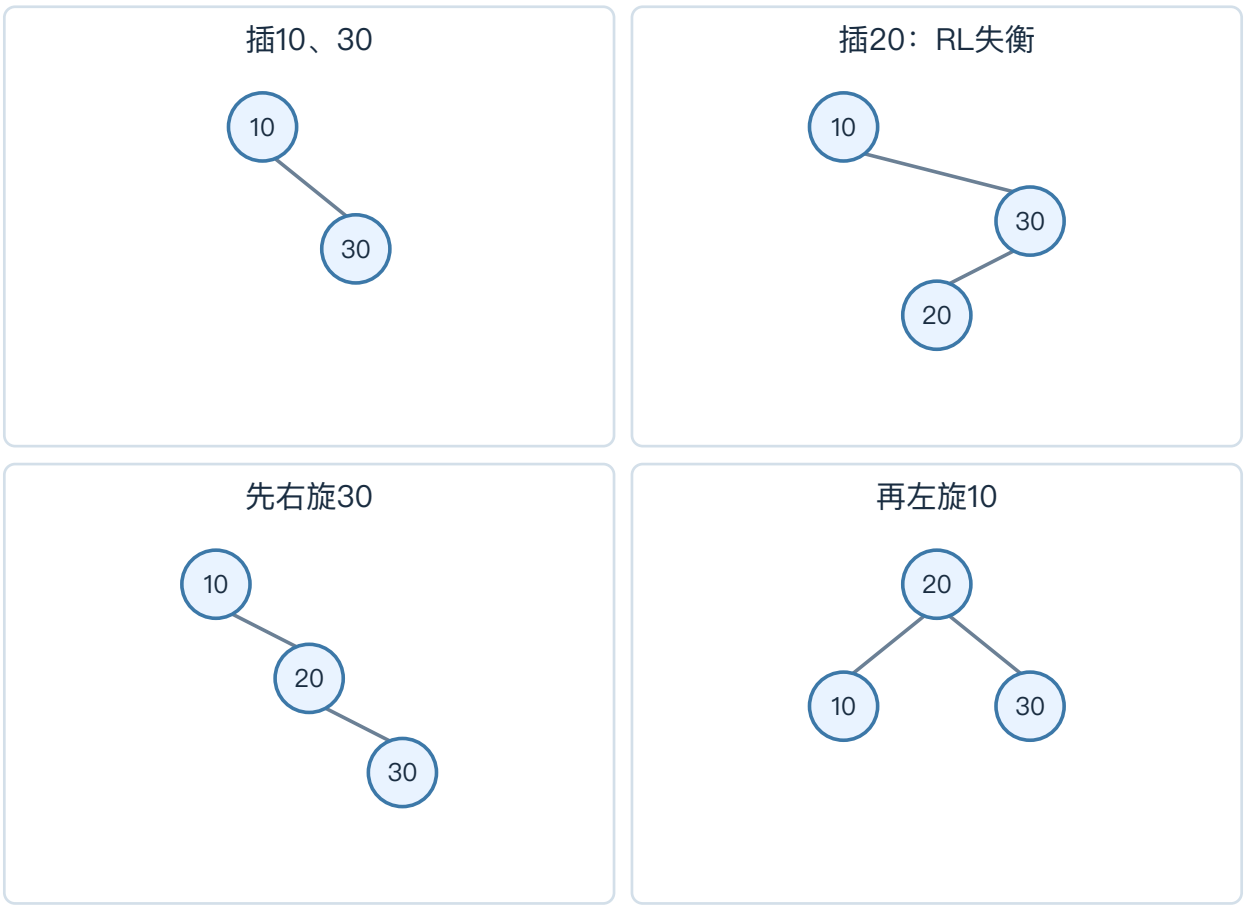


图 97　AVL 右左插入的两个中间旋转状态

三个关键节点都能按中序排成 a<b<c，最终 b 为局部根；真正带侧子树的情况必须保留四块值域，不能只背三节点形状。

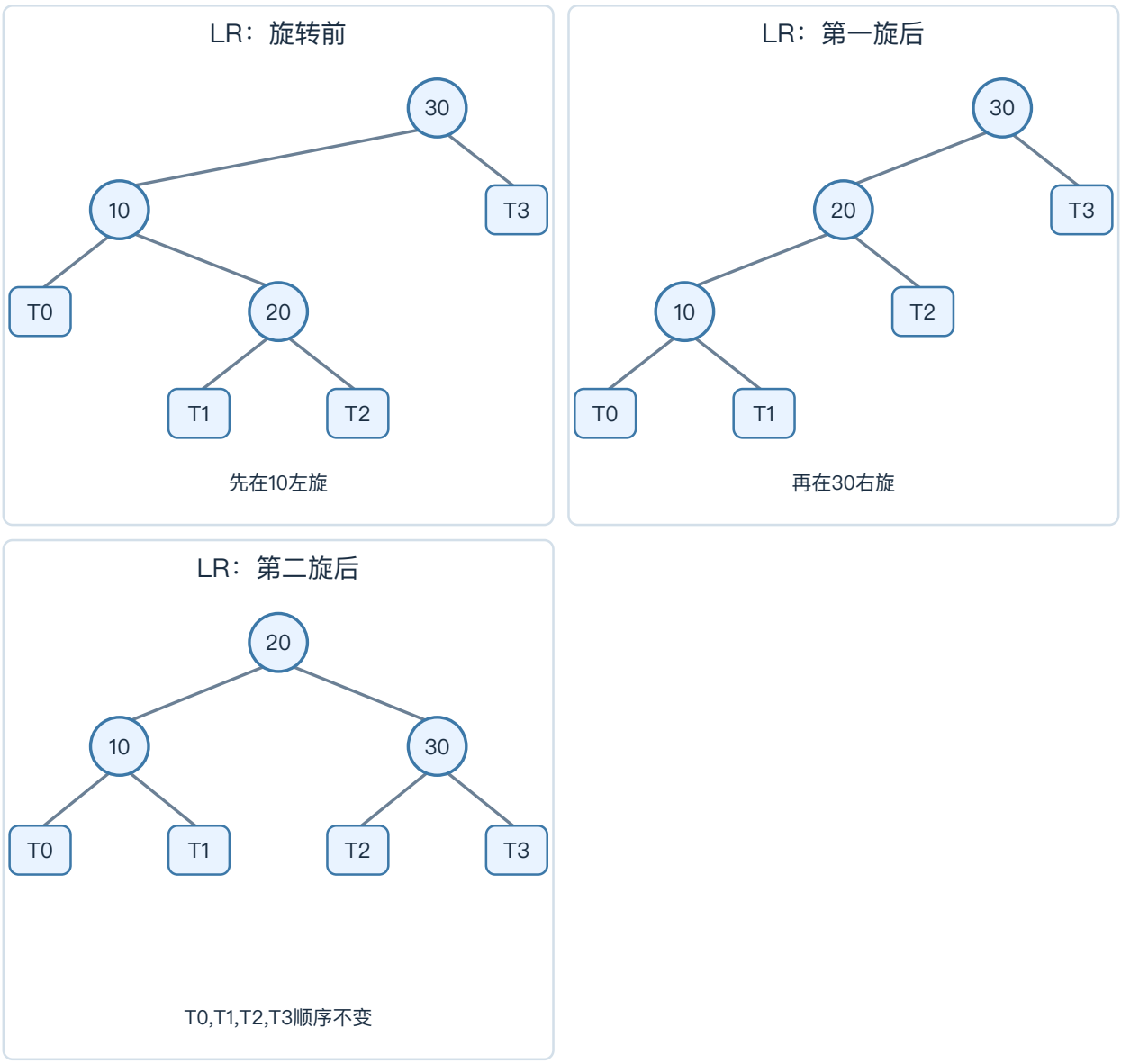


图 98　带四块侧子树的 AVL 旋转及转接位置

插入修复从新节点向根回溯，找最低失衡祖先。修复后这棵子树高度恢复到插入之前，故更高祖先不会因这次插入继续失衡；需要更新沿路高度，但旋转修复次数为常数。

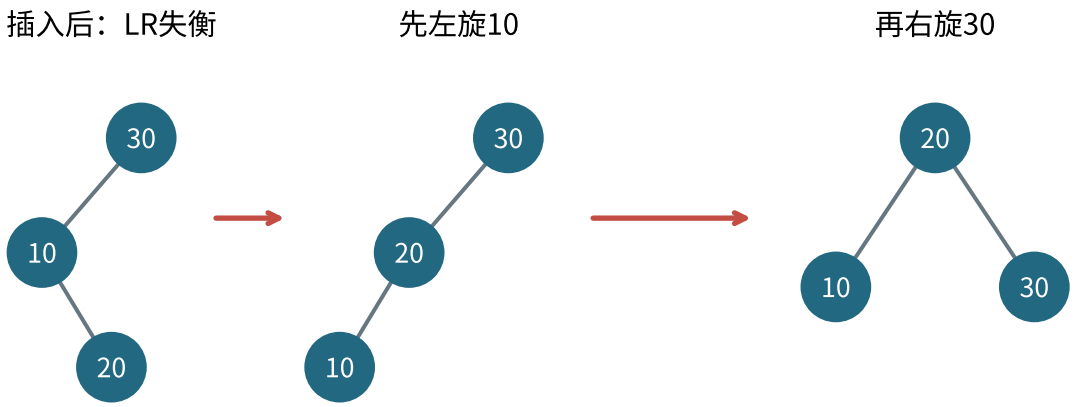


图 99　LR 失衡的两步旋转

左图三十的左孩子是十，十的右孩子是二十，形成内折。第一根箭头表示先在十处左旋，使二十上升；第二根箭头表示再在三十处右旋，最终二十成为局部根。三个节点的中序始终是十、二十、三十，图只省略了空子树；真实树中还要按值域接回四棵侧子树。

12.6 删除修复与插入不同

删除可能让子树高度减少 1，修复后仍比原来矮，因此失衡可能继续向上传播，最坏需要 O(log n)层的旋转修复。不能把“AVL 插入最多一次局部重构”直接套到删除。

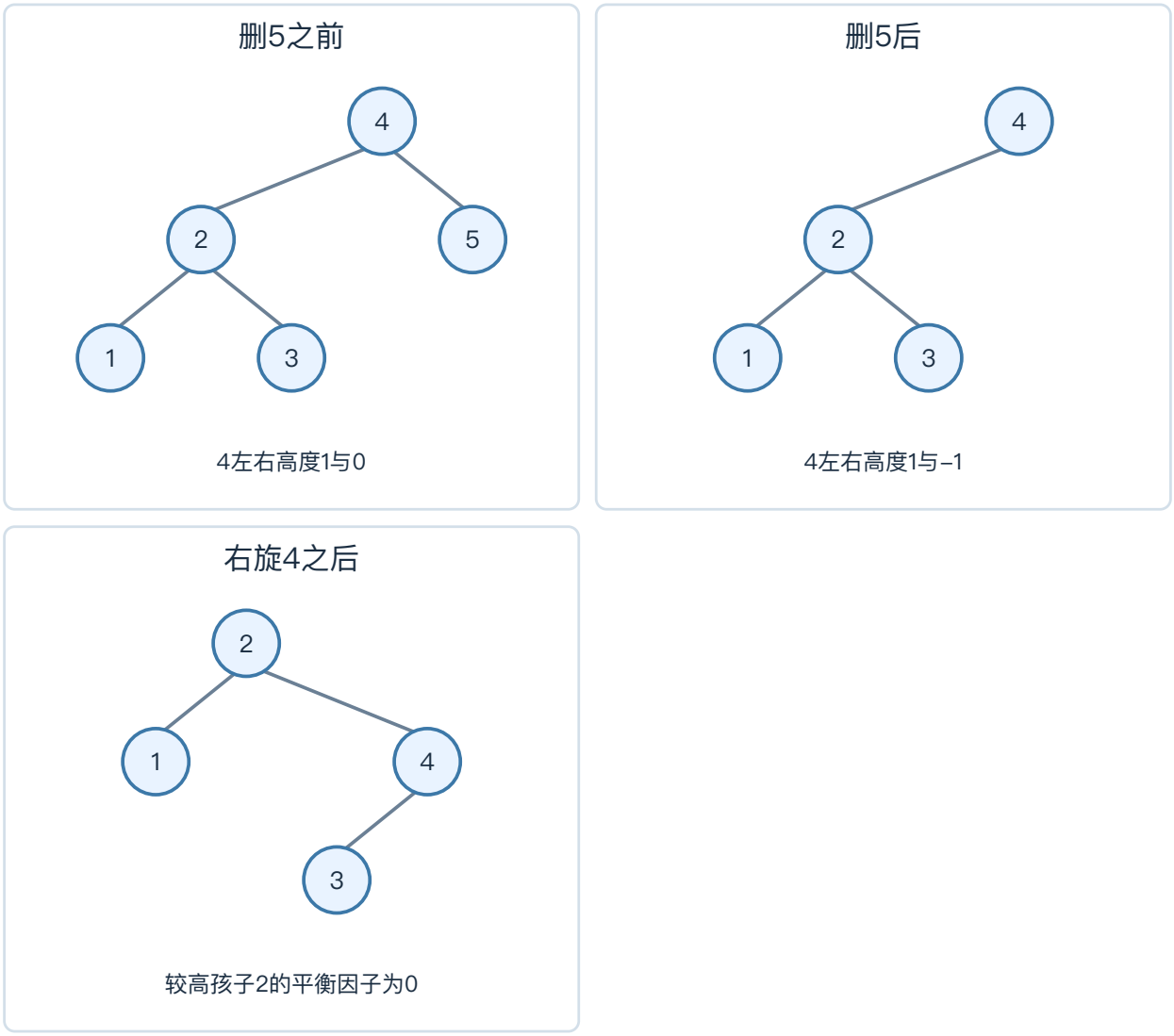


图 100　AVL 删除 5 后较高孩子平衡因子为零的修复

考虑根 4，左子树为 2 带孩子 1、3，右孩子 5 为叶。删 5 后，4 的左高 1、右高−1，平衡因子 2；较高孩子 2 的左右等高。对 4 右旋后根 2，左 1，右 4 且 4 左 3，树重新平衡。这里孩子平衡因子为 0，是删除中需要特别覆盖的一种情况；若只写插入时的 LL/LR 判断，可能遗漏。

每次回溯先重新计算高度，再判断因子，旋转后继续从新局部根的父节点向上。实现最好将“更新高度”“重新连接父链接”“局部旋转”分别封装，避免四种情况各写一份不一致的指针操作。

12.7 子树大小还能支持第 k 小

每个节点额外存 size=1+左 size+右 size。要找从 0 起的第 k 小，令 L 为左子树大小：k<L 就去左边；k=L 返回当前节点；k>L 就去右边，新的 k 减去 L+1。每次只走一条根叶路径，在 AVL 中 O(log n)。

例如原 7 节点树找第 4 小，根 8 左边 3 项，4>3，去右子树，k 改为 0；节点 12 左子树含 10 一项，0<1，去 10，返回 10。旋转不改变局部元素总数，但改变各节点子树范围，所以必须像更新高度一样更新 size，先下后上。

12.8 AVL 删除为什么可能一层接一层修复

插入使局部高度增加，最低失衡点重构后常恢复插入前高度，因此上方不再继续受影响。删除使局部高度降低，旋转后这棵子树可能仍比删除前矮 1，父亲就可能因另一侧相对变高而失衡。回溯必须继续，不能看到第一次旋转成功就直接返回。

准确实现时，从被物理移除节点的父亲开始，而不一定从最初指定删除键的位置开始。两孩子删除若用后继替换，真正少掉节点发生在右子树最左端；沿那里的祖先链更新才覆盖实际高度变化。若只从原根位置改一次高度，会漏掉右子树内部的修复。

12.9 三节点四子树重构，把双旋展开成一次有序接回

设按中序排序的三个关键节点为 a<b<c，它们之间四棵子树 T_0、T_1、T_2、T_3也按值域依次排列。重构后 b 为局部根，a 为左孩子、c 为右孩子；a 左右接 T_0、T_1，c 左右接 T_2、T_3。先更新 a、c 的 height 与 size，再更新 b，最后把 b 接回原祖父。

例如 LR 结构根 30，左 10，10 右 20。若 10 左 T_0、20 左 T_1、20 右 T_2、30 右 T_3，则重构根 20，左 10 挂 T_0,T_1，右 30 挂 T_2,T_3。四个值域从未改变相对顺序，所以搜索性质保留；中间两子树不再依原父节点身份挂接，而依它们数值区间挂接。这比只画 10、20、30 三个圆更接近真实指针操作。

12.10 有序统计的 rank 查询，和第 k 小方向相反

若问键 x 之前有多少键，比起查第 k 小，需要沿查找路径累计。当前节点键小于 x 时，它的整个左子树和自己都可计入，答案增加左 size+1，再去右侧；当前键≥x 时去左侧，不增加。不存在 x 也能得到它的插入秩。

原树键 1、3、6、8、10，查 x=7：根 8 太大去左，节点 3<7，计左 1 及自己，共 2；去 6，6<7 再计 1，总 3。恰有 1、3、6 三个键小于 7。若重复键以计数保存，当前节点贡献是 count 而非 1，size 也应统计所有记录数，接口到底返回不同键秩还是记录秩要先说明。

12.11 搜索失败不代表没有可用信息

BST 查找 5 最终停在 6 的空左槽，路径已经给出前驱与后继范围。可在下降时保存“最后见到的小于目标节点”与“最后见到的大于目标节点”，失败后仍能返回边界。平衡树除了是否存在，还能回答 lower_bound、upper_bound 与范围遍历，这些能力来自全局顺序，不是散列桶可以直接提供的。

12.12 等值键保存为计数，删除一个与删除全部不同

若节点键五的 count 为三，removeOne 只将 count 减到二，树形无需改变，但路径上 size 都要减一；removeAll 才需要物理删除节点，并将相关记录数量从增广 size 里扣除。若 size 统计不同键数，而查询要求含重复的第 k 小，两者就不匹配。

例如键一一份、键五三份、键九一份，记录序列为一、五、五、五、九。第五小是九，第三小是五；在节点五处，左边记录数一，自身覆盖接下来的三个秩。用左 size 加一替代左 size 加 count，会把后面秩算错。多重集合不是只允许重复插入这一个改动，查找、秩与删除语义都需一致。

搜索树的键应满足一致的比较顺序。若同一对键这次比较小于、下次又大于，或者键在树内被外部修改，原来指向左子树的范围可能不再成立。修改键通常应先删除旧记录，再按新键重新插入；只改节点字段而不重新定位，会让查找沿错误分支走失。浮点特殊值也可能不满足普通全序假设，比较器需明确定义它们怎样排列。

旋转前后还应核对总记录数不变，局部根接回以后，原祖先的孩子指针与新根的父指针必须互相对应。

12.13 练习与答案

练习 1。 依次插入 40、20、30，写出失衡类型与两次旋转后结构。

解答：40 左孩子 20，20 右孩子 30，是 LR。先 20 左旋，让 30 挂到 40 左边、20 成为 30 左孩子；再 40 右旋，让 30 成为根，左 20、右 40。中序始终 20、30、40，旋转后高度 1，左右高 0。

练习 2。 删除 BST 中有两个孩子的节点时，能否用前驱而非后继？

解答：可以。前驱是左子树最右节点，它不可能有右孩子，所以替换后也归约到 0 或 1 孩子删除。左右论证对称。应复制完整记录并更新原前驱位置的链接、parent 与增广信息。若重复键策略不同，前驱或后继替换需与等值放置规则一致。

练习 3。 左右子树节点数为 7 与 3 的节点，能否是 AVL 合法节点?

解答：可能。7 节点子树若高度 2，3 节点子树若高度 1，高度差 1，合法。AVL 约束高度，不约束节点数相等。反过来，节点数相近也不保证高度接近：某侧若退化为链仍可能很高，但它本身就违反 AVL 递归平衡条件。判断时必须检查每个节点的高度关系。

13 伸展树、B 树、红黑树与跳转表

同样是保存一组有序键，代价会随使用方式改变。连续查询同一个键，刚查过的节点值得靠近根；数据放在外存，一次读取一页，值得在一页里装进多个键；要求每次操作都有时间上限，就需要始终控制高度。下面分别操作四种结构。键互异，记录随键一起移动；相等键更新记录，不再插入一个节点。

13.1 伸展：访问 10 以后，10 怎样成为根

伸展树是一棵二叉搜索树。查找仍按“小往左、大往右”。查找结束后，命中的节点升到根；若查找失败，则把最后到达的非空节点升到根。这个升根过程叫伸展，英文 splay。空树没有可升的节点。

旋转只改变局部父子关系，不能改变中序次序。以下把目标记为 x、它的父亲记为 p、祖父记为 g。图里的 T0 到 T3 代表整棵子树，各自可能为空；移动方框时，其内部节点不需要重排。

13.1.1 同向：先转祖父，再转父亲

考虑 g=30、p=20、x=10，三者形成“30 的左孩子 20，20 的左孩子 10”。给侧子树各放一个键，T0={5}、T1={15}、T2={25}、T3={35}，于是完整中序是 5、10、15、20、25、30、35。

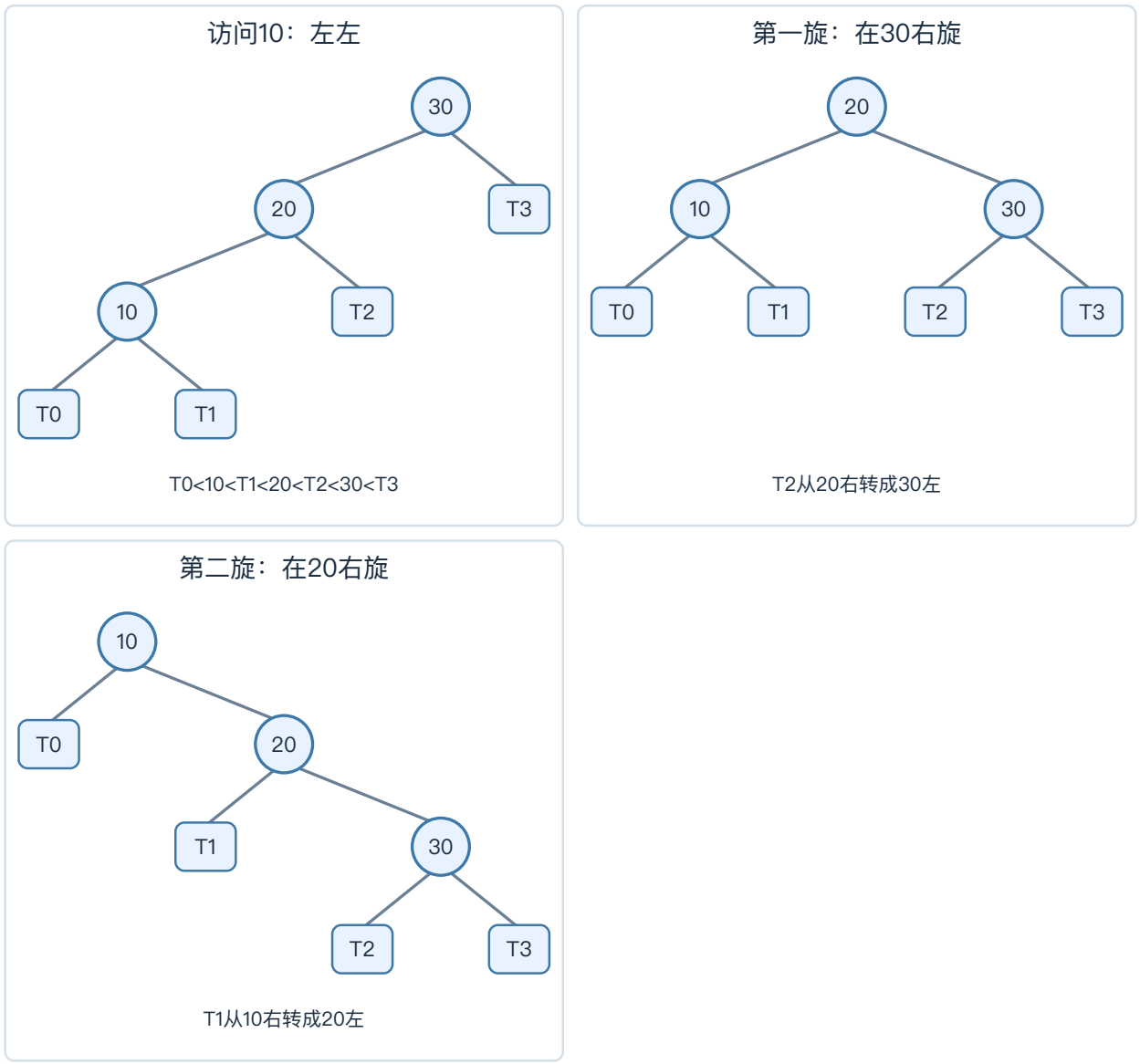


图 101　伸展左左情形的两次旋转与全部侧子树

第一幅中，10 在 20 左边，20 又在 30 左边，称左左，即 zig-zig。先对 30 右旋：20 顶替 30 的位置，30 成为 20 的右孩子，原来的 T2 不能继续挂在 20 右边，因为那里已由 30 占据。T2 里的键在 20 与 30 之间，恰好应当成为 30 的左子树。10 与它的 T0、T1 在这一步保持不动。

第二步对 20 右旋：10 顶替 20，20 成为 10 的右孩子；T1 里的键在 10 与 20 之间，因此改挂到 20 左边。最后从根读得到 10，左 T0，右 20；20 左 T1、右 30；30 左 T2、右 T3。逐项检查 5<10<15<20<25<30<35，可以确认没有丢键，也没有把某块子树挂错。

右右情形完全镜像：g 右 p、p 右 x，先对 g 左旋，再对 p 左旋。这里的顺序是标准双层伸展的一部分。若每次只让 x 与父亲交换，虽然最终也能提到根，中间树形不同，不能直接引用标准伸展的摊还证明。

13.1.2 异向：先把内折摊开

改成 30 左 10，10 右 20。x=20 在父亲右侧，父亲又在祖父左侧，称左右，即 zig-zag。侧子树仍按 T0<10<T1<20<T2<30<T3 排列。

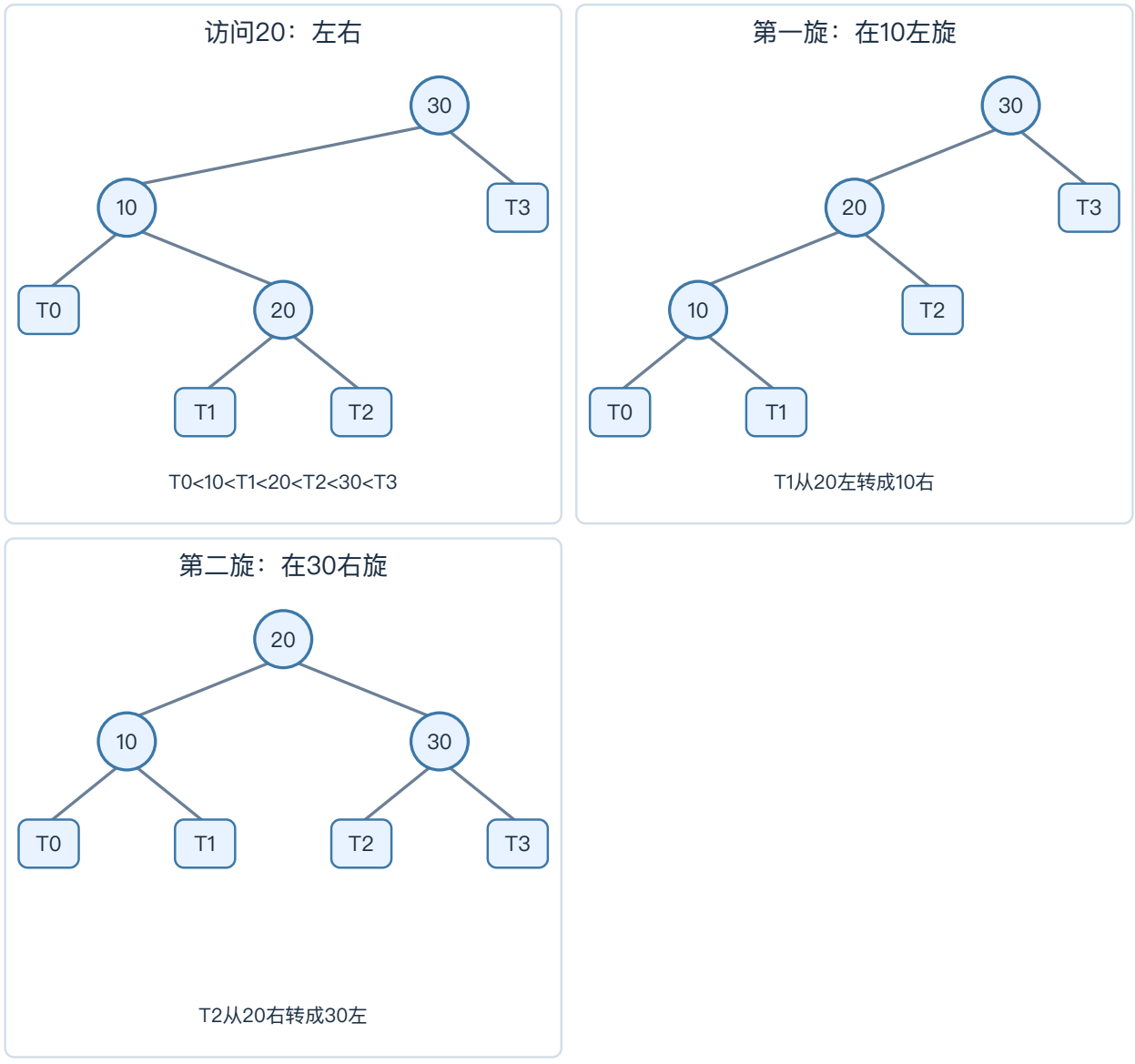


图 102　伸展左右情形的两次旋转与转移子树

先对 10 左旋，让 20 升到 10 原来的位置。20 原左子树 T1 转接成 10 的右子树，因为 T1 的键恰在 10 和 20 之间。此时局部根仍是 30，它的左孩子变成 20，20 左 10、右 T2。

再对 30 右旋，让 20 升上来。20 原右子树 T2 转成 30 的左子树。最终 20 左 10、右 30，10 挂 T0/T1，30 挂 T2/T3。右左情形按左右镜像执行，先右旋 p，再左旋 g。

如果 x 只有父亲，没有祖父，只做最后一次单旋。每处理一个祖孙三代，x 便向上两层；循环直到 x 没有父亲。旋转期间还须把新局部根接回原来 g 的父亲，并同步 parent 指针。若保存子树 size，高度或其他

摘要，要先更新降下去的节点，再更新升上来的节点。这里真正改变的链接只有常数条，所以每次旋转是常数时间。

13.1.3 分裂、插入、合并、删除在同一棵树上执行

设伸展 5 之后的树为根 5，左子树根 3 且左 1，右子树根 9 且左 7。按 5 分裂并单独返回 5：断开 5 的左、右链接，把两棵子树根的 parent 清空，得到 L={1,3}与 R={7,9}。它们各自仍为 BST，而且 L 中每个键都小于 R 中每个键。

向这两部分中间插入 6，可以直接新建根 6，左接 L，右接 R。此时中序 1、3、6、7、9，所有链接范围成立。如果输入的两个集合是 L={1,8}、R={3,9}，便不能这样接：8 会出现在 6 左侧而破坏顺序。合并的前提必须是所有左键小于所有右键。

删除新根 6 后，先保存 L、R，再释放 6。合并采用课程教材的“右边取最小”版本：从 R 根 9 向左查 6，查到 7，再往左为空；失败的最后节点 7 伸展到 R 根。7 没有左孩子，把整个 L 接到 7 左边即可。最终根 7，左 3 且 3 左 1，右 9。也可以把 L 的最大 3 升根后右接 R，两种版本对称；一次实现选择一套规则即可。

下面的等宽树专门对照断开的接口，和上面的双旋图结合阅读：

```
删除6前                删除6、暂不相接          R中7升根后合并
       6                L: 3      R: 9                 7
     /   \                /         /                / \
    3     9              1         7                3   9
   /     /                                         /
  1     7                                         1
```

查找 5 失败时为什么能据此插入？伸展后根是 5 的前驱或后继。以根为后继 7 为例，原根的左子树中所有键都小于 5，否则搜索路径不会在 7 左侧结束。新建 5，把原左子树给它，再把 7 作为其右孩子，并清空 7 原左链接，即可保持所有键的顺序。插入、分裂和合并都依赖这个相邻关系，不能只把“新节点当根”当成任意接树的许可。

伸展树允许某次查找走过一条长度 n 的链，因此单次最坏是线性。它的保证针对一串操作：标准实现的总账给出摊还对数界，并计入任意初始树的势能。后面会把这一证明展开。

13.2 B 树：先固定容量，再画分裂与借位

二叉树一层只有一次分叉，查一次键可能读取许多分散的页面。B 树的一页保存多个排好序的键，用这些分隔键决定下一页。比如节点[20,40]有三个孩子，依次保存小于 20、20 与 40 之间、大于 40 的键。键数为 r 时，恰好切成 r+1 个范围。

本节数值例固定采用最小度数 t=2：一个非根节点至少 1 键、至多 3 键；内部节点有 2 至 4 个孩子；所有叶子深度相同。根非空时至少 1 键，根叶可以是整棵树。一般 t 的范围为 t−1 至 2t−1 键，孩子数为 t 至 2t。课程 2026 教材还用(a,b)-树记最少和最多分支数，本例就是(2,4)-树。不能把“阶 4”“t=2”和“最少 2 键”混用：此处最少是 2 个孩子、1 个键。

13.2.1 插入 1 到 10：每个分裂发生在哪一步

采用向下走之前先分裂满孩子的版本。它在进入孩子前保证孩子还剩容量，父亲也因此不会在回程中突然溢出。教师课件另展示“先插入、溢出再向上分裂”，触发时点不同；下面图与伪代码统一按前一种执行，不拿两套中间状态相互判断对错。



图 103　B 树先在根叶依次插入 1、2、3

按图内从左到右、再到下一行的顺序阅读。插 1、2、3 都只在根叶有序放入，成为[1,2,3]。插 4 之前，根已经满。把中间 2 移到新根；小于 2 的[1]成为左叶，大于 2 的[3]成为右叶。随后 4 走右侧，右叶成为[3,4]。所有旧叶一起下降一层，叶深仍一致。

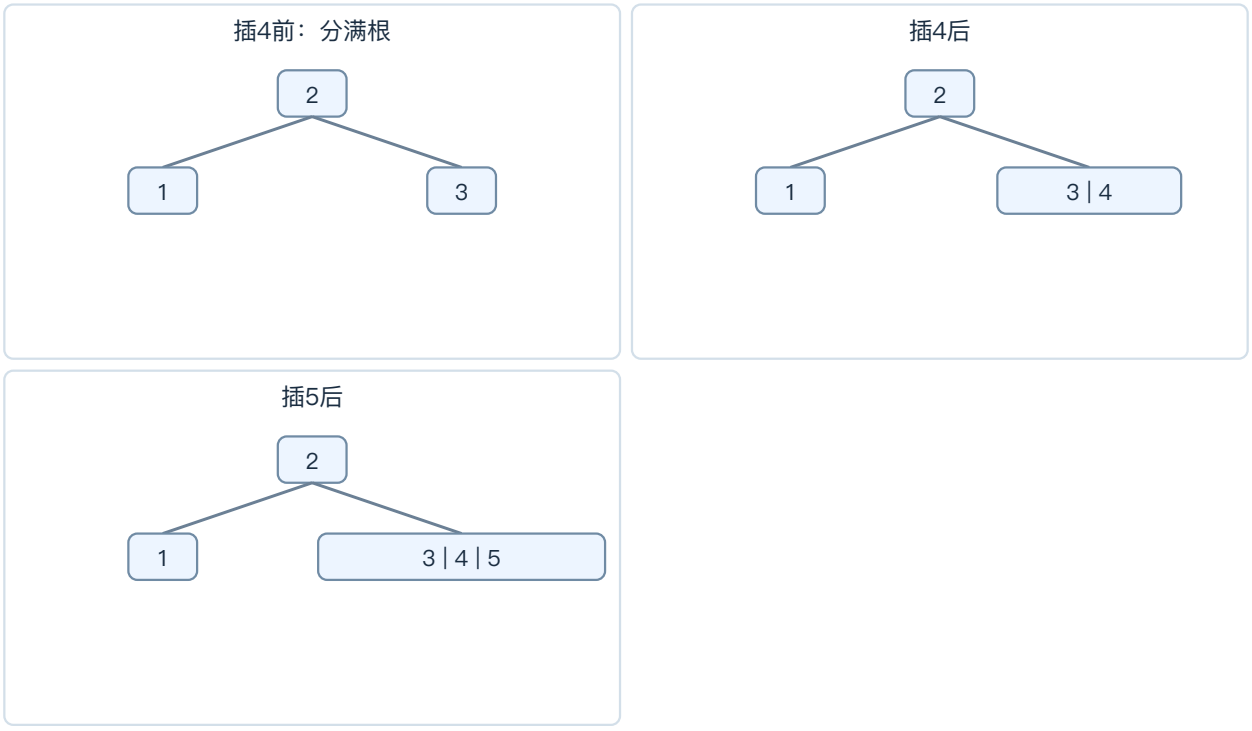


图 104　根分裂后插入 4 和 5

插 5 到右叶，得到[3,4,5]。插 6 前，读根[2]发现该往右走，但右孩子已满，于是中键 4 升到父亲，原叶拆成[3]与[5]，父亲成为[2,4]。现在再次和新分隔键 4 比较，6>4，去最后一片叶，变成[5,6]。中键上升以后必须重新选孩子，不能沿用分裂前的孩子编号。

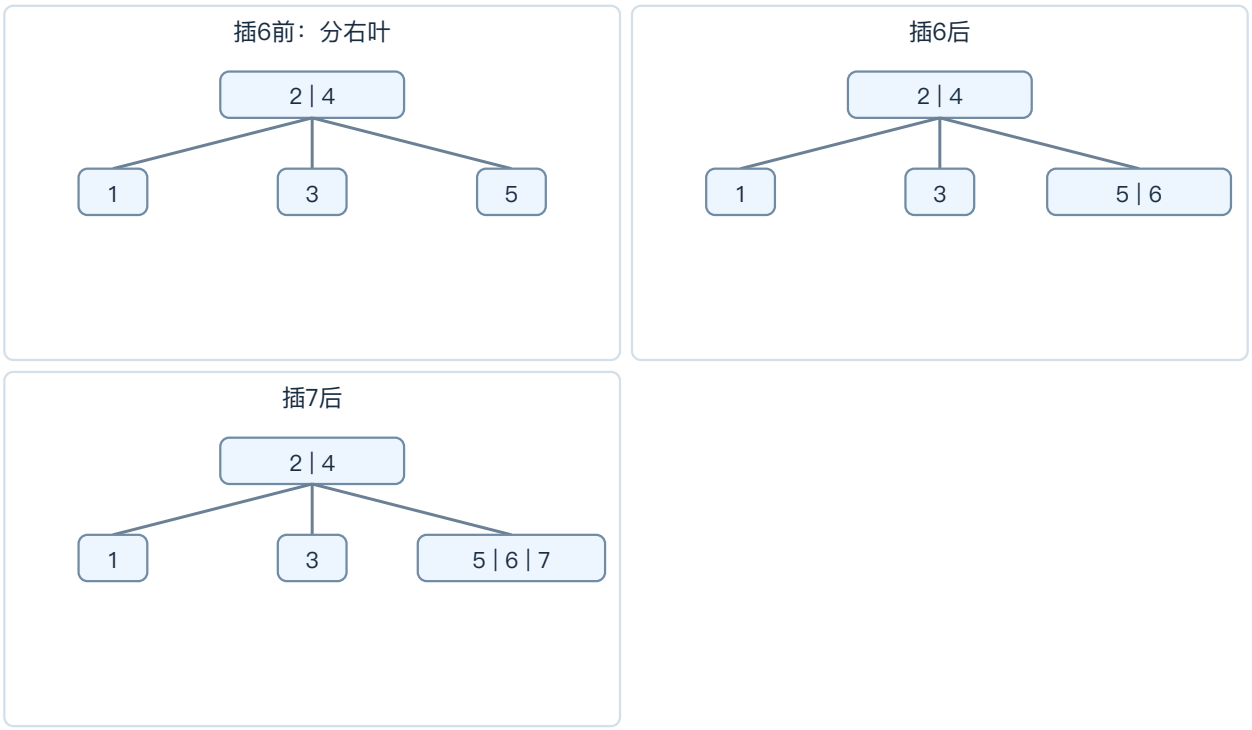


图 105　插 6 之前的叶分裂，随后插 6 和 7

插 7 后末叶[5,6,7]。插 8 前分它，中键 6 上升，根成为[2,4,6]，叶依次[1]、[3]、[5]、[7]；再把 8 放入[7,8]。插 9 前根满，先把 4 升为新根。左内部节点[2]接[1]/[3]，右内部节点[6]接[5]/[7,8]。此时旧根的四个孩子按顺序分两组，不能只分键、不分孩子指针。9 进入[7,8,9]。

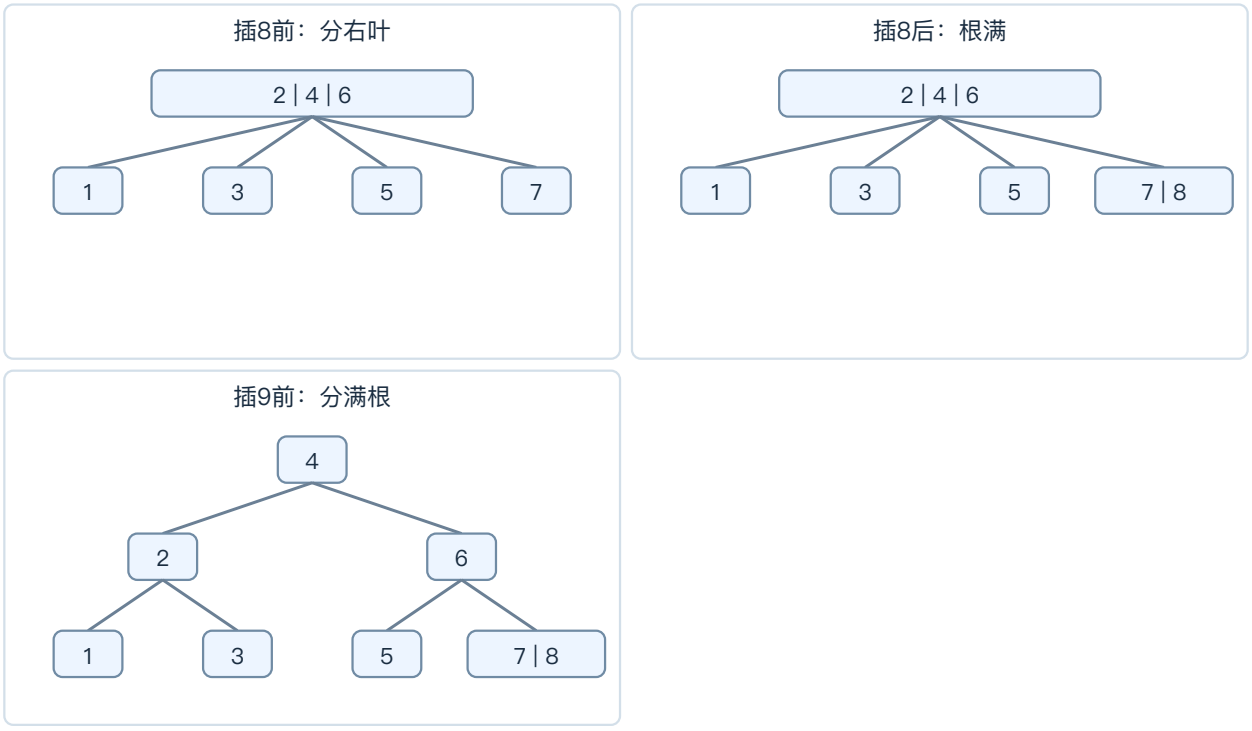


图 106　插 8 前分叶与插 9 前分根

插 10 时，新根[4]不满，右孩子[6]也不满，继续准备进入[7,8,9]才发现叶满。把 8 上交右内部节点，变成[6,8]，对应叶[5]、[7]、[9]，最后 10 进入[9,10]。树顶没有变化。这说明分裂可能发生在不同深度，只有根分裂会增加整棵树高度。

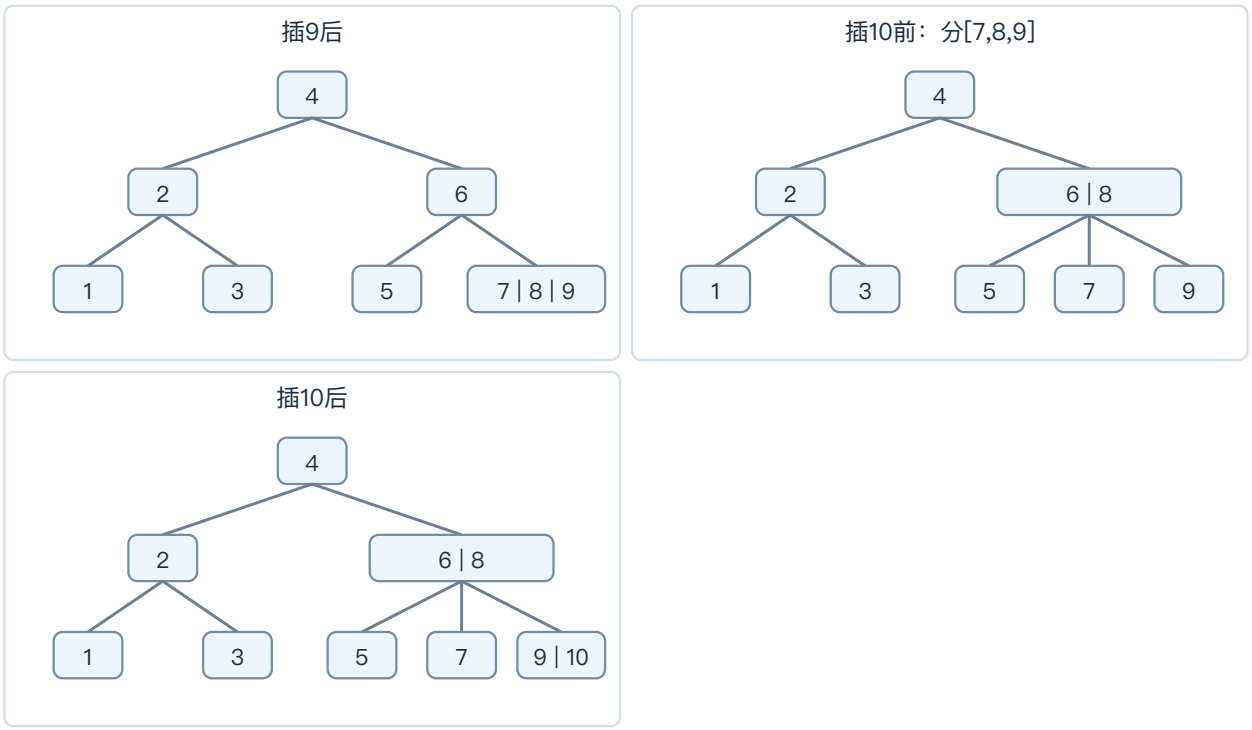


图 107　插 9 和 10 时较深一层的分裂

一般满节点有 2t−1 个键。分裂取第 t 个键上交父亲，左右各留 t−1 键；若它是内部节点，2t 个孩子也各分 t 个。左边所有键小于中键，右边所有键大于中键；两组孩子维持原来的间隙顺序，所以局部查找范围不变。

13.2.2 删除前先准备容量：左借、右借与合并

删除叶键很容易，困难是删后不能少于 1 键。本节使用“下降前先让目标孩子至少有 t 个键”的版本。因此若某个孩子仅有 1 键，先修复容量再进入它。以下借位与合并例彼此独立，都从各自第一幅合法树开始。

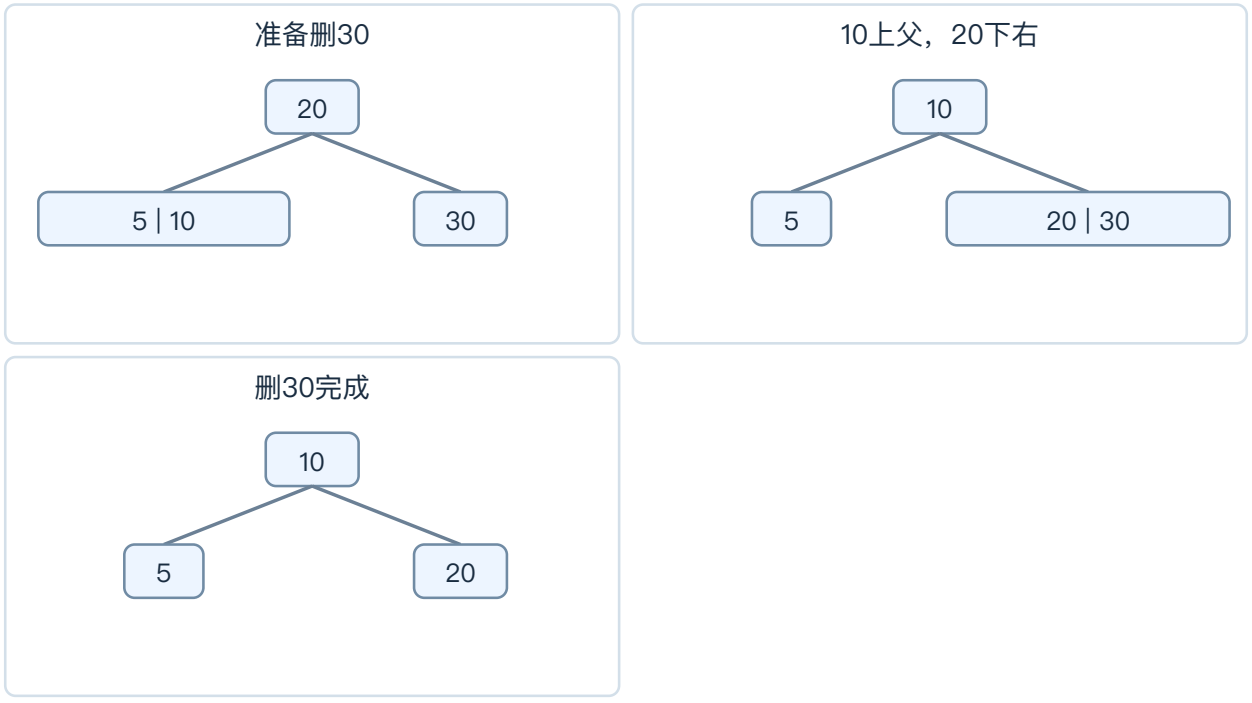


图 108　B 树从左兄弟借键并删除 30 的每一步

先看从左兄弟借的例子：要删右叶 30。父[20]，左兄弟[5,10]有多余键。父 20 下降到右孩子开头，左兄弟最大 10 上升到父亲，得到父[10]、左[5]、右[20,30]。现在才删除 30，右叶剩[20]。键没有直接从左叶穿到右叶；它经过父分隔键轮换。若把 10 直接移入右叶而父仍 20，右叶出现 10<20，查找范围会立即错误。

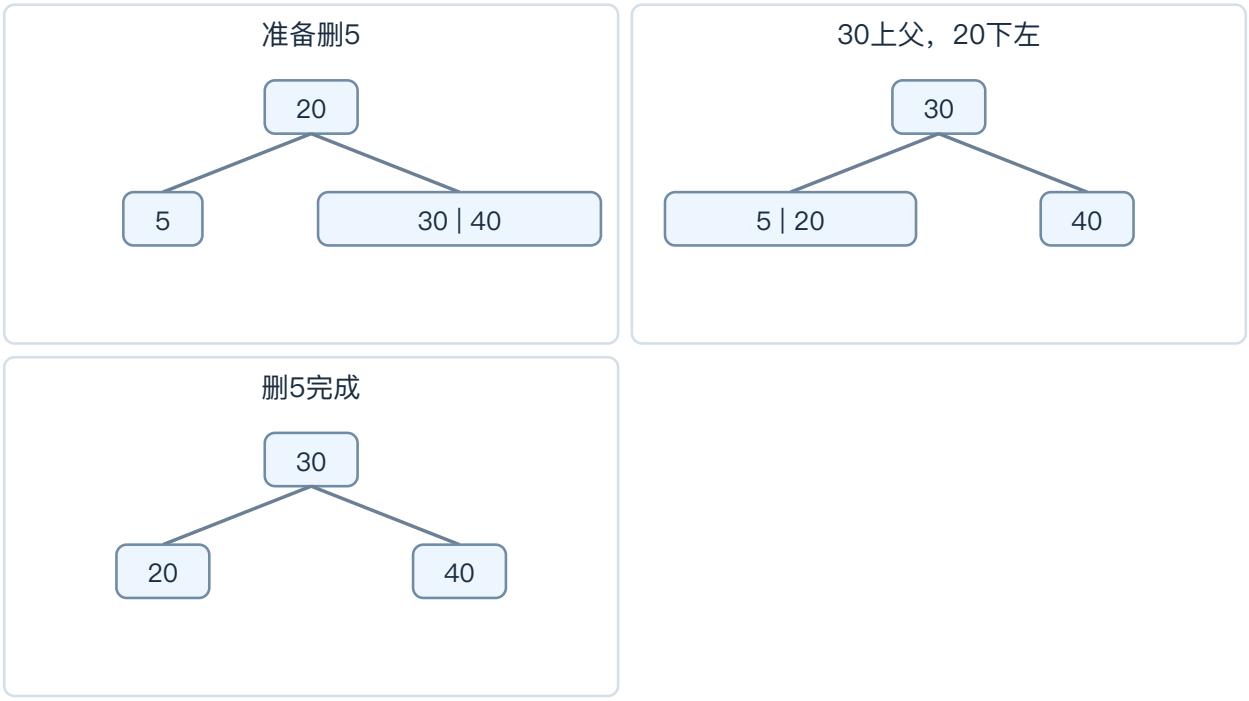


图 109　B 树从右兄弟借键并删除 5 的每一步

下一例要删左叶 5。父[20]，右兄弟[30,40]可借。父 20 下降到左叶尾部，右兄弟最小 30 上升，得到父[30]、左[5,20]、右[40]。再删 5，左叶剩 20。这与第一行镜像，借键方向改变时，下降的位置从开头改成末尾。

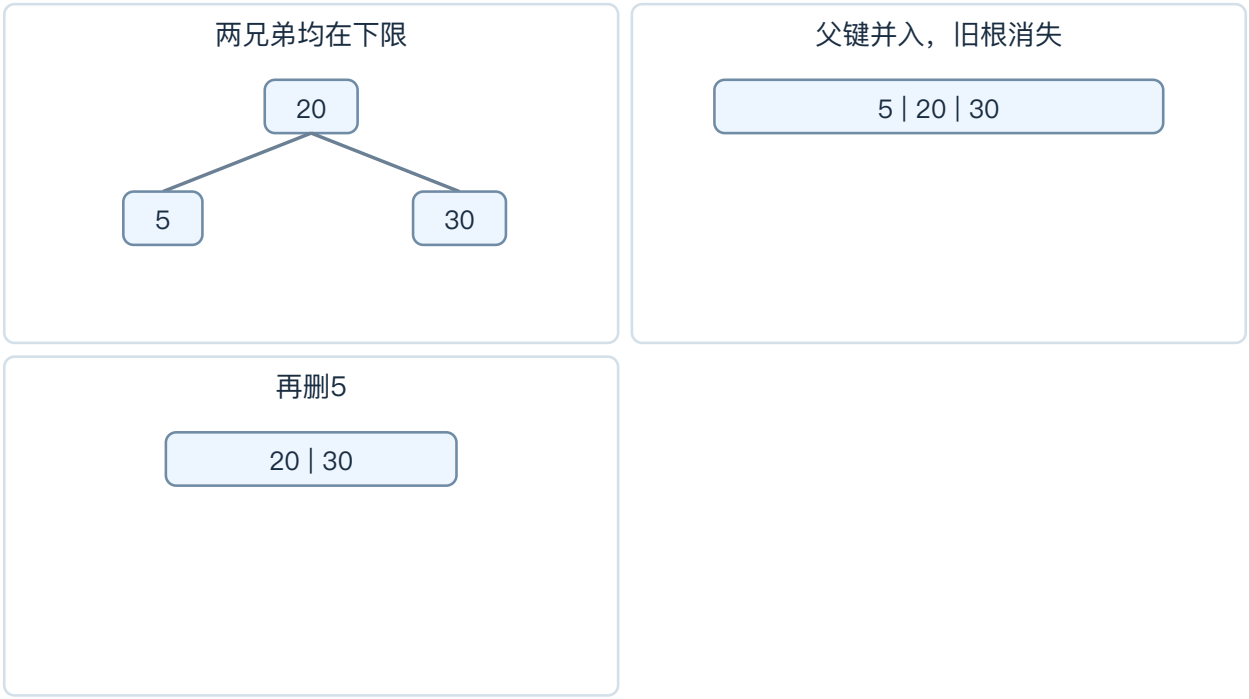


图 110　B 树兄弟与父键合并、根下降，再删除 5

合并例中父[20]，两叶[5]、[30]都在下限，谁也不能借。把左叶、父 20、右叶依次合成[5,20,30]。父失去唯一键与一个孩子链接，成为空根，于是合并页直接接任新根。再删 5，根叶[20,30]。所有叶一同升一层，叶深仍相同。非根不能无条件变空；下降前准备容量正是为了避免回程出现没有预案的下溢。

13.2.3 借的是分隔键，连着子树也要移动

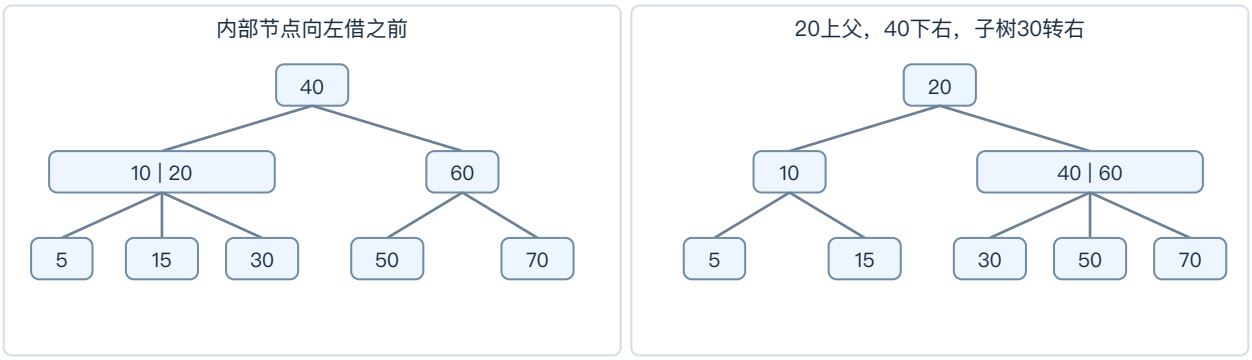


图 111　内部 B 树借位时子树 30 随范围转移

左图父[40]；左孩子[10,20]带三棵叶[5]/[15]/[30]，右孩子[60]带[50]/[70]。右孩子向左借：20 上父、40 下降成为右孩子第一个键。原来在 20 与 40 之间的整棵[30]子树，现在应位于新右孩子[40,60]的最左边，因为 30<40。因此它要从左孩子末尾转到右孩子开头。

调整后父[20]，左内部节点[10]带[5]/[15]，右内部节点[40,60]带[30]/[50]/[70]。逐个节点检查“孩子数=键数+1”和分隔范围，就能找到漏搬孩子指针的错误。内部节点合并时也同样需要拼接两侧全部孩子，数量从 t+t 变为 2t。

13.2.4 删除内部键：先确定哪一侧够用

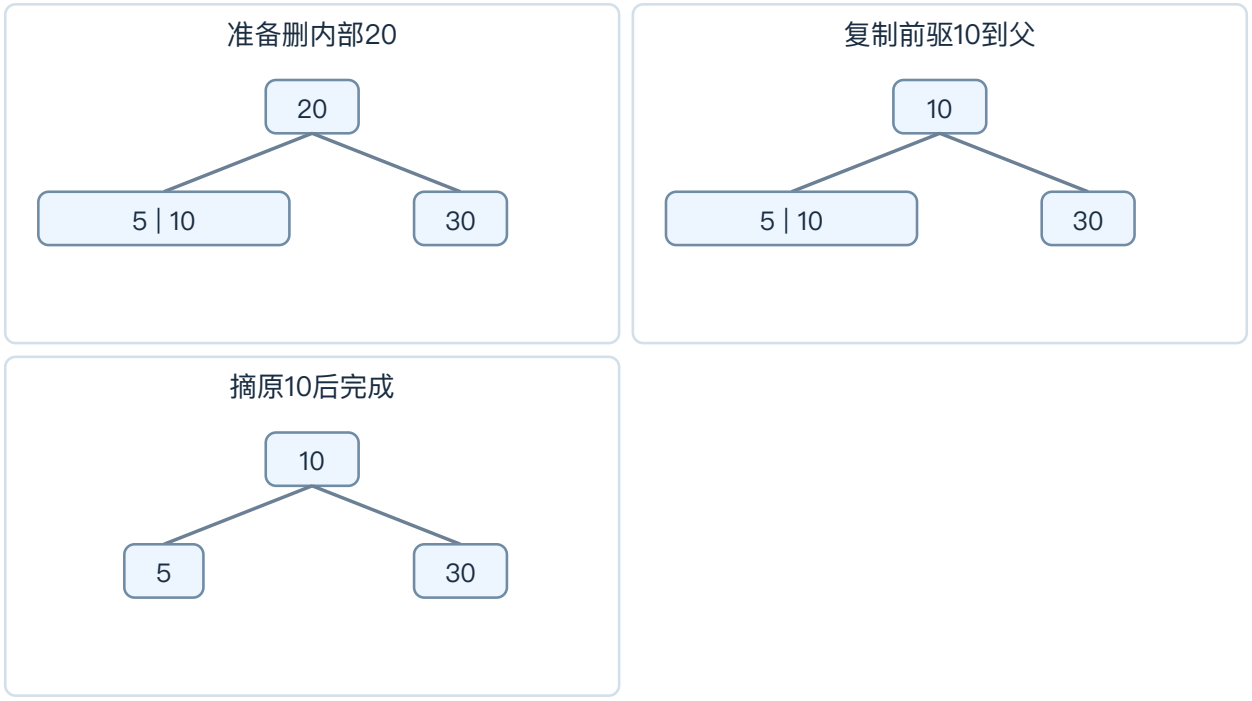


图 112　B 树删除内部键时先复制前驱、再摘原位置

这个例子删父 20：左孩子[5,10]有至少 t 键，可以用前驱 10 替换 20，再递归删除左孩子中的原 10。中间父和孩子暂时都显示 10，这只是“复制记录后尚未摘除原节点”的算法状态；完成后父[10]，左右叶[5]/[30]，键只保存一份。整条记录一起复制，不能把键 10 配上原来 20 的值。

如果左边仅[5]，右边[30,40]，改用后继 30：先替换父，再在右叶删原 30，最终父[30]、左右[5]/[40]。若两边都仅 1 键，例如父[20]、叶[5]/[30]，先合并为[5,20,30]，然后在合并页删 20，得到[5,30]。这三种条件覆盖内部删除的所有分支。

查找、插入、删除每次只沿一条根叶路径。现在把高度界按层数出来：n 是整树实际保存的键数，t≥2 是最小度数，h 按边数计算，从根到叶经过 h 条边。非空根至少 1 键；若 h≥1，根至少有 2 个孩子；其他内部节点至少 t 个孩子。所以深度 1 至少 2 个节点，深度 2 至少 2t 个，深度 i 至少有$2t^{i-1}$个节点。每个非根节点至少 t−1 键，因此

$$n \ge 1 + 2(t-1)\sum_{i=0}^{h-1} t^i.$$

括号里的和是 1+t+...+t 的 h−1 次方。令这串和为 S，tS 减 S 后中间项抵消，得到(t−1)S=t 的 h 次方−1，所以

$$n \ge 1 + 2(t-1)\frac{t^h - 1}{t-1} = 2t^h - 1.$$

移项得$t^h \le (n+1)/2$。因为 t>1，取以 t 为底的对数保持不等号方向，得到

$$h \le \log_t \frac{n+1}{2}.$$

h=0 时求和为空，根叶至少 1 键，仍满足这条式子；空树 n=0 另外处理。一次查找最多读 h+1 页，常写 O(log_t n)，小规模时可写 O(1+log_t(n+1))避免对数为零的歧义。每层常数次分裂、借位或合并，每次读写常数页，因此插入、删除的页访问数也由同一高度界控制。页内移动最多 O(t)个键/指针，CPU 时间需另计 O(t log_t n)的基本上界；不能把页操作和单键移动混成同一种成本。

13.3 红黑树：先知道颜色约束，再数黑节点

红黑树仍满足 BST 顺序，另外有以下约束：根黑；每个空孩子 NIL 视为黑；红节点的孩子黑；从同一节点到它任何后代 NIL 的路径，黑节点数相同。图中节点同时标 B 或 R，分别是黑或红。下一张图先画全黑色 NIL，后面的中间图才按同一约定省略它们。

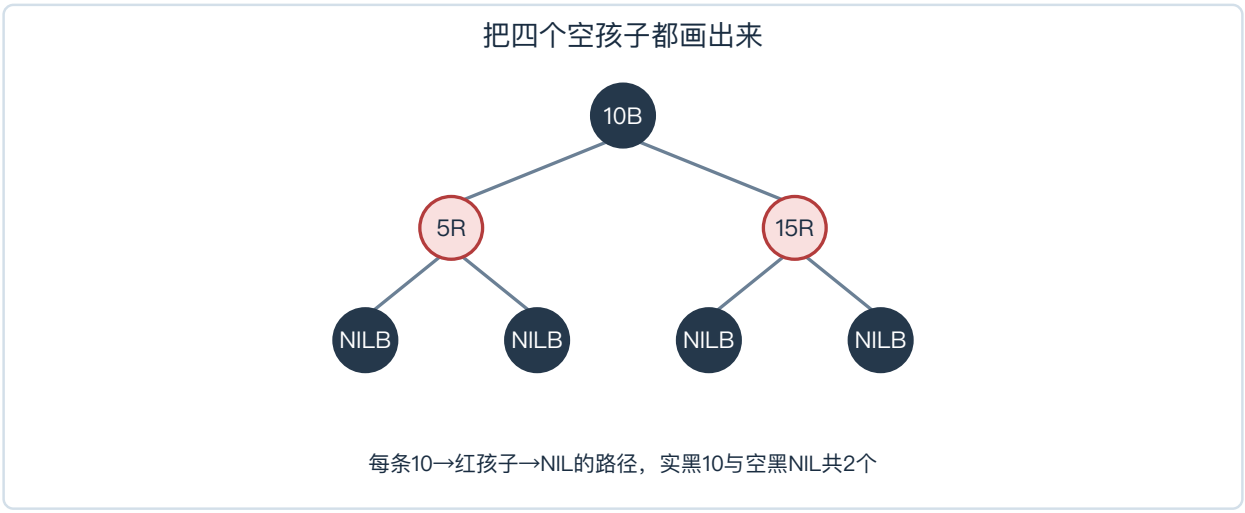


图 113　红黑树的四个黑色 NIL 与完整路径

NIL 表示一条空孩子链接，不是一个额外保存的键。5 的左、右链接为空，各画一个 NIL；15 也同样有两个，共四个。每一条路径都从根出发，一直走到其中一个 NIL 才结束：10B→5R→左 NIL，10B→5R→右 NIL，10B→15R→左 NIL，10B→15R→右 NIL。每条都数到根 10 一个黑、末尾 NIL 一个黑，共 2 个；5/15 红色不计。以后中间图省略这些不变的 NIL 时，按这里补齐再数，缺口位置 x*则始终画出。

把上图红孩子与黑父收成一个多键节点，可以对应 2-3-4 树。红红冲突类似一个多键节点放不下；删除黑节点后的缺口类似多键页少了分隔键。

13.3.1 插入先涂红：黑高不变，可能出现红红

新叶涂红，原来通向 NIL 的路径只是多经过一个红节点，黑数不变。父亲黑便完成；父亲红则违反“红节点的孩子黑”，要继续修复。以下例子把叔叔定义为父亲的兄弟，不能把目标的兄弟认成叔叔。



图 114　从空树插 10、5、15 的完整状态

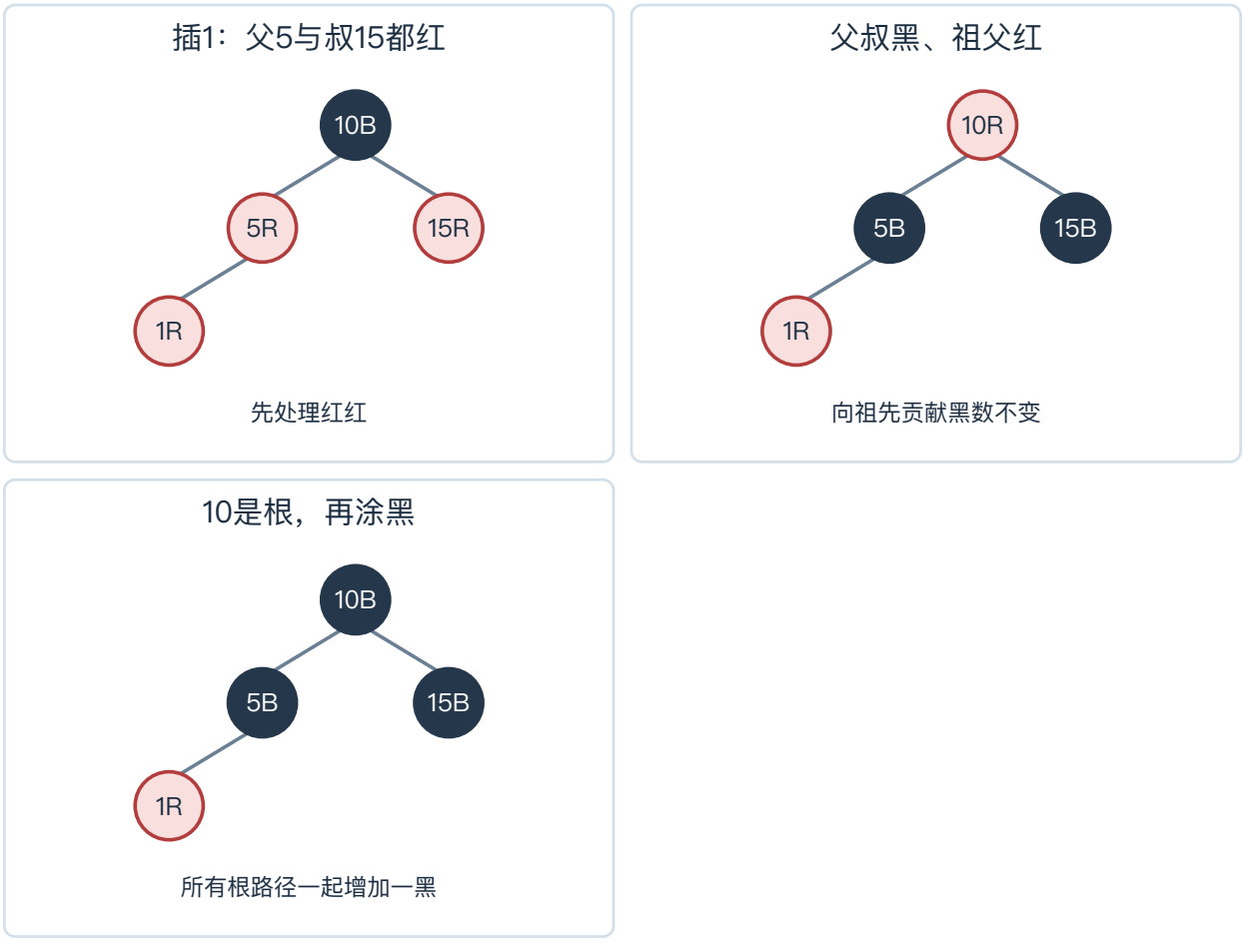


图 115　红叔情况下插 1、变色、根再转黑

这两组图连续插 10、5、15、1。10 是根，直接涂黑；5、15 以红孩子加入合法。1 加入 5 左边时，父 5 红，叔 15 也红。把 5、15 涂黑，把祖父 10 涂红：从 10 往下的各条路径都因孩子转黑多一黑，又因 10 转红少一黑，向更高祖先贡献不变。若 10 还有红父亲，冲突就转移到 10 与其父之间，循环上移。此例 10 已是根，再涂黑，所有根路径同时多一黑，合法结束。

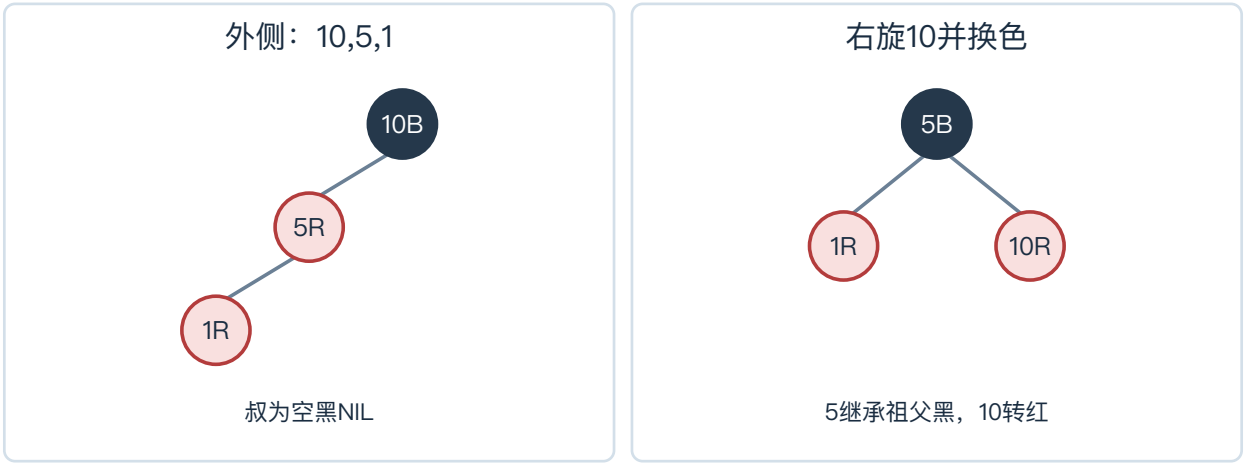


图 116　黑叔外侧插入通过一次旋转和变色完成

接着看外侧例 10、5、1，叔叔为空黑。把 5 升到 10 原位置，对 10 右旋；5 继承祖父原来的黑色，10 涂红，1 保持红。结果根 5B、左 1R、右 10R。原出口路径黑数保持，红红也消失。右右方向则左旋祖父。

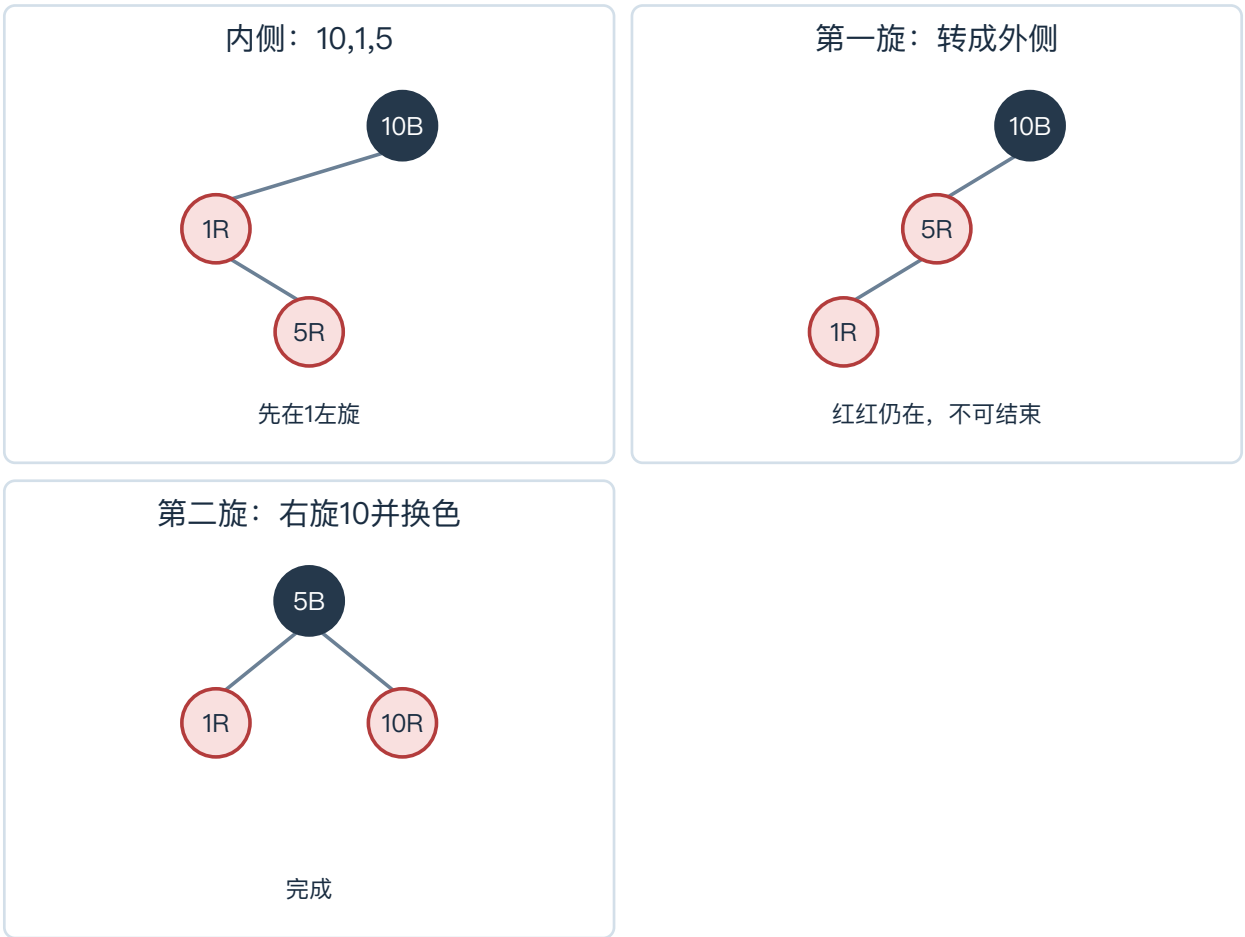


图 117　黑叔内侧插入的两次旋转和中间红红

内侧例 10、1、5，父 1 红、目标 5 红、叔空黑。第一旋在 1 左旋，让 5 成为 10 的红左孩子，1 成为 5 的红左孩子。至此红红还在，只是转成外侧；第二旋在 10 右旋并换色，才得到 5B 与两红孩子 1/10。左右镜像得到右左例。四种路径方向可以按图操作，不需要仅靠名字记住最后形状。

插入可能沿祖先做多次变色，但旋转修复以后局部黑高恢复、红红消失，故旋转次数是常数；沿高度走一遍为 O(log n)。

13.3.2 删除先按 BST 摘节点，再检查被摘走的颜色

两个孩子的节点仍先用前驱/后继记录替换，把真正摘除的位置移到至多一个孩子的节点。决定修复的是这个物理摘除节点的颜色，而非最初指定键所在位置的颜色。

摘红节点不会少黑；摘黑节点而替代孩子是红，将替代孩子涂黑就补齐。余下情况是摘黑、由黑孩子或空 NIL 接替。将替代位置记为 x，并临时给它记一份“还欠一黑”的标记，图中写 x*。这不是第三种节点颜色。它表示若额外补一黑，整棵树才恢复各出口黑数一致。

下面统一 x 在父 p 左侧，兄弟 w 在右侧。近侄是 w.left，远侄是 w.right。x 在右时，方向全部镜像。每次旋转后必须重新确定父、兄弟和两个侄子。

13.3.3 删除情形一：红兄弟先转成黑兄弟

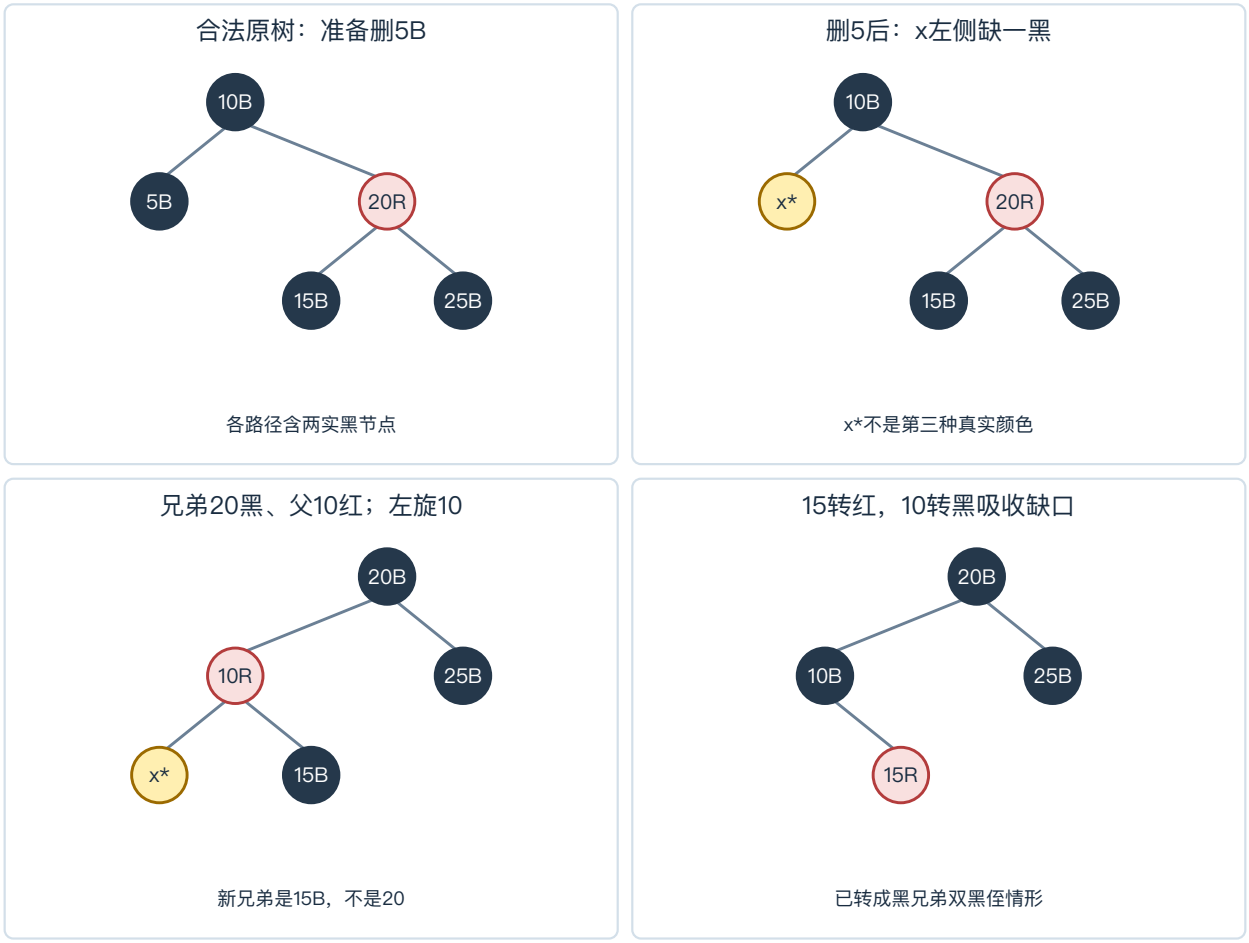


图 118　红兄弟删除情形的原树、缺口、旋转与后续吸收

原树根 10B、左 5B、右 20R，20 带 15B/25B。删 5 后 x*在 10 左侧，兄弟 20 红。红兄弟的两个孩子必黑，父 10 也必黑；否则原树已经有红红。

将 20 涂黑、10 涂红，再在 10 左旋。新局部根 20B，左 10R 且 10 右 15B，右 25B；x*还在 10 左侧。缺口没被旋转凭空填上，新兄弟改成 15B。然后进入下一类：15 的两个孩子都是 NIL 黑，把 15 涂红，将欠账推到 10。10 原为红，涂黑正好吸收欠账。最终 20B 左 10B（右 15R）、右 25B。图最后一幅展示的是完成这一步后的状态，不能在第一旋后就停止。

13.3.4 删除情形二：黑兄弟与两个黑侄，把缺口向上移

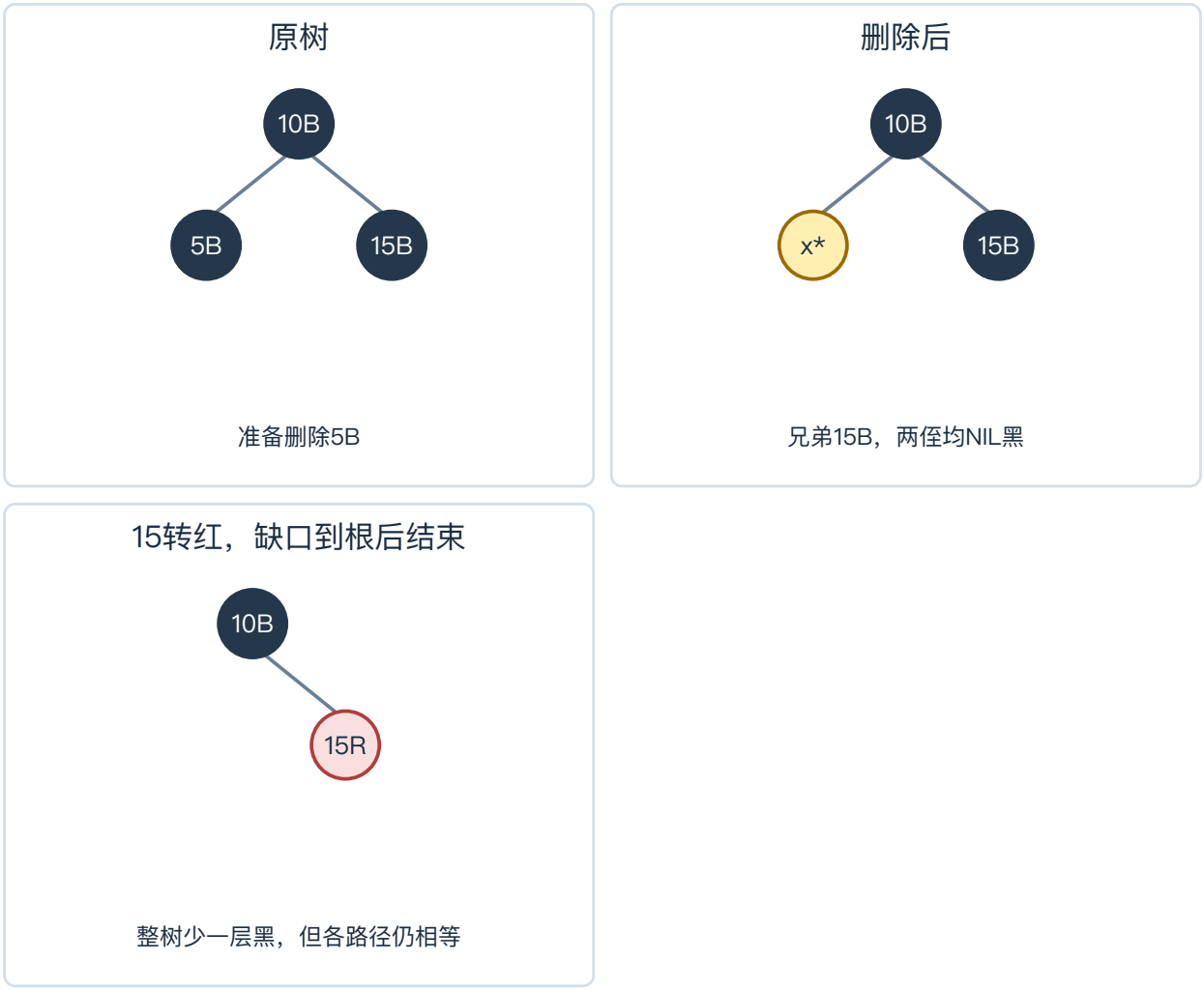


图 119　黑兄弟两个黑侄的缺口上移

原树 10B 带 5B/15B。删 5 后，右边 15B 方向比左边多一黑。将 15 变红，就让右方向也少一黑，父 10 下方两侧重新相等；但整棵 10 子树相对原先仍少一黑，所以欠账移到 10。10 是根，全树所有出口同时少一黑，直接结束。

若 p 是红，将 p 涂黑即可吸收；若 p 黑且不是根，就把 x 改为 p，继续向上检查它的新兄弟。这里是可能重复 O(log n)层的分支。把 w 转红后若忘记上移或吸收，局部看起来齐了，相对祖先仍然缺一黑。

13.3.5 删除情形三：近侄红、远侄黑，先转换方向

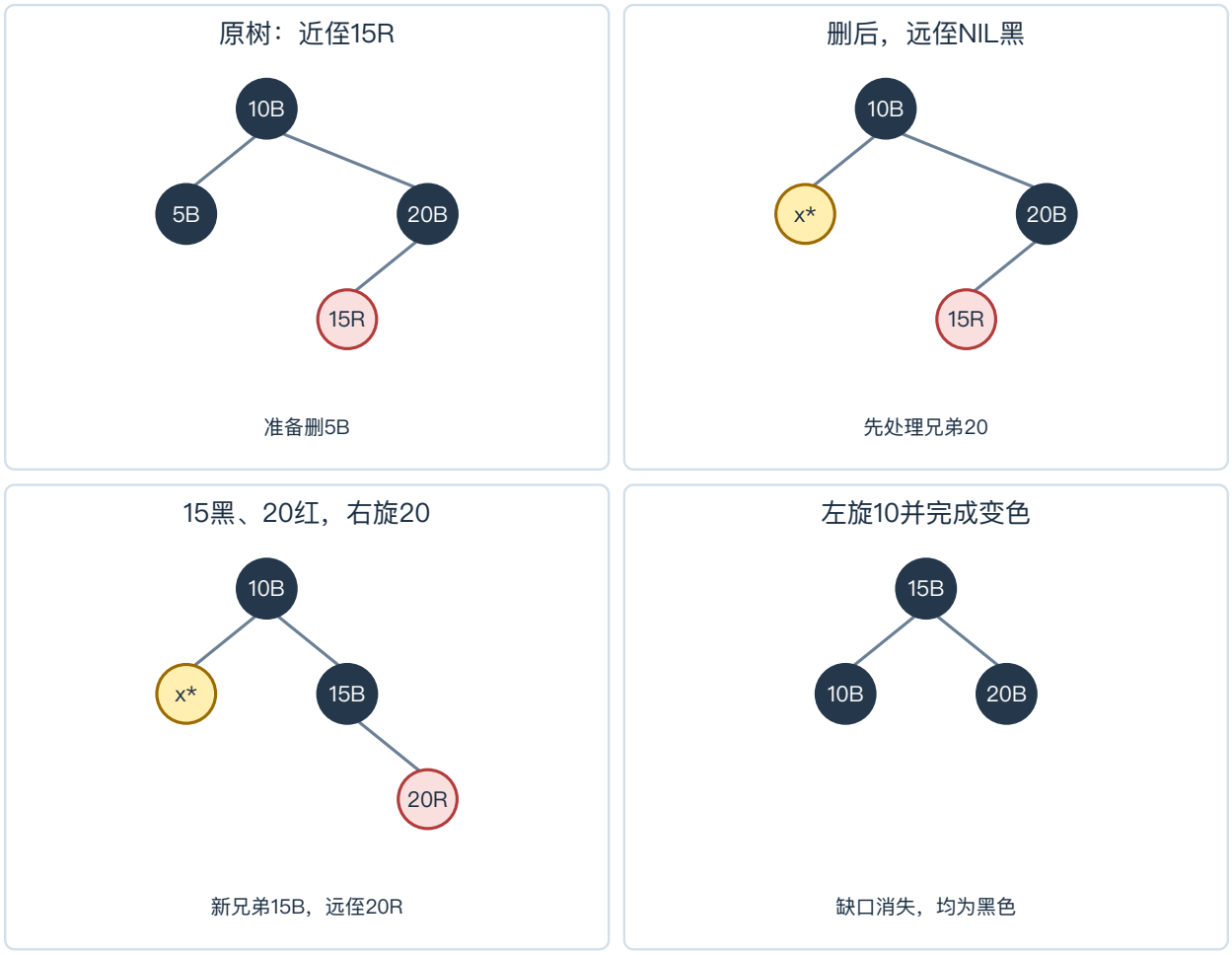


图 120　近红远黑先在兄弟处旋转，再进入远红情形

原树 10B 左 5B，右 20B 且 20 左 15R。删 5，兄弟 20 黑，近侄 15 红，远侄 NIL 黑。先把 15 涂黑、20 涂红，对 20 右旋。10 的右孩子改为 15B，15 右 20R；x*仍未改变。现在兄弟是 15、远侄是 20，已经转为下一类。这次旋转保持兄弟方向对父的黑数贡献。它的目的在于把一层可借的红节点移到远侧，不是立即解决欠账。下一步在父 10 处左旋并完成变色，得到 15B 带 10B/20B，欠账消除。

13.3.6 删除情形四：远侄红，一次旋转补齐

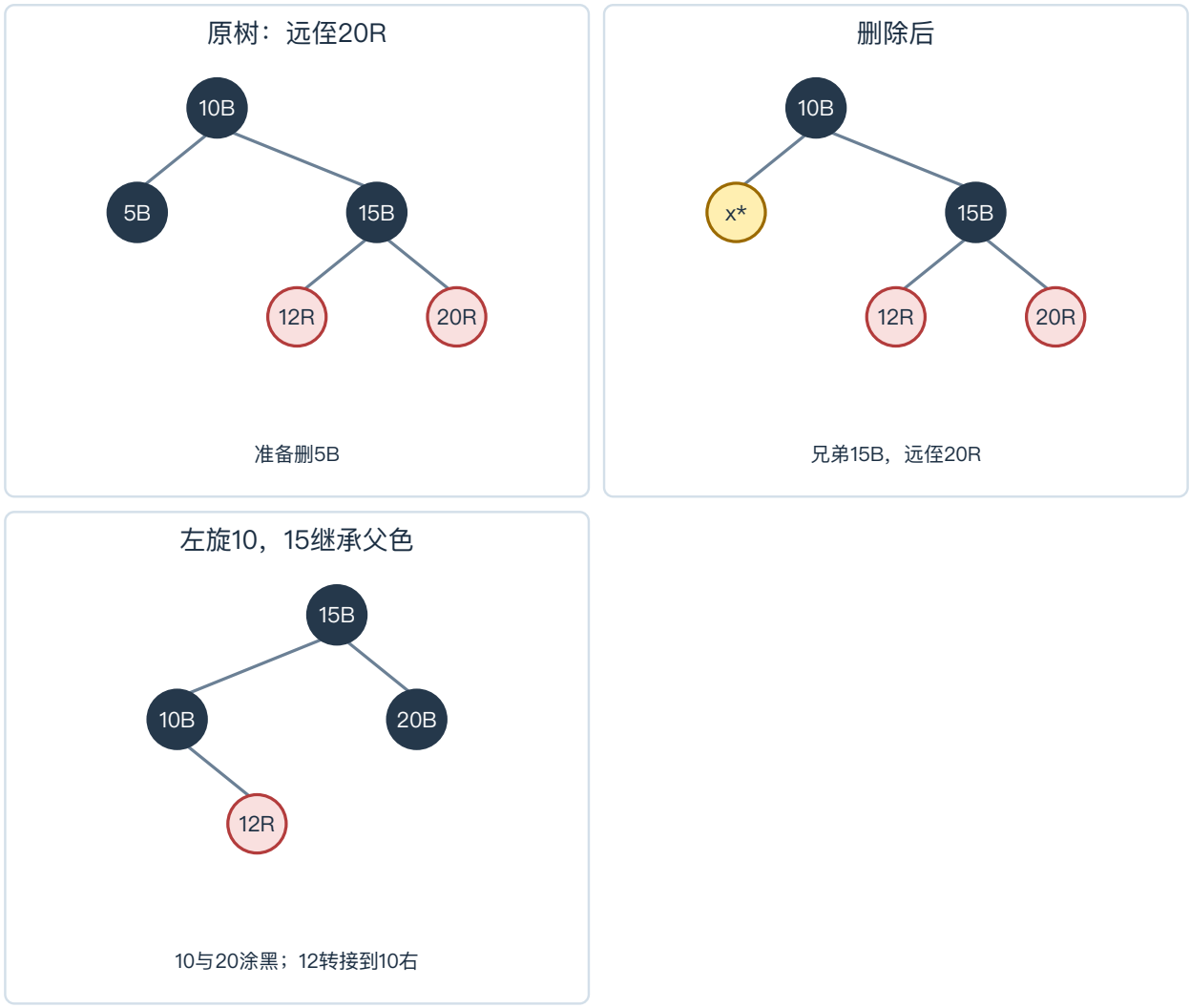


图 121　远侄红时父、兄弟、远侄的变色和中间子树转移

原树 10B 左 5B，右 15B 带 12R/20R。删 5 以后，兄弟 15 黑、远侄 20 红。让 15 继承父 10 原颜色，10 涂黑，20 涂黑，再对 10 左旋。12 在 10 与 15 之间，从 15 左边转接成 10 右边。最终 15B 左 10B（右 12R）、右 20B。

逐路数：15 到左空经过 15、10 两个实黑；15 到 12 再到空也经过 15、10 两个实黑；15 到右侧空经过 15、20 两个实黑，已相等。父 p 若原先红，新根 w 要继承红，才能保持这棵局部树向祖先贡献的黑数不变，不能无条件把新根涂黑。近侄可以红也可以黑，对这一步不构成额外条件。

四类判断可以写成一条执行路径：红兄弟先转换；之后兄弟黑，若两侄黑则上移，否则若远侄黑先把近红转成远红，最后远红终结。教材把“黑兄弟有红孩子”的重构合成 BB-1，将父红/父黑的合并分成 BB-2R/BB-2B，编号与这里四分法不同，树的动作与黑高账对应。

合法红黑树为何不会退成链？定义 b 为从某节点往下、不计该节点而计末尾 NIL 的黑高度。若子树黑高为 b，它的每个孩子黑高至少 b−1，因为向孩子走一步最多消耗一层黑。令 N(b)表示这种黑高至少需要的实节点数，N(0)=0，便有

$$N(b) \geq 1 + 2N(b-1).$$

从下往上代：b=1 至少 1 个实节点，b=2 至少 1+2=3 个，b=3 至少 1+2×3=7 个；归纳得到$N(b) \geq 2^b - 1$。实际 n≥N(b)，所以$2^b \leq n+1$，取对数得$b \leq \log_2(n+1)$。

沿一条根到 NIL 的路，红节点不能相邻；每遇一个红节点，它的下一层必黑。因此最长实节点路径的边数 h 不超过 2b。代入上式，$h \leq 2\log_2(n+1)$。这一步把颜色规则转成了实际路径长度，而不只是在说黑节点少。查找走一条这样的路径，插入与删除沿它修复，每层只做常数次检查，故单次最坏 O(log n)；伸展树则给摊还界，保证的对象不同。

Princeton 的平衡搜索树说明使用左倾红黑树，其中红链接还要求倾向左侧；官方 RedBlackBST 实现的旋转、颜色翻转要连同该附加约束一起读。不要把它的少量代码分支与本节一般红黑树删除四类逐行混用。

13.4 跳转表：沿每层找前驱，留下 update 数组

底层是一条保存全部键的有序链表，上层只保存一部分键。一个键出现的各层称一座塔。本节将底层编号 0，每个节点独立抛公平硬币：先有底层，正面就再升一层，直到第一次反面。高度 2 表示出现在第 0、1 层。另有每层头尾哨兵−∞、+∞，它们不算用户键。

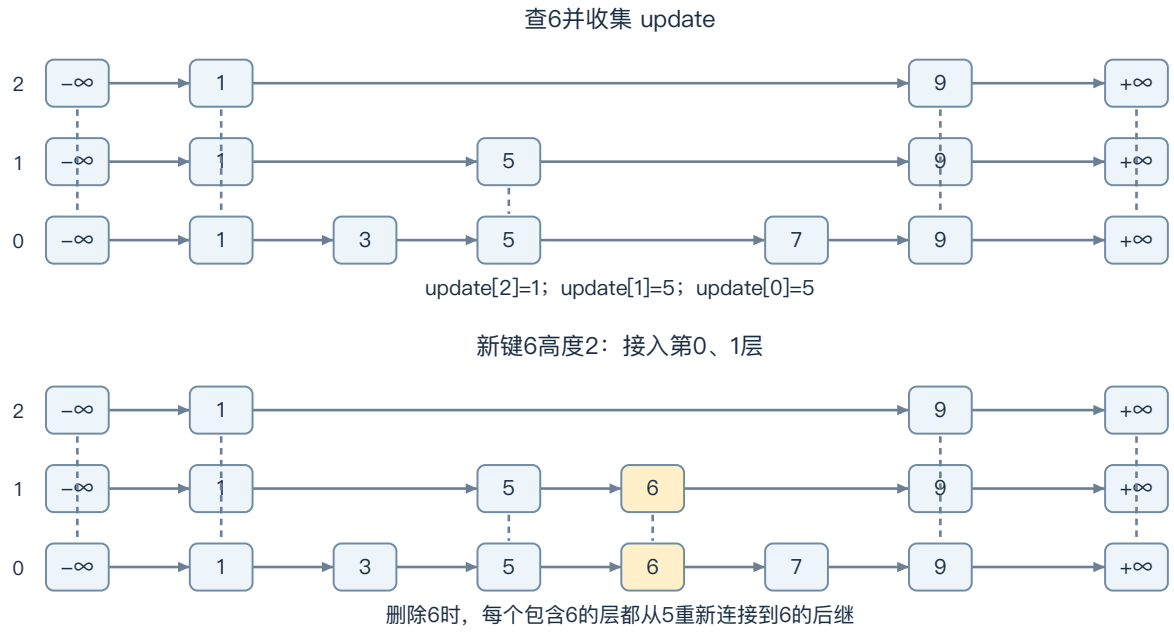


图 122　跳转表查询 6 时的前驱数组与插入后的两条新链接

初始第 0 层 1、3、5、7、9；第 1 层 1、5、9；第 2 层 1、9。查 6 时用“下一键严格小于 6 便向右，否则下降”的规则：

当前层与节点	看到的下一键	动作	离开该层时留下什么
2 层，−∞	1<6	右移到 1	继续看下一键
2 层，1	9>6	不能右移，下降	update[2]=1
1 层，1	5<6	右移到 5	继续看下一键
1 层，5	9>6	下降	update[1]=5
0 层，5	7>6	停止下降	update[0]=5

底层前驱是 5，它的后继 7 不等于 6，故查找失败。插 6 时已经知道每层应该改哪条链接，不必再从头搜索。假设这次抛币得到高度 2，则第 0 层做 `6.next[0]=5.next[0]`（指向 7），再做 `5.next[0]=6`；第 1 层同样先让 6 指向 9，再让 5 指向 6。第 2 层不出现 6，保持 1 直达 9。

先保存后继再改前驱，才能不丢原链。若相反先覆盖 5 的 next，再读它，会把 6.next 误设成 6，形成自环。若随机高度超过当前最高层，先为新层建头尾哨兵，并把该层 update 填为头哨兵，再接新塔。

随后查 7：2 层−∞到 1 后降；1 层 1 到 5 再到 6，看到 9 太大便降；0 层 6 的后继 7 正好命中。再删 6，复用查 6 得到的 update：第 0 层把 5 直接接 7，第 1 层把 5 直接接 9，第 2 层本来无 6 不动。只有确实指向 6 的层才断链。若最高层删得只剩哨兵，就回收该空层。

为什么期望空间是线性？每个键达到第 j 层的概率为 2 的−j 次方，期望层数为 1+1/2+1/4+…=2，n 个键期望约 2n 份塔节点/前向链接。为什么期望查找对数？高度约 $\log_2 n$；把搜索路径逆向看，每次在一层向左寻找可升高的塔，独立硬币使平均检查次数为常数，乘以层数得到 O(log n)。这些结论依赖独立升层，不承诺每个具体随机结果都好；所有键恰好只有底层时，找最后一项仍需线性时间。

13.5 对照原资料继续核对

2026 教材第 8 章（本地 PDF198—237 页附近）分别给出伸展、B 树、红黑、跳转表。老师《08.高级搜索树.pdf》PDF9—10 页有两种双旋，54—55 页有上溢分裂，62—65 页有借位和合并，100—106 页有双黑修复。阅读原页时先核对它的左右方向、容量符号和临时状态，再与本章图逐链接对照。页脚编号在课件分节后重新开始，以上均为 PDF 查看器页码。

13.6 伸展树的势能为什么选子树大小的对数

令 s(v)为节点 v 子树大小，rank r(v)=$\log_2 s(v)$，势能 Φ 为所有节点 rank 之和。子树大小反映一次访问后有多少节点被放到某节点下面；取对数使“规模翻倍”变成“rank 增加 1”，便于将一长串旋转的成本与树形变化相抵。

单次伸展的访问引理给出：实际旋转步数加势能变化，不超过常数倍 r(根)−r(x)再加常数，常见表述为 3 倍加 1。因为根子树大小为 n，x 大小至少 1，rank 差至多 $\log_2 n$，于是得到摊还 O(log n)。下面将 zig、zig-zig、zig-zag 中改变 rank 的节点逐项列出，验证这个界。

这也解释了为什么只做单旋提根不能随便复用同一引理：成对旋转的局部结构决定势能怎样下降。某次长路径花很多旋转，但它改变许多子树大小，使势能下降足以抵消大部分实际成本。若初始势能不为 0，总账要保留 Φ 初这一项；n 个节点每项 rank≤log n，初始势能上界为 n log n。

用三节点链 30 左 20 左 10 看账：原子树大小分别 3、2、1，Φ=$\log_2 3+1$；伸展 10 成反向链后大小仍 3、2、1，只是归属换了，势能总和相同，这次两旋仍需要实际支付 2，不是每次都能靠势能下降变便宜。摊还界容许常数或对数支出，并不宣称每次 Φ 都会降。

13.7 访问引理的三种局部账，逐式看常数怎样消掉

下面把一次旋转成本计 1，r 为旋转前 rank，r′为旋转后 rank，δ=r′(x)−r(x)≥0。只有参与旋转的 x、父 p、祖父 g 的 rank 可能变化，其他节点的子树元素集合不变。对数有一条常用不等式：若 a+b≤c，则 $\log_2 a+\log_2 b\le 2\log_2 c-2$，这是 $ab\le((a+b)/2)^2$得到的。

zig 只有 x、p 变化，p 子树缩小，因此摊还成本 A=1+[r′(x)−r(x)]+[r′(p)−r(p)]≤1+δ。每次完整伸展最多在最后发生一次 zig，所以这一个加 1 不会被每层重复累积。

zig-zag 后 x 占据原 g 整棵子树，故 r′(x)=r(g)。实际两旋加势能变化，约去这两项，得 A=2+r′(p)+r′(g)−r(p)−r(x)。旧 p 包含旧 x，所以 r(p)≥r(x)，得 A≤2+r′(p)+r′(g)−2r(x)。新 p 与新 g 为新 x 两棵不相交子树，大小和小于新 x 大小，故 r′(p)+r′(g)≤2r′(x)−2。代入得 A≤2δ≤3δ。

zig-zig 也先约去 r′(x)=r(g)，得到同样的 A 式。再用 r(p)≥r(x)、r′(p)≤r′(x)，得到 A≤2+r′(g)+r′(x)−2r(x)。这次旧 x 子树与新 g 子树在新 x 内不相交，二者大小和不超过新 x 大小，所以 r(x)+r′(g)≤2r′(x)−2。代入即 A≤3r′(x)−3r(x)=3δ。

将一连串双旋及至多一个单旋的 A 相加，节点 x 在本步后的 rank 正是下一步前的 rank，中间项抵消。最终 x 成为根，rank=$\log_2 n$，得到整次伸展摊还成本≤1+3($\log_2 n$−原 r(x))。这证明了旋转阶段的对数界；查找路径长度与相应伸展步数成常数倍关系，所以查找也纳入这份界。插入、删除还需考虑新节点加入、旧节点移除及分裂合并带来的势能变化，标准实现可据同一大小与 rank 关系继续得到摊还对数总界。

原书习题 8-2 在官方合体 PDF580 到 582 页使用同类势能分析。读总账时保留初始势能：Σ 实际=Σ 摊还+Φ 初−Φ 末。非负势能只保证实际≤摊还+Φ 初，不能省去任意初始树的 Φ 初就说实际总不超过摊还总。

13.8 练习与答案

练习 1。 伸展树单次查找最坏 Θ(n)，是否与摊还 O(log n)矛盾？

解答：不矛盾。摊还约束一串操作的总成本，并允许某次很贵。访问深节点后结构发生改变，后续操作可能便宜；分析还要计入初始树势能。如果只拿一棵精心构造的深树做一次访问，再忽略构造与初始势能，就无法用它否定标准总账结论。

练习 2。 最小度数 t=3 的 B 树，非根节点键数范围是多少？为什么孩子数比键数多 1？

解答：键数 2 到 5，内部孩子数 3 到 6。r 个有序分隔键把数轴切成 r+1 个间隙：最左、相邻键之间以及最右，因此对应 r+1 个子树范围。叶子没有实际孩子，但容量限制仍适用。根可少于 2 键，例如仅 1 键 2 孩子。

练习 3。 跳转表中所有节点恰好只在底层，查找最后一个键要多久？红黑树会出现同样的 n 层链吗？
解答：跳表要走过近 n 个节点，$\Theta(n)$，这是一种可能的最坏随机结果。合法红黑树不能形成任意长单链，因为黑高一致和无连续红节点会被破坏；其高度始终 $O(\log n)$。一个给期望界，一个给最坏界，选择时要看是否需要严格延迟上限。

14 搜索一段范围，为什么不能只照搬找一个键

预约系统保存许多时间区间，要问“下午 3 点哪些预约正在进行”；数组需要反复求某段总和；地图要找矩形框里的点。这些任务都在查询范围，但数据对象和输出不同：区间可能互相覆盖，数组位置固定，平面点有两个坐标。因此要先区分问题，再选择区间树、线段树、范围树或 kd 树。

本章默认区间端点规则会明确给出。闭区间[1,3]包含 3，半开区间[1,3)不含 3。两段是否相交、边界点属于哪边，都依赖这个约定。

14.1 例 1：区间相交，先把判定写出来

闭区间[a,b]与[c,d]相交，当且仅当 `max(a,c) <= min(b,d)`，也等价于 `a<=d and c<=b`。例如[1,4]与[4,6]在 4 相交。半开区间若长度为正，相交应改成严格小于；[1,4)与[4,6)不相交。

“穿刺查询”就是给一个点，报告所有包含它的区间。“范围报告”中的报告表示把每个答案实际写出来，若有 k 个答案，输出本身至少需要 k 步。

对区间[1,3]、[2,8]、[6,9]、[10,12]，查询点 7，答案是[2,8]与[6,9]。朴素扫描全部 n 段，成本 $\Theta(n)$。若查询很多次且区间集固定，值得预处理。

一种区间搜索树按左端点建 BST，每个节点再存子树最大右端点 maxEnd。查找与查询区间[qL,qR]相交的任意一段时：当前节点若相交便返回；否则若左子树 maxEnd<qL，左边所有区间都在查询左侧结束，可整棵跳过；若左侧仍可能相交，先去左边。与平衡 BST 结合，找一个相交区间可 O(log n)。

把这四段具体接成根[6,9]，左孩子[2,8]且它左孩子[1,3]，右孩子[10,12]。各自 maxEnd 依次是 12、8、3、12。查[4,5]：根[6,9]不相交，但左 maxEnd=8≥4，进入左孩子，[2,8]相交，返回。查[9.5,9.8]：根不相交，左 maxEnd=8<9.5，跳过左边；右孩子[10,12]的左端 10>9.8，也不相交，沿可用链接走到空，报告不存在。每次比较保留了具体排除理由。

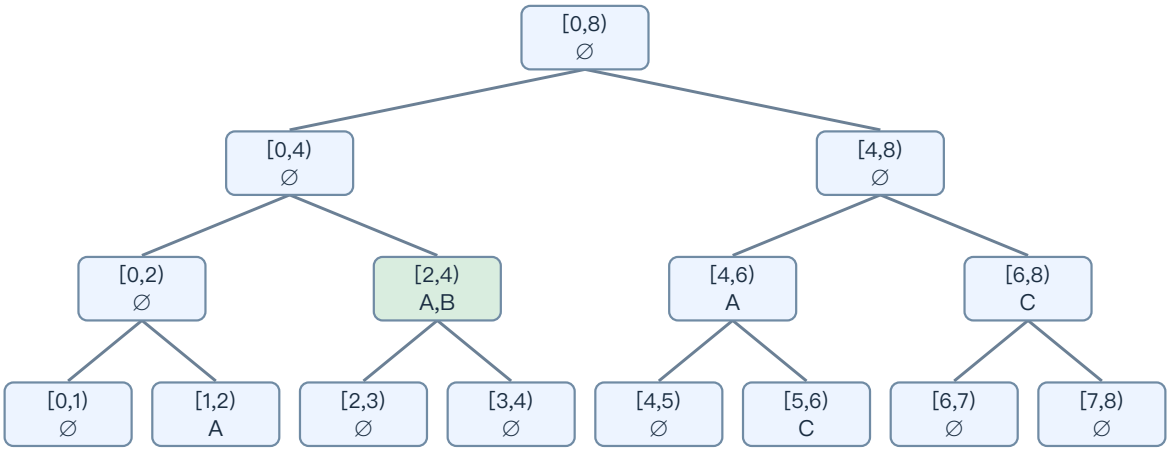
为什么左 maxEnd≥qL 时可以只走左侧找一个？若左侧最终没有相交，达到 maxEnd 的区间虽右端够大，它的左端必大于 qR；而左子树左端不大于根的左端，因此根和右边的左端也都大于 qR，右边同样无解。这个论证依赖 BST 按区间左端排序；不能把 maxEnd 当成无需顺序条件的剪枝咒语。

这里“找一个”与“报告全部”不同。全部输出 k 项至少要 $\Omega(k)$时间；遍历剪枝算法的界还取决于具体增广结构。不能将“有一个 maxEnd”自动当作所有报告查询均 O(log n+k)的充分证明。

课程里的中心分割区间树是另一种结构：选中心点 c，跨过 c 的区间留在当前节点，左边完全小于 c 的递归进左树，右边完全大于 c 的进右树；当前节点的跨中心区间分别按左端升序、右端降序保存。查询 q<c 时，只需在左端有序表中报告左端≤q 者，随后走左子树；q>c 时对称。合适的平衡构造能达到对数定位加输出成本，读课件时注意它与 maxEnd 增广 BST 不是完全同一种树。

14.2 区间报告线段树：把一段预约拆到少数完整节点

老师课件中的线段树先解决区间穿刺，而下面数组求和是同一分段骨架的另一种用途。先用半开区间统一边界：A=[1,6)、B=[2,4)、C=[5,8)，询问给定 q 落在哪些区间里。将端点和需要的分界排好，把根范围[0,8)二分至单位段。真实连续坐标可以先将端点排序后用相邻端点构成基本区间，不要求输入坐标本身是小整数。



A=[1,6)，B=[2,4)，C=[5,8)；节点下行保存完整覆盖它的区间编号

图 123　区间报告线段树中每个输入区间存到哪些节点

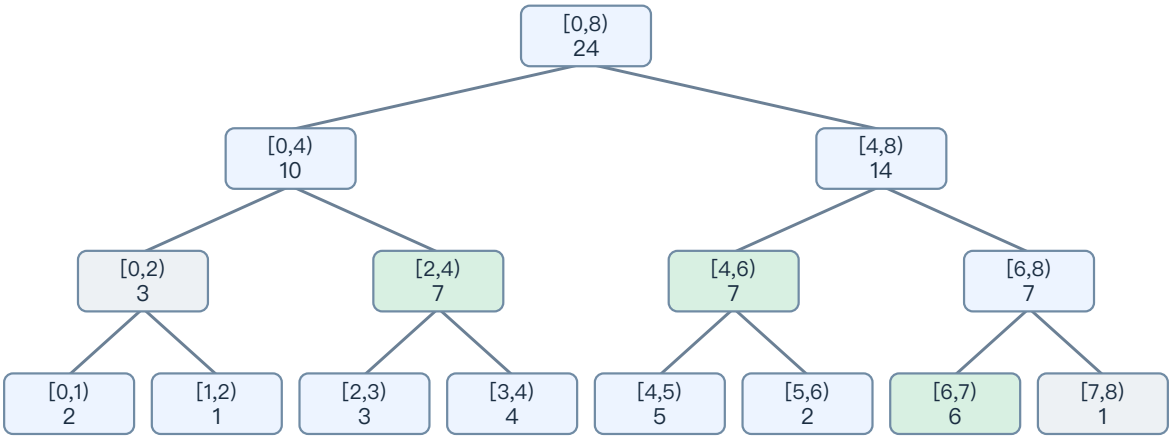
插 A 时，从根出发：[0,8)只是部分覆盖，拆两侧；[0,4)仍部分，分到[1,2)与[2,4)；[4,8)仍部分，取完整[4,6)。于是只在[1,2)、[2,4)、[4,6)三处存 A，放好后不再向这些节点后代重复存。B 恰好覆盖[2,4)，只在该处加 B。C=[5,8)拆成[5,6)、[6,8)，分别存 C。图中的∅表示这里没有直接存区间编号，不表示这棵子树没有数据。

查 q=3.5，沿包含它的唯一子区间下降：[0,8)→[0,4)→[2,4)→[3,4)。依次收集各节点列表，前两处空，[2,4)输出 A/B，最后为空，答案 A、B。查 q=5.5，路径[0,8)→[4,8)→[4,6)→[5,6)，在[4,6)输出 A、在[5,6)输出 C，答案 A、C。每个区间拆成互不重叠的完整块，单个点最多落入其中一块，所以同一区间不会沿一次路径输出多次。

单个连续区间在每层至多留下左右两条尚需细分的边界路径，中间完整块遇到即停止，因而每个输入区间占 O(log n)个列表位置；n 个区间总空间 O(n log n)。查询只走一条高度 O(log n)路径，再输出 k 个实际答案，耗时 O(log n+k)。这里节点保存区间编号列表；下一节节点保存数组和，不能把这两种载荷的数据结构接口直接混用。

14.3 例 2：线段树，每个节点保存一段数组的摘要

数组`[2,1,3,4,5,2,6,1]`，要查询半开区间[2,7)的和，并支持单点修改。线段树根代表[0,8)，左右孩子代表[0,4)、[4,8)，继续二分直到单点。节点存其区间和。



绿：本次查询直接取这一段的和；灰：与查询范围不相交

图 124　线段树查询下标 2 至 7 的半开区间：整块取值与两侧排除

图中每个节点上行是下标半开区间，下行是该段总和。绿色节点[2,4)、[4,6)、[6,7)互不重叠，恰好拼成查询区间；灰色块完全不属于查询。先看区间再看数字，不能把节点标签的 2 或 7 当数组元素值。

查询[2,7)时，根与查询部分重叠，继续分解。左子树[0,4)中，[0,2)不相交，跳过；[2,4)完全包含在查询中，直接取 3+4=7。右子树[4,8)中，[4,6)完全包含，取 5+2=7；[6,8)继续分，取[6,7)的 6，跳过[7,8)。答案 7+7+6=20。完全覆盖时为何可以不继续？节点保存的摘要已经等于该整段的答案，继续访问叶子只会重复工作。不相交时为何可跳过？这段没有任何位置属于查询，因此贡献为加法单位元 0。

单点把下标 3 的 4 改成 10，只需要更新叶[3,4)及它到根的祖先。[2,4)从 7 变 13，[0,4)从 10 变 16，根从 24 变 30。每层只改一个节点，高度 O(log n)，因此单点更新 O(log n)。构建时每个节点只合并一次，总节点 O(n)、总时间 O(n)。

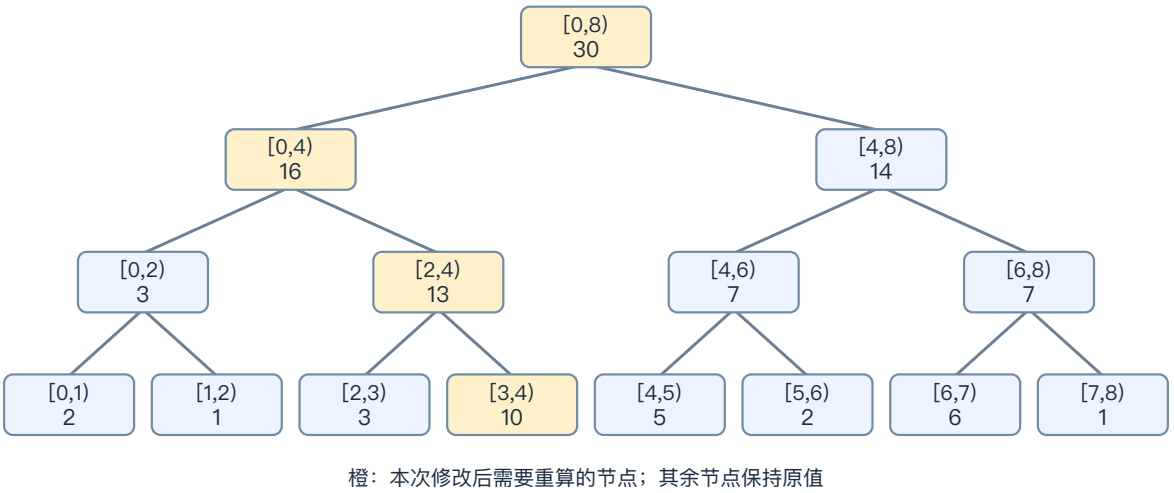


图 125　单点 3 改为 10 后只有一条祖先链发生变化

更新图的橙色链依次是[3,4)、[2,4)、[0,4)、[0,8)。新叶 10 写入后向上合并为 13、16、30；其余节点仍保存原来的和。

求和可以换成最小值、最大值、最大连续区间四元组等，只要能从左右摘要合成父摘要。组合顺序可能重要，例如字符串拼接不可交换，但只要按左后右顺序合并仍可使用。查询空区间时要给出适当单位元，最小值查询通常用正无穷，而不是 0。

14.4 区间修改，懒标记究竟懒了什么

若把[0,4)所有元素加 3，逐叶更新要做 4 次；更一般宽区间可达 Θ(n)。节点已知自己覆盖长度 len，整段加 d 会让 sum 增加 d×len。可在这个节点记一个 lazy=d，表示“该加法已计入本节点和，但尚未传给孩子”。以后若查询或修改只覆盖它的一部分，就必须先将 lazy 下推：两个孩子 sum 分别增加 d×各自长度，各自 lazy 加 d，再把父 lazy 清零。若整段仍被完全覆盖，可以直接使用父 sum，不必展开。lazy 保存的是尚欠孩子的动作，不能重复给父 sum 加一次。

在长度 8 数组的根上加 2，根 sum 增加 16 并记 lazy2；随后查[0,4)，需要先将左右孩子各加 8 并记 lazy2，父 lazy 清零，再返回左 sum。下推之后根总和不应再改变，它早已算过这一笔。



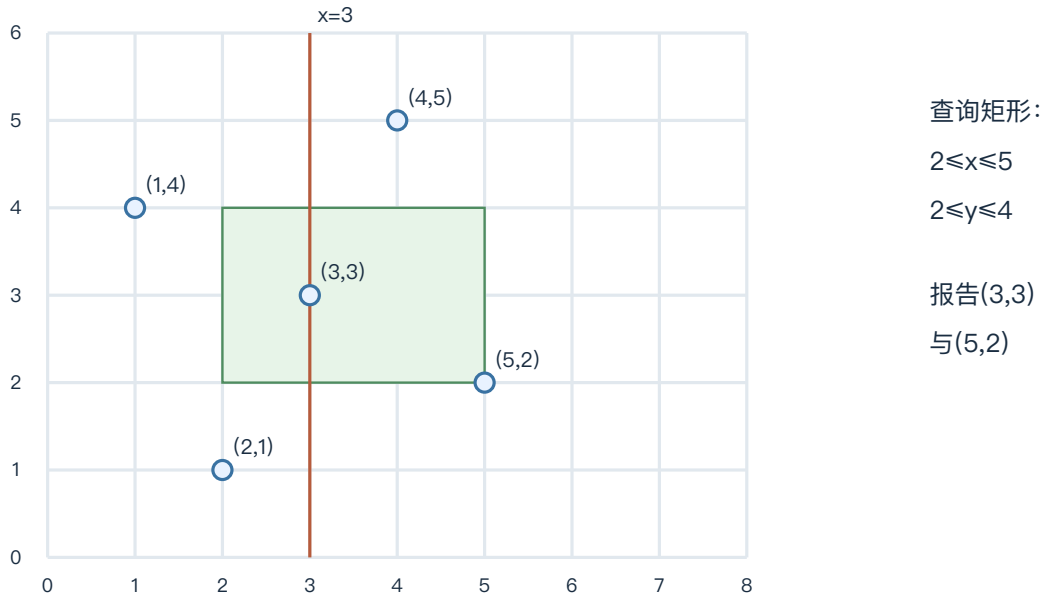
三列依次为根[0,8)、左[0,4)、右[4,8)；tag是尚欠孩子的加法

图 126　整段加 2 和第一次部分查询时懒标记的转移

图中第二行孩子还写旧 sum，因为根的 tag=2 记着这笔欠账；第三行父 tag 清零，左右分别记 tag=2。根 sum 始终 40，下推没有给整个数组再加一遍。之后若只改下标 1，就继续把左孩子 tag 下推，直到触及需要分开的范围。

加法标记可相加，赋值标记却会覆盖旧赋值；赋值与加法混合时先后顺序不能交换。单个旧值 2，先赋 5 再加 3 得 8；先加 3 再赋 5 得 5。同一段的标签若先记“加 3”又收到“赋 5”，新赋值会抹掉旧加法，而反序要保留“赋 5 后加 3”。若题目要两种操作，标记需表示一个明确的函数组合，不能只多放两个变量而不定义它们的优先关系。

14.5 例 3：二维矩形查询，范围树利用两层顺序



绿色边界也属于闭矩形；点(2,1)和(4,5)在y范围之外

图 127　闭矩形查询与五个输入点的位置

点为(1,4)、(2,1)、(3,3)、(4,5)、(5,2)，查询 x 在[2,5]、y 在[2,4]内的点，答案(3,3)、(5,2)。只按 x 排序，可先二分出 x 范围，再逐项看 y，最坏仍扫很多项。

二维范围树先按 x 建立平衡树，每个 x 子树再存一份该子树点按 y 排序的辅助结构。查询 x 区间可拆成 $O(\log n)$棵完整子树，对每棵辅助表二分 y 区间并输出。基础版本预处理空间 $O(n \log n)$，查询 $O(\log^2 n+k)$，k 为输出数量。分数级联可继续改进二分开销，但需要额外链接及证明，不是把公式里的平方直接删去。

对这五个点按 x 建根 3、左组 1/2、右组 4/5。查询 x∈[2,5]可拆成单点 x2、根点 x3、右整组{x4,x5}。三块对应的 y 表分别[1]、[3]、[2,5]。在 y∈[2,4]内，第一块空，第二块输出(3,3)，第三块在[2,5]中二分出仅 y2，输出(5,2)。右整组不必再沿 x 树拆成两叶；辅助 y 表就是在此发挥作用。

14.5.1 分散级联：在下一张表中少做一次完整二分

多个完整 x 子树各有一张 y 表，基础版本对每张表都二分，才有 $\log^2 n$。分散级联（也叫分数级联）在表之间补导航副本与指针，让一次定位的信息传给下一张表。先用三张小表看它具体省在哪里。

原表 A=[1,3,5,8]，B=[2,4,6,9]，C=[0,3,7,10]。从最下 C 开始，把 C 的第 0、2 项 0/7 复制到 B 并合并，B 增广表成为[0,*2,4,6,7*,9]；再把这张表第 0、2、4 项 0/4/7 复制到 A，A 增广表成为[0,*1,3,4*,5,7*,8]。星号仅表示导航副本，同值时还需区分原项身份。



*为从下一表复制的项，只用于导航；输出仍只报告原始项

图 128　分散级联三张增广有序表的隔项副本

查询每张原表中第一个≥1.5 的位置。先在 A 增广表二分，找到 3。在这个位置预存“B 增广表第一个≥3”的指针，它指向 4；但 B 的前一项 2 也≥1.5，因此向左修正一格到 2。2 所在位置又有“C 中第一个≥2”的指针，指向 3，前一项 0<1.5，停止。得到原表 A 从 3 开始、B 从 2 开始、C 从 3 开始。

为什么每次只需常数格修正？下一张增广表的每隔一项都有副本被提到上一表。若需要越过两项，二者之间就有一个抽样副本也应在上表的定位结果之前被发现，与二分找到第一个≥查询值矛盾。实际实现给每项保存到本表原始位置、下一表位置的桥指针，重复值再用同一套 lower_bound/upper_bound 规则处理。查询右端 6.5 也做一遍，三表输出分别 3/5、2/4/6、3，导航副本不作为额外结果输出。

课程范围树还有一个更直接的条件：父节点的 y 表本来就是左右孩子 y 表的归并，已经包含两个孩子的所有项，因此可直接保存“父表前 i 项中有多少项来自左/右孩子”，不必再额外抽样。父表 Y=[0,1,2,3,4,5,6,7]，左表 L=[1,3,4,5,7]，右表 R=[0,2,6]。查询闭区间[2,5]在父表的边界为[2,6)：前 2 项里左 1 项、右 1 项；前 6 项里左 4 项、右 2 项，所以左表边界直接变[1,4)，输出 3/4/5；右表变[1,2)，输出 2。两个父边界各查一次预存计数，就把两次二分变成常数工作。原课件第 39 页画的正是从父 y 表到两个孩子 y 表的捷径。

范围树把这些桥安排在相关的父子辅助表之间。第一次定位花 O(log n)，之后 O(log n)张相关表各花常数定位，加上输出，得到二维范围报告 O(log n+k)。这需要增广表和桥指针，只有原始 y 表的基础实现仍是 $O(\log^2 n+k)$。

每个点会出现在从叶到根的 O(log n)份辅助表里，所以空间增加。它用重复保存顺序信息换查询速度。若数据频繁动态更新，维护这些辅助表比静态构建复杂得多，应确认题目是静态还是动态。

14.6 例 4：kd 树，轮流按坐标切开空间

二维 kd 树在根按 x 切，下一层按 y 切，再下一层按 x 切，轮流继续。通常选当前子集的中位数附近分割，使左右点数接近。节点对应一个包围盒，查询矩形 Q 时：若盒与 Q 不交，整棵跳过；若盒完全在 Q 内，整棵输出；部分相交才递归。

例如一组点被分在 x<3 与 x≥3 两盒，查询框 x∈[4,6]时，左盒可完全排除。到右盒若按 y=2 分成上下两片，而查询 y∈[3,5]，下片又可排除。它用空间包围关系剪枝，不是对两个坐标同时做普通二分。

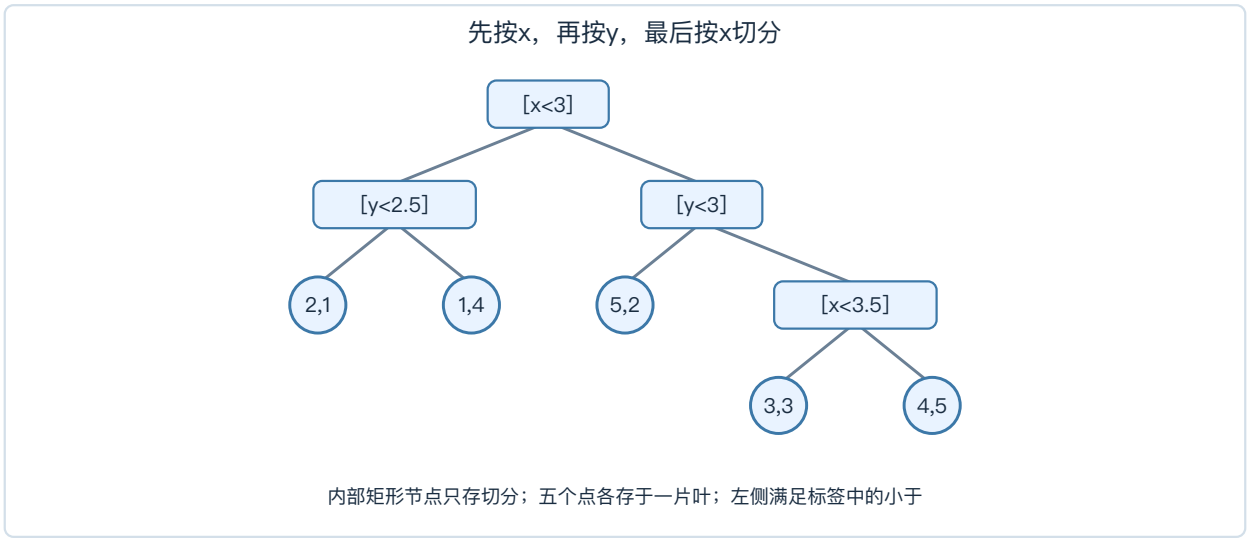


图 129　课程叶存点式 kd 树的完整切分结构

这一图采用课程常用的“内部节点只存切分、所有输入点在叶中”的版本。根按 x=3 切，左组{(1,4),(2,1)}，右组{(3,3),(4,5),(5,2)}。左组再按 y=2.5 分成两个叶；右组按 y=3 分出下叶(5,2)，上组再按 x=3.5 分成(3,3)/(4,5)。每次切分都严格减少子集，五个点各出现一次。

查询同一矩形[2,5]×[2,4]：根两侧都可能相交，分别深入；左侧两片叶检查坐标，(2,1)因 y<2 排除，(1,4)因 x<2 排除。右侧下叶(5,2)在闭边界，输出；上组继续分，(3,3)输出，(4,5)因 y>4 排除。对这个小输入剪枝不一定比扫描省很多，结构价值要在足够多点和查询上衡量。若某个子盒全在查询内，则可整棵输出，不再做每点坐标判定；若完全不交，则不访问它的后代。

对平衡二维静态 kd 树，轴对齐范围报告经典界为 O(√n+k)，空间 O(n)，比二维范围树省空间但查询界不同。这个平方根来自查询边界切过多少小格：先看矩形的一条竖边。在按 x 切的一层，它通常只需进入边所在的一侧；下一层按 y 切，这条竖边可能穿过上下两侧。两层合计最多把边界路径数翻倍，而树中点数规模约分成四份。到约 $\log_2 n$ 层，边界穿过的叶数因此约为$2^{(\log_2 n)/2} = \sqrt{n}$。各层边界节点数按 1、2、4 等几何增长，总和与最后一层同阶；矩形只有四条边，仍为 O(√n)。完全在框内的子树不再做几何测试，只遍历输出它们的点，合计 O(k)，所以得到 O(√n+k)。这一论证针对按中位数平衡切分、边界归属一致的标准二维构造；同坐标可按秩和固定次序处理，不可让一次切分原样递归同一整组。最近邻搜索时可用“查询点到包围盒的最短距离”作为下界：若该下界已不小于当前最佳距离，盒内所有点都不能改善答案，便可剪枝。最近邻最坏仍可能访问 Θ(n)点，不能保证每次 O(log n)；高维时剪枝还可能明显变差。

距离计算可用平方距离比较，避免开方；若坐标为大整数，差的平方也要防溢出。坐标相等时的分割与叶容量要有规则，否则全相等数据可能让递归无法缩小。

14.7 选择前先写清问题

数组区间求聚合并更新，用线段树时节点的区间是位置范围。已有区间做穿刺查询，用区间树时节点里保存的是区间对象。平面正交范围查询，范围树或 kd 树利用多维坐标。它们名字里都有“树”和“范围”，并不代表可以互相替代。

输出敏感复杂度里的+k 是实际输出数量，而不是可有可无的附注。如果查询要报告全部十万个点，任何算法都得花时间写出十万条结果；若只问个数，可能通过节点计数直接汇总，输出需求的变化又带来不同优化。

14.8 中心区间树，查询点左移时到底扫描哪张表

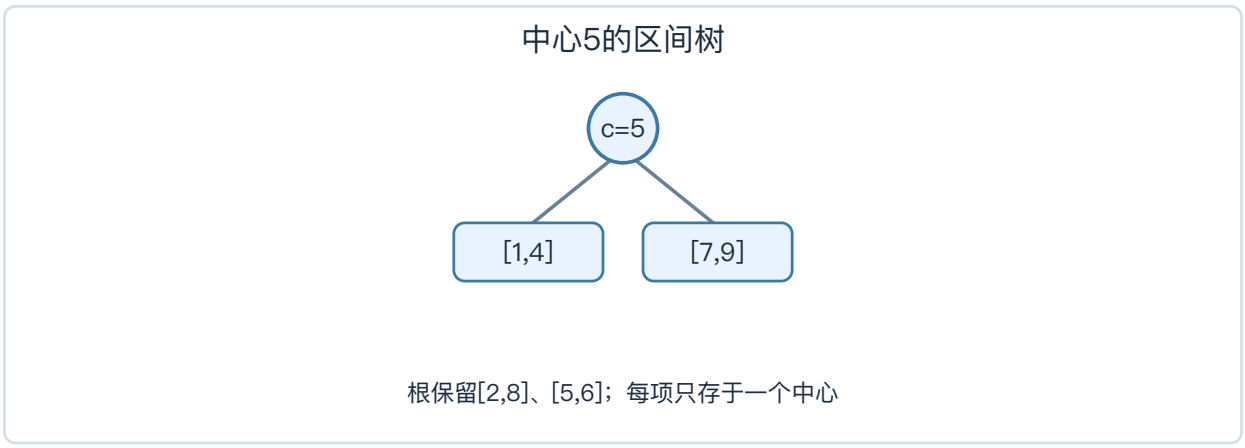


图 130　中心区间树每层保存的跨中心区间和两侧子树

区间为[1,4]、[2,8]、[5,6]、[7,9]，选中心 c=5。跨中心的[2,8]与[5,6]留在根，完全左侧[1,4]进左树，完全右侧[7,9]进右树。根里左端升序表为[2,8]、[5,6]，右端降序表也恰好同序。

查 q=3<c，根里所有区间右端至少 5，因此只要左端≤3 就包含 3。扫描左端表，报告[2,8]，遇左端 5>3 便停止；再走左树，报告[1,4]。不访问右树，因为那里所有区间左端都>5>3。查 q=7>c，则根里所有左端≤5，只需右端≥7，报告[2,8]，遇右端 6<7 停止，再走右树报告[7,9]。

每层只走一边，扫描表中除停止项外都产生输出，因而合适平衡构造下总成本是树高加输出数。若只保存一张按左端排序的表，q 在右边时无法遇到第一个不满足便停止，可能需扫描很多不输出项；两种排序正是在支付两侧查询的不同需求。

14.9 线段树的查询为何只有对数数量的摘要块

看某一层，查询区间中间被完全覆盖的节点可以直接取摘要；只有包含左端点与右端点的最多两个边界节点还要继续拆开。下一层从这两个节点产生至多四个孩子，其中中间完整块又可停止，仍只有两端继续。树高 O(log n)，每层只新增常数数量需要处理的节点，所以单个连续区间查询为 O(log n)。

若查询的是任意散乱的一万个位置，就不能继续沿用“每层最多两个边界”的证明；那已是多个区间或点集查询。算法界来自查询形状，而非只要叫 query 就自动对数。

懒标记也只对能从摘要快速更新的操作成立。整段加 d 更新 sum 只需 len；若只存区间最小值，整段平方不能在含正负值时仅凭旧最小值推出新最小值。例如[−5,2]与[−5,4]旧最小值同为−5，平方后的最小分别 4、16。状态缺少信息，就不能强加一个懒函数。

14.10 kd 最近邻，实际比较一次下界

查询点 Q=(4,4)，先找到候选 P=(5,4)，平方距离 1。某子树盒为 x∈[0,2]、y∈[0,6]，Q 到盒的最近横差 2、纵差 0，平方下界 4，大于当前 1，所以整盒可跳过。另一盒 x∈[3,6]、y∈[0,2]，最近纵差 2，下界也 4，可跳过。

若盒 x∈[3,6]、y∈[3,6]包含 Q，下界 0，即使盒中心离 Q 较远，也不能剪枝。盒内可能恰好有 Q 本身，距离 0。计算下界时每维取：点在区间内贡献 0，在左边贡献到左边界的距离，在右边贡献到右边界的距离，再平方相加。这是可靠几何下界，盒中心距离不是。

通常先搜索包含查询点或更近的一侧，尽早找到小的当前上界，让另一侧更容易被剪掉。这改变实际访问量，不消除最坏可能访问所有点的情况。最近邻算法的“平均表现不错”不能代替题目需要的最坏延迟保证。

14.11 练习与答案

练习 1。 线段树保存 `[1,2,3,4]`，整段加 5 后再把[1,3)加 2，最终总和与各项是多少？

解答：第一次变为[6,7,8,9]，总和 30；第二次变为[6,9,10,9]，总和 34。若根第一次记 lazy5，第二次部分覆盖前必须下推到左右孩子；再在各自相关叶或区间加 2。最后父 sum 由左右重新合并。若不下推就更新旧孩子值，合并后会丢掉第一次的加 5。

练习 2。 maxEnd 增广 BST 查询[8,9]，左子树最大右端点为 7，可以跳过吗？若为 8 呢？

解答：闭区间情形，7<8 可排除；若为 8，可能有区间恰在 8 结束，与查询在 8 相交，不能跳过。若输入改成半开区间，边界比较也要改；区间端点规则会直接进入剪枝条件。

练习 3。kd 树最近邻已经找到距离 5 的点，另一盒到查询点的最近距离为 6。盒里是否还需要逐点查看？下界若为 4 呢？

解答：下界 6 意味着盒内每一点距离至少 6，不可能优于 5，可跳过；下界 4 只说明可能存在更近点，并不保证有，需要继续检查。剪枝使用的是可靠下界，不能拿盒中心距离当下界，因为盒边缘可能更接近查询点。

15 散列表：算出一个位置以后，还要确认是谁

希望用学号查成绩，且不要求按学号排序输出。直接用学号当数组下标可能浪费巨大空间；散列函数把键映射到较小的桶号，例如 `h(k)=k mod 7`。不同键可能得到同一桶，叫冲突。冲突不是偶然的程序错误，只要键范围比桶数大，就不可避免。

散列表首先通过哈希值缩小候选，最终仍要比较真实键是否相等。两个对象哈希相同，不代表对象相等；若对象相等，则哈希必须相同。键若在入表后改变参与哈希的字段，后续按新值查找可能找不到原位置，因此通常要求键在表内保持相应不可变性。

15.1 例 1：拉链法，每桶放一串候选

桶数 m=7，插入 10、17、24、11。前三者都余 3，进入桶 3；11 余 4，进入桶 4。每次插入后的桶 3 依次是[10]、[10,17]、[10,17,24]；插 11 不再改变桶 3，桶 4 成为[11]。本例统一尾插，让手算比较次序可复现。若实现采用头插，最后会是 24⟶17⟶10，正确性不变，但逐次比较的顺序要随之改写。

查 17，先算余 3，再沿桶 3 比较 10、17，找到。查 31 也余 3，要比较 10、17、24 后失败。

装载因子 α=n/m 表示平均每桶有多少元素。在散列足够均匀等假设下，拉链查找期望 O(1+α)。当 α 保持常数，称期望 O(1)；若所有键都挤在同一桶，最坏 O(n)。不能只背“散列表 O(1)”而删掉概率条件与最坏情况。

字符串键的哈希计算还要读取字符，长度 L 的键通常至少 O(L)处理成本；若已缓存哈希值，可复用一部分，但相等性比较也可能读到很多字符。字典操作的常数时间结论常默认键本身可在常数时间哈希与比较。

删除某一键时只删对应节点，不影响同桶其他键。若支持相同键更新值，插入前需要先在桶内确认已有键，不能每次直接追加而产生多个相同逻辑键，除非接口本来就是多重映射。

15.2 例 2：线性探测，空槽为什么有两种



·从未使用；×墓碑；插31仍需确认后面没有已有31，不能遇墓碑立刻插

图 131　线性探测每次插入、墓碑删除、查询与复用的完整数组

图中最上方是槽下标，不是键值。圆点表示从未使用，叉表示曾用过但已删除。每行黄色格是这次操作检查过的位置，插 31 的倒数第二行还没写入，最后一行才把它放回首个墓碑。

开放定址把所有记录放在数组中。桶 m=7，线性探测依次试 `h(k),h(k)+1,...` 对 m 取模。插 10 放 3；插 17 先看 3 被占，放 4；插 24 再看 3、4，放 5；插 11 的本位 4 已占，再看 5，放 6。

查 24 沿 3、4、5 找到。若删除 17 后把 4 直接设成“从未使用”，查 24 会在 4 提前停止，误判不存在。因此删除槽通常标成墓碑：查询遇墓碑继续，遇从未使用槽才知道这条探测链上不可能有目标。

插入可记住沿途第一个墓碑作为候选空位，但仍需继续查到真正空槽或确认不存在已有同键。否则可能在墓碑处先插一个 17，而后面其实已有 17，造成重复键。若扫描一整圈都无从未使用槽，也要有明确终止条件，避免满表无限探测。

墓碑不算有效记录，却会增加查找路径。重建触发条件往往不仅看有效装载因子，还看有效记录加墓碑占据的比例。桶数远大于 n 也不必然快：若几乎所有槽都被墓碑覆盖，失败查找仍可能走很长。

15.3 线性聚集与其他探测方式

以上生命周期继续：删 17 以后，查 24 仍检查槽 3 的 10、槽 4 墓碑、槽 5 的 24。插 31 的本位也为 3，依次检查 3、4、5、6、0；先记住 4，但直到 0 的从未使用槽才确认后面不存在 31，再把 31 放到 4。此后槽 4 已是 31，不再是墓碑。

另从“刚删 17、还未插 31”的状态看更新 11：先遇槽 4 墓碑，不能立即在这里插一个 11；继续探测 5、6，在 6 找到已有 11，只更新该记录。这个分支说明为什么记住墓碑后仍要继续找同键，和插 31 完成后的表不是同一时刻。

线性探测形成连续占用块，新键撞到块中任一位置都会并入这个块，使块越来越长，这叫主聚集。二次探测用平方偏移，双重散列用第二个函数给步长，试图分散探测位置。

双重散列`(h1(k)+i*h2(k)) mod m`要确保步长与 m 互质，才能遍历全部桶。例如 m=8、步长 2 只会访问 0、2、4、6，另一半槽即使全空也到不了；m 为质数且步长介于 1 与 m−1，则互质。二次探测是否能找到空位也依桶数与装载条件，不能把“改变步长”当作完整正确性证明。



第一行四格循环；第二行步长3与8互质，恰访问全部8格

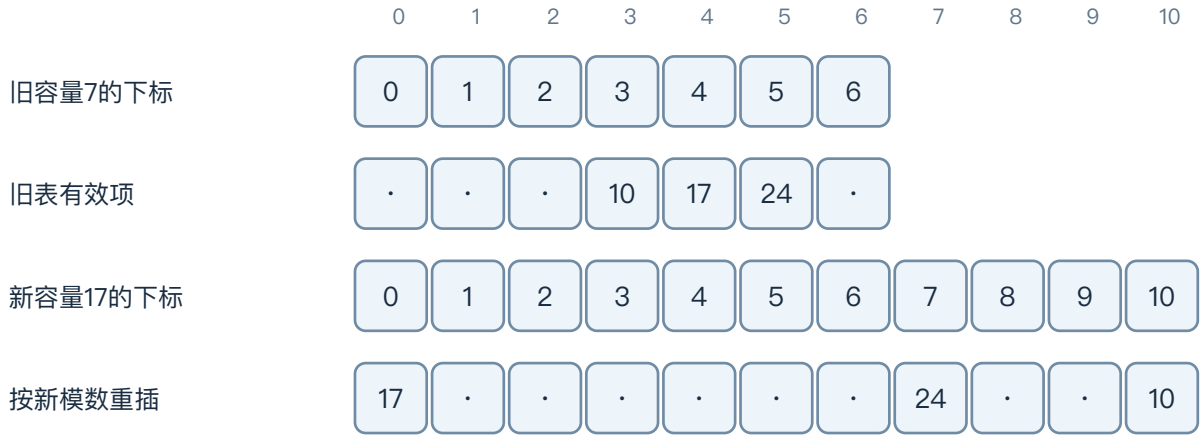
图 132　同一容量下两种步长的实际探测周期

二次探测也应真走一个输入：容量 7，本位 0，偏移 i^2，i=0 至 6 访问 0、1、4、2、2、4、1，只覆盖 0/1/2/4 四格。若这些位置都满，即使 3、5、6 空也找不到。因此实现必须配合能保证成功的容量/负载条件，或明确检测探测周期后重建。

在理想均匀探测模型下，失败查找期望探测数可由装载率估计为不超过约 1/(1−α)。α 接近 1 时成本急剧上升。这个上界可以按“还没找到空位”的概率求和：检查 j 个槽仍全满的概率不超过α^j，所以期望探测数不超过$1+\alpha+\alpha^2+\cdots=1/(1-\alpha)$。这里假设探测次序是理想均匀的随机排列，剩余空槽占比只会让后续全满概率更小。α=1/2 时上界 2，α=0.9 时上界 10；它没有说每次查询恰好这么多步。线性探测的具体期望公式与均匀独立探测模型不同，不能把一个模型的公式搬给所有冲突策略。

15.4 例 3：扩容以后，为什么所有键都要重新算位置

旧桶数 7，键 10 在 3、17 在 4、24 在 5。扩成 17 桶以后，h(k)=k mod17，三者本位分别 10、0、7。若只是把旧数组复制到新数组前三格相同位置，查 10 会先去 10 而找不到。



新表未画11..16；它们均空。不能原位复制旧桶3/4/5。

图 133　容量 7 扩到 17 时每个有效键重新定位

重散列需要新建空表，遍历所有有效记录，按新桶数重新插入，不复制墓碑。若表内 n 项且新表规模 O(n)，重建成本 O(n)期望或与具体散列规则相应的成本。按固定比例扩容并控制装载阈值，可以把这种昂贵重建摊还到之前的多次插入。

它同时包含两层不同分析：冲突处理的成本通常是期望界；扩容总账是摊还界。因此完整说法可能是“期望摊还 O(1)插入”，而不是两种条件被一个 O(1)遮住。

取模前的哈希也会影响分布。若整数键都为某个因子的倍数，桶数选择可能造成大量冲突。字符串多项式哈希按字符顺序累积，例如 `h = h*b + code(c)`，比简单相加能区分换位字符，但仍可能碰撞。密码学抗攻击要求另有安全设计，课程普通哈希不能自动承担密码学用途。

15.5 Trie：键是字符串时，沿字符路径找

用 car、cart、cat 建立一棵 Trie。空根不代表字符，每条边消费一个字符，节点另记“是否到这里就结束一个完整词”。

插入词	复用已有前缀	新建边/节点	新增词尾标记
car	无	根-c-a-r 三条边	r 处标词尾
cart	c-a-r	从 r 新建 t	t 处标词尾，保留 r 词尾
cat	c-a	从 a 新建另一分支 t	新 t 处标词尾

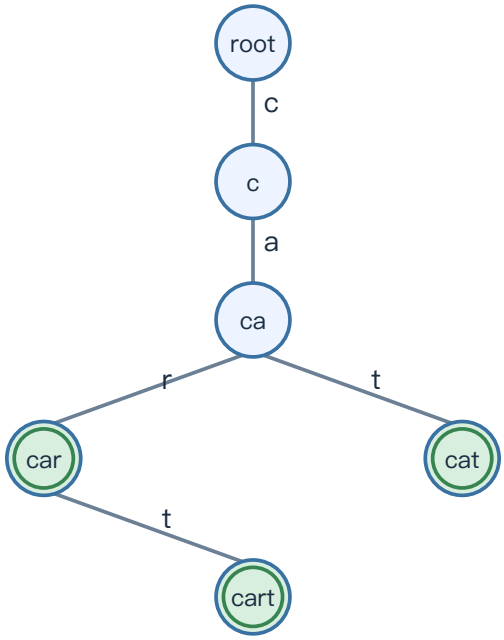


图 134　插入 car、cart、cat 之后共享前缀与词尾

查 ca：从根读 c，再读 a，路径存在，但 a 未标词尾，因此不是保存的完整词；查前缀 ca 则成功。查 cart：依次走 c/a/r/t，在最后 t 的词尾标记为真，找到。查 cab：c/a 仍可走，但 a 没有 b 孩子，立即失败，不必扫描 car 和 cat 的完整字符串。

时间与键长 L 有关，而非词典总词数 n 的对数；每步孩子若用固定字母表数组查找，则 L 步为 O(L)。若每个孩子集合用平衡树或散列表，字符转移的成本也要相应计入。所有词的总输入字符数若为 S，新建节点至多 S+1（包括根），共享前缀只会减少数量。

字母表很大时，每节点开全套孩子数组很浪费，可用压缩分支、映射或基数树。路径压缩把只有一个孩子的连续路径合成一条带字符串片段的边，节省节点，但查找需逐字符比较该片段。

15.6 例 4：用散列表计数，输出需求会改变下一步

输入`[4,2,4,7,2,4]`，依次更新计数：4⟶1，2⟶1，4⟶2，7⟶1，2⟶2，4⟶3，最后映射{4:3,2:2,7:1}。若只问 4 出现几次，直接查 4。若要按数值从小到大输出频数，普通散列表不提供这种顺序，还需把键排序，得到 2:2、4:3、7:1。

若键范围只有 0 到 9，数组计数可能更直接；若还要找前缀相同的字符串，Trie 更贴合；若要随时问最小键和前驱，平衡搜索树更方便。字典只是接口，底层结构应按需要选择。

15.7 为什么随机选哈希函数能帮助，而不是把输入称作随机

设从一个函数族中随机选 h，保证任意两个不同键冲突概率不超过 1/m。固定要查的键 x，以及已经存入的 n 个不同键，为每个其他键 y 定义冲突指标。该桶候选数期望至多 1+(n−1)/m=O(1+α)，由线性期望直接得到，不需要假设输入键本身均匀。

这个模型将随机性放在哈希函数选择上。若始终使用固定简单取模，输入恰好全是 m 的倍数，就不能引用上述随机函数族保证。若对手能看见 h 以后专门选键，假设也可能改变；课程一般散列分析与面向恶意输入的哈希设计需区分。这里要学的是概率到底对哪个对象取，而非把“随机”当万能形容词。

15.8 重新散列也要检查删除后的负载

假设表容量 16，最多容纳某阈值比例有效项。大量删除后只有 2 项，却留下 12 个墓碑。只看 α=2/16 似乎很空，但失败查询可能穿过连续墓碑长链。重建到一个新空表，重新插入这 2 项，才清除历史探测痕迹。

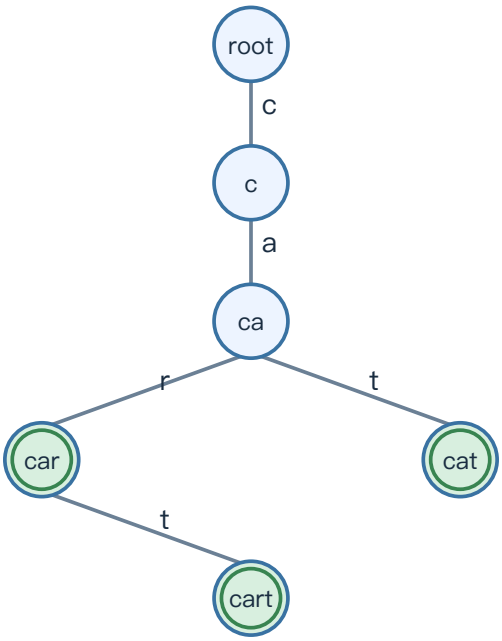
如果每次有一个墓碑就立即重建，会让频繁删插序列反复支付全表成本。可在有效加墓碑比例过大、墓碑相对有效项过多时触发，并与容量增长、收缩阈值配合留缓冲。每条策略都需要总账，不是只说“偶尔重建”就得到摊还常数。

15.9 装载因子同样大，冲突形状仍会影响成本

容量 8、4 项，α=1/2。若占位为 0、2、4、6，各新失败探测可能很快遇空；若为 0、1、2、3，哈希到 0 的失败查找会检查 5 格才遇空。α 描述全局密度，不完整描述具体簇形。期望公式依赖散列与探测的随机模型，单个具体表仍可能偏离期望。

拉链法 α 可以大于 1，因为每桶可有多个节点；开放定址有效项不能超过容量，α 接近 1 时探测成本暴涨。把两种结构的相同字母当作相同使用范围，会得出满表仍能常数插入之类错误结论。

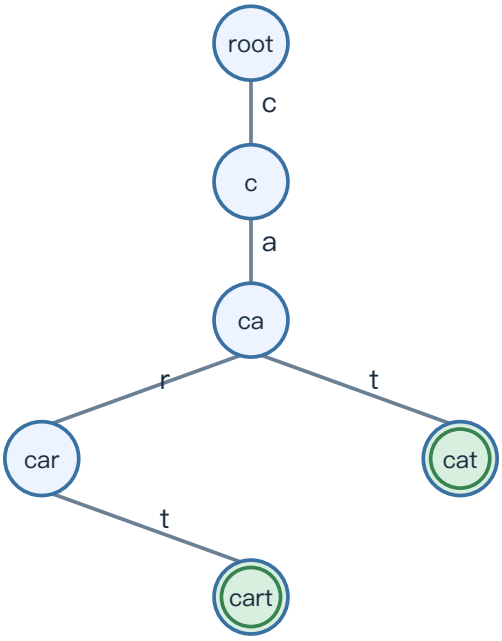
15.10 Trie 删除与前缀报告，逐项保留仍有用途的节点



双圈=完整单词结束；路径存在不代表这里已经保存一个词

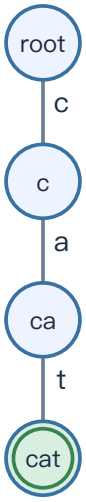
图 135　Trie 中 car 同时是词尾和 cart 的前缀

词典含 car、cart、cat。删除 car 只清 car 末节点的结束标记，不能删其节点，因为 cart 仍沿它向下。



双圈=完整单词结束；路径存在不代表这里已经保存一个词

再删 cart 后，t 节点无孩子且不结束任何词，可删；向上 car 的 r 节点也无孩子且已非词尾，可删；到 a 节点仍有通往 cat 的 t 分支，停止。每层检查是否仍被别的词需要，才不会拆掉共享前缀。



双圈=完整单词结束；路径存在不代表这里已经保存一个词

图 136　再删除 cart 后回收 t 与 r，仍保留 cat 路径

图中双圈表示单词在这里结束。第一张的 car、cart、cat 都是双圈；第二张 car 不再双圈，但 r 和它的 t 仍在；第三张只剩 cat，a 仍不能删。

报告前缀 ca 下所有词，先用两步走到 ca，再遍历它的后代并在词尾输出；时间至少包括输出字符总数。若每次拼接复制整个前缀字符串，成本可能超过只按节点数算的界，通常用可回退字符缓冲区记录当前路径。

15.11 迭代顺序也是接口承诺的一部分

普通散列表遍历通常不按键大小，扩容重散列后内部顺序还可能改变。有些语言字典另外承诺插入顺序，那是实现额外维护的关系，不是哈希函数自动产生有序。

若程序依赖按成绩从低到高输出，应显式排序键或选择搜索树，不能拿一次本地桶顺序碰巧递增当保证。相同集合经过不同插入、删除历史，内部布局也可能不同，但存在查询仍应给同样结果。

墓碑保留的是探测历史，不是一个仍然存在的用户键，计数与遍历都应跳过它。

15.12 练习与答案

练习 1。 m=5，线性探测插入 1、6、11，再删 6。查询 11 访问哪些槽？随后插 16 应怎样利用墓碑？

解答：1 在 1，6 在 2，11 在 3，删 6 后 2 为墓碑。查 11 从 1 到 2 再到 3，不能在 2 停。插 16 从 1 开始，记住 2 为首墓碑，仍继续检查 3 以免后面已有 16；到 4 发现从未使用槽，确认不存在后将 16 放回 2。有效记录仍沿同一探测规则可达。

练习 2。 相同对象的哈希能否不同？不同对象的哈希能否相同？

解答：按相等性定义判为相同的键必须产生相同哈希，否则它们会被送到不同桶而无法正确更新和查找。不同键可以同哈希，这正是冲突，需要真实相等比较处理。哈希值只用于定位候选，不能单独证明键相等。

练习 3。 Trie 中已存 `a` 与 `ab`，删除 a 能否直接删掉字符 a 对应的节点？

解答：不能，那会同时断掉 ab 的路径。先清除 a 节点的结束标记；只有节点不再标记单词且没有孩子时，才可向上回收无用节点。一个节点可以同时是某个单词的终点和另一个单词的内部路径。

16 图：把“谁和谁相连”变成可追踪的状态

地图中的地点与道路、课程与先修关系、计算任务与依赖，都是图。顶点代表对象，边代表关系。无向边表示双向连接；有向边 u→v 只承诺从 u 到 v。图 G=(V,E)中，|V|为顶点数，|E|为边数；本章讨论有限图，算法细节会区分是否允许重边与自环。

树可以看作没有环的连通无向图，但一般图允许环、多个父方向与不连通部分。树遍历直接递归孩子不会重复；图遍历必须记录访问状态，否则绕一圈又回到起点，可能无限递归。

16.1 怎样存边，决定遍历要付多少成本

邻接矩阵用 V×V 表格，a[u][v]记录是否有边或边权，判断两点是否相邻 O(1)，空间 Θ(V²)。枚举 u 所有邻居通常扫一整行 Θ(V)。邻接表为每个顶点保存出边，空间 Θ(V+E)，枚举所有邻居总成本与实际边数有关。无向边在邻接表中通常保存两份，u 表中有 v，v 表中有 u。于是所有度数和为 2E。有向图中出度总和与入度总和都为 E。复杂度写 O(V+E)时，通常默认邻接表等能按实际边数枚举邻居的表示；同一算法换成矩阵，遍历可能 Θ(V²)。

16.2 例 1：BFS，队列保证无权最短路

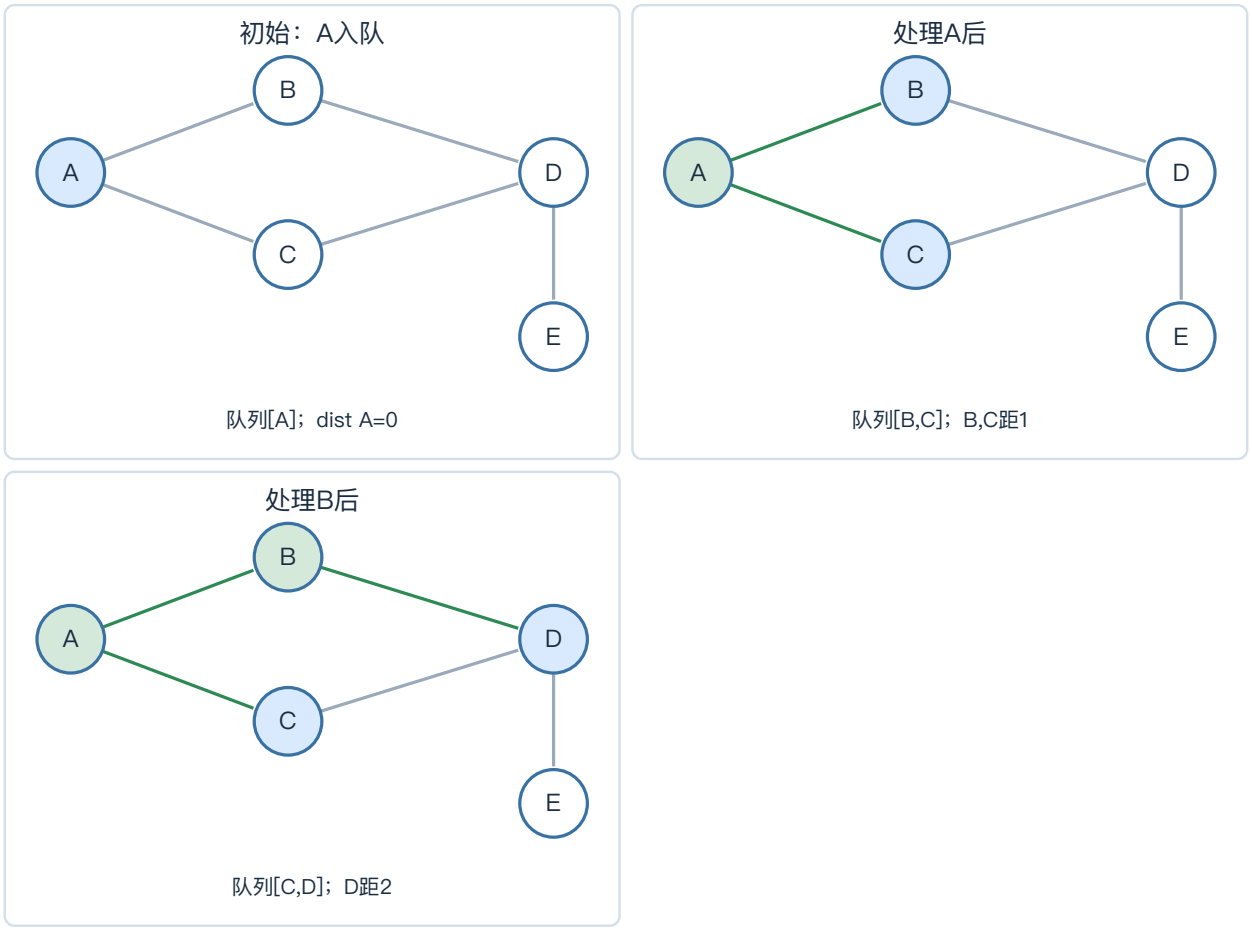


图 137　BFS 开始到处理 B 时的队列与父边

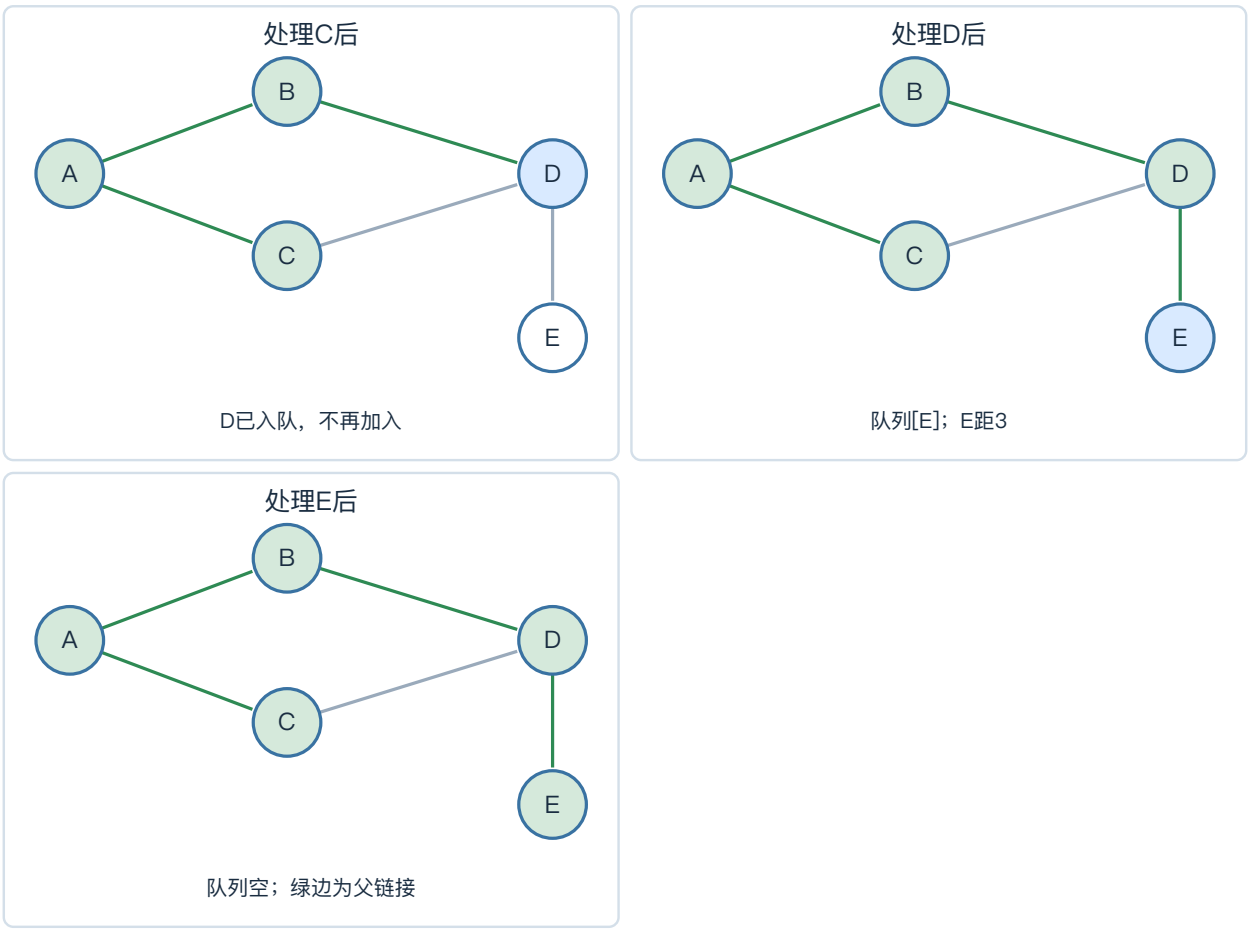


图 138　BFS 继续处理 C、D、E 直到队列为空

图中蓝点已经发现并在队列中，绿点已经处理完，白点尚未发现；绿边是首次发现时保存的父链接。BFS 树的边数距离可沿这些父边数出来，原图里 C-D 边存在但没有成为父边，因为 D 先由 B 发现。

无向图边为 A-B、A-C、B-D、C-D、D-E。源点 A，初始化 dist[A]=0，其他未发现。先将 A 入队，并立即标记已发现。

出队顶点	新发现的邻居	新距离	出队后的队列
A	B,C	B=1,C=1	B,C
B	D	D=2	C,D
C	无，D 已发现	不变	D
D	E	E=3	E
E	无	不变	空

为什么入队时就标记，而非出队时才标记？B 和 C 都邻接 D，若 D 入队后仍保持未发现，C 会再次将 D 入队，造成重复处理。第一次发现时设置 dist 和 parent，记录它由谁到达，既避免重复，也能恢复路径。E 沿 parent 倒走 E-D-B-A，反转得到 A-B-D-E。

队列中顶点距离按非降顺序出现。处理距离 d 的点时，只会新发现距离 d+1 的邻居；若某点存在更短路径，它的前驱距离更小，应已更早出队并发现它，矛盾。因此 BFS 首次发现给无权图的最少边数距离。边权不同就不再成立，例如 A 到 B 直边权 100，经 C 两条权 1 的路更短，BFS 只最小化边数。

邻接表中每点入队出队一次，每边检查常数次，时间 O(V+E)、额外空间 O(V)。若图不连通，从 A 只得到 A 所在连通分量；要遍历全图，外层再对每个未发现顶点启动一次 BFS。

16.3 例 2：DFS，进入与退出时间分别说明什么

深度优先搜索沿一个邻居继续走，走到没有未发现邻居才回退。顶点可以分白、灰、黑：白为未发现，灰为已进入但尚未完成，黑为所有出边已检查完。进入时记发现时间 d[u]，退出时记完成时间 f[u]。

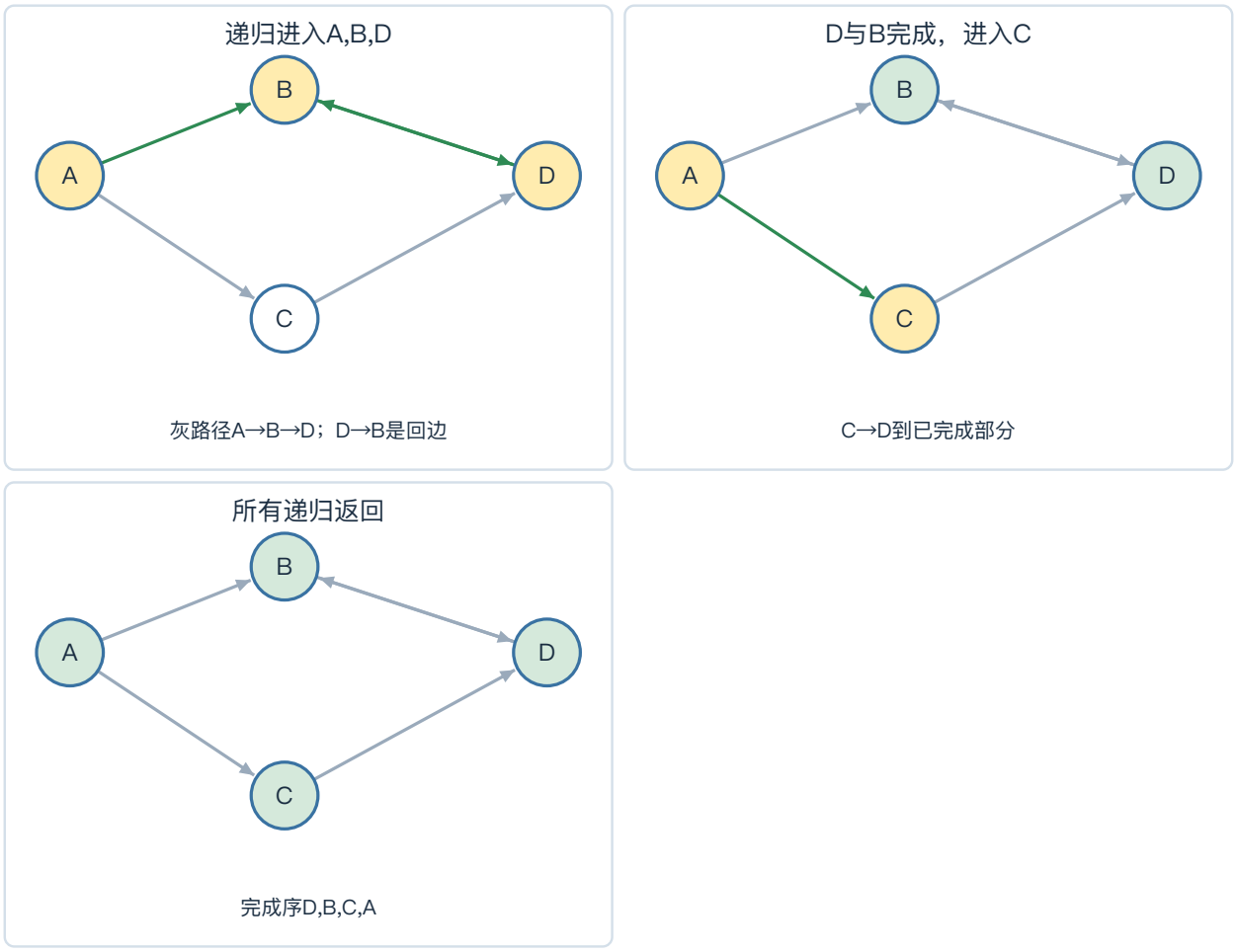


图 139　DFS 活动路径与已经完成的部分

有向图中 B→D 与 D→B 是两条相反的边，图中重合的连线由正文的箭头次序区分。黄色点表示当前递归栈，绿色点表示已退出；白点还没进入。

对有向边 A→B、A→C、B→D、D→B、C→D，按字母邻居次序从 A 开始。进入 A，进入 B，进入 D；D 看到灰色 B，发现一条指向当前递归祖先的回边 D→B，说明有环 B→D→B。D 完成，B 完成；回 A 后进入 C，C 看到黑色 D，D 不在当前递归祖先链上，是跨到已经完成部分的边；最后 C、A 完成。

把时钟在每次进入、退出都加 1，实际记录为：

点	进入 d	退出 f	父亲	退出时递归栈
D	3	4	B	A,B
B	2	5	A	A
C	6	7	A	A
A	1	8	无	空

比如 C 的区间[6,7]与 D 的[3,4]分离，所以 C→D 不是指向本次递归祖先。D→B 时 B 的区间尚未结束，是活动祖先，闭合出 B→D→B。

DFS 树中，一个节点子树的进入退出区间嵌套在其祖先区间内。对任意两个节点，区间要么嵌套，要么分离。灰节点恰好是当前尚未退出的递归路径，指向灰节点的有向边因此构成环。

无向图每条树边有反向记录，孩子看到父亲时不能把这条原树边当新环。如果允许平行边，应按边编号跳过“进入时那条边的反向副本”，而不能跳过所有通向父顶点的边，否则会漏掉另一条平行边形成的环或影响桥判定。

16.4 例 3：拓扑排序，顺序必须尊重箭头

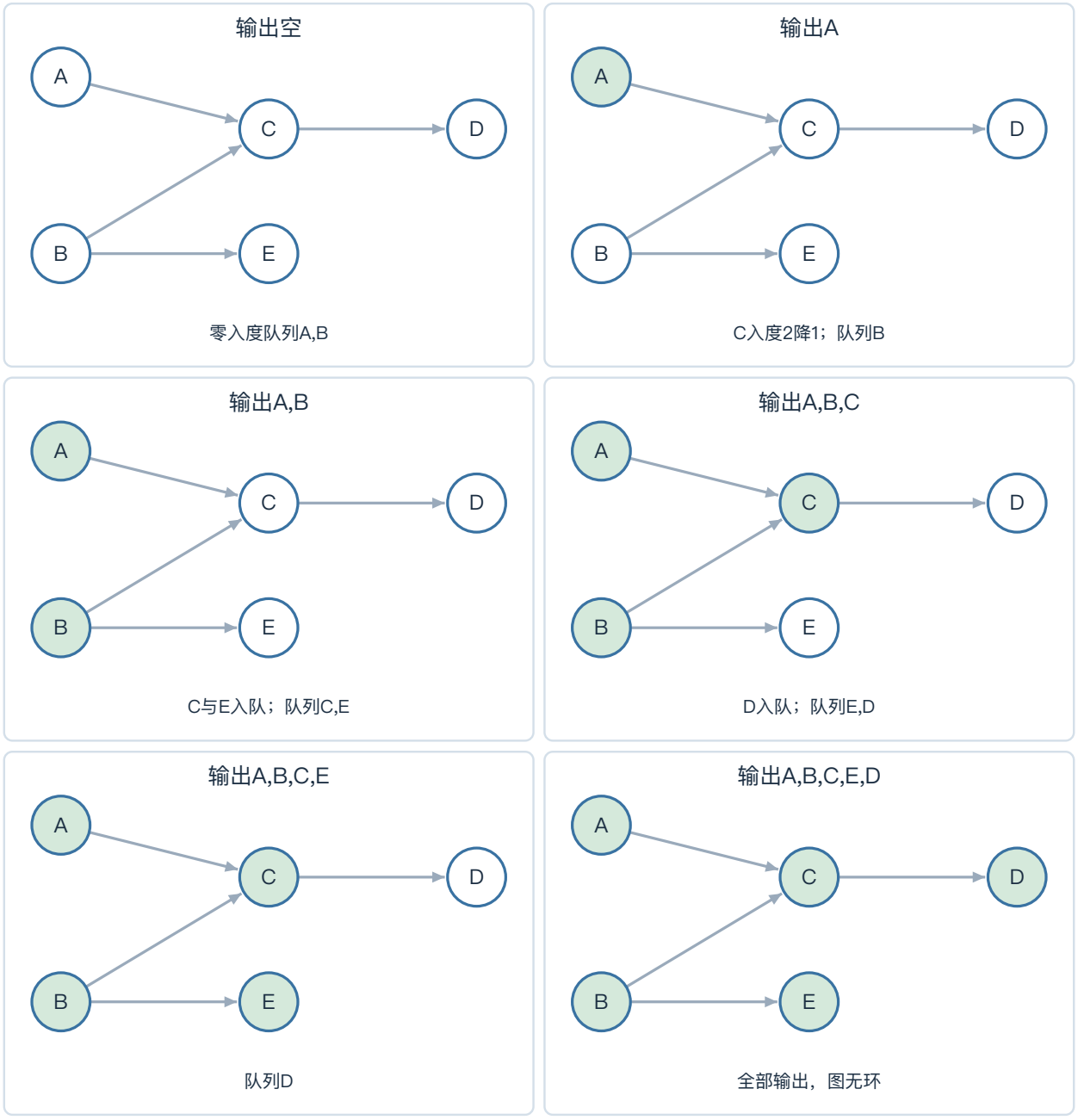


图 140　Kahn 逐次输出顶点与剩余依赖

图中绿色点已经从剩余图删除，连接它们的原边只作为位置参照，不再计入后续入度。每一步只有从新删除顶点发出的边使后继入度减 1。

课程依赖：A→C，B→C，C→D，B→E。意思是 A 和 B 必须先于 C，C 必须先于 D，B 必须先于 E。拓扑序并不唯一，例如 A B C D E 或 B E A C D 都合法。

Kahn 算法先计算各点入度。初始 A、B 入度 0，入队。出 A，删除 A→C，C 入度从 2 变 1；出 B，C 变 0、E 变 0，入队 C、E；出 C 使 D 变 0；最后得到 A、B、C、E、D。每次挑入度 0 顶点，说明剩余图里没有任何未完成前驱挡住它。

把 Kahn 的每一步都写出，便能检查“减少的是谁的入度”：

刚输出	剩余 C/D/E 入度	新入队	等待队列	已输出
开始	2/1/1	A,B	A,B	空
A	1/1/1	无	B	A
B	0/1/0	C,E	C,E	A,B
C	已删/0/0	D	E,D	A,B,C
E	已删/0/已删	无	D	A,B,C,E
D	全已删	无	空	A,B,C,E,D

若最后输出少于 V 个点，剩余图每个点入度至少 1。在有限图中不断往前追前驱，必有重复顶点，从而存在环。反过来，有环上的点不可能在环尚未破坏时变成入度 0，所以无法全部输出。时间 O(V+E)。

另一种方法是 DFS 完成时间逆序。对 DAG 任意边 u→v，DFS 若从 u 走到 v，则 v 先完成；若 v 已完成，也仍有 f[v]<f[u]；不可能指向灰祖先，否则有环。因此按完成时间递减排列，每条边都从前指向后。应先检查回边，不能对含环图照样输出然后叫拓扑序。

16.5 强连通：彼此都能走到，与无向连通不同

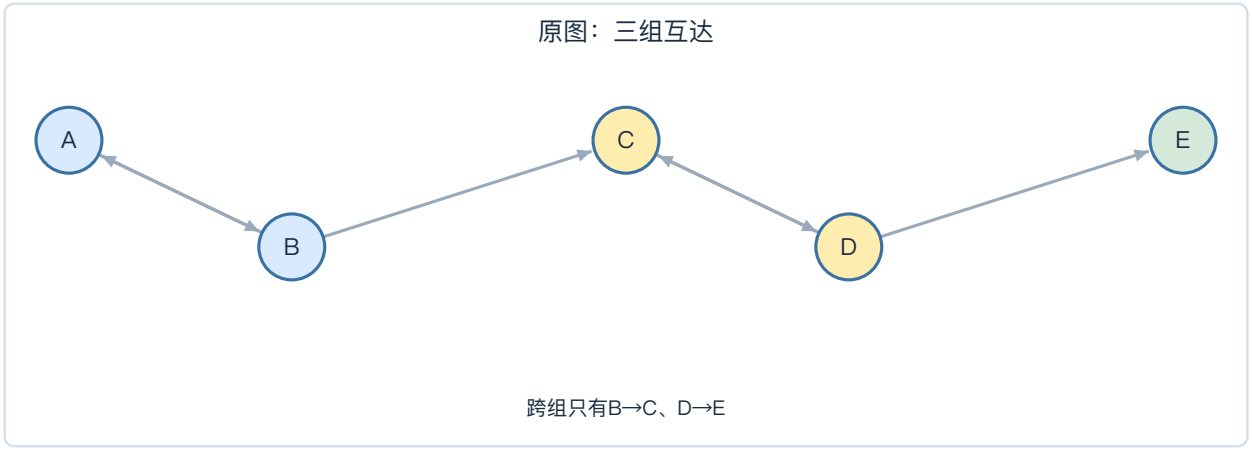


图 141　强连通分量与两个跨分量的有向连接

图中 A 与 B 双向可达、C 与 D 双向可达；A 到 E 可走，E 却没有回到 A 的路径。

有向图中，u 可达 v 且 v 可达 u，二者属于同一强连通分量。把每个强连通分量缩成一个点，得到的凝聚图一定是 DAG；否则分量之间形成有向环，沿环便能互达，应合为一个更大的强连通分量。

Kosaraju 算法先在原图做 DFS，记录完成次序；再将所有边反向，按原图完成时间递减顺序启动 DFS，每棵新搜索树给一个强连通分量。为什么要用这个次序？凝聚图中的“靠源头”的分量会在适当完成次序中先被选到；反图里它没有通向尚未处理其他分量的出口，因此一次搜索不会越过分量边界。反向图与次序必须配套，不能只记“两次 DFS”便随意改变方向。

例：A→B、B→A、B→C、C→D、D→C、D→E。强连通分量为{A,B}、{C,D}、{E}，凝聚图是三点链。A 能走到 E，但 E 不能回到 A，所以绝不能把整个弱连通图当作一个强连通分量。

Tarjan 强连通算法也用 lowlink 和栈，但它与下面无向图割点算法的栈条件不同，尤其要判断邻居是否仍在分量栈中。两个算法都出现 low 变量，不代表更新规则可以互换。

16.6 Tarjan 强连通：完成一组时才从分量栈弹出

仍用上一图，按 A、B、C、D、E 顺序进入。idx 记录首次进入编号，lowlink[u]表示在已搜索的部分，从 u 沿 DFS 子树向下，再至多用一条边到仍在分量栈中的节点，能取得的最小 idx；也包括不走边的 idx[u]。该栈中节点不一定是当前递归链上的祖先；另设 onStack 判断节点是否还在“尚未分配分量”的栈。这个栈可以比递归活动路径长，节点递归退出后不一定已经归组。

规则分两种：遇未访问邻居 v，递归处理 v 后用 lowlink[v]更新自己；遇已经访问且仍 onStack 的邻居 v，用 idx[v]更新自己；遇已从分量栈弹出的点，不能再借它降低 lowlink。完成 u 时若 lowlink[u]=idx[u]，u 就是一个分量的栈底，弹到 u 为止。

处理位置	分量栈（底到顶）	实际动作
进入 A、B	A,B	idx 为 1、2，B⟶A 指向栈中 A，lowlink[B]=1
继续进入 C、D	A,B,C,D	idx 为 3、4，D⟶C 把 lowlink[D]降到 3
进入并完成 E	A,B,C,D,E	idx[E]=lowlink[E]=5，立即弹 E，得{E}
返回 D	A,B,C,D	D 的 lowlink=3≠idx4，D 暂不弹；已归组 E 不参与回边更新
返回 C	A,B,C,D	lowlink[C]=3=idx3，依次弹 D、C，得{C,D}
返回 B，再返回 A	A,B	B 的 lowlink 仍 1，不弹；A 满足 1=1，弹 B、A，得{A,B}

每个节点只压弹一次，每条有向边只处理一次，因此邻接表实现 O(V+E)。接下来无向割点的 low 也追踪“绕回祖先”，但它跳过父边、使用的是无向 DFS 性质；不要把 onStack 条件直接从本段移过去。

16.7 例 4：无向割点，哪一个节点拿掉会断开

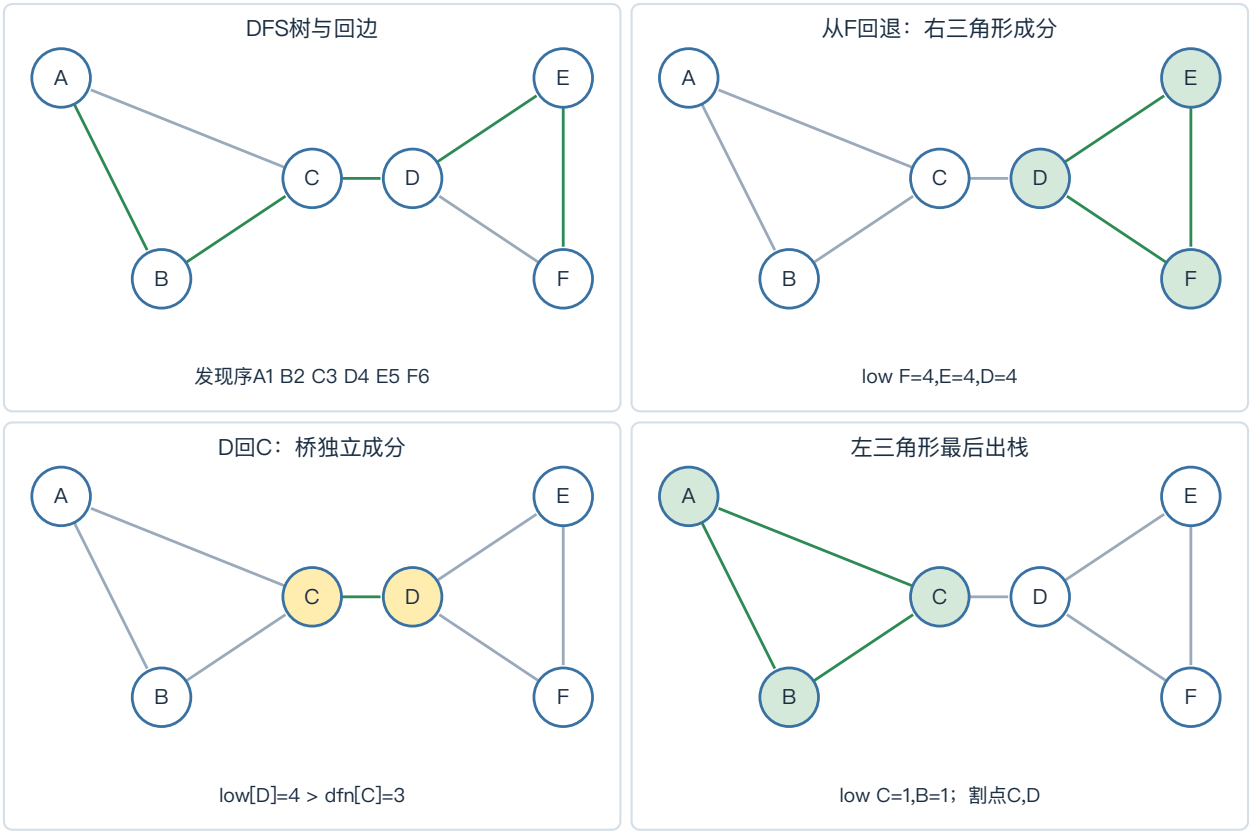


图 142　双连通例子的原图、DFS 父边和三个边分量

图中实绿边在不同格表示当前讨论的树边或分量边，白色底图始终保留原来的七条无向边。按 A、B、C、D、E、F 进入，发现序为 1 到 6。

无向图由三角形 A-B-C-A，连接边 C-D，以及三角形 D-E-F-D 组成。删 C 后，A、B 与右侧分开；删 D 后，E、F 与左侧分开。因此 C、D 是割点，C-D 是桥。三角形内部任意一条边都不是桥，因为还有另一条绕路。

DFS 进入时记录发现序号 dfn[u]。定义 low[u]为：从 u 子树中的节点出发，沿若干树边向下，再最多用一条指向祖先的非树边，能到达的最小发现序号。它描述这棵子树能不能绕过父连接到更早祖先。

对子树孩子 v，搜索完成后做 `low[u]=min(low[u],low[v])`。遇已发现且不是父边的邻居 w，在无向 DFS 中可用 `dfn[w]` 更新 low[u]。不要用 low[w]替代 dfn[w]乱传，因为定义只允许一条向祖先的非树边，不允许继续借别的子树绕多次。

非根节点 u 若有某个孩子 v 满足 `low[v] >= dfn[u]`，说明 v 子树无法绕到 u 的严格祖先；删 u 会把该子树与上方分开，所以 u 是割点。等号仍是割点条件：子树只能回到 u，u 被删就断。

树边 u-v 若满足 `low[v] > dfn[u]`，则 v 子树连 u 本身也绕不到，删除这条边就断，因此是桥。这里必须严格大于；若 low[v]=dfn[u]，说明另有回到 u 的边，这条树边处在环里，不是桥。

DFS 根没有“上方”部分，要单独判断：根有至少 2 个 DFS 树孩子才是割点。注意是树孩子数，不是邻接顶点数。三角形从 A 出发时，A 可能邻接 B 和 C，但 DFS 经 B 已经发现 C，根只有一个树孩子，不是割点。

16.8 双连通分量，为什么边栈比顶点集合更清楚

点双连通分量常指极大的、没有割点的连通子图。不同分量可以共享割点，却不会共享普通边。按本章“极大无割点连通子图”的约定，孤立顶点也单独构成一个单点分量；边栈从未入边时无法自动输出它，需要在外层对度为 0 顶点另外报告。若课程采用仅报告含边分量的约定，应明确不输出这些单点。DFS 时将经过的边压栈；当某孩子 v 满足 `low[v]>=dfn[u]`，从边栈弹到树边 u-v，弹出的边构成一个分量。

在两个三角形通过 C-D 相连的例子中，分量包括三角形 A B C、桥边 C D 以及三角形 D E F，割点 C、D 可出现在多个分量里。是否将桥单独视为一个二顶点分量，是资料中的定义约定；做题应对照课程，而不能因为“两个顶点不够形成环”就强行删除它。

算法每顶点、每边处理常数次，时间 O(V+E)。实现时递归深度最坏 V，大图可能栈溢出；可以用显式栈保存当前顶点、父边与下一条邻边位置，但必须保留“子调用返回后更新 low”的阶段信息。

16.9 Kosaraju 在一个有方向的例子上完整跑两遍

边 A→B、B→A、B→C、C→D、D→C、D→E，从 A 开始按给出的邻接次序 DFS。进入顺序 A,B,C,D,E；E 先完成，再 D、C、B、A，因此完成逆序为 A,B,C,D,E。反图里有 A↔B、C→B、C↔D、E→D。

第二遍先从 A 走反图，只能到 B，不能从 B 走到 C，因为原 B→C 反成 C→B，得到分量{A,B}。接着 C 未访问，从 C 可到 D 和已处理 B，忽略后者，得到{C,D}。最后 E 得到{E}。如果第二遍仍用原图，A 会一路走到 E，错误合成一团，说明反向边是隔离分量的关键。

对一般凝聚 DAG，可用完成时间证明该隔离次序：若原图有不同分量 X→Y，则两个分量的最大完成时间满足 F(X)>F(Y)。第二遍选尚未处理的最大完成时间分量，在反图里它不可能走到另一个尚未处理分量，否则原图会有该分量指向它并具有更大 F，矛盾。因此一次搜索恰好覆盖一个分量。每遍 O(V+E)，建立反向邻接表也 O(V+E)。

16.10 双连通分量的边栈，真的弹一次

在两个三角形 A-B-C-A 与 D-E-F-D 之间接 C-D。DFS 先压 A-B、B-C、C-A，再沿 C-D 进入右侧，压 D-E、E-F、F-D。右三角形回边让 low[F]、low[E]回到 dfn[D]，当 E 返回 D，low[E]≥dfn[D]成立，从边栈弹 F-D、E-F、D-E，得到右三角形分量。

按边栈底到顶，完整过程是：

时刻	栈内容	low 变化/弹出原因
沿 A-B-C 并看回边 C-A	AB,BC,CA	low[C]=1
沿 C-D-E-F 并看回边 F-D	AB,BC,CA,CD,DE,EF,FD	low[F]=4
F 返回 E	同上	low[E]=4
E 返回 D	AB,BC,CA,CD	low[E]=4≥dfn[D]=4，弹 FD/EF/DE
D 返回 C	AB,BC,CA	low[D]=4>dfn[C]=3，弹 CD
C 返回 B，再 B 返回 A	空	low[B]=1≥dfn[A]=1，弹 CA/BC/AB

只在第一次遇到非树边、且它指向祖先时入边栈，反向副本不再重复压入。这样每条无向边恰好压一次、弹一次。

D 返回 C 时，low[D]>dfn[C]，再弹 C-D，得到单桥边分量。最后左三角形返回根的对应孩子时，弹 C-A、B-C、A-B，得到左分量。每条边只入栈出栈一次，总线性；割点 C 和 D 可由多组弹出的边共同出现。

如果用顶点栈而不区分分量边界，容易把共享割点从前一个分量弹掉，后面缺了它。点双连通与边双连通也不同：去掉桥后的连通块是边双连通的一种分解，不能直接冒充去掉割点条件下的点双连通分量。

16.11 遍历次序不唯一

邻接表中邻居排列不同，DFS 树、BFS 父亲与拓扑序可能不同。检查正确性应验证层数、依赖关系与分量，而非要求每次打印同一串顶点。

16.12 练习与答案

练习 1。 有向边 A→B、A→C、B→D、C→D，给出两个拓扑序。再加 D→A 后，Kahn 算法发生什么？

解答：A B C D 与 A C B D 均可。加 D→A 以后，每个点入度至少 1：A 来自 D，B、C 来自 A，D 来自 B、C，初始入度 0 队列为空，输出 0 点，检测到环。不能因 A 原先是源点便继续把 A 强行放队列。

练习 2。 无向链 A-B-C，根选 B。B 有几个 DFS 树孩子？A、C 的 low 与割点结论是什么？

解答：B 分别搜索 A、C，两者互不连通，各为一个树孩子，共 2 个，所以 B 是割点。A、C 无返回祖先的其他边，low 等于自身 dfn；两条树边都满足孩子 low 严格大于父 dfn，是桥。根判定不能使用非根的“有一个孩子 low≥dfn”条件，否则单个孩子的根也会被误判。

练习 3。 BFS 若在出队时才标记，结果距离一定错误吗？为什么仍不推荐？

解答：某些配套实现仍可最终得到正确距离，但会重复入队，若反复覆盖 parent 或 dist 可能产生错误，复杂度保证也更难保持。标准实现发现时就标记，使每点只入队一次，证明与代码都更直接。要评价一个非标准版本，应读完它处理重复条目的全部规则，而不能只凭一行断言必错或必对。

17 最短路和最小生成树，究竟在最小化什么

从宿舍到图书馆，要让这一次走的路尽量短，是最短路径。给所有宿舍铺网络，要让整个连通网络的总线长尽量短，是最小生成树。前者关心从一个源点到目标的路径长度，后者关心选出的全部边的总权重。它们都可能用贪心与优先级队列，但不能互换答案。

以 A-B 权 2、A-C 权 3、B-C 权 2 为例，从 A 出发的最短路树会保留 A-B、A-C，总权 5；最小生成树选 A-B、B-C，总权 4，却让 A 到 C 的树上路径长 4，大于原图直接走 3。这个三点例子足以区分两种目标。

17.1 松弛：当前找到的路还能不能变短

距离数组 dist[v]记录目前已知从源点 s 到 v 的一条路径长度上界。初始 dist[s]=0，其他为正无穷。若有边 u→v、权 w，且 dist[u]+w<dist[v]，就更新 dist[v]并令 parent[v]=u，这个动作叫松弛。

它不会凭空低估答案，因为每次新距离都来自一条真实可走路径；但尚未发现的更短路可能让当前值过大。各种最短路算法的差别，在于按什么顺序做松弛，以及什么时候可以宣布某个值不会再变。

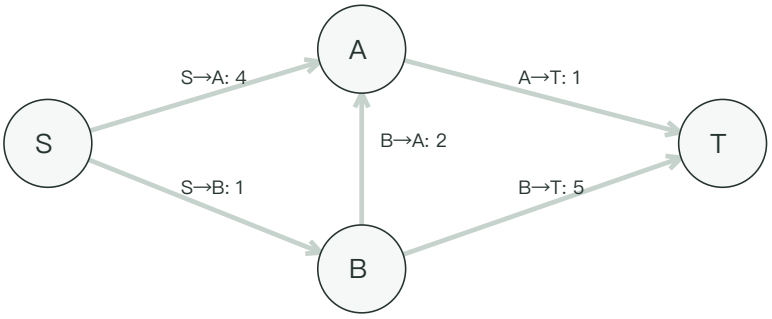
17.2 例 1：Dijkstra，每次拿出当前最小距离

使用有向边 S→A 权 4、S→B 权 1、B→A 权 2、B→T 权 5、A→T 权 1，源点 S。S 是起点，T 是目标，A/B 是中间点。非负权是这个算法的前提。

逐步图显示每条有向边、距离与前驱；下方表格另列惰性堆的入出与过期记录。距离记作 d，与正文 dist 同义；前驱记录新路径进入该点之前所在的顶点。

Dijkstra：一次只检查一条出边

1. 初始化距离



点	距离d	前驱
S	0	无
A	∞	无
B	∞	无
T	∞	无

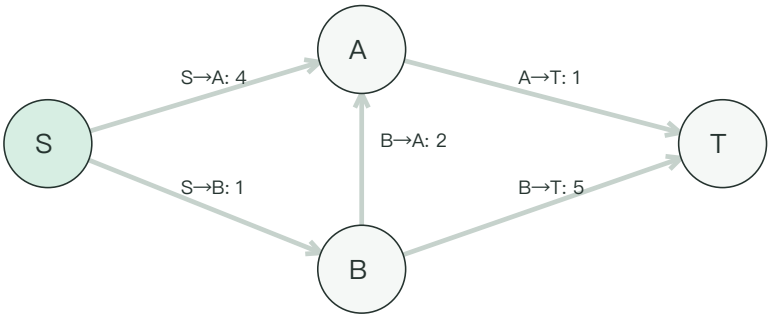
绿色＝距离已确定；边标签明确写出方向。

d[S]=0，其他点为∞，确定集合为空。
原因：∞表示目前还没有找到从S到该点的路径。

图 143　逐步演示：Dijkstra：一次只检查一条出边

Dijkstra：一次只检查一条出边

2. 取出未确定点中距离最小者



点	距离d	前驱
S	0	无
A	∞	无
B	∞	无
T	∞	无

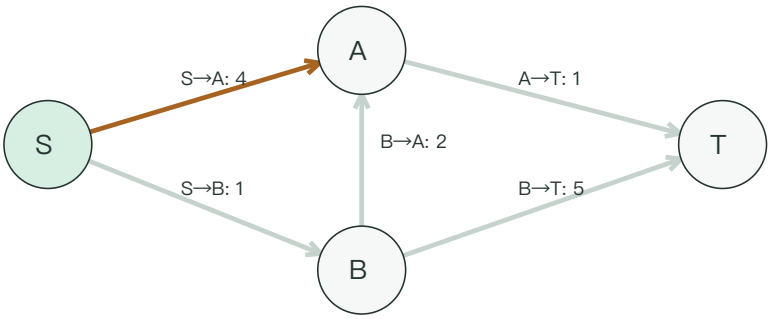
绿色＝距离已确定；边标签明确写出方向。

选S，d[S]=0，将其标绿。
原因：本图边权都非负，绕经尚未确定点的路径不可能把这个最小值变小。

图 144　Dijkstra：一次只检查一条出边，第 2 步

Dijkstra：一次只检查一条出边

3. 检查一条出边并松弛



点	距离d	前驱
S	0	无
A	4	S
B	∞	无
T	∞	无

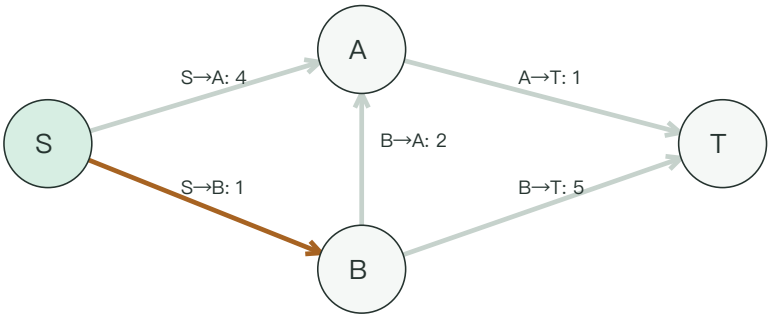
绿色＝距离已确定；边标签明确写出方向。

d[A]从∞改为0+4=4，前驱改为S。
原因：尝试把当前到S的路径后面接上S→A；只保留更短的候选。

图 145　Dijkstra：一次只检查一条出边，第 3 步

Dijkstra：一次只检查一条出边

4. 检查一条出边并松弛



点	距离d	前驱
S	0	无
A	4	S
B	1	S
T	∞	无

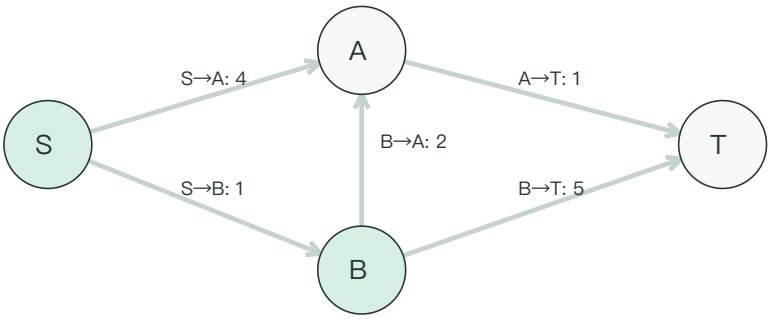
绿色＝距离已确定；边标签明确写出方向。

d[B]从∞改为0+1=1，前驱改为S。
原因：尝试把当前到S的路径后面接上S→B；只保留更短的候选。

图 146　Dijkstra：一次只检查一条出边，第 4 步

Dijkstra：一次只检查一条出边

5. 取出未确定点中距离最小者



点	距离d	前驱
S	0	无
A	4	S
B	1	S
T	∞	无

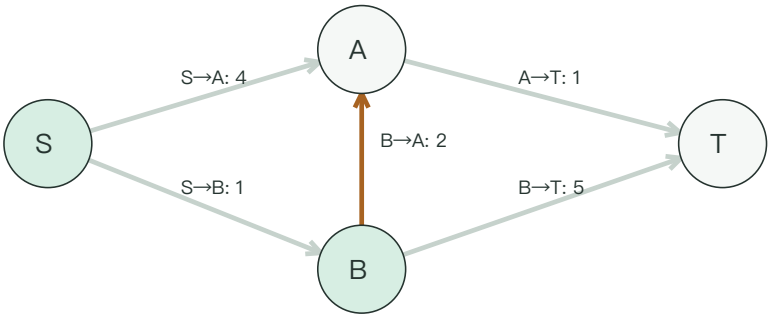
绿色＝距离已确定；边标签明确写出方向。

选B，d[B]=1，将其标绿。
原因：本图边权都非负，绕经尚未确定点的路径不可能把这个最小值变小。

图 147　Dijkstra：一次只检查一条出边，第 5 步

Dijkstra：一次只检查一条出边

6. 检查一条出边并松弛



点	距离d	前驱
S	0	无
A	3	B
B	1	S
T	∞	无

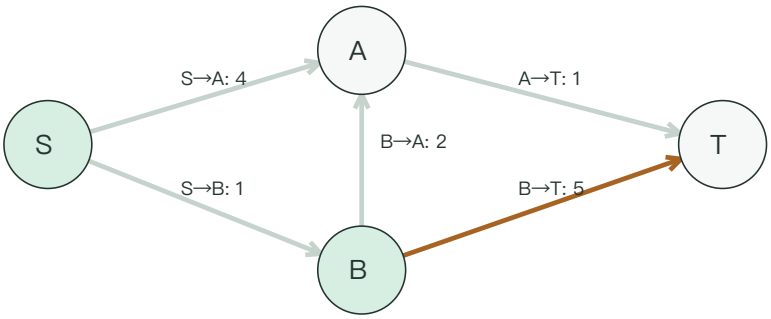
绿色＝距离已确定；边标签明确写出方向。

d[A]从4改为1+2=3，前驱改为B。
原因：尝试把当前到B的路径后面接上B→A；只保留更短的候选。

图 148　Dijkstra：一次只检查一条出边，第 6 步

Dijkstra：一次只检查一条出边

7. 检查一条出边并松弛



点	距离d	前驱
S	0	无
A	3	B
B	1	S
T	6	B

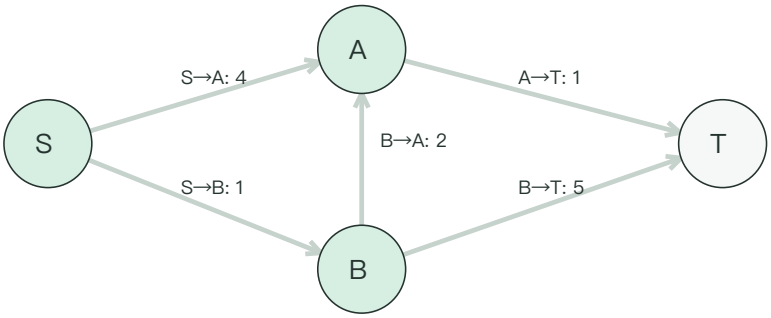
绿色＝距离已确定；边标签明确写出方向。

d[T]从∞改为1+5=6，前驱改为B。
原因：尝试把当前到B的路径后面接上B→T；只保留更短的候选。

图 149　Dijkstra：一次只检查一条出边，第 7 步

Dijkstra：一次只检查一条出边

8. 取出未确定点中距离最小者



点	距离d	前驱
S	0	无
A	3	B
B	1	S
T	6	B

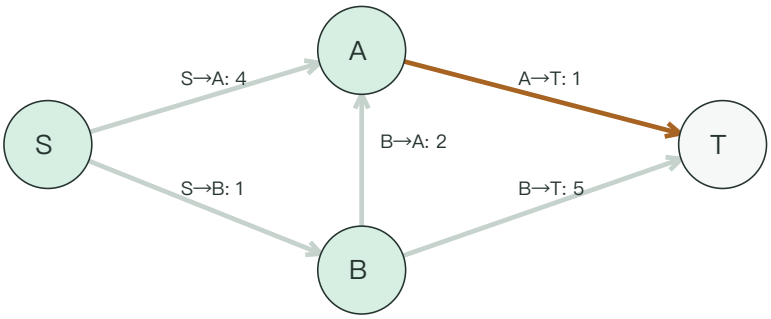
绿色＝距离已确定；边标签明确写出方向。

选A，d[A]=3，将其标绿。
原因：本图边权都非负，绕经尚未确定点的路径不可能把这个最小值变小。

图 150　Dijkstra：一次只检查一条出边，第 8 步

Dijkstra：一次只检查一条出边

9. 检查一条出边并松弛



点	距离d	前驱
S	0	无
A	3	B
B	1	S
T	4	A

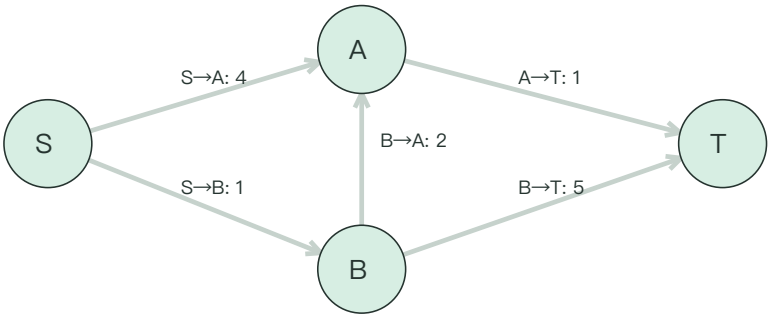
绿色＝距离已确定；边标签明确写出方向。

d[T]从6改为3+1=4，前驱改为A。
原因：尝试把当前到A的路径后面接上A→T；只保留更短的候选。

图 151　Dijkstra：一次只检查一条出边，第 9 步

Dijkstra：一次只检查一条出边

10. 取出未确定点中距离最小者



点	距离d	前驱
S	0	无
A	3	B
B	1	S
T	4	A

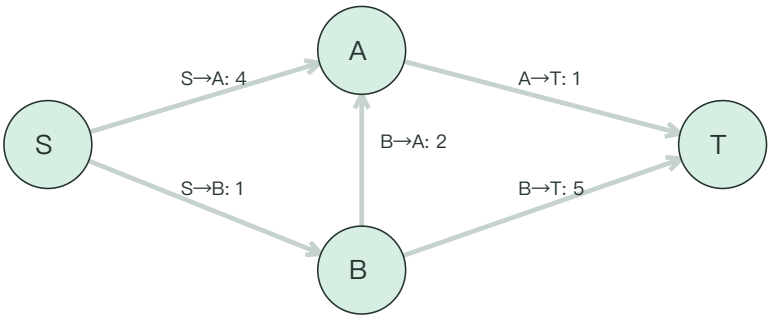
绿色＝距离已确定；边标签明确写出方向。

选T，d[T]=4，将其标绿。
原因：本图边权都非负，绕经尚未确定点的路径不可能把这个最小值变小。

图 152　Dijkstra：一次只检查一条出边，第 10 步

Dijkstra：一次只检查一条出边

11. 全部确定



点	距离d	前驱
S	0	无
A	3	B
B	1	S
T	4	A

绿色＝距离已确定；边标签明确写出方向。

最短路径S→B→A→T长度为1+2+1=4。
原因：每次改距离也应记录前驱；逆向读前驱才能恢复具体路径。

图 153　Dijkstra：一次只检查一条出边，第 11 步

绿色点的距离已经确定，白色点尚待确定，橙色有向边是本步正在检查的边。右侧表同时保存距离和前驱。例如松弛 B⟶A 后，A 的前驱由 S 改成 B；S⟶A 原边仍在图中，但不再是当前到 A 的最佳候选路径的最后一条边。

弹出或处理	计算	dist[A],dist[B],dist[T]	懒堆内容，按距离列出
初始	dist[S]=0	∞,∞,∞	(0,S)
弹(0,S)，看 S⟶A	0+4<∞	4,∞,∞	(4,A)
同一 S，看 S⟶B	0+1<∞	4,1,∞	(1,B),(4,A)
弹(1,B)，看 B⟶A	1+2<4	3,1,∞	(3,A),(4,A 旧)
同一 B，看 B⟶T	1+5<∞	3,1,6	(3,A),(4,A 旧),(6,T)
弹(3,A)，看 A⟶T	3+1<6	3,1,4	(4,A 旧),(4,T),(6,T 旧)
弹(4,A 旧)	4 不等于 dist[A]=3	不改	(4,T),(6,T 旧)
弹(4,T)	T 无出边	不改	(6,T 旧)
弹(6,T 旧)	6 不等于 dist[T]=4	不改	空

这里相同距离时约定 A 旧记录先出，换成 T 先出也不影响答案。每次先核对弹出值是否过期，过期记录不再扫描出边。最后从 T 沿 parent 倒走 T、A、B、S，反转得到 S⟶B⟶A⟶T，总长 1+2+1=4。若只要 T 的最短路，可以在有效的(4,T)出堆时结束；在首次把 T 放入堆、距离还是 6 时不能结束。

为什么 u 可以确定？假设还有一条更短路。沿那条路从源走，取第一个尚未确定的点 y，其前驱 x 已确定；处理 x 时就已给 y 一个不大于这条路前缀的候选。边权非负，前缀长度不大于整条路，因此 dist[y]会小于当前 dist[u]，与 u 最小矛盾。证明用到了非负边权，负边会破坏这个前缀比较。

用邻接表和支持 decrease-key、每顶点至多一条记录的二叉堆，时间 O((V+E)log(V+1))，空间 O(V+E)。不用堆而每次扫描全部顶点找最小，时间 $O(V^2+E)$，稠密图中也可能合适。若堆不支持 decrease-key，可以每次更新后再插一条`(新距离,v)`，出堆时发现距离不是 dist[v]就丢弃旧条目；不能把过期条目再次当作确定事件。这个懒堆可能同时保存 O(E)条新旧记录，一般界可写 O(V+E log(E+2))，堆空间 O(E+1)；对简单图 $E \le V^2$时可化到常见的 log V 量级，允许任意多平行边时不能无条件删掉 E 的对数。这个总账可直接数：初始化 dist 读写 V 项；每个有效出堆顶点的出边只扫一次，共 E 条；每条边至多导致一次成功松弛，也就至多增加一条堆记录。堆中最多 E+1 条记录，每条入堆和出堆各沿 O(log(E+2))高度路径，总计 O(V+E log(E+2))。过期条目只付出堆成本，不再扫描它对应顶点的邻接表。

17.3 例 2：一个负边就能让 Dijkstra 的确定过早

边 A⟶B 权 2、A⟶C 权 5、C⟶B 权−4。Dijkstra 先确定 A，再选距离 2 的 B；但真正最短路 A-C-B 长 1。错误出在“距离 2 的 B 不可能再改善”的判断，而不是松弛公式。

含负边而没有相关负环时，可以用 Bellman–Ford。每轮扫描所有边做松弛，经过 k 轮以后，所有使用至多 k 条边的最短路径都已被考虑，理由是将最后一条边拆出来归纳。若最短路可选为简单路径，最多 V−1 条边，所以 V−1 轮足够。再做一轮若仍能降低某个从源可达的距离，则存在源可达负环。

负环不是对所有目的地都让答案无穷小。只有从源可达该负环，并能从该负环再到达目标，目标的最短路才可通过反复绕环无限下降。图中完全不可达的负环不影响当前源点可达范围的有限答案。

Bellman–Ford 时间 O(VE)，可在整轮无更新时提前结束，但最坏界不变。SPFA 是队列式调度优化，把距离刚变小、需要重新检查出边的顶点放进队列，通常用 inQueue 避免同一顶点同时排队多次。它改变工作顺序，不改变松弛定义；最坏仍可达到 O(VE)，不能给普遍线性保证。下文在同一 Bellman–Ford 输入上逐次执行。

17.4 DAG 与全点对：结构能换一种顺序

有向无环图可按拓扑序处理，每到一个点，其所有前驱都已处理，直接松弛出边即可。允许负边，时间 O(V+E)，因为没有环，无需反复回头。若图不是 DAG，拓扑顺序不存在，不能只取某个 DFS 序代替。

具体取 S⟶A4、S⟶B5、A⟶B−2、B⟶C3、A⟶C10。拓扑序 S,A,B,C。处理 S 后 A4/B5；处理 A 后 B 降 2、C 暂 14；处理 B 后 C 降 5；处理 C 结束。每个前驱都已经完成，因此到 B 时它不可能再被一个尚未处理的前驱改善。整个过程不需要最小堆，负边 A⟶B 也不妨碍这一顺序。

Floyd–Warshall 维护 D[i][j]，逐步允许编号更小的顶点作中间点。加入中间点 k 后，最短路要么不用 k，保持 D[i][j]；要么经过 k，候选 D[i][k]+D[k][j]。因此 `D[i][j]=min(D[i][j],D[i][k]+D[k][j])`。k 必须在最外层，确保读到的状态具有正确的“允许中间点集合”含义。时间 $\Theta(V^3)$、空间 $\Theta(V^2)$，可发现 D[i][i]<0 的负环现象。

正无穷加权时要防溢出，不能拿最大整数加正数后绕成负数。若距离使用浮点数，相等与误差规则也应根据题意谨慎处理。

17.5 最小生成树：无向连通图，选 V−1 条边连通全部点

生成树不含环，并覆盖所有顶点。最小生成树 MST 在所有生成树中使边权总和最小。边权可以为负，MST 不受 Dijkstra 的非负要求限制；若图不连通，得到的是最小生成森林，不能承诺一棵覆盖所有点的树。

割把顶点分成两组。设已选边集合 A 能包含在某棵 MST 中，且 A 没有边跨过当前割，称该割尊重 A。取割上最轻边 e，它可以安全加入：先取一棵包含 A 的 MST T，若 T 已有 e，结论直接成立；否则加 e 成环，环上必有另一条跨割边 f。因为割尊重 A，f 不在 A 中，所以删 f 不会破坏已选集合。新树仍包含 A 与 e，且 $w(e)\le w(f)$，成本不增，因此仍是 MST。这个交换论证同时保住“最优”与“之前已选边”，是 Prim 与 Kruskal 的共同基础。

17.6 例 3：Prim 只关心跨出当前树的边

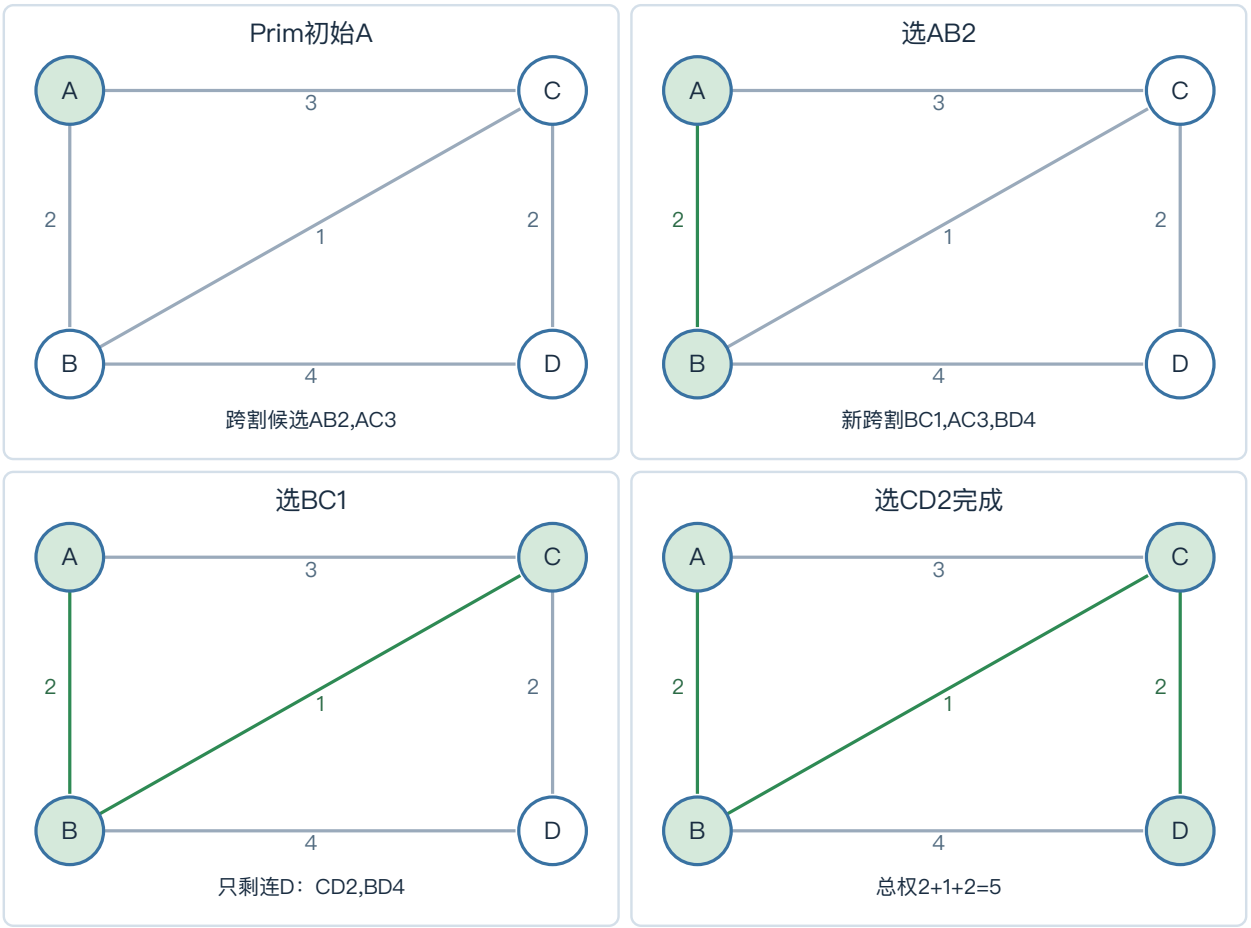


图 154　Prim 逐次扩张当前树与跨割候选

图中绿色点属于当前树，候选只能一端绿、一端白。两个绿点之间的旧边已经在树内部，不再是连接新点的候选。

无向图边 A-B2、A-C3、B-C1、B-D4、C-D2。从 A 开始，当前树{A}，候选 A-B2、A-C3，选 A-B2。现在树{A,B}，跨割候选 B-C1、A-C3、B-D4，选 B-C1。树{A,B,C}，到 D 的最轻是 C-D2，完成，总权 5。

Prim 对未进入树的顶点 v 保存 key[v]：它与当前树之间最轻连接边的权重。Dijkstra 的 dist[v]则是从固定源点走到 v 的路径总长。上例 C 加入前 Prim key[C]=1，而从 A 经 B 到 C 的路径长是 3；若把松弛写成 `key[u]+w`，就把 Prim 改成了另一种问题。

使用堆时成本 O(E log V)量级，稠密图用矩阵与线性选点可 O(V²)。初始起点可以任意，多个最优树可能不同，但最小总权一致。

17.7 例 4：Kruskal 与并查集，避免选边绕成环

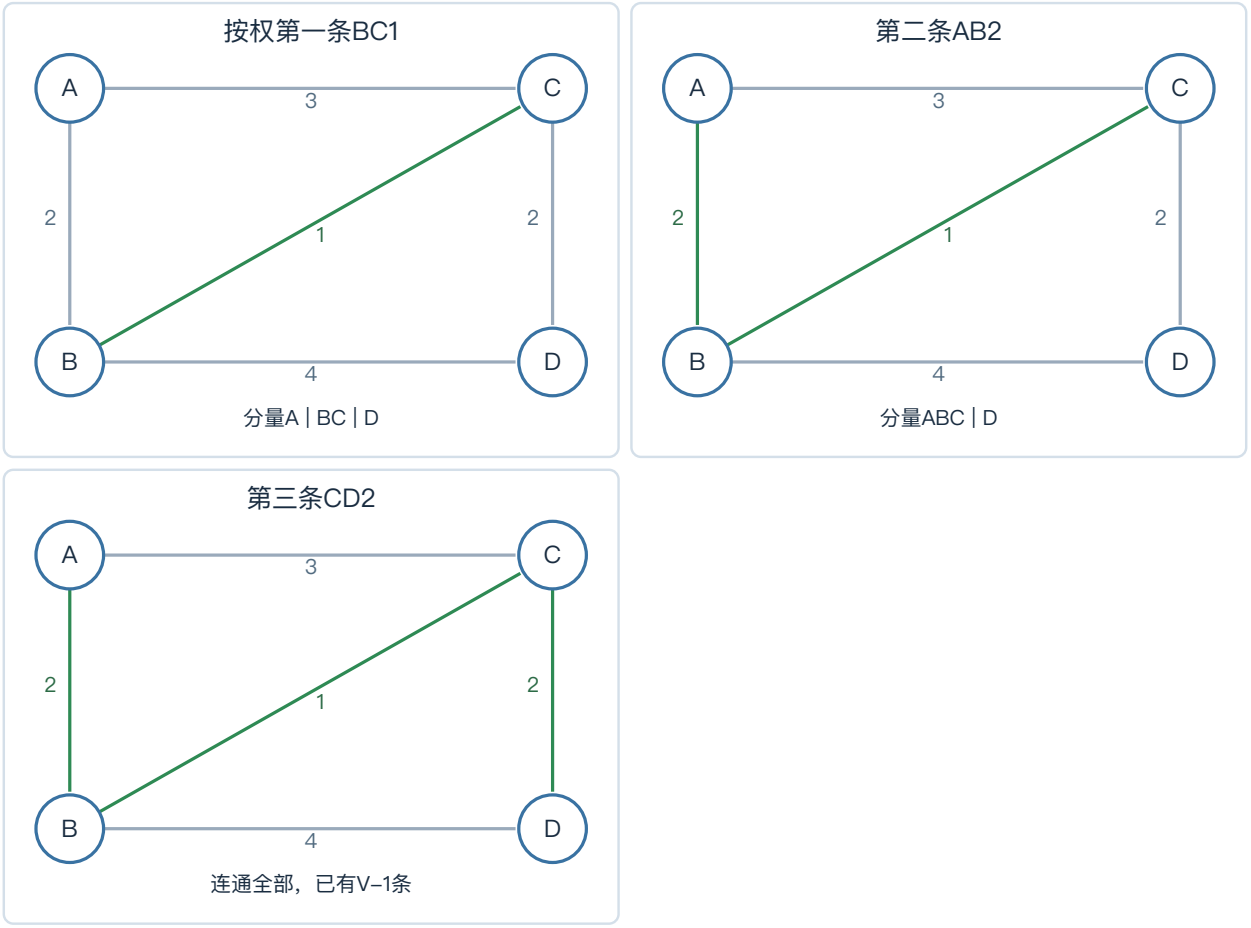


图 155　Kruskal 按全局边权合并不同连通分量

Kruskal 不要求已选边始终是一棵树，它开始是一片森林；每次只要两端当前不连通，就能合并两棵树。图中绿边是已选边，而白点仍然属于某个并查集分量。

将同一图的边按权排为 B-C1，A-B2，C-D2，A-C3，B-D4。先选 B-C，分量为{B,C}；选 A-B，分量变{A,B,C}；选 C-D，所有点连成一体，已选 3=V−1 条，完成。

若继续看到 A-C，两端已属同一分量，加上会成环，跳过。并查集用 find 判定分量代表，用 union 合并分量。按秩或大小合并加路径压缩后，m 次操作有近线性的逆 Ackermann 摊还界；不必把这个极慢增长函数当真正常数，常写 O(m α(V))。

排序通常 O(E log E)，其余并查集成本较小。对于简单无向图 E≤V²，可写 O(E log V)同阶上界；若允许任意多平行边，应保留 E 的表达更稳妥。重边可以被算法自然处理，自环永远不能帮助生成树连通两个分量。

17.8 权重相等与“必须选”的区别

割上唯一最轻边会出现在每一棵 MST 中；若最轻边有并列，只能保证存在某棵 MST 包含其中某条，不能说每条都必选。环上唯一最重边不属于任何 MST；若最重并列，也不能随意断言每条都不出现。

例如三角形三边全为 1，任取两条都是 MST。没有一条边是唯一必须选的，但每个算法仍能稳定得到某棵最优树。证明中的“存在”与“所有”不能互换。

17.9 Bellman–Ford 逐轮表，证明的状态与原地实现稍有不同

用同步数组演示：每轮只从上一轮 dist 读取，写入新数组，状态恰好代表至多 k 条边的最短路。边 S→A 权 4、S→B 权 5、A→B 权−2、B→C 权 3、A→C 权 10。初始 S0，其余无穷；第 1 轮 A4、B5、C∞；第 2 轮 B 降 2、C 降 8；第 3 轮 C 降 5；第 4 轮无变化。最短到 C 为 S-A-B-C，长 5，使用 3 条边。

用表核对同步版本：

至多允许边数 k	S	A	B	C	C 当前可用路径
0	0	∞	∞	∞	无
1	0	4	5	∞	无
2	0	4	2	8	S-B-C
3	0	4	2	5	S-A-B-C
4	0	4	2	5	不再更新

SPFA 按队列处理同一输入：初始[S]；处理 S 使 A4、B5，队列[A,B]；处理 A 使 B2、C14，B 已在队中，不再重复入队，得到[B,C]；处理 B 使 C5，C 已在队，得到[C]；处理 C 没有出边，队列空，结束。若 C 已经出队后又被改善，它必须重新入队，否则更短距离无法向它的后继传播。这个例子没有可达负环，只说明调度如何做，不据此推断所有图都只处理每点一次。若输入可能有可达负环，需要负环检测，否则队列可以永远有新改善；前述限定轮数的 Bellman–Ford 可稳妥地检测这种情况。

原地扫描可以在同一轮中连续传播多条边，因此某个顶点可能比同步表更早变好；“k 轮以后至少已涵盖至多 k 边路径”的保证仍成立，但“恰好只含这些路径”就不一定成立。读证明时用同步版本最清楚，实现原地版本时不要拿轮号当精确边数。

负环例子 A→B 权 1、B→C 权−3、C→A 权 1，绕一圈总−1。只要源可达 A，反复绕圈可让 A、B、C 距离持续下降；第 V 轮仍有松弛，反映不存在有限最短值。若要输出受负环影响的所有顶点，可从仍可更新的点再沿有向边传播标记，不能只标记环上一个被更新点。

17.10 Floyd 的一格更新，允许中间点集合必须一致

三点 A,B,C，初始 A→B 为 3，B→C 为 2，A→C 为 9，对角 0，其他无边无穷。允许 A 作中间点暂不改善；允许 B 时，D[A][C]取 min(9,3+2)=5；允许 C 时保持 5。D[A][C]由两段都只允许先前中间点的路径拼成，正好覆盖经过 B 的候选。

按行、列都为 A,B,C 写完整矩阵，便能检查更新改了哪一格：

$$D^{(0)}=\begin{pmatrix}0&3&9\\ \infty&0&2\\ \infty&\infty&0\end{pmatrix},\qquad D^{(A)}=D^{(0)}.$$

$$D^{(B)}=\begin{pmatrix}0&3&5\\ \infty&0&2\\ \infty&\infty&0\end{pmatrix},\qquad D^{(C)}=D^{(B)}.$$

上标表示已允许的中间点，不是矩阵乘方。加入 B 那轮只把 A 到 C 从 9 改为 5；A 到 B 的第一跳仍 B，故 next[A][C]改为 next[A][B]=B。输出 A→B→C，而不是只输出中间点 B。

为什么 k 放最外层？状态定义是“当前统一允许哪些中间点”。如果 i、j 在外，某些格已经用了编号较大的点，另一些格还没用，同一轮读取的信息不再对应同一个限制集合。个别小图即使错序也算对，不能据此证明该循环顺序正确。

恢复路径可存 next[i][j]，表示从 i 去 j 的第一跳；经 k 改善时赋 next[i][j]=next[i][k]。打印时从 i 反复跳向目标，若无路径要先判断，若存在相关负环也不能当有限最短路无限打印。距离值和路径证据需要同步更新。

17.11 并查集合并后，代表编号不代表最小顶点

集合{A,B}与{C,D}合并，按大小可能把 C 挂在 A 下，也可能相反；find 返回的是内部根，不承诺字典序最小元素。若题目要查询分量最小编号，必须额外在根保存 min 字段并随 union 合并。

路径压缩把访问链上的节点直接连到根，只改变内部表示，不改变集合划分。Kruskal 只关心两端是否同集合，因此可安全使用。若还维护距离、奇偶关系或历史回滚，简单压缩规则可能不再足够，额外信息也要随父关系变化调整，不能直接拿普通并查集复制过去。

17.12 零权边与相等距离，不需要禁止

Dijkstra 允许零权边，证明只要求路径前缀不会比整条路径更长，不要求严格增加。多个点距离相同，可以任意顺序确定；如果还要求某种字典序最小路径，需要额外处理并列，而非假设最小堆自然给出全部路径规则。

无穷距离的点从堆里永不出现，代表源不可达；矩阵扫描版本若选出的最小值仍为无穷，可停止当前源搜索。不能把无穷当普通大数去松弛，否则溢出或哨兵相加可能伪造一条很短的路径。

恢复最短路径时，从目标沿父链接到源，再反转。不可达目标没有这条父链，应报告无路，而非打印一个未初始化的父编号。

同距时不随意覆盖父链。

17.13 练习与答案

练习 1。边 S→A 权 10、S→B 权 1、B→A 权 1。Dijkstra 中 A 可能在堆里出现几次，怎样避免旧条目干扰？

解答：处理 S 时插(10,A)，处理 B 后更新 dist[A]=2，再插(2,A)。先出(2,A)并正确处理；以后出(10,A)时发现 10≠当前 dist[A]，直接跳过。不能每次出堆都把旧值重新写回 dist，否则会把正确距离覆盖成更大值。

练习 2。三点边权 A-B 为−2、B-C 为 1、A-C 为 5，MST 是什么？存在负边是否导致无法求解？

解答：选 A-B 与 B-C，总权−1。生成树只选两条且不成环，负边不会造成反复绕圈的问题。最短路允许重复走路时负环会导致无下界，生成树不允许环，是另一个优化集合。

练习 3。无向图上有一条负权边，若把每条无向边理解成两条有向边用于最短路，会怎样？

解答：沿该边过去再回来，总权为负的两边回路，可重复降低路径长度。因此普通允许重复边的最短路会出现可达负环。若题目要求简单路径、不能重复顶点，则定义不同且问题性质改变，不能继续直接套普通最短路模型。

阅读对照：Princeton 最短路与官方 DijkstraSP 使用可修改键值的索引堆；本章完整追踪的是允许过期条目的懒堆，空间与对数因子的账要相应保留。最小生成树说明可对照割与环的交换性质。

18 堆：只保证冠军站在最上面

任务队列要不断插入新任务，并取出优先级最高者。每次完整排序做了很多不需要的工作；只用无序数组则每次找最高都要扫描。堆维持一种较弱的顺序：父节点优先级不低于孩子，因此最高项在根，但左右子树之间不必有序。

优先级队列是接口，可以用多种结构实现；二叉堆是常见实现。最大堆取最大，最小堆取最小。本章以最大堆为主，切换到最小堆时所有比较方向都要一起反转。

18.1 完全二叉树的形状，让链接变成下标算式

堆按层序紧凑存在数组里，节点 i 左孩子 2i+1、右孩子 2i+2，父为 $\lfloor (i-1)/2 \rfloor$，根下标 0。最后一层必须从左到右填满，形状由 size 唯一确定，所以无需为每节点另存孩子指针。

数组 `[9,6,8,2,4,7]` 是合法最大堆：9≥6、8；6≥2、4；8≥7。但数组并未有序，8 在 6 后面，7 在 2、4 后面。若要列出所有项从大到小，仍需反复删除最大或另做排序。

18.2 例 1：插入，只有一条向上路径可能出错

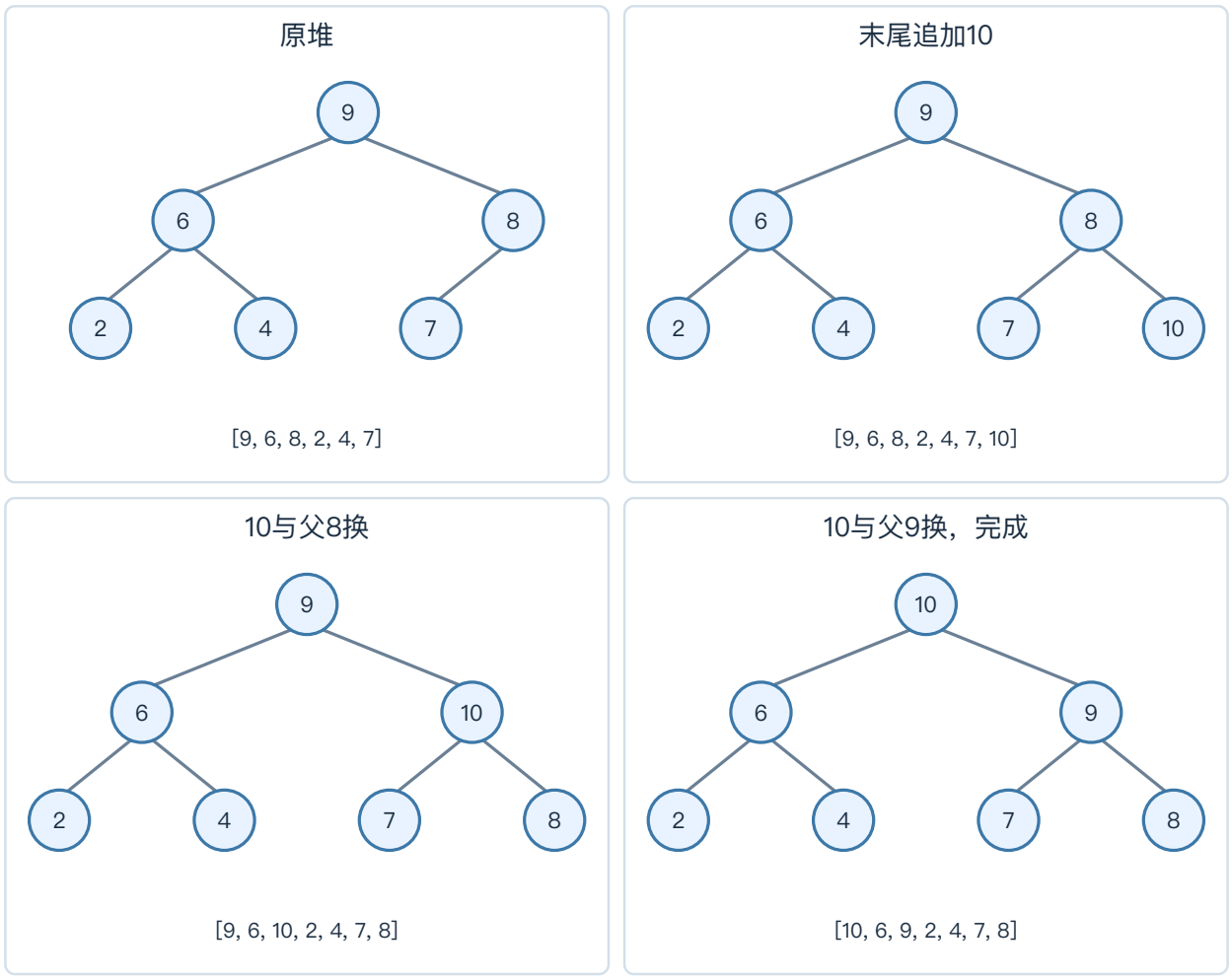


图 156　最大堆插入 10 时每一次上滤交换

图下方的数组与树的层序一一对应。先根据数组下标确认父子，再比较键；同一层左右两项大小不同不需要交换。

在上述堆插 10，先放到末尾下标 6，形状仍完全，但 10 可能大于父 8。与 8 交换，数组变 `[9,6,10,2,4,7,8]`；10 再与父 9 交换，变 `[10,6,9,2,4,7,8]`，到根结束。

为什么不检查其他路径？新项之外所有旧父子关系未变；每次把较小父项向下移到新项原位置，该父项仍不小于原来新项的旧兄弟子树，因为它原先就在这些后代之上。唯一尚可能违反的是新项与新父之间。因此沿祖先不断修复即可。

树高 $\lfloor \log_2 n \rfloor$，最多交换这么多次，最坏 O(log n)。如果新值不大于父，只需一次比较，最好 O(1)。底层数组还可能扩容，插入的完整账可以包含向量的摊还扩容成本。

18.3 例 2：删除根，洞口应往哪边走

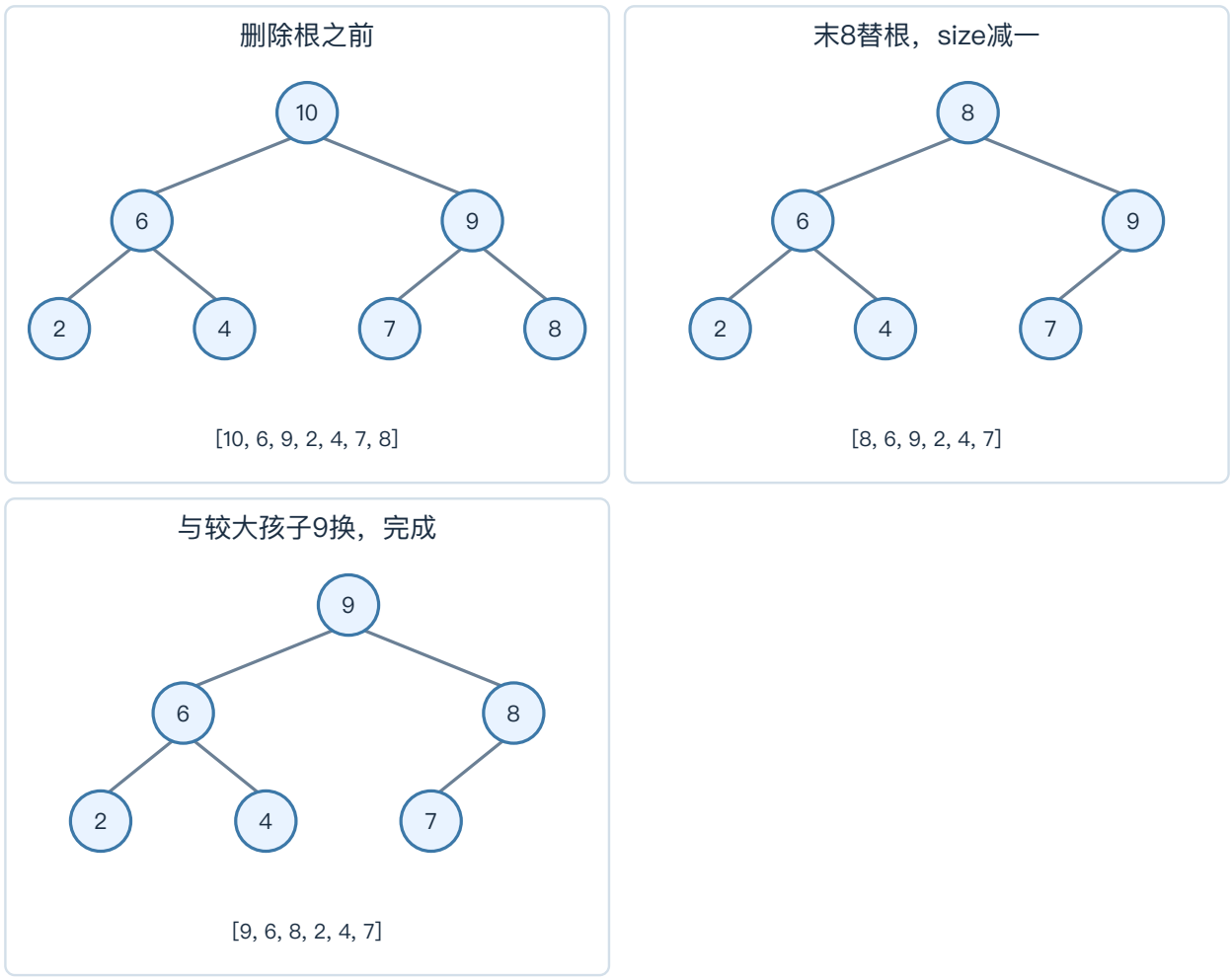


图 157　最大堆删除根时末项替入与较大孩子上升

从 `[10,6,9,2,4,7,8]` 删除最大 10。将末项 8 移到根并缩小 size，得 `[8,6,9,2,4,7]`。8 小于两个孩子中的最大 9，因此与 9 交换，得 `[9,6,8,2,4,7]`；此时 8 只有孩子 7，已不小于 7，结束。

为何必须与较大的孩子交换？若 8 与 6 换，9 仍在根的另一个孩子位置，根 6<9，堆性质立刻失败。选择较大孩子可确保升上来的新父同时不小于两个孩子，只留下被下移项与它的新孩子之间可能冲突。

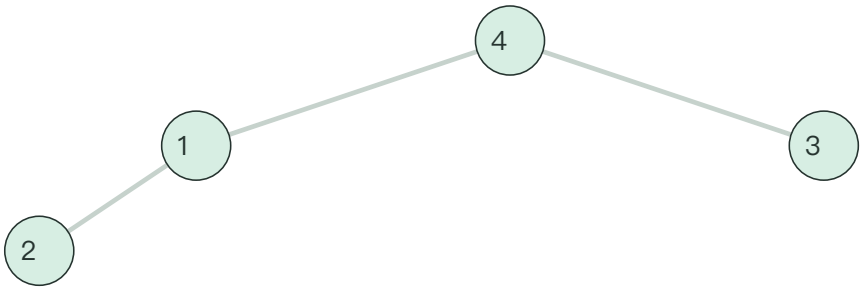
可以用“挖洞”减少交换：保存末项 x，根留下洞；不断把较大孩子提上来，直到适合放 x 的位置，再写入 x。比较逻辑相同，减少了反复交换带来的赋值次数。它仍是最坏 O(log n)，修改常数不改变阶。

18.4 例 3：自底向上建堆，为什么不是 n log n

使用数组[4,1,3,2]。n=4，最后一个非叶下标为 $\lfloor n/2 \rfloor - 1 = 1$。叶下标 2 和 3 本身各是一棵单节点堆，无需处理。逐步图把树和数组放在一起。数组下标从 0 开始，左右孩子下标为 2i+1、2i+2；只有小于当前长度的下标才存在。

最大堆：建堆和删除最大值

1. 建堆前



对应数组

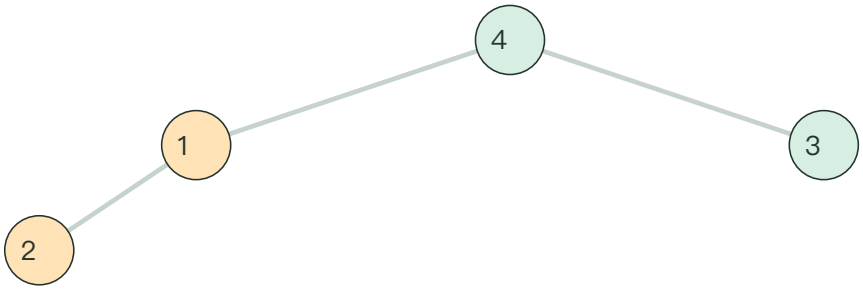
4	1	3	2
0	1	2	3

数组按层序对应图中的完全二叉树。
原因：数组下标i的孩子在2i+1和2i+2，超出长度的孩子不存在。

图 158　逐步演示：最大堆：建堆和删除最大值

最大堆：建堆和删除最大值

2. 选择较大的孩子



对应数组

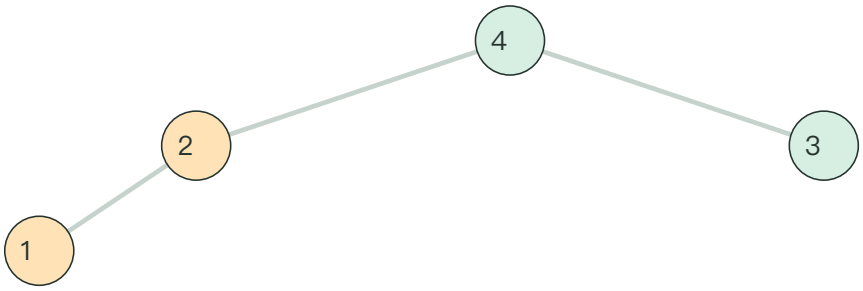
4	1	3	2
0	1	2	3

父下标1的值1，较大孩子下标3的值2。
原因：父若需下移，就与较大孩子交换，才能保证换上来的父不小于另一孩子。

图 159　最大堆：建堆和删除最大值，第 2 步

最大堆：建堆和删除最大值

3. 交换父与孩子



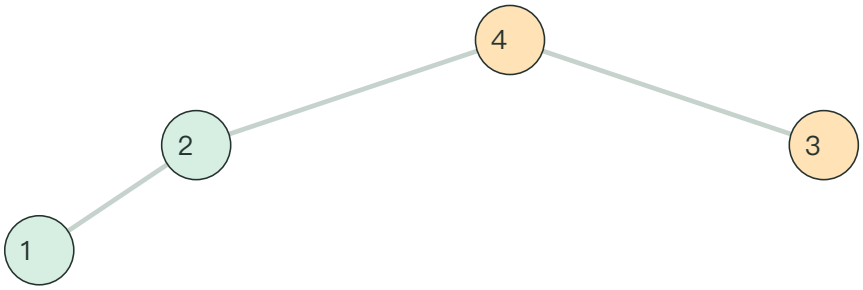


交换下标1和3。
原因：原父值只可能继续在它移入的子树中违背堆序。

图 160　最大堆：建堆和删除最大值，第 3 步

最大堆：建堆和删除最大值

4. 选择较大的孩子



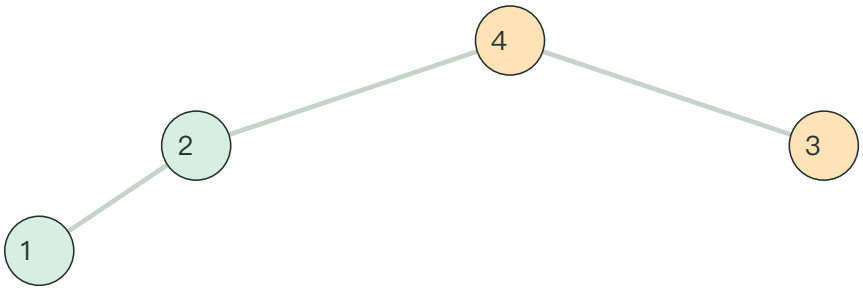


父下标0的值4，较大孩子下标2的值3。
原因：父若需下移，就与较大孩子交换，才能保证换上来的父不小于另一孩子。

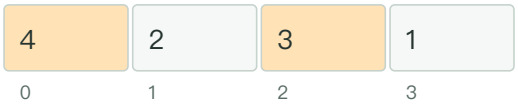
图 161　最大堆：建堆和删除最大值，第 4 步

最大堆：建堆和删除最大值

5. 这一子树已经满足堆序



对应数组

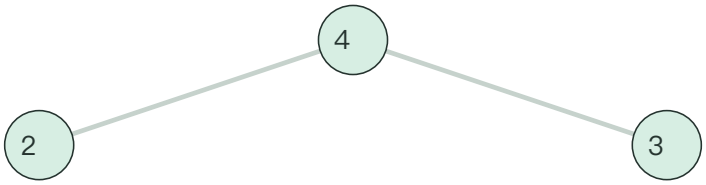


4≥3，停止本次下滤。
原因：两个孩子的子树此前都已成堆。

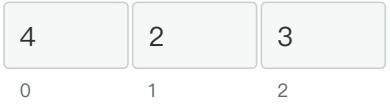
图 162　最大堆：建堆和删除最大值，第 5 步

最大堆：建堆和删除最大值

6. 删除末尾格，暂存其值



对应数组

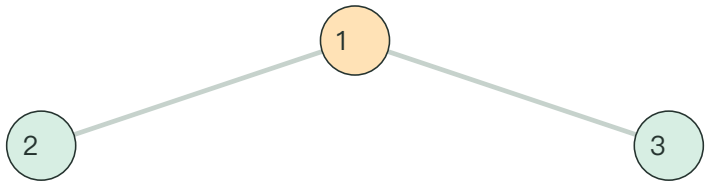


最大值4已经暂存返回；另取末尾值1。
原因：物理删除必须发生在末尾，才能保留完全二叉树的形状。

图 163　最大堆：建堆和删除最大值，第 6 步

最大堆：建堆和删除最大值

7. 末尾值补根



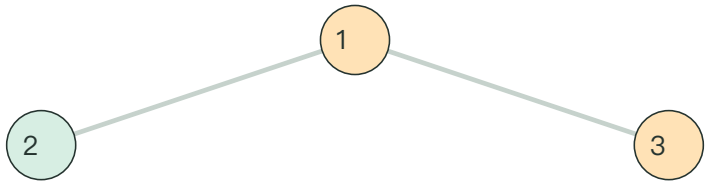


根暂时写为1。
原因：形状正确后，再通过下滤恢复大小关系。

图 164　最大堆：建堆和删除最大值，第 7 步

最大堆：建堆和删除最大值

8. 重新选择较大孩子



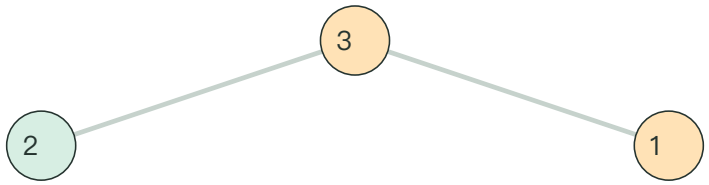


比较父1与孩子3。
原因：仍按较大的孩子决定是否下滤。

图 165　最大堆：建堆和删除最大值，第 8 步

最大堆：建堆和删除最大值

9. 下滤交换



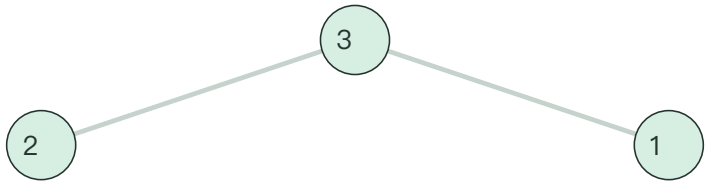


交换下标0与2。
原因：交换后上层满足堆序，继续检查原父值落到的位置。

图 166　最大堆：建堆和删除最大值，第 9 步

最大堆：建堆和删除最大值

10. 删除最大值完成





返回4，剩余堆为[3, 2, 1]。
原因：形状与堆序都已恢复；数组整体不必从大到小排列。

图 167　最大堆：建堆和删除最大值，第 10 步

先在 i=1 下滤，值 1 只有左孩子 2，1<2，交换，数组变[4,2,3,1]。i=0 时根 4 大于孩子 2 和 3，不动。到此建堆完成，数组就是[4,2,3,1]，不能因为“i=0 没交换”而把这一步检查漏掉。

图后两幅继续对建好的堆 delMax：返回 4，将末项 1 填根并把 size 从 4 减到 3，得到[1,2,3]；比较左右孩子 2 与 3，选 3 升根，得到[3,2,1]。1 已经到叶，结束。建堆与删除共用下滤，但起点、循环边界和当前有效 size 不同。

再用六项输入核对两层下滤：[3,1,6,5,2,4]，i=2 的 6≥4，不动；i=1 时 1 与 5 换，得[3,5,6,1,2,4]；i=0 时 3 先与 6 换，得[6,5,3,1,2,4]，再与孩子 4 换，得到[6,5,4,1,2,3]。最后一次下滤有两个交换，不是只把最大值放根便完成。

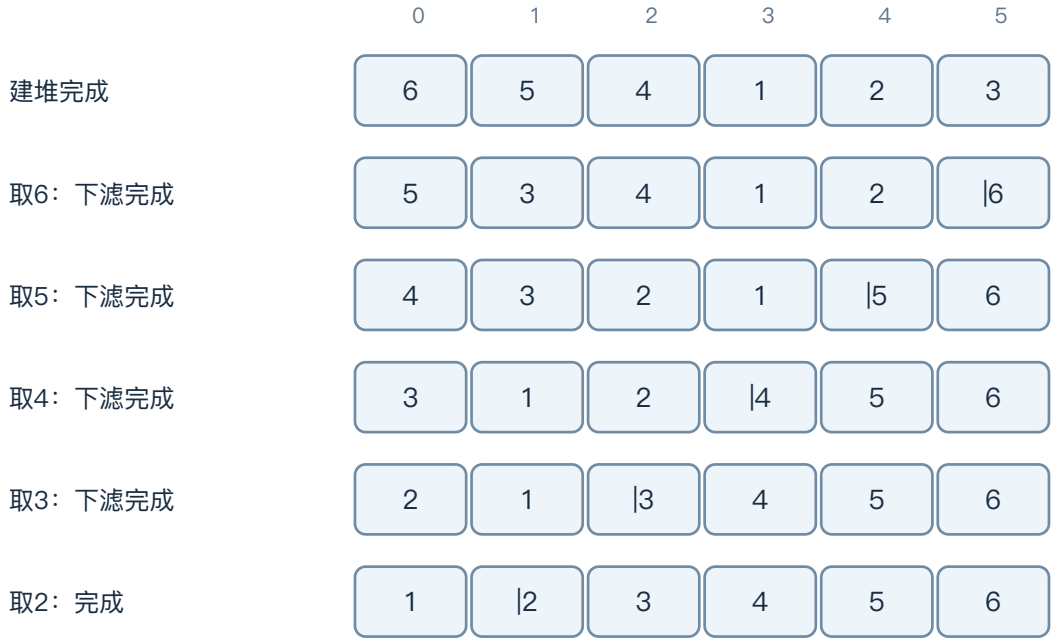
处理 i 时，它的左右子树都已经是堆，所以一次下滤即可使整个 i 子树成堆。叶子不需要下滤。若把每个节点都按最坏 log n 相乘，会得到 O(n log n)的松上界；实际上大多数节点靠近底层，最多只下移一两层。高度至少 h 的节点数约不超过 n/2^h，因此总下移工作可按高度上界求和：

$$\sum_{h\ge 1}\left(O\frac{n}{2^h}\right)=O(n).$$

也可将高度恰为 h 的节点数乘 h，得到 n 乘收敛级数 Σ h/2^h。所有元素至少要读一次，故建堆为 Θ(n)。这里的关键是按节点实际可能下移的高度记账，而不是所有节点都支付根的成本。

18.5 堆排序：根与末尾交换后，缩小有效堆

最大堆根是当前最大值，与有效区间末项交换，最大值便到最终位置。有效堆 size 减 1，对新根下滤，再重复。数组后缀逐渐变成升序的最终位置，最后得到整体升序。



|右侧是已经到最终位置的有序后缀；下滤只看左侧有效堆

图 168　堆排序每次下滤完成后的有效堆与最终后缀

图中竖线右边已经就位，下滤不能越过它。每一步先把根与有效末项交换，再缩小 size，最后只修复前缀。对[6,5,4,1,2,3]：6 交换到尾部，前 5 项下滤成[5,3,4,1,2]，后缀[6]；再把 5 移到下标 4，前 4 项下滤；持续至剩 1 项。总共 n−1 次删除根，每次 O(log n)，时间 O(n log n)，加上线性建堆仍同阶。额外空间可 O(1)，但通常不稳定，因为根与末项的远距离交换会改变相等记录的相对顺序。

18.6 第 k 大与前 k 大，不必把全部排好

先约定有限输入有 n 项，且 1≤k≤n；k=0 不表示合法名次，k>n 没有第 k 大，应由接口明确报告不存在。数据流尚未读到 k 项时，也只能报告“尚不足 k 项”。维护一个容量 k 的最小堆，保存当前最大的 k 项。读到新值 x，堆未满就插入；满后若 x≤堆顶，它不可能挤进前 k 大，跳过；若 x 更大，替换堆顶再下滤。扫描 n 项时间可统一写 O(n log(k+1))，空间 O(k)。k≥2 时通常简写 O(n log k)，k=1 时仍须扫描全部 n 项，成

本 Θ(n)，不能把 log1=0 代入后误判为零工作。结束时堆顶为第 k 大，其余堆内是前 k 大集合，但未按大小顺序排列。

例如 k=3，输入 5、1、8、3、9。以下列的是实际最小堆数组，不是把集合临时排序后的列表：

读入	操作	堆数组	当前第 3 大
5	追加	[5]	项数不足 3
1	追加并上滤	[1,5]	项数不足 3
8	追加，无需上滤	[1,5,8]	1
3	替根 1，3≤5/8 无需交换	[3,5,8]	3
9	替根 3，与较小孩子 5 换	[5,9,8]	5

最后保存的是前 3 大记录，数组[5,9,8]并未升序。重复值是否分别计入名次，应由题目说明；若要不同值的第 k 大，还需额外去重。

18.7 左式堆：如果主要需求是合并两堆

二叉堆的紧凑数组很适合取极值，但合并两堆通常需要重新组织全部元素。左式堆是一种可合并的堆序二叉树，额外维护空路径长度 npl：空节点记−1，非空节点 `1+min(npl(left),npl(right))`，并要求左 npl 不小于右 npl。

合并两个最小左式堆，先比较根，较小根保留；递归合并它的右子树与另一堆；回来时若左右 npl 次序反了，交换两孩子，再更新 npl。沿右链递归，右链长度 O(log n)，因为 npl 为 r 的树至少有指数于 r 的节点。这里平衡约束的是右链，不要求整树高度 O(log n)，左侧可很深。

插入等价于与单节点堆合并，删除根等价于合并左右堆。最坏合并 O(log(n+m))量级。要处理父指针、内存所有权和重复优先级，仍需与普通树一样认真检查。

18.8 优先级变化，不是改完数值就结束

若最大堆中某节点优先级增加，应上滤；若减小，应下滤。首先需要找到它的数组位置。只给任务 ID，额外映射“ID→当前堆下标”可 O(1)期望或按映射结构相应成本定位；每次堆交换都要同步更新映射。

也可以用懒删除：每次更新插一条新记录，出堆时检查版本是否过期，过期则跳过。Dijkstra 常采用这种方式。它简化修改接口，却让堆里可能暂存多条旧记录，需要计入空间和总操作数。

18.9 左式堆合并，走完所有返回与交换



图 169　两个最小左式堆的完整初始输入

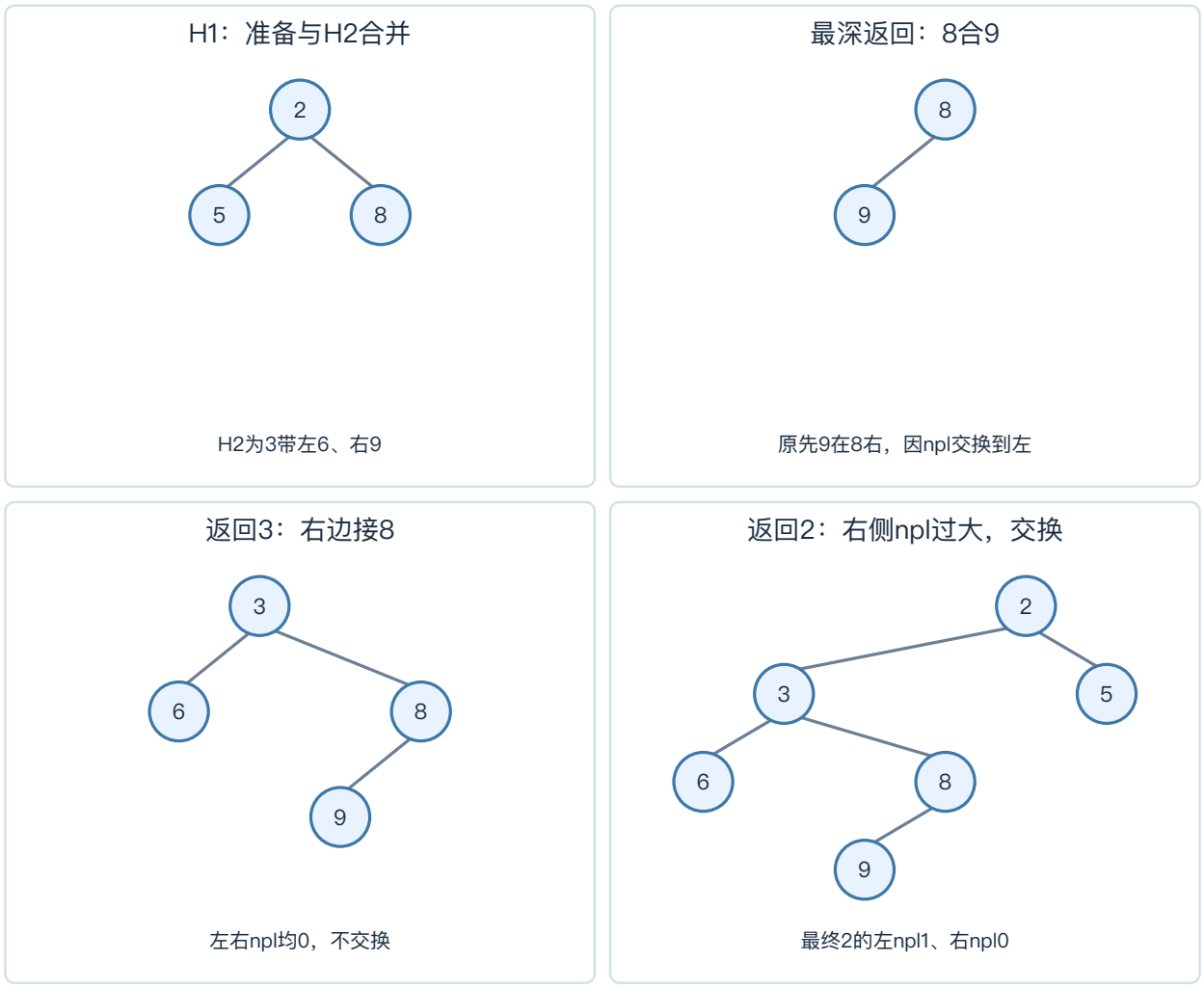


图 170　左式堆递归合并的返回顺序与每次孩子交换

有最小左式堆 H_1，根 2、左叶 5、右叶 8；H_2根 3、左叶 6、右叶 9。两个根比较，保留 2，把它右子树 8 与 H_2合并。8 与 3 比，保留 3，再把 3 的右叶 9 与 8 合并；保留 8，让 9 成为它的右孩子，回来时左空、右非空，交换成 8 左 9、右空。

回到 3，左 6 的 npl 为 0，右 8 的 npl 也为 0，不必换，3 的 npl 为 1。回到 2，左 5 的 npl0，小于右 3 的 npl1，交换左右，最终根 2 左 3、右 5，3 左 6 右 8，8 左 9。每处仍父≤子，而且左 npl≥右 npl。总节点六个，一个也没复制或丢失。

把调用和返回分开看：

调用/返回	保留哪个根	尚待合并	返回时检查
merge(2 树,3 树)	2	merge(8,3 树)	暂不判断 2 的 npl
merge(8,3 树)	3	merge(9,8)	暂不判断 3 的 npl
merge(9,8)	8	merge(空,9)	把 9 暂接到 8 右
merge(空,9)返回	9	无	直接返回现成 9 子树
回到 8	8	无	左空 npl−1<右 9 的 0，交换孩子，8.npl=0
回到 3	3	无	左 6 与右 8 的 npl 同为 0，不换，3.npl=1
回到 2	2	无	左 5 的 0<右 3 的 1，交换，2.npl=1

只有递归返回后才知道新右子树的 npl，所以不能在递归之前盲目换孩子。

npl 为 r 意味着最短到空路径长度为 r+1；左右 npl 至少 r−1，令 L(r)为 npl=r 至少需要的节点数，L(−1)=0；左右子树 npl 都至少 r−1，因此$L(r) \geq 1 + 2L(r-1)$。从叶 r=0 得到 1 个，r=1 至少 3 个，r=2 至少 7 个，归

纳为$L(r) \geq 2^{r+1}-1$。有 n 个节点时，$r+1 \leq \log_2(n+1)$；由于左 npl≥右 npl，节点 npl 恰好等于 1+右孩子 npl，所以沿右边每走一步 npl 减 1，整条右链长至多 r+1=O(log n)。左链可以长，不影响合并只沿右链递归的界。若把 npl 误写成整树高度再强制左高右低，会维护另一种更苛刻结构，不能照搬左式堆证明。

18.10 自底向上建堆的下界，不是所有节点都要下移

线性建堆说总工作 Θ(n)，不表示每个节点都移动一次。有些输入本来已成堆，只需比较确认；约一半节点是叶，无需下滤。但算法至少要读取并纳入全部输入，且最后非叶需要进行相应检查，总工作仍为线性量级。

按高度求和的精细版本：高度至少 1 的约 n/2 个节点，各可能付第一步；高度至少 2 的约 n/4 个，再付第二步；高度至少 3 的约 n/8 个，再付第三步。把每个节点的下移路径分成一层一层的账，所有账合为 n/2+n/4+...<n。这避免了“每个节点都是根”那种重复收取 log n 的松估计。

18.11 优先级相等时的服务次序

堆本身通常不稳定，同优先级任务的出队顺序未必与入队一致。若系统要求同优先级先到先服务，可把比较键设计为(优先级,反向到达序号)供最大堆使用，或最小堆比较(负优先级,到达序号)。要小心序号溢出与比较方向，写两三个同优先级任务测试比只看说明可靠。

例如优先级均为 5 的 A 先入、B 后入，最小堆键(-5,0)、(-5,1)，A 先出。后来高优先级 7 的 C 键(-7,2)，C 先于 A、B，这同时满足主优先级与并列到达次序。额外序号改变比较关系，不改变堆的操作复杂度。

18.12 任意位置删除，末项替换以后可能上也可能下

删除堆中下标 i，不必搬移整个后缀。取末项 x 填入 i 并缩小 size；若 x 比新父更大，最大堆应上滤；否则可能比某孩子小，应下滤。不能无条件只做下滤，因为末项来自另一分支，未必小于 i 的父。

例如最大堆根一百，左分支父五十、其孩子十与二十，右分支父九十、其孩子八十与七十。删左下的十，末项七十填进去，七十大于父五十，必须上滤。堆序只约束祖先后代，不能比较不同分支推断替换方向。

18.13 堆数组下标变化，需要同步任何外部定位表

任务编号到下标的映射若只在插入时写一次，后续上滤交换后便过期。更新某任务优先级时，可能改到另一任务。每次交换数组项，也交换它们在映射中的位置记录；删除时清除被删编号，替入末项也更新它的新下标。

调试可对每个数组下标检查 map[id]恰等于当前位置，再反向检查映射指向的记录编号相同。堆性质正确但映射错误，取最大可能一直对，按 ID 修改却错，必须分别验证这两份不变量。

堆空时 getMax 和 delMax 没有有效记录，应该返回失败或抛出错误，不能读数组首格。容量尚在、size 为零也仍是空堆。相反，size 为一时删除根后只需变空，无孩子可下滤；若代码仍拿左右孩子比较，会访问越界。单元素与双元素测试能直接覆盖这些分支，比只用大型随机堆更容易看清结构变化。

若优先级不允许改变，可省去定位映射；接口需求少一项，实现也可以更简单，不能把额外索引开销当成堆必有成本。

18.14 练习与答案

练习 1。 在最大堆`[8,5,7,2,3]`插 6，写出数组变化。

解答：6 追加在下标 5，父为下标 2 的 7。6≤7，无需上滤，得到`[8,5,7,2,3,6]`。不能因为 6 比另一个孩子 5 大就交换，它不是 5 的孩子；堆约束只比较真实父子关系，不要求同一层有序。

练习 2。 最小堆能否 O(1)获得最大值？

解答：一般不能。最大值必在某个叶子，但叶子之间没有大小顺序，最坏需检查线性数量叶子。堆只承诺根极小，不承诺另一端极值。若两端都要高效查询，需要同时维护额外结构或使用双端优先级队列。

练习 3。 维护前 2 大，扫描`[4,9,4,7]`，最后小根堆保存什么？

解答：前两项堆为 4、9；第三项 4 不大于堆顶 4，可跳过，仍有两个最大的记录 4、9；读 7 替换 4，保留 7、9，顶 7 为第 2 大。若要求所有与第 2 大并列的记录，堆容量固定 2 不足以完整输出全部并列者，输出规格要先确认。

阅读对照：Princeton 优先级队列与官方 IndexMinPQ 可检查堆位置数组和反向定位数组怎样随每次交换一起更新；该实现用 1 基下标，本章用 0 基，孩子公式不能直接抄同一套。

19 字符串匹配：已经比对过的字，怎样不再重读

文本 T=ABABABAC，模式 P=ABABAC，想找 P 第一次完整出现的位置。朴素法先把 P 对齐 T 下标 0，前 5 个字符匹配，最后 C 对上 B 失败；然后右移一格，从头再比。失败之前得到的“前面若干字符相同”是一份信息，KMP 会用它跳过不可能的对齐位置。

字符串是字符序列，长度 n、m 分别表示文本与模式的字符数量。本章先把字符看作可在常数时间比较的符号。真实 UTF-8 字节、Unicode 码点与用户看到的字形不是同一个单位；实现查找中文时必须知道接口返回的是字节偏移还是字符下标。

19.1 例 1：朴素匹配，最坏为何会接近 nm

T=AAAAAAAB，P=AAAAB。对齐下标 0，先匹配 4 个 A，再在 B 处失败；对齐 1，仍先匹配 4 个 A 再失败；继续右移。把这个小输入完整执行：起点 0 的第 5 次比较是 P[4]=B 对 T[4]=A，失败；起点 1 同样在 T[5]失败；起点 2 在 T[6]失败；起点 3 时 AAAA 与 B 全部相同，得到起点 3。四次对齐各比 5 次，共 20 次字符比较。文本只有 8 个字符，也会重读同一位置多次。

每次失败都重比大量相同 A，若文本长 n、模式长 m，可达到 Θ(nm)量级。

朴素算法并非所有输入都慢。若每次第一字符就失败，总时间近线性；模式很短、数据量小，简单实现也可能足够。KMP 提供的是最坏线性保证，并避免文本指针后退。

空模式的定义要提前决定。常见约定认为空串在下标 0 匹配；查找所有匹配则空串可在每个边界出现。模式长于文本时无非空匹配。这些边界不应等代码崩了以后再补。

19.2 前缀函数：一段字符串自己与自己能重合多少

对模式 P，π[i]表示 P[0:i+1]的最长真前缀，同时也是后缀的长度。“真”指不能取整段自身。例如 ABABA 的前后都可取 ABA，长度 3；整段长度 5 不允许，否则失配时不会缩小。

下面表的候选 j 表示当前正在尝试的边界长度，比较的是新字符 P[i]与 P[j]：

i	当前前缀	起始 j	比较/回退	π[i]
0	A	无	没有真前缀	0
1	AB	0	B≠A，保持 0	0
2	ABA	0	A=A，j 到 1	1
3	ABAB	1	B=B，j 到 2	2
4	ABABA	2	A=A，j 到 3	3
5	ABABAC	3	C≠B，退 1；C≠B，退 0；C≠A	0

P=ABABAC，逐项得 π=[0,0,1,2,3,0]。第 0 项只有 A，没有非空真前缀，得 0；AB 没有；ABA 前后都是 A，得 1；ABAB 前后都是 AB，得 2；ABABA 得 3；加 C 后 ABABAC 没有以 C 结尾的非空前缀，得 0。

19.3 例 2：计算 π 时，失败候选也沿着同一条链退

```
def prefix_function(p):
    pi = [0] * len(p)
    for i in range(1, len(p)):
        j = pi[i-1]
        while j > 0 and p[i] != p[j]:
            j = pi[j-1]
        if p[i] == p[j]:
            j += 1
        pi[i] = j
    return pi
```

以 P=ABABAC 的最后 C 为例，上一前缀 ABABA 的最长边界长 3，候选是 ABA，期望再接 P[3]=B，但当前为 C，失败。下一候选长度 π[2]=1，期望 P[1]=B，又失败；再退到 0，比较 P[0]=A 仍失败，最终 π[5]=0。

为什么下一候选必须是 π[j−1]，不能随意 j−1？当前长度 j 的前缀已经等于之前的后缀，若还要缩短并继续保持前后相同，新的候选必定是这段长度 j 前缀自身的边界。π[j−1]给其中最长者，沿链可枚举所有可能长度，既不漏也不重试不可能项。

每次外层 i 只增加，j 在匹配时至多增加 1，在 while 里严格减小。总增加不超过 m，总减少也不超过总增加，所以构造 π 为 O(m)。有嵌套循环仍是线性，总账要看 j 的所有变化，而非每轮拿 m 作上界相乘。

19.4 例 3：KMP 匹配，失配时文本指针原地等一下

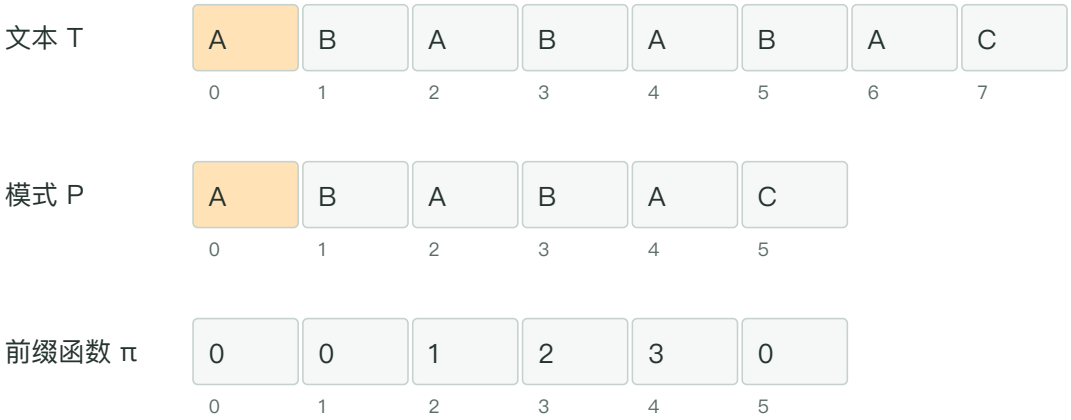
```
def find_all(t, p):
    if not p:
        return list(range(len(t) + 1))
    pi = prefix_function(p)
    ans = []
    i = j = 0
    while i < len(t):
        if t[i] == p[j]:
            i += 1
            j += 1
            if j == len(p):
                ans.append(i - j)
                j = pi[j-1]
        elif j > 0:
            j = pi[j-1]
        else:
            i += 1
    return ans
```

固定上方文本，移动下方模式；两行纵向对齐的橙格就是当前比较的字符。i 指下一待比文本位置，j 指已匹配前缀长度。π 表保存含当前项的最长真前后缀长度，失配时读取 π[j−1]，不是直接读取 π[j]。

KMP：失败时保留已经匹配的字符

1. 比较两个当前字符

i=0，j=0，模式起点对齐文本下标 i−j=0



i=0，j=0；T[i]=A，P[j]=A，相同。
原因：绿色为已经确认匹配的前缀，橙色两格纵向对齐，正是这次比较的字符。

图 171　逐步演示：KMP：失败时保留已经匹配的字符

KMP：失败时保留已经匹配的字符

2. 同时向后移动

i=1，j=1，模式起点对齐文本下标 i−j=0



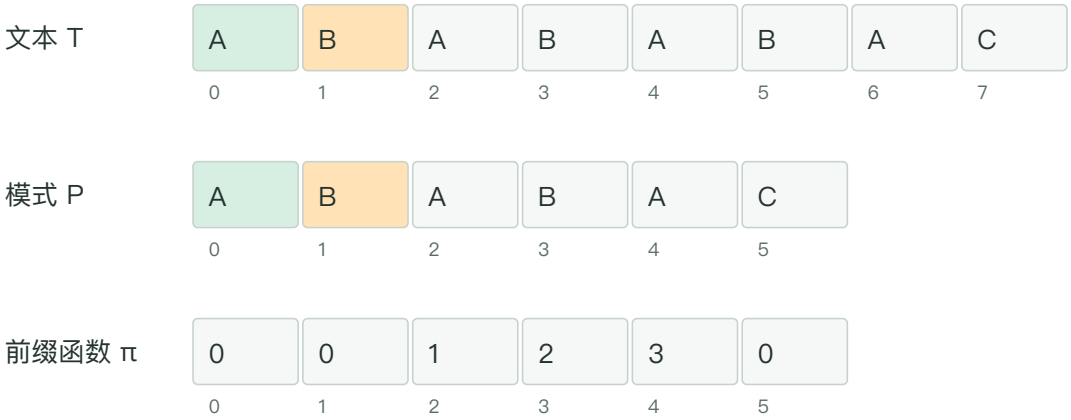
i←1，j←1。
原因：刚比较的字符已匹配，匹配前缀长度增加1，模式的起始对齐位置不变。

图 172　KMP：失败时保留已经匹配的字符，第 2 步

KMP：失败时保留已经匹配的字符

3. 比较两个当前字符

i=1，j=1，模式起点对齐文本下标 i−j=0



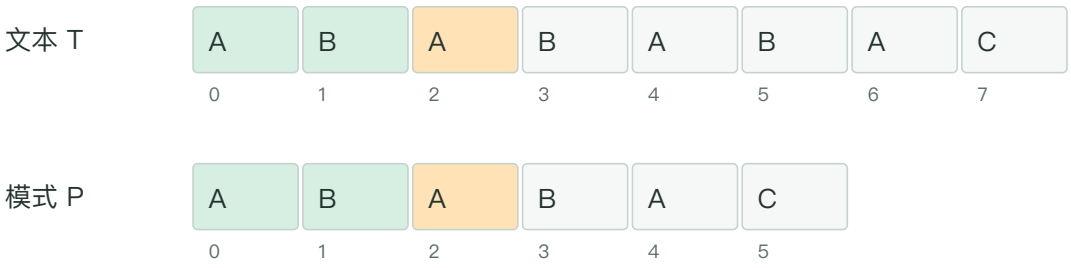
i=1，j=1；T[i]=B，P[j]=B，相同。
原因：绿色为已经确认匹配的前缀，橙色两格纵向对齐，正是这次比较的字符。

图 173　KMP：失败时保留已经匹配的字符，第 3 步

KMP：失败时保留已经匹配的字符

4. 同时向后移动

i=2，j=2，模式起点对齐文本下标 i−j=0



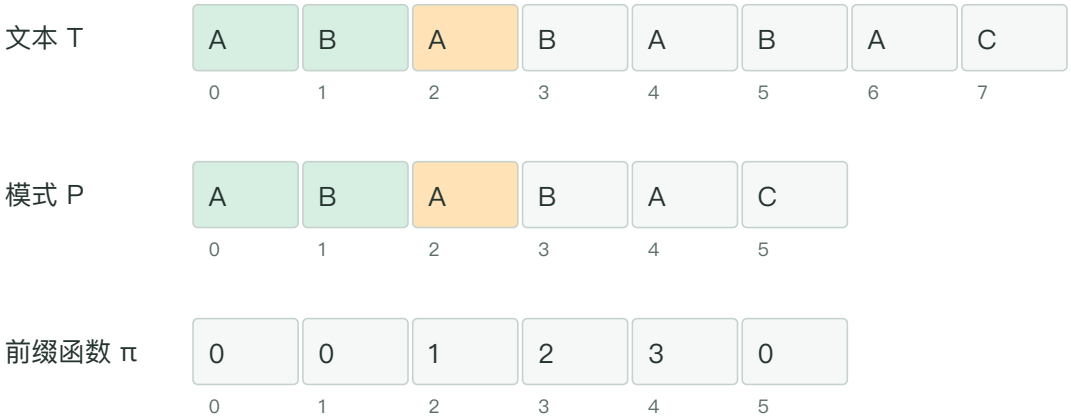
i←2，j←2。
原因：刚比较的字符已匹配，匹配前缀长度增加1，模式的起始对齐位置不变。

图 174　KMP：失败时保留已经匹配的字符，第 4 步

KMP：失败时保留已经匹配的字符

5. 比较两个当前字符

i=2，j=2，模式起点对齐文本下标 i−j=0



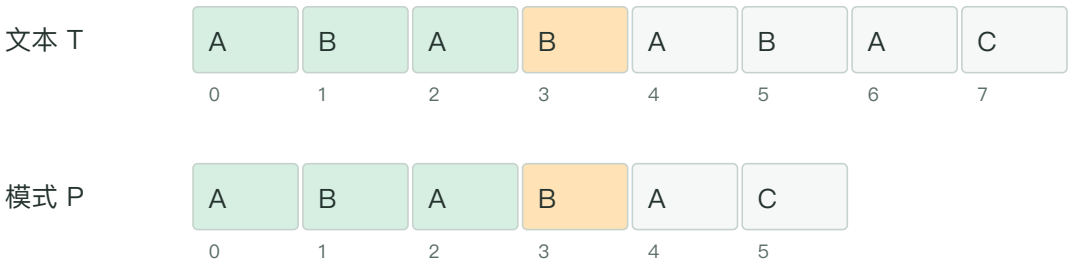
i=2，j=2；T[i]=A，P[j]=A，相同。
原因：绿色为已经确认匹配的前缀，橙色两格纵向对齐，正是这次比较的字符。

图 175　KMP：失败时保留已经匹配的字符，第 5 步

KMP：失败时保留已经匹配的字符

6. 同时向后移动

i=3，j=3，模式起点对齐文本下标 i−j=0



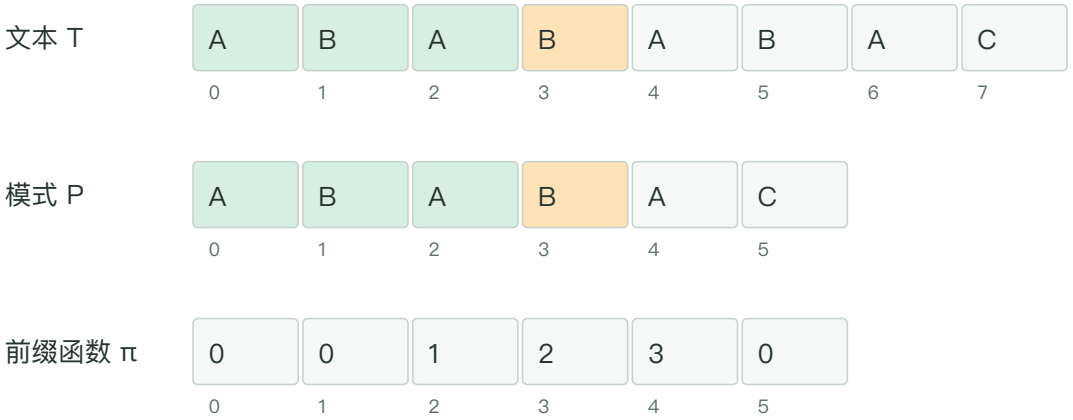
i←3，j←3。
原因：刚比较的字符已匹配，匹配前缀长度增加1，模式的起始对齐位置不变。

图 176　KMP：失败时保留已经匹配的字符，第 6 步

KMP：失败时保留已经匹配的字符

7. 比较两个当前字符

i=3，j=3，模式起点对齐文本下标 i−j=0



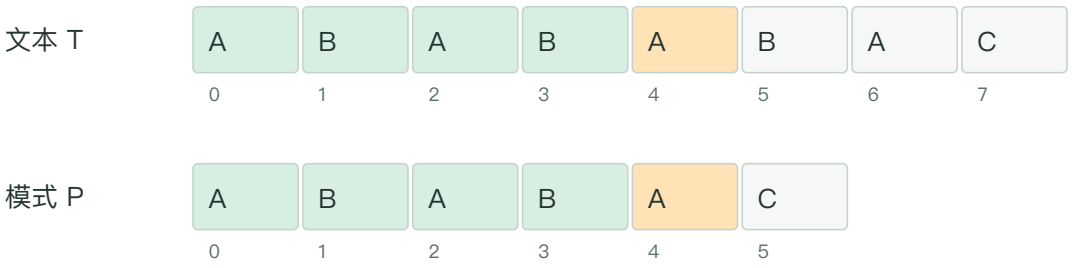
i=3，j=3；T[i]=B，P[j]=B，相同。
原因：绿色为已经确认匹配的前缀，橙色两格纵向对齐，正是这次比较的字符。

图 177　KMP：失败时保留已经匹配的字符，第 7 步

KMP：失败时保留已经匹配的字符

8. 同时向后移动

i=4，j=4，模式起点对齐文本下标 i−j=0



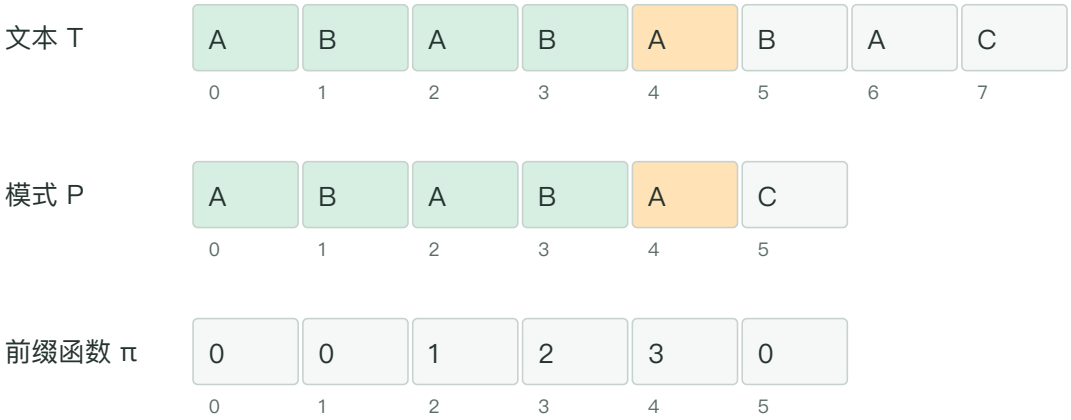
i←4，j←4。
原因：刚比较的字符已匹配，匹配前缀长度增加1，模式的起始对齐位置不变。

图 178　KMP：失败时保留已经匹配的字符，第 8 步

KMP：失败时保留已经匹配的字符

9. 比较两个当前字符

i=4，j=4，模式起点对齐文本下标 i−j=0



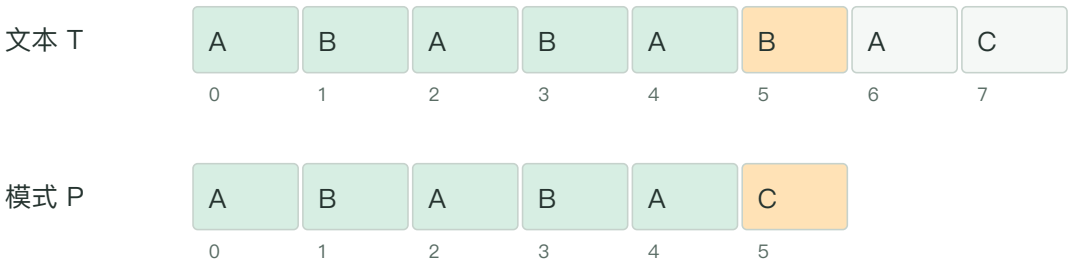
i=4，j=4；T[i]=A，P[j]=A，相同。
原因：绿色为已经确认匹配的前缀，橙色两格纵向对齐，正是这次比较的字符。

图 179　KMP：失败时保留已经匹配的字符，第 9 步

KMP：失败时保留已经匹配的字符

10. 同时向后移动

i=5，j=5，模式起点对齐文本下标 i−j=0



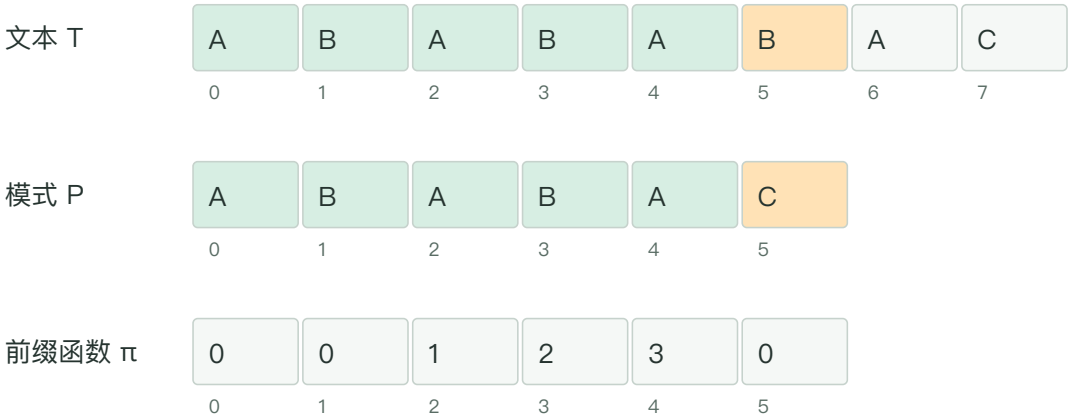
i←5，j←5。
原因：刚比较的字符已匹配，匹配前缀长度增加1，模式的起始对齐位置不变。

图 180　KMP：失败时保留已经匹配的字符，第 10 步

KMP：失败时保留已经匹配的字符

11. 比较两个当前字符

i=5，j=5，模式起点对齐文本下标 i−j=0



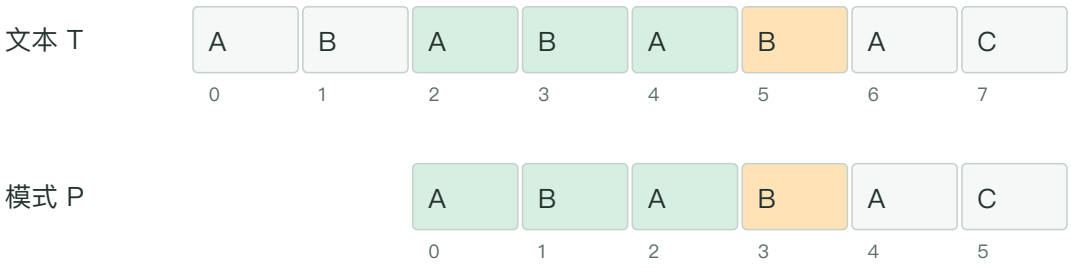
i=5，j=5；T[i]=B，P[j]=C，不同。
原因：绿色为已经确认匹配的前缀，橙色两格纵向对齐，正是这次比较的字符。

图 181　KMP：失败时保留已经匹配的字符，第 11 步

KMP：失败时保留已经匹配的字符

12. 只退模式位置，文本不回退

i=5，j=3，模式起点对齐文本下标 i−j=2



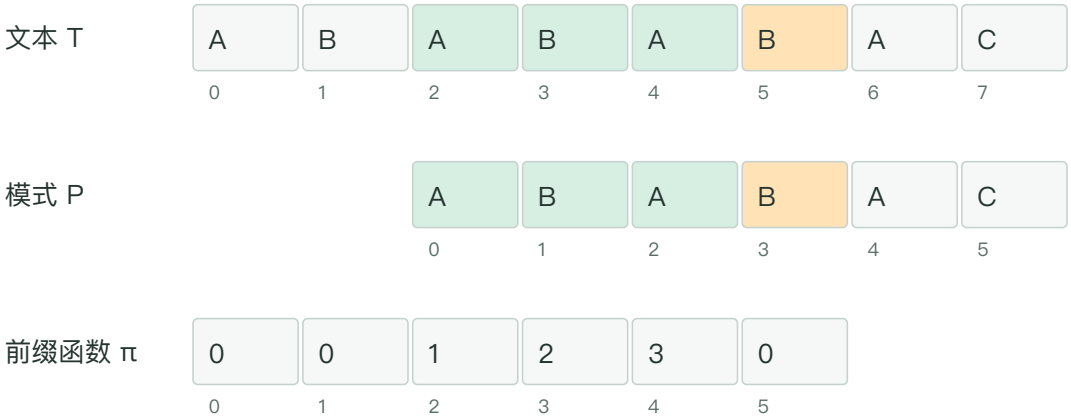
j←π[4]=3；i仍为5；模式向右移动2格。
原因：已匹配的ABABA有最长相同真前后缀ABA，长度3；这3个字符可以直接保留。

图 182　KMP：失败时保留已经匹配的字符，第 12 步

KMP：失败时保留已经匹配的字符

13. 比较两个当前字符

i=5，j=3，模式起点对齐文本下标 i−j=2



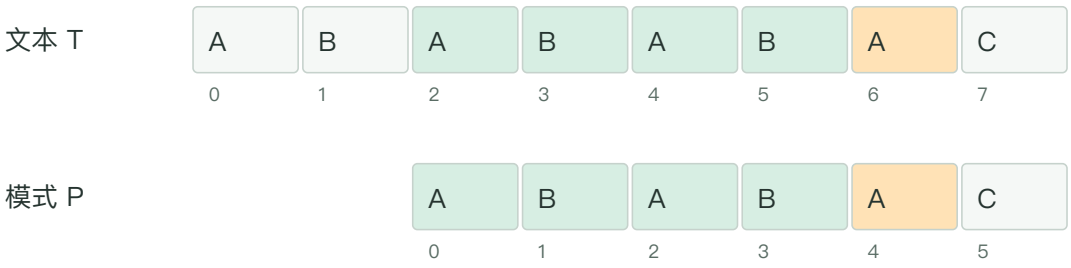
i=5，j=3；T[i]=B，P[j]=B，相同。
原因：绿色为已经确认匹配的前缀，橙色两格纵向对齐，正是这次比较的字符。

图 183　KMP：失败时保留已经匹配的字符，第 13 步

KMP：失败时保留已经匹配的字符

14. 同时向后移动

i=6，j=4，模式起点对齐文本下标 i−j=2



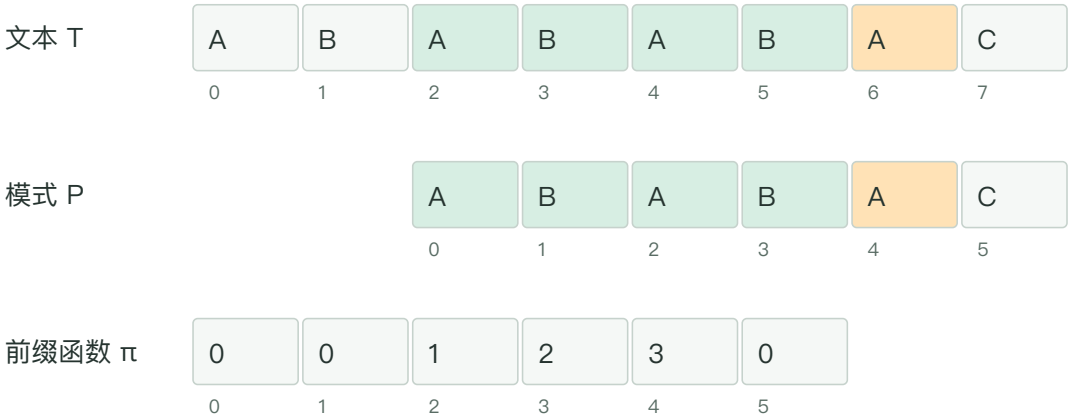
i←6，j←4。
原因：刚比较的字符已匹配，匹配前缀长度增加1，模式的起始对齐位置不变。

图 184　KMP：失败时保留已经匹配的字符，第 14 步

KMP：失败时保留已经匹配的字符

15. 比较两个当前字符

i=6，j=4，模式起点对齐文本下标 i−j=2



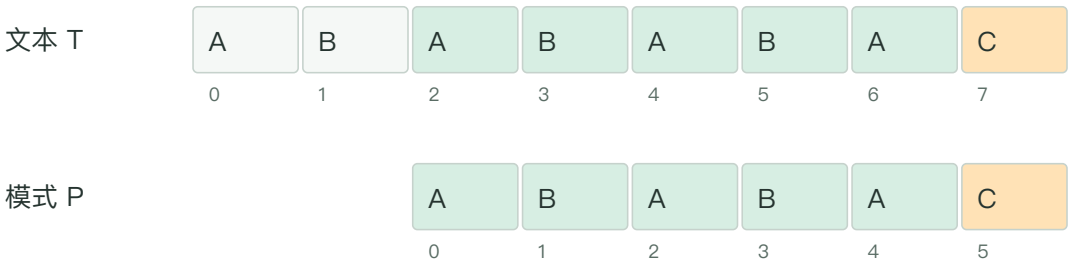
i=6，j=4；T[i]=A，P[j]=A，相同。
原因：绿色为已经确认匹配的前缀，橙色两格纵向对齐，正是这次比较的字符。

图 185　KMP：失败时保留已经匹配的字符，第 15 步

KMP：失败时保留已经匹配的字符

16. 同时向后移动

i=7，j=5，模式起点对齐文本下标 i−j=2



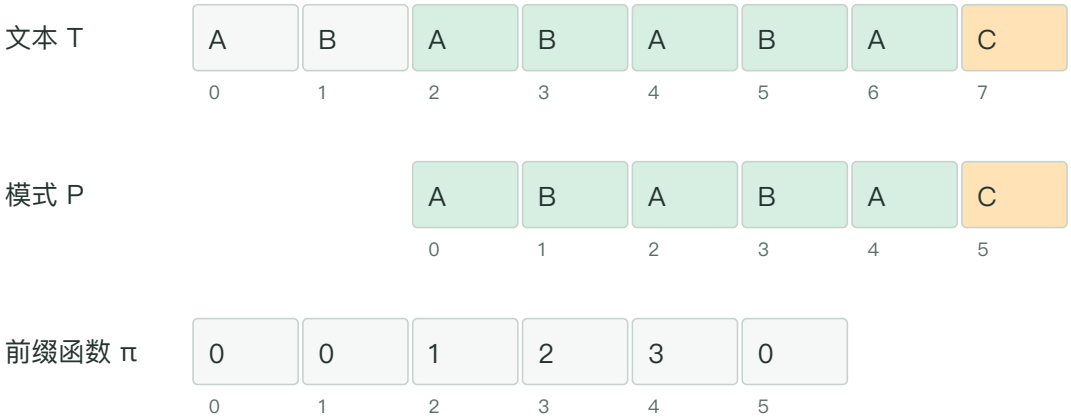
i←7，j←5。
原因：刚比较的字符已匹配，匹配前缀长度增加1，模式的起始对齐位置不变。

图 186　KMP：失败时保留已经匹配的字符，第 16 步

KMP：失败时保留已经匹配的字符

17. 比较两个当前字符

i=7，j=5，模式起点对齐文本下标 i−j=2



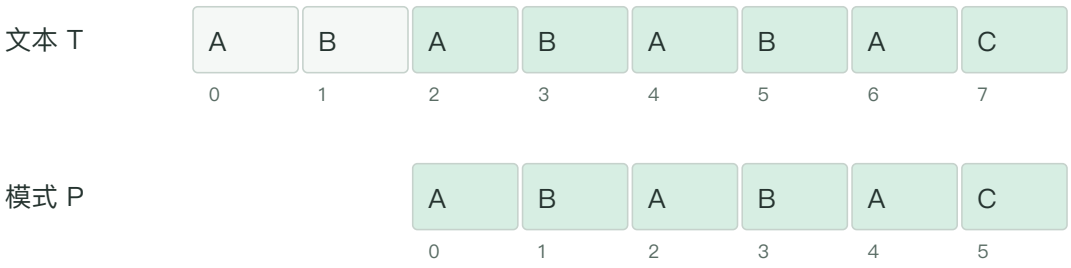
i=7，j=5；T[i]=C，P[j]=C，相同。
原因：绿色为已经确认匹配的前缀，橙色两格纵向对齐，正是这次比较的字符。

图 187　KMP：失败时保留已经匹配的字符，第 17 步

KMP：失败时保留已经匹配的字符

18. 同时向后移动

i=8，j=6，模式起点对齐文本下标 i−j=2



i←8，j←6。
原因：刚比较的字符已匹配，匹配前缀长度增加1，模式的起始对齐位置不变。

图 188　KMP：失败时保留已经匹配的字符，第 18 步

KMP：失败时保留已经匹配的字符

19. 模式完整出现

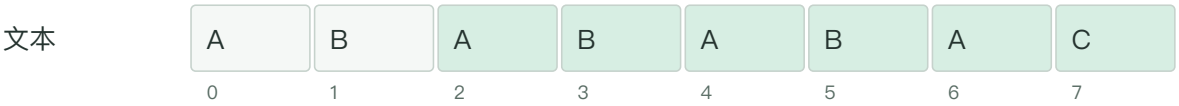


起点=i−j=8−6=2，把2记入结果。
原因：i已移到尾后位置8，j等于模式长度6，前面连续6个字符都已确认匹配。

图 189　KMP：失败时保留已经匹配的字符，第 19 步

KMP：失败时保留已经匹配的字符

20. 保留可重叠前缀，再检查文本是否结束



结果=[2]；下一待比位置i=8；保留前缀长度j=0。

j←π[5]=0；i=8已到文本末尾，返回所有起点[2]。
原因：完整匹配后通常保留最长真前后缀以继续找重叠；本模式该长度恰为0。

图 190　KMP：失败时保留已经匹配的字符，第 20 步

失配时 T[5]的橙格保持不动，P[5]的 C 被右移后的 P[3]的 B 接替。回退后已有的三个匹配字符用绿色保留；下一步重新比较的正是刚才没能匹配的那个文本 B。
本段与演示都令 i 指向下一次要比较的文本字符，j 为已经匹配的模式前缀长度。比较相同时 i、j 各加 1；失配且 j>0 时只改 j，同一个文本字符留待下一次比较。
对 T=ABABABAC、P=ABABAC：读前 5 个字符 ABABA 后 i=5、j=5。读 T[5]=B 时，本来期望 P[5]=C，失配；令 j=π[4]=3。此时 P 前 3 项 ABA 等于已经读过的文本后 3 项，不必重比。当前 B 与 P[3]=B 匹配，i 升 6、j 升 4；接着读 A 后 i=7、j=5；读 C 后 i=8、j=6，完整匹配，起点 i−j=8−6=2。i 指向尾后位置，计算起点不再额外加 1。
把每一次字符比较与 j 的改动列出：

文本 i/字符	比较的模式位置	结果	处理完此字符后的 j
0/A	0/A	相同	1
1/B	1/B	相同	2
2/A	2/A	相同	3

文本 i/字符	比较的模式位置	结果	处理完此字符后的 j
3/B	3/B	相同	4
4/A	4/A	相同	5
5/B	先 5/C，再 3/B	先失败，j 退 3；再相同	4
6/A	4/A	相同	5
7/C	5/C	相同，输出 2	先达到 6，再退 π[5]=0

共 9 次字符比较，i 没有后退。j 减小只是在寻找已经匹配后缀的另一种解释。

不变式是：读过的文本前缀中，当前 j 为某个与其后缀相同的模式前缀长度；失配退到下一边界后仍保持这个关系。未被尝试的中间对齐位置不存在合适的前后重合，故跳过去不会漏答案。整个文本只向前读，j 总上升 O(n)、下降 O(n)，匹配 O(n)，加 π 构造共 O(n+m)。

完整匹配后 j 不能无条件归 0，否则会漏重叠答案。T=AAAA、P=AA，应得起点 0、1、2。每匹配一次退到 π[1]=1，保留末尾 A 作为下次模式开头，于是能继续找到重叠。

19.5 教师图中的 next，与本章 π 不是同一个数组定义

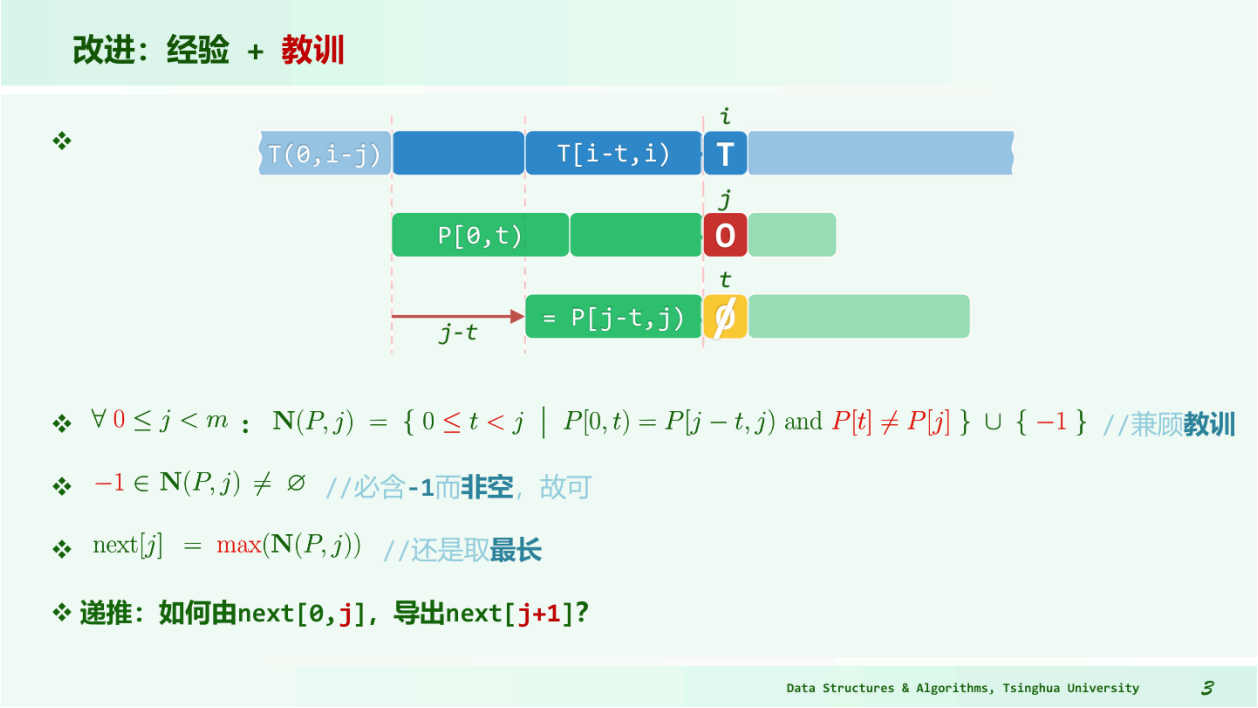


图 191　教师展示的改进失配跳转

图中 i 是文本当前位置，j 是模式中正要比较的位置。上方已知文本片段等于模式前缀；失配以后，模式滑动，把一个前缀与已知匹配后缀重合。图中特别要求候选位置 t 满足 `P[t] != P[j]`，因为若 P[t]与刚失败的 P[j]相同，再拿它比同一个文本字符必然还失败，可以继续跳。

普通位置式 next[j]回答的是“失配于 j 之前，已经匹配的 P[0:j]还能留下多长前缀”。所以 j>0 时 next[j]=π[j−1]，而 next[0]=−1 是额外终止哨兵，不能把 π[0]=0 直接当同一位置的 next[0]。

例如本章 ABABAC 的普通位置式 next（第 0 项用−1）为[−1,0,0,1,2,3]；若跳过与原失败字符相同的候选，改进 next 为[−1,0,-1,0,-1,3]。看位置 4 的 A：普通候选 2 也是 A，既然同一文本字符已经与位置 4 的 A 不等，再比位置 2 也必失败，可以继续退到位置 0 的 A，最后退−1。位置 5 的 C 退到 3 的 B，字符不同，必须实际比较，不能一并跳掉。−1 的含义是模式开头也失败，这时推进文本一格并把 j 恢复 0。

这是改进 next 的定义，常带−1 哨兵；本章 π[i]只是前缀 `P[0:i+1]` 的最长边界长度，允许下一个字符相同。两者数值不能逐项强行相等。使用课程代码时，先写清 next[j]描述“位置”还是“长度”、失配时 i 是否增加、−1 如何处理，再做对照。

19.6 Boyer–Moore 与滚动哈希，换一种利用失败信息的方法

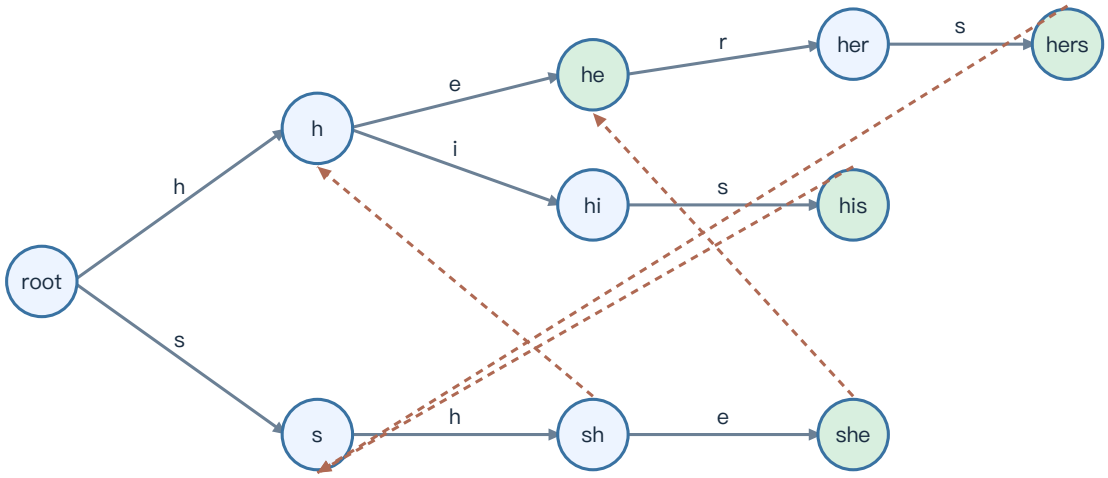
Boyer–Moore 从模式右端往左比。坏字符规则利用文本中导致失败的字符在模式中的最后出现位置来跳；好后缀规则利用已经匹配的模式后缀寻找其他对齐。它常能跳过许多文本位置，但具体最坏保证依版本与附加规则，不能把经验上的快自动叫 O(n)。

滚动哈希把长度 m 窗口编码成数，窗口右移时减去旧字符贡献、乘基数、加新字符，从而 O(1)更新某种固定模数哈希。哈希不相等可确定字符串不同；哈希相等仍可能冲突。若要求确定正确，应在候选处再逐字核对，最坏核对成本可能增大；双哈希降低碰撞概率，也不把概率变成数学上的 0。

例如用十进制直观表示数字串，窗口 123 右移为 234：先减 1×100 得到 23，再乘 10 加 4 得到 234。真实字符哈希常在模数下做同样操作。若模数太小，不同窗口容易碰撞；如果攻击者能选择输入，简单随机假设更需要谨慎。

19.7 多模式匹配：Trie 加失败链接

若同时找 `he,she,his,hers`，每个模式单独跑 KMP 会重复扫描文本。Aho–Corasick 在 Trie 节点上加失败链接：当前前缀失配时，跳到它的最长后缀、且该后缀也是某模式前缀的节点。这个思路与 KMP 的边界链相通，但状态是 Trie 节点，不再只是一个整数前缀长度。



实线读入字符；虚线是选出的失败链接；she还输出其后缀模式he

图 192　四模式 Trie 和用于跳转的失败链接

节点标签是从根走到这里得到的完整前缀，实边只消费一个新字符，虚线跳转不消费当前字符。构造失败链接可按 BFS：根的孩子 h、s 失败都回根；处理 sh 时，先看 s 的失败根，从根能读 h，故 fail(sh)=h；处理 she 时，从 fail(sh)=h 继续读 e 到 he，故 fail(she)=he。his 的最长可用后缀是 s，hers 也同样退 s。

完整扫描文本 ushers：u 在根无边，留根；s 到 s；h 到 sh；e 到 she，输出 she 并沿失败输出 he；r 在 she 无边，先退 he，再从 he 读 r 到 her；最后 s 到 hers，输出 hers。各命中起点按长度回算：she 从 1，he 从 2，hers 从 2。跳 fail 时不重新消费 r，否则会漏掉 her 状态。

输出还要处理“某个状态对应多个模式后缀”的情况。例如读到 she，同时可能命中 he。实现复杂度依转移表、失败构建与输出方式，若实际命中 k 次，至少要计入 O(k)输出。先掌握单模式 KMP 的状态含义，再学多模式结构，会更容易检查失败链接究竟保存了什么。

19.8 一次坏字符跳转，先把对齐下标画对

模式 P=ABCD，先对齐文本 T=ABXDABCD 的下标 0，从右向左比较。D 对 D 相同，接着模式位置 2 的 C 对文本位置 2 的 X 失败。X 在模式中未出现，最后位置记−1，坏字符规则可让模式起点向右移动 j−last[X]=2−(−1)=3。新起点 3 不是直接就匹配，还要重新比较；随后可继续跳到起点 4 找到 ABCD。

这个坏字符例继续走完：右移 3 后起点为 3，窗口 DABC，右端 P[3]=D 对 T[6]=C 失败；last[C]=2，位移 max(1,3−2)=1，再到起点 4，ABCD 四个字符全部相同，找到 4。第一次安全位移并不保证下一次马上成功。如果失配字符在模式中最后出现在 j 右侧，j−last 可能非正，至少仍需向右移 1，不能让模式倒退。好后缀规则还能利用已匹配的 D，可能给更大的安全位移；最终取安全规则中的较大值，但每个规则都要独立保证不漏匹配。

19.8.1 好后缀的完整例子：AAABAB 中找 ABAB

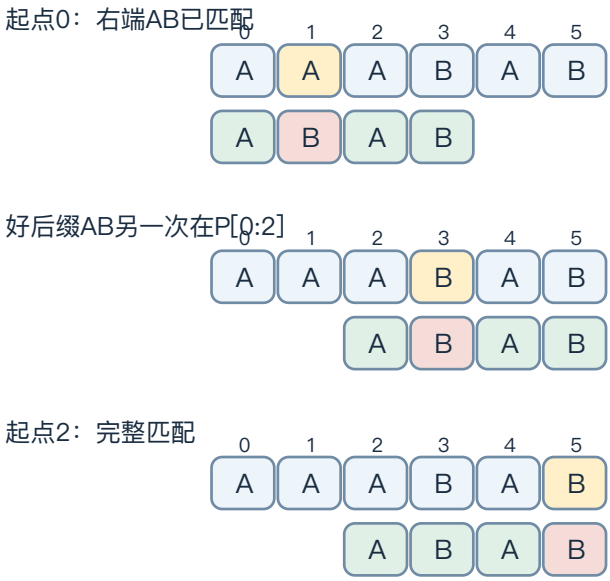


图 193　Boyer–Moore 好后缀 AB 复用并移动两格

起点 0，从右向左比较：P[3]=B 与 T[3]=B 相同，P[2]=A 与 T[2]=A 相同；P[1]=B 与 T[1]=A 失败，已经知道文本位置 2/3 是后缀 AB。模式中还有一段 AB 在位置 0/1，把这一段对齐刚匹配的文本 2/3，需要移动 2 格。由于这是模式开头，没有一个相同的前置失败字符会迫使它再次失败。起点 2 后窗口恰为 ABAB，逐个从右比较全相同，返回 2。

此时坏字符 A 在模式最后位置 2，比失配位置 j=1 还靠右，坏字符规则只允许保守移动 1；好后缀规则允许安全移动 2，取两者较大值 2。这一例才显示两张表做的工作不同。一般寻找好后缀的另一出现时，要排除“前置字符与刚失败的模式字符相同”的候选，因为同一文本坏字符对它仍会失败；若找不到，则把好后缀中最长的、也是模式前缀的部分对齐；连这一部分都为空才可跳过整段。

再看 P=ABCD、文本 AXCDABCD：起点 0 已匹配 CD，在 B 对 X 处失败，模式中无另一 CD，也无非空前缀等于 CD 的后缀，故好后缀位移 4，直接到起点 4 匹配 ABCD。预处理表把这些搜索提前完成，匹配时只查表。

19.8.2 从模式本身算出好后缀表

好后缀的位移只由模式和失配位置决定，可以预先算表。先定义 ss[j]：在 P[0:j+1]的所有后缀中，找一个与整个 P 的某个后缀相同的最长者，记其长度。比如 P=ABAB：前缀 A 不能与以 B 结尾的模式后缀相同，ss[0]=0；AB 本身等于模式末尾 AB，ss[1]=2；ABA 末尾为 A，ss[2]=0；整个 ABAB 等于自身，ss[3]=4。所以 ss=[0,2,0,4]。这里允许整段前缀被取出，与 KMP 的“真前缀边界”定义不同。

逐个 j 从右往左直接比后缀，可以正确构造 ss，但最坏需平方时间。课程的线性算法再利用一段已经比过的窗口。设已知 P 的区间(a,b]等于模式末尾同样长的一段，向右平移 d=m−b−1 就对齐；a 在窗口左边界之外，b 包含在内。若当前 j 处在这段已知窗口里，就可借已经算好的 ss[j+d]，不必重新比较窗口内部。

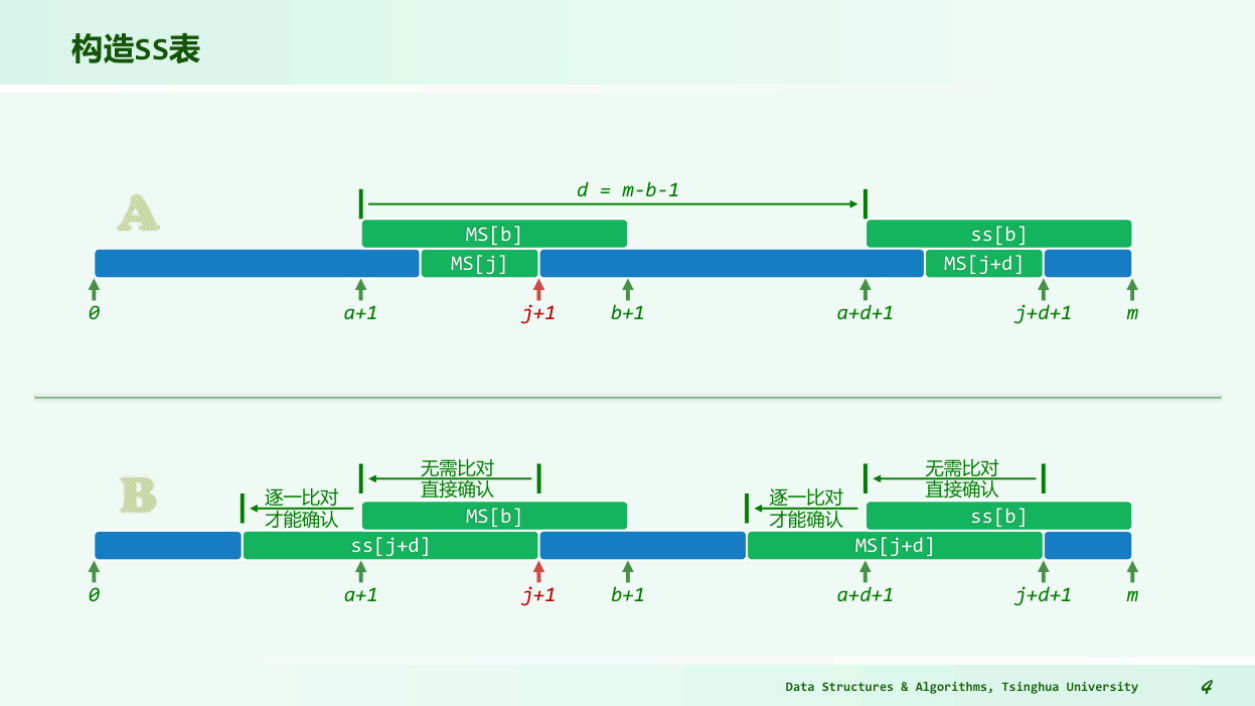


图 194　教师完整原页：SS 窗口中可复制与必须继续比较的两类情况

原页来自《13.串.pdf》PDF55 页，页脚 4。上图 A 中，ss[j+d]的匹配段完全落在已知窗口内部，它左边那个不等字符也还在已知范围内，因此可以直接复制长度。下图 B 中，借来的匹配已经碰到窗口左边界，边界外的字符尚未比过，只能从 a 开始再往左核对，直到不等或越过 0。

```
def build_ss(p):
    m = len(p)
    if m == 0:
        return []
    ss = [0] * m
    ss[m - 1] = m
    a = b = m - 2
    d = 1
    for j in range(m - 2, -1, -1):
        if ss[j + d] < j - a:
            ss[j] = ss[j + d]
        else:
            a = min(a, j)
            b = j
            d = m - b - 1
            while a >= 0 and p[a] == p[a + d]:
                a -= 1
            ss[j] = b - a
    return ss
```

在 ABAB 上从右往左完整走：

当前 j	进入时 a,b,d	能直接复制吗	实际比较/更新	ss[j]
初始	2,2,1	先设 ss[3]=4	空窗口，从右往左计算	4 位于下标 3
2	2,2,1	ss[3]=4 不小于 j−a=0	a=2,b=2,d=1；A 与 B 不等，a 不动	0
1	2,2,1	ss[2]=0 不小于 j−a=−1	改 a=1,b=1,d=2；B=B 使 a 到 0，再 A=A 使 a 到−1	2
0	−1,1,2	ss[2]=0 小于 j−a=1	直接复制，无字符比较	0

外层 j 每次减 1，共 m 次；显式字符比较的内层中，a 只会继续减小，更新 a=min(a,j)也不会让它增加，因此全部成功比较合计不超过 m。每个外层至多再有一次失败比较，总工作 O(m)。保存窗口是为了让“已经比过的字符”不被反复收费。

SS表构造算法

```
void buildSS(char* P, int m, int* ss) { //O(m = |P|)
   ss[m-1] = m; //确定末字符对应的项；以下，自后向前地扫描P，依次计算出其余各项
   for (int a = m-2, b = m-2, d = 1, j = m-2; j >= 0; j--) //d=m-b-1, (a,b]=(a+d,m)
      if (ss[j+d] < j-a) //情况一：虽自然有[j-ss[j+d],j] = [j+d-ss[j+d],j+d]
         ss[j] = ss[j+d]; //但[j+d-ss[j+d]] <> [m-1-ss[j+d]]，故ss[j] <= ss[j+d]
      else { //情况二：已有(a,j] = (a+d,j+d]，MS[j]的左边界还有可能进一步拓展
         a = min(a,j); b = j; d = m-b-1; //更新(a,b]与d
         while ( (0 <= a) && (P[a] == P[a+d]) ) //逐个比对，向左拓展(a,b]
            a--; //所有的while循环均由a控制，而a全程从m-2到0单调递减，故累计仅O(m)
         ss[j] = b-a; //MS[j] = 拓展之后的(a,b]
      } //else
} //buildSS
```

Data Structures & Algorithms, Tsinghua University 5

图 195　教师完整原页：线性 SS 构造代码与单调指针

原页来自同一文件 PDF56 页、页脚 5。先对应上面的 a/b/d 定义，再看原程序的区间记号；不能把 a 当成已经匹配的第一个字符，它始终在窗口之外。

有了 ss，再构造 gs[j]，它表示失配在模式位置 j 时好后缀允许的安全位移。先令所有 gs 为 m，再做两遍：一遍处理“模式前缀可接上好后缀末尾”的情况，一遍处理“模式内部另有完整好后缀”的情况。

```
def build_gs(p):
    m = len(p)
    ss = build_ss(p)
    gs = [m] * m
    i = 0
    for j in range(m - 1, -1, -1):
        d = m - 1 - j
        if ss[j] == j + 1:
            while i < d:
                gs[i] = d
                i += 1
    for j in range(m - 1):
        gs[m - 1 - ss[j]] = m - 1 - j
    return gs
```

第一遍在 ABAB 中：j=3 时 d=0，没有空位要填；j=2 不满足；j=1 满足 ss[1]=2=j+1，说明开头 AB 也是末尾 AB，位移 d=2 可用于失配位置 0 和 1，填 gs[0]=gs[1]=2；j=0 不满足。此时 gs=[2,2,4,4]。i 只增不减，每项第一次收到的 d 最小，不能在每次 j 重新把 i 设 0。

第二遍 j=0，ss=0，对应失配位置 m−1−0=3，候选位移 3，gs[3]=3；j=1，ss=2，对应位置 1，位移 2，gs[1]=2；j=2，ss=0，又对应位置 3，位移 1，覆盖成 1。最终 gs=[2,2,4,1]。这一遍 j 从左往右、位移从大往小，后写入的才是较小安全位移。

这个下标 m−1−ss[j]来自图的对齐：一个以位置 j 结尾、长度 ss[j]的匹配后缀，平移 d=m−1−j 后恰好接到模式末尾；它前面的失配位置就在 m−1−ss[j]。ss 的最大性又保证其前一个字符与原失配模式字符不同，否则还能再向左多匹配一位。

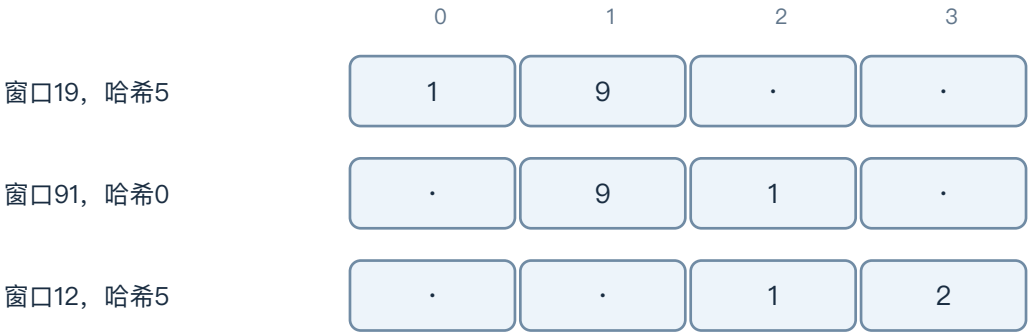
可直接核对表：失配 j=1 时已匹配 AB，另一个 AB 允许移 2；失配 j=2 时只匹配末 B，若移 2 会用模式位置 0 的 A 接替刚失败的位置 2 的 A，同一坏字符必再失败，不能选，最小安全位移为 4；失配 j=3 时没有好后缀，但前一个字符 A 与 B 不同，允许移 1。

两遍外层各 m 次，第一遍的 i 全程最多增 m 次，所以 ss 到 gs 为 O(m)，连同 ss 构造仍线性。真正匹配时，在每次从右端比较结束后，将起点增加 max(坏字符安全位移,gs[j])；完整匹配后若还找后续出现，可按模式最长边界继续移动，ABAB 为 2，不应直接跳 4 漏掉 ABABAB 中的重叠出现。

Boyer–Moore 的表格定义有多种版本，last 通常以字符为下标、返回最后出现位置；KMP 的 π 以模式位置为下标、返回长度。两个数组都叫“跳转信息”，却不是同一种输入输出函数。先写一个表项的中文含义，再抄公式，能避免把字符编码拿去索引长度 m 数组的越界错误。

19.9 哈希窗口也可以有一个很短的碰撞反例

以十进制数对 7 取模作教学哈希，串 12 的值 12 mod7=5，串 19 的值 19 mod7 也=5。它们长度相同、哈希相同，字符却不同。真实滚动哈希基数与模数更大，只是让某些模型下碰撞概率更低，并没有改变“有限值空间可能碰撞”的事实。



文本1912，模式12也哈希5；第一窗碰撞需逐字排除，第三窗才真匹配

图 196　滚动哈希扫描 1912 时的碰撞、排除与真匹配

用文本 1912 找模式 12，固定两位窗口、基数 10、模数 7。初窗 19 的哈希 5 与模式相同，逐字核对 1=1 但 9≠2，排除。右移得到 91，哈希 0，不相同，直接跳过逐字核对；再右移为 12，哈希 5，核对两字都相同，报告起点 2。滚动时只保留哈希即可：10 在模 7 下为 3，19⟶91 算((5−1×3)×10+1) mod7=0；91⟶12 算((0−9×3)×10+2) mod7=5。负中间值必须按 0 至 6 的余数规范化，不能把语言的负余数直接当数组下标。

查询模式 12 时若直接接受哈希 5，窗口 19 就会成为假阳性。再逐字比较能排除它，但如果恶意构造许多碰撞窗口，核对总成本可能很大。要给严格最坏线性且确定正确的单模式匹配，KMP 提供了不同性质的保证；要用哈希，便明确报告概率假设或验证成本。

19.10 从匹配结果恢复位置，字符单位必须一路相同

假设模式长度 m=3，刚消费文本下标 7 的字符以后，i 已经移到 8、j 达到 3，起点为 i−m=8−3=5。这与本章 i 指向下一待比字符的约定相同。i 与 m 必须按同一种符号单位计数；若 i 按 UTF-8 字节数、m 按 Unicode 字符数，公式相减就无意义。中文“数据结构”有四个常见汉字，但 UTF-8 通常每个占多个字节，工具返回的字节偏移不能直接当第几个汉字。

如果用户看到的一个字形由多个码点组成，按码点切片也可能切开显示单元。课程算法通常抽象为符号序列，不要求先解决所有文本规范化问题；实际应用必须在读入、比较、长度与返回偏移之间统一层次。字形看似相同也可能有不同编码序列，是否先规范化取决于产品语义。

19.11 模式预处理一次，多个文本可以复用

前缀函数只依赖模式，不依赖文本。固定关键词反复搜索多份文件，先构造一次 π，每份文本从 j 为零开始扫描，节省重复预处理。如果把上一份文件的 j 直接带到下一份，会把两个独立文件当成拼接文本，可能报告跨文件边界的假匹配；只有明确处理连续分块的同一文本时才应保留 j。

流式处理同一大文本时，块尾已匹配前缀长度正是下一块需要的状态，无需保存全部旧文本。要报告全局起点，另维护已经读过的总字符数；局部下标必须加块偏移。模式本身及 π 仍需保存，空间随模式长而非文本总长增长。

如果只要第一次匹配，找到即可返回，输出成本常数；若要所有匹配，结果个数可达文本长度量级，例如重复字符，输出列表也占相应空间。说算法额外空间仅为模式长度时，要说明是否把输出空间排除。

文本结束而 j 小于模式长，表示最后只有一个未完成前缀，不应输出匹配。模式比文本长时，即使所有文本字符都与模式开头相同，也不能把这段前缀当完整命中。

找到首次命中后立即返回，适用于只问第一次出现的接口。

19.12 练习与答案

练习 1。 写出 P=AAAA 的 π，并用它找 T=AAAAA 中的全部匹配。

解答：π 为 0、1、2、3。前 4 字符匹配后输出 0，j 退到 3；读第 5 个 A 再升 4，输出 1。答案 0、1。若匹配后归 0，第二个重叠出现会漏掉。重复字符不是特殊补丁，而是边界定义自然覆盖的输入。

练习 2。 P=ABABC，最后字符 C 到来时，π 构造的候选怎样退？

解答：此前 ABAB 的边界长 2，候选下一个字符 P[2]=A，与 C 不同；退 j=π[1]=0，再比 P[0]=A 仍不同，最终 π[4]=0。注意不是从 2 逐个试 1 再 0，因为长度 1 不是 ABAB 的边界，根本不可能接上。

练习 3。 两个窗口哈希相同，可以直接认定字符串相同吗？

解答：一般不能。有限哈希值比可能字符串少，必有不同串碰撞。若题目允许概率算法，要清楚说明随机选择与碰撞风险；若要求严格确定答案，应做足够的核对或使用无碰撞的结构。不能把“本地测了一百万次没撞”替代正确性证明。

20 划分、选择与步长：排序还有哪些做法

归并排序先分成两半，再花时间合并；快速排序先花时间划分，使轴点左侧不大于它、右侧不小于它，再各排各的。两者都用分治，但本层工作发生的位置不同。读快排实现时，先确定划分函数返回的是轴点位置、两段分界，还是一整段等值区间；递归范围由这个约定决定。

20.1 例 1：三路划分，重复值不必反复递归

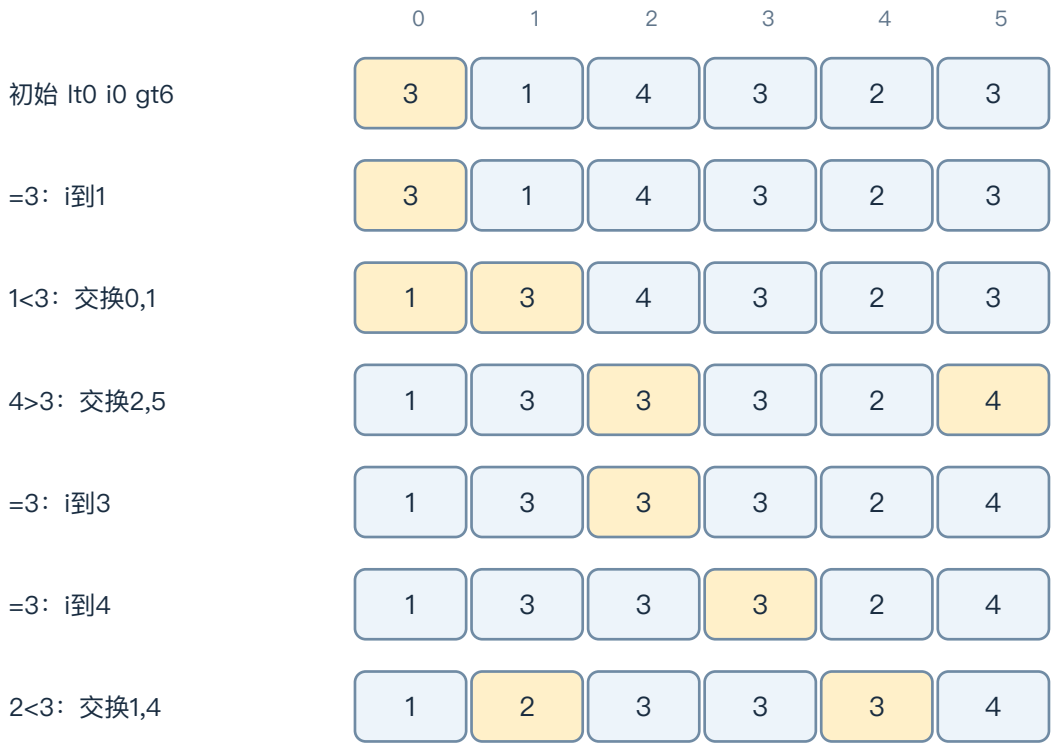


图 197　三路划分每次分类后的数组与活动下标

图中下标从 0 起。黄色格是本步比较或交换的位置，数组值可能相同，交换后外观不变也要更新相应下标。轴值先复制为独立的 3，随后交换不会改变它。

以轴值 v=3 划分`[3,1,4,3,2,3]`，维护四段：`[lo,lt)`小于 3，`[lt,i)`等于 3，`[i,gt)`尚未检查，`[gt,hi)`大于 3。初始 lt=i=lo，gt=hi。

读 a[i]若小于 v，与 a[lt]交换，lt 与 i 都加 1；若等于 v，只让 i 加 1；若大于 v，先 gt 减 1，与 a[gt]交换，i 不动，因为换来的项还没看。

```
初始   [3,1,4,3,2,3] lt0 i0 gt6
等于   [3,1,4,3,2,3] lt0 i1 gt6
小于   [1,3,4,3,2,3] lt1 i2 gt6
大于   [1,3,3,3,2,4] lt1 i2 gt5
等于   [1,3,3,3,2,4] lt1 i3 gt5
等于   [1,3,3,3,2,4] lt1 i4 gt5
小于   [1,2,3,3,3,4] lt2 i5 gt5
```

结束 i=gt，未知段为空，返回两条边界 lt=2、gt=5。只递归`[lo,lt)`与`[gt,hi)`，等于区间无需再排。每轮未知段长度 gt−i 至少减少 1，因此本次划分 $\Theta(n)$。这里返回的是一段等值区间；某些二路划分返回轴点最终秩，另一些返回左右分界，递归区间不能互换。

完整快速排序还要走完递归，而不是划分结束便已经有序。上述第一次划分后是[1,2]、[3,3,3]、[4]三块。中块直接完成；右块仅一项，直接完成；左块选轴 1，先把 1 留在等值区，看到 2>1 便归右，返回[1]与[2]两个单项区间。全部调用返回后才得到[1,2,3,3,3,4]。这一次只需再排两项，是因为最初的等值块很大，不是所有快排输入都会如此。

20.2 快排为什么平均快，又为什么会平方

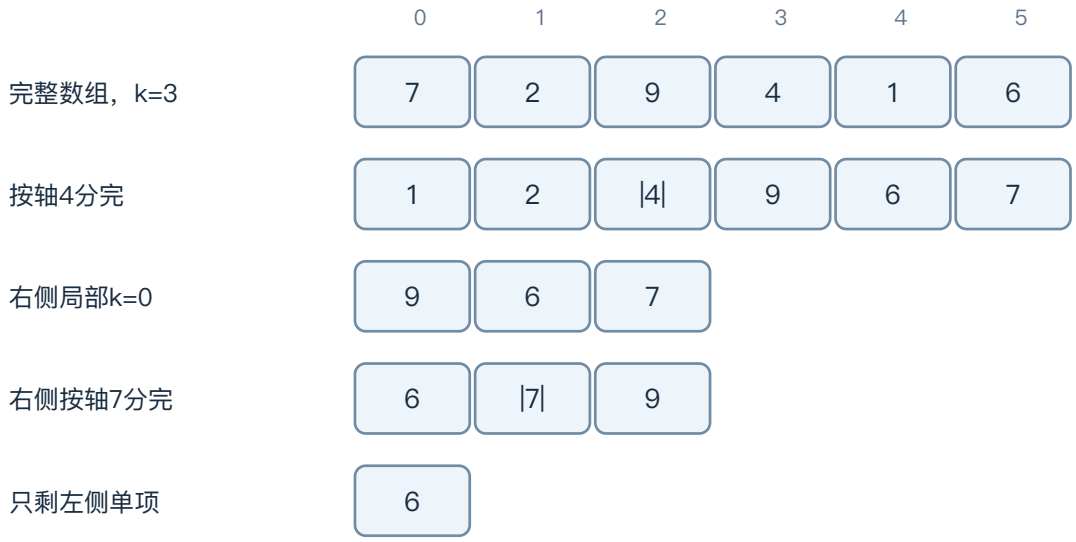
若每次轴点接近中间，递推约为 $2T(n/2)+\Theta(n)$，得到 $\Theta(n \log n)$。若总选到最小值或最大值，一边为空、一边 $n-1$，总代价 $n+(n-1)+\dots$，为 $\Theta(n^2)$。用第一个元素当轴，对已排输入的某些二路实现就会遇到这种情况。

随机从当前区间选轴，在互异元素、随机选择独立且均匀等假设下，期望比较次数为 $O(n \log n)$。这给每个固定输入上的随机期望，而不是最坏保证。三路划分能避免大量相等值把递归搞坏，但仍不能让所有不平衡轴点的最坏情况自动消失。

实现上可每次递归较小半边，较大半边改成循环，使调用栈最坏 $O(\log n)$，即使总运行时间仍可能平方。这是时间与栈空间分别优化的例子。实现也可以给递归深度设置预算：过深时用前章堆排序完成剩余区间，小区间则用插入排序。这样选择后备算法时，仍需把完整最坏成本加到总账。

20.3 例 2：第 k 小，只追答案所在的一边

找[7,2,9,4,1,6]中从 0 起第 3 小，即排序后第 4 项。使用前面同一套三路划分，第一轴复制为 4。



竖线标等值区；放弃整块不含目标秩的区间，不再回头排序它

图 198　Quickselect 每次只保留含目标秩的那一侧

第一次划分内部也逐步看：7>4，与末 6 换，得[6,2,9,4,1,7]；6>4，与当前未知末 1 换，得[1,2,9,4,6,7]；1<4、2<4，依次扩大小于区；9>4，与 4 换，得[1,2,4,9,6,7]；新来的 4 等于轴，划分结束。小于区长 2，等值区长 1，轴占全局秩 2。目标 3 在右侧，右侧局部秩改为 3−2−1=0。

只在[9,6,7]继续，选轴 7：9>7，与末 7 换，得[7,6,9]；7 等于轴，扫描向右；6<7，与等值区开头 7 换，得[6,7,9]。此时小于区长 1，等值区长 1，局部目标 0 在左侧，只剩[6]，直接返回 6。被放弃的[1,2]无需内部排序，右侧 9 也无需再看，因为它们不可能改变目标秩的值。

Quickselect 每次只递归一边，随机轴的期望时间 $O(n)$，最坏仍 $O(n^2)$。期望线性也能按阶段看：随机轴至少有约 1/2 概率落在当前秩的中间一半，这时保留下来的目标侧最多约 $3n/4$。一次没选好也只会让子问题缩小，不会变大；等到一次选好的期望尝试数至多约 2。因此把规模降到原来的 3/4，期望划分工作不超过常数乘 n；下一阶段不超过常数乘 $3n/4$，再下一阶段 $9n/16$。几何级数 $n+(3/4)n+(3/4)^2n+\dots=4n$ 收敛为线性，给出 $O(n)$ 期望界。极小规模的取整误差可并入固定终止阈值。

不能把快排的“两个孩子”递推直接照搬给选择，也不能因只递归一边就无条件说线性：若每次只丢掉 1 项，仍是平方和。

确定性线性选择可用“中位数的中位数”：每 5 项一组，求各组中位数，再递归选这些中位数的中位数作轴。忽略常数边界后，至少约 3n/10 项不小于轴，也至少约 3n/10 项不大于轴，最大递归侧不超过约 7n/10。递推 `T(n) <= T(n/5)+T(7n/10+O(1))+O(n)` 可证明 O(n)。它牺牲常数与实现简洁，换来最坏界。

20.4 例 3：希尔排序，先让远处的元素接近位置

希尔排序选一串逐渐减小、最后为 1 的步长 gap。固定 gap 时，把下标相差 gap 的元素分成子序列，各做插入排序。大步长让元素一次跨越较远距离，最后 gap1 时整个数组已较接近有序。

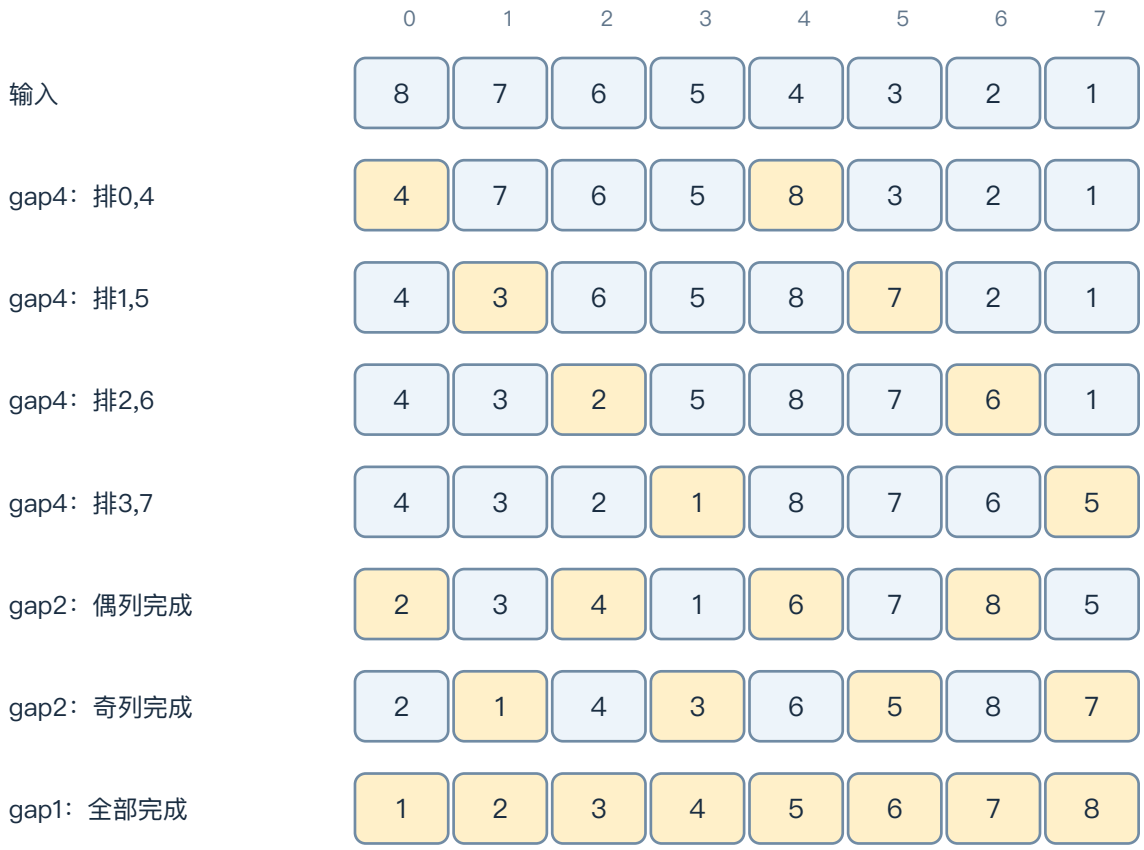
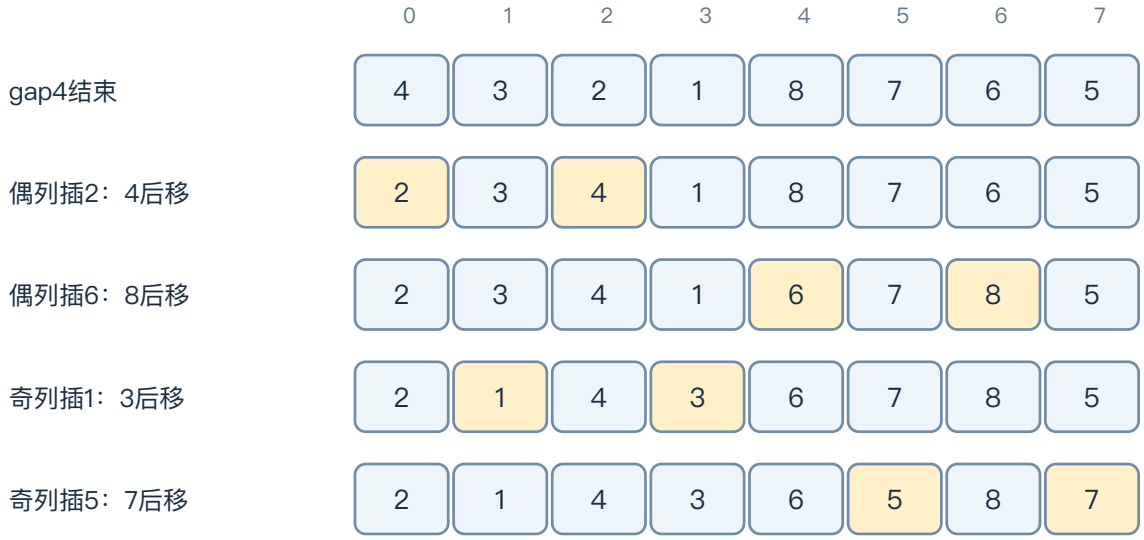


图 199　希尔排序各列完成之后的数组

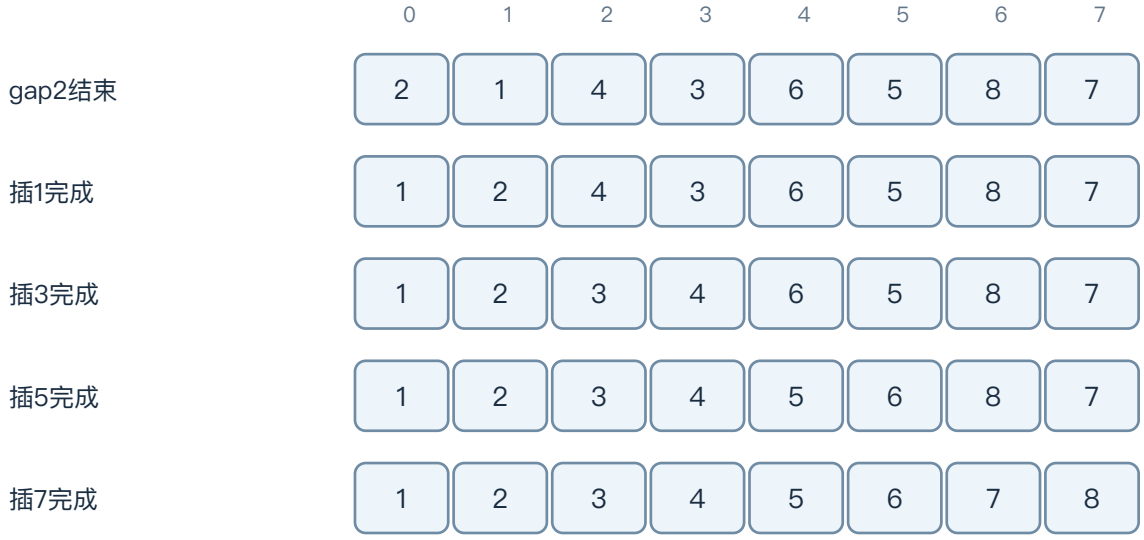
数组 `[8,7,6,5,4,3,2,1]`，先 gap4，分别排下标(0,4)、(1,5)、(2,6)、(3,7)，得到 `[4,3,2,1,8,7,6,5]`。再 gap2，偶数位置 4,2,8,6 排成 2,4,6,8，奇数位置 3,1,7,5 排成 1,3,5,7，得 `[2,1,4,3,6,5,8,7]`。最后 gap1 插入，得到 1 到 8。



此处逐列处理：先全部偶位置，再全部奇位置；每行是一项插入完成后的状态

图 200　gap2 中每次发生移动的插入

图按列执行 gap2 插入：偶列[4,2,8,6]先把 2 插到 4 前，8 随后不动，再把 6 插到 8 前；奇列[3,1,7,5]同样先插 1、再插 5。图省略的是没有移动的插入，检查仍然做过。



间隔1的其他插入无需移动；不动的检查仍要执行

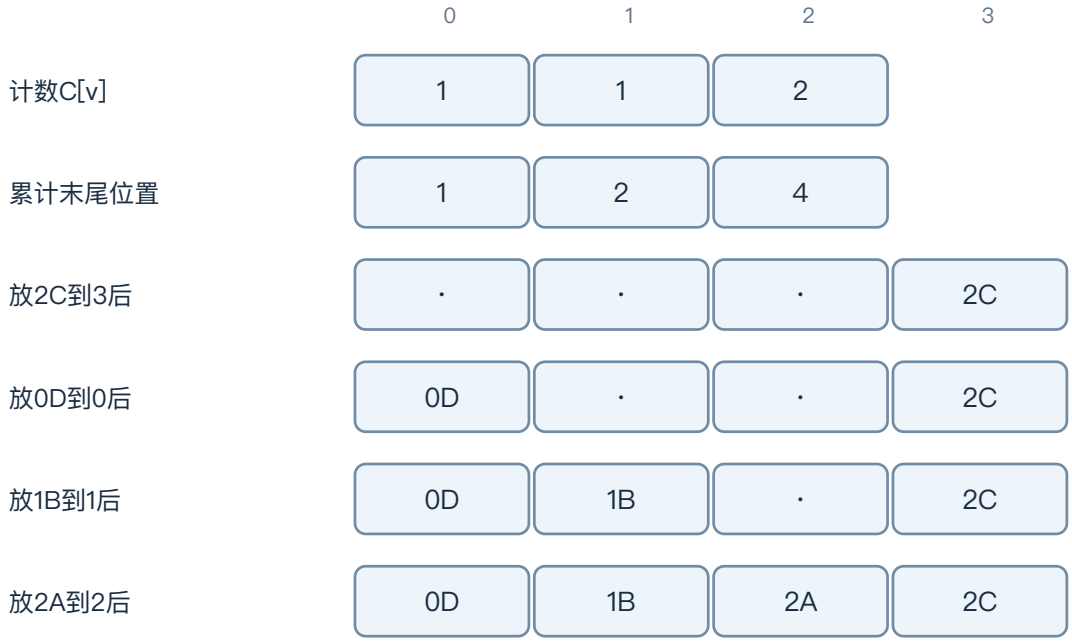
图 201　最后 gap1 中每个发生移动的插入

最后 gap1 在[2,1,4,3,6,5,8,7]上继续。插 1 移 2，插 3 移 4，插 5 移 6，插 7 移 8，才得到整体有序。

gap 有序并不等于整体有序，最后必须包含 1。较小 gap 排序后仍保持此前某些 gap 有序性质，但复杂度强烈依赖步长序列。朴素折半序列的最坏界可达平方级；其他序列有不同理论界。不能笼统给所有希尔排序写一个统一的 n log n。它通常原地但不稳定，因为远距离移动会越过相等记录。

20.5 计数排序：键范围小，就直接数有多少个

若 n 个键都是`[0,U)`整数，可以统计每个键出现次数，再计算前缀位置，将记录稳定放入输出数组。时间 Θ(n+U)、额外空间 O(n+U)，跳过了只做两两比较的模型，因此不受比较排序 Ω(n log n)下界直接约束。



前两行下标是键0/1/2；其后各行下标是输出位置0/1/2/3

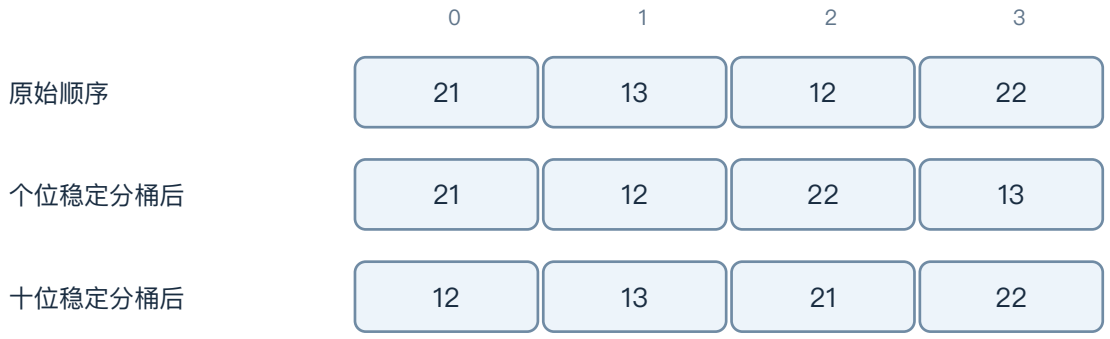
图 202　稳定计数排序的计数、前缀位置与每次写入

图的前两行下标指键值，后四行下标指出口位置，不要把这两种数组的下标混用。累计数 C[v]表示不大于 v 的记录总数，所以键 v 的当前最后空位在 C[v]−1。

例如记录`(2,A),(1,B),(2,C),(0,D)`，计数为[1,1,2]，前缀累计[1,2,4]。从右往左放：2C 放输出下标 3 并减 2 的累计；0D 放 0；1B 放 1；2A 放 2。得到 0D、1B、2A、2C，两个 2 仍保持 A 在 C 前。若用“每种值打印次数”替代放完整记录，就会丢掉标签等附加数据。

U 若远大于 n，比如 64 位任意整数范围，直接开计数表不可行。可以做基数排序，按若干位一轮稳定排序；每轮时间与基数有关，总成本还取决于位数。固定 32 位与随 n 增长的长整数，是不同的模型。

20.6 例 4：最低位优先基数排序，为什么每一轮要稳定



个位1桶[21]、2桶[12,22]、3桶[13]；十位1桶保留[12,13]

图 203　最低位优先两轮稳定分桶的完整顺序

排序两位十进制数`[21,13,12,22]`。先按个位稳定排序，得到 21、12、22、13；再按十位稳定排序，得到 12、13、21、22。第二轮同十位内部保留上一轮已确定的个位顺序，所以整体正确。

把每个桶也写出来：个位 1 桶[21]，2 桶[12,22]，3 桶[13]，按桶号收回得到 21/12/22/13；十位 1 桶按当前读入顺序得到[12,13]，2 桶得到[21,22]，收回得到 12/13/21/22。

最高位优先法在同一输入上先按十位分成 1 桶[13,12]、2 桶[21,22]，再分别按个位排成[12,13]、[21,22]，按十位桶次序拼接。这里不能把所有桶取出后全局再按个位排，否则会得到 21/12/22/13，把十位次序破坏。如果第二轮不稳定，十位为 1 的 12、13 可能被翻成 13、12，前一轮信息丢失。对最低位优先法，稳定性就是把新一位加到已有低位次序上的桥梁。最高位优先法则先按高位分桶，再在各桶内递归处理低位，结构不同，不能把两者的过程表混用。

20.7 比较算法时，先确定实际工作

完全排序输出全部有序序列，第 k 小只输出一个秩位置，前 k 大只关心一个集合，找众数关心频率。这些输出要求不同。图方便而先把全部排序，可能多做 n log n 工作；但若后面还要反复区间查询，一次排序也可能值得。

稳定性、额外内存、输入近有序程度、重复值数量、最坏延迟与元素移动成本，都影响选择。复杂度是重要限制，却不是只拿一个大 O 就能替代的全部条件。

20.8 线性选择的好轴，怎样保证至少丢掉常数比例

每 5 项一组，组内排序取第 3 项作中位数。再递归求这些组中位数的中位数 p。至少约一半组的中位数≥p，每个这样的完整组至少有 3 项≥其中位数≥p；另一半对称。因此至少约 3n/10 项不小于 p，至少约 3n/10 项不大于 p，扣除轴所在组和末尾不足 5 项组只损失常数。

取一个完整的 25 项输入，把组和组中位数都写出。每组排序只含 5 项，是固定规模工作：

原组	组内有序	中位数
9,1,8,2,7	1,2,7,8,9	7
20,3,19,4,18	3,4,18,19,20	18
25,5,24,6,23	5,6,23,24,25	23
17,10,16,11,15	10,11,15,16,17	15
22,12,21,13,14	12,13,14,21,22	14

中位数列表 7/18/23/15/14 有 5 项，直接排成 7/14/15/18/23，取中间 15 作轴。比如中位数 18 和 23 所在两组各有至少 3 项≥15，中位数 7 与 14 所在两组各有至少 3 项≤15，轴所在组本身还提供若干项。这个具体输入恰含 1 至 25 全部整数，所以小于 15 的有 14 项，大于 15 的有 10 项。

要找从 0 起第 12 小，以“子问题不超过 5 项就直接排序”为终止规则。为了让追踪不依赖原地交换顺序，本段划分按原顺序收集小于、等于、大于三块：

待选序列	目标局部秩	组中位数→选轴	划分后保留
上述 25 项	12	7,18,23,15,14⟶15	左侧[9,1,8,2,7,3,4,5,6,10,11,12,13,14]，仍秩 12
这 14 项	12	7,5,13⟶7	右侧[9,8,10,11,12,13,14]，秩 12−7=5
这 7 项	5	10,14⟶14	左侧[9,8,10,11,12,13]，仍秩 5
这 6 项	5	10,13⟶13	等值区恰占秩 5，返回 13

不足 5 项的尾组在本例取排序后下标 ⌊长度/2⌋ 的项，即偶数项时取较大的中间项；小规模中位数列表也用同一规则。因此两项 10/14 选 14，不是随意取平均 12。尾组与轴所在组只影响一般证明中的常数项，不破坏两侧各丢掉约 3n/10 项的规模界。

递归选择 p 花 T(n/5)，分组和划分花 cn，目标侧至多约 7n/10+常数。要证明总 O(n)，猜 T(n)≤An，代入右边得到约 A(n/5+7n/10)+cn=0.9An+cn。选 A≥10c 并用足够大的基础阈值吸收常数，就能压回 An。决定可行的是两个递归比例之和 0.9 小于 1，不是“选了中位数”这几个字。

如果每 3 项一组，简单比例分析可能得到 T(n/3)+T(2n/3)+cn，比例之和等 1，不能用同一代入证明线性。更细算法可能另有性质，但不能把 5 组法证明中关键常数删掉再声称所有分组大小都一样。

20.9 快排的期望比较数，也能按元素对来数

把互异元素按最终秩标 1 到 n。秩 i<j 的两项会直接比较，当且仅当在秩区间[i,j]中，最先被选作轴的是 i 或 j。若先选中间某项，它会把两项分到不同子问题，此后不再直接比较；若端点先成为轴，则比较一次。均匀随机轴下概率为 2/(j−i+1)。

将所有对的比较指标取期望并求和，得到 `Σ_{i<j} 2/(j−i+1)=O(n log n)`。把差 d=j−i 相同的项合起来，该差有 n−d 对，所以

$$\sum_{i<j}\frac{2}{j-i+1}=\sum_{d=1}^{n-1}\frac{2(n-d)}{d+1}\le 2n\sum_{d=1}^{n-1}\frac{1}{d+1}.$$

最后的倒数和按分母区间[1,2)、[2,4)、[4,8)分组，每组至多“项数×最大单项”≤1，共约 $\log_2 n$ 组，故总和 O(log n)，乘前面的 n 得到 O(n log n)。

这个证明不要求不同对是否比较相互独立，线性期望即可。它也说明随机化为何有效：固定输入顺序不影响最终秩区间内哪个轴最先出现的均匀性。

有大量重复时，普通两路比较与这份互异秩证明不直接匹配。三路划分将所有等于轴的项一次归并成不再递归的区间，更适合这种输入。对全相等 n 项，扫描一次后两侧都空，时间线性；若实现仍逐个将等值项留到一侧，就会与这个结论不同。

20.10 希尔排序为什么可能打乱相等项

考虑相等记录 5A 在前、5B 在后，中间穿插其他值。大 gap 子序列里某一 5 可能因与更大值比较向前移动多格，而另一 5 位于不同子序列保持不动，二者可以跨越而从未直接比较。这与稳定插入排序只移动严格较大且相邻逻辑区间的情况不同。

具体用 gap2 再 gap1 处理`[(2,A),3,1,(2,B)]`。gap2 的偶位置 2A、1 排好后，数组变`[1,3,2A,2B]`；奇位置 3、2B 排好后变`[1,2B,2A,3]`。最后 gap1 即使采用稳定插入排序，也只会保持当前等值次序 B 在 A 前，原来 A 在 B 前的顺序已无法恢复。

所以“每一趟子序列插入排序稳定”并不能推出整个希尔排序稳定：不同子序列之间在全局顺序上交错，跨步移动会改变它们的相对位置。需要稳定性时，除非使用带原始序号的完整比较键或专门变体，不应依赖普通希尔排序。

20.11 划分轴值与数组元素分开保存

划分过程中会交换数组，若只保存指向某格的轴引用，而该格被交换覆盖，后续比较可能在用一个变化的轴值。教学代码通常先把 pivot 的值或稳定记录拷贝出来，或者保证轴所在格在最终归位前不被改变；两种设计都可以，但实现要兑现这个不变式。

若元素很大，复制轴记录可能有成本，可保存比较键或使用合适引用策略；若键又随记录修改，则还要确认比较期间不变。泛型算法的元素类型假设，决定某个看似简单赋值是否真能视为常数。

20.12 第 k 小的 k，必须说明是否从零开始

六项数组从零起第 3 小是排序后的第四项，从一起第 3 小是第三项。进入右子问题时，零基秩减去小于区长度与等值区长度；若用一基，判定与减量要同步调整，不能只在入口减一次后又把内部条件改回一基。

等值区长度大于一时，目标只要落在这个整段秩范围内，返回轴值即可，无需再挑其中一项递归。若要返回原记录身份，多个相同键之间必须另定并列次序。数值答案唯一与记录答案唯一，也不是一回事。

快速排序的空区间与单项区间应立即返回。若递归区间包含轴点本身，轴在边界时可能再次传入原区间，造成无限递归；划分后大小必须严格缩小，这个终止检查比仅看输出是否有序更早发现问题。

20.13 练习与答案

练习 1。 三路划分遇到 a[i]>v，与右端交换后为何不增加 i？

解答：从未知区右端换来的值尚未被分类，可能小于、等于或大于 v。若直接 i++，这项会被错误归入等值区或已处理区。只有看过它并按规则放好，才可缩小未知段左边界。与左端交换时换来的是已知等值项，因此那个分支可同时增加 i。

练习 2。 Quickselect 求第 0 小，恰好每次选最大轴，会怎样？

解答：每次最大轴落在末尾，目标仍在大小 n−1 的左侧，只丢掉 1 项。划分成本 n、n−1、……，总 $\Theta(n^2)$。只递归一边不足以证明线性，需要证明被保留规模在整体上缩得足够快，随机期望或确定性好轴都在解决这件事。

练习 3。 计数排序 n=100、U=10^9，时间 $\Theta(n+U)$是否比比较排序更好？

解答：这里 U 主导，约十亿规模的初始化与扫描很不划算，比较排序的几百次比较量级通常更合适。渐近表达式中两个变量都要代入，不能只看到计数排序“线性”二字便丢掉 U。

阅读对照：Princeton 快速排序可核对二路划分和三路划分返回值的差别；老师《14.排序.pdf》PDF47—51 页给出选择算法，54—59 页给出 gap 排序与折半步长的输入敏感性。以上均按 PDF 查看器页码。

21 PA1：把题目变成能一步步执行的程序

这一章对照 PA1 的两份教师讲解，分别做 Filename、Interview、Gift、Graphics 和二维偏序。它们的共同困难是：老师写“使用动态规划”“分块”“二分”时，程序究竟存什么，下一步究竟改哪里？下面用另设的小输入把全过程展开。正式输入格式、数据规模和输出要求仍应对照当次题面；现有材料不足以把这五题称作全学期所有 OJ 题，也没有据此声称已经通过 OJ。

21.1 Filename：两个文件名只许插字符、删字符

设原串是 `ABC`，目标串是 `ABXC`。允许做两种事：删掉原串的一个字符，或者在任意位置插入一个字符。这里**没有成本为 1 的替换操作**。例如 `A` 变 `B`，必须先删 A，再插 B，共 2 次。

先不考虑大输入。我们可以问一个较小的问题：“原串的前几个字符，变成目标串的前几个字符，需要几次？”前缀允许为空。把这个问题的答案放进一个表：

$$D[i][j] = \textbf{把原串前 } i \textbf{ 个字符变成目标串前 } j \textbf{ 个字符的最少插删次数}.$$

这里i在0到原串长度n之间，j在0到目标串长度m之间。`D[0][3]=3` 的意思是，空串变成目标的前三个字符，必须插 3 次。于是表的第一行、第一列先填好：

$$D[0][j] = j, \qquad D[i][0] = i.$$

看右下角某格时，最后一步只有以下可能。

- 删原串的第i个字符。之前已经把前$i-1$个字符变成目标前j个，故成本是$D[i-1][j]+1$。
- 插目标的第j个字符。之前已经得到目标前$j-1$个，故成本是$D[i][j-1]+1$。
- 两边最后一个字符相同，直接保留。成本是$D[i-1][j-1]$。

末字符不同时，第三种“免费保留”不合法，也不能擅自写成对角线加 1，那会引入题目没有的替换。取所有合法方案的最小值。因为每种最后一步都列到了，且之前的前缀问题更小，这个递推既不会漏掉最优方案，也不会凭空制造方案。

21.1.1 完整填一次表，再沿表找回操作

对 `ABC` 和 `ABXC`，表如下。行标题是原串前缀，列标题是目标前缀。

$D[i][j]$	空	A	AB	ABX	ABXC
空	0	1	2	3	4
A	1	0	1	2	3
AB	2	1	0	1	2
ABC	3	2	1	2	1

别一次盯着整张表。只算第三行的后三格：

1. `AB→AB`：B 与 B 相同，可以继承 `A→A` 的 0，所以填 0。
2. `AB→ABX`：B 与 X 不同。删 B 需要 `D[1][3]+1=3`；在 `AB→AB` 以后插 X 只需 `D[2][2]+1=1`，所以填 1。
3. `ABC→ABX`：C 与 X 不同。上方 1 加 1 为 2，左方 1 加 1 也是 2，所以填 2。这个格子成本超过预算 1，但它并不会污染右下角：`ABC→ABXC` 的 C 可以匹配 C，直接取左上方 `AB→ABX` 的 1。

最后答案是右下角 1。恢复一条操作路径时，从右下角反走：C 相同，走到`(2,3)`；这一格来自左边加 1，所以记录“插 X”；B 相同走到`(1,1)`；A 相同走到`(0,0)`。反转记录后得到：保留 A、保留 B、插 X、保留 C。表需要$(n+1)(m+1)$格，每格只检查常数个来源，因此完整算法时间$O(nm)$，存全表空间$O(nm)$。只求次数、不恢复路径时，当前行只依赖上一行和本行左格，可以滚动成两行。

21.1.2 教师给的 LCS 等式怎么来的

L表示两串最长公共子序列的长度。“子序列”允许跳过字符，但保留下来的先后次序不能变。在 `ABC` 与 `ABXC` 里，`ABC` 就是长度 3 的公共子序列。保留它，只插 X 即可。

一般地，保留这L个字符，原串另外$n-L$个必须删，目标另外$m-L$个必须插，得到一个成本为

$$n+m-2L$$

的可行方案。还要证明没有更便宜的：任意插删方案最终留下来的原字符，一定按原次序同时出现在两串里，因此留下的数量r最多为L。至少要删$n-r$个、插$m-r$个，所以至少花$n+m-2r \geq n+m-2L$次。可行上界和不可突破的下界相同，等式才成立。

这个证明也告诉我们为什么不能把它照搬到允许替换的题。`A→B` 的 LCS 长度 0，公式给 2，而一次替换只花 1。

21.1.3 预算只有k：哪些格根本不必算

若题目只关心最少次数是否不超过k，状态`(i,j)`首先必须满足$|i-j| \leq k$。原因很直接：从长度i变成长度j，每次插删最多改变 1 个长度，光补长度差就至少要$|i-j|$次。一个总成本不超过k的编辑路径，每个中间前缀格也不可能已经花掉超过k次。

例如刚才$k=1$时，只保留主对角线左右各 1 格。横线表示本次不计算，读取时按无穷大处理：

行	$j=0$	1	2	3	4
$i=0$	0	1	—	—	—
1	1	0	1	—	—
2	—	1	0	1	—
3	—	—	1	2	1

最后仍是 1，最优路径没有被剪掉。带内的 2 可以保留为 2，也可以截成$k+1=2$；我们只需区分“0 到k”和“已超预算”。**在带内与成本够小是两回事**，`ABC→XYZ` 始终存在相同长度的格，答案仍然是 6。

滚动数组最容易错在“这个槽现在对应哪一列”。把每行存成“最左列号+一段数组”，访问原列j时，用j减去本行最左列得到槽号。不在这一行保存范围的格，统一读成$k+1$。下面的完整实现每行只清带内的格；若仍每次清空$m+1$格，内层虽缩窄，初始化就会把时间拖回$O(nm)$。

```
def insert_delete_with_budget(a, b, k):
    if k < 0:
        raise ValueError("budget must be nonnegative")
    n, m = len(a), len(b)
    if abs(n - m) > k:
        return None
    inf = k + 1
    prev_lo = 0
    prev = list(range(min(m, k) + 1))
    for i in range(1, n + 1):
        lo, hi = max(0, i - k), min(m, i + k)
        cur = [inf] * (hi - lo + 1)

        def above(j):
            pos = j - prev_lo
            return prev[pos] if 0 <= pos < len(prev) else inf

        for j in range(lo, hi + 1):
            if j == 0:
                cur[0] = i
                continue
            delete = above(j) + 1
            insert = cur[j - 1 - lo] + 1 if j > lo else inf
            best = min(delete, insert)
            if a[i - 1] == b[j - 1]:
                best = min(best, above(j - 1))
            cur[j - lo] = min(inf, best)
        prev_lo, prev = lo, cur
    answer = prev[m - prev_lo]
    return answer if answer <= k else None
```

`above(j)`通过原列号取上一行；`cur[j-lo]`通过原列号写当前行。两者各自减自己的最左列。`inf` 不是某个非常大的神秘数字，而是“已超预算”的统一标记，超过它的候选可以截断。行内从左向右，所以插入所需的本行左格已经算好。

本例第 2 行保存$j=1,2,3$的`[1,0,1]`，第 3 行保存$j=2,3,4$的`[1,2,1]`。两行的槽 0 对应的原列不同；算第 3 行原列 2 时，正上方在上一行槽 1，左上方在上一行槽 0。带的位置移动并没有改变递推式，只改变了坐标到数组槽的换算。

每行至多$2k+1$格，清空、填写各一次。总时间为$O((n+1)(k+1))$，辅助空间$O(k+1)$，字符串本身的存储另计；当k大于串长时，实际上不超过完整表的格数。只求次数时这是完整算法，若要恢复具体编辑路径，则需另外保留来源或采用分治恢复，不能说“两行数组里仍保存了全部历史”。

教师讲解PDF 第 1 页关于可达目标有一句“大于k”的文字，与前后递推意图相反；本章按长度差下界和“不超过预算”的目标核对，不沿用该笔误。

21.2 Interview：倒着恢复入座，环上的“第几个”怎样查

教师讲解把原过程按时间倒放：先放最后到来的人，再找前一个人的座位。每放一个人，一把空椅变成占用；以后只需查询剩下空位里的第m个。这一步让删除变成了逐个标记，不必模拟已经离开的人。

先固定本章约定：座位从 1 编号，顺时针递增，末尾绕回 1；**从当前位置之后开始数，当前位置不参与本次数数**。正式题面如另有约定，需要相应调整。

8 把椅子，当前已占用第 2 把，其余空位为`[1,3,4,5,6,7,8]`。向后数第 3 个空位，是 3、4、5，故放到 5。再从 5 之后数第 3 个空位，是 6、7、8，放到 8。再从 8 之后数第 3 个空位，是 1、3、4，放到 4：

```
椅子编号       1 2 3 4 5 6 7 8
初始空位标记   1 0 1 1 1 1 1 1   当前2
放到5后        1 0 1 1 0 1 1 1   当前5
放到8后        1 0 1 1 0 1 1 0   当前8
放到4后        1 0 1 0 0 1 1 0   当前4
```

朴素实现每次沿环检查椅子。做n次查询，每次可能绕过$O(n)$个位置，总计$O(n^2)$。下面改变“跳过一段”的方式，数数规则不变。

21.2.1 分块：一次跳过的不是一把，而是一整块

把 8 把椅子分成 4 块，块长$B=2$。在“放到 5 后”的状态里，每块空位数是：

块	椅子	空位标记	块计数
0	1、2	1、0	1
1	3、4	1、1	2
2	5、6	0、1	1
3	7、8	1、1	2

从 5 后找第 3 空位，先处理起点所在块的尾部：椅 6 空，把剩余名次从 3 减到 2。到块 3，计数 2 正好覆盖剩余名次，不能跳过它，逐格扫描 7、8，第二空位是 8。若某块只有 1 空而还要第 3 空位，跳过整块即可把剩余名次从 3 减到 2，不必检查块内每把椅子。

一般需要处理起点的零碎部分、跳过整块、扫描目标块。最多$O(B)$个单格和$O(n/B)$个块，所以一次查询是

$$O\left(B+\frac{n}{B}\right).$$

B太小，块多；B太大，块内扫描多。取B接近$\sqrt{n}$使两项接近，得到$O(\sqrt{n})$一次，$O(n\sqrt{n})$总时间。教师PDF 同段先写了$O(1)$的目标、后又给$O(\sqrt{n})$分块过程；此实现应按后者记账，不能称为常数查询。占用一个座位只改一个标记、把对应块计数减 1，修改是$O(1)$。

21.2.2 树状数组：先把环转成全局名次

若已经会前缀和树状数组，可以把每把空椅记为 1。`sum(p)`是从椅 1 到椅p的空位总数；全部空位数记为S。若当前位置p已经占用，设$r=\mathrm{sum}(p)$。从p之后数第m个空位，换成从椅 1 开始数的全局名次为

$$q=((r+m-1)\bmod S)+1.$$

为什么先减 1 再取模？普通名次是从 1 数，取模结果却从 0 数；先把第 1 名移到 0，绕环，再加 1 恢复。刚才放到 5 后，空位`[1,3,4,6,7,8]`，$S=6$、$r=3$、$m=3$，所以$q=6$，全局第 6 空位正好是 8。接着$p=8$被占，$S=5$、$r=5$，再数第 3 个，$q=((5+3-1)\bmod 5)+1=3$，全局空位`[1,3,4,6,7]`的第 3 个是 4。“找全局第q个 1”也能在树状数组中直接做。8 格在最初占用 2 后的树状数组是：

```
原始标记 a[1..8] = 1 0 1 1 1 1 1 1
树状节点 t[1..8] = 1 1 1 3 1 2 1 7
```

`t[4]=3` 管椅 1 到 4，`t[6]=2` 管椅 5 到 6，`t[8]=7` 管全部。寻找第 4 个 1 时，保留 `pos=0,need=4`，依次试 8、4、2、1 的跳长：

跳长	试探位置	那一块的计数	处理后 `pos,need`
8	8	7	7 不小于 4，目标在此前，保留 `0,4`
4	4	3	跳过这 3 个，改为 `4,1`
2	6	2	2 不小于 1，保留 `4,1`
1	5	1	1 不小于 1，保留 `4,1`

结束时 `pos=4` 表示“肯定在目标之前的最后位置”，答案是 5。只在块计数**严格小于**剩余名次时跳过，否则可能越过目标。位数每次减 1，所以一次$O(\log n)$。更新椅 5 为占用，依次把 `t[5]`、`t[6]`、`t[8]`减 1。查询、更新均保持“树状节点存它所管区间的空位和”，整个恢复$O(n\log n)$。

上面走的是树状数组的“按块跳”。普通前缀和的块也可以明确算出：定义 `lowbit(p)=p & -p`，它是二进制 p 最低位的 1 所代表的数，即整除 p 的最大 2 的幂。例如 6 的二进制是 110，lowbit 为 2，所以 `t[6]`管末尾 2 格，即 5、6；减去 2 到 4，再取 `t[4]`管 1 到 4，两块不重叠，合起来恰是前 6 格。

以下三个函数足以执行本题的查名次与占座。t 采用从 1 开始的树状数组，`t[0]`不用，已经按当前空位标记建好；开始全空时 `t[p]=lowbit(p)`。`occupy` 的前提是这把椅子当前确实为空，不能同一位置重复减 1。

```python
def free_prefix(t, p):
    result = 0
    while p > 0:
        result += t[p]
        p -= p & -p
    return result


def kth_free(t, rank):
    n = len(t) - 1
    if not 1 <= rank <= free_prefix(t, n):
        raise ValueError("rank outside free seats")
    pos = 0
    step = 1
    while step + step <= n:
        step += step
    while step:
        nxt = pos + step
        if nxt <= n and t[nxt] < rank:
            rank -= t[nxt]
            pos = nxt
        step //= 2
    return pos + 1
```

```
def occupy(t, p):
    if not 1 <= p < len(t):
        raise ValueError("seat index outside tree")
    while p < len(t):
        t[p] -= 1
        p += p & -p
```

`kth_free` 每次丢掉一个二进制位的尝试，留下“pos 及以前已经整块跳过，rank 是余下还要数的名次”。越过一块时从 rank 扣掉的就是这一块空位数，不能扣座位数。到跳长 1 也试完，目标紧挨在 pos 之后。占座则向上改所有覆盖该格的树状节点；例如占 5 会改 5、6、8，不改只管 1 到 4 的 `t[4]`。

只有$S > 0$时公式才有意义。m很大时先对S取模可避免计数溢出，但应保留从 1 开始数的规则；`m=0` 若题面不允许，就不能悄悄解释成“第 0 个空位”。

21.3 Gift：四组选择，怎样少算一半指数

输入n组数对(a_i, b_i)，每组必须恰好选一个，问总和不超过预算P的方案数。不同组的两个选项即使数值相同，也仍然代表不同选择。下面这 4 组贯穿整个算法：

```
第1组 (1,4)    第2组 (2,5)
第3组 (0,3)    第4组 (1,2)    预算 P=7
```

直接递归每组分两个分支，共$2^4 = 16$个叶。大到 40 组就是2^{40}，约一万亿。中间相遇先切成两半：前 2 组列全部左半和，后 2 组列全部右半和。一个完整方案唯一对应“一份左选择+一份右选择”，所以只要统计和不超过 7 的左右配对。

左选择	左和	右选择	右和
1、2	3	0、1	1
1、5	6	0、2	2
4、2	6	3、1	4
4、5	9	3、2	5

对每个右和r，需要左和$\ell \le P - r$。左数组`[3,6,6,9]`排序后，可以用 `upper_bound(P-r)`找第一个超过阈值的位置；这个位置前有几项，就有几个合法左方案。

右和r	左和阈值$7 - r$	合法左数组项	本轮贡献	累计
1	6	3、6、6	3	3
2	5	3	1	4
4	3	3	1	5
5	2	无	0	5

两个 6 分别来自“1、5”和“4、2”，删掉任何一个都会少算。这里在数方案，不是在列不同总和。

21.3.1 教师拓展里的两次线性归并怎么做

第一处归并用于生成有序半和。从只有空选择的`[0]`开始，每添一组(a, b)，把旧列表分别整体加a、加b，得到两个有序列表，再归并。左半的全过程：

```
空组                      [0]
添(1,4): [0]+1, [0]+4  -> [1,4]
添(2,5): [1,4]+2       -> [3,6]
         [1,4]+5       -> [6,9]
归并                      [3,6,6,9]
```

右半同理：`[0]→[0,3]→merge([1,4],[2,5])=[1,2,4,5]`。每次列表翻倍，各层处理量是$2+4+\cdots+2^h<2^{h+1}$，其中h是这一半的组数，所以全部有序枚举为$O(2^h)$，无需最后再排序。

第二处归并用于计数。右和从小到大走，允许的左和阈值不断下降，左指针只会往回退。初始把指针放在左数组末尾 9：

```
r=1，阈值6：9太大，退到第二个6。可用3项，累计3。
r=2，阈值5：两个6依次退掉，停在3。可用1项，累计4。
r=4，阈值3：3仍可用。再加1，累计5。
r=5，阈值2：3也退掉，指针变−1。再加0，仍为5。
```

右指针走完一次，左指针也最多退完一次，配对阶段是$O(|L|+|R|)$。加上有序生成，总时间与空间都是$O(2^{\lceil n/2\rceil})$量级；这里假设一次加法和计数的机器字运算成本为常数，且相应数值放得下。普通“生成后排序、每项二分”版本也完全正确，时间通常多一个n量级因子。

负数不妨碍排序后的阈值计数，但会破坏“部分和已超过预算就立刻剪枝”的推理。例如先选 5，后面还能选−10，最后可能合法。数值范围也分开估：和的绝对值最多是各组最大绝对值之和；方案数最多是2^n。和装得下 32 位，方案数未必装得下。

中间相遇的“把一次完整选择唯一拆成两份”是正确性的关键。它也用来检查：奇数组怎么分不影响配对一一对应，空的一半要保留一个和 0，不能生成空列表。

21.4 Graphics：点在线哪边，为什么能连续砍掉一半线

教师讲解的核心是“将端点分别排序后连成有序、不相交的分界线，按叉积二分”。先画一个明确的几何小例。下边界$y=0$有端点$x=1,3,5$，上边界$y=4$有端点$x=2,4,6$，依次连成 3 条线。第j条从下向上定向，查询点$Q=(3.5,2)$。

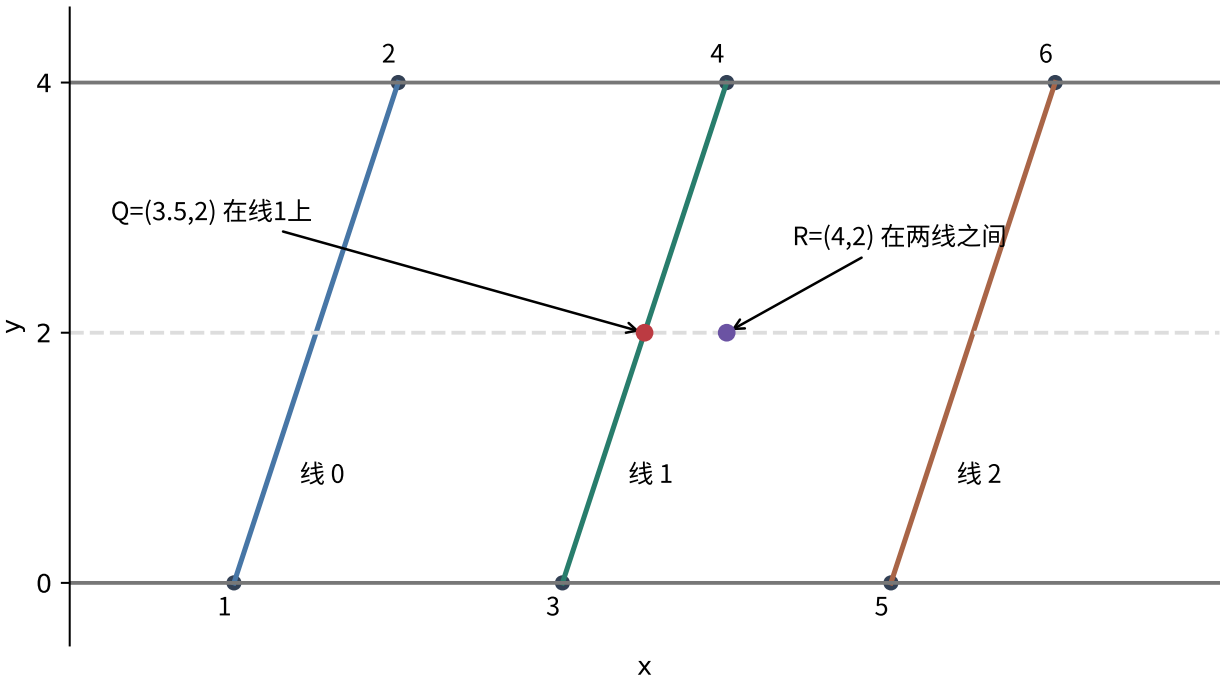


图 204　三条分界线在上下边界的顺序一致，因此在中间高度也保持左到右顺序；查询点 Q 恰在线 1 上；线上归属需题目约定。

二维向量(u_x,u_y)、(v_x,v_y)的叉积在这里是一个数：

$$\operatorname{cross}(u,v)=u_xv_y-u_yv_x.$$

对线段起点A、终点B和点Q，先作差得到$u=B-A$、$v=Q-A$。本图y向上，线从下向上，叉积为负表示 Q 在线的右侧，正表示左侧，零表示线上。不要把这个正负约定搬到屏幕y向下的坐标系而不检查。

三条线逐条手算：

线	A	$B-A$	$Q-A$	叉积	Q 的位置
0	$(1,0)$	$(1,4)$	$(2.5,2)$	$1\cdot 2-4\cdot 2.5=-8$	右侧
1	$(3,0)$	$(1,4)$	$(0.5,2)$	$2-2=0$	线上
2	$(5,0)$	$(1,4)$	$(-1.5,2)$	$2+6=8$	左侧

本例 Q **恰好在线 1 上**。若题目规定线上归右边区域，则问“有多少条线在 Q 左侧或经过 Q”，应把叉积≤ 0算进前半；若线上归左区，就用<0。图中的区域说明也应按这一边界约定理解，不能把线上点说成严格处于两线之间。

为演示普通内部点，使用图中的$R=(4,2)$，叉积依次为−10、−2、6，谓词“Q 在此线右侧”依次为真、真、假。找第一个假，过程是：

```
初始候选线下标 [0,3)
mid=1：cross=-2<0，线1也在点左边，答案边界在[2,3)
mid=2：cross= 6>0，线2在点右边，答案边界在[2,2)
结束：有2条线在点左边，点在第2个区域（从0编号）。
```

二分合法是因为谓词只变一次。证明也不难：在高度y令$t=y/4$，第j条线横坐标为

$$x_j(y)=(1-t)x_j^{\text{下}}+tx_j^{\text{上}}.$$

上下两列端点都按相同次序递增，且$0\le t\le 1$，所以$j<k$时两端差都为正，其非负加权和仍为正，得到$x_j(y)<x_k(y)$。查询点从右侧走到左侧只会经过一次分界。若端点顺序交叉，这个证明就失败，二分也可能错；先排序的依据必须来自题面结构。

朴素逐条测试是$O(n)$一次；有序后每次二分$O(\log n)$，预处理排序$O(n\log n)$。整数坐标可以用整数叉积精确判零。若坐标绝对值至多M，差可到$2M$，两个乘积各可到$4M^2$，相减的绝对值可到$8M^2$，中间运算的类型要按这个上界选择。先用窄整数做乘法，再赋给宽整数，救不了已经发生的溢出。

21.5 二维偏序：先算清楚谁可以接谁，再加速

教师 PPT 给三元组(a_i,b_i,c_i)：选一些**不重复索引**，可以按坐标顺序组织成链，要求相邻两项的a、b都不下降，最大化所选c之和。这里允许重新组织所选索引，不能擅自改成“必须保持原输入顺序的子序列”问题。后者的依赖关系不同。

取 5 项：

编号	a	b	权重c
A	1	3	4
B	2	2	5
C	2	4	6
D	3	3	7
E	4	5	2

先按a、再按b排序，本例顺序就是 A、B、C、D、E。A 不能接到 B 前，因为b从 3 降到 2；B 可以接 C；A 和 B 都能接 D；C 和 D 彼此不可接，但都能接 E。

设f_i为“必须以第i项结尾的最大链权”。以i结尾的链，要么只有自己，要么在一条合法前驱链后加自己。因此

$$f_i=c_i+\max\left(0,\max_{j<i,\ b_j\le b_i} f_j\right).$$

外面的 0 表示从自己重新开始；里面的j只能在已处理项里，并满足第二坐标不下降。按排序顺序，第一坐标已经不会下降。

当前项	可以接的前驱及其f	选中前驱	当前f	最优链
A	无	无	4	A
B	无，A 的$b=3>2$	无	5	B
C	A:4、B:5	B	11	B⟶C
D	A:4、B:5；C 的$b=4>3$	B	12	B⟶D
E	A:4、B:5、C:11、D:12	D	14	B⟶D⟶E

这张表本身就是一个正确的$O(n^2)$算法：每行扫所有前驱。如果加速版输出不是 14，可以回到某一行查是哪个前驱漏了。

21.5.1 前缀最大值树状数组：整块记住“最好的一条链”

第二坐标原值为 2、3、4、5，压成秩 1、2、3、4。压缩只保存大小关系，不保存距离。定义 `query(r)`返回所有第二坐标秩不超过r的已处理项中，最大的f。在本例中树状节点管的区间是：

```
t[1] 管秩1       t[2] 管秩1..2
t[3] 管秩3       t[4] 管秩1..4
```

初始各项为 0，逐点处理：

项	秩	查询经过哪些节点	查得前驱最大值	更新后 `t[1..4]`
A	2	`t[2]`	0	`[0,4,0,4]`
B	1	`t[1]`	0	`[5,5,0,5]`
C	3	`t[3]`、`t[2]`	5	`[5,5,11,11]`
D	2	`t[2]`	5	`[5,12,11,12]`
E	4	`t[4]`	12	`[5,12,11,14]`

为什么 C 没有被 D 接上？D 只查秩 1 到 2，而 C 在秩 3；节点 `t[4]`虽存着 11，查询秩 2 时并不读它。查询向下减去最低位 1，更新向上加最低位 1，都只经过$O(\log n)$个区间。每个节点总存其所管区间里已经出现过的最大f，所以查询恰好等于平方算法里那一项“所有合法前驱的最大值”。总时间$O(n\log n)$。

对应查询与更新也可照着节点区间写出。t 初始为 0，表示允许空前驱链；r 是当前第二坐标压缩后的 1 起名次，value 是已经算好的f_i：

```
def best_prefix(t, r):
    answer = 0
    while r > 0:
        answer = max(answer, t[r])
        r -= r & -r
    return answer


def improve(t, r, value):
    if not 1 <= r < len(t):
        raise ValueError("rank outside tree")
    while r < len(t):
        t[r] = max(t[r], value)
        r += r & -r
```

本例 C 的 r=3，查询先读 `t[3]`负责原秩 3，再减 lowbit(3)=1 到 2，读 `t[2]`负责原秩 1、2，再减 2 到 0 结束。得到前 3 秩的最大值；更新则反向改所有覆盖 r 的块。每个节点只需不断增大其最大值，不能用这段代码实现任意删除或降低旧f，那时原最大值可能失效，需要另一种维护办法。

这里的聚合是 max，不能仿照前缀和把两个查询相减求任意区间最大值。若要恢复链，节点同时保存“最大值、取得它的前驱编号”，每项把查询得到的编号存为 parent[i]。从 E 回溯得到 E、D、B，再反转就是 B→D→E。

相等条件会直接改程序：若a要求严格增加，同a的 B、C 必须先一起查询、再一起更新，否则 B→C 被错误放行；若b也要求严格增加，查询上界应为当前秩减 1。本题教师原模型是两坐标不下降，不要把“防同组污染”的技巧不加区分地加进来，反而漏掉本来合法的同a链。

同坐标的正权项在允许重排、非降的模型中可以全部选，合并成权重和；非正权项在允许空解时可丢弃。若要求至少选一项且全部权重为负，答案应是其中最大的负数，不能直接输出 0。每个预处理都应对应题面的一个条件。

21.5.2 教师 PPT 的分治为什么一定先左、传播、再右

还可以用分治加速同一个 DP。取 3 个两坐标都递增的点，权重 5、7、11。正确的f是 5、12、23。

```
初始下界 f=[5,7,11]
先解左点1：f1=5
左点贡献到右点：f2=max(7,5+7)=12，f3=max(11,5+11)=16
再解右区间[2,3]：点2贡献给点3，f3=max(16,12+11)=23
```

若先把右半算完，右内部只能得到$f_2 = 7, f_3 = 18$，再传播左点会把f_2改成 12，却未必重新让它贡献给点 3，最后漏掉 23。普通归并排序的“两半都排完再合并”不能原封不动套来解带依赖的 DP。

教师分治子任务的入口承诺是：“进入区间$[l, r]$前，更早区间对本区间的贡献已经写入f。”先递归左半使其最终定值，再按b有序扫描，把左侧合法前驱的最大值传给右半，然后才递归右半。维护按原编号的左右归属与按b的扫描顺序，需稳定划分或辅助索引，不能排序后把原依赖次序弄丢。每层各项被线性处理，总层数$O(\log n)$，所以也能做到$O(n \log n)$。

21.6 先自己走，再对答案

1. Filename：AB→BA，只准插删，预算 1 能成功吗？指出表里哪条路径达到最小值。
2. Interview：空椅为[1,3,4,6,7]，当前位置 8，从之后数第 7 个空位，落在哪里？
3. Gift：左半和[1,1,4]、右半和[2,3]，预算 4，写出每个右和贡献的数量。
4. 二维链若要求a严格增加，前面 5 项的最优值是否还为 14？

解答：

1. 不成功。保留 A，删原来的 B，再在 A 前插 B，共 2 次；或保留 B，删 A 再在后面插 A。长度为 2 的公共子序列不存在，LCS 长度只有 1，故至少$2 + 2 - 2 = 2$次。
2. $S = 5, r = 5, m = 7$，全局秩$((5 + 7 - 1) \bmod 5) + 1 = 2$，第 2 空位是 3。手走也一样：1、3、4、6、7、1、3。
3. 右 2 允许左≤2，两份 1 都算，贡献 2；右 3 允许左≤1，仍贡献 2，总 4。
4. 仍是 14，链 B→D→E 的a为 2、3、4，合法。但 C 不能再接同a的 B，故f_C从 11 改为 10（A→C）；最终最大值碰巧不变，不能只比较最后答案就以为同组处理写对了。

本章主要依据课程 PA1 两份讲解。动态规划的“状态先定义为明确子问题、再按依赖次序求值”可对照 Jeff Erickson 的 Dynamic Programming 第 3 章；其教材采用通用编辑距离模型，使用时必须重新检查本题只准插删这一操作集合。本章的小输入、状态表和实现边界均按上述 PA 模型重新推演。

22 PS01：把“会背复杂度”变成“自己能数出来”

教师原始习题单：第 01 次习题共 32 题。下面保留原题号，重新解释题目要你做什么，再把能落在纸上的计算、证明或实验办法写出来。编译、测时间和个人经历类问题没有一个替所有人编好的“标准经历”；这些题给出可照着做的操作和检查结果的办法。

读这一章时，准备一张纸。遇到程序，先写变量下一步变成多少；遇到证明，先试三个小输入；遇到复杂度，先决定到底数什么。答案里的上界、下界、平均值和实测时间是四种东西，不能相互顶替。

22.1 1 · 计算工具：算法不一定写在电脑里

算盘、尺规和纸笔都能执行计算。区别在于，它们允许的基本动作不同。算盘可以拨珠，尺规可以画直线、画圆、取交点，纸笔可以逐位写数。一个过程要说清楚，首先要说明输入如何放进去。例如用算盘算 $27+35$，先用十位和个位的珠位表示 27，再加 3 个十和 5 个一；个位从 7 加 5 得到 12，要留下 2 并向十位进 1，最后读出 62。只有“把数加起来”这句话还不是给执行者的完整步骤。

可以把十进制竖式加法写成程序：从最低位起，把两位数字和进位相加；结果对 10 取余作为本位，除以 10 取整作为新进位；移动到高一位，直到两串数字和进位都用完。每次处理一位，长度为 d 的输入需要 $O(d)$ 个这样的逐位动作。

C++把动作写成有精确语法的语句，变量、条件和循环都有规定的含义；自然语言描述更容易夹带“适当处理”之类的空白。反过来，尺规的“取两个圆的交点”是它的原语，在普通电脑里却要另外表示坐标、处理精度、求方程。比较两种算法快慢以前，需要把基本动作放在同一个计算模型里。

可以自己试一个检查：让完全没参与设计的人照步骤算 $99+1$。若他不清楚最后的进位要不要额外写一位，说明描述漏了边界，而不是他“应该懂”。

22.2 2 · 曹冲称象与刻舟求剑：保存的究竟是什么

曹冲称象需要的量是“大象使船增加的排水重量”。把大象放上船，在船舷记录水线；移走大象，再逐块放石头，直到同一条水线重新与水面重合。在船本身、船中其他载荷、水的密度等条件保持一致时，相同吃水意味着相同排水体积，因此石头总重量等于大象重量。最后称石头就把一个难称的大物体转成了若干容易称的小物体。

这里的记号标在船上有用，是因为水线要表示船相对水面的吃水状态。刻舟求剑要找的却是剑相对河床的位置；船移动后，刻线跟着船走，剑没有跟着走。记号保存了“剑落水时相对船的位置”，而人想恢复“现在相对河床的位置”，中间缺了船的位移。若剑在落水后还受水流影响，甚至连固定的河床位置也要另作假设。

用算法语言说，输入信息必须足以确定输出。设落剑时船在地面坐标 x_0，记号相对船的位置为 r，则剑的初始地面坐标是 x_0+r。船后来到 x_1，拿同一记号推断的位置是 x_1+r，误差正是 x_1-x_0。把坐标写出来，就能指出错误发生在哪一步。

这个题也提醒我们：程序可以按预定步骤顺利结束，却算错问题。有穷性保证会结束，正确性还需要输入、模型和推理都对。

22.3 3 · 死循环：局部能查，万能检查器做不出来

先看最常见的一类错误：

```
while (lo < hi) {
    int mid = lo + (hi - lo) / 2;
    if (a[mid] < target) lo = mid; // 长度为1时不再缩短
    else hi = mid;
}
```

若 `lo=4, hi=5` 且 `a[4]<target`，就有 `mid=4`，更新后还是 `[4,5)`，此后完全重复。修正通常是 `lo=mid+1`，但仍要检查这是否符合想要的返回语义。调试时把 `lo,hi,mid` 打印出来，比一直盯着循环体更容易看见状态没变。

证明一个循环终止，可以找“非负整数、每轮严格减少”的量。这里取 $hi-lo$。它不能降到负数，每次又至少减 1，因此轮数有限。只写“区间变小”不够，要检查每一条分支，特别是长度 0 和 1。机器整数溢出还可能让本应下降的量突然变负或绕回去，实际程序必须另查类型。

为什么不能写一个检查任意程序的万能算法？假设存在 `halts(P,x)`，能判断程序 P 在输入 x 上是否结束。构造程序 D：输入程序 P，若 `halts(P,P)` 说会结束，D 就永远循环；若说不会结束，D 立刻结束。再问 D 在输入自己时会怎样。判断说会结束，D 偏不结束；判断说不结束，D 偏结束，产生矛盾。因此这个假设不成立。

这不妨碍我们检查具体程序、有限状态程序，或要求循环带下降量的受限语言。正确的实践是组合使用不变量、边界测试、超时和审查；超时发现“这次没在限定时间结束”，本身不能证明永远不结束。

22.4 4 · Failstone：别因为长得像冰雹序列，就停止证明

原题递归在 p=q 时返回 1；否则变换参数并把返回值加 1。下面明确讨论正整数 $p,q\ge 1$，并假定算术精确、没有溢出。若允许 p=0,q>0，奇数 q 时会原样重复 `(0,q)`，结论就变了。“自然数”是否包含 0 必须先约定。

可以不用深递归，而是直接数经过多少个状态：

```
def failstone(p, q):
    if p <= 0 or q <= 0:
        raise ValueError("本实现只接收正整数")
    states = 1
    while p != q:
        if p > q:
            p, q = p - q, p
        elif q % 2:
            p = 2 * p
        else:
            q //= 2
        states += 1
    return states
```

例如从 `(1,3)` 开始，依次得到 `(2,3)`、`(4,3)`、`(1,4)`、`(1,2)`、`(1,1)`。共有 6 个状态，所以返回 6。要把末尾的基础情形也数一次；只数变换次数会少 1。

困难在于，单步最大值不总下降：`(2,3)` 变成 `(4,3)` 时还变大了。我们把有限的几步捆成一组。若 p<q 且 q 为偶数，减半 q 后，新的最大值小于原 q。若 p<q 且 q 为奇数，就不断将 p 加倍。若中途相等立即结束；否则一定能到某个 $p^*<q<2p^*$，因为正整数反复加倍会超过固定 q，而且奇数 q 不可能等于偶数 $2p^*$。接下来三步是

$$(p^*,q)\to(2p^*,q)\to(2p^*-q,2p^*)\to(2p^*-q,p^*).$$

最后一步合法，因为第二项 $2p^*$ 是偶数，且 $0<2p^*-q<p^*$。这组变换后两项都小于原来的 q。若起初 p>q，先走一步得到 `(p-q,p)`，再进入刚才的小项在前的情形，后续一组同样让最大值降到原 p 以下。

因此，每组都在有限步内结束，并让正整数最大值严格下降；这样的组不可能无限多。这就证明了原过程对正整数一定结束。它与尚未普遍解决的另一道递推问题相似，并不能把那个问题的未知状态搬过来。

22.5 5 · 几何分布：要数到第一次成功为止

每轮独立，成功概率固定为 p，直到第一次成功的轮数记为 K，且成功那轮也计数。那么

$$P(K=k)=(1-p)^{k-1}p,\quad k=1,2,\dots$$

前 $k-1$ 轮失败，第 k 轮成功，两部分概率相乘。求期望有个很省事的办法：先做一轮，用掉 1；若失败，以后仍然面对同样的问题。因此 $E[K]=1+(1-p)E[K]$，移项得 $E[K]=1/p$。

取$p=1/2$，先列出最短的几类运行：第 1 轮成功的概率为$1/2$；先失败再成功为$1/4$；先失败两次再成功为$1/8$。它们的成本分别是 1、2、3。第三类不会因为写了“平均 2 次”就被删除。若最多做 3 轮，未成功也停止，实际轮数的期望是$1+1/2+1/4=7/4$，而未成功概率是$1/8$；这是另一个有明确失败出口的程序。

例如独立地掷公平硬币直到第一次正面，平均需要 2 次；成功率为 0.2 的独立尝试平均需要 5 次。平均 5 次不表示第五次一定成功，前 20 次都失败的概率仍是 $0.8^{20} > 0$。

符合模型的计算例子是：从固定集合均匀独立抽样，直到抽到某个目标；或者每轮独立选择随机候选，命中合法集合的比例恒为 p。网络重试常有服务器状态变化、限流、相关故障，不能只因为“也在重试”就套几何分布。若允许最多重试 m 次，随机变量还变成了截断版本，期望也要重新算。

22.6 6 · 分解大整数：先忠于题目的蛮力算法

原流程依次试素数 2、3、5、7.....；找到因子就除掉，直到剩余数为 1。这里没有写“试到平方根便停止”，所以不能在分析中悄悄加上这项优化。

最坏例子可以取接近 10^{100} 的素数 N。每个小于 N 的候选素数都试不出因子，直到 N 本身才整除成功，测试量约为素数计数函数 $\pi(N)$。用素数分布的渐近估计 $\pi(N) \sim N/\ln N$，在这个量级约为 $10^{100}/(100\ln 10) \approx 4.34 \times 10^{97}$ 次。这个数只是数量级估计，不能当成对某一个具体 N 的精确素数数量；光生成所有候选素数还另有成本。

最好让待分解数是很小素数的幂。取 $N = 2^k$，每次都能除以 2，约经过 $k = \log_2 N$ 次成功除法。10^{100} 附近的 2 的幂需要约 332 或 333 次，具体取决于选的是哪一个幂。“同样约 100 位十进制数字”，计算量能相差几十个数量级。

若采用平方根停止并在最后输出剩余因子的改进版，素数输入只需试到 $\sqrt{N}$。那是另一个更好的算法，回答时应先分析原流程，再单独说明改进。大整数除法本身也不必是单位成本；若按位模型收费，还要乘上相应位长的运算成本。

22.7 7 · 图灵机递减：借位时到底改了哪几位

设纸带上是最高位在左的二进制正整数，读头先移到最低位。做减 1 时：读到 0 就把它改成 1 并向左，继续借位；读到第一个 1 就把它改成 0，借位结束。之后根据题目约定删去多余前导 0、复位读头、进入停机状态。

用 `1000` 减 1 逐步看：最低位 0 改 1 得到 `1001`，再向左把 0 改 1 得到 `1011`，再改一位成 `1111`，最后把最左的 1 改 0，得到 `0111`，去掉前导 0 即 `111`。中间的 `1111` 不是结果，因为“仍在借位”的状态还没有结束。

`1011` 减 1 只改最低位 1 为 0，得到 `1010`；`1` 减 1 得到 0。输入 0 若属于允许集合，不能继续向左无限找 1，应该单独输出边界错误或按明确规定返回 0。减法所需改写次数等于尾部连续 0 的数量再加 1，最坏与数字位数成正比。

图灵机的状态名不是装饰。至少要区分“去最低位”“借位”“返回起始处”“结束”，否则同一个符号 1 在不同阶段应执行什么动作就说不清。复位读头产生的移动也属于时间成本。

22.8 8 · 允许读头原地停留，会不会多出一种计算能力

只允许 L 和 R 时，每走一步位置的奇偶性都会改变。若读头最后回到起点，向左和向右的步数相等，移动总次数是两者之和，因此为偶数。这是对位移计数的结论，不是说程序中所有操作都天然成双。

加入 N 表示头不移动。设某条可达转移在读到某符号、处于某状态时，既不改符号也不改状态，且动作为 N，那么执行后整个配置与执行前完全相同。确定性的机器下一步还会选同一转移，于是永久重复。要让一条有效计算路径最终结束，这类转移必须改变符号或状态，或者本身就是停机规则。

时间应该按转移次数数，而不是只数头移动了多远。连续 100 次改状态但不移动仍做了 100 步，不能算零时间。N 也能由原模型模拟：先向一侧移动，进入一个记住待返回状态的临时状态，再向另一侧移回；邻格读到什么就原样写回。这样只增加常数倍转移，没有改变哪些问题可计算。

这项模拟说明模型的可计算能力相同，但不保证逐步时间完全相同。比较复杂度时要保留“至多常数倍模拟开销”这一层说明。

22.9 9 · RAM：只用加减，怎样把乘除真的做出来

RAM 可以通过地址直接访问寄存器或数组格，图灵机则只能读写头下的格。原语集合决定一次动作的成本。本题要自己实现乘除时，就不能在循环里先偷用一个没提供的除法，再宣布已经实现了乘法。

最直接的非负整数乘法是加b次a。算13×11，累计值依次是 0、13、26、39、52、65、78、91、104、117、130、143。每轮只做加法、计数减 1 和判断，共$\Theta(b+1)$时间。循环不变量是“已经累计的和+还没执行的次数乘a，始终等于原乘积”。余次从b降到 0，所以会结束。

还可以在允许 RAM 随机访问数组时，只用加法、减法和比较构造倍增表。先同时倍增权重和对应的a倍数，权重超过b后停止；再从大到小选能容纳的权重。这个实现没有整数除法或乘法：

```
def ram_multiply(a, b):
    if a < 0 or b < 0:
        raise ValueError("nonnegative integers required")
    weights, values = [], []
    w, v = 1, a
    while w <= b:
        weights.append(w)
        values.append(v)
        w, v = w + w, v + v
    answer, remaining = 0, b
    for i in range(len(weights) - 1, -1, -1):
        if weights[i] <= remaining:
            remaining -= weights[i]
            answer += values[i]
    return answer
```

对 13 与 11，建表后为：

权重	对应倍数
1	13
2	26
4	52
8	104

倒着读：8≤11，选 104，剩 3；4>3，跳过；2≤3，选 26，剩 1；1≤1，选 13，剩 0。答案 104+26+13=143。建表期间每行满足“倍数=a乘权重”；选择期间满足“已累计乘积+a乘剩余权重=原乘积”。处理完某个权重后，余数小于该权重，最终余数小于 1 且非负，只能为 0，证明没有漏掉还需累计的部分。表长$\lfloor \log_2 b \rfloor + 1$ $(b > 0)$，时间和辅助空间都是$O(1+\log(b+1))$；若 RAM 已有按位指令，可省掉表并用常数个字完成，但那要另写原语假设。

除法也建一张倍增表。以$37 \div 5$为例，表是`(倍数,商贡献)=(5,1),(10,2),(20,4)`，40 已经超过 37，停止。倒着逐行：

试减	试前余数	能减吗	试后余数	已得商
20	37	能	17	4
10	17	能	7	6
5	7	能	2	7

每一步保留等式$37 = 5q + r$，其中q是已得商、r是余数。倒着扫完后$0 \le r < 5$，因此$q = 7$就是向下取整商；若还要向上取整，余数非 0 时再加 1，得到 8。

```
def ram_divide(a, b):
    if a < 0 or b <= 0:
        raise ValueError("a >= 0 and b > 0 required")
    multiples, powers = [], []
    v, w = b, 1
    while v <= a:
        multiples.append(v)
        powers.append(w)
        v, w = v + v, w + w
    quotient, remainder = 0, a
    for i in range(len(multiples) - 1, -1, -1):
        if multiples[i] <= remainder:
            remainder -= multiples[i]
            quotient += powers[i]
    return quotient, remainder
```

若$a < b$，表为空，直接返回商 0、余a；除数 0 不合法。假定精确机器字加减，时间与表空间为$O(\log(a/b + 1) + 1)$。定长机器上倍增前还要检查下一次加法是否会溢出，不能靠回绕后的负数结束循环。任意长整数的加减又需按位长付费。

C++还规定类型转换、对象生命期、资源释放等语言行为。把 C++中的 `string` 一次复制映射成 RAM 的一条整数赋值，会改变成本模型；回答模型差异时，应举出这样能实际改变计算量的例子。

22.10 10 · 小 o 与小 ω：比“同阶”更强的判断

$f(n) = o(g(n))$ 表示 $f/g \to 0$，即 f 相对 g 越来越可忽略；$f(n) = \omega(g(n))$ 表示 $f/g \to \infty$。这里先假定 g 最终为正。大 O 只要求比值最终有一个固定上界，小 o 则要求不管选多小的正上界，最终都能低于它。

例如 $n/(n \log n) = 1/\log n \to 0$，所以 $n = o(n \log n)$，也就是 $n \log n = \omega(n)$。但 $3n$ 与 n 的比值恒为 3，所以 $3n = \Theta(n)$，却不是 $o(n)$。

再看一个容易误判的例子：$n/1000$ 在很多输入上比 n 小很多，但二者仍同阶，因为“1000”是固定常数。反过来 $n/\log n$ 与 n 的相对差别会随 n 增长，前者才是 $o(n)$。这些符号描述极限趋势，不表示某个小规模输入上哪一个实际更快。

22.11 11 · 乘法、除法能不能当$O(1)$

若课程明定“机器字内的整数乘除”属于 RAM 原语，分析时就可以算一次基本操作。但这个约定含有一个前提：操作数能放进规定字长，而且不把任意大整数悄悄塞进一个寄存器。

用只支持加减、移位的指令实现乘法，第 9 题展示的算法要按乘数位数做多轮；用小学竖式乘两个 L 位二进制数，最朴素需要 $O(L^2)$ 个逐位动作。这和“CPU 有一条乘法指令”的单位成本说法并不冲突，它们在不同模型里计费。

因此写复杂度报告时可以明确说：“以下 n 个元素均为固定宽度整数，比较和算术按单位成本计。”若题目让你处理有百万位的数，就必须把位长作为规模的一部分。用一句“乘法都是常数”会抹掉题目真正的难处。

22.12 12 · 取对数：正向可保留上界，反向只得到多项式关系

原题问的是：由 $T(n) = O(f(n))$，是否有 $\log T(n) = O(\log f(n))$，反向又怎样。先设 $T(n), f(n) \geq 1$，且 f 最终至少为某个大于 1 的常数；更常见的充分条件是 $f(n) \to \infty$。这样对数非负，分母不会一直为 0。

若 $T \leq Cf$，两边取对数得到 $\log T \leq \log C + \log f$。在 $\log f$ 最终有正下界时，常数 $\log C$ 能被另一个常数倍的 $\log f$ 吸收，所以正向成立。若 f 恒为 1、T 恒为 2，虽然 $T = O(f)$，但 $\log f = 0$、$\log T > 0$，就不能写成后一关系。这说明前提不是可省略的装饰。

反向通常不成立。取 $T(n) = n^2, f(n) = n$：有 $\log T = 2 \log n = O(\log f)$，但 $T/f = n \to \infty$，所以 $T \neq O(f)$。对数把幂次从指数位置拉成了乘法常数，而大 O 恰好会忽略这个常数。

从 $\log T \leq C \log f$ 真正能推出的是 $T \leq f^C$，即 T 受 f 的某个固定次幂约束；不能擅自把这个幂 C 改回 1。核对方法很简单：每次把两边指数化，看看常数究竟跑到哪里去了。

22.13 13 · 指数枚举：输出数量本身可能就很大

n 个元素的每一个子集都可用长度 n 的 0/1 串编码：第 i 位为 1 表示取第 i 项。每位有两个选择，候选共有 2^n 个。枚举 n 个布尔变量的所有真值赋值、逐一检查所有子集的重量，也都有这层枚举成本。

n 从 20 增加到 40 时，候选量从约一百万增加到约一万亿；输入只翻倍，候选却平方。这正是先估算复杂度再写 PA 的理由。若每个候选还要重新扫描 n 项，时间是 $O(n2^n)$；若用增量维护，单个候选成本可能下降，但候选数本身没有消失。

要区分两个结论："这个算法枚举了 2^n 个候选"与"这个问题的所有算法都必须指数时间"。前者直接数流程就能证明，后者是问题下界，通常难得多。对于特定子集问题，折半枚举还能把主要枚举规模降到 $2^{n/2}$，这已经足以推翻"只能按原算法那么慢"的断言。

22.14 14 · 32 位、64 位无符号整数有几位十进制

正整数 x 的十进制位数是 $\lfloor \log_{10} x \rfloor + 1$。因为 d 位数恰好落在 $10^{d-1} \le x < 10^d$ 之间，两边取对数即可得到这个式子。

w 位无符号整数最大值为 $2^w - 1$。32 位最大值是 4294967295，共 10 位；64 位最大值是 18446744073709551615，共 20 位。估算时用 $\log_{10} 2 \approx 0.30103$，32 乘它约 9.633，64 乘它约 19.266，分别向下取整再加 1。

这里问的是"十进制字符要写几位"，不是"数有多大就需要多少字节"。64 位数在机器的定长二进制表示里仍占 8 字节，而把它打印成十进制文本最多要 20 个数字字符，若用 C 字符串还要给末尾零字符留位置。

22.15 15 · 时限预算：给自己的程序做一个能被反驳的估计

先从题面记下 n 的上限、时限、内存限额、编译器和允许的库，再写出候选算法的操作量。例如 n 为 20 万时，二重循环约 4×10^{10} 对，排序约 $n \log_2 n \approx 3.5 \times 10^6$ 次比较；即使常数有差异，这个数量级对比也足够先淘汰朴素二次方案。

不能把"每秒一亿次"当硬件定律。一次比较可能是两个整数，也可能比较长字符串；访问连续数组和随机追指针的缓存代价也不同。正确做法是在目标环境用与题目相似的数据做小型基准，确认随 n 增长的趋势，并保留一定余量。

内存也要算。例如两个长度 20 万的 64 位数组约占 $2 \times 200000 \times 8 = 3.2 \times 10^6$ 字节；若递归每层复制数组，或每个元素另建一个带指针的节点，实际占用会增加。记录的是你实际运行的配置和结果，不把本书的示例数字当作 OJ 实测记录。

22.16 16 · 典型循环：先给每个位置写"收费项目"

官方《习题解析》1-32 中的 16 段程序已经在第 32 章逐段展开。这道题练的是数法，建议先遮住答案，把每段程序分成"外层取哪些值""每个外层值的内层做多少次""条件判断本身贵不贵"三列。

例如 `for(i=1;i<=n;++i) for(j=1;j<=i;++j)` 的收费是 $1 + 2 + \cdots + n = n(n+1)/2$。把内层改成 `j*=2`，第 i 轮就只有 $\lfloor \log_2 i \rfloor + 1$ 次，累计才是 $\Theta(n \log n)$。外层同样写着 n 轮，并不能决定整体是几次方。

特别留意循环条件中的函数：`while (i < n/G(i))` 每检查一次条件都可能重新计算 G。若 G 本身递归，不能因为它没写在花括号里面就免费。第 32 章的开方递归例正是把条件成本漏掉后容易答错的题。

22.17 17 · 分数级数怎样估整数次数

运行次数当然是整数。例如内循环按步长 i 扫到 n，真实次数是 $\lfloor n/i \rfloor$。分析时把它夹住：

$$\frac{n}{i} - 1 < \left\lfloor \frac{n}{i} \right\rfloor \le \frac{n}{i}.$$

对 i 从 1 加到 n，得到 $nH_n - n < \sum_i \lfloor n/i \rfloor \le nH_n$。上下界只差至多 n，而 $nH_n = \Theta(n \log n)$，因此真实整数和也为 $\Theta(n \log n)$。

拿 n=5 手算，次数为 5、2、1、1、1，总计 10；分数和为 $5(1 + 1/2 + 1/3 + 1/4 + 1/5) \approx 11.42$。我们没有声称程序做了 11.42 次操作，只是用一个更容易计算的数给真实次数定界。

22.18 18 · 调和级数：把一块块矩形放在曲线上

记 $H_n = 1 + 1/2 + \cdots + 1/n$。函数 $1/x$ 单调下降，所以在区间 $[k, k+1]$ 上，函数面积不超过高度为 $1/k$ 的矩形；把这些区间拼起来，得到

$$\int_1^{n+1} \frac{dx}{x} \le H_n.$$

另一方面，从第 2 项起，每一项 $1/k$ 不超过区间 $[k-1, k]$ 下的面积，所以 $H_n \le 1 + \int_1^n dx/x$。积分就是自然对数，于是

$$\ln(n+1) \le H_n \le 1 + \ln n.$$

上下都是对数增长，故 $H_n = \Theta(\log n)$。换对数底只乘一个固定常数。若只记住“调和级数发散”还不够：发散表示最终能超过任意常数，这道题还要知道它慢到什么程度。

22.19 19 · 为什么 $\log(n!)$ 是 $n \log n$ 量级

把乘积取对数：$\log(n!) = \sum_{i=1}^{n} \log i$。上界很直接，每项至多 $\log n$，所以和至多 $n \log n$。下界只拿后半段，至少有 $\lfloor n/2 \rfloor$ 项，每项至少 $\log(n/2)$，因此和至少约 $(n/2)\log(n/2)$。这已经证明了 $\Theta(n \log n)$。

Stirling 公式给出更细的近似：

$$n! \sim \sqrt{2\pi n}(n/e)^n, \qquad \ln(n!) = n \ln n - n + \frac{1}{2}\ln(2\pi n) + o(1).$$

例如 n=10 时，$\log_2(10!) \approx 21.79$。这表示区分 10 个互异元素的所有排列，二元比较树至少需要足够深，不能用十次左右的比较普遍完成排序。精细近似能估常数；只求渐近阶时，前面的上下夹逼更短，也更不容易记错。

22.20 20 · 递归空间：数同时没回家的调用，不数一共来过几次

线性递归 `sum(a,n)=sum(a,n-1)+a[n-1]` 在最深处会同时保留 n 层左右的调用帧。每层若只存固定几个下标、返回地址和局部量，辅助空间是 $\Theta(n)$。把输入数组本身排除以后，这些栈帧仍然是额外空间。

二分求和先算左半，再算右半。递归树总节点仍有 $\Theta(n)$ 个，因此总时间为 $\Theta(n)$；但任一时刻只在一条根到叶的路径上，深度为 $\Theta(\log n)$，辅助空间只有 $\Theta(\log n)$。顺序执行的左右子树不会同时保留各自所有内部调用。

取 4 项`[2,5,7,1]`，二分求和在最深处的存活链可以逐时刻画成：

```
进入sum[0,4)
  进入sum[0,2)
    进入sum[0,1)，返回2
    进入sum[1,2)，返回5
  返回7，sum[0,2)的调用帧消失
  进入sum[2,4)
    进入sum[2,3)，返回7
    进入sum[3,4)，返回1
  返回8
返回15
```

在`sum[2,3)`执行时，左半的各个子调用已经结束，只剩根帧保存左结果 7。总共调用 7 次，最多同时 3 层；把 7 误当最大栈深，就把“累计出现”与“同时存在”混在一起。对 8 项最多 4 层，对 16 项最多 5 层，因此是对数。

迭代求和只用一个累计值和一个循环下标，空间为 $O(1)$。若每次递归都先复制子数组，再调用函数，复制出的数组可能还没释放，空间就不能只按“每层常数”算了。若并行启动左右递归，也要重新检查同时存活的任务和缓冲。

时间和空间有联系，但不必同阶。二分求和就是时间 n、空间$\log n$；缓存所有子问题可能多占空间却大幅省时间。判断时画“某个时刻哪些对象仍活着”，比把递归树所有节点直接相加更可靠。

22.21 21 · reverse：交换放在递归前后，为什么结果一样

采用闭区间 [lo,hi]。当 lo>=hi 时不做事；否则交换两端，再反转内部 [lo+1,hi-1]。交换的两个位置和内部递归处理的位置不重叠，因此可以调换这两步。

例如 [1,2,3,4]：先交换外端得到 [4,2,3,1]，再交换内部得到 [4,3,2,1]。先做内部则先得到 [1,3,2,4]，再换外端，同样得到 [4,3,2,1]。这不是“所有两行代码都能交换”，而是这里两步对互不重叠的数据起作用。

每层做一次常数时间交换，规模减 2，层数约 n/2，所以两种顺序时间都为 $\Theta(n)$。普通递归实现保留 $\Theta(n)$ 层帧。递归放最后时有尾调用形式，某些编译器可能优化；C++不保证一定消去栈帧，因此若必须保证常数辅助空间，明确写双指针循环更稳妥。

22.22 22 · Master 定理：从每层的账单证明，而不是背三行结论

先处理 $T(n) = aT(n/b) + g(n)$，其中 $a \geq 1, b > 1$ 为常数，基础规模的成本为正常数。为看清结构，先取 $n = b^L$，省掉取整；常用取整版本还需保证递归规模确实下降。记 $d = \log_b a$。

第 i 层有 a^i 个节点，每个规模 n/b^i，本层非递归成本为 $a^i g(n/b^i)$；第 L 层叶数是 $a^L = n^d$。所以

$$T(n) = \Theta(n^d) + \sum_{i=0}^{L-1} a^i g(n/b^i).$$

若 $g(n) = O(n^{d-\varepsilon})$，代入第 i 层，有 $a^i g(n/b^i) = O(n^{d-\varepsilon} b^{i\varepsilon})$。这是向叶层增长的几何级数，和为 $O(n^d)$，叶层本身又提供 $\Omega(n^d)$，故 $T = \Theta(n^d)$。例如 $4T(n/2) + n$ 得 $\Theta(n^2)$。

若 $g(n) = \Theta(n^d)$，每层都为 $\Theta(n^d)$，共 L 层，故 $T = \Theta(n^d \log n)$。例如 $2T(n/2) + n$ 得 $\Theta(n \log n)$。

若 g 比 n^d 多一个固定正幂，并且存在 c<1 使 $ag(n/b) \leq cg(n)$，第 i 层成本至多 $c^i g(n)$，整棵树的非递归成本至多 $g(n)/(1-c)$。根层又必须支付$g(n)$，叶成本可被 g 吸收，因此 $T = \Theta(g(n))$。例如 $2T(n/2) + n^2$ 得 $\Theta(n^2)$。

把$n = 8$真正代一次，三种递推的层账单很容易看清。下面省略常数基础项，每个叶收费 1：

递推	第 0 层	第 1 层	第 2 层	第 3 层叶子	总量
$4T(n/2) + n$	8	16	32	64	120
$2T(n/2) + n$	8	8	8	8	32
$2T(n/2) + n^2$	64	32	16	8	120

第一行越往下越贵，第二行每层一样，第三行根附近最贵。两个 120 相同只是$n = 8$的数值巧合：第一行总量随n^2增长，第三行也随n^2增长，但它们的钱花在不同层；第二行是n乘层数。层成本计算依赖的是“节点数×每节点工作量”，不要只看一条路径上规模变小。

注意课程把根主导情况放在“第一种”，有些书把它编号为第三种。应按条件认定情形，不能只记编号。这个证明也没有覆盖 $n/\log n$ 一类恰在边界附近的所有函数，遇到它们要回到层成本和。

22.23 23 · 正则条件已经包含增长下界吗

假设 $ag(n/b) \leq cg(n)$，其中 0<c<1，且基础区间中的 g 有正下界。把式子改写为 $g(n) \geq (a/c)g(n/b)$。重复 k 次，有

$$g(n) \geq (a/c)^k g(n/b^k).$$

选 k 使 n/b^k 落到固定基础区间，k 约为 $\log_b n$。因此 $g(n) = \Omega(n^{\log_b(a/c)})$。由于

$$\log_b(a/c) = \log_b a + \log_b(1/c),$$

后面多出的 $\varepsilon = \log_b(1/c) > 0$，正是所需的多项式间隔。若只讨论 $n = b^k$，证明直接沿这条序列成立；要推广到所有 n，需要基础区间统一下界等常规条件。

反过来，光有增长下界不能约束相邻尺度的起伏。取 a=b=2，在 $n = 2^k$ 上规定 g：k 为偶数时 $g(n) = n^2$，k 为奇数时 $g(n) = n^3$。无论哪种都有 $g(n) \geq n^2$，已比 $n^{\log_2 2} = n$ 多一幂。

但当 k 为偶数时，$2g(n/2)/g(n) = 2(n/2)^3/n^2 = n/4$，随 n 变大，连小于 1 都做不到，更别说统一小于某个固定 c。这个明确的反例说明，“整体长得够快”与“向下一层时有规律地衰减”是两个不同要求。

22.24 24 · 带对数幂的递推：别漏掉 p=−1

设 $g(n)=\Theta(n^k(\log n)^p)$，在足够大 n 上取对数，避免基础规模出现 log1=0。仍写 $n=b^L$。第 i 层的主要大小为

$$n^k(a/b^k)^i(L-i)^p,$$

其中换底常数已经吸收到 Θ 里。令 j=L-i，就能看见几何权重和幂次怎样叠加。

若 $a<b^k$，从根向下的几何因子衰减，任何固定对数幂都无法推翻固定多项式间隔，所以 $T=\Theta(n^k(\log n)^p)$。若 $a>b^k$，叶层主导，$T=\Theta(n^{\log_b a})$。

若 $a=b^k$，几何部分恰好抵消，剩下 $n^k\sum_{j=1}^{L}j^p$。当 p>−1，和为 $\Theta(L^{p+1})$；当 p=-1，和是调和级数，给 $\Theta(\log L)$；当 p<−1，和有常数上界，叶成本也同阶。对应地：

$$T(n)=\begin{cases}\Theta(n^k(\log n)^{p+1}), & p>-1,\\ \Theta(n^k\log\log n), & p=-1,\\ \Theta(n^k), & p<-1.\end{cases}$$

以底数 2、$n=16$、$k=1$、$p=-1$为例，离叶子还有 4、3、2、1 层时，本层成本分别是16/4、16/3、16/2、16。相加得到$16(1+1/2+1/3+1/4)$，不是只有一个 16。把n换成2^L，括号内就变成H_L，所以多出的是$\log L=\log\log n$。基础层的工作另外计入$\Theta(n)$，避免在叶子上算$1/\log 1$。

例如 $2T(n/2)+n/\log n$ 是 $\Theta(n\log\log n)$，不能机械套成 $\Theta(n)$。对于恰为平滑函数 $n^k(\log n)^p$，根主导时的比值最终小于 1；若只知上下常数包住它，原函数可能起伏，正则条件未必逐点成立，但可用平滑上下包络和递推比较得到同一渐近界。不要把“结论可由夹逼得到”写成“所有起伏函数都自动满足逐点正则条件”。

22.25 25 · Akra–Bazzi：不等长的递归树，怎样加出积分式

先看具体树，再进入一般证明。对$T(n)=T(n/3)+T(2n/3)+n$，取 n=27，前两层是：

```
                27                  本层工作27
          /             \
         9               18         本层工作9+18=27
       /    \          /    \
      3      6        6      12     本层工作3+6+6+12=27
```

每个框里是该节点处理的规模，也是它自己要付的非递归工作。左支占原规模 1/3，右支占 2/3。若只沿一条路径走，为了代表整树，就不能把两种支路一律各算一半；下面会用 1/3、2/3 作为归一化后的路径权重。具体例题的$n\log n$结论也可先用第 27 题的“完整浅层+最长路径”证明。以下一般证明用到加权平均、换元和期望，可以先读第 28 章对应推导，再回来看。

现在证明一个覆盖本课常用多项式与对数幂成本的版本。设固定基础阈值$x_0>0$，在$x>x_0$时

$$T(x)=\sum_{i=1}^{m}a_iT(b_ix)+g(x),\qquad a_i>0,\quad 0<b_i<1.$$

m是子问题种类数，a_i是该类的系数，b_i是规模缩小比例；它们都是与x无关的常数。基础区间的T有统一的正常数上下界。$g(x)$是当前节点的非递归成本，先设其为正，并满足以下“不要在一个固定比例区间剧烈振荡”的条件：令$b_{\min}=\min_i b_i$，存在常数K，当u,v充分大且$b_{\min}u\le v\le u$时，$g(u)/u^p$与$g(v)/v^p$之比都不超过K。常见的$x^q(\log x)^r$在远离$x=1$后满足此条件。若$g\equiv 0$，下面只剩基础项，单独得到$T=\Theta(x^p)$。

指数p由

$$\sum_{i=1}^{m}a_ib_i^p=1$$

确定。左侧关于p连续且严格递减，$p\to-\infty$时趋无穷，$p\to+\infty$时趋 0，所以有且只有一个实数解。这里p不必是整数。对$T(n)=T(n/3)+T(2n/3)+n$，代入$p=1$，两项是$1/3+2/3=1$。

要证明的结论是

$$T(x) = \Theta\!\left(x^p \left[1 + \int_{x_0}^{x} \frac{g(u)}{u^{p+1}}\,du\right]\right).$$

下面把积分从递归树里推出来，而不是只报公式。

第一步，把每条分支变成一个总和为 1 的权重。 设$w_i = a_i b_i^p$，由p的定义有$\sum_i w_i = 1$。再设

$$F(x) = \frac{T(x)}{x^p}, \qquad h(x) = \frac{g(x)}{x^p}.$$

原式两边除x^p，得到

$$F(x) = \sum_i w_i F(b_i x) + h(x).$$

右边的第一项是较小规模F的加权平均，第二项是本节点归一化后的成本。我们可以用一条随机路径计算这个平均：从$X_0 = x$出发，每一步独立以概率w_i选择第i类分支，令$X_{t+1} = b_i X_t$；第一次不超过x_0时停下，停下步数记为τ。记号$E[Z]$表示把每条可能路径上的数值 Z，按该路径被选中的概率加权平均；它不是取最大，也不是假设每条路径等可能。沿递推展开，有

$$F(x) = \Theta\!\left[F(X_\tau) + \sum_{t=0}^{\tau-1} h(X_t)\right].$$

这条等式只是“平均的平均仍按所有路径的概率加权”。在本来并不随机的递推分析中引入一条随机路径，是为了方便给每棵子树分配正确权重，不是在假设原算法随机。

第二步，基础项是常数量级。 停止时，X_τ落在固定区间$[b_{\min} x_0, x_0]$。这个区间不随初始x增长，基础T和x^p都有正常数上下界，因此$F(X_\tau) = \Theta(1)$。公式外面的“1”就是从这里来的，不能随意删掉；若节点工作极小，总成本仍有叶子贡献。

第三步，把一条路径上的和换成面积。 对规模取自然对数，记$S_t = \ln X_t$。每步向下走的距离是$-\ln b_i$，介于两个正常数c和C之间。因此路径把从$\ln x$到基础区间的一段数轴分成若干长度介于c, C的区间。

在区间$[S_{t+1}, S_t]$上，e^s与X_t至多相差固定比例。前面给的正则条件保证$h(e^s)$与$h(X_t)$至多相差固定倍数。因此面积

$$\int_{S_{t+1}}^{S_t} h(e^s)\,ds$$

被两个固定正常数乘$h(X_t)$夹住：区间长度既不会缩到 0，也不会无限大，函数值也没有突然跳高或降为 0。把各段相加，就得到对每一条路径都成立的

$$1 + \sum_{t<\tau} h(X_t) = \Theta\!\left(1 + \int_{\ln x_0}^{\ln x} h(e^s)\,ds\right).$$

跨过基础点的最后一小段只带来常数，可以放进前面的 1。夹逼常数对所有路径相同，因此取期望后仍成立。最后令$u = e^s$，则$ds = du/u$，代入$h(u) = g(u)/u^p$，积分恰好变成$\int g(u)/u^{p+1}\,du$。再乘回x^p，定理得到证明。

这个证明说明了为什么分母是$p+1$：p来自按树的规模权重归一化，多出的 1 来自对数尺度的换元$ds = du/u$。它也说明为什么需要成本函数的正则条件：不能未经控制就把节点上的离散采样当成连续区间的面积。

现在计算两例。对$T(n) = T(n/3) + T(2n/3) + n$，$p = 1$、$g(u) = u$，积分为$\int du/u = \ln n + O(1)$，答案$\Theta(n \log n)$。若节点工作改成$n^2$，积分为$\int 1\,du = \Theta(n)$，外面再乘$n$，答案$\Theta(n^2)$。同一棵不齐递归树，工作分布不同，主导位置就不同。

完整文献还有更一般的成本正则条件、$b_i x + h_i(x)$扰动、向上向下取整版本。本段证明的是已明确写出的固定比例、无扰动版本；不能据此把$n-1$当作某个固定$b_i n$，也不以“Akra–Bazzi 适用”跳过实际条件核对。对本课取整后的固定比例线性合并，还可直接用每层总规模上界和完整浅层下界证明，与下一题一起互相检查。

22.26 26 · 最大和区间：把“哪个区间”也放进状态

先采用非空区间，使用半开写法 `[l,r)`，并约定和相同时取起点更小者，再取终点更小者。答案要保存三元组 `(sum,l,r)`，所有更新都使用同一个比较规则。

线性算法中，记 `ending` 为“必须在当前下标 i 结束的最佳非空区间和”，记 `start` 为它的起点。读到 `a[i]` 时，只能二选一：独立从 i 开始，或接到前一个结束于 i-1 的区间后面。若旧 `ending<0`，延长只会更差，就令 `ending=a[i], start=i`；若旧和为 0，延长与重开同和，按“更小起点”应保留旧起点。

例如 `[-2,3,-1,3,-5]`：i=0 时当前和-2；i=1 重开得到 3、起点 1；i=2 延长得到 2；i=3 延长得到 5，最佳区间变为 `[1,4)`；i=4 降到 0，但全局最佳仍为 5。最后输出的不只是 5，还有元素 `[3,-1,3]`。

原题说的四种算法都能带上位置，但保存的位置不完全相同。

- 三重枚举：枚举左端l、右端r，再从头累加区间。每算完一个候选，就连同(l,r)一起比较更新，时间仍$O(n^3)$。
- 二重枚举：固定l后令 `sum=0`，右端每延长一项只加新值，同样更新三元组，时间$O(n^2)$。
- 分治：左解与右解本来带位置；跨中点候选由“左边最大后缀+右边最大前缀”拼成，同时保留两端。三个候选按同一规则比较，时间$O(n \log n)$。
- 线性：保存当前结尾最优区间的和与起点，以及全局答案的两端，时间$O(n)$。

例如对`[-2,3,-1,3,-5]`按中点 2 切，跨中点时，左后缀从下标 1 取 3 优于从 0 取 1；右前缀依次为−1、2、−3，最大是到下标 3 的 2，于是跨越候选的区间为`[1,4)`、和 5。不是分别挑左右最大任意区间再拼，它们必须贴住分界，否则拼出来可能中间有洞。

如果允许空区间，则全负数组的最优和为 0，可以约定输出 `[0,0)`。这需要改变初始化和并列规则；不能初始化为 0，却又在题面声称必须选至少一个元素。只多保存常数个下标，四种原有算法的时间阶不变，线性版额外空间仍为$O(1)$。

22.27 27 · 最大区间按 25% 与 75% 切，为什么没有变成二次

每个子问题在线性时间内求跨越分界的最佳区间，再加上左右递归。固定按 1/4 和 3/4 切，递推是$T(n) = T(n/4) + T(3n/4) + \Theta(n)$。一层各节点代表互不重叠的输入区间，总长度约 n，所以合并工作每层线性。最长路径沿 3/4 那边下降，约经过 $\log_{4/3} n$ 层；这虽然比 $\log_2 n$ 深，但仍是常数倍的$\log n$。因此总时间为$\Theta(n \log n)$，下界可按第 25 题的完整浅层论证得到。

对任意固定 $\alpha \in (0,1)$ 都一样。但“固定”不能删掉：若每次切成 1 与 n-1，最长路径有 n 层，层成本依次 n、n-1……，合计 $\Theta(n^2)$。看见两个递归分支，不能自动认定是 nlog n。

22.28 28 · 记忆化：缓存的键必须能决定整个子问题

以 Fibonacci 为例，直接递归 `F(n)=F(n−1)+F(n−2)` 会反复计算同一个 `F(3)`、`F(4)`。把已求得的 Fk存起来，再次遇到同一个 k 就直接读，只有 n 个不同状态，每个状态合并两项，因此单位字模型下时间$O(n)$、缓存$O(n)$。

可以手画 `F(5)`：左分支 F4里会计算 F3，右分支又要 F3。不缓存时重新展开，缓存后第二次只取已有结果。缓存节约的是重复子问题，不会神奇地消除互不相同的所有分支。

对 `F(5)`，约定 `F(0)=0,F(1)=1`，先递归左分支时，第一次计算并写回缓存的顺序是：

```
F2 = F1 + F0 = 1，缓存 {0:0,1:1,2:1}
F3 = F2 + F1 = 2，缓存再添3:2
F4 = F3 + F2 = 3，F2直接读缓存
F5 = F4 + F3 = 5，F3直接读缓存
```

`F3` 的第二次调用不再展开 `F2+F1`。若用自底向上，直接按 2、3、4、5 的顺序算同一批格；若只求最终值，又只需保留前两项，因为每次用完更早项后没有其他依赖。缓存所有答案与滚动两个变量是两种空间选择，不应混说成记忆化自动只占常数空间。

一个常见错误是求“从位置 i 出发、不许用已经选过的元素”的答案，却只用 i 作为键。两个调用虽然位置相同，已选集合不同，剩余合法选择也不同，不能共享答案。正确键可能是 `(i,used_mask)`，状态数也随之大增。先问“相同键是否保证以后面对完全相同的问题”，再写缓存。

22.29 29 · LCS 恢复：表里存的是长度，路径带出字符位置

设 $D[i][j]$ 表示 A 前 i 个字符与 B 前 j 个字符的 LCS 长度。若末字符相同，取 $D[i-1][j-1]+1$；否则取上格和左格的较大值。第一行、第一列为 0，因为有一串为空。

以 A=ABC、B=AC 为例，完整表是：

A 前缀 / B 前缀	空	A	AC
空	0	0	0
A	0	1	1
AB	0	1	1
ABC	0	1	2

从右下角 (3,2) 回溯：C 相同，记录 A 中下标 2、B 中下标 1，走左上到 (2,1)；B 与 A 不同，上格大于左格，向上到 (1,1)；A 相同，记录 (0,0)，再走左上。得到的是逆序的 C、A，把记录反转就是 A、C 及两串中的位置。

相等的候选格可能产生多条最优路径。规定“并列优先上格”便能得到一个确定答案，但它不自动保证字典序最小的 LCS；那是更强要求。保存完整长度表，计算时间$O(mn)$，回溯$O(m+n)$，额外输出长度不超过$\min(m,n)$。若原先只保留滚动两行求长度，直接回溯所需的信息已经丢掉；不能声称不增加信息就自然恢复路径。更省空间的恢复算法需要另讲分治技巧。

22.30 30 · 自顶向下和自底向上：同一个依赖图的两种走法

自顶向下从 D[m][n] 开始，需要哪个依赖就递归求哪个，算过的格子缓存。自底向上按行或列填写，保证填某格时左、上、左上已存在。两者使用同一递推关系，差别是访问顺序和实现开销。

LCS 的密集最坏情形中，两个方法都可能碰到 $\Theta(mn)$ 个状态。自底向上用连续数组，容易滚动行，且没有递归栈；自顶向下贴近定义，并可能避开没用到的状态。例如两串每个末字符都相等时，沿对角线就能一路下降，不必访问整个矩形。

但若依赖图是稀疏、不规则的搜索空间，先把所有可能状态铺成表可能很浪费；反过来，状态几乎全用到而且规则，逐格填表通常更便于控制常数。先比较访问了多少个不同状态，再比较每个状态的递归、查表与内存成本，不用“递归差、循环好”概括所有情况。

22.31 31 · 读源码：把接口的约定也读进来

打开课程代码包后，不必第一遍就追进每个模板细节。先选一个最小调用，例如对 [2,5,7] 求和，找到入口、参数、递归基和返回值。把调用链写成 sum(3)→sum(2)→sum(1)→sum(0)，再沿返回方向累计。

对每个接口写四件事：合法输入是什么、输出保证什么、哪些对象被修改、谁负责释放资源。例如 [lo,hi) 中 hi 不属于区间；若错按闭区间传参，复杂度公式完全正确也会越界。再用空数组、单元素、全相等、最大合法下标和非法边界分开测试。

如果本机平台无法直接打开 Visual Studio 工程，可以读取源文件并用合适编译器编译选定项目，并分别记录所用编译器、选取项目和运行结果。测试通过意味着这些输入符合预期；正确性证明还要覆盖其他合法输入。

22.32 32 · 看演示：先预测下一步，再按播放

演示最有价值的用法是让自己暴露误解。以反转或二分查找为例，先写当前数组、lo、hi，预测下一次交换或轴点；只播放一步，再比较。若预测错了，要指出自己把哪个区间端点或更新顺序理解错。

可以做一次具体记录：数组 [1,4,6,9]，目标 5，用返回最后一个不大于目标的位置的 C 版二分。第一次看 6，应把 hi 改到 2；再看 4，应把 lo 改到 2；此时 lo=hi=2，返回 1，对应值 4。返回值是前驱位置，不是“没找到，所以算法失败”。

动画只展示所选输入的一条执行轨迹。要把观察变成普遍结论，还要提炼不变量和终止量。纸笔预测、单步核对、写出理由，才是把演示变成可复用理解的过程。

22.33 本章练习：离开答案再做一次

1. $T(n)=n^3, f(n)=n^2$，判断 $T=O(f)$ 与 $\log T=O(\log f)$。说明两个判断为什么能不同。
2. 用 Failstone 规则追踪 `(3,5)`，数清返回值，并指出哪几步组成最大值下降的一组。
3. 非空最大和区间在 `[0,0]` 上，采用本章并列规则应输出哪个区间？若误把“旧和为 0”也当作必须重开，会违背哪一条规则？

第一题：T/f=n 无界，故第一个不成立；对数比值为 3/2，第二个成立。第二题依次为 `(3,5),(6,5),(1,6),(1,3),(2,3),(4,3),(1,4),(1,2),(1,1)`，返回 9；前三次变换后最大值从 5 降到 3，是一组。第三题应为 `[0,1)`，和 0、起点 0，并列时终点 1 比 2 小。若在第二项重开会选起点 1，损害先比起点的规则；若总覆盖为延长结果又会选 `[0,2)`，损害最后比终点的规则，所以全局比较也要一致。

23 PS02：向量里的每一次搬动，都要知道搬了谁

教师原始习题单：第 02 次习题共 43 题。统一把有效区间写为 `[0,n)`，含 0、不含 n；容量是能容纳多少项，规模是已经存在多少项。元素复制、比较暂按单位成本计；大对象的构造、移动和析构需要另计。

23.1 1 · 面向对象：从一个能自己维护状态的小类开始

对象是一次具体存在，例如某份成绩记录；类说明这些对象有哪些数据、能做哪些操作。属性保存状态，方法读写状态。构造函数建立合法初态，析构函数在对象生命结束时释放它独占的资源。把数据放进类，却允许外部随手把规模改成比容量大，仍没有保护合法状态。

```
template<class T>
struct Pair {
    T first, second;
    Pair(T a, T b) : first(a), second(b) {}
    T sum() const { return first + second; }
};
struct AddTo {
    int delta;
    void operator()(int& x) const { x += delta; }
};
```

`Pair<int> p(2,5)` 把模板参数 T 具体化成 int，`p.sum()` 得 7。`AddTo plus3{3}; plus3(x);` 看起来像函数调用，其实调用对象的 `operator()`，delta 则保存在对象里。

把继承和虚方法也落到一次调用上：

```
struct Shape {
    virtual int corners() const { return 0; }
    virtual ~Shape() = default;
};
struct Triangle : Shape {
    int corners() const override { return 3; }
};
Triangle t;
Shape& s = t;
// s.corners() 返回3，因为它是虚方法，实际对象为Triangle。
```

`s` 的静态类型是 `Shape&`，但它引用的对象是 `t`。若把 `virtual` 去掉，并相应去掉派生类的 `override`，通过 `Shape&` 调用便选基类方法而返回 0。虚析构使将来通过基类指针释放派生对象有合适的析构行为；它与“运行时挑哪一个普通方法”是相关但不同的规则。模板则是在编译时按 `T` 生成合适代码，不能把模板和虚函数当同一种机制。

重载是在同一名称下提供不同参数签名；重写是派生类替换基类的虚方法实现。通过基类指针调用非虚方法，并不会自动按对象实际类型改选。阅读 Vector 模板时，先判断当前是哪个对象、哪个类型、哪份资源，再研究语法，别让一串名词把状态藏起来。

23.2 2 · std::vector：reserve 以后，为什么仍不能写 v[0]

空向量的 size 为 0。`reserve(10)` 只保证容量至少 10，不创建 10 个元素，所以 `v[0]=7` 仍越界。`push_back(7)` 才创建第一个有效元素；`resize(4)` 则把有效规模改成 4，新建整数按该接口的初始化规则得到相应初值。

照着做：空表→reserve(10)→push_back(7)→push_back(8)→erase(begin())。规模依次 0、0、1、2、1，最后元素为 8。打印每一步 size、capacity 和 data 地址；容量除标准明确的下界等保证外，不能把某个实现的数值照抄成普遍规律。

若保存元素指针，然后扩容迁移，旧指针不会自动指向新数组。释放后继续解引用违反有效性要求，即使偶尔还能读到旧数字也不能据此判断安全。只比较地址数值作观察，不去访问已经失效的对象。

标准原文：vector 容量与 reserve/resize，可核对规模不变、容量保证与重分配失效规则。

23.3 3 · 课程 Vector：测试报告必须对照哪个不变量

通常有 _size、_capacity、_elem 三项核心状态。首先检查 $0 \le size \le capacity$，其次检查有效对象恰好落在 [0,size)，最后检查谁拥有并释放底层数组。

从容量 4、元素[2,4,6]开始，依次头插 1、尾插 8、删中段、复制、修改副本。头插后应是[1,2,4,6]；尾插可能扩容；副本的修改不应改变原表。若改变了，复制很可能只复制了地址，没有建立独立资源。

把这条操作序列的内容、规模都补齐，就能写出不会猜答案的断言。设删去半开区间[1,3)，结果如下：

操作	有效数组	size	应检验的性质
初态	[2,4,6]	3	size≤capacity
头插 1	[1,2,4,6]	4	后缀顺序保持
尾插 8	[1,2,4,6,8]	5	新数组若迁移，内容仍相同
删[1,3)	[1,6,8]	3	删除的是 2、4，不是 6
复制到 w	v 与 w 均[1,6,8]	各 3	内容同，资源独立
令 w[0]=99	v 为[1,6,8]，w 为[99,6,8]	各 3	原表不受修改

capacity 只在确实取得所用实现的扩容规则后才能填写精确值；表里故意不猜某个标准库的增长倍数。

把空表、单元素、刚好满、自赋值、空删除、删除到触发缩容分别作为测例。随机通过 100 次只说明这些轨迹没有暴露错误。阅读测试主程序的每条断言，确认它查的是内容、规模、地址失效，还是资源泄漏；同一个“通过”不代表所有性质都已验证。

23.4 4 · 批量接口：别把同一后缀反复搬开

向长度 1000 的表头插 100 项。一次批量插入，原 1000 项整体右移 100 格，再写 100 个新项，主要约 1100 次写入。反复在同一位置单项插入，搬移量为 $1000 + 1001 + \cdots + 1099 = 104950$，还要加 100 次新值写入，而且新项次序会反转。

若每次递增插入位置以保留新项顺序，之前新插的项可能不用再搬，但原 1000 项仍各搬 100 次，共 100000 次。具体公式随调用次序改变，重复移动的浪费却很清楚。

因此先做区间接口，再把单元素视作长度 1 的特例，可以共享一次搬移和边界处理。删除也一样：一口气删除头部 100 项，只需压缩一次后缀；逐个删会重复压缩。批量设计的收益来自减少真实工作，不来自函数名字更漂亮。

23.5 5 · insert：插入多少项，原后缀又有多长

令 $k = hi - lo$，原规模 n，在秩 r 前插入。没有扩容时，旧后缀 [r,n) 的 n-r 项右移 k 格，再复制 k 项，故成本为 $\Theta(1 + k + n - r)$。非平凡情况常省掉 1，写成 $\Theta(k + n - r)$；触发扩容还要计入旧数组迁移。

原表[10,20,30,40,50]，r=2，插入[7,8]。先从末尾搬：50 到 6、40 到 5、30 到 4，再写 7 到 2、8 到 3。结果为[10,20,7,8,30,40,50]，搬 3 项、写 2 项，对应公式里的 $n - r + k = 5$。

逐次赋值也可以画出来。下划线表示已申请但还未写入结果的格：

```
原态        [10,20,30,40,50, _, _]
50: 4→6     [10,20,30,40,50, _,50]
40: 3→5     [10,20,30,40,50,40,50]
30: 2→4     [10,20,30,40,30,40,50]
写7到2      [10,20, 7,40,30,40,50]
写8到3      [10,20, 7, 8,30,40,50]
```

中间还出现旧 40、50 没有关系，因为当前在构造结果，尚未宣布新的有效区间完成。循环不变量是：已搬过的右端元素已在最终位置，尚未搬的原后缀仍完整。倒序使下一次读取总在未被覆盖的区域。

若源区间来自同一个向量，扩容可能让源地址失效，移动又可能覆盖尚未复制的项，必须遵守接口的别名约束或暂存源。不能在证明里假定源独立，实现里却把自身的一段直接传进去。

23.6 6 · 移动方向：右移倒着来，左移正着来

把[A,B,C,D]的 B、C、D 右移一格。若先搬 B 到 C 的位置，原 C 就丢了；下一轮读到的已经是 B，后面可能连着复制同一个值。正确顺序是先 D，再 C，再 B，让每次读到的源都还没被覆盖。

错误的正序搬移也值得走完一次。若把 B、C、D 右移一格，先做 `a[2]=a[1]` 得到 `[A,B,B,D,_]`；再做 `a[3]=a[2]` 读到的已经是 B，变 `[A,B,B,B,_]`；最后再写一个 B。C、D 都被覆盖了，后面再小心也找不回来。正确顺序则是 `D:3→4`、`C:2→3`、`B:1→2`，最后 `[A,B,B,C,D]`，留出的下标 1 才用于新项。

删除后的后缀左移则从前向后，因为目的地在源左侧，不会破坏更靠右的未读项。源和目的完全不重叠时，方向可以不同；判断依据是区间重叠关系。

`memcpy` 要求不重叠，`memmove` 允许重叠。但是含字符串等非平凡 C++对象不能简单按字节搬来搬去：构造、析构、移动和所有权都有类型规则。画整数格子帮助理解方向，写模板容器还要把对象生命期处理正确。

23.7 7 · remove：没删东西，却可能搬完整个表

删除 `[lo,hi)` 后，旧后缀 `[hi,n)` 左移 hi-lo 格。整数搬移模型的成本为 $\Theta(1+n-hi)$，固定接口成本不能在 n=hi 时解释成“完全零时间”。

原循环如果不判空，lo=hi=0 时不删任何项，却让每个元素做一次自赋值，正好仍是线性最坏例。加 `if(lo==hi) return 0;`，便能让空区间常数返回。真实删除首项也要搬 n-1 项，仍为线性。

通用对象还可能需要析构被删项，所以仅以 n-hi 计费隐含了元素处理模型。多一次分支值不值，要看空删除频率与元素移动代价；能给出实验设计，不能凭空报提速百分比。

23.8 8 · 几何扩容：一次昂贵迁移由很多次追加分担

容量每次乘固定 $\alpha>1$。从空表一路追加到 N 项，旧容量迁移量形成几何级数，累计小于约 $\alpha N/(\alpha-1)$，加上 N 次新项写入仍为 O(N)，所以按单元素追加均摊 O(1)。

倍增时搬过 1、2、4、8 项，合计 15，不是每次都搬 16 项。$\alpha=1.1$ 仍满足常数倍增长，但复制常数大致随 $1/(\alpha-1)$ 增加；增长后装填率约 $1/\alpha$。更小的闲置空间通常意味着更频繁的迁移。

以初始容量 1、单项追加为例，逐次迁移账单是：

第几次追加	追加前 size/capacity	本次搬旧项	追加后容量
1	0/1	0	1
2	1/1	1	2
3	2/2	2	4
4	3/4	0	4
5	4/4	4	8
6、7、8	5/8、6/8、7/8	各 0	8
9	8/8	8	16

前 9 次共搬 15 项，另外写 9 个新项，总 24 次元素写入。单次第 9 次确实很贵，摊还分析没有把它改成便宜；它说的是这 9 次合计只花了常数倍 9。

一般增长率$\alpha>1$固定。最后一次迁移的旧容量小于N，前面的容量至多依次除以α。所以总迁移小于$N(1+1/\alpha+1/\alpha^2+\cdots)=\alpha N/(\alpha-1)$。分母越小，复制常数越大；若增长率随规模趋近 1，就不能再把这个因子当固定常数。

批量插入不能按“调用一次”无条件均摊 O(1)：一次插百万项，光写新项就得线性；在一个已有巨大满表上触发扩容，也可能有大迁移。应按完整操作序列和插入元素总数算账。标准库没有规定每个 vector 实现都恰好倍增，具体倍数要查所用库版本并打印 capacity 核对。

23.9 9 · 为什么不能只平均真正扩容的调用

倍增到 N，实际迁移约 $\log N$ 次，总迁移量为 $\Theta(N)$。只对这些昂贵调用取平均，结果是 $\Theta(N/\log N)$；把所有追加检查算进来，才是每次摊还 O(1)。两者换了分母，没有矛盾。

两次扩容之间的便宜操作为下次“搬家”积攒费用。若把这些操作从统计集合删除，自然不能继续引用原来的均摊结论。摊还也不要求随机输入；对任何遵守接口的合法序列，总账都受控制。

写分析时明确列出你平均的是所有 push_back、所有 expand 调用，还是仅有搬迁的 expand。先把样本集合讲清楚，后面的公式才有具体含义。

23.10 10 · 半满就缩会抖动，四分之一才缩留出了间隔

容量 C、规模刚低于 C/2 就缩成 C/2，接着只需插很少几项就再次满；删一点又缩。对手用常数次插删，就能迫使约 C/2 项反复迁移，昂贵操作之间没有足够便宜操作来分摊。

取“严格小于半满才缩”的规则，从 `size=8,capacity=16` 开始：

```
删除1项：size=7 < 8，搬7项，容量缩到8。
追加1项：size=8，容量8，未搬迁。
再追加1项：原来已满，搬8项，容量扩到16，size=9。
删除1项：size=8，不满足严格小于8，暂不缩。
再删1项：size=7，再搬7项缩到8。
```

从第一次缩容后的状态起，反复“加 2、删 2”，每 4 次普通插删就搬约原规模的项。若阈值用≤半满，抖动甚至能更频繁。不能用“原来追加时已经证明过均摊”来覆盖这条新操作序列。

改成低于 C/4 才缩为 C/2，缩后约半满。要再扩容得插入约 C/4 项，要再缩还需删掉一批项。这个间隔使累计收费能够覆盖搬移。

例如容量 16、规模 8，删到 7 先不缩；删到 4 附近满足具体阈值时才缩为 8。“小于”还是“小于等于”会改变边界那一步，但不改变要留间隔的理由。最小容量要有固定下限，空表也不应反复分配无意义的零长数组。

23.11 11 · 去重里的哨兵：当前项本来就在停止位置上

查 `_elem[k]` 是否出现在 `[0,k)` 时，目标就在下标 k。若 find 靠第 k 格的哨兵让扫描最终停下，就不必再保存、写入和复原同一个目标，因为它已经在那儿。

例如`[5,2,5]`处理下标$k=2$。要查目标 5 在前缀`[0,2)`里是否出现，哨兵查找可从 0 顺扫到相等即停。即使前缀没有 5，走到下标 2 也一定停，因为那一格本来就是目标。所谓省掉设置与恢复，就是省去“暂存第 2 格、把 5 写入第 2 格、查完写回”的三次多余动作；它没有允许搜索结果把第 2 格本身当作前缀命中。返回位置若等于 2，仍应解释为前缀中不存在。

若处理`[(5,A),(2,B),(5,C)]`时命中前面的 A 并删除它，最后保留的是 C。改删当前 C 就保留 A；两者 key 相同，记录身份却不同，必须先决定接口承诺。

每轮省常数次操作，n 轮省 O(n)。如果整体仍有二次比较或移动，总阶没有改变。常数优化值得解释，但不能说它把算法改成了线性。

查到前面的 i 时删 i，会保留较新的 k；改删 k 则保留先出现的一份。附加字段可能不同，所以这不只是下标替换。删 k 以后，下一个未处理项移到 k，不能还无条件 k++，否则会漏项。

23.12 12 · 无序去重：先减少移动，再讨论比较

若保留最后一次出现，可以让 i 扫描原表，find 检查未读后缀 `[i+1,n)` 是否还有同值项；有则跳过，没有才把整条记录写到 w 并令 w++。w 始终不超过 i，不会破坏未读后缀。

例：[A1,B1,A2,C1,B2]，A1 和 B1 后面还有同值记录，跳过；A2、C1、B2 依次写入，得到[A2,C1,B2]。每项最多写一次，移动不超过 n。已写前缀始终是已经确认不再有后继重复的记录。

压缩过程可以保持旧数组尾部不动：

读下标 i	当前记录	后缀还有同 key 吗	写前 w	本轮动作	已确定输出前缀
0	A1	有 A2	0	跳过	[]
1	B1	有 B2	0	跳过	[]

读下标 i	当前记录	后缀还有同 key 吗	写前 w	本轮动作	已确定输出前缀
2	A2	无	0	写 a[0]=A2，w=1	[A2]
3	C1	无	1	写 a[1]=C1，w=2	[A2,C1]
4	B2	无	2	写 a[2]=B2，w=3	[A2,C1,B2]

输出写指针从未超过读指针，故不会覆盖还要搜索的后缀。处理结束才令 size=w；不能每跳过一项就缩容，否则原读下标的含义会改变。比较次数仍取决于后缀查找遇到相等多早，写入次数则至多 n，这两笔账分开。

全相等时，若 find 见到相等项立即返回，可每轮很快确认重复，整体 O(n)；全不同时仍要扫描各后缀，比较为 $n(n-1)/2$。减少移动没有自动减少所有比较。

若允许排序，可 O(nlog n)后线性去重，但顺序可能改变；允许散列表时可在相应散列模型下得到期望 O(n)，空间也增加；小整数值域可用位图。对于任意元素只提供相等测试的模型，不应承诺同一改进。题面所说移动比比较贵，是优化动机；通用临界长度还取决于元素类型和机器，不能编一个数字。

23.13 13 · 函数对象：累加到的是外部对象还是内部副本

求和访问器保存一个累加状态：

```
struct Sum {
    long long value = 0;
    void operator()(const int& x) { value += x; }
};
// 假设 traverse 按引用接收访问器
Sum s;
v.traverse(s);
std::cout << s.value;
```

对[2,-1,5]，状态 0⟶2⟶1⟶6。若 traverse 按值接收，内部副本变成 6，外部 s 仍是 0。先看函数签名，再判断它有没有工作。也可让接口返回访问器副本，但设计必须明确。

空表求和保持初值 0；整数元素多了以后可能超过 int，所以累加器范围仍要算。函数对象能把“访问时顺便保存状态”组织起来，不能免掉数值范围和复制语义。

23.14 14 · 用遍历做冒泡：访问器要记住前一项

一趟冒泡比较相邻元素。访问器保存上一项地址与交换标志；访问当前项时，若前项更大就交换，然后把上一项地址更新到当前项。对[3,1,2]，先交换 3 和 1 得[1,3,2]，再交换当前 3 和 2 得[1,2,3]。

若访问器只收到值的副本，就不能修改原表；要限制未排序区间，还需区间遍历或计数器。把循环换成 traverse 只是换了表达方式，邻接关系和边界状态仍然要维护。

对`[3,1,2]`，访问器的状态应是：第一次访问下标 0，只记住地址 0；访问下标 1，比较 `a[0]=3` 与 `a[1]=1`，交换后数组`[1,3,2]`，把前项地址改为 1；访问下标 2，比较 3 与 2，交换后`[1,2,3]`，前项地址改为 2。若第二步交换后仍让前项地址停在 0，第三步会比较 1 与 2，错过 3 与 2。保存的是“刚经过的格子地址”，不是“当初见到的那个对象值”。

一趟线性，最坏 n-1 趟，比较仍约 $n(n-1)/2$，总时间 $\Theta(n^2)$。接口复用与渐近效率要分别评价。

23.15 15 · 判等与比较：同分数的两条记录算相同吗

记录(score,id)按分数排序，(90,A)与(90,B)互不小于，属于同一排序等价类；若相等运算符还比较 id，它们又不相等。去重按分数还是按完整记录，要由业务定义。

严格弱序比较器的等价可写为 `!less(a,b)&&!less(b,a)`。这不一定与 `a==b` 一样。排序按 score、去重按全字段，并不会自动得到“每个分数只留一条”。

比较规则还必须在整个算法中一致。若同一个 a 满足 less(a,a)=true，或排序过程中临时改变顺序标准，二分与归并的证明都失去了前提。

23.16 16 · 缩容后：整数秩还在，旧地址不会自己更新

旧底层数组在 P，缩容后迁到 Q。秩 k 只是一个整数；再调用当前 `_elem[k]` 会用 Q 寻址，只要 k 仍合法，就能访问新数组。

旧指针 P+k 却不会自动改成 Q+k。原数组释放后它悬空；引用和某些迭代器也一样。删除还会改变秩：旧 k 可能现在指向原来的下一项，即使没越界，逻辑也可能错。

检查去重循环时分开问：“保存的是秩还是地址？”“删后下一项移到了哪里？”它们分别对应地址失效和扫描漏项，不能混成一句“扩容后小心”。

23.17 17 · 懒惰 uniquify：前面没重复时，为什么还写自己

read 向前读，write 指向最后保留位置；新值与保留值不同才推进 write 并写入。遇到首个重复以前 read=write，赋值只是自己写回自己。

可先找第一对相邻相等项；若没有便直接返回，找到了再开始压缩。例[1,2,3,3,4]的前三项不用写，最后 4 才移到下标 3。始终维持“输出前缀等于已读部分的去重结果”。

具体下标是：先比较 `a[0],a[1]`，再比较 `a[1],a[2]`，都不同，尚不写；比较 `a[2],a[3]` 发现两个 3，令 write=2、read=4；4 与保留的 3 不同，把 write 增为 3，执行 `a[3]=4`，最后 size=4。结果 `[1,2,3,4]`，只发生一次有效写入。若数组 `[1,2,3]` 没有重复，检测完直接返回，无需写 3 次自己；若空或单元素，则在读取相邻格以前就返回。

这减少写操作，却增加了首段检测或分支。对无重复、大量重复和昂贵对象，实际收益不同；总读次数仍线性，所以时间阶不变。优化不能以更难验证的边界代码为代价。

23.18 18 · 本题保留最后一份：按组扫描就能做到

原题把稳定约定为每组相等记录保留最后一项。它与排序稳定性的通常定义相关，但要求不完全相同，回答时要忠于这条题面约定。

有序表逐组扫描：i 是组起点，j 一路走到组末，把整条 a[j]写到输出位置。[(1,A),(1,B),(2,C),(2,D)]最终得到[(1,B),(2,D)]，不能只改 key 而丢掉附加字段。

组内最后项的定位可用如下边界：从 i 开始令 j=i+1，当 j<n 且 key(a[j])等于 key(a[i])时一直加 j。循环退出后，本组是 `[i,j)`，最后项在 `j-1`。写 `a[w]=a[j-1]`，再令 w++、i=j。例中第一组 `[0,2)` 选 B，第二组 `[2,4)` 选 D。每轮至少吃掉一项，j 总向前，所以终止；整个记录一起复制才保留最后一项的所有附加字段。

每项被访问常数次，总时间 O(n)，几个下标只需 O(1)空间。从右向左也可选最后记录，但要把输出恢复到正常顺序，并把那部分移动计入成本。

23.19 19 · 分支次序：大 O 相同，不等于比较次数相同

一次查找要区分小于、大于、等于轴点三种情况。用两个 if 和末尾 else 表达，排在第一种的情况只用一次关键字比较，其余用两次。若本层三类概率为 p_L, p_R, p_E，把概率最大的类别放第一，理想化的期望比较次数为 $2 - \max(p_L, p_R, p_E)$。

无论怎么排，只要每次把区间按固定比例缩短，最坏迭代轮数仍是 $\Theta(\log n)$。但相等命中若极少，把相等测试放第一，就可能每层几乎都要多做一次比较。取百万个互异 uint32_t 关键字、目标在全部 2^{32} 个数中均匀抽取，总命中概率约 $10^6/2^{32} \approx 0.000233$，即 0.0233%。

这还不能推出每一层左右概率恰好各半。若关键字和查询都分布平衡，可粗估相等优先约每层 2 次，方向优先约每层 1.5 次；约 20 层时分别接近 40 和 30。若值集中、查询偏斜或比较代价不对称，估计会变。CPU 分支预测也会影响真实时间，所以这里估的是关键字比较，不能把它直接当纳秒。

23.20 20 · 左转 1、右转 2：课程为什么忽略循环下标的比较

原 A 版先测试 `e<S[mid]`，为真就左转；否则再测试另一方向，不等于才右转。按关键字比较计费，左转 1 次、右转 2 次。`lo<hi`、计算 mid 和更新下标也做工作，但课程把它们作为统一的控制开销，或把昂贵的关键字比较作为单独的分析对象。

若元素是长字符串，关键字比较可能远贵于一次整数下标运算，这个模型很有意义。若所有元素是短整数，要估实际运行时间，必须把这些成本和内存、分支都纳入。没有哪条物理定律保证右转恰好耗时左转两倍。

所以不是随意把 1:2 改成 2:3 或 3:4；那相当于在每轮额外加入了另一种固定收费。模型可以换，但模型换后，黄金分割最优常数的推导也应跟着换。

23.21 21 · 中点溢出：最终合法，计算途中也可能先出事

在 `0<=lo<=hi<=INT_MAX` 下，`hi-lo` 非负且不超过 INT_MAX，半差也在范围内，`lo+(hi-lo)/2<=hi`，所以这条中点公式不会因正整数加法超上界而溢出。

而 `(lo+hi)/2` 先算和。例如 32 位有符号数里 lo=1500000000、hi=2000000000，两者分别合法，总和 3500000000 已经超范围，不能指望最后再除 2 救回来。

若改用无符号类型，算术可能按模回绕，仍不等于算出了正确中点；若 lo、hi 来自不同符号类型，转换也要另查。库代码不应因为“当前机器放不下那么多元素”就省掉几乎不增加复杂度的安全写法；不过本题的具体小规模实测可能根本触不到这个边界。

23.22 22 · 重复项：“随机返回”其实是确定轨迹

A 版遇到等于轴点就返回。数组[2,2,2,2]、区间[0,4)第一次 mid=2，立即返回 2；区间换成[0,3)，mid=1，立即返回 1。同样的代码、输入和区间会重复返回同一个位置，所谓“随机”只是接口没有承诺首个还是末个。

需要第一项时寻找第一个不小于 e 的位置；需要最后一项时寻找第一个大于 e 的位置，再减 1 并检查是否等于 e。相等时继续向哪边收缩，决定了边界语义。

做测试时给重复元素附不同 id，才能看出返回的是哪一条。只打印 key，全是 2，很容易误以为测试已经验证了“保留哪份”。

23.23 23 · Fibonacci 切分：便宜的一边为什么分得更大

原 A 版左转收费 1，右转收费 2，轴点在 lo 加 $\lambda(hi-lo)$ 处。左边约占 λ，右边约占 $1-\lambda$。若最坏比较数近似 $C\log_2 n$，两种路径分别要求

$$C \ge \frac{1}{\log_2(1/\lambda)}, \qquad C \ge \frac{2}{\log_2(1/(1-\lambda))}.$$

要把两者的最大值压到最低，使两项相等，即 $\log(1-\lambda) = 2\log\lambda$，得到 $1-\lambda=\lambda^2$，正根为 $\lambda=(\sqrt{5}-1)/2\approx 0.618$。较大的子区间交给便宜的左转，最坏比较主项系数约 1.4404。

若镜像取 0.382，昂贵右转反而分到约 0.618，最坏系数约 $2/\log_2(1/0.618)\approx 2.8808$，并不仍然最优。这个最优性依赖 1 与 2 的收费模型，若使用三向比较器或不同硬件成本，结果要重新推导。

两条不等式可以从一个分支直接推出。若左转一次后规模约λn，剩余最坏成本约$C\log_2(\lambda n)$，加本次 1 次比较，应不超过$C\log_2 n$；消去公共项，得到$1 \le C\log_2(1/\lambda)$。右转同理收 2 次，于是得到第二条。λ增加时，左子问题变大、第一约束变紧，右子问题变小、第二约束变松；两条相等处才使较大者最小。

Fibonacci 切分可以不用每轮算浮点数。若区间长度接近F_k，取左跨度F_{k-1}，余下约F_{k-2}；因为$F_k = F_{k-1}+F_{k-2}$，转到哪一边，就把“当前 Fibonacci 尺度”改成对应较小项。譬如长度 13 附近用 8 与 5，转左后用 5 与 3，再转右后用 2 与 1。实际轴点还需扣除轴点本身并夹到合法下标，不能把 8/5 比例当成每个长度都精确可分的整数公式。

Fibonacci 数满足相邻两项比值趋向这个比例，可以用递减的 Fibonacci 跨度定位轴点，避免每层浮点乘法。但若关键字比较很便宜，维护额外状态的成本未必值得；“比较次数更少”是理论结论，是否更快还应实测。

23.24 24 · B 版为什么先判 e 小于最小值

课程 B 版维持至少一个候选，循环到区间长度 1，然后直接返回 lo 作为“不大于 e 的最后一项”。若 e 连最小值都不到，却不提前排除，最后仍会留下下标 0，错误地声称它是合法前驱。

例如[3,7,9]查 2。候选不管怎么缩都不可能出现合法元素，所以应返回起始 lo 减 1 的失败哨兵；这就是开头判断的作用。A 版查相等项，可以一直缩到空区间而报告失败；C 版允许空区间，并从统一边界减 1，故不需要同样的最小值特判。

原题 b 行写着 A、C“都需要如此特判”，与这两版实际代码的处理不同；应把源码行为讲清楚，不能因为题面一句话便给它们强加多余分支。B 版读取 `S[lo]` 前还必须保证非空，这又是另一项前置条件。

23.25 25 · 空区间：返回哨兵，不代表可以解引用哨兵

C 版求 upper_bound 后减 1。若目标小于最小项，所有比较都向左，最终 lo 停在起始位置，返回 lo-1。若起初 lo=hi，循环根本不执行，也返回起始 lo-1。

这只是表示“本区间没有不大于目标的项”。若对全表[0,n)得到-1，不能接着访问 `S[-1]`。若 Rank 改为无符号类型，减 1 会回绕，所以接口还需改用可表达失败的类型或显式结果。

A 版空区间自然不进入循环；B 版如果一开始就读 `S[lo]` 则越界，必须先判空或把非空写成前置条件。把“返回值有定义”和“该值可以当元素位置访问”分开，是这组题的重点。

23.26 26 · C 版的另一种证明：按区间长度归纳

C 版每次选 mid，若 e 小于轴点，则 `[mid,hi)` 全部大于 e，不可能包含前驱，剩下左区间；否则 `[lo,mid]` 全部不大于 e，它们已经可以归入确定的左侧，未知部分只剩 `[mid+1,hi)`。

先把被证明的程序写出来，返回语义才不会漂移：

```
def predecessor(a, target, lo, hi):
    while lo < hi:
        mid = lo + (hi - lo) // 2
        if target < a[mid]:
            hi = mid
        else:
            lo = mid + 1
    return lo - 1
```

例如`[2,2,5,8]`查 2：`(lo,hi,mid)=(0,4,2)`，5 太大，hi 改 2；再`(0,2,1)`，等于 2 也走右，lo 改 2；未知区间变空，返回 1。已确定左侧是两个 2，右侧是 5、8，前驱就是左侧最后位置。若相等立即返回，就不再是本题的 C 版。

以初始区间长度为归纳参数。长度 0 时，未知区域已经空，边界 lo 左侧全不大于 e、右侧全大于 e，返回 lo-1 正确。长度大于 0 时，两个分支都只删除已确定的一侧，并保留同样的边界含义，且剩余长度严格更小，因此可用归纳假设。

这个证明同时需要有序性、边界语义和严格缩短。只说“规模小了所以对”没有证明删掉的那一半不会藏着答案。若存在重复项，相等时也把 mid 归到左侧，正好确保最后找到最后一个不大于目标的项。

23.27 27 · 平均查找长度：先讲“平均哪些输入”

令 n 个关键字互异，成功时各关键字等概率，失败时 n+1 个间隙等概率；计费是 C 版循环中的关键字比较。若 $n = 2^k - 1$，决策树的 n+1 个终点间隙都在深度 k，所以任何查询均做 k 次比较，两个平均值恰好都等于 k。

先把$n = 3$列全，键为 10、20、30。每次先看下标 1 的 20，下一步无论左右都会再做一次比较。成功目标 10、20、30 分别返回 0、1、2，比较数都 2；失败 4 个间隙`<10`、`(10,20)`、`(20,30)`、`>30` 的比较数也都是 2。这是$k = 2$的完整小树：

```
                target < 20 ?
               /             \
        target < 10 ?      target < 30 ?
         /       \          /       \
        <10    [10,20)    [20,30)   >=30
返回    -1       0          1        2
```

图中叶子写的是整个有序数轴区间。等于某个键与紧挨它右边的失败间隙走相同的 C 版路径，所以成功查询并不在内部节点提前结束。

对一般n，令$N=n+1$表示间隙数。最浅叶深$k=\lfloor \log_2 N \rfloor$，叶深只会是$k$或$k+1$；设浅叶数$u$、深叶数$v$。叶数给$u+v=N$，完整二叉决策树的容量给$u/2^k+v/2^{k+1}=1$。两式相减解出$v=2(N-2^k)$，再算深度总和$D=ku+(k+1)v=kN+v=N(k+2)-2^{k+1}$。这就是后面公式中每个量的来源。

一般 n 就不能把“同一主项”说成精确相等。n=2 时，间隙依次小于首项、两项之间、大于末项，比较次数为 2、2、1，失败平均为 5/3；目标等于两个关键字时为 2、1，成功平均为 3/2。两者已经不同。

设 $N=n+1, k=\lfloor \log_2 N \rfloor$，这类对半决策树的间隙深度为 k 或 k+1。深层叶数为 $2(N-2^k)$，因此叶深总和 $D=N(k+2)-2^{k+1}$，失败均值为 D/N。成功等于第 j 个键的轨迹与其右侧间隙相同，所以成功均值是 $(D-d_0)/n$，其中 $d_0=\lceil \log_2(n+1) \rceil$ 是最左间隙深度。

这些式子共同给出 $\log_2 n+O(1)$，即主项系数为 1。题面用大 Theta 串起的表达应按渐近意义理解；若要求任意 n 时两个精确均值相等，上面的 n=2 就是反例。对数底与概率模型也必须说明。

23.28 28 · 比较 A、B、C：别把三版的返回语义混起来

n 个互异有序键把数轴切成 n+1 个失败间隙，因此“失败类比成功类多 1”来自顺序结构，而不是某份二分源码的偶然。A 版成功是遇到相等项，B/C 版主要返回前驱；若需回答是否命中，还要在返回位置上再判等。

对 $n=2^k-1$，A 版成功键所在层 d 有 2^{d-1} 个。若只数访问层数，平均深度为

$$\bar{d}=\frac{\sum_{d=1}^{k} d\,2^{d-1}}{2^k-1}=k-1+\frac{k}{2^k-1}.$$

但按原代码关键字比较，左 1 右 2，成功末层还需判等，平均约 $1.5\bar{d}+0.5$；失败间隙等概率时平均约 1.5k。访问一次节点与做一次关键字比较不能混算。

C 版在这个规模上每次固定 k 个方向比较，成功与失败都 k。B 版循环到长度 1，另有最小值特判；大多数路径同样为 log n 量级，但其精确平均应按课件源码把开头特判和最终命中检查分别计入。第一个失败间隙常能在特判处结束，它与其他间隙并不对称。先列清收费表，再求平均，才能解释几版常数为何不同。

23.29 29 · 终止：为什么 lo 不会一脚跨过 hi

C 版在 lo<hi 时，取 `mid=lo+(hi-lo)/2`，总有 $lo \le mid < hi$。左分支令 hi=mid，仍 lo≤hi，长度从 m 降到 $\lfloor m/2 \rfloor$；右分支令 lo=mid+1，仍 lo≤hi，剩余长度也严格小于 m。

所以长度是非负、严格下降的整数，循环必停；出口因为不能 lo>hi，又不再 lo<hi，只能 lo=hi。长度 1 时左分支把 hi 落到 lo，右分支把 lo 落到 hi，两种都一步结束。

B 版的持续条件为长度大于 1，更新 lo=mid 时不会卡住，因为长度至少 2 保证 mid>lo；出口长度 1。若误把同样的更新配上 C 版 lo<hi 条件，长度 1 时就会死循环。三版更新规则与持续条件必须成套理解。

23.30 30 · 插值查找：一个极端值就能把猜测拉歪

取序列[0,1,2,...,n-2,M]，其中 M 远大于 n，例如 $M=n^3$，查找靠近 n-2 的值。按端点线性插值，估计位置大致为 $(n-1)e/M$，常常接近区间起点；一次仅排掉一两个小值，反复执行就可能线性。

把$n=6, M=216$代入，数组为`[0,1,2,3,4,216]`，查 4。闭区间端点插值第一次猜$\lfloor 5\times 4/216 \rfloor=0$，只排除 0；第二次在`[1,5]`猜$1+\lfloor 4\times 3/215 \rfloor=1$，只排除 1；第三次又猜 2，再猜 3，最后才到 4。每次猜的位置合法，正确性仍在，但进展退化成顺序查找。极端值 216 把其他值的秩距离挤得几乎看不见了。

序列确实有序，坏的是“数值距离能预测秩距离”的假设。二分不依赖值分布，每次按位置砍半，因此最坏仍对数。插值实现可设置进展检查或回退二分，避免长期猜在边缘。

还要处理端点值相等的分母 0、乘法溢出、目标超范围和重复值。理论上在均匀模型下很快，不表示可以省略边界检查。

23.31 31 · 非均匀分布：用累计概率猜秩，而非硬套直线

若已知值服从分布 F，$F(x)$ 表示随机值不大于 x 的概率。长度 n 的独立样本里，不大于 x 的数量期望约 nF(x)，因此可用累计概率替代原始数值做插值。

对高斯分布，先标准化 $z = (x - \mu)/\sigma$，再用其 CDF 估计累计比例。中心很密、尾部很稀，直接按 x 的线性距离猜位置会失真，CDF 映射才反映这些密度差别。

但参数估错、有限样本起伏、离群点和重复值都会使预测不准。CDF 只负责提出一个候选轴点，仍需把它限制在合法区间，并靠有序比较维持正确性。不能把统计模型的期望当作每份数据的精确秩。

23.32 32 · 二分何时切顺序：做一个不会骗自己的实验

固定编译优化、元素类型、比较器和查询集合；分别测试长度 8、16、32、64 等小表，既有成功也有失败，既有均匀也有聚集值。每种重复足够多次，防止单次计时噪声淹没差异。

结果应累计到校验值并输出，避免编译器删除“算了却没人使用”的循环。把数据生成时间移出计时区，改变测试次序，报告中位数和波动，再选在代表数据上稳定的切换阈值。

顺序扫描虽然比较多，但控制简单、内存连续；小表可能胜过多分支二分。长字符串、缓存状态和 CPU 改变后，阈值可能变。切换阈值应来自相同接口、相同数据上的本机实测。

23.33 33 · 冒泡提前停：一整趟没交换才证明已经排好

每趟开始 sorted=true，只要交换一次就设 false。若整趟结束仍 true，所有相邻项非降，因此整个序列有序，可以结束。

[1,2,3]第一趟无交换；[2,1,3]第一趟交换得到有序，但标志已经 false，若没有别的逻辑，还要下一趟确认；[2,3,1]中的 1 每趟只能向左移动一格，需要更多趟。全相等也应不交换，否则既损害稳定性又失去提前停机会。

最坏逆序仍二次，最好有序为线性。平均省多少趟，需要指定随机排列模型和是否把最后确认趟算入；只挑一个近乎有序样本，不能推出总体平均提速。

23.34 34 · last 跳跃：最后交换之后的区域，为什么可以不再碰

一趟从左向右比较，把最后一次交换的右端位置记为 last。本趟结束后，last 之后没有逆序交换需求，且大项已向右传递，因此下一趟可把活动右边界缩到 last。

例如[3,2,1,4,5,6]的乱序只在前三项，第一趟最后交换发生在前部，后面 4、5、6 可以直接排除。若只按 n-1、n-2 缓慢缩小，会重复比较这些已经安稳的后缀。

把它走到结束，设活动区间为`[0,hi)`，last 存最后一次交换的右端下标：

```
hi=6: [3,2,1,4,5,6]
  (0,1)交换 -> [2,3,1,4,5,6], last=1
  (1,2)交换 -> [2,1,3,4,5,6], last=2
  后三对均不换，本趟结束令hi=2。
hi=2: 只比较(0,1)，交换 -> [1,2,3,4,5,6], last=1，令hi=1。
hi=1: 不足两项，结束。
```

下一轮只需处理下标 0、1；下标 2 上的 3 已经是原活动区间前三项的最大值，且后面 4、5、6 也都不小于它。若把 last 理解成“下次还要比较到的右端下标”，会多比较或漏比较，必须与半开 hi 的约定配套。

last 初值和边界含义要配套：若 last=0 表示未发生交换，就可以结束；若存左端下标，下一轮界限又差 1。画清“还比较哪些相邻对”，比套一个神秘赋值更可靠。逆序最坏仍二次。

23.35 35 · 非对半归并：固定比例仍是 nlog n

设固定 $\alpha \in (0,1)$，递推为 $T(n) = T(\alpha n) + T((1-\alpha)n) + \Theta(n)$。每层区间总长度不超过 n，最长路径 O(log n)，上界 O(nlog n)；前常数倍 log n 层都尚未触到基础规模，层成本为 Θ(n)，给出匹配下界。

例如 1/4 与 3/4 切分比对半树更深，但深度只差固定常数。若比例随 n 变化成 1 与 n-1，就变成 $T(n) = T(n-1) + \Theta(n) = \Theta(n^2)$。

固定比例偏斜通常使常数变差，还可能改变实际缓存表现；渐近同阶不代表实际一样快。先分清常数比例与每次只减固定个数。

23.36 36 · 复用归并缓冲：不必每个节点都 new 一遍

排序入口一次申请长度 n 的辅助数组，递归时把区间边界传下去，每次归并使用对应片段，退出整个排序后统一释放。顺序递归的兄弟区间不会同时使用同一片段，可以复用。

原来每个归并都申请、释放缓冲，分配调用次数可能达到线性数量；统一缓冲把这部分调用降为常数次。元素比较和迁移总量仍为 O(nlog n)，峰值辅助空间仍 O(n)，不能把少做分配说成空间阶从 n 降成常数。

若改成并行归并，要检查不同任务的片段是否重叠。C++还需选择缓冲中哪些元素已经构造、哪些需移动赋值，不能只把裸指针传来传去。计时应对同一输入副本比较两版，并把复制测试输入放在计时区之外。

23.37 37 · 一边耗尽：右半剩下的值为什么可能不用再复制

课程常把左半复制到缓冲，右半仍在原数组，写入位置从原区间左端向右移动。左半先耗尽时，写指针与右半读指针会重合，右半剩余项已经处在正确位置，无需原地复制。

例如左[1,2]、右[3,4]，前两次把 1、2 写回，随后读右半的位置恰好就是接下来输出的位置。若右半先耗尽，缓冲中的左半还没在正确位置，必须继续写。

再用交错例核对写指针为何追得上但不会超过右读指针。原数组`[1,4,7 | 2,3,8]`，左缓冲 `B=[1,4,7]`；写指针 k=0，右读指针 j=3。依次写 1、2、3、4、7，对应来源为左、右、右、左、左，此时 k=5、j=5、左缓冲耗尽，原下标 5 的 8 已经在最终位置。任何时刻“尚未输出的左缓冲项数”恰是 j−k，因此左侧不空时 k<j，不会写坏未来还需读的右项；左侧为空时 k=j，后缀自然就位。

如果实现把两半都复制到了外部缓冲，那么两边的剩余项都要拷回。答案取决于源、目的在哪以及读写指针的关系，不是“归并永远只处理一边耗尽”。

23.38 38 · 已经整体有序：只检查分界那一对就够吗

左右分别有序且非空。若左半最大值不大于右半最小值，即 `a[mid-1]<=a[mid]`，那么任意左项都不大于任意右项，整段已经有序，可以直接返回。

这个检查多一次比较；在已经有序输入上，每个递归节点都能跳过整段归并，总成本可降到线性量级的节点检查。普通输入若经常不满足，只是多付一个小常数，不保证所有机器都一定更快。

要先处理空段和长度小于 2，避免 mid-1 越界；比较器若只提供 less，用 `!less(a[mid],a[mid-1])` 表达非降关系。相等时不必交换，稳定性也能保留。

23.39 39 · inplace_merge：接口名字没有保证完全不申请缓冲

C++的 `inplace_merge` 表示结果仍放回原区间，不等于所有实现都禁止临时存储。要以标准规定的语义、复杂度条件和手头库源码为准，不能照题面一句“彻底避免辅助空间”推广到所有库。

一种低元素缓冲的思路是：在两段中选切点，把中间错位的两块通过旋转交换，再递归处理两个较小归并。旋转可用反转实现，能省大块元素缓冲，却增加移动和控制工作；递归本身还可能用 O(log n)栈空间。

例如[1,4,7]与[2,3,8]，可先找 4 在右段应插入的位置，把[4,7]与[2,3]旋转成[2,3,4,7]，再处理更小边界。要证明稳定，切点的 lower_bound 或 upper_bound 取法也必须正确。空间紧张、大对象移动昂贵、数据规模等因素共同决定是否值得，不能只看“原地”两个字。

上例的块旋转可用三次反转实现：原中段`[4,7 | 2,3]`，先分别反转得`[7,4 | 3,2]`，再整体反转得`[2,3 | 4,7]`。块内相对次序最终恢复，所以这个旋转本身不打乱各块内部的相等记录；但二分切点若把左右相等记录分错边，整个递归归并仍可能不稳定。真正的通用算法还要递归处理两侧的新归并任务，这个特例旋转后碰巧全部有序，不代表任意两块转一次就完成排序。

标准对非并行 inplace_merge 明确区分额外内存是否足够：有足够内存时比较至多 N−1，否则允许 O(Nlog N)。核对合并接口与复杂度。

23.40 40 · 位图复位：清零所有格子，还是宣布旧记录失效

第 k 位在字节 `k/8`，偏移 `k%8`；置位用掩码或，清位用掩码取反再与。若值域大小 U，普通位图整体清零要处理约 U/字长个存储字，成本依赖 U。

例如值 13 在字节下标 1、位偏移 5。若字节当前是二进制 `00000100`，掩码是 `00100000`，或以后得到 `00100100`；清 13 则与 `11011111` 相与，恢复 `00000100`。字节中的低位约定必须统一，跨字节时 13 不是“第 13 个字节”。位图扩容也要拷贝旧字节、初始化新增字节为 0；若只追加未初始化内存，随机旧位可能被误认成存在。

`std::bitset<N>`的 N 是编译期固定大小，提供按位逻辑、set/reset/test 等接口；课程可扩 Bitmap 通常运行时扩容。二者不只名字不同，大小何时确定、越界时接口保证、能否整体位运算都影响用法。

Hopcroft 结构用稠密表 dense 保存当前存在的值，稀疏表 sparse 给出候选位置，另存逻辑长度 m。判断 x 存在，需要同时满足 `sparse[x]<m` 与 `dense[sparse[x]]==x`。只把 m 设为 0，就让所有旧候选位置失去校验效力，逻辑清空 O(1)。

另一种方法给每格存时间戳，清空时增加当前世代号，格子时间戳等于当前世代才有效。世代号有限宽，回绕时需要真正重置；首次分配、初始化和空间都不能消失。C++读取未初始化整数也不合法，所以“逻辑清空 O(1)”不等于首次创建一个巨大数组也完全免费。

23.41 41 · 普通位图去重并排序：时间里有值域这一项

把 n 个值的对应位设为 1，耗 O(n)；再从 0 到 U-1 扫描置位项并输出，耗 O(U)，合计 O(n+U)。扫描按数值递增，所以同时完成去重与排序。

若 U≤cn，便可简写为 O(n)。若 n=1000、U=十亿，扫描十亿位范围的需要处理的范围由 U 决定，不能只拿 1000 估计；位图还可能占约 125MB，具体单位和额外对象另计。

可以只在读输入时检查并保留首次出现来避免全域输出扫描，但那保留的是输入顺序，不再自动排序。输出要求不同，复杂度公式也不同；不能拿去重的改进偷偷替代“去重且排序”的原任务。

23.42 42 · 校验环删除：拿最后一项填洞

设 dense=[4,9,2]、m=3，sparse[4]=0、sparse[9]=1、sparse[2]=2。删 9 时，位置 1 会留下洞；把末尾 2 搬到 1，令 sparse[2]=1，再 m−，有效 dense 变成[4,2]。

旧 sparse[9]即使还等于 1 也没关系，因为 dense[1]现在是 2，双向校验失败，9 被判为不存在。若删的本来就是末项，直接 m−即可，交换自身的写入也可省掉。

将“查不到的旧值”也走一次：删除 9 前，查询 9 读 `sparse[9]=1`，1<3 且 `dense[1]=9`，两关都过；删除后 m=2，旧 `sparse[9]`仍为 1，第一关 1<2 还过，但第二关 `dense[1]=2!=9`，因此拒绝。若只检查位置在有效范围，就会把 9 错判为仍存在。后来再插入 9，写 `dense[2]=9,sparse[9]=2,m=3`，双向关系重新建立。

每次只改常数个格子，因此删除 O(1)。前提是先确认 x 确实存在；若拿一个无效 sparse 位置直接搬，会破坏其他记录。删后 dense 的顺序可能变化，这也是它与保持插入顺序容器的差别。

23.43 43 · Hopcroft 位图：免掉全域扫描，付出什么代价

扫描 n 个输入，每个用双向校验判断是否已存在，不存在就加入 dense，并更新 sparse。已有结构的逻辑清空 O(1)，每个输入处理 O(1)，因此这一轮去重 O(n)，不用把整个值域逐格清零或扫描。

但直接寻址的 sparse 仍随 U 占空间；相较一位表示一个值，它常需要一整个整数位置，空间明显增加。首次建立可合法读取的存储仍有初始化成本，若把 U 也计入本次从零开始的总成本，不能声称完全与 U 无关。

dense 保存的是处理顺序或交换后的顺序，去重结果不再天然按数值排序。若还要求有序输出，需要另排序，或者重新扫描值域。节约的是反复清空和稀疏访问的时间，代价包括空间、顺序与初始化条件。

23.44 本章练习与核对

1. 容量 8、规模 8 的表从头插 1 项，为什么“追加均摊 O(1)”不能推出这次头插 O(1)?
2. 对[2,2,5]查 2，用 C 版列出 lo、hi、mid，返回哪一个 2？若查 1 又怎样?
3. dense=[6,1,8,3]，删除 1 后，至少需要改哪几项？旧 sparse[1]为什么可以暂时不擦?

第一题：几何扩容只控制整条追加序列的迁移账；头插还需要移动原后缀，单次本来就是 Θ(n)，扩容也可能单次 Θ(n)。第二题：起初[0,3)，mid=1，2 不小于 2，lo=2；再 mid=2，2 小于 5，hi=2，返回 1，即最

后一个 2。查 1 先 hi=1，再 hi=0，返回-1，不可解引用。第三题：把 3 写到 dense[1]，sparse[3]改成 1，m 由 4 减到 3；旧 sparse[1]=1，但 dense[1]不再是 1，所以校验拒绝。

24 PS03：节点身份、接线顺序与排序成本

教师原始习题单：第 03 次习题共 37 题。这里把节点身份、逻辑次序和数据值分开：移动节点改变链接；交换 data 改变值属于哪个节点；删除并新建还会改变对象生命期。它们有时得到相同的数值序列，却不一定对外部指针、稳定性和资源管理产生相同效果。

24.1 1 · 静态与动态：看逻辑结构，不能只看地址动没动

静态操作查询现有数据，动态操作改变集合内容或结构。游标链表把节点放在固定数组格子里，用 succ 下标说明后继；把链接从 2⟶5⟶7 改成 2⟶7⟶5，物理格子都没挪，逻辑顺序已经改变，因此仍是动态操作。

反过来，底层把整个存储搬到新地址，只要抽象集合与顺序保持，用户看到的逻辑内容未必变化。接口分析应先说明以哪一层为准，而不是把“动态”误解成“指针数值变了”。

可以拿三个卡片做演示：卡片固定贴在桌上，只改“下一张编号”，输出顺序也能改变。这就是列表逻辑链接与数组物理邻接的差别。

24.2 2 · 秩与位置：写成 p+100，也还是走了 100 步

秩是第几个，位置是某个节点句柄。向量从首地址加秩乘元素大小可直接寻址；列表一般只能从一个已知节点沿链接走。给列表重载 `p+k`，可以把走 k 步封装得好看，不能因此让它变成常数时间。

在长度 n 的链表上循环访问 `L[i]`，若每次从头找第 i 项，总访问量是 $0+1+\cdots+(n-1)=\Theta(n^2)$。改为保存当前节点 p，每轮 p=p->succ，总共只走 n 次，就成线性。

因此课程不建议频繁用模仿数组的重载语法。阅读代码时主动问“这一行内部要走几条边”，不要让一个加号把复杂度藏起来。

24.3 3 · 两节点距离未知方向：两边同时试，不必先找头

假设 p、q 是同一合法双向链表内节点。若已知 p 在 q 前面，从 p 沿 succ 走，数到 q 即可。距离 d 按边数计算，p=q 时返回 0，整体可写 O(1+d)。

方向未知时，可以从 p 同时向前、向后扩展，每轮各走至多一步，任一方向碰到 q 就结束；碰到哨兵的方向停止，另一边继续。最多各探索 d 步，所以仍 O(1+d)。也可从 p、q 分别向后探索，看谁先遇到对方。

例如链 `H↔A↔B↔C↔D↔E↔T`，p=C、q=A。第 0 轮两探针都在 C；第 1 轮向前到 B、向后到 D，都不是 A；第 2 轮向前到 A，返回距离 2，不再继续。若 q=E 则同样在第 2 轮从另一边找到。p 恰在首项时，向前探针第 1 轮碰到 H 就冻结，向后探针仍继续；不能让探针穿过哨兵去读无效 pred。每轮付至多 2 条边，找到后至多付 2d，所以是$O(d+1)$。

这需要同表与有效节点的前提。若 q 不属于该表，走到边界也找不到，必须返回错误，不能把哨兵当正常数据继续走。为了防御不可信句柄而先全表验证，可能另付 O(n)，与在已保证前提下的距离算法分开计费。

24.4 4 · 倒数第 k 项：双指针省的是重复定位，不是神奇少读所有节点

单链无长度时，先遍历数 n，再从头走 n-k 步，总边访问大致 $n+(n-k)$。双指针先让 fast 领先 k 步，再让两者同步；fast 到末尾时，slow 到倒数第 k 项。

以[10,20,30,40,50]找倒数第 2 项：fast 先越过 10、20，到 30；同步三轮：`(slow,fast)`从`(10,30)`依次变成`(20,40)`、`(30,50)`、`(40,tail)`，这时 slow 到 40。初始化究竟把 fast 放在“第 k 项”还是“越过 k 项”取决于尾哨兵约定，必须按所用表示画一遍。

对应代码假定空链的 first 就是 tail，fast 每领先一步都先检查是否已到 tail：

```
slow = first; fast = first
重复k次：若fast==tail则k过大；否则fast=fast.succ
当fast!=tail：slow=slow.succ；fast=fast.succ
返回slow
```

两指针之间始终相隔 k 条链接。fast 到尾后，slow 后面恰好还剩 k 个真实节点（包括 slow 自己），所以 slow 是倒数第 k 项。k=n 时 fast 初始化后已经到 tail，slow 不动，返回首项；k=1 时两者最后分别在末项与 tail。

两种方法都 O(n)，双指针仍可能合计走近 2n 条边；其优势是一次向前流程、避免重新从头定位，也可能有不同缓存行为。k≤0 或 k>n 时必须拒绝。实验要固定内存布局和 k 的比例，而不是只比较一条很短链的偶然耗时。

24.5 5 · 哨兵：空表也有能接线的邻居

双向链表保留 head、tail，空表为 head↔tail。把 x 插入两者之间，用与一般位置同样的四条连接：x.pred=head，x.succ=tail，head.succ=x，tail.pred=x。删除唯一元素 x，再把 head.succ 指回 tail、tail.pred 指回 head，就恢复空表。

这消除了“首节点没有前驱”“尾节点没有后继”的多套分支，但并不把所有 O(n)接口改成 O(1)。它主要统一边界并减少错误机会，精确运行时间仍应实测。

要测试空表插入、删唯一项、连续清空两次、首尾移动和复制空表。哨兵不保存普通元素，不参与查找比较，也不该被普通 remove 删掉。若遍历到 tail 还读取 data，哨兵并没有替你保证安全。

24.6 6 · 四条链接的顺序：先保存旧关系，再改图

把新节点 x 插在 p 与 q 之间，先记住 q=p->succ，再写 x 的两条边，最后改 p 与 q 反向指向 x。若一上来 `p->succ=x`，随后又用 `p->succ` 当旧 q，就把旧后缀丢掉了。

令原图 `P↔Q`，新节点 X 刚分配好，还不在表中。按下面顺序写，每行需要的旧信息都仍保留：

步	写入	这一步之后已确定的边
0	`q=p.succ`	局部变量 q 记住 Q
1	`x.pred=p`	X 向左认 P
2	`x.succ=q`	X 向右认 Q
3	`p.succ=x`	P 向右改认 X；Q 仍由 q 保存
4	`q.pred=x`	Q 向左改认 X，最终 P↔X↔Q

第 1 与第 2 行可互换，第 3 与第 4 行在前两行准备好后也可互换；第 0 行若拖到改写 `p.succ` 之后，就会把 q 错记成 X。中间图尚未双向一致不表示这段单线程、无回调代码必错，但在一致性恢复之前不能把半成品暴露给可能遍历的代码。

删除 x 时先保存 p=x->pred、q=x->succ，再令 p->succ=q、q->pred=p，最后释放 x。释放后再读 x->succ 违反生命周期要求。两条互不依赖的写入可以交换，读旧边之前覆盖旧边则不行。

检验顺序的办法是每写一行就画出所有箭头，标出下一行还会读取哪条旧边。局部最终图相同不够，执行途中也不能读取已经丢失或释放的对象。

24.7 7 · 把节点移动到自己前面：这次应当什么都不做

若 `this==s`，通常把 move 定义成无操作，直接返回。若没有检查，先把 this 摘链，再试图插到 s 之前，s 恰好就是刚摘下的节点，后继和前驱已不再代表原位置，容易接成自环或断链。

相邻情形也要检查：x 本来就在 s 前面，再移到 s 前面应当保持原样；从另一个表转移，则还要同步两表的 size 和所有权。若接口不允许跨表或自身移动，应把它写成前置条件并断言，而非含糊地期待调用方永远猜对。

真正移动 B 到 A 前时，先摘 B：保存 `oldLeft=A,oldRight=T`，写 `A.succ=T,T.pred=A`，此时表为 `H↔A↔T`，B 独立；再保存目标 A 的前驱 H，写 `B.pred=H,B.succ=A,H.succ=B,A.pred=B`，得到 `H↔B↔A↔T`。必须先判断 B 不是目标自己。对 A 移到 B 前的相邻无操作，也可直接早退；若仍用通用摘接，要在摘掉后重新取得目标的当前前驱，不能使用已过时的相邻关系。

一个很小的测试表 head↔A↔B↔tail 就够，分别移动 A 到 A 前、A 到 B 前、B 到 A 前，逐次核对正向和反向遍历是否一致。

24.8 8 · insertFirst 复制：反着读，才能正着留下

原表 A→B→C，从头顺序读并每次 insertFirst，目标变化为空→A→B,A→C,B,A，结果反序。若从尾向前读，先 C、再 B、再 A，目标最终 A,B,C，顺序保持。

双向链表能直接沿 pred 反向遍历。单链若每次重新找前驱，会退化；可先用辅助栈保存节点或采用尾插，别为了使用 insertFirst 而制造无谓二次扫描。

独立复制 n 个节点需要 Θ(n)时间、Θ(n)新存储。即使额外工作空间是常数，新表本身的 n 个对象也必须分配；回答空间复杂度时说明是否把输出存储排除。

24.9 9 · 单链析构：先记住下一家，再拆当前这家

```
Node* p = first;
while (p != tail) {
    Node* next = p->succ;
    delete p;
    p = next;
}
```

单链没有 pred 不妨碍这个流程，因为销毁时只向前走。每个节点读一次后继、释放一次，时间 Θ(n)。若 tail 也是动态分配的哨兵，循环后再按所有权规则释放它；不能当普通节点提前删。

错误写法是先 delete p，再读 p->succ；另一个低效写法是每轮找最后节点并删除，每次都从头走，成本 n+(n-1)+…，变成 $\Theta(n^2)$。保存一个 next 就解决的问题，不需要用重新遍历补救。

24.10 10 · 链表无序去重：删一个节点，不必搬整个后缀

维护已经处理的无重复前缀。当前节点 p 在此前缀里找到同值 q，可以删 q 保留较新的 p；若改删 p，则必须先保存 p 的后继，再释放，随后从保存的节点继续，不能再访问 p。

全相等输入，每轮很快找到相等项，且删除只改常数条链接，最好 Θ(n)。全互异输入，第 k 项要查 k 个前缀节点，累计 $\Theta(n^2)$。课程原 Vector 版本即使比较少，也可能不断移动后缀，所以“同样的去重思路”在不同存储上有不同最好情况。

原题还问第一次 find 查空前缀是否安全。合法的计数式查找先检查“还需比较的节点数 r 是否大于 0”，r=0 时直接返回前边界，根本不读节点 data。若接口约定先取 pred 再循环，返回 h 仍可合法，但绝不能把 h.data 当普通元素判等。

对 `H↔A1↔B1↔A2↔T` 保留最后一份：A1 查空前缀，留下；B1 查 A1，不等，留下；A2 查 B1 再查 A1，命中后摘掉 A1，得到 `H↔B1↔A2↔T`。若改成删当前 A2，必须先保存 next=T，摘并释放 A2 后令 p=T，不能继续用 p 取 succ。两个版本输出不同但都可实现，先声明保留哪一份。

若改维护无重后缀，可从尾向前扫描，在后缀找重复；双向表能够对称实现。空表、单项、全等、首尾重复都要测。用哈希额外索引可改变期望比较成本，但已经增加了另一数据结构，不能不计其空间与假设。

24.11 11 · 遍历：访问器有没有资格修改链接

求和、计数、打印、验证有序性都可用访问器。计算平均值时让访问器保存 sum 与 count，遍历完再除；空表 count=0 时不能直接除 0。

若访问器仅修改 data，普通遍历一般还能沿 succ 继续；若它删除当前节点，遍历框架随后读取当前 succ 可能已经访问释放内存。需要安全删除协议：预存 next，允许回调返回新的位置，或明确禁止改链接。

因此“都能用遍历实现”不表示任何回调都能塞进同一个接口。先读参数是值、引用还是节点位置，再判断它能读写什么。

24.12 12 · 用 traverse 写 find/search：接口可能让你无法提前停

find 访问每个元素，若第一次相等就保存命中节点；search 在有序表里保存最后一个不大于 e 的节点。后者遇到第一项大于 e，本来就可以结束。

但若 traverse 的回调只返回 void、框架必定扫完整表，访问器虽然可以不再做比较，仍然发生后续指针移动。要真正提前停，可设计返回 bool 的回调或单独提供短路遍历。

例如[2,5,9,11]查 3，前驱是 2，看到 5 即可确认；通用遍历若继续走 9、11，只是功能正确而未达到同样的常数优化。接口能力和算法成本需要一起检查。

24.13 13 · 有序去重：不必为了少一行特判去冒险读哨兵

少于两项，没有一对真实相邻项可比较。先判 size<2 可以直接返回，避免构造第二节点位置后读到 tail。也可以令循环显式要求当前 p 和其后继 q 都是真实节点。相等时删除其中一项，不等时推进；若删除 q，p 不动并重新取新后继，才能处理[1,1,1]连着三个相等值。

对 `H↔1A↔1B↔1C↔2D↔T` 保留首项时，p 先在 1A，q 在 1B，相等删 q；p 仍在 1A，重新令 q=p.succ 即 1C，再删；下一次 q 变 2D，不等，才把 p 推进到 2D。若每次删除后也推进 p，可能跳过 1C，留下两个 1。停止条件应先保证 q 不是 T，再读 q.data。最终链接为 `H↔1A↔2D↔T`，真实节点 3 次相邻判定，删除 2 个节点。

测试空、单项、两项同值、两项不同、整表全等、首尾各一组重复。无论哪种改写，证明都应是“已经越过的前缀无重复，且保持指定的保留规则”，而非单纯说代码行数少了。

24.14 14 · search 返回前驱：返回了一个位置，不等于找到了相等值

在有序区间(h,t)里，search 找最后一个不大于 e 的项。若 e 比所有项小，结果是头边界 h；插在它后面，就让 e 成为首项，顺序仍正确。

例如[2,5,9]查 7，返回 5 的位置。接着在其后插 7 得到[2,5,7,9]，这就是前驱接口的用途。若只想判断 7 是否存在，需要先确认返回值不是 h，再用一致的等价关系比较返回元素与 7。

查 2 返回 2、查 1 返回 h、查 12 返回 9，三个例子可以把“精确匹配”“前驱”“插入位置”区分清楚。把任意非空指针当命中会把后两种误判。

24.15 15 · 有序单链搜索：少比较，也未必少走路

只有相邻链接、无额外索引时，要到中间节点就得先经过前面的链接。即使关键字有序，对手也能把答案或最后确认位置放在很远处，最坏需要 Ω(n)次边访问。

在链表上做二分，比较次数可能降成 O(log n)，但找各次中点的总步数仍可为 O(n)，不会像数组那样获得 O(log n)总时间。若每次又从头定位中点，甚至可能更差。

取 8 项，第一次中点在第 4 项，需要从首走 3 条边。若目标在后半，下一次在余下 4 项中定位中间，再走约 2 条；再下一次约 1 条。总步数是$4+2+1=7$量级，虽然只比较 3 次，仍与 n 同阶。即使刻意把每层的走路量做得最省，几何级数$n/2+n/4+\cdots$也有$\Theta(n)$，省比较并没有省掉第一次抵达中点的路程。

增加跳表层、树索引或按秩定位结构能改进，但这已改变数据结构和维护成本。题目问原链表能不能做到，答案必须带着这个访问模型，不能用另一个结构的能力冒充。

24.16 16 · 选择排序的交换、平移和删建，是三种不同的动作

交换法每轮找到无序前缀最大项 m，与前缀末项 x 交换 data，再把边界左移。它只做常数次元素移动，不必分配节点。若用 remove 取出值、再 insert 新节点，会多做分配、析构和复制，原节点身份也丢了。

若“平移”指摘下同一个节点再接到后缀前面，则无需新建，代价主要是链接写入，且其他元素相对顺序可以保留。比较两个实现时必须读清 remove 返回的是值，还是只摘链返回节点。

例如外部保存了 m 的节点地址。交换 data 后，该地址里的值可能变了；移动原节点后，该地址和值仍然配对，只是邻居变了。只检查最终数值排序，检验不到这项接口语义。

24.17 17 · 最后一个最大值：能帮助稳定，但不能救任意交换

从后向前找最大，初始化为最右项，只在遇到严格更大值时更新，就自然保留最后一个最大值。相等项不会更新；某个值即使多次出现，也最多在它第一次严格超过现有最大值时触发一次记录更新。

若从前向后扫描，为了仍取最后一个最大值，要在大于或等于时更新。全相等长度 r 就会更新到每一项，最多 r-1 次（不计初始化）。扫描方向与相等处理必须配套。

把最大记录更新过程单独走一次，前缀`[4A,7B,7C,2D,7E]`从右扫描：初始化 m=7E；遇 2D 不换，遇 7C 相等不换，遇 7B 相等不换，遇 4A 更小不换，最终保留 7E。若从左扫描并用≥更新，则初始化 4A，依次更新到 7B、7C、7E，三次；同一个最大 key 造成多次更新。全相等 r 项时达到 r−1 次更新。

“每组相等元素最多使 m 更新一次”依赖的是从右向左、严格大于才更新：某组的最右成员若第一次刷新了最大值，之后同值成员都不再严格大；若它没能刷新，后面的最大值只会更大，这一组以后也不可能刷新。

选最后最大再把整个节点稳定地摘到后缀，可保留等值次序；但任意交换 data 仍可能破坏稳定。对[(2,A),(1,B),(2,C),(1,D)]，先将 2C 换到末尾，再将 2A 换到剩余末尾，可使 1D 跑到 1B 前面。因此要审查整套排序，不把 getMax 的一个好性质扩展成整体保证。

24.18 18 · 向量选择排序：二次比较，线性移动确实能同时成立

长度 r 的无序前缀找最大需要 r-1 次比较，r 从 n 降到 2，合计 $n(n-1)/2$。每轮最多做一次交换，而交换由常数次元素赋值组成，所以元素移动 O(n)。

例如[3,1,4,2]：先选 4 与末项 2 交换得[3,1,2,4]；再选 3 与 2 交换得[2,1,3,4]；最后换 2 与 1，得到有序。比较扫描很多，真正交换仅 3 次。

它适合需要控制移动次数的模型，但若比较也很贵，二次比较仍是明显缺点。普通交换版不稳定；若改成数组区间平移来保序，可能引入二次移动，不能同时保留原来的全部优点。

24.19 19 · 交换与轮换：先把排列画成箭头

将每个位置连向其元素的最终位置，整个排列分成若干不相交的环。已经就位的元素是长度 1 的环。交换两个位置，只改变这两个位置发出的归属关系，其余箭头保持。

若两位置在同一环上，交换会把环切成两个；若在不同环上，交换会把两环接成一个。可以用[2,3,1]：原箭头 1⟶2⟶3⟶1，交换位置 1 与 3 后成为[1,3,2]，拆成固定点 1 与环(2,3)。

再补“不同环”的情况。排列`[2,1,4,3]`有两个环`(1 2)`和`(3 4)`。交换位置 2 与 3，数组变`[2,4,1,3]`，箭头依次为 1→2→4→3→1，两个环合成一个。把这次交换再做一次，当然又把一个环拆回两个。由此可见一次任意交换不会保持环数不变，而是准确改变 1；下界论证才能按每次最多增 1 收费。

因此一次交换让环数改变 1，奇偶性也改变。环不是“数组里相邻的一段”，两节点在环中相邻与在物理序列中相邻完全不同。

24.20 20 · 交换选择排序：为什么有效交换数是 n 减环数

无序前缀末位置 x 应该放当前最大值 m。因为 m 的目标就是 x，m 所在位置的箭头直接指向 x 所在位置，所以两者属于同一环且沿一个方向紧邻。交换后 m 就位成为固定点，原环少一个成员。

若初始有 c 个环，每个长度 l 的环需要 l-1 次这种有效交换，合计 n-c。课程若连最后长度 1 的步骤也做，每个环最终留下的那个固定点对应一次原地交换，所以无效交换共 c 次；若省去最后一步，计数相应少 1，必须统一约定。

在`[2,3,1,5,4]`上可以把“每个有效交换拆一个固定点”全部看见：

```
初始环：(1 2 3)、(4 5)，c=2。
r=5：最大5在位置4，交换4与5 -> [2,3,1,4,5]；环(4 5)拆成(4)(5)。
r=4：最大4已在位置4，原地动作。
r=3：最大3在位置2，交换2与3 -> [2,1,3,4,5]；三环拆出(3)。
r=2：最大2在位置1，交换1与2 -> [1,2,3,4,5]；拆出(1)(2)。
r=1：原地动作。
```

有效交换 3 次，等于$n-c=5-2$；含 r=1 的原地动作 2 次，等于初始环数。若课程循环在 r=2 之后结束，就不执行最后一次原地动作，此时原地次数为 c−1。计数前先说明最后一步是否执行。

互异均匀随机排列中，当前 r 项的最大值等概率位于任一位置，在末端的概率 1/r。对 r 从 1 到 n 求和，原地交换期望为 $H_n=\sum_{r=1}^{n}1/r$。由逐输入“原地次数=环数”，得到期望环数也是 H_n，因而 Θ(log n)。

这不是假设每次事件独立。期望可相加，不需要独立；但最大位置均匀来自输入排列的随机模型，不能无条件套到人为构造数据。

24.21 21 · 换成平移：逐输入的环数等式已经失效

取[3,1,2]，原排列是一个三环。平移最大 3 到末尾后变成[1,2,3]，随后长度 2 与长度 1 的最大项都已经在末尾，发生两次原地动作。初始只有 1 个环，因此“原地次数恰好等于初始环数”不再成立。

仍在[3,1,2]上分别画两种状态：交换法先把 3 与 2 互换得到[2,1,3]，还要交换 2 与 1；平移法摘掉 3，把后面的 1、2 向前接起来，再把 3 放到末尾，立即得到[1,2,3]。二者剩余排列不同，所以不能沿用同一条输入的环数计数。

为什么平均数还能相同？固定 r−1 个较小元素的一种相对次序，把最大元素插入 r 个不同位置，就得到 r 个等可能的原排列；删掉最大项都回到同一剩余次序。所以每一种剩余次序恰好有同样多的来源，均匀性保留。交换法也能用“指定最大原位置后逆向交换”做相同的来源计数。

但均匀随机排列中，删去最大项后，其余项保留相对次序，仍是均匀随机排列。因此当前 r 项的最大值处在末端仍以 1/r 概率发生，原地次数期望依旧 H_n（若从 r=2 计则 H_n-1）。

两个方法能有相同的平均数，却对同一个具体排列给出不同的次数。证明平均结论应重新基于剩余相对顺序的均匀性，不能拿已经失效的逐输入等式继续推。

24.22 22 · 在线与输入敏感：抓一张牌就插进去

插入排序看见新牌，就在已排好前缀里找位置并插入，任何时刻已收到的牌都有序，所以适合“还会继续来牌”的在线维护。用平衡搜索树接收新键也能在线维护顺序，但结构和输出方式不同。

输入敏感指成本还取决于某种输入难度。插入排序可按逆序对数 I 写成 O(n+I)的常见数组移动界；已经有序时线性，完全逆序时二次。带提前终止的冒泡也对某些接近有序输入更省。

选择排序每轮仍要扫描剩余集合，比较数不因输入已序而减少。它的优势是总交换 O(n)。因此不能只问“哪个排序更好”，应说清比较、移动、在线响应和稳定性分别要求什么。

24.23 23 · 数组插入：二分找到了位置，格子还是要腾开

保存当前值 x，把比 x 大的前缀元素依次右移，最后把 x 填入空位。例[2,4,7,3]：先存 3，7 右移，4 右移，再把 3 写到 2 之后，得到[2,3,4,7]。

这一轮的完整数组状态如下，空洞只在逻辑上标记，x 已经存到临时变量：

```
x=3，洞在3：[2,4,7,_]
7>3，7右移：[2,4,_,7]，洞移到2
4>3，4右移：[2,_,4,7]，洞移到1
2<=3，停止：[2,3,4,7]
```

循环不变量是：洞右侧到当前轮末尾已经是应当位于 x 之后的元素，原前缀尚未被覆盖；每次把洞左边较大项搬到洞里，洞向左走 1。保留 x 直到最后填洞，避免把它在第一次右移时丢掉。

逆序输入时，第 i 项要越过前 i 项，移动数是 1+2+…+(n-1)，所以 $\Theta(n^2)$。不能像链表那样只改常数条边，因为数组的逻辑连续性正依靠连续格子，必须给插入位置腾空间。

二分能把查位置的比较降到 O(log n)，却不能消除 Θ(n)后缀移动。链表则相反：按秩查位置不便宜，但已知位置后接入节点可以常数改链。每种结构省的工作不同。

24.24 24 · move 保留对象，remove 加 insert 可能重建对象

节点转移先摘下现有节点 x，再接到目标位置；x 的 data 对象可以完全不复制，外部持有的节点地址也保持。remove 若销毁节点再返回值，insert 又新建节点，则增加分配、构造和析构。

对于大字符串、不可复制对象或有身份的记录，这些差别比几条指针赋值大。对于只有几个字节的简单对象，差别可能主要是分配器常数，也应通过实际实现确认。

例如 H↔2A↔4B↔7C↔3D↔8E↔T，待插节点 x=3D，原后继 next=8E。已找插入前驱 2A：先摘 3D 使 7C 连到 8E；再把 3D 接到 2A 与 4B 之间。得到 H↔2A↔3D↔4B↔7C↔8E↔T。下一轮必须处理保存的 8E，若沿移动后的 3D.succ 走，会回到 4B，把已经排好的节点再处理一次。

移动后扫描指针必须指向原后继，而不是沿 x 的新后继回到已经排好的前缀。先保存 next，再移动 x，是避免重复处理和无限循环的关键。

24.25 25 · 插入排序：原地不动几次，比整体有序容易得多

互异均匀随机排列中，完全有序只有 1 种，概率为 1/n!。但某次插入不移动，只要求新项是当前前缀最大值；在前 k 项中，每个位置等概率容纳最大值，所以该事件概率 1/k。

若排序从第 2 项开始，原地插入的期望次数为 $\sum_{k=2}^{n} 1/k = H_n - 1$；若把首项初始化也计一次，就是 H_n。n=4 时，期望为 1/2+1/3+1/4=13/12，不需要接近 1/n!那么罕见。

平均逆序对为 n(n-1)/4，标准插入排序每次右移消除一个相应逆序，所以平均仍 Θ(n²)。偶尔许多项无需移动，与总体二次工作并不矛盾。

24.26 26 · 为什么链表插入排序不直接二分

有序只说明比较后能排除一半候选，不保证可以常数时间跳到中点。链表从当前边界走到中点已经需要一段指针移动，所有层累计仍可能线性；每次从头重新找位置还会增加重复工作。

要真正像数组那样实现 O(log n)定位，需要高效随机按秩访问，或增加可跳跃的索引。同时插入后还要高效维护这个索引，否则定位省下的时间会在更新时付回。

即便对数组采用二分插入，移动仍可能二次；对链表普通顺序找位置虽然比较多，却能常数接链。先把“比较次数”“走链接次数”“移动数据次数”三张账分开，再决定该优化哪一个。

24.27 27 · 逆序对：每一对位置各自交一张账单

互异元素中，逆序是 i<j 却 a[i]>a[j]。共有 n(n-1)/2 对位置，完全递减时全部逆序，这是最大值。

随机排列中任意一对大小方向对称，成为逆序的概率 1/2。令每对的指示变量为 1 或 0，期望相加得 $E[I] = n(n-1)/4$，无需假设所有对之间独立。

例如[3,1,2]逆序为(3,1)、(3,2)，共 2；相邻交换 3 和 1 后剩(3,2)，再交换便有序。普通冒泡每次只消一个逆序；插入右移也能用这些对逐项收费，所以随机平均 Θ(n²)。

24.28 28 · 冒泡只跑 n 趟：先查清最后一趟是不是确认趟

原题长度为 2n，采用课件第 75 页提前终止版。该版在最后一次有交换之后，还要执行一趟“完全没交换”的确认，才让外循环停止。下面把这趟也计入 n，因此有交换的最多左移深度为 $d = n - 1$。

一个元素每趟最多向左移动一格。若需要 d 趟有交换，至少存在一个元素要越过 d 个较大前项，所以逆序至少 d。取前 d+1 项为[2,3,...,d+1,1]，其他值递增，恰有 d 个逆序，并需要 d 次有交换加一次确认。

每个有交换的第 i 趟至多比较并交换 2n-i 对，所以逆序总数至多

$$\sum_{i=1}^{d}(2n-i) = 2nd - \frac{d(d+1)}{2}.$$

取前 d 项为最大的 d 个数、递减排列，后面其余数递增：每趟都能把一个大项推到末端，达到这个上界。把较小的$n = 3, d = 2$走全，最少逆序构造为`[2,3,1,4,5,6]`：

```
第1趟：[2,1,3,4,5,6]，1只左移1格，有交换。
第2趟：[1,2,3,4,5,6]，1再左移1格，有交换。
第3趟：不改变数组，无交换，确认结束。
```

最多逆序构造为`[6,5,1,2,3,4]`：

```
第1趟：[5,1,2,3,4,6]，6一路右移，5次交换。
第2趟：[1,2,3,4,5,6]，5一路右移，4次交换。
第3趟：无交换。
```

总逆序分别 2 和 9。最大构造中，前 d 个大元素各自与后面 m−d 个小元素逆序，另有大元素彼此的$d(d-1)/2$个逆序，总数为$d(m-d) + d(d-1)/2 = md - d(d+1)/2$。这既核对了逐趟上界，也证明构造确实达到上界。

n=15 时，d=14，最少 14、最多 315。最少例为[2,3,...,15,1,16,17,...,30]；最多例为[30,29,...,17,1,2,...,16]。若采用另一计数习惯，把“n 趟”仅指有交换的扫描，便应取 d=n，得到最少 15、最多 330。不要把两种趟数口径混写；课件这份源码可以实际打印 sorted 来确认。

24.29 29 · 交换远处逆序：能多消一些，但距离不是充分条件

互异数 a>b 位于 i<j。交换后，它们这对逆序消失；中间每个值 x 若满足 b<x<a，又会同时消掉两对逆序。因此逆序减少量为 $1+2k$，k 是中间落在两值之间的元素数。

距离远意味着可能跨过更多合适的 x，却不保证一定多消。例如中间值全大于 a 或全小于 b，仍可能只减少 1。若有重复值，还要单独计与端点相等的情况，不能照搬奇数公式。

把三种中间元素各算一次，能看到公式的来源。端点$a > b$交换后：若$b < x < a$，原来有 a>x 和 x>b 两对，交换后都消失，贡献 2；若$x < b$，端点与 x 的逆序只是从 a 转到 b，净减少 0；若$x > a$同理为 0。端点本身的 a>b 再减少 1。若允许重复，$x = a$或$x = b$会各净减少 1，所以完整式为

$$\Delta I = 1 + 2\#\{x : b < x < a\} + \#\{x : x = a\} + \#\{x : x = b\},$$

其中只数两端位置之间的元素。交换端点外的某个元素与这两个端点的相对比较，总数不变，因此不用额外计入。

任意交换模型下，环分解给出 n-c 次就能排序，至多 n-1。选择排序保证 O(n)次交换，堆排序和快排一般不保证这个线性界；冒泡只用相邻交换，完全逆序需 n(n-1)/2 次。允许交换的位置不同，最少次数也不同。

24.30 30 · 不等长二路归并：仍以两边总长度计上界

两有序段长 m、n。比较两头，输出较小者并推进对应一边；每次至少消耗一项，直到一边空。关键字比较至多 m+n-1，输出和访问总量 O(m+n)，并不要求 m 与 n 接近。

例如[1,100]与[2,3,4,5,6]，先出 1，连续出 2 到 6，最后接 100，长的一侧并没有被反复从头扫描。若链表剩余尾段可整段接上，某些输入的真实工作还可更少。

线性上界仍成立；但不能因此说所有输入都必须逐节点复制尾段。要按接口是否维护 size、尾指针、统计信息，判断一次接链之外还有没有必须访问的工作。

24.31 31 · 节点转移归并：保存 next，稳定地从左边先取

遇到右段 x 小于左段 y，把 x 摘到 y 前；先存 x 在原段的后继，再改链接，后续扫描从保存位置继续。若直接用移动后的 x->succ，它已经指向 y，可能反复处理错误位置。

相等时让左段先出，保留来自原左半的记录在前，才能稳定。对左[2A,4]、右[2B,3]，输出应为 2A、2B、3、4。

将例子画成带身份的两条链：

```
L: H_L ↔ 2A ↔ 4C ↔ T_L
R: H_R ↔ 2B ↔ 3D ↔ T_R
```

比较 2A 与 2B，相等先保留左 2A，左指针到 4C；比较 4C 与 2B，先存右后继 3D，摘 2B 并接到 4C 前，左链成 2A↔2B↔4C；再比较 4C 与 3D，保存右后继 T_R，摘 3D 接到 4C 前，得 2A↔2B↔3D↔4C。右链为空，结束。A 仍在 B 之前，且 A、B、C、D 的节点地址都没变。若相等先从右取，就会变成 2B 在 2A 前，稳定性在第一次比较已经破坏。

remove 加新 insert 也能造出正确值序，但会分配、复制并改变身份。原节点转移的优势是复用资源；如果还有外部节点句柄，语义差别尤其需要写清。

24.32 32 · 向量与列表归并：同为 nlog n，代价落在不同地方

向量连续，扫描常有较好的局部性，按秩分半方便；常见稳定归并需要 O(n)元素缓冲，并多次复制或移动值。列表可以拆链、重接原节点，避免大对象复制，额外元素存储可以很少。

列表节点却可能散在内存，走 succ 会发生缓存失效，节点分配本身也占额外指针空间。找中点若每层扫描，仍能并入每层线性成本，整体 Θ(nlog n)，但常数不同。

比较时使用同一元素类型、相同数据与实际内存布局。只有少量大对象和很多小整数两种场景，得到不同胜负很正常；不能用渐近符号决定所有实测结论。

24.33 33 · 固定 15/16 与每次 n−1：看似都偏斜，结论却不同

按 2/3 与 1/3、15/16 与 1/16，或任意固定 $\lambda : (1-\lambda)$ 切分，两边仍以固定比例缩小，每层总长度 O(n)、深度 O(log n)，整体 Θ(nlog n)。

15/16 那条长路径深约 $\log_{16/15} n$，常数比对半大，却仍是对数。若改成 n-1 与 1，路径 n 层，递推为 $T(n) = T(n-1) + \Theta(n)$，累计二次。

把单项不断与已排序的大段合并，效果接近逐项插入排序。是否从前还是后剥离，决定具体移动方向；判断渐近时关键是每次只减少 1，而非减少一个固定比例。

24.34 34 · 归并数逆序：右项先出时，整批左项一起记

左[2,5,7]、右[1,4,6]。先出 1，左边剩 3 项都比 1 大，增加 3；出 2 不增；出 4 时左边剩 5、7，增加 2；出 5 不增；出 6 时剩 7，增加 1，总跨段逆序 6。

递归先数两半内部逆序，再加跨段逆序，每层只扫线性，总 O(nlog n)。相等时先取左项而不加逆序，因为逆序要求严格大于。总数最大约 $n^2/2$，累加器要有足够位宽。

把计数与输出状态同时记下：

左尚未输出	右尚未输出	本次输出	新增逆序	累计跨段逆序
2,5,7	1,4,6	1	3	3
2,5,7	4,6	2	0	3
5,7	4,6	4	2	5
5,7	6	5	0	5
7	6	6	1	6
7	空	7	0	6

“左边剩多少就加多少”成立，因为左半已经有序：右头小于左头，便小于左半所有剩余项。左头先出时，它不大于右头，也就不大于右半所有剩余项，不新添跨段逆序。两半内部逆序在递归中已各算一次，因此没有重计。

还可压缩秩后用树状数组：从右向左读，每次查询已有多少个更小秩，再插入当前秩，同样 O(nlog n)。若小整数值域允许更专门的操作，复杂度可变化；在一般比较模型里不应无条件宣称线性。

24.35 35 · 游标链表：数组下标充当地址

节点放进数组，每格保存 data、pred、succ，succ 是下一个节点的下标。另用一条 free 链管理空闲格。申请时从 free 头摘一格，释放时把该格接回 free 头。

例如有效链 2⟶5⟶1，物理下标既不递增也不连续。删除 5 只改 2.succ=1、1.pred=2，再回收 5；不需要把数组第 6 格以后整体前移。

把池的有效链接和空闲链接分开保存。例如有效 2→5→1→空，free 为 3→4→6→空。插入新项 X 到 2 之后：从 free 头取 3，free 改成 4；令格 3 的数据为 X，succ=5、pred=2，再改 2.succ=3、5.pred=3。删除 5：先让 3.succ=1、1.pred=3，然后把格 5 接到 free 头，即 5.succ=4、free=5。一个格只属于有效链或空闲链之一，这就是分配器必须维护的不变量。

它适合固定池、需要序列化引用或不便动态分配的场景。回收后的下标若被外部继续使用，可能指向后来新建的另一个对象；可以加代际号或限制句柄生命期。游标替代地址，不会自动解决悬空身份问题。

24.36 36 · Java：垃圾回收不会帮你接正确的链

interface 规定可调用能力，implements 表示类提供这些方法。Java 变量常持有对象引用，不像 C++某些写法那样直接复制整个节点对象；模板与泛型的具体机制也应按语言规则区分。

摘掉节点后，若没有任何可达引用，垃圾回收最终可释放它；若你仍无意保留一条引用，它可能继续存活。GC 不会自动把前驱接到正确后继，也不会修复环、错误 size 或逻辑重复。

对照课程版本时，把插入后前后遍历结果、size、外部引用指向对象身份逐项比较。运行测试只能报告实际运行的平台与版本，不能把“语言有 GC”当作算法正确性的证明。

24.37 37 · Python list：名字叫 list，底层却可能是数组

Python 内置 list 是动态数组，尾部追加或弹出通常有摊还常数表现，头部插删需要移动后缀；课程自己实现的链式 List 是另一类型。先确认 reverse.py 操作的是哪一种对象。

双指针原地反转只交换首尾，再向中间收缩，时间 O(n)、额外空间 O(1)。若递归版本每层切出剩余子串或子列表，再与首项拼接，每层复制长度 n、n-1……，就会出现二次时间和大量临时对象。

对`[A,B,C,D]`，双指针版只做`(0,3)`交换得到`[D,B,C,A]`，再做`(1,2)`得到`[D,C,B,A]`，两次交换。若写成`reverse(a[1:]) + a[:1]`，第一层先复制 BCD，下一层复制 CD，再下一层复制 D；返回时又拼接长度 2、3、4 的列表。切片与拼接并不因为写在一行里就成为常数，累计复制数量形成$1+2+\cdots+n$，所以会是二次量级。

同样是“递归反转”，若仅传左右下标、不切片，时间仍可线性，只增加递归栈。解释性能差距时要读具体切片、拼接和访问成本，不能把原因缩成“Python 慢”或“递归一定慢”。

24.38 本章练习与详细答案

1. 对[2,3,1]画出置换环；任选一次把元素放回最终位置的交换，环数怎样改变？
2. 交换[5,2,4,1,3]首尾的逆序 5 与 3，会消掉几个逆序？先用中间值判断，再枚举核对。
3. 为长度 6、含最后确认趟共 3 趟的提前终止冒泡，各构造最少和最多逆序的输入。

第一题：位置 1→2→3→1，一个三环。交换位置 1 与 3 得[1,3,2]，拆成固定点 1 和环(2,3)，环数增加 1。

第二题：端点之间仅 4 严格位于 3 与 5 之间，故减少 1+2=3；原逆序为 7 个，交换成[3,2,4,1,5]后剩 4 个。

第三题：d=2，最少[2,3,1,4,5,6]含 2 逆序；最多[6,5,1,2,3,4]含 9 逆序，分别两趟交换后再一趟确认。通过逐趟打印数组，可检查自己有没有漏数确认趟。

25 PS04：先把栈里的东西写出来，再谈递归、表达式和单调结构

教师原始习题单：第 04 次习题共 35 题。表达式题采用课件第 41—42 页的实际优先关系表来解释原程序；特别提醒：原表的 ^ 对 ^ 是先归约，因此原程序是左结合。数学惯例通常右结合，修改版要明确说明，不能把两者混成同一个答案。

25.1 1 · 向量做栈：把栈顶搬到头部，哪些成本跟着变

原 Vector 尾端作为 top，push 就是尾插，pop 就是尾删；没有扩容时只改末项，几何扩容下 push 摊还 O(1)，top 读末项 O(1)。

若把首端作为 top，每次 push 必须把旧 n 项右移一格，每次 pop 把剩余项左移，单次 Θ(n)。连续压入 n 项，移动量 0+1+…+(n-1)，总 $\Theta(n^2)$。top 仍是读首项，仍 O(1)。

栈的“后进先出”规则没有变，改变的是底层接口代价。实现一种 ADT 不只要输出顺序正确，还要检查承诺的时间复杂度。可以用[10,20,30]各压弹一次，画出每次真正移动的格子。

25.2 2 · 列表做栈：双向与单向不能混答

双向表有首尾哨兵和尾指针，两端插入、删除都能常数改链，交换 top 与 bottom 的角色不会改变渐近阶。节点分配和缓存常数仍可能不同。

单链若有尾指针，尾插可以 O(1)，但尾删还要找到前驱，一般 O(n)；首插、首删则 O(1)。所以“列表两端对称”需要双向链接或额外结构，不是所有叫 List 的类型都成立。

读示例项目时找 push、pop 实际调用哪个接口，再看那个接口需要多少次走链接。先确认表示法，才知道对称性是不是实质的。

25.3 3 · 迷宫里的粉笔：调用栈不会记住所有来过的地方

调用栈保存“当前还没做完的路径”。函数返回后那一帧就消失，所以以后从另一路走到同一图节点，栈未必知道它曾被处理。

要避免图中的重复或环，可以把 visited 放在节点字段、外部数组或哈希集合里。C++并没有自动防止递归环的内建机制；静态或全局变量能跨调用保存状态，但也可显式传入一个共享的访问表，并非只能靠全局变量。

例如 A→B→A，进入 A 就标记，再进入 B，看到 A 已在当前访问路径上便识别环。只在返回时标“处理完成”，却不区分正在访问的灰色状态，可能还没返回就再次递归 A。粉笔记录的状态含义需要明确。

25.4 4 · 每帧减半，理想规模为什么可能平方

设调用栈预算 B，每帧 s 字节，允许深度约 B/s。固定比例分治若深度 $h = c\log n$，在同一栈预算下帧减半，允许深度翻倍，故 $c\log n' \approx 2c\log n$，得到 $n' \approx n^2$。

减到十分之一则允许约 10 倍深度，理想上 $n' \approx n^{10}$。这不是“内存容量变成原来的平方”，只是对数深度与输入规模之间的指数换算。

若算法实际是线性递归 h≈n，则帧减半只能把规模约翻倍。若输入数组、输出和堆缓冲占用了主要空间，也不能按这个理想算式承诺真实机器能力；题目已明确暂忽略这些消耗。

25.5 5 · 递归都能消去，但未必能消掉需要保存的信息

普通递归可以用显式栈模拟：每个帧保存参数、尚需使用的局部量、返回后该接着执行哪一段。while 循环读取栈顶状态，遇到递归调用就压一个新帧，遇到返回就弹出并把结果交给父帧。

例如二叉树后序遍历，帧需说明“左边还没做”“左边已做、等右边”“左右都完成”；只保存节点地址而不保存阶段，就不知道返回后该去哪。

例如计算树 `A(B,D)`的后序，叶子 B、D 没有孩子。把帧写为`(节点,下次阶段)`，阶段 0 准备左、1 准备右、2 准备访问自己，完整栈从底到顶为：

```
[(A,0)]                          进入根
[(A,1),(B,0)]                    父帧先改成“等左返回后做右”，再压B
[(A,1),(B,1)]                    B没有左孩
[(A,1),(B,2)]                    B没有右孩
[(A,1)]，输出B                   弹B
[(A,2),(D,0)]                    A保存“等右后访问自己”，再压D
[(A,2),(D,1)] -> [(A,2),(D,2)]
[(A,2)]，输出D                   弹D
[]，输出A                        弹A
```

如果只在栈里存 A、B，B 返回以后看到 A 却不知道左子树已经完成，就可能反复压 B。阶段就是递归调用返回地址原本替你保存的信息。

这消除了语言层面的递归调用，却仍可能使用 O(h)辅助栈。一般程序需要记住哪些待办任务，是问题本身的信息需求，不会因为把函数改成 while 就自动变成 O(1)。

25.6 6 · 第二个递归是尾调用：可以省那条等待链

若某函数先递归处理左半，左半返回后直接返回右半的递归结果，右半调用是真尾位置，可以把当前帧参数改成右半并跳回循环起点，第一支仍正常递归。

这能减少“父帧等第二支结束”的堆叠，但第一支的深度仍存在。对快排一类过程，常见办法是递归较小一侧、循环处理较大一侧，可控制同时活跃的递归深度；需要证明每次递归那侧至多减半。

以“访问根、处理左子树、处理右子树”的先序为例，右递归在最后，可改为：

```
preorder(v):
    while v不是空:
        visit(v)
        preorder(v.left)
        v = v.right
```

树 `A(B(D,E),C)`的执行是：外层访问 A，递归进入 B；B 访问后递归 D，D 访问结束，B 的循环改成 E 并访问；B 整个调用返回后，A 的循环改成 C，访问 C。输出 A、B、D、E、C。右边的一条长链现在由循环走，不会一层层保留旧帧；左边的递归仍会保留父节点，最坏左链仍线性深。

如果右递归返回后还要合并左右结果，它就不是尾位置，不能直接覆盖父帧；必须保留左结果和后续合并任务。看源码中的“调用以后还有没有实际需要的工作”，而非只看它是不是第二个调用。

25.7 7 · 阶乘：把待完成乘法装进一个累加器

普通阶乘 `fac(n)=n*fac(n-1)` 返回后还要乘 n。引入 acc，保持不变量“acc 乘上尚未处理的 1 至 n，等于原目标阶乘”，便能迭代：

```
def factorial(n):
    if n < 0:
        raise ValueError("阶乘输入须为非负整数")
    acc = 1
    while n > 1:
        acc *= n
        n -= 1
    return acc
```

n=4 时，(acc,n)依次(1,4)→(4,3)→(12,2)→(24,1)。固定宽度模型下只用常数个变量；若使用任意精度整数，保存 n!本身需要的位数仍在增长，不能叫实际字节 O(1)。

推广要检查剩余操作能否以固定大小状态组合，顺序能否改变、副作用是否允许。浮点乘法重排可能改舍入，打印输出的顺序也不能随意交换。“递归后只剩一行”不等于可以自动塞进一个标量。

25.8 8 · 头递归：参数推进也算计算

进制转换先调用 `convert(n/base)`，返回后输出 n%base。它的主要输出在递归之后，所以常被归为头递归；但实参中的除法已经使问题缩小，并不是调用之前完全没有计算。

如果所谓“纯头递归”是状态与参数完全不变，就再次调用同一个实例，那么确实不可能靠自己接近基例。例如 `f(n){if(n==0)return; f(n);}` 对 n>0 不断压栈。若参数改成 n-1，便已经有了推进，不能套这条断言。

去递归时，只需保存将来输出所需的余数，而不必保存编译器帧中的所有机器细节。例如十进制 13 转二进制，依次保存余数 1、0、1、1，反向弹出 1101。保存什么，取决于返回后还要用什么。

25.9 9 · 单链析构：后序删与顺序删都能线性

头递归可先销毁后继，再 delete 当前；它保证每个节点只删一次，但长 n 的链会同时压 n 层帧。调用前必须先判断 p 是否为空或尾哨兵，实参 p->succ 的读取本身也是工作，不属于“毫无状态推进”。

迭代版先保存 next，再释放当前，再 p=next。它不用返回旧节点处理剩余事，所以可常数辅助空间，时间仍 Θ(n)。释放前保存后继，释放后绝不能再从 p 读链接。

如果数据元素的析构必须遵循某种业务次序，头到尾与尾到头可能有语义差别；普通独立节点可选迭代，但不能把所有带副作用的析构顺序视为无关。

25.10 10 · 自定义括号：先分清字符串，再谈栈

词法阶段把输入识别成左记号、右记号与普通内容，栈保存(类型,起始位置)。遇左记号压栈；遇右记号，若栈空就报告多余关闭，类型不同就报告当前位置与期待的关闭类型；输入结束还剩左记号则报告未关闭的起点。

例如 `<a>[x)</a>`，读到右圆括号时，栈顶是方括号起点，应说明“这里期待]，它与位置 4 的[对应”。这比单纯返回 false 更能帮助修改。

固定一个只支持`()`、`[]`的简单语言，输入`([x)]`，用从 0 开始的位置：

位置	字符	动作	栈（底→顶）
0	(	压入类型与位置	`[(圆,0)]`
1	[	压入类型与位置	`[(圆,0),(方,1)]`
2	x	普通内容，不动栈	同上
3	)	与栈顶方括号冲突，停止	报错位置 3，期待]以匹配位置 1

不能在失配时先把栈弹掉再继续假装合法。若输入是`([]`，扫描结束后剩`(圆,0)`，错误应报告为“位置 0 的左圆括号尚未关闭”，与中间遇错右括号是不同情况。

JSON 字符串中的 `"["` 是内容，不是结构括号，反斜杠转义也影响引号结束位置。HTML 还涉及空元素、属性、注释与复杂语法，简单标签栈不等于完整 HTML 解析器。扩展算法要先说支持哪一种受限语言，不能只按字符匹配就宣称解析所有格式。

25.11 11 · 加法结合方向：表中同类相遇的那一格决定顺序

课件 + 对 + 为大于，意味着栈顶先归约。2+3+4 读到第二个+时，先算 2+3=5，再把+压入等待 4，得到(2+3)+4。要改成 2+(3+4)，把这格改为小于，让新+先入栈，之后从右边归约。对只含合法加法的表达式，主循环不需换框架；若同时有减法，则必须检查相同优先级的交叉格，不能只改一个角落却声称整个语法都一致。

先看完整教师原表，横向是“刚读到的运算符”，纵向是“栈顶等待的运算符”。>表示先处理栈顶，当前输入先不前进；<表示压入当前符号并前进；匹配格表示消去相应括号或结束哨兵。

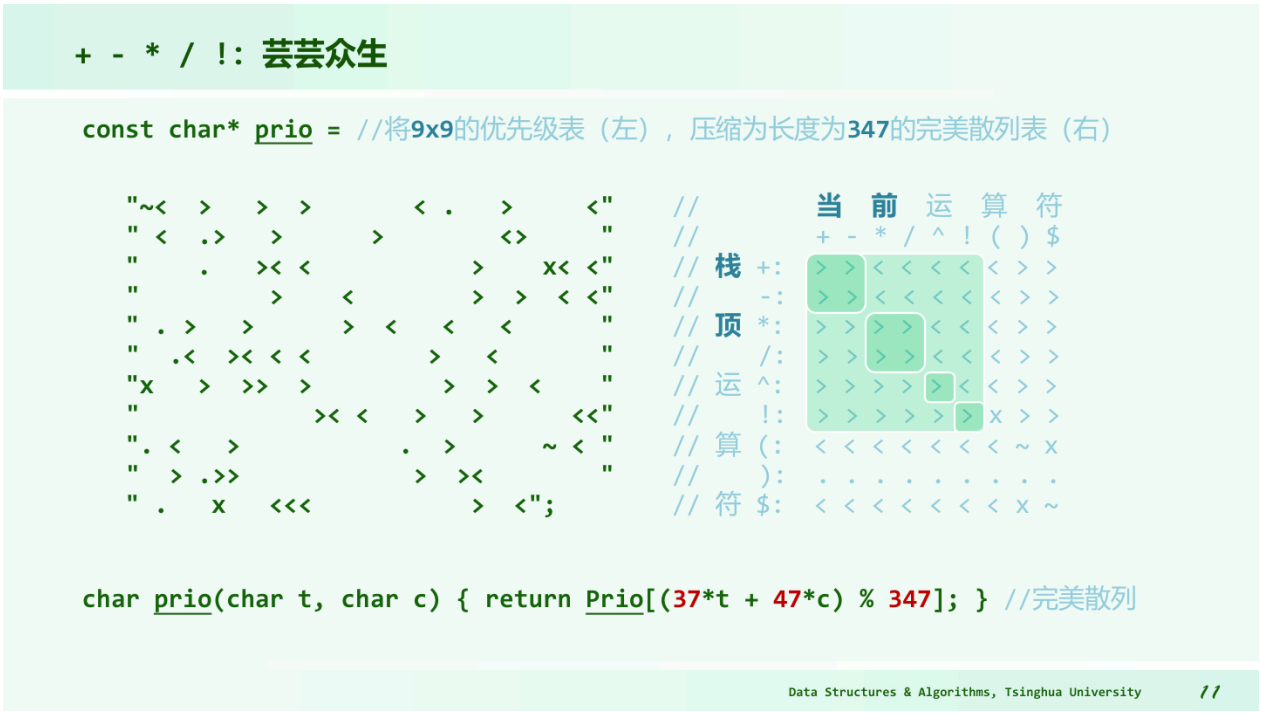


图 205　课程《04.栈与队列》分章 PDF 第 42 页完整原图（幻灯片页脚 11）。例如栈顶乘号、当前加号应读第*行第+列，得到先归约；乘方对乘方也是先归约。

对 2+3*4，让 # 在本章表示原图的结束哨兵，逐步求值如下。每行只做一个动作，同一个当前符号可能被连续看多次：

当前输入	查表或动作	运算符栈	数值栈
2	数字入数值栈	#	[2]
+	# < +，压+	# +	[2]
3	数字入栈	# +	[2,3]
*	+ < *，压*	# + *	[2,3]
4	数字入栈	# + *	[2,3,4]
结束 #	* > #，弹*并算 3×4	# +	[2,12]
仍为 #	+ > #，弹+并算 2+12	#	[14]
仍为 #	哨兵匹配，结束	空	[14]

最容易漏的是后 3 行：归约只弹栈顶，没有消费当前输入。若每归约一次也把输入指针前移，最后的加法可能根本没机会执行。

精确整数无溢出时两者同值，浮点却不保证结合律。时间仍线性，但连续右结合运算符会积在栈里，某些输入的峰值空间可由常数增加到线性。只说“时间空间都没变”会漏掉这一输入族上的变化。

25.12 12 · 原程序的乘方是左结合，修改后才符合通常数学惯例

原表 ^ 对 ^ 为大于，因此 2^3^4 先算 2^3=8，再算 8^4=4096。通常数学写法解释为 $2^{(3^4)} = 2^{81}$，两者差很多。必须先报告原表实际做什么，再说要怎样修改。

用较小的 2^3^2 核对两个版本，不用算很大的数：

时刻	原表左结合：符号栈/数值栈	修改后右结合：符号栈/数值栈
刚读完 2、第一乘方、3	# ^ / [2,3]	相同
遇第二乘方	先弹旧乘方算 8，当前仍未消费	直接压新乘方，变 # ^ ^ / [2,3]

时刻	原表左结合：符号栈/数值栈	修改后右结合：符号栈/数值栈
消费第二乘方并读 2	`# ^ / [8,2]`	`# ^ ^ / [2,3,2]`
输入结束	8 的平方=64	先 3 的平方=9，再 2 的 9 次方=512

两个版本都遵守“先弹右数、再弹左数”，差别出在何时决定旧乘方已经拥有完整的右子表达式。

把这格改为小于，连续^先入栈，最后从右向左归约。读取、弹出和计算框架不必改变，但测试应同时包含括号、阶乘与多个乘方；C++里的字符^本身是按位异或，这里只是自定义表达式语言的乘方 token。

总时间依然 O(N)，最坏总空间依然 O(N)；但无括号纯乘方链原先可即时归约，改右结合后会积压 O(N)个运算符。复杂度类别与具体输入上的峰值，应分别说明。

25.13 13 · 右括号那一行为什么全空

当前右括号遇到左括号时只匹配并消去，不作为待执行运算符长期压栈。因此“栈顶已经是右括号，再看下一个 token”的状态不应出现，表中对应整行为空。

另一些空格表示语法不合法，例如没有左括号却遇到右括号，或输入结束还有未配对左括号。它们不是“优先级差不多，随便处理”，而是应报告的错误。

区分三种表项：小于表示移进，大于表示归约，匹配标记表示相消；空白表示不允许。若程序默认输入合法，可以断言；面向真实输入的解析器应返回位置和原因。

25.14 14 · 双阶乘：6!!不是(6!)!

定义 $n!! = n(n-2)(n-4)\cdots$，到 1 或 2 停止，并约定 0!!=1。6!!=6×4×2=48；(6!)!完全是另一个大得多的数。

间接方案把一个新 token 映射成一元后缀双阶乘，计算器按步长 2 累乘。它与阶乘一样只弹一个操作数，必须检查非负整数定义域和溢出。直接读!!则需词法分析的最长匹配：先看下一个字符是否也是!，一次识别双阶乘。

若语法还允许两个连续普通阶乘，必须额外规定用括号如 `(6!)!` 表示，否则同一字符流有歧义。3!!!也要说明是 3!!后接!，还是别的扩展；不能依靠“人看着懂”让程序猜。

25.15 15 · 操作数顺序：先弹右边，后弹左边

读完 3、8 后，栈底 3、栈顶 8。遇减号先弹 b=8，再弹 a=3，算 a-b=-5；如果反过来做 b-a，就变 5。加法乘法的交换律会掩盖这个错误，所以测试要包含减、除和乘方。

一元阶乘只需一项；二元运算需两项；栈不足必须在执行前报告。除数为 0 也要先判断，不应偷偷压 0 继续算，因为后面看似合法的结果会隐藏真实失败。

整数截断、精确有理数、浮点是三种数值语义。第 30 章的表达式核对采用精确有理数；若课程 double 求值，最后小数应允许相应舍入，不把两种执行结果机械按字符串判等。

25.16 16 · 运算符种类有限，栈为什么仍能任意深

相邻待处理符号能同时入栈，是因为前一符号相对后一符号为移进关系。这个表关系不传递：+对(为小于，(对+也为小于，但+对+为大于。因此不能说“每层严格升高，符号只有九种，所以最多九项”。

构造很多尚未闭合左括号，栈就能线性深；运算数栈也能用嵌套的 `1+(1+(1+...))` 积累待算左值。输入结束前某个时刻的峰值需要计入，不能只看最终剩一项。

若问原优先表下“达到峰值 n 最少多少对括号”，计入底部结束哨兵时，一层括号内最多可暂留+、*、^、！四个不同优先级运算符；!后不能直接接(，所以每增加一个嵌套层最多增加四个待留项。无括号最多 5 项（含哨兵），故下界是 $\max(0, \lceil (n-5)/4 \rceil)$ 对，利用 `0+0*0^(0+0*0^0!)` 这类递归嵌套可达到相应峰值，其余 n 用较短运算链补齐。

这里把“补齐”实际做一遍。含底部哨兵时，下列四个式子都只用一对括号：

表达式	读到最内层!、尚未归约时的运算符栈（底→顶）	峰值
`(0+0*0^0!)`	`# ( + * ^ !`	6

表达式	读到最内层!、尚未归约时的运算符栈（底→顶）	峰值
`0+(0+0*0^0!)`	`# + ( + * ^ !`	7
`0+0*(0+0*0^0!)`	`# + * ( + * ^ !`	8
`0+0*0^(0+0*0^0!)`	`# + * ^ ( + * ^ !`	9

每加一个完整外层，外层可留下+、*、^三个运算符，再加左括号，共增加 4 项。!是后缀符号，原表不允许!后直接移进左括号，不能让每层都多留一个!再继续往里嵌套。对更深的目标，重复完整外层后，在最外层选上表对应的短前缀，就能补齐余数；例如峰 10 可在峰 9 的式子外再包一对括号。

若不计底部哨兵，最大峰变成 4P+4，最少括号相应为 $\max(0, \lceil (n-4)/4 \rceil)$；若先把^改成右结合，纯乘方链就能不加括号积到任意深，刚才的最少括号式不再适用。回答本题必须带上表版本与栈计数口径。

25.17 17 · 转 RPN：每个运算符都应等自己的孩子先输出

把表达式看成树，数字是叶，运算符是内部节点。后序遍历先输出左子树、右子树，再输出父运算符，就是 RPN。按树规模归纳：两个更小子表达式各自输出正确，接上运算符，就恰好表示它们的组合。

把 `2*(3+4)`画成树，两个孩子的顺序也标清：

```
      *
     / \
    2   +
       / \
      3   4
```

树的后序是：访问叶 2；进入右子树，访问 3、4 后访问+；最后访问*，所以 RPN 为 `2 3 4 + *`。归纳命题必须说“输出恰好对应这棵子树的完整 RPN”，而不能只说“运算符在数字后面”：后者不足以保证左右孩子归属正确。

在线栈算法还没看到右操作数完整范围时，先保存运算符；只有后续 token 说明该子表达式已闭合，才归约输出它。因此数字顺序不变，运算符输出顺序遵从相同的树依赖。

例 *2(3+4)*：读 *2* 先输出，等待；括号内 3、4 先输出，再输出+；括号闭合后最后输出*，得到 `2 3 4 + *`。手工画树是整体分析，栈法是逐 token 保存尚未确定的工作，二者得到同一后序。

25.18 18 · 转换也要线性，RPN 还有什么价值

只计算一次时，中缀解析加 RPN 求值仍是线性，并没有把 O(N)降到更低阶。它的收益是将“理解括号与优先级”同“执行运算”拆开。

如果一个带变量表达式需要对一万组输入求值，语法只解析一次，后面反复执行简单指令序列；虚拟机也可以让每条指令注明弹入几个值。错误位置和类型检查可在转换阶段先完成。

这是一种可复用表示，而非复杂度魔法。若表达式只用一次、程序很小，直接求值也可能更简单；应根据任务选择，不必把 RPN 描绘成所有情形都更快。

25.19 19 · 21 个长表达式：用短片段先确定读法

完整 21 式的 RPN、数值与逐步栈追踪在第 30 章。这里先拆第一式：括号内 0+1+2-3+4+5+6 按左结合依次输出 `0 1 + 2 + 3 - 4 + 5 + 6 +`；另一个括号输出 `7 8 +`；两段相乘后输出，再读 *9*，最后输出。

合起来是 `0 1 + 2 + 3 - 4 + 5 + 6 + 7 8 + * 9 *`。数值第一括号 15，第二括号 15，所以 15×15×9=2025。转换与求值要分别核对：一个是 token 顺序，一个是算术结果。

含!!的式子不能逐字符当两个!；含连续减除的式子要按左结合；第 30 章使用同一套原表约定；这 21 式也没有无括号连续^造成的冲突。若另加 `2^3^4` 作为测试，必须标明使用原表还是修改表。

25.20 20 · 前缀 PN：从右往左读也能用栈

前缀 `- 8 * 2 3` 表示 8 减 2×3。倒序扫描 3、2 入栈，遇*弹出的第一个是左操作数 2、第二个是右操作数 3，压 6；再读 8 入栈，遇-弹 8 与 6，压 2。

这和 RPN 从左读时“先弹右操作数”的顺序恰好不同，因为扫描方向变了。固定元数让程序知道每个运算符要几个孩子；单目、双目应在 token 定义里标明。

每个 token 进出常数次，时间 O(N)，最坏栈 O(N)。也可递归按“运算符后接几个子表达式”解析，仍需检查 token 耗尽、操作数不足和最后剩余多份结果。

25.21 21 · 禁形：晚读入的大数，把中间数压在了小数上面

统一按输入 1、2、…、n 依次压入，输出按实际弹出顺序记录。设 i<j<k，目标输出却含 k...i...j。为了先弹 k，必须已经把 i、j、k 按输入顺序读入；若 j 尚未弹出，它就在 i 上面，因此 i 不能先于 j 弹出，矛盾。特例 j+1...i...j 只是令 k=j+1，机制相同。它们说的是三个元素的相对次序，不要求三个在输出中紧挨着。例如想输出 3,1,2：压 1、2、3，弹 3 以后栈顶是 2，要得到 1 必须先弹 2，却又要求 2 在 1 后面，卡住。输出 3,2,1 则可行。若教材图把最终输出画成另一只栈，视觉阅读方向可能反着，需要先换成实际弹出顺序，再使用禁形名称。

25.22 22 · 依最后或最先输出分类：两种视角不必有同样简单的公式

设最后输出 k。所有小于 k 的元素必须在 k 入栈前弹完，否则被 k 压住以后就不可能赶在最后 k 之前弹出。大于 k 的元素在 k 上方完成另一套独立栈过程，因此输出分为“小于 k 的混洗、大于 k 的混洗、最后 k”。两块分别有 C_{k-1}, C_{n-k} 种，分类互斥且穷尽，组合可逆，于是

$$C_n = \sum_{k=1}^{n} C_{k-1} C_{n-k}, \qquad C_0 = 1.$$

用$n = 3$列出按最后一项的三组：最后 1 是 231、321（前两项都属于大于 1 的子问题），最后 2 只有 132（小于 2 的 1 必须先于大于 2 的 3 完成），最后 3 是 123、213。组数为 2、1、2，恰是C_0C_2、C_1C_1、C_2C_0。

卷积的可逆性也需要说清：任取左子问题的合法操作，再压入并暂存 k，执行大于 k 的那部分，最后弹 k，得到且只得到这一组输出。不同 k 由最后项区分，不会重复计数。

按第一个输出 k 分类时，要先压 1 至 k 再弹 k，剩余栈有 k-1 项，还有 n-k 个输入没读。后续不是两块简单相接，应该数从高度 k-1 回到 0、始终非负的入出栈路径。

令 u=n-k、h=k-1，可用完成数 F(u,h)递推：允许再压入时加 F(u-1,h+1)，栈非空允许弹时加 F(u,h-1)，u=0 时只能依次弹完，完成数为 1。也可由反射法得

$$F(n-k, k-1) = \binom{2n-k-1}{n-k} - \binom{2n-k-1}{n-k-1},$$

差式里的两项分别在数什么？首弹 k 后，初始高度$h = k-1$，剩$u = n-k$次入栈、$u+h$次出栈，总步数$L = 2u+h$。不管会不会弹空，任意排列这些步共有$\binom{L}{u}$种。对会从高度 0 走到−1 的坏路径，保留它首次到−1 的前缀，把这之后的所有上下动作互换。原来后缀从−1 回到 0，反射后改为从−1 到−2，整条新路径比原来少 1 个上步、多 1 个下步，故只有$u-1$个上步。反过来，任何从高度h结束在−2 的这种步串都必经过−1，在首次到−1 后反射即可恢复原路径。所以坏数为$\binom{L}{u-1}$。两者相减就是合法完成数。这个反射计数需要保留初始高度，不能默认每次从空栈开始。

越界二项式视为 0。n=3 时按首项 1、2、3 分别为 2、2、1，总 5。不能为了得到熟悉的 Catalan 卷积，硬把这次分类也拆成并不存在的两块。

25.23 23 · CSTDSA：两个 S 不能直接让 Catalan 数除以 2

先给两个 S 编号，输入顺序 C、Sa、T、D、Sb、A。最后弹 Sa 时，C 不可能留在 Sa 下面等最后才出，所以 C 必须先出；中间 T、D、Sb、A 可独立混洗，故这类有 $C_4 = 14$ 种。

最后弹 Sb 时，A 在它上面，必须紧挨末尾先弹 A；更早四项已经完成，所以这类也是 14 种。再要求以 C 开头、末两项为 A 和 Sa 或 Sb，分别留下 3 项自由混洗，各 $C_3 = 5$ 种。

把原题 e、f 两类各 5 个可见串完整列出来，交集就不必靠猜：

以 C 开头、以 A Sa 结尾	以 C 开头、以 A Sb 结尾
CDSTAS（左类独有）	CSTDAS（右类独有）

以 C 开头、以 A Sa 结尾	以 C 开头、以 A Sb 结尾
CDTSAS	CDTSAS
CSDTAS	CSDTAS
CTDSAS	CTDSAS
CTSDAS	CTSDAS

每列内部的 S 仍分别来自有标记的合法操作，只是此表省去标记显示。四个相同字符串可能由不同 S 身份的路径产生，计“可见串”时应合并；计“有标记路径”时不能合并。

去掉Sa、Sb身份，只显示S后，这两组会有重叠。交集的可见输出是CDTSAS、CSDTAS、CTDSAS、CTSDAS，共 4 种；于是末项为 S 的不同可见字符串为 14+14-4=24。两个特定 5 元素集合的对称差有 2 种，交集 4 种；各自独有 1 种。

可以用递归枚举全部 132 条六元素合法栈操作，再分别放入集合核对。去重的作用发生在输出字符串层面，不是在操作路径层面。因为重复字符的两种标号不一定都对应合法过程，直接 132 除 2 没有依据。

25.24 24 · 栈混洗判定：当前目标不在顶上，只有一个可能的补救

按目标输出逐项处理。如果栈顶就是目标，立刻弹；否则继续从输入压入，直到目标成为栈顶。若所有输入已压完仍不匹配，之后没有新动作能把埋在下面的目标提前取出，判失败。

贪心正确性来自动作被迫：想输出当前目标，栈顶若是其他值，不能把它先弹到输出；唯一可尝试的是继续压尚未读入的值。如果目标仍在输入里，压到它会成功；如果它已被压在错误的上层元素下面，过程就会卡住。

成功与失败各走一次，输入都为 1、2、3：

目标输出	当前要的数	连续压入	弹出后栈	结果
2,1,3	2	压 1、2	[1]	输出 2
同上	1	无	[]	输出 1
同上	3	压 3	[]	输出 3，成功
3,1,2	3	压 1、2、3	[1,2]	输出 3
同上	1	输入已耗尽	顶仍是 2	不能先弹 2，失败

失败返回当前尚未匹配的输出下标，故小于 n；全部匹配才返回 n。若目标含重复或漏项，不再是“互异输入的一个排列”，应先验证输入规格或让模拟按相同定义拒绝。

虽然外 for 最多 n 轮、某次 while 也可能压 n 项，但每个输入只压一次，每个输出只弹一次，内循环跨所有外层的总次数最多 n，整体 O(n)。二重循环不能不看变量变化就相乘。

25.25 25 · 括号与混洗：对应的是合法操作路径

入栈写(，出栈写)。任何前缀里出栈次数不能超过入栈次数，所以左括号数不少于右括号数；结束时各 n 个。这恰好是 n 对合法括号串的条件。

反向读一个合法括号串，遇(就压下一个输入，遇)就弹栈，非负前缀保证不会从空栈弹出。这个过程唯一，因此合法括号串与合法入出栈操作路径一一对应。

对互异且输入顺序固定的元素，可行输出也唯一决定其被迫的入出过程，于是输出混洗数等于 Catalan 数。两个字符相同后，可见输出可能由多条路径得到，刚才的 CSTDSA 正说明此时不能再把输出字符串与括号串一一对应。

25.26 26 · 枚举、第 k 个、均匀抽样：先知道每条分支下面有多少串

设已用 o 个左括号、c 个右括号。o<n 时可加左括号；c<o 时可加右括号。完成数 A(o,c)满足这两个合法后继的和，终点 A(n,n)=1，其余非法状态为 0。用表或记忆化计算，共 $O(n^2)$个状态。

按约定的字典序“(在)前”深度优先枚举，n=3 依次得到((()))、(()())、(())()、()(())、()()()。取从 1 编号的第 k 个时，先看左分支完成数 L：k≤L 就走左，否则 k 减 L 再走右。例如第 4 个跳过左分支的前三个，进入以()开头的部分，最终为()(())。

“第 4 个”的决策不要跳过。n=3 时第一字符只能(；到状态$(o,c)=(1,0)$，再写(的分支共有 3 个完成串((()))、(()())、((()))，写)的分支有 2 个。k=4 先跳过前 3 个，改 k=1，走到前缀()；接着只能写(，在前缀()(之后先试(，有 1 个完成串()(())，k=1 命中。每一步都减的是整棵尚未进入的分支的完成数，不是每跳一个符号就减 1。

完成数表也可手填 n=3 的几个格：$A(3,3)=1$，$A(3,2)=1$，$A(2,2)=A(3,2)=1$，$A(3,1)=1$，$A(2,1)=A(3,1)+A(2,2)=2$，$A(2,0)=A(3,0)+A(2,1)=1+2=3$，$A(1,0)=3+2=5$。状态只允许$c\le o\le n$，非法格为 0。

均匀随机抽样也按完成数分支：在总数 L+R 里均匀取整数，小于 L 选左，否则选右。每一整串的路径概率相乘后恰为 1/Cn。若每次在可选的左右之间直接各取一半，不同串经过的自由选择次数不同，概率会不等；n=3 的((()))可有 1/4，而(()())仅 1/8。

大数需要大整数计数和无偏随机整数，不能用溢出的机器整数或有偏取模。这里给出一套可验证的动态规划方案，解决原题三项任务；它不是把 Knuth 所列 P/U/W 算法的原始实现逐行复刻，阅读原书时还需比较其不同优化目标。

25.27 27 · 循环向量队列：出队只移动头，不搬整队

保存 head 和 size，容量 C>0 时，第 i 个逻辑元素位于 `(head+i)%C`。入队写到 `(head+size)%C` 后 size++；出队读 head，再令 head=(head+1)%C 并 size−。

容量 4，依次入 A、B、C，出 A、B，再入 D、E，物理数组可能是[E,旧 B,C,D]，逻辑顺序却是 C、D、E。旧格中留下的字节不等于仍有有效元素，size 决定有效范围。

把本例的每次物理写也列出来，_表示本来为空，带旧字样的内容已经失效：

```
初态      [_,_,_,_]      head=0,size=0
入A       [A,_,_,_]      head=0,size=1
入B       [A,B,_,_]      head=0,size=2
入C       [A,B,C,_]      head=0,size=3
出A       [旧A,B,C,_]    head=1,size=2
出B       [旧A,旧B,C,_]  head=2,size=1
入D       [旧A,旧B,C,D]  写(2+1)%4=3, size=2
入E       [E,旧B,C,D]    写(2+2)%4=0, size=3
```

逻辑第 0、1、2 项的物理下标分别为 2、3、0。若这时扩到 8 格，应依次复制 C、D、E 到新下标 0、1、2，再令 head=0；原数组从物理 0 直接往后复制，会把逻辑队首误改成 E。

扩容时按逻辑顺序把两段复制到新数组 0 起点，head 重置 0。记录 size 可区分 head==tail 时究竟空还是满；也可以保留一个空位，但容量含义改变。容量 0、恰好满、绕回和扩容后顺序都要测。

25.28 28 · FIFO 应用：按到达次序处理的真实任务

BFS 把刚发现的节点加入队尾，从队首取下一个处理，因而先处理距离近的一层。打印任务、生产者消费者缓冲、离散事件里同一优先级的等待队列，也常使用先进先出。

但“最早到达”与“最紧急”不同。需要先处理最高优先级时应使用优先队列；需要先处理时间戳最小事件时，时间顺序不等于消息到达顺序，也可能要堆。

设计应用例子时写清入队事件、出队事件、满时如何处理。只报“操作系统用队列”没有说明它在保存哪种待办任务。

25.29 29 · 双栈当队：三种证明的常数为何不一定一样

in 接收入队，out 负责出队；只有 out 空时才把 in 逐项弹出并压进 out。一个元素最多入 in 一次、从 in 弹出一次、入 out 一次、最后从 out 弹一次，总共常数个栈动作，所以任意 m 次合法队列操作总 O(m)。

势能取 $\Phi = 2|in|$：入队实际 1、势能增 2，摊还 3；out 不空出队实际 1、势能不变，摊还 1；倒 k 项再出队实际 2k+1，势能减 2k，摊还仍 1。

对“入 A、入 B、出、入 C、出、出”，两只栈从底到顶的实际状态为：

操作	in	out	栈动作数	返回
入 A	`[A]`	`[]`	1	—
入 B	`[A,B]`	`[]`	1	—
出	`[]`	`[B]`	5：倒 2 项各 2 次，再弹 1 次	A
入 C	`[C]`	`[B]`	1	—
出	`[C]`	`[]`	1	B
出	`[]`	`[]`	3：倒 1 项再弹	C

第五步 out 还有 B，必须直接弹 B，不能因为 in 里有新 C 就再次倒入 out，否则新元素会越过旧元素。势能依次为 0、2、4、0、2、2、0，实际总 12，摊还成本 3、3、1、3、1、1 也合计 12。

记账法若每次入队收更多费用，会得一个更松的常数；聚合法也可能把尚未出队元素按全流程预收。只要是上界，它们不必精确相等。比较常数前先统一 push/pop、空检查、返回值复制是否计费，而非看到 2 与 3 就认定证明互相矛盾。

25.30 30 · 最大直方图矩形：面积相同，左边界也要比

每个柱宽 1、高非负。矩形覆盖半开区间[l,r)，高度不能超过区间最矮柱，面积为 $(r-l)\min h$。逐根柱子向两边扫描直到更矮处，单调输入可使总扫描 $\Theta(n^2)$；枚举所有区间又每次重扫最小值的另一种朴素写法甚至 $O(n^3)$，不要混为同一算法。

“最靠左”应明确定义为同面积时 l 最小；若 l 也相同，再规定 r 最小或最大，本章取 r 最小。更新答案时比较(-area,l,r)的字典序，或写等价的逐项条件。

例如[2,0,2]，左右两个单柱面积都为 2，应选[0,1)。有的栈实现先弹右边候选，仅用面积严格大于更新就会留错；有的扫描顺序恰好先见左侧，又碰巧符合，不能把偶然顺序当证明。核对课件 Excel 时，可以输入同一个`[2,0,2]`，记录它最后返回的左端是否为 0；只看面积 2 不能区分左右两个候选。

25.31 31 · 单趟比两趟多做了什么即时工作

两趟方案先算每根柱左侧最近更矮位置，再反向算右侧边界，最后统一算面积。它便于把所有边界画在表上，但需要存储相应数组或中间状态。

单趟保持递增栈，当前更矮柱一到，就确定被弹柱的右边界，立即结算面积。对[2,1,5,6,2,3]，读到后面的 2 时，先弹 6 得面积 6，再弹 5 得宽 2、面积 10，不必等整份右边界数组建完。

这个例子的单趟完整过程如下，栈保存柱下标，只在新高度严格更小时弹出，末尾补高度 0：

当前下标 i/高度	本轮弹出及面积	压入后栈（下标:高度）
0/2	无	`0:2`
1/1	弹 0，左边空记−1，宽 1，面积 2	`1:1`
2/5	无	`1:1,2:5`
3/6	无	`1:1,2:5,3:6`
4/2	弹 3，左 2，宽 1，面积 6；弹 2，左 1，宽 2，面积 10	`1:1,4:2`
5/3	无	`1:1,4:2,5:3`
6/0	弹 5 得 3；弹 4 宽 4 得 8；弹 1 宽 6 得 6	`6:0`

弹出柱 p 时，右边界就是当前 i，不包括 i；左边界为弹后栈顶 left 的后一格。覆盖`[left+1,i)`，宽度为 $i-left-1$。等高柱在本版本中暂时保留，left 可能等高而非严格更矮；较右等高柱先结算窄矩形，较左等高柱以后获得完整宽度。例如`[2,2,0]`先结算右柱面积 2，再结算左柱面积 4，最大值仍正确。

单趟可流式前进、较早给出已结算候选，但仍可能把一整段递增输入存进栈。流式不等于常数空间，也不表示未来不可能出现更大的全局答案。

25.32 32 · 单趟线性：一次 while 很长，不代表每根柱都很长

聚合法：每根真实柱恰入栈一次，最多出栈一次，所以总 push≤n、总 pop≤n；外扫描 n 步，合计 O(n)。结尾追加高度 0 只负责把未结算部分清出，还需明确相等高度的保留规则。

记账法：每次入栈收两份，一份支付本次 push，一份存给未来 pop；从未弹出的柱保留信用，也不影响上界。每根柱收费常数，所以总线性。

势能法取栈大小。一步弹 k 根再压当前根，真实栈成本 k+1，势能变化 1-k，摊还为 2；再加常数比较等控制成本，仍常数。若把一次弹栈的比较也逐项收费，改变固定系数即可。这个证明完全不需要随机高度。

25.33 33 · 单时刻平均与全程峰值：不能把期望搬进 max

严格递增输入会让 n 根柱同时留在栈，最坏 Θ(n)；严格递减时每次弹掉旧项，最好峰值 O(1)。整个运行空间是 $M = \max_{1\le r\le n} S_r$，其中 Sr 是第 r 根真实柱处理后的栈长，最后清空之前的状态也必须算。

假设高度互异、相对顺序为均匀随机排列。栈恰保存此前缀从右往左看到的记录最小值；一个后缀长度 j，其中最左项成为最小值的概率 1/j，所以 $E[S_r] = H_r$。

r=3 时可以不带公式列全。6 种相对排列处理完后，栈内后缀最小记录分别为：

输入前缀	留下的高度	S_3
1,2,3	1,2,3	3
1,3,2	1,2	2
2,1,3	1,3	2
2,3,1	1	1
3,1,2	1,2	2
3,2,1	1	1

平均$11/6 = 1 + 1/2 + 1/3$。同一条输入 2,3,1 的前缀栈长却是 1、2、1，峰值 2 而最终只有 1，说明“最后栈长”和“运行空间峰值”连在同一个样本上都可能不同。

要证明峰值期望也为 Θ(log n)，还需控制“某一步偶然特别大”的概率。先补上乘积公式所需的独立性：从右向左读一个长 r 的均匀随机排列，把第 j 个读到的数在此前 j 个数中的名次记为 R_j，最小者名次为 1。每个 R_j 可取 1 到 j。

为什么这些名次独立？给定一组 $R_1, \dots, R_r$，从 j=r 往前恢复：先在尚未使用的 r 个数中取第 R_r 小者，作为最后读入的数；删掉它，再在剩余 r−1 个数中取第 R_{r-1} 小者，依此继续。每组合法名次恰好恢复一个排列。名次组合一共 $1 \cdot 2 \cdots r = r!$ 组，而每个排列等可能，所以联合概率为 $1/r! = \prod_{j=1}^{r}(1/j)$。这就证明各名次独立且分别均匀；只证明单个事件概率 1/j，本来不足以得到独立性。

令 $B_j = 1$ 表示 $R_j = 1$，即这次读到了新的最小记录，否则为 0。于是这些 B_j 独立，且 $P(B_j = 1) = 1/j$；栈长 $S_r = \sum_{j=1}^{r} B_j$。独立变量的指数期望可以相乘，单项为 $E[e^{B_j}] = 1 + (e-1)/j$，因此

$$E[e^{S_r}] = \prod_{j=1}^{r}\left(1 + \frac{e-1}{j}\right) \le e^{(e-1)H_r} \le Cr^{e-1}.$$

由 Markov 不等式，$P(S_r \ge t) \le Cr^{e-1}e^{-t}$；对所有 r 加起来，$P(M \ge t) \le Cn^e e^{-t}$。取 $t = A\ln n + s$、A>e，尾概率随 s 指数衰减，尾和给出 $E[M] = O(\log n)$。另一方面 M≥Sn，所以 $E[M] \ge H_n$，得到匹配下界。这些步骤解释了为什么答案成立，也暴露它依赖的随机模型。重复、相关或刻意排序的高度不能自动引用此概率界；最坏保证仍是线性。

25.34 34 · MaxQueue 摊还时间：每项只可能离开一次

影子队列按值单调递减。新项 x 到来，把尾部不可能在未来胜过 x 的较小项删掉，再把 x 接尾；原队列出队时，若影子头正对应离开的项，也删头。用索引或计数处理重复值，避免删错同值的另一项。

每项进影子一次，之后要么从尾被更大新值淘汰，要么从头因过期离开，至多一次。m 项总操作 O(m)，所以摊还 O(1)，不需要题名 Expected 暗示的随机假设。

取原队列依次入 3、1、4、2，再依次出队，影子保存(下标,值)。严格更小才从尾淘汰：

```
入(0,3)：影子[(0,3)]，最大3
入(1,1)：影子[(0,3),(1,1)]，最大3
入(2,4)：尾1、尾3依次淘汰，影子[(2,4)]，最大4
入(3,2)：影子[(2,4),(3,2)]，最大4
原队出下标0、1：它们已不在影子里，影子不动
原队出下标2：影子头就是2，删头，剩[(3,2)]，最大2
原队出下标3：影子清空
```

被新 4 淘汰的旧 3、1 即使还在原队列中，也不可能再成为最大值：4 更大而且更晚离开。这个“更大且更晚离开”同时保证淘汰安全，单有更大却马上过期并不够。用下标区别相同数值，也能准确判断离开的到底是哪一份。

记账法给每次加入预存一次未来删除费；势能取影子长度，一步删 k 项再加 1 项，实际 k+1 加势能 1-k 为 2。若维护原队列 X，其常数入出也另加，仍不改变摊还界。

25.35 35 · MaxQueue 空间：一扇窗口的平均，与长时间的最大值

固定窗口长 w，窗口内值独立连续同分布时，影子保存后缀最大记录。距窗口右端 j 项的位置成为记录的概率 1/j，影子在这一固定时刻的期望长度为 Hw，即 Θ(log w)。严格递减的一窗却会保留 w 项，最坏为 Θ(w)。

若观察越来越长的整段运行，不能仍把峰值期望写成 Hw。取互不重叠窗口，每窗恰好递减的概率 1/w!；观察 q 窗至少一次递减的概率为 $1-(1-1/w!)^q$，q 增加时趋近 1，因此固定 w 下的长期峰值趋近 w。

另外，若原队列 X 显式保存全部 w 项，它本身就占 Θ(w)，即使影子平均仅 log w，整个数据结构也不是总共 log w 空间。需要明确题目是在分析额外影子，还是包含原始元素的完整队列。

25.36 本章练习与答案

1. 原优先表和右结合修改表分别如何转换 2^3^2？写 RPN 与数值。
2. 高度[2,1,2]的最大矩形覆盖哪里？把每次弹栈对应的宽度写出来。
3. n=3 按第一个弹出元素分类，为什么不是三个类别都一样大？

第一题：原表 2 3 ^ 2 ^，值 64；修改表 2 3 2 ^ ^，值 512。第二题：最后 0 清栈时，中间高度 1 的柱可以扩到[0,3)，面积 3；高度 2 的左右单柱面积各 2，所以最佳是整个区间，高度 1。第三题：先弹 1 后剩 2 项自由混洗有 2 种；先弹 2 后还有两种合法完成；先弹 3 时栈中剩 2 压着 1，只能 2 再 1，所以 1 种，总 5。分类计数要由剩余状态决定，不能凭对称感猜。

26 PS05：树的箭头、遍历顺序和编码成本，一步一步核对

教师原始习题单：第 05 次习题共 37 题。路径长度按边数，根深度 0，叶高度 0，空树高度约定为-1。先序为根–左–右，中序为左–根–右，后序为左–右–根。“真二叉树”指每个内部节点恰有两个孩子，不等于每层都满。

26.1 1 · 路径按边计：空走一步都没走，就是 0

节点到自己的路径长 0。u 到 v 长 a，v 到 w 长 b，若可沿树中同一路径接起来，总长就是 a+b，不需要因为连接点 v 被数了两次再减 1。

例如根 R 到 A 一条边，A 到 B 一条边，R 到 B 深度 2。若按节点数，前两段各有 2 个节点，连接后却是 3 个，公式总要额外修正；按边计数与“每过一条链接深度加 1”一致。

计数约定本身都可以定义，但后续高度递推、路径拼接、复杂度边界必须统一。看到别的书叶高 1 时，先做整体偏移换算，再判断公式是不是矛盾。

26.2 2 · 深度加高度不超过全树高度

设 v 深度 d，从根到 v 有 d 条边；v 的最深后代距 v 为 h(v)，拼起来就是根到某个叶的路径，长 d+h(v)。全树高度 H 是所有根叶路径的最大值，因此 $d + h(v) \le H$。

若 v 的子树中含有全树某条最深路径，取等；若 v 在较短的侧枝，便可能严格小于。例：根左链深 4，右孩子就是叶。右孩子 d=1、h=0，总 1<4；左链中任一节点的深度加剩余高度都等于 4。

不能把全树最长路径误当“必须经过每个节点”。这条不等式是在比较 v 所在的那条候选路径与所有候选的最大值。

26.3 3 · 三种树表示：多存一条边，省下哪种搜索

只有 parent 时，判断 u 是否为 v 祖先，可从 v 向上走到根，最坏 O(h)。接入新节点时设置 parent 即可，但若要列出它的所有孩子，就可能扫描全体节点。删除整棵子树若无孩子索引，也需要先找到哪些节点属于它，不能只看摘根那一步。

只有孩子列表时，从 u 向下 DFS 能判断 v 是否属于其子树，最坏 O(size(u))；要从全树删除子树根，还需知道它在父节点孩子列表中的位置，若没有 parent 或外部位置，可能先从根搜索。已知插入父亲和孩子序号时，插入成本还取决于孩子容器是数组还是链表。

parent 加孩子列表兼顾上溯和下行，但每次接入、移动、删除都要两边同步。删除含 s 个节点的子树至少要处理这 s 个对象，故实际释放 Ω(s)。

用一棵小树核对父表示的查询：A 是根，B、C 的 parent 为 A，D、E 的 parent 为 B。问 B 是不是 E 的祖先，从 E 走到 B 就命中；问 C 是不是 E 的祖先，则 E→B→A→空都没遇到 C，拒绝。接入 F 为 C 的孩子，只写 `parent[F]=C` 便能表达这条父关系；但问“C 有哪些孩子”时，只有 parent 数组就需扫描所有节点找 parent 等于 C 的项。一个方向的边能让上行容易，不会自动提供反向目录。

相同树下求 D 和 C 的 LCA：深度 D 为 2、C 为 1，先把 D 移到 B，使两边都深 1；B 与 C 不同，一起上移到 A，得到答案 A。每次走向 parent 都会使深度减 1，故至多高度量级。

求 LCA 时，有 parent 可先算深度、把更深节点上移，再同时上移，O(h)时间；没有 parent，可从根找两条路径再比较最后公共位置，普通实现 O(n)时间、O(h)路径空间。预处理倍增或时间戳能改进多次查询，但需要额外结构和预处理成本。

26.4 4 · 叶子总比双分支多一个：同一条边数两遍

非空树 n 个节点有 n-1 条边。另一方面，每个单分支节点向下贡献 1 条，双分支贡献 2 条，所以

$$n_1 + 2n_2 = n_0 + n_1 + n_2 - 1.$$

消去 n1，得到 $n_0 = n_2 + 1$。再写 $e = n - 1 = n_1 + 2n_2$，可看见单分支越多，边数里越多是单链贡献。

真二叉树 n1=0，于是 $n = 2n_2 + 1$、$n_0 = (n+1)/2$、$e = 2n_2$。完全二叉树可以有一个单孩子节点，不能不看结构就套真树公式。

例：根有两个孩子，其中左孩子还有一个孩子。n0=2、n1=1、n2=1，边 3=1+2，叶 2=双分支 1+1。手数一次，再对照公式，比直接背三个字母更稳。

26.5 5 · 长子—弟弟编码：一棵二叉树可能代表一片森林

约定左指针表示第一个孩子，右指针表示下一个兄弟。多叉树的根没有兄弟，因此它编码后的根右指针为空。任意二叉树若根有右孩子，按这套含义就代表根还有兄弟，更自然地对应森林，而非单棵树。

例如二叉根 A 的右指针指 B，左指针指 C，解码为两棵树：第一棵根 A、孩子 C，第二棵根 B。不能把 B 同时当 A 的普通孩子，那会混用左右指针的含义。

把整树根在某些接口里视为虚拟父节点的右孩子，是为统一“指向该子树的入口引用”的约定；它不让真实根凭空多一个父亲，也不意味着根在长子—弟弟编码里有真实的右兄弟。

26.6 6 · 有序森林与二叉树：正反变换都写出来才叫一一对应

森林各根按从左到右用右兄弟指针串起来；每个根的孩子列表，首孩子接左指针，后续孩子也用右指针串起来。所有节点只保留这两类指针，就得到二叉结构。

具体把森林 A 的孩子为 [B,C]；B 的孩子为 [D] 和另一根 E 编码：

```
原有序森林：              二叉编码（左=首孩，右=弟弟）：
    A       E                    A --右--> E
   / \                          /
  B   C                        B --右--> C
 /                            /
D                            D
```

真实的原树边有 A→B、A→C、B→D；编码中的 B→C 却是兄弟边，不表示 B 是 C 父亲。解码 A 的孩子时，从 A.left=B 开始沿右链收集 B、C；解码 B 的孩子时，从 B.left=D 开始，只收 D。根 A 的右链给森林另一根 E，不应把 E 放进 A 的孩子列表。

解码时，从二叉根沿右指针拆出森林的根序列；每个节点的左指针给第一个孩子，再沿这条孩子的右链列出其余孩子，逐个递归。顺序没有丢，层级也由左指针与兄弟链的位置唯一决定。

正变换后立刻反变换会恢复每个原孩子列表，反过来也恢复每一条二叉指针，所以既不会两个森林编码成同一结构，也不会遗漏某个二叉结构。无序树若本来没有兄弟顺序，就不具备这项同样的一一对应前提。

26.7 7 · 原题要求的真树：每个空位置都要补，包括旧叶子的两边

以下先设原树非空，n=n0+n1+n2，n0=n2+1。原题要求新树双分支数 $2n_2 + n_1 + 1 = n$，叶数 $2n_2 + n_1 + 2 = n + 1$。正确转换是：把每个原节点都保留为内部节点，原来缺失的左、右孩子各补一个带标记的外部空叶。

原叶有两个空位置，要补两个；原单分支补一个；原双分支不用补。新外部叶总数 $2n_0 + n_1 = 2n_2 + n_1 + 2$；每个原节点如今都有两个孩子，新双分支数正是 n，单分支为 0。

例：只有一个原根 A。补后是 A 连两个外部叶，内部 1、外部 2。若只给单分支节点补另一边，A 一开始没有孩子，完全不补，这时仍只有一个叶，达不到题面要求的两个外部叶。只给单分支补孩子也是一种变成真树的方法，但它的节点数与本题指定的转换不同。

原树 A 只有左孩子 B 时，转换可以全部画出来。圆点 # 是带标记的外部叶，不是原数据节点：

```
原树                  扩充后
  A                      A
 /                      / \
B                      B   #
                      / \
                     #   #
```

原来有 2 个节点、1 条边、4 个孩子指针槽，其中 1 个非空、3 个空。一般 n 个原节点有 2n 个孩子槽，非空槽恰好对应 n−1 条边，所以空槽为$2n-(n-1)=n+1$。补每个空槽一个外叶，正好得到 n+1，不依赖原树是否有单分支。这是与$n_0=n_2+1$推导独立的第二次核对。

保留外部叶标记，逆变换删除所有外部叶即可唯一恢复原树。标记不能与普通数据值混用，否则真实节点恰好也存“空”字样时会有歧义。

26.8 8 · 高度更新：某处没变，往上的高度也不会变

高度由两个孩子高度决定，$h(v)=1+\max(h_L,h_R)$。一次局部修改只影响一条祖先路径；若更新到某祖先 a，发现 h(a)与旧值相同，那么 a 的父亲看到的这个孩子高度没变，另一孩子又没改，因此父高也不变，往上同理。

插叶、删叶、接入整棵子树、分离子树都适用这个逻辑，但必须从真正受影响的入口向上，并在改完孩子链接后重算。高度不变不表示 size 没变；若还维护节点数、和或其他统计，不能一并提前停。

需要把其他缓存与高度分开：原节点 A 的左子树高度 2、右子树高度 0，所以 A.height=3。向右子树插一个叶，让右高变 1，A.height 仍为 3，可以停止高度传播；但 A.size 已经加 1，所有祖先的 size 也都应加 1。若把 `if(newHeight==oldHeight) break` 放在同时更新 height 和 size 的循环里，会少更新上层 size。提前停止只适用于已经证明不变的那一项统计。

例：根两边高度分别 3 和 1，在较短一边增加到 2，根高度仍 4，继续往更高处传播高度就是多余。最坏插在唯一最长链末端，高度一路变到根，优化仍 O(h)，只是某些修改能提前结束。

26.9 9 · 勤奋缓存与懒惰计算：用操作序列算总账

若每次求 height 都遍历子树，单次可能 O(n)；缓存 height 后查询 O(1)，但修改时需向上维护。size 也一样：即时维护使 size 查询常数，代价是每次接入、删除、旋转等都正确更新。

例如一天修改 100 次、查询一百万次，缓存常常合适；一天改一百万次、只查一次，按需遍历可能更简单。要把树高、子树规模和查询频率代入，而不是认为“勤奋一定更好”。

维护多个缓存还会增加一致性风险：旋转后必须先更新低节点，再更新高节点，使父读到的新孩子信息正确。测试除了最终值，还要在每次修改后用独立遍历结果核对缓存。

26.10 10 · 中序在单侧链上：空调用忽略后，看剩余工作

全右链中序先遇空左树，然后访问当前，再递归右树，右递归之后没有剩余访问工作，可按尾递归化。全左链则先递归左边，再返回访问当前，访问工作都积在返回路上，是头递归形状。

一般树左右都有非空分支，不能同时把两条路径都压成同一种单链循环。空子树调用仍有固定检查成本，讨论形状时可以忽略它，但实际计费不要说从未执行过。

以 A 右连 B 右连 C，中序就是 A、B、C，可直接循环；A 左连 B 左连 C，中序应为 C、B、A，要记住祖先或借助结构回溯，不能从根一路输出。

26.11 11 · 单链先序：只向前走，不需要 parent

每节点最多一个孩子时，先访问当前，再把 p 设为唯一的非空孩子，直到空。因为没有另一棵尚未处理的侧树，不需要返回祖先寻找分支。

时间 Θ(n)，辅助空间 O(1)，即使没有 parent 也完全可行。若递归版编译器没有尾调用优化，它会保留 n 层；显式循环才把空间保证写进实现。

这项简化依赖“最多一个孩子”。一般树中若先深入左边，右边还没处理，需要保存任务；删去 parent 并不会删去这些待办任务。

26.12 12 · size 访问器：只要不重不漏，哪种遍历都能数

定义 Count 含 value，visit 每到一个真实节点就加 1。先序、中序、后序、层序都访问每个节点一次，所以最终都为 n；空树为 0，外部空叶是否算节点必须按所求结构定义。

求所有 data 总和、检查每项是否在某范围，也可这样写。若要计算“每个节点子树大小”，单纯给全局计数器加 1 不够，需要把左右子结果汇回父节点，后序更自然。

函数对象按引用还是值传递也要核对，避免最后外部计数仍 0。遍历的抽象复用建立在任务真正只需要一次单节点访问的前提上。

26.13 13 · clone：中序、后序都能复制，但要保存结构位置

后序做法：先复制左子树得到 L，再复制右子树得到 R，然后新建根并接上 L、R。先序则先建根，再递归填两边。二者都能保持形状，且每个原节点对应一个独立新节点。

中序也可以先复制左边，保存结果 L，创建当前根，再复制右边并连接；不能因为“根在中间创建”就说不可能。要保存孩子身份，而非仅把中序值输出，再当作 BST 插入，那会改变原形状。

后序复制的最小实现可直接对照结构：

```
clone(p):
    若p为空，返回空
    L = clone(p.left)
    R = clone(p.right)
    q = 新建与p数据相同的节点
    q.left=L；若L非空则L.parent=q
    q.right=R；若R非空则R.parent=q
    更新q的height/size
    返回q
```

对原树 A 左 B 右 C，先创建新 B’，再 C’，最后 A’，连接 A’.left=B’、A’.right=C’。根数据相同但地址不相同，且 B’、C’都不指回原 A。只复制中序 `B,A,C` 并逐项插入 BST，可能得到根 B、右 A、A 右 C，与原形状不同。复制值序列不能代替复制孩子位置。

无论哪种都需 Θ(n)新输出存储，递归栈最坏 O(h)。若构造中途失败，还要释放已经复制的部分或使用资源管理对象，避免只在完全成功的路径上没有泄漏。

26.14 14 · 打印树：缩进只说明深度，还需要说明哪条边

若每层缩进两格，只能看出节点深浅，不一定知道它是哪个父亲的左或右孩子。完整打印还需保存左右分支身份，以及哪些祖先层还有未打印的兄弟，用来决定竖线是否继续。

可以先打印右子树、再根、再左子树，让树向侧面展开；也可以先序打印带“L/R”的路径。选择一种后把图例写清，避免读者把上方节点误当左孩子。

先序带路径打印适合最早的自检。例如树 A 左 B 右 C，B 只有右孩子 D，应输出：

```
root: A
L:    B
L/R:  D
R:    C
```

`L/R` 表示从根先走左边、再走右边，所以 D 是 B 的右孩子。若只缩进，`D` 出现在深度 2，读者仍可能误以为 B 有左孩子。路径文字与图中空位置一起保留，就能验证分支方向。

用一棵左右高度不同、有单孩子且有空位的小树核对，比用满树更能暴露错误。打印是显示接口，不能通过交换真实孩子指针来“画得好看”而改变树。

26.15 15 · 清空不只允许后序，但绝不能释放后再读

后序先清左右，再删根，最容易保证仍能找到未处理节点。先序也可先保存 left、right，再 delete 根，再用保存的地址清左右；删除根后不能再读它的字段。

中序可先清左，保存右指针，再删根，然后清右。对象析构若会自动递归释放孩子，就不能同时再手工递归，否则双重释放；所有权策略必须统一。

各种顺序都能做到 Θ(n)，差别在于保留哪些地址、析构副作用和代码可验证性。正确性由生命周期与剩余对象可达性决定，不是某个遍历名字天然授予安全。

26.16 16 · 从 RPN 建树：栈里放的从数字变成子树

遇操作数就建叶节点入栈；遇二元运算符，先弹右子树 R，再弹左子树 L，新建运算符根并接上 L、R；遇一元后缀运算则只弹一棵树。

例如 `2 3 4 + *`：先有叶 2、3、4，+弹 4 和 3，形成根+的子树；再弹这棵右子树和叶 *2*，得到 *2*(3+4)。最后栈恰有一棵树，便是答案。

栈里每项是一整棵已经建好的子树，不能把“栈中多个节点”误连成一棵链：

读 token	操作	栈（底→顶；用括号表示子树）
2	建叶 2	`[2]`
3	建叶 3	`[2,3]`
4	建叶 4	`[2,3,4]`
+	弹右 4、左 3，建`+(3,4)`	`[2,+(3,4)]`
*	弹右`+(3,4)`、左 2，建`*(2,+(3,4))`	`[*(2,+(3,4))]`

归纳不变量是：已经消耗的 RPN 前缀被栈中这些互不相交的完整子表达式恰好分解。遇运算符合并的就是最近完成的几个子表达式，顺序由先左后右的后序决定。每个 token 创建一个节点、每条孩子边写一次，故时间线性，输出树也必须占线性空间。

执行前检查元数，结束检查剩余棵数；`2 +` 操作数不足，`2 3` 留下两棵，都不合法。固定 token 与元数后，树结构可唯一恢复；若同一符号有不明确的一元、二元含义，还需词法或语法信息。

26.17 17 · 沿右藤的“对称版”，会不会把先序变成根右左

直接访问根后一路深入右边、把左子树暂存，得到的是根–右–左，并非要求的根–左–右。若只是把 lc 与 rc 交换，必须同时承认遍历定义变了。

沿右藤分解时，可以把基本工作改成“访问藤上当前节点，完整处理它挂着的左子树，再沿右边走一步”。左子树仍必须先完成，不能把整条右藤预先访问完。直接写出来是：

```
def preorder_right_vine(node, visit):
    while node is not None:
        visit(node)
        preorder_right_vine(node.left, visit)
        node = node.right
```

while 负责沿当前右藤前进；递归负责侧面的整棵左子树。没有左孩子时，空调用立即返回；有左孩子时，调用结束前不访问当前节点的右孩子。这正好保持根–左–右。

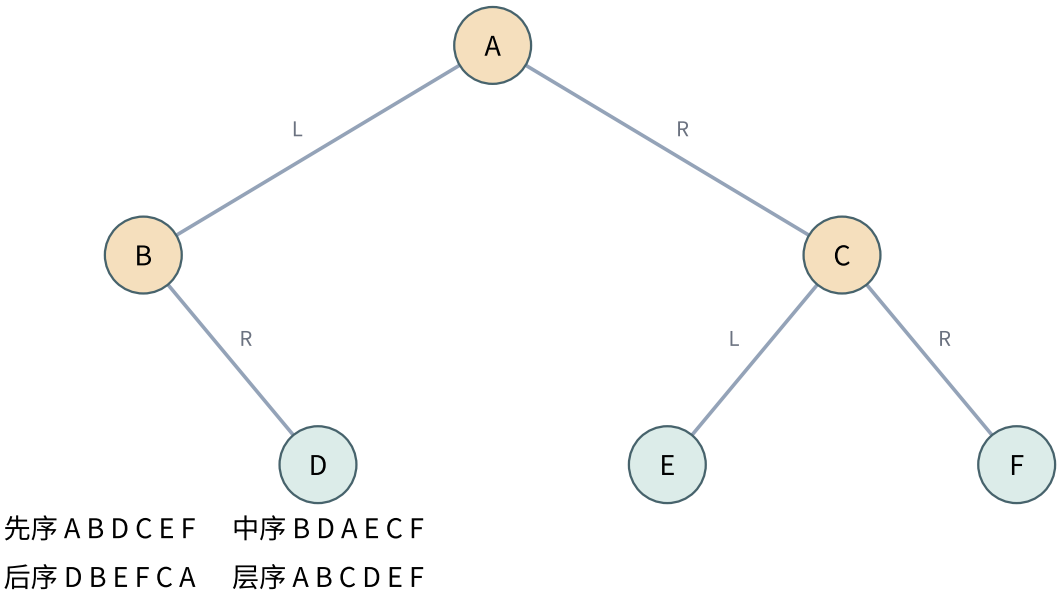


图 206 后面逐步走读的小树。L/R 标在真实边上，B 没有左孩子；图下四种访问次序可以先遮住，用来核对自己的栈和队列。

取 A 左 B、右 C，B 右 D，C 左 E、右 F。访问 A 后，先进入左子树 B；访问 B，左边空，沿 B 的右藤访问 D。整个 B 子树完成，才回到 A 那一层沿右藤到 C；访问 C，递归访问 E，再沿右藤访问 F。输出为 A、B、D、C、E、F。若先扫 A、C、F 再补 B、D、E，先序就被破坏了。

这个版本把右方向改成循环，递归栈只为尚未处理完的左侧调用保存返回位置。全右链只需常数层调用，全左链却要 Θ(n)层；时间仍 Θ(n)，一般辅助空间 O(h)。沿左藤的迭代版则显式保存待办右子树，两者保存的位置不同。递归版在长链上还可能超过语言的调用深度限制。

若要把上述左递归也改成显式栈，离开当前节点进入左边之前，就保存它尚未继续的右藤起点；左侧完成再恢复：

```
def preorder_right_vine_iter(root, visit):
    resume = []
    node = root
    while node is not None or resume:
        if node is None:
            node = resume.pop()
            continue
        visit(node)
        if node.left is not None:
            if node.right is not None:
                resume.append(node.right)
            node = node.left
        else:
            node = node.right
```

栈内每项都是“当前左侧工作完成后，接着处理这段右藤”的任务，最后存入者最先恢复。它与常见的“先压右、后压左”任务栈在访问顺序上等价，但这里不必把立即要走的左孩子也压栈。两种写法都没有更好的普遍时间阶；优缺点要比较具体树形上的待办数量、调用开销与实现清楚程度。

26.18 18 · 先序正确性：按整树规模归纳，侧枝也要包括

空树无输出；单节点访问一次正确。对更大树，沿左藤访问根及各左后代，并把每处右子树压栈。最深处的右侧任务最后压入，最先弹出，正好符合先做完当前左子树内部，再返回更高祖先处理右树的顺序。

每个被压的右子树规模小于原树，且彼此不重叠，归纳假设保证它们各自按先序完成。藤节点与这些侧树恰好覆盖原树，所以不漏、不重、次序正确。

若只归纳藤长度，外挂侧树可能比整条藤大得多，原归纳假设未必覆盖它们。可以增强命题，让它对任意合法侧树也成立；不能只把参数名字从“规模”换成“藤长”就算证明。

26.19 19 · 不压空右树：减少无效栈动作，但加了一次判断

原先无条件压右孩子，可能压入 nullptr；弹出后又立刻发现为空。改成右孩子非空才压，就消去这些无效 push 和 pop。

对全左链，若原版每层都压空右树，会积累很多空任务；去掉后待办栈可以显著缩小。对几乎每个节点都有右孩子的树，新增判断很少省动作。

总时间仍 Θ(n)，在不同编译器与数据下实际净收益需要测量。能严格说少做了哪些栈操作，不应直接保证所有机器上运行时间都按同一比例下降。

26.20 20 · 先序栈空间：高度是上界，待办右树数更贴切

沿左藤版栈保存还没访问的右子树根。当前路径每经过一个“走向左孩子、右边还有任务”的祖先，最多增加一个待办，因此栈长不超过相应左祖先数，进而不超过 O(h)。底部哨兵或当前节点另加常数。

全右链高度 n-1，几乎没有左祖先，待办栈可常数；全左链若不压空右树，也可以常数。若长左链每个祖先还挂一个真实右叶，走到底时会同时积累约 h 个右任务，达到高度阶。

原题 c、d 要求构造极端树，不只报一个$O(h)$。若只统计非空右任务：全右链长 n，高度 n−1，最大左祖先数 0，显式待办栈峰值 0（连当前指针一起计也只多常数）；高度上界可比实际大线性倍。要让左祖先数与高度相同，可取每一步都走左的深链；若还要求实际栈任务达到高度阶，就在左链每个非末节点挂一个右叶，下降时分别留下这些右叶，走到底同时有 h 个待办。

```
      A
     / \
    B   a
   / \
  C   b
 / \
D   c
```

访问 A、B、C、D 时，栈中的非空右任务依次为[a]、[a,b]、[a,b,c]、[a,b,c]，底层任务 c 最先恢复。树高 3，待办峰值 3。左链若完全不挂右叶，左祖先数仍大，实际非空任务却为 0，说明这个数量是上界而非每棵树的精确等式。

题目问两个上界之比的数学期望，必须说明树形与选点分布。左右方向独立对称的一条固定路径，其左转次数期望是路径长一半；但“全树最大左祖先数/全树高度”的期望不是因此自动一半。max 与期望以及比值都不能随意交换。没有随机模型，不能给出普遍数字。

26.21 21 · 中序归纳：左藤只是拆解入口，不是全部规模

中序先沿左藤把祖先压栈，直到最左节点；弹出当前访问，再把其右子树按同样方式展开。整棵树规模归纳可以直接覆盖左右两个更小子树。

对文中的弯曲树，完整中序栈过程是：

```
A左B，B右C，C左D。
沿左压A、B，栈[A,B]，当前到空。
弹B并访问，输出B；转B.right=C。
沿左压C、D，栈[A,C,D]，当前到空。
弹D访问，输出B,D；D.right为空。
弹C访问，输出B,D,C；C.right为空。
弹A访问，输出B,D,C,A；结束。
```

每次弹出的节点，其左子树已经完成，自己访问后再转右子树。等待中的 A 不会在 B 结束时被马上访问，因为 B 的右子树 C 已经被安排在 A 之前。这就是栈的后进先出与中序要求相符的具体位置。

若以左藤长度归纳，需要把“藤上每个节点挂着任意右子树时，算法仍正确处理它们”写进命题，并独立保证这些侧树的正确性。只讨论藤本身的访问顺序不足以证明整棵树。

用根 A、左 B、B 右 C、C 左 D 的小树测试：中序应 B、D、C、A。它很容易揭穿只按一条直链推理、漏掉弯曲侧枝的证明。

26.22 22 · 最后一个中序节点：一路向右后，再向上越过根

中序最后访问整树最右节点。它没有右子树，否则右子树中还有更晚节点。求 succ 时，若有右子树应取其最左节点；没有就向上，直到第一次从某祖先左侧返回，该祖先才是后继。

对于整树最右节点，向上始终处在祖先右侧，最终越过根得到 nullptr。因此算法应在访问 parent 前判空，不能再从根去读不存在的父节点。

最小树只有一个根，它也是最后节点，succ 直接为空。测试这个例子能检查代码是否把“根无父亲”的边界遗漏。

26.23 23 · 没有 parent，也能后序：自己保存返回阶段

栈保存(node,stage)。stage=0 时准备处理左边，改成 1 后压左孩子；stage=1 时左边已完成，改成 2 后压右孩子；stage=2 时左右都完成，访问当前并弹出。

```
def postorder(root, visit):
    stack = [(root, 0)] if root is not None else []
    while stack:
        node, stage = stack.pop()
        if stage == 0:
            stack.append((node, 1))
            if node.left is not None:
                stack.append((node.left, 0))
        elif stage == 1:
            stack.append((node, 2))
            if node.right is not None:
                stack.append((node.right, 0))
        else:
            visit(node)
```

以 A 左 B 右 C 走程序：栈先`[(A,0)]`；处理阶段 0 后变`[(A,1),(B,0)]`；B 没有孩子，依次改为 B 的阶段 1、2，然后访问 B 并弹掉；恢复 A 阶段 1，改存 A 阶段 2 并压 C；C 同样完成三个阶段、访问 C；最后弹 A 阶段 2 并访问。输出 B、C、A。每个父帧在压孩子以前已改好下一阶段，所以孩子返回后不会再次从阶段 0 开始。

这相当于把语言调用帧的程序位置显式存出来，时间 Θ(n)、辅助空间 O(h)。parent 指针是获得返回方向的一种办法，不是唯一办法。

26.24 24 · 后序正确性：访问条件就是“两棵子树已完成”

对每个帧，阶段只按 0→1→2 向前推进。进入阶段 1 前左子树任务已执行完，进入阶段 2 前右子树也已执行完；因此访问根时，恰好符合左–右–根。

每个非空孩子只由唯一父帧压一次，树没有共享孩子或环，所以不会重复。空孩子直接跳过，不影响阶段含义。按子树规模归纳，两个子任务各自输出正确，接上当前根便正确。

如果输入其实是共享子图或有环，这份树遍历可能重复或不结束，需要 visited 和另一种语义。先把“是合法树”写成前提，证明才不会偷换对象。

26.25 25 · 迭代遍历总时间：一次下藤走 h 步，不能给每个节点都收 h 步

先序每个节点访问一次，其右任务至多压弹各一次；中序每个节点压一次、弹一次；阶段式后序每个节点只经历固定三阶段。合计都 Θ(n)。

内层 while 偶尔走很长一条藤，但这条藤上的节点随后不会在别处再次作为新的未访问节点下藤。把所有内循环迭代相加是 n 量级，不能看到嵌套循环就相乘。

若算法每次求 succ 都从根重新搜索，或每次重算整棵子树信息，已经不是这里被分析的迭代版本，可能增加成本。复杂度结论应绑定到真实的节点进出次数。

26.26 26 · 层序遍历：队列里为什么只会混着相邻两层

先把根入队。每次弹一个节点 v 访问，再按左后右把其孩子入队，孩子深度仅比 v 大 1。FIFO 保证当前层尚未处理节点在前，刚发现的下一层孩子排在后，所以队列深度非降，最多差 1。

归纳可知根先到，每个非根节点恰由唯一父亲发现一次。每个节点入队一次、出队一次，因此总 Θ(n)。同层左先右后来自孩子入队顺序；若调换左右，深度顺序仍正确，但同层阅读方向会变。

用原题下一问的 5 节点例列全队列：根 R 左 A 右 B，A 左 C 右 D。

本次弹出访问	新加入的孩子	操作后的队列（左为队首）	深度序列
尚未开始	先放 R	`[R]`	`[0]`
R	A、B	`[A,B]`	`[1,1]`
A	C、D	`[B,C,D]`	`[1,2,2]`
B	无	`[C,D]`	`[2,2]`
C	无	`[D]`	`[2]`
D	无	`[]`	`[]`

处理 A 以后当前层还剩 B，下一层 C、D 已经到尾部，两层同时存在；但深度差从不超过 1。每一轮一定弹 1 个，加入可以是 0、1、2 个，所以队列长度并不单调。

“左边更早”只能对同层按该有序树定义解释。一个很浅的右节点会早于深处左节点，不能把全树横向位置当成绝对处理次序。

26.27 27 · 层序空间：队列可能比某一层还稍宽

设最大层宽 w。队列可能同时包含当前层剩余部分与下一层已发现部分，故不一定恰好≤w，但最多是相邻两层的总宽，仍 O(w)。单链每次队列最多 1，完全树可线性大。

更精确地，队列每个节点代表一棵互不相交的尚未处理子树，每棵至少包含一个不同叶子，所以队列长度不超过叶子总数。非空二叉树叶子最多 $\lfloor (n+1)/2 \rfloor = \lceil n/2 \rceil$，完全二叉树可以达到这个数量级，且这种 BFS 队列也能达到该上界。

例如 n=5 完全树，根孩子 A、B，A 有 C、D。处理 A 以后队列为 B、C、D，长度 3；最大单层宽却只有 2。这一小例解释了为何空间 Θ(w)不表示每时每刻精确等于某层宽。

26.28 28 · 完全树放进数组：父子下标怎么算

零基存储时，节点 i 的左孩子 2i+1、右孩子 2i+2；i>0 的父亲为 $\lfloor (i-1)/2 \rfloor$。存在与否只需检查孩子下标是否小于 n，不能先访问再判断。

根 0，下一层 1、2，再下一层 3、4、5、6。这些区间连续，因此层序扫描局部性很好；沿一条深路径，下标却按倍增跳跃，不能保证每对父子都物理紧挨。

完全性保证中间无空洞；任意稀疏树硬放这种数组，深单链可能需要指数大的空位置。适用条件与缓存收益都要按访问模式说明。

26.29 29 · 真树先后序重建：第二个先序项给出左子树范围

假设标签互异，且给的是同一棵真二叉树。先序首项和后序末项必须是同一个根；若不止一个节点，先序第二项就是左子树根。到后序中找它，可得到整个左子树的末端与长度，从而把两个序列都切成左右两块递归。

例如先序 A,B,D,E,C，后序 D,E,B,C,A。根 A，左根 B 在后序下标 2，左块长度 3；递归左块得到 B 的孩子 D、E，右块只有 C。真树保证根不可能只有一支，消除了“这段到底左子树还是右子树”的歧义。

把两次递归切分写得更具体。下标采用从 0 开始的半开区间：

```
pre = [A,B,D,E,C]       post = [D,E,B,C,A]
根A：pre[0] == post[4]
左根B在post[2]，左子树长度=2-0+1=3
左：pre[1:4]=[B,D,E]，post[0:3]=[D,E,B]
右：pre[4:5]=[C]，     post[3:4]=[C]
左根B：其左根D在左post的第0位，左长度1
于是B.left=D，B.right=E；C是叶。
```

得到的形状是：

```
      A
     / \
    B   C
   / \
  D   E
```

若不是叶，真树要求左右两块都非空，且每块节点数为奇数；例如只剩 2 项却试图构造真树，应报告输入不合法。保留四个边界下标而不反复切片复制，才能达到线性总工作；用 Python 切片直接实现会额外复制子数组，不能仍把这些复制当免费。

用标签到后序位置的字典可使定位期望常数，总体 O(n)；若每层线性找标签，退化树形可二次。还要验证左右块均合法、根标签一致、标签集合一致，不能把不一致序列强行拼成树。

26.30 30 · 增强遍历：空标记能救先序、后序，救不了中序

先序读一个 token，若是 # 返回空；否则建根，再依次读取左、右子树。对 `A B # # C # #`，先建 A，左建 B 并读两个空，右建 C 并读两个空，边界被递归结构唯一确定。

增强先序的反向恢复不用猜哪个子树在哪结束：读到 A 要继续恢复两个孩子；读到 B 也要继续恢复两个孩子；两个 # 分别让 B.left、B.right 为空，返回 B；再读 C，读完它的两个 # 返回 C；最后 A 的两个孩子都齐，返回 A。递归中的“还欠几个孩子”就是边界信息。

```
读取后：A              读取B,#,#后：  A       全部完成：  A
                                    /                  / \
                                   B                  B   C
```

反读增强后序时规则对称：最后一个真实 token 是根，接着最先读到的是右子树的末根，所以必须先建右、后建左。先后顺序弄反会得到镜像树，值集合一样，结构却错。

后序从末尾反读，遇根后先恢复右、再恢复左，同样唯一。每个真实节点和空位置读一次，时间线性，还要检查最后有没有多余 token。

中序增强不唯一：A 只有右孩子 B，与 B 只有左孩子 A，都会给 `# A # B #`。空标记本身不自动记录根在哪，所以“所有增强序列都能唯一重建”是错误推广。

26.31 31 · 组合重建：每个“唯一”都带着前提

互异标签下，先序给根、中序给左右分界，故先+中唯一；后+中同理。普通先+后对单孩子方向无解：A 左 B 和 A 右 B 的先序都 A,B，后序都 B,A。

加上真树条件可排除单孩子歧义，如前题。层序加中序也能逐层选出各区间最早出现的根来重建，但单独层序只有访问顺序，不能决定哪些边存在。

层序+中序也走一个例：层序`[A,B,C,D,E]`、中序`[D,B,E,A,C]`。层序最早的 A 为根，中序把左边切成 D、B、E，右边为 C；在左集合中，层序最早出现的是 B，故左根 B，再由中序分出左 D 右 E；右边只有 C。得到与前题同一棵树。朴素每层筛一遍层序可能$O(n^2)$，唯一性成立不代表任意构造实现都已经线性。

标签重复时，连先+中也可能不唯一，必须为不同节点加身份或另外约束。先检查输入条件，再选重建方法，不能只拿两个遍历名称就保证有解且唯一。

26.32 32 · 交换子树：成本差由总权与深度差决定

两棵互不为祖先后代的子树根深 d1、d2，总权 W1、W2。交换后各叶内部相对深度不变，第一棵整体移动 d2-d1 层，第二棵反向移动，因此

$$\Delta C = (W_1 - W_2)(d_2 - d_1).$$

若较大权 W1 原来更深，即 W1>W2 且 d1>d2，则差为负，成本下降。权相等或深度相等只保证不变，不能说严格下降。

把差分逐项展开。交换前这两棵子树的每个叶深都可写成“子树内部深度+子树根深度”。内部深度不变，其对应成本会完全抵消；只剩根深度项：

$$C_{\text{后}} - C_{\text{前}} = W_1 d_2 + W_2 d_1 - W_1 d_1 - W_2 d_2 = (W_1 - W_2)(d_2 - d_1).$$

W_1是第一棵子树所有叶权之和，不是子树根原来的 data；d_1是交换前该根距整树根的边数。若把单个叶权与整棵子树总权混用，计算会少掉内部其他叶整体移动的贡献。

例 W1=10、W2=3，深度 3 与 1，交换差(10-3)(1-3)=-14。若一棵包含另一棵，直接交换会破坏树，不能把同一公式不加结构限制地套用。

26.33 33 · Huffman 最优：贪心选择要能嵌进某棵最优树

先取最小两权 a、b。在一棵最优真编码树里选最深的一对兄弟叶，把 a、b 交换到这两个位置。最小权放得更深不会增加成本，故存在一棵最优树让它们为兄弟。

把这对兄弟缩成权 a+b 的叶，原树成本恰等于缩小树成本再加 a+b。若缩小树不是最优，换成更优的小树再展开，就会改进原最优树，矛盾。因此在缩小问题上继续做同样贪心，按叶数归纳得到最优。

交换论证中，“最小权放深处”可以直接算：若$a \le w$而 a 原在深度$d_a \le D$，把它与深度 D、权 w 的叶交换，成本变化为$(a-w)(D-d_a) \le 0$。先放 a，再放 b；最深的两个兄弟位置深度相同，第二次不会增加已放好的 a 的成本。最深叶的兄弟必须也是叶，否则兄弟的后代更深，矛盾。于是找到的是**一棵同样最优且具有贪心第一步的树**，不需要断言所有最优树都只有同一个形状。

缩成一叶时，原 a、b 比合并叶都多 1 层，所以展开额外成本恰好是$a+b$。这也是“每次合并和的总和等于最后 WPL”的来由：每次合并为其子树的每个叶深加 1，累计到叶上就是权重乘最终深度。

对[2,3,7,8]，先合 2+3=5，再 5+7=12，再 12+8=20，WPL 为 5+12+20=37。展开树的叶深分别 3、3、2、1，直接算 2×3+3×3+7×2+8×1 也为 37。两个算法得到不同左右码字不一定有错，最优编码树可能不唯一。

26.34 34 · 框架与森林容器：换容器要重新带入成本

核心框架只有：取两个最小根、合并为新根、把新根放回，直到剩一棵树。它依赖优先队列能力，不必知道底层是无序数组、堆还是两个有序来源。

无序数组每轮线性找最小，n 轮可二次；小根堆每次取最小、插入均 O(log n)，总 O(nlog n)。初始建堆可以线性，但后面 n-1 轮合并仍在。

接口隔离让框架可复用；复杂度分析却不能对不同容器照抄一个数字。源码中找到森林类型、取最小和插回的位置，再把每项成本相加。

26.35 35 · 双队列 Huffman：线性的是已经有序之后的合并阶段

原始权重升序放在一个来源，新合并的权放第二个队列。每次只比较两个队首，从中取全局最小的两个。新合并权按生成顺序不减：下一轮取到的两个可用最小值，不会产生比上次合并和更小的新和。

原输入[1,2,4,8]：取 1、2 得 3，合并队列[3]；取 3、4 得 7；取 7、8 得 15。每个原项与合并项进出常数次，有序合并阶段 O(n)。

每次取出两项时都重新比较两个来源的队首，而不是先在一个来源拿齐两项。例如原权`[1,2,4,8]`：

轮	轮初原队列	轮初合并队列	依次取两最小	轮后合并队列
1	`[1,2,4,8]`	`[]`	1、2	`[3]`
2	`[4,8]`	`[3]`	3 来自合并队、4 来自原队	`[7]`

轮	轮初原队列	轮初合并队列	依次取两最小	轮后合并队列
3	[8]	[7]	7、8	[15]

有序性的严格理由是：设本轮取$x \le y$，此前所有未取项都≥y；插回$x+y$也≥y，因为权非负。下一轮取的两个数都≥y，所以新和至少$2y \ge x+y$。因此合并队列尾部追加仍非降。若允许负权，这一步就失去依据，题目的非负频数条件不能省。

用栈存原权时，应让最小项在栈顶，即压入方向相反；换成升序队列也一样。若原权未排序，前面的排序一般 O(nlog n)，因此原题总复杂度仍 O(nlog n)，不能把后半线性当无条件总线性。

26.36 36 · 平均比较排序下界：少数短路径救不了所有排列

互异 n 项共有 n!个等概率排列，每个对应决策树一片可区分叶，深度 di 是比较次数。二叉树满足 Kraft 界 $\sum 2^{-d_i} \le 1$，且函数 2^{-x} 凸，所以

$$2^{-\bar{d}} \le \frac{1}{n!}\sum_{i} 2^{-d_i} \le \frac{1}{n!},$$

得到 $\bar{d} \ge \log_2(n!) = \Omega(n\log n)$。

上式中的 Kraft 界可以不用新术语直接看成空间分配：给二叉决策树根一个长度 1 的区间，每比较一次把当前区间平分给两个孩子。深度d_i的叶占长度2^{-d_i}，不同叶区间不相交，所以总长≤1。设$N=n!$，这 N 个长度的几何平均不超过算术平均，于是

$$\left(\prod_{i=1}^{N} 2^{-d_i}\right)^{1/N} = 2^{-(d_1+\cdots+d_N)/N} \le \frac{\sum_i 2^{-d_i}}{N} \le \frac{1}{N}.$$

两边取以 2 为底的对数再改符号，便得到平均深度$\bar{d} \ge \log_2 N$。最深叶只约束一个输入；这一平均数不等式把所有叶都算进来，因此才能回答本题。

也就是说，哪怕某些输入一眼就能认出，其他分支必须付出足够比较，均匀平均仍不能突破此界。若输入分布高度集中，平均下界应该用其信息量重新描述；若采用整数桶、位运算等超出比较模型的操作，也不能把这条下界照搬过去。

随机算法还需说明平均是在输入随机、内部随机，还是两者共同取期望。证明固定比较树的均匀平均后，对内部随机选择的树再平均，仍能得到相应下界。

26.37 37 · 编码构造下界：限定比较模型，再构造足够多的输出

这里讨论任意未排序正实权重、仅通过比较区分大小、输出带原字符标签的最优树。已排序权重的线性合并没有违反下界，因为排序信息已经作为输入给出了；受限整数模型也可能有其他手段。

可取一组增长极快的固定权重，后一权重远大于前面所有权重之和的 n 倍，再把字符标签任意置换。这时最大权必须在深度 1：让它多一层的损失，大于其他所有叶在至多 n-1 层内调整所能补偿的总量。移去最大叶后对剩余子树重复，最优树被迫呈梳状，依次用深度暴露各权重名次，只有最小两叶可互换。

例如权重 1、10、100、10000，四叶的梳形最优深度为 3、3、2、1。10000 若被放到深度 2，光它一叶就多付 10000；把它换到深度 1 后，其余权总和 111 即使多移动最多 3 层，也补不了这个损失。接着在剩下 3 叶里，100 也被迫比 1、10 更浅，最后 1、10 同深可以互换。把四个字符标签任意分配到这四个权重，树的叶深至少区分$4!/2 = 12$种名次信息。

一般构造要求每个更大权W大于n倍所有较小权之和 S。若最大权不在深度 1，成本至少$2W$；而存在一棵把它放深度 1、其余叶深不超过 n−1 的候选树，成本至多$W+(n-1)S < 2W$，矛盾。去掉这个最大叶后，另一侧子树必须也是剩余权的最优树，再递归使用相同条件。这个候选成本比较补上了“增长足够快为何强迫梳形”的必要一步。

所以至少有 n!/2 类必须区分的带标签输出，二叉比较树需 $\log_2(n!/2) = \Omega(n\log n)$ 深度。也可把问题看成：构造后在线性时间读出深度顺序，便得到这个受限排序族的排序信息。

Huffman 树是最优树的一种，因此同一组强分离输入也约束其构造。不能把“任意树的任意层序遍历一定按所有叶权排序”当成无条件事实；左右孩子可交换，同层叶也可能需要额外安排。下界应建立在明确可区分的信息上。

26.38 本章练习与核对

1. 原树仅 A 左孩子 B。扩充全部空位置后，内部、外部叶各几项？若只给 A 补右孩子，会漏什么？
2. 给先序 A,B 和后序 B,A，画出两棵可能树；加什么条件才能排除这种歧义？
3. 权重 1、2、4、8 做双队列 Huffman，每轮取哪两项，最终 WPL 多少？

第一题：原节点 n=2，扩后内部 2、外部叶 3；A 右边补 1 个，B 左右各补 1 个。仅给 A 补右边会让 B 仍是普通叶，达不到原题“每个原节点成为内部”的计数。第二题：A 左 B 与 A 右 B 均符合；要求真二叉树后，这个双节点输入本身不合法，并非随便选一棵就解决。第三题：1+2=3，3+4=7，7+8=15，WPL=3+7+15=25；叶深 3、3、2、1，直接 1×3+2×3+4×2+8=25 核对。

27 一次完整的调试：从错误输出，找到第一步走歪的位置

算法在纸上正确，不表示你写的程序还在执行同一个算法。这里做三次小实验：抓住二分不前进、矩形宽度多 1、Gift 重复方案被删的问题。输入都很小，错误发生在哪一行可以当场看见。之后再把同一套办法用于链表、树和图。

27.1 先给程序一个可以被反驳的承诺

以 `lower_bound` 为例：输入数组 `a` 已经非降排列，返回下标 p，满足

$$0 \le p \le n;\quad i < p \Rightarrow a_i < x;\quad i \ge p \Rightarrow a_i \ge x.$$

n是数组长度，x是查询值。p 可以等于 n，表示整个数组都小于 x；这时 p 是合法的边界，却不是可以读取的元素位置。空数组返回 0。存在重复时，返回的是第一项不小于 x 的位置。

这个承诺描述答案应该具有什么性质，没有规定必须用二分。因此另一个从左到右扫描的程序就能独立提供答案：看见第一项不小于 x 便返回，否则返回 n。它在大输入上慢，但对长度 6 的测试完全够用，而且不容易和二分共享同一个更新错误。

先把两个常被混淆的接口放在纸上：数组`[2,2,5]`查 2，`lower_bound` 返回 0；课程 C 版查找返回“最后一个不大于 2”的位置 1。两份程序若各自实现这两个不同承诺，输出不同并不说明其中一个算错。测试前必须先固定规格。

27.2 实验一：让错误二分停下来，交出重复的状态

错误更新是 `lo=mid`，持续条件却写 `lo<hi`。用数组`[1,2]`查 3，半开区间初始`[0,2)`：

轮	lo	hi	mid	判定	更新后未知区间
1	0	2	1	2<3，向右	`[1,2)`
2	1	2	1	2<3，向右	`[1,2)`
3	1	2	1	完全重复	`[1,2)`

一个输入就说明“每轮一定缩短”是假的。若测试器只等待返回，它自己也会卡住。调试版可设置步数上限，或者每轮断言新长度严格小于旧长度。这里改为 `lo=mid+1`，第 1 轮后未知区间立即变空`[2,2)`，返回 2，表示没有满足条件的项。

下面是一个完整小程序。`bad` 参数只用来有意注入这一处已知错误；正常版始终用 `mid+1`。`trace` 保存每轮的输入状态，报错时不需要再凭记忆复现。

```
def linear_bound(a, x):
    for i, value in enumerate(a):
        if value >= x:
            return i
    return len(a)


def binary_bound(a, x, bad=False):
    lo, hi = 0, len(a)
    trace = []
    while lo < hi:
        if len(trace) > len(a) + 2:
            raise RuntimeError(trace)
        mid = lo + (hi - lo) // 2
        trace.append((lo, hi, mid))
        if a[mid] < x:
            lo = mid if bad else mid + 1
        else:
```

```
            hi = mid
    return lo


a, x = [1, 2], 3
print("slow:", linear_bound(a, x))
print("good:", binary_bound(a, x))
try:
    binary_bound(a, x, bad=True)
except RuntimeError as error:
    print("bad repeats:", error.args[0])
```

输出的慢答案和正确二分都是 2；错误版会交出`(0,2,1)`后连续重复`(1,2,1)`的轨迹。步数限制只是让一次坏测试有限结束，不能作为二分正确性的数学证明。真正的终止依据仍是：当 lo<hi 时，`lo≤mid<hi`，两条正确分支都使非负整数 `hi-lo` 严格下降。

27.2.1 把一个反例扩成有限域穷举

从值域`{-1,0,1}`生成长度 0 到 6 的所有非降数组，对每个查询`{-2,-1,0,1,2}`比较两版答案。非降数组可用 `combinations_with_replacement` 生成，不必先生成全部乱序数组再排序。长度为 r 时有$\binom{r+2}{2}$个这样的数组；总数为$\binom{9}{3}=84$，每个查 5 次，共 420 次。

```
from itertools import combinations_with_replacement

checks = 0
for length in range(7):
    for values in combinations_with_replacement([-1, 0, 1], length):
        a = list(values)
        for x in [-2, -1, 0, 1, 2]:
            actual = binary_bound(a, x)
            expected = linear_bound(a, x)
            assert actual == expected, (a, x, expected, actual)
            checks += 1
print("bound checks:", checks)
```

420 次全过，准确结论是“在这个有限域里与独立扫描全部一致”。它能抓住很多空区间、重复值、末端返回的错误，但没有穷尽任意长度、任意类型，也没有验证 C++的整数溢出和对象生命期。测试覆盖范围与算法证明各自写清。

27.3 实验二：一根柱就能抓到矩形宽度多 1

直方图每柱宽 1。慢答案枚举每个连续区间`[l,r]`，从左向右延长时维护最小高度，面积是最小高度乘 `r-l+1`。快版弹出一根柱时，当前下标 i 是右侧不可用位置，弹后栈顶 left 是左侧暂时不能跨过的位置，因此可用格为 `left+1` 到 `i-1`，宽应为

$$i-left-1.$$

用只有一根高度 2 的柱，末尾补 0。原柱下标 0 被弹出，i=1，left=−1，可用位置只有 0，宽$1-(-1)-1=1$，面积 2。若漏减 1，会算出宽 2、面积 4。这个反例比一屏随机柱子更能让人看到“多出来的是哪一格”：右边下标 1 的零柱不该包括，左边-1 也不是真柱。

慢程序可以直接写成：

```
def rectangle_slow(heights):
    best = 0
    for left in range(len(heights)):
        minimum = heights[left]
        for right in range(left, len(heights)):
            minimum = min(minimum, heights[right])
            best = max(best, minimum * (right - left + 1))
    return best
```

先手验几组有不同目的的数据：

输入	答案	专门检查什么
`[]`	0（若接口允许空）	不读首元素
`[2]`	2	左右边界不多 1
`[2,2]`	4	等高柱如何处理
`[2,1,2]`	3	最矮柱覆盖整段
`[1,2,3]`	4	输入结束要结算未弹柱
`[3,2,1]`	4	一轮连续弹多根
`[0,0]`	0	面积 0 与并列边界

若快程序还要返回“面积相同取最靠左，仍相同取最短”的区间，慢程序也应枚举并比较`(-area,left,right)`。只比较面积会漏掉位置错误；只比较位置也可能漏掉面积错误。测试对象的承诺有几部分，验证器就查几部分。

可以再枚举长度至多 6、高度 0 到 3 的全部数组，与慢答案比较。枚举范围增大前，先故意把快版宽度改错，确认测试立即拒绝`[2]`。若错误版仍全绿，应该先检查测试是否真的调用了快版、失败是否被吞掉、预期答案是否误用同一个函数算出。

27.4 实验三：重复的和，为什么不是重复的方案

Gift 有两组`(1,1)`、`(2,2)`。每组仍有“选左”“选右”两个动作，四个方案为：

选择身份	总和
左、左	3
左、右	3
右、左	3
右、右	3

预算 3 时答案 4，预算 2 时答案 0。若优化版把左右半和都放进集合去重，会把`[1,1]`变成`[1]`、`[2,2]`变成`[2]`，只数到 1。这次错误不是二分边界，也不是整数算错；它发生在表示法丢掉了选择身份。

独立慢答案枚举每个二进制选择串，对每组按相应位取值，再求和。对小组数，可以把全部预算从“最小总和减 1”测到“最大总和加 1”。优化版的输出不仅要等于慢答案，还应满足以下可单独核验的关系：交换某组的左右两个选项，方案总数不变；增大预算，答案不减；把每个选项都加 c，并把预算加nc，答案不变。负数还可以专测错误剪枝。两组`(5,6)`、`(-10,0)`，预算 0，合法总和−5、−4，共 2 种。如果第一组刚取 5 或 6 就因“部分和超预算”剪枝，会把两个合法方案都删掉。只有能证明后面不可能降低总和时，这种剪枝才安全。

27.5 链表错误要保存操作历史，不能只保存最后一组值

从 `H↔A↔B↔T` 把 B 移到 A 前，预期 `H↔B↔A↔T`。验证不能只正向读出 B、A，还应从 T 逆向读出 A、B，并检查每对相邻节点互相承认对方：`p.succ.pred==p`、`p.pred.succ==p`。若忘了改 A.pred，正向内容可能正确，逆向或后续删除才出错。

保存外部句柄 p 指向 A，再做节点移动、值交换、删除重建，这三个操作的句柄承诺不同。调试日志应记下节点 ID、链接 ID、size 以及操作序列。只保存最终集合{A,B}无法复现“先移动再删除”才触发的错误。

发现 100 步操作后失败，可以试删去前半操作；若仍失败，保留删减，继续缩小。每次删减都要确认前置条件仍合法，且独立验证器仍报告同一种规格失败。逐步缩成 3 步的反例，比猜 100 行日志容易得多。

随机种子有助复现，但最终还应保存完整失败输入；生成器版本和调用次序变了，种子相同也可能得到不同序列。

27.6 整数范围先算中间式，再决定类型

逆序数最多$n(n-1)/2$。n=100000 时最大为 4999950000，超出 32 位有符号整数。下面两句不是等价的：

```
long long wrong = n * (n - 1) / 2;  // n若为int，乘法可能先溢出
long long right = 1LL * n * (n - 1) / 2;
```

宽变量只能保存右侧计算完成后的值，不能追溯修复窄乘法。叉积的两个大乘积最终相减成小数，乘积仍可能先溢出；带状 DP 的 `inf` 若取最大整数，再无条件加 1，也可能回绕成负数，伪装成更优解。

内存也按具体对象计算。一个 10000×10000 的 32 位表是 4 亿字节，约 381MiB，还没计行对象、指针、输入和递归栈。滚动两行能省表，但若每行仍初始化一万个格，时间成本仍要算。扩容迁移时旧数组与新数组暂时同时存在，峰值内存也应计双份。

27.7 图算法的输出，怎样交出可验证的证据

最短路程序除距离 dist，还可以保留前驱 parent。例：边 S→A 权 2、S→B 权 5、A→B 权 1。若输出 dist[B]=3、parent[B]=A、parent[A]=S，沿前驱恢复 S→A→B，逐边相加为 3，能证明 3 确实是一条可行路径的长度。再检查每条边的松弛不等式$dist[v] \le dist[u] + w(u,v)$，说明没有任何边能进一步改善。

这两类检查结合才有力度。只查不等式，给所有点写 0 在非负图上也可能蒙混过关；它们未必都由源点可达路径产生。只查路径，可接受长为 5 的 S→B，虽然另一条路径长 3。还要检查不可达点的无穷表示，前驱不能形成环，源点的边界也应正确。

MST 类似：先查所选边连通、无环、边数为 V−1，再在小图中枚举所有生成树比较最小权。只看总权巧合相同，可能接受不连通边集；只看是一棵树，又不能证明权最小。

27.8 性能实验要比较真实工作量

测 n 与 2n，线性、平方、排序的理论操作量大致分别乘 2、4、略多于 2。实际墙钟时间还受分配、缓存、解释器启动、系统调度影响。先用操作计数确认增长，再用固定环境、相同输入副本、重复测量检验实际延迟。数据生成和打印应移出计时区，计算结果必须被使用，避免优化器把被测循环删掉。

输入形状要能逼出不同路径：BST 递增插入会变长链；快排全相等可暴露错误分区；散列表大量墓碑可能让失败查询变慢；带状 DP 要测长度差恰好 k 与 k+1；单调栈测严格递增与严格递减。不要把“随机数据上很快”写成最坏保证。

本书已有的`示例/算法检查.py`、`示例/复杂度计数.py`与`示例/习题核对.py`支持若干具体算法及算例，它们不是课程 OJ 验收，也没有证明全部未纳入的算法。发布时应保留它们的范围和实际输出，不把一个绿色数字当作读者已经理解的证据。

公开复现数据使用本章这些另设小例。带个人身份的日志、个人提交与受限样卷仍保留在本地原处，不因调试方便混进公开仓库。

27.9 自己试做

1. 设某个 lower_bound 程序返回 p=1，数组`[2,2,5]`、查询 2。它满足下标合法，为什么仍应被拒绝？
2. 对快矩形算法，`[2,2]`结果只报面积 2。怎样写最短的状态说明，帮助作者看出等高处理的问题？
3. AVL 中序已经有序，还要检查哪些结构性质？举一项中序遍历查不到的错误。

答案：

1. p 之前的下标 0 存 2，不满足“p 之前所有值严格小于 2”。验证左半性质就能拒绝，不能只查`a[p]>=x`。

2. 补 0 时应让右边 2 先算宽 1、面积 2，再让左边 2 算宽 2、面积 4。若只留下右边 2 却没保存最左起点，宽度会丢失；若保持两个等高柱，必须继续结算左边那根。提交两次弹栈的 `i,p,left,width` 即可定位。
3. 每个节点高度差至多 1、缓存 height/size 正确、parent 与孩子互指、无环、节点数与所有权正确。例如某个孩子 parent 指向错误，中序递归不读 parent，仍可能完全正确，后继查询却会走错。

28 把证明里省掉的那一步写出来

读到“平均对数”“势能抵消”“按归纳假设”，很容易把结论记住，却不知道自己能不能重做。本章从实际状态和小样本开始：先说明数的是什么，再把符号放上去。前半的摊还与期望适合直接读；“全程峰值”一节用到指数、尾概率，可以在理解固定时刻的例子以后再进入。

28.1 摊还分析：先做完一段操作，再摊它的总账

摊还不需要掷硬币，也不假设输入随机。它要证明：无论合法操作怎样安排，一整段的总工作量都受控制。某一次很贵完全可以。

双栈队列有 in 和 out 两只栈，均从底到顶书写。in 接新项，出队只弹 out；out 空时才把 in 倒过去。把一次基本 push 或 pop 各计 1，空检查等固定开销暂时单列：

操作	in	out	实际成本
初始	[]	[]	—
入 A	[A]	[]	1
入 B	[A,B]	[]	1
出 A	[]	[B]	5：弹 B 压 B、弹 A 压 A、再弹 A
出 B	[]	[]	1

总成本 8，第三次单独花 5。每个元素一生最多做 4 个栈动作：进 in、离 in、进 out、离 out。所以 m 次合法队列操作总成本$O(m)$。元素如果还没出队，实际工作更少；即使预先给它收齐 4 份费用，也仍是上界。

28.1.1 为什么要人为定义“势能”

前两次便宜的入队，可以替第三次倒栈预留费用。给一个状态定义

$$\Phi = 2|\text{in}|.$$

竖线表示 in 中的元素个数，前面的 2 表示每个尚未倒过去的元素，还需要一次 pop 和一次 push。Φ是待付工作的记账量，不是额外真的执行这些动作。

设第i次操作花的实际成本为c_i，操作后势能为Φ_i，操作前为Φ_{i-1}。定义第i次摊还成本

$$\hat{c}_i = c_i + (\Phi_i - \Phi_{i-1}).$$

势能增加，表示这次除支付实际工作，还存下未来要用的费用；势能减少，表示这次从已有存款里支付了一部分。对上面 4 步，逐列算：

操作	c_i	操作前后Φ	势能变化	$\hat{c}_i$
入 A	1	0⟶2	+2	3
入 B	1	2⟶4	+2	3
出 A	5	4⟶0	−4	1
出 B	1	0⟶0	0	1

实际合计 8，摊还合计也 8。若做完“入 A、入 B”就停止，实际只花 2，摊还收 6，还剩势能 4；收得多一些不会破坏上界。

把 m 步的定义逐个相加，可以亲眼看到抵消：

$$\begin{aligned}\hat{c}_1 + \hat{c}_2 + \cdots + \hat{c}_m &= c_1 + c_2 + \cdots + c_m \\ &\quad + (\Phi_1 - \Phi_0) + (\Phi_2 - \Phi_1) + \cdots + (\Phi_m - \Phi_{m-1}) \\ &= \sum_{i=1}^{m} c_i + \Phi_m - \Phi_0.\end{aligned}$$

每个中间Φ_i有一次加、一次减，只剩初态与末态。若初态为 0、所有状态势能非负，实际总成本就不超过摊还总成本。如果允许势能跌到任意大的负数，它就像无限透支账户，不能用小额收费控制实际账。

把 3 类队列操作分别代入也能普遍核验：入队$c=1,\Delta\Phi=2$，摊还 3；out 非空出队$c=1,\Delta\Phi=0$，摊还 1；out 空、in 有 k 项时，$c=2k+1,\Delta\Phi=-2k$，摊还仍 1。这里 k 再大也消掉了，所以总界对任何合法序列成立。

这种从二进制计数器出发、依次用总和、记账、势能解释同一成本的组织，可对照 Jeff Erickson Amortized Analysis，第 1–5 页。本章改用本课双栈队列和下面的向量状态计算，便于直接接上课程实现。

28.1.2 向量加倍：一个完整的非负势能

只考虑尾部追加，初始容量 1、规模 0，满时容量翻倍。令 n 是当前有效元素数、C 是当前容量，取

$$\Phi(n,C)=\max(0,2n-C).$$

这个定义在空表也非负。非扩容追加只写新项，成本 1；势能最多增加 2，摊还至多 3。扩容前 n=C，势能 C；搬 C 项、写 1 项，成本 C+1；扩容后规模 C+1、容量 2C，新势能 2。因此摊还为$(C+1)+(2-C)=3$。

第几次追加	追加后 n,C	本次实际写入	操作后势能	本次摊还
0	0,1	—	0	—
1	1,1	1	1	2
2	2,2	2	2	3
3	3,4	3	2	3
4	4,4	1	4	3
5	5,8	5	2	3

前 5 次实际 12，摊还 14，差 2 正好等于末态势能。势能函数不是猜出后便自动正确，必须检查初态、便宜操作、昂贵操作和下界。若允许头插、批量插入、缩容，这个只分析尾追加的证明不能直接覆盖那些新操作。

28.2 概率与期望：先列“可能发生什么”

随机变量是给每个可能结果算出的一个数。公平硬币抛两次有 4 个等可能结果：正正、正反、反正、反反。令 X 为正面数，四个结果对应 2、1、1、0，所以

$$E[X]=(2+1+1+0)/4=1.$$

E读作期望，意思是按各结果的概率加权平均，不是“最可能发生的值”，也不保证某次结果正好等于它。一般写成

$$E[X]=\sum_{x} x\,P(X=x),$$

即把 X 可能取的每个值 x 乘出现概率，再加起来。上例也可按数值合并：X=0 概率 1/4，X=1 概率 1/2，X=2 概率 1/4，得到$0/4+1/2+2/4=1$。

28.2.1 指示变量为什么省事，为什么不用先证明独立

令I_1表示“第一次正面”，发生记 1，否则 0；I_2同理。逐个样本上都成立$X=I_1+I_2$。对于任一 0/1 变量 I，

$$E[I]=0\cdot P(I=0)+1\cdot P(I=1)=P(I=1).$$

因此$E[X]=E[I_1]+E[I_2]=1/2+1/2$。把期望分配到加法上，称为线性期望。它不要求独立，因为在加权求和里只是交换两次求和次序。

相关的例子更能说明这一点：公平掷一颗六面骰，X 为顶面点数，Y 为底面点数，故 Y=7−X。它们显然相关，X 变大时 Y 一定变小；但$E[X+Y]=7=3.5+3.5$仍成立。不能因为“可以把期望相加”就误断独立。

独立性是在乘积时另需的条件。例如只有独立时才能一般地写$E[XY]=E[X]E[Y]$。顶底骰子这里，$E[XY]=(6+10+12+12+10+6)/6=28/3$，而$E[X]E[Y]=49/4$，并不相等。相关条件是会在数值上产生差别的。

这些基础定义与模型可对照 Jeff Erickson Discrete Probability，第 5—9 页。原讲义把几何变量定义成“成功前失败了几次”；下面按课程常见习惯数“包含成功的总轮次”，因此期望相差 1。

28.2.2 几何分布：平均 2 轮，为什么仍可能失败很多轮

每轮独立、成功概率固定$p>0$，K 表示首次成功的轮次，成功那轮也计入。K=k 意味着前 k−1 轮失败、第 k 轮成功，因此

$$P(K=k)=(1-p)^{k-1}p,\qquad k=1,2,\dots.$$

例如$p=1/2$，最短三类运行是：成功，概率 1/2、成本 1；失败成功，概率 1/4、成本 2；失败失败成功，概率 1/8、成本 3。还存在更长的运行，不能在第 3 类截掉后便说列全了。

求平均轮数可以按“这一轮还需要执行吗”计数。任意整数 K=k 都可拆成 k 个 1；对所有样本取平均，便有

$$E[K]=\sum_{j=1}^{\infty}P(K\ge j).$$

第 j 轮会发生，当且仅当前 j−1 轮全失败，概率$(1-p)^{j-1}$。于是

$$E[K]=1+(1-p)+(1-p)^2+\cdots=\frac{1}{p}.$$

最后一步是等比级数：把整个和乘$(1-p)$再从原和减去，只剩 1，故 p 乘原和等于 1。$p=1/2$时平均 2 轮，但前 20 轮全失败仍有2^{-20}的正概率。期望有限没有给出统一最坏轮数。

若程序最多做 3 轮就停止，实际轮数是$\min(K,3)$，平均$1+(1-p)+(1-p)^2$，失败概率$(1-p)^3$；它保证有限结束，却允许报告失败。若每轮p变化或试验相关，也不能继续把失败概率写成同一个幂。

28.3 随机排列的环：为什么最少交换平均是$n-H_n$

排列`[2,3,1,5,4]`表示位置 1 的元素应该去位置 2，位置 2 去 3，位置 3 去 1，另有 4 和 5 互换。因此环为`(1 2 3)`、`(4 5)`，共 c=2 个。

```
[2,3,1,5,4]  交换4、5 -> [2,3,1,4,5]
               交换2、3 -> [2,1,3,4,5]
               交换1、2 -> [1,2,3,4,5]
```

共 3 次，正好$n-c=5-2$。任何一次交换最多让环数增加 1，目标排列有 n 个单点环，所以至少要 n−c 次；逐环拆又恰好能达到，故这是真正的最小值。

现在才加入随机模型：n 个互异标签的 n!个排列等可能。按标签 1、2、...构造环，加入第 k 个标签时有 k 种选择：让它单独成一个新环，或把它插入已有 k−1 个标签中的某个标签之后。每种最终排列可以唯一逆向恢复这些选择：删去最大标签 k，若它原来独立，就记“新环”；否则记下它原来的前驱，再把环接回。共有$1\cdot 2\cdots n=n!$组选择，与排列一一对应。

令J_k表示第 k 步是否选“新环”，是则 1、否则 0。k 种等可能选择中只有一种新增环，所以$E[J_k]=1/k$。最终环数是$J_1+\cdots+J_n$，线性期望给

$$E[c]=1+\frac{1}{2}+\cdots+\frac{1}{n}=H_n,\qquad E[n-c]=n-H_n.$$

H_n叫第 n 个调和数。n=3 时六个排列的环数分别为：恒等 123 有 3 环；213、132、321 各 2 环；231、312 各 1 环。平均$(3+2+2+2+1+1)/6=11/6=H_3$，最少交换平均为$3-11/6=7/6$。小样本可以核对一般推导，而不是替代一般证明。

28.4 单调栈某一时刻：从右往左看，谁还没有被淘汰

递增单调栈遇到更小的新高度就弹旧项。假定高度互异、相对次序均匀随机。处理完前 r 项后，一个旧项还在栈中，当且仅当它小于自己右边所有已处理项；否则右边的某个更小项早已把它弹掉。留下的是“从右往左看到的新最小值”。

例输入`[4,2,5,1,3]`：

```
读4: [4]
读2: 弹4，剩[2]
读5: [2,5]
读1: 弹5、弹2，剩[1]
读3: [1,3]
```

从右看原数组，先看到 3，是记录；再看到 1，比 3 小，是新记录；5、2、4 都比已经见到的 1 大，不是记录。栈恰好留下 1、3。

在长度 j 的后缀中，最左那项成为最小值的概率是 1/j。将每一项“是否留下”写成 0/1 变量相加，得到固定时刻栈长S_r的期望

$$E[S_r] = H_r = \Theta(\log r).$$

这里不需要独立。r=3 时六种排列栈长为 3、2、2、1、2、1，平均 11/6；但严格递增的一份输入还是会留下 3 项，概率平均没有删除最坏输入。

28.4.1 全程峰值为什么还要另证

令$M = \max(S_1, ..., S_n)$。它是每次完整运行中出现过的最大栈长，先在同一样本里取最大，再对不同样本取平均。不能把 E 随意搬到 max 里面。

一个只有两个样本的反例：一半概率$(X, Y) = (1, 0)$，另一半为$(0, 1)$。X、Y 各自平均 1/2，但每个样本里的最大值恒为 1，所以$E[\max(X, Y)] = 1$。知道每个时刻平均小，还没有排除不同运行各自在不同时间出现大峰值。

下面给全程峰值的真正上界。若只需要最坏空间，看到$M \le n$已经足够；以下是随机模型中的更细结论。

28.4.2 第一步：先证明记录指标独立

把一个 r 项均匀排列从右向左读。第 j 次读到的项，在已经读过的 j 项中的名次记为R_j，最小名次是 1。成为新最小记录正好是$R_j = 1$。名次变量不是原数值，只比较相对大小。

先对三个数从左读的版本列全（反向读仍同样均匀）：

排列	R_1, R_2, R_3
1,2,3	1,2,3
1,3,2	1,2,2
2,1,3	1,1,3
2,3,1	1,2,1
3,1,2	1,1,2
3,2,1	1,1,1

所有“第 1 名只能 1、第 2 名可 1 或 2、第 3 名可 1 或 2 或 3”的组合恰好各出现一次。

一般也是如此：给定一组合法名次，从 j=r 倒着恢复。在尚未使用的有序标签中取第R_j小的，放到第 j 个读入位置，然后删掉它。剩下的标签相对次序不变，继续恢复$j-1$。例如名次[1,2,1]：从{1,2,3}取第 1 小即 1 放第 3 位；余{2,3}取第 2 小即 3 放第 2 位；最后 2 放第 1 位，恢复 231。

合法名次元组共$r!$个，排列也$r!$个，且一一对应、等可能，所以每个元组的概率是

$$\frac{1}{r!} = \frac{1}{1} \cdot \frac{1}{2} \cdots \frac{1}{r}.$$

这恰等于每个分量取其名次的概率之积，证明所有R_j相互独立、各自在 1 到 j 中均匀。于是$B_j = [R_j = 1]$也独立，概率 1/j。一个固定前缀从右读，栈长便是$S_r = \sum_j B_j$。仅说每项概率 1/j，本来不足以证明独立；这里的一一对应补上了缺口。

28.4.3 第二步：用“指数后的平均”控制大值概率

取自然常数$e \approx 2.718$。对 0/1 变量B_j，e^{B_j}只能为 1 或 e，因此

$$E[e^{B_j}] = (1-1/j)\cdot 1 + (1/j)\cdot e = 1 + (e-1)/j.$$

又因为$e^{S_r} = \prod_j e^{B_j}$，独立性允许乘积的平均拆成平均的乘积：

$$E[e^{S_r}] = \prod_{j=1}^{r}\left(1 + \frac{e-1}{j}\right).$$

使用$1 + x \le e^x$，每个因子不超过对应指数，乘起来就是

$$E[e^{S_r}] \le e^{(e-1)H_r} \le Cr^{e-1}, \qquad C = e^{e-1}.$$

最后一步用$H_r \le 1 + \ln r$；常数 C 不随 r 增长。它把所有大栈长按指数放大后，平均仍受一个多项式控制。非负变量 Z 满足$P(Z \ge a) \le E[Z]/a$：发生事件的那部分样本每个至少贡献 a，所以整个平均至少$aP(Z \ge a)$。这就是 Markov 不等式的完整理由。取$Z = e^{S_r}$、$a = e^t$，得到

$$P(S_r \ge t) \le Cr^{e-1}e^{-t}.$$

28.4.4 第三步：把所有时间加进来，再用尾和求平均

M≥t 表示至少一个时刻r有$S_r \ge t$。把这些事件的概率相加可能重复计算，却一定是上界；这一步不需不同时间独立。因此

$$P(M \ge t) \le \sum_{r=1}^{n} P(S_r \ge t) \le Cn^e e^{-t}.$$

当然概率至多 1，实际取二者的较小值。设$n \ge 2$，选整数阈值

$$t_0 = \lceil (e+1)\ln n + \ln C \rceil.$$

那么$Cn^e e^{-t_0} \le 1/n$。前t_0个尾概率每个最多 1，后面的概率每增加 1 格就至少乘e^{-1}，所以

$$E[M] = \sum_{t\ge 1} P(M \ge t) \le t_0 + \frac{1}{n}\sum_{s\ge 1} e^{-s} = O(\log n).$$

另一方面每份运行都有$M \ge S_n$，取平均仍得$E[M] \ge H_n = \Omega(\log n)$，上下合为$\Theta(\log n)$。n=1 单独为 1，不影响渐近结论。高度重复、相关或刻意排好时，独立相对排列模型不再成立；最坏峰值仍可以线性。

28.5 一扇固定窗口，与无限期运行的最大队列

宽度 w 固定，值独立、连续、同分布，单调最大队列影子留下的是后缀最大记录，和前面的最小记录对称。固定时刻平均候选数为H_w。

w=3 时窗口 `[3,2,1]` 会留下全部 3 项；这类严格递减顺序的概率为 1/6。取 q 个互不重叠窗口，它们独立。一次都没有递减的概率是$(5/6)^q$；至少遇到一次的概率是$1-(5/6)^q$。q 越大，后者越接近 1，因此长期峰值达到 3 的概率趋 1。

一般 w 的递减排列概率为$1/w!$，相同计算给$1-(1-1/w!)^q$。所以固定 w、观察时间越来越长，峰值趋向 w，而不是一直保持平均$\log w$。若原始队列本身也存了 w 项，整个结构更本来就需要$\Theta(w)$空间；“影子候选平均很少”只是在说额外索引。

28.6 平均比较排序下界：把每片叶子占的空间相加

n 个互异元素有$n!$种排列，若均匀出现，每种概率$1/n!$。比较决策树每次问一个两分结果，叶深d_i就是第 i 种输入的比较次数。给根一条长度 1 的区间，每往下比较一次，区间平分。第 i 片叶占2^{-d_i}长度，不同叶不重叠，所以

$$\sum_{i=1}^{n!} 2^{-d_i} \le 1.$$

这叫 Kraft 界，在这里就是“所有叶区间放得进长度 1”。令$N = n!$，这 N 个正长度的几何平均不大于算术平均：

$$2^{-\bar{d}} = \left(\prod_i 2^{-d_i}\right)^{1/N} \le \frac{1}{N}\sum_i 2^{-d_i} \le \frac{1}{N},$$

其中$\bar{d} = (d_1 + \cdots + d_N)/N$。取以 2 为底的对数、再把负号移过去，得到$\bar{d} \geq \log_2 N = \log_2(n!) = \Omega(n \log n)$。

n=3 时，有一棵合法比较树的 6 片叶深为 2、2、3、3、3、3。平均是 8/3，约 2.667，大于$\log_{26} \approx 2.585$；平均并不必等于最坏值 3。只证明某一片叶很深，约束不到平均；刚才的长度总和把所有叶一起控制住了。该结论限定一般比较模型与均匀互异排列。整数桶利用关键字位值，已超出仅比较模型；输入几乎总是某个已知顺序时，分布也不同，不能不改条件地引用同一个平均下界。

28.7 终止与归纳：选的量必须真的覆盖完整状态

“每次变小”要指出哪个非负整数变小。二分可用 hi−lo，树递归可用子树节点数。沿树的一条左藤归纳时，外挂右子树可能很大，只证明藤长减少，没有保证侧树的过程正确；按整树规模归纳更容易覆盖它们。Failstone 甚至不是每一步最大值都减小。正整数状态`(3,5)`会先变`(6,5)`，最大值还增加了。把后续三步一起看：

```
(3,5) -> (6,5) -> (1,6) -> (1,3)
```

阶段端点的最大值由 5 降到 3。一般在 p<q、q 为奇数时，有限次翻倍会经过$p^* < q < 2p^*$，随后到$(2p^* - q, p^*)$，两项都小于 q；q 为偶数时减半可直接降低最大值。每个阶段有限，阶段端点的正整数最大值严格下降，才证明整个过程有限。p=0 时不能沿用“翻倍总能超过 q”，这正是要写正整数条件的理由。

28.7.1 对数尺度换元，为何会多出一个$1/u$

第 22 章 Akra–Bazzi 证明把规模 u 改成$s = \ln u$，这样每次乘固定比例就变成减固定距离。例如 u 从 27 到 9，s 减少$\ln 3$；从 27 到 18，s 减少$\ln(3/2)$，都与当前 u 无关。

连续微小变化满足$ds = du/u$：这来自$\ln u$的导数是$1/u$。若归一化后的工作为$h(u) = g(u)/u^p$，在 s 轴上累加面积就是

$$\int h(e^s)\, ds = \int \frac{g(u)}{u^p} \frac{du}{u} = \int \frac{g(u)}{u^{p+1}}\, du.$$

p来自递归树规模权重，多出来的 1 来自换元，不是定理中随手加的指数。例$p = 1, g(u) = u$时，面积是$\int du/u = \ln u$，乘回规模 n 便给$n \log n$。这一步只负责解释积分形式，离散节点和面积之间的夹逼仍需第 22 章明确写出的正则条件。

28.8 遮住答案，再做几笔账

1. 双栈队列只做入 A、入 B、入 C 就停止，实际、势能和摊还总账分别多少？
2. 公平硬币抛两次，X 为正面数、Y 为反面数。X 与 Y 独立吗？期望相加是否仍成立？
3. 对排列`[3,1,2]`求环数、最少交换数、完整单调栈的各步长度和峰值。
4. 为什么固定时刻记录概率 1/j 还不足以直接把指数期望写成乘积？

答案：

1. 实际 3，末势能 6，摊还每次 3、合计 9；关系为$3 = 9 + 0 - 6$。未来即使永远不出队，这 6 份未用费用仍只是上界里的余款。
2. 不独立，因为 Y=2−X，X=2 会强迫 Y=0；但$E[X + Y] = 2 = 1 + 1$，线性期望不要求独立。
3. 箭头 1→3→2→1 是一个三环，最少$3 - 1 = 2$次交换。单调递增栈处理 3 得到`[3]`，处理 1 弹 3 得`[1]`，处理 2 得`[1,2]`，长度 1、1、2，峰值 2。
4. 单个概率只给每个变量的边缘分布，没有给联合分布。两个变量可以始终同时发生，也可以互相排斥，单个成功概率却一样。相对秩与排列的一一对应证明所有名次组合等可能，才提供拆乘积需要的联合独立性。

29 题目没有告诉你用什么结构时，先算一个小输入

下面八题都先确定操作和答案，再选择要维护的状态。读到一个步骤时，可以盖住下一行，自己画一次数组、树或队列；旁边的状态不是装饰，它应当能用来查出自己的第一处错误。

29.1 订单按到达顺序处理，还要随时知道最大金额

输入操作为：入 A 金额 5、入 B 金额 2、查最大、出单、入 C 金额 7、查最大、出单、查最大、出单。出单必须先进先出。若只用普通队列，每次查最大要扫当前所有订单，重复查询可能二次。

用两个栈组成队列，每个栈格保存(订单,金额,从栈底到此格的最大金额)。这样一只栈的最大就是顶格附带的摘要，弹出后新栈顶自动给出剩余部分的最大，无需重扫。

in 接收新单，out 负责出单，只有 out 为空时才倒栈。每个格以下简写为订单:金额/本格最大，均从底到顶书写：

操作	in	out	返回
入 A5	[A:5/5]	[]	—
入 B2	[A:5/5,B:2/5]	[]	—
查最大	同上	空	5
出单	[]	倒完为[B:2/2,A:5/5]，再弹 A，剩[B:2/2]	A
入 C7	[C:7/7]	[B:2/2]	—
查最大	同上	同上	max(7,2)=7
出单	[C:7/7]	[]	B
查最大	[C:7/7]	[]	7
出单	[]	C 倒入后再弹，清空	C

为什么 in 里新来的 C 不会越过旧 B？out 非空时始终先用 out，不因为新单到来而重倒。倒栈又把 in 的后进先出反转一次，使最早入 in 的单处于 out 顶。两只栈各自的最大取 max，恰好覆盖当前队列全部元素；空栈的摘要用“无值”表示，别用 0，否则全负金额模型会误报 0。

每项至多进 in、离 in、进 out、离 out 各一次，所以 m 次操作总$O(m)$，查询最大最坏$O(1)$，空间与当前订单数同阶。某一次出单倒很多项仍可能线性，这个方案给摊还保证；要求每次都严格常数延迟的系统，需要另外的逐步迁移方案。

29.2 成绩序列：两个 2 能不能都选，先看等号

记录按到达顺序为[3,1,2,2,4]，允许跳过，禁止重排。找最长不下降子序列时，1、2、2、4 合法，长度 4；找严格上升时不能连两个 2，答案长度 3。

先写慢 DP。设f_i表示必须以第 i 项结束的最佳长度，初值 1；每个前驱 j 需满足 j<i 且$a_j \le a_i$。把所有合法前驱的f_j+1取最大：

i / 值	合法前驱	当前f_i	一条达到它的链
0 / 3	无	1	3
1 / 1	无	1	1
2 / 2	下标 1 的 1	2	1,2
3 / 2	下标 1 的 1、下标 2 的 2	3	1,2,2
4 / 4	前面全部	4	1,2,2,4

每项扫前面全部，时间$O(n^2)$。若把≤改<，下标 3 不能接 2，得到$f=[1,1,2,2,3]$，精确体现题意差别。

加速时，`tails[l-1]`只保存“已经见过的、长度为 l 的不降子序列中，最小的末尾值”。长度相同，末尾越小，未来可接的新值至少不会更少。新值 x 应接在所有末尾≤x 的长度后面，因此找第一个**严格大于 x** 的位置，使用 upper_bound：

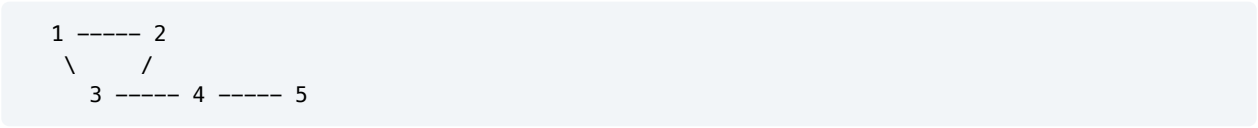
```
读3：tails=[]，位置0，追加        -> [3]
读1：首个>1是下标0，替换          -> [1]
读2：没有>2，位置1，追加          -> [1,2]
再读2：没有>2，位置2，追加        -> [1,2,2]
读4：没有>4，位置3，追加          -> [1,2,2,4]
```

为什么替换不损失长度？替换位置 p 时，之前长度 p 的某条链末尾≤x，可以把 x 接上形成长度 p+1；把旧末尾换成更小的 x，只改善未来接续条件。其他长度的最优摘要保持不变。用二分每项$O(\log n)$，总$O(n \log n)$。

`tails` 本身未必是一条真实子序列。例如读`[2,3,1]`后 tails 为`[1,3]`，但 1 出现晚于 3。要恢复答案，再存每项的前驱 `parent[i]`，和每个长度的代表下标 `end[p]`。新项落 p 时，前驱是更新前的 `end[p-1]`（p=0 则无）；然后更新 `end[p]=i`。本例第二个 2 的前驱是第一个 2，4 的前驱是第二个 2，最终沿 4⟶2⟶2⟶1 回溯再反转。只求长度与恢复链需要的信息不同。

29.3 道路网络：哪个路口一删，连通块就增多

无向简单图边为 1—2、2—3、3—1、3—4、4—5。前面是一个三角形，后面接一条链：



慢办法逐个删除顶点后重跑连通性。一次 DFS 可同时找割点与桥：`dfn[v]`记首次发现次序；`low[v]`记 v 子树沿树边向下、最多再用一条回祖先边，能到达的最小发现号。只看“有没有回边”不够，还要知道能绕到父亲之上哪一层。

固定邻居访问顺序，使 DFS 树走 1⟶2⟶3⟶4⟶5：

事件	dfn/low 发生的变化
发现 1、2、3	各自 dfn 为 1、2、3，low 先等于自己 dfn
3 看到已访问祖先 1	`low[3]=min(3,dfn[1])=1`
发现 4、5	dfn 为 4、5，low 初值 4、5
5 返回 4	没有回边，low5=5；5>dfn4=4，所以 4—5 是桥；5≥4，所以非根 4 是割点
4 返回 3	low4=4；4>dfn3=3，所以 3—4 是桥；4≥3，所以 3 是割点；low3 仍 min(1,4)=1
3 返回 2	low3=1<dfn2=2，子树可绕过 2 到 1，2 不是割点；low2 改 1
2 返回 1	根 1 只有一个 DFS 树孩子 2，所以不是割点

根虽然与 2、3 都相邻，但 3 在 2 的递归过程中已经被发现，不能把两条邻边当两个 DFS 树孩子。最终割点 3、4，桥 3—4、4—5。割点条件用≥，桥用>：若孩子能回到父亲本身，边不一定是桥，但删掉父亲仍可能断开子树。

每条无向边扫描常数次，时间$O(V+E)$。若有平行边，要按边 ID 识别“刚才来的父边”，否则把另一条回父边一并跳过会误报桥。这个简单例子的证明不能靠省掉多重边约定来冒充一般实现。

29.4 反复插入、删除以后，仍要能查第 k 小

依次插 8、3、10、1、6，重复值暂不允许，查询第 3 小；再删 3，再查第 3 小。只用普通散列表找不到全局秩；可以用平衡 BST，并给每个节点额外保存子树节点数 size。

插入后的合法 AVL 形状可以是：

```
       8(size=5)
      /         \
  3(size=3)     10(size=1)
   /     \
1(1)    6(1)
```

查询节点 v 时，左子树大小记 L。按中序，左边恰占前 L 名，v 自己是 L+1 名；目标 k≤L 就去左边，k=L+1 返回 v，否则去右边，并把 k 减去 L+1。

当前节点	左大小 L	当前 k	决定
8	3	3	3≤3，去左
3	1	3	越过左边 1 项和根，k 改 1，去右
6	0	1	1=L+1，返回 6

删 3 时用其后继 6 替换并删除原 6 节点，左子树变根 6、左孩子 1。原根 8 的左大小从 3 变 2，所以第二次查询 k=3，直接在 8 返回。全序也确实由 `1,3,6,8,10` 变为 `1,6,8,10`。

维护式是 `size(v)=1+size(left)+size(right)`，空子树 size=0。每次修改或旋转都要更新；旋转先更新变低的节点，再更新新根，否则新根会读到旧摘要。高度没变时可以停止高度传播，size 却可能继续变，不能一起停止。平衡树高度$O(\log n)$，所以四种接口均可对数；k 必须在 1 到当前 size 之间，空树没有第 1 小。

29.5 矩形内的点：计数摘要与输出全部是两种任务

点为(1,1)、(2,4)、(3,2)、(4,5)、(5,3)，查询闭矩形$[2,5]\times[2,5]$。逐点答案是后 4 个点，计数 4。

设 kd 树根点为(3,2)，左子树保存前两点，包围盒$[1,2]\times[1,4]$；右子树保存后两点，包围盒$[4,5]\times[3,5]$。每个节点保存 size 和包围盒：

要检查的对象	与查询的关系	计数操作
根点(3,2)	在闭矩形内，包括边界	+1
左子树包围盒	部分相交	继续分开检查
左点(1,1)	外部	+0
左点(2,4)	边界上	+1
右子树包围盒	整个包含	直接加 size=2，不逐点看

共 1+1+2=4。安全跳过的理由是包围盒确实包住所有后代：不相交则整棵无解，完全包含则每项都合法，部分相交才需要深入。若包围盒在插入后没更新，这个剪枝就不再安全。

若要求报告四个坐标，右子树即使完全包含，也必须把其中两个点输出，至少再付 2 步。一般输出 k 项有$\Omega(k)$成本。节点摘要能常数回答“这段一共有多少”，不会常数吐出任意多条数据。kd 树的最坏查询也可能访问许多节点，不能因为小例里跳了一棵就声称所有矩形查询都$O(\log n)$。

29.6 接线总价最低，与从源点出发路程最短

用能区分两种目标的三角形：A–B 权 2，A–C 权 3，B–C 权 2。

```
  A
2/ \3
B---C
  2
```

最小生成树只需连接全部顶点，总边价最小。Kruskal 按权排序：先选 A—B2，分量变{A,B},{C}；再选 B—C2，连成一棵树，总价 4；A—C3 加入会形成环，跳过。

A 作为源的最短路却应保留 A—B2、A—C3，距离分别 2、3，总边价 5。若走 MST 到 C，要 A→B→C 花 4，比直接 3 长。MST 优化“一次性买下哪些边”，最短路树优化“从指定源到每个点怎样走”，两个目标不同，不能只看它们都画成树。

权互异是 MST 唯一的充分条件；有重复却未必不唯一。例如图本来就是一条两边的链，两边都权 1，也只有这一棵生成树。回答“唯一吗”要给能区分的结构，不能把充分条件误读成必要条件。

29.7 数据流的第 3 大：保留集合和堆数组都画出来

输入依次 4、9、1、7、2、10，要求读满 3 项后随时报告第 3 大，重复值按记录计名次。保留容量 3 的小根堆，根是当前保留的前三大中的最小值，也就是第 3 大。普通存全部再排序可正确但空间随流增长，反复排序也浪费。

数组按完全树层序保存，根在下标 0；以下是一种合法的实际堆状态：

新值	动作	堆数组（不是排序数组）	第 3 大
4	入堆	[4]	未满 3 项
9	入堆，无须上滤	[4,9]	未满
1	入堆后与 4 交换上滤	[1,9,4]	1
7	替换根 1 得[7,9,4]，与较小孩子 4 换	[4,9,7]	4
2	不大于根 4，丢弃	[4,9,7]	4
10	替换根 4 得[10,9,7]，与 7 换	[7,9,10]	7

第 4 步后堆是根 4、左 9、右 7，父子关系合法，但数组没有排序。要输出前三名的降序顺序，需另处理，不能直接打印堆数组。

不变量是堆中恰好保存已读前缀最大的 3 条记录。新项不大于根时，已经有 3 项不小于它，它不会进入前三；较大时，旧根是这 4 个候选里该被淘汰的最小项。丢掉的项未来也不会重新变成历史前缀的前三，因为未来只会增加更大候选，不会删掉已有项。若题目改成滑动窗口、旧项会过期，这一保留规则就不够了。

对正整数 k，每项$O(\log(k+1))$，整个 n 项流$O(n\log(k+1))$、辅助空间$O(k)$。写 k+1 是为了包含 k=1：维护最大值仍需逐项看，整体$\Theta(n)$。k=0 应拒绝，未读满 k 项应报告未就绪；一边说流式只占 k 空间、一边另存全体输入，会违背自己的空间承诺。

29.8 同一张图，BFS 求的是边数，Dijkstra 求的是权和

有向边 S→T 权 10，S→A 权 1，A→B 权 1，B→T 权 1。BFS 第一次从 S 就发现 T，按最少边数取直接边；Dijkstra 不能把“第一次发现”当距离定值。

Dijkstra 确定的点	松弛以后 dist[A],dist[B],dist[T]	T 的前驱
初始 S 为 0	∞,∞,∞	无
处理 S	1,∞,10	S
处理最小候选 A1	1,2,10	S
处理最小候选 B2	1,2,3	改为 B
处理 T3	终止，路径 S→A→B→T	B

堆里如果还留着早先的 T10，懒删除版弹到它时应发现 10 不等于当前 dist[T]=3 并丢弃，不能再次当最终距离覆盖 3。维护一条记录/顶点的 decrease-key 堆与允许多份旧记录的懒堆，内存、时间口径也不同。

若所有边权都为 1，最小权和就是最少边数，BFS 每点每边常数处理，$O(V+E)$更简单。若只含 0、1 权，可用 0-1 BFS 的双端队列：改善路径用 0 边就放队头，用 1 边就放队尾；任意正权不满足这一两层距离结构，不能继续同样声称线性。

29.9 再做五题，先写判断依据

1. `ABCA→ACBA` 只允许插删，预算 2。给具体操作，并说明预算 1 为何不行。
2. 有人承诺：任意 n 个互异实数，只用两项比较，最坏不超过 3n 次。怎样用下界否定它对所有 n 成立？
3. 向量`[1,3,5,7]`在秩 1 插 2，未扩容。原下标 2 的物理格现在装什么？保存“指向原 5”的旧迭代器能否照用？
4. 随机快排对固定数组的内部随机轴取期望为$O(n\log n)$，是否保证每次运行都如此？
5. 有序数组允许任意点修改，同时求区间和与第一个≥x。一棵只存 sum 的线段树够不够？

解答：

1. 删原串下标 2 的 C，得到 ABA；在首 A 后插 C，得到 ACBA，共 2 步。长度相同、串不同，不可能只用一次插或删，因为任何一次都会改变长度；因此预算 1 不行。等价地 LCS 长 3，距离$4+4-2\times 3=2$。
2. 比较树至少 n!叶，最坏深度至少$\log_2(n!)$。取后半至少 n/2 项，每项不小于 n/2，得粗下界$(n/2)\log_2(n/2)$；对足够大 n（如偶数 n>128）已经超过 3n。这里不追求最小矛盾规模。整数位操作不属于“只比较”模型，不能把同一下界套在计数排序上。
3. 底层格子模型中结果`[1,2,3,5,7]`，原下标 2 的格装 3，原 5 移到下标 3。`std::vector` 插入点及其后的旧迭代器、引用会失效，不能通过旧句柄读取来“验证仍可读”；链表插入新节点通常保持其他未删节点的身份。物理地址、合法生命期和逻辑身份需要分别判断。
4. 不保证。连续选极端轴仍可能平方；期望是对每一固定输入的内部随机选择平均，不必假设输入本身随机。若实现固定选首项，则没有这份内部随机模型，不能借用同一个期望证明。
5. 任意修改可能破坏全局有序，原二分前提就不在了。只存 sum 也不能判断某区间是否含≥x：`[1,9]`与`[5,5]`和都 10，对阈值 8 却不同。需额外 max 摘要才能向左优先寻找满足阈值的位置；区间和仍由 sum 回答。若修改保证始终有序，可另对原数组二分，但应说明维护这份保证的方式。

30 PS04 的 21 个表达式：把每一次压栈、弹栈写全

本章对照 PS04 第 19 题的 21 式。每式先拆语法上的关键子表达式，再列 RPN，最后让每个 token 真实走过数值栈。表中栈从底到顶书写，最右项是下一次被弹出的项；数字的压入也单独列一行，因此可以从空栈逐行重做，不需要自行补隐含步骤。

这里!是阶乘，!!是双阶乘。例如 3!=6，3!!=3，4!!=8。双阶乘是一个 token，不能逐字符当两次普通阶乘。阶乘优先于乘方，乘方优先于乘除，乘除优先于加减。使用教师原表的左结合乘方约定；这 21 式没有连续无括号乘方链，故改右结合也不会改变它们。C++自身的^是异或，本章只是用这一字符表示自定义语言的乘方。

除法保留精确有理数，3/4 就存分数 3/4，不截成 0。阶乘只对非负整数执行。二元运算先弹**右值 b**，再弹**左值 a**，计算 a 运算 b。输入 8 3 -算 5，输入 3 8 -算−5；加法与乘法会掩盖左右弹反的错误，所以必须用减除检查。

30.1 先做一次从中缀到 RPN 的完整转换

取 2*(3+4)。数字直接输出；运算符暂存；遇更低或同级左结合运算时先弹旧运算符；左括号阻隔比较，右括号让内部运算符弹完，再消去左括号。# 表示结束哨兵，本章只用来画状态。

当前 token	本次动作	运算符栈（底→顶）	本步新输出
初始	放底哨兵	#	无
2	数字直接输出	#	2
*	哨兵让它入栈	# *	无
(	压左括号	# * (	无
3	数字直接输出	# * (	3
+	左括号内暂存+	# * (+	无
4	数字直接输出	# * (+	4
)	先弹+，当前右括号不前进	# * (	+
仍为)	消去左括号，消费右括号	# *	无
结束 #	弹*，当前结束符不前进	#	*
仍为 #	哨兵相遇，结束	空	无

从上到下拼接“本步新输出”，就是 2 3 4 + *。括号没有进入 RPN，因为它只负责决定子树边界，不执行数值运算。它对应的表达式树是：

```
      *
     / \
    2   +
       / \
      3   4
```

后序先 2，再 3、4、+，最后*，与栈转换相同。下面每式给出的 RPN 都可以按这个规则从中缀重建；数值表则独立验证“这个 RPN 到底算成什么”。表中偶尔连续出现多个数字，是在保存尚未凑齐右子表达式的左值，不表示该立即把它们相加。

30.2 第 1 式

原式：(0+1+2−3+4+5+6)*(7+8)*9

先把左括号内的连加减从左到右算完，得到 15。另一个括号 7+8 也是 15，最后 15×15×9。两个独立子表达式只有在各自算完后才交给乘法。

```
0 1 + 2 + 3 - 4 + 5 + 6 + 7 8 + * 9 *
```

步骤 / 读到 token	这一动作	动作后的完整数值栈
1 / 0	压入数字 0	[0]
2 / 1	压入数字 1	[0, 1]
3 / +	先弹右 1，再弹左 0；(0) + (1) = 1	[1]
4 / 2	压入数字 2	[1, 2]
5 / +	先弹右 2，再弹左 1；(1) + (2) = 3	[3]
6 / 3	压入数字 3	[3, 3]
7 / -	先弹右 3，再弹左 3；(3) - (3) = 0	[0]
8 / 4	压入数字 4	[0, 4]
9 / +	先弹右 4，再弹左 0；(0) + (4) = 4	[4]
10 / 5	压入数字 5	[4, 5]
11 / +	先弹右 5，再弹左 4；(4) + (5) = 9	[9]
12 / 6	压入数字 6	[9, 6]
13 / +	先弹右 6，再弹左 9；(9) + (6) = 15	[15]
14 / 7	压入数字 7	[15, 7]
15 / 8	压入数字 8	[15, 7, 8]
16 / +	先弹右 8，再弹左 7；(7) + (8) = 15	[15, 15]
17 / *	先弹右 15，再弹左 15；(15) * (15) = 225	[225]
18 / 9	压入数字 9	[225, 9]
19 / *	先弹右 9，再弹左 225；(225) * (9) = 2025	[2025]

最终值为 2025，数值栈峰值 3 项。

30.3 第 2 式

原式：`(0*1+2)*3!/4!*5*6*(7+8)*9`

先算 0×1+2=2，阶乘 3!=6、4!=24。乘除同级按左结合：2×6÷24=1/2，再乘 5、6、15、9。中间出现分数并不代表错误，不能在这里擅自做整数截断。

```
0 1 * 2 + 3 ! * 4 ! / 5 * 6 * 7 8 + * 9 *
```

步骤 / 读到 token	这一动作	动作后的完整数值栈
1 / 0	压入数字 0	[0]
2 / 1	压入数字 1	[0, 1]
3 / *	先弹右 1，再弹左 0；(0) * (1) = 0	[0]
4 / 2	压入数字 2	[0, 2]
5 / +	先弹右 2，再弹左 0；(0) + (2) = 2	[2]
6 / 3	压入数字 3	[2, 3]
7 / !	弹 3，算 3!=6，压回	[2, 6]

步骤 / 读到 token	这一动作	动作后的完整数值栈
8 / *	先弹右 6，再弹左 2；(2) * (6) = 12	[12]
9 / 4	压入数字 4	[12, 4]
10 / !	弹 4，算 4!=24，压回	[12, 24]
11 / /	先弹右 24，再弹左 12；(12) / (24) = 1/2	[1/2]
12 / 5	压入数字 5	[1/2, 5]
13 / *	先弹右 5，再弹左 1/2；(1/2) * (5) = 5/2	[5/2]
14 / 6	压入数字 6	[5/2, 6]
15 / *	先弹右 6，再弹左 5/2；(5/2) * (6) = 15	[15]
16 / 7	压入数字 7	[15, 7]
17 / 8	压入数字 8	[15, 7, 8]
18 / +	先弹右 8，再弹左 7；(7) + (8) = 15	[15, 15]
19 / *	先弹右 15，再弹左 15；(15) * (15) = 225	[225]
20 / 9	压入数字 9	[225, 9]
21 / *	先弹右 9，再弹左 225；(225) * (9) = 2025	[2025]

最终值为 2025，数值栈峰值 3 项。

30.4 第 3 式

原式：`0!+(1+2)!!*3-4-5!-6-7-8+9`

0!=1，括号 1+2=3，3!!=3，所以前面 1+3×3=10。另一个阶乘 5!=120，随后从左到右减 4、120、6、7、8，再加 9。注意双阶乘与连续两次阶乘的差别。

```
0 ! 1 2 + !! 3 * + 4 - 5 ! - 6 - 7 - 8 - 9 +
```

步骤 / 读到 token	这一动作	动作后的完整数值栈
1 / 0	压入数字 0	[0]
2 / !	弹 0，算 0!=1，压回	[1]
3 / 1	压入数字 1	[1, 1]
4 / 2	压入数字 2	[1, 1, 2]
5 / +	先弹右 2，再弹左 1；(1) + (2) = 3	[1, 3]
6 / !!	弹 3，算 3!!=3，压回	[1, 3]
7 / 3	压入数字 3	[1, 3, 3]
8 / *	先弹右 3，再弹左 3；(3) * (3) = 9	[1, 9]
9 / +	先弹右 9，再弹左 1；(1) + (9) = 10	[10]
10 / 4	压入数字 4	[10, 4]
11 / -	先弹右 4，再弹左 10；(10) - (4) = 6	[6]
12 / 5	压入数字 5	[6, 5]
13 / !	弹 5，算 5!=120，压回	[6, 120]
14 / -	先弹右 120，再弹左 6；(6) - (120) = -114	[-114]

步骤 / 读到 token	这一动作	动作后的完整数值栈
15 / 6	压入数字 6	[-114, 6]
16 / -	先弹右 6，再弹左-114；(-114) - (6) = -120	[-120]
17 / 7	压入数字 7	[-120, 7]
18 / -	先弹右 7，再弹左-120；(-120) - (7) = -127	[-127]
19 / 8	压入数字 8	[-127, 8]
20 / -	先弹右 8，再弹左-127；(-127) - (8) = -135	[-135]
21 / 9	压入数字 9	[-135, 9]
22 / +	先弹右 9，再弹左-135；(-135) + (9) = -126	[-126]

最终值为-126，数值栈峰值 3 项。

30.5 第 4 式

原式：`(0+1+2)/3!/4!*5!*6*(7+8)*9`

括号和为 3。依次除 3!=6 与 4!=24，得到 1/48，再乘 5!=120 得到 5/2，继续乘 6、15、9。若每次除法都截成整数，第一步 3/6 就变 0，整式会完全换掉。

```
0 1 + 2 + 3 ! / 4 ! / 5 ! * 6 * 7 8 + * 9 *
```

步骤 / 读到 token	这一动作	动作后的完整数值栈
1 / 0	压入数字 0	[0]
2 / 1	压入数字 1	[0, 1]
3 / +	先弹右 1，再弹左 0；(0) + (1) = 1	[1]
4 / 2	压入数字 2	[1, 2]
5 / +	先弹右 2，再弹左 1；(1) + (2) = 3	[3]
6 / 3	压入数字 3	[3, 3]
7 / !	弹 3，算 3!=6，压回	[3, 6]
8 / /	先弹右 6，再弹左 3；(3) / (6) = 1/2	[1/2]
9 / 4	压入数字 4	[1/2, 4]
10 / !	弹 4，算 4!=24，压回	[1/2, 24]
11 / /	先弹右 24，再弹左 1/2；(1/2) / (24) = 1/48	[1/48]
12 / 5	压入数字 5	[1/48, 5]
13 / !	弹 5，算 5!=120，压回	[1/48, 120]
14 / *	先弹右 120，再弹左 1/48；(1/48) * (120) = 5/2	[5/2]
15 / 6	压入数字 6	[5/2, 6]
16 / *	先弹右 6，再弹左 5/2；(5/2) * (6) = 15	[15]
17 / 7	压入数字 7	[15, 7]
18 / 8	压入数字 8	[15, 7, 8]
19 / +	先弹右 8，再弹左 7；(7) + (8) = 15	[15, 15]
20 / *	先弹右 15，再弹左 15；(15) * (15) = 225	[225]

步骤 / 读到 token	这一动作	动作后的完整数值栈
21 / 9	压入数字 9	[225, 9]
22 / *	先弹右 9，再弹左 225；(225) * (9) = 2025	[2025]

最终值为 2025，数值栈峰值 3 项。

30.6 第 5 式

原式：(0!^1/2*3!!/(4+5)*6-7-8)*9

0!=1，1^1=1。接下来除 2、乘 3!!=3、除(4+5)=9、乘 6，连续乘除结果是 1。括号内再减 7、减 8 得−14，最后乘 9。这里的负值由明确的减法链产生。

```
0 ! 1 ^ 2 / 3 !! * 4 5 + / 6 * 7 - 8 - 9 *
```

步骤 / 读到 token	这一动作	动作后的完整数值栈
1 / 0	压入数字 0	[0]
2 / !	弹 0，算 0!=1，压回	[1]
3 / 1	压入数字 1	[1, 1]
4 / ^	先弹右 1，再弹左 1；(1) ^ (1) = 1	[1]
5 / 2	压入数字 2	[1, 2]
6 / /	先弹右 2，再弹左 1；(1) / (2) = 1/2	[1/2]
7 / 3	压入数字 3	[1/2, 3]
8 / !!	弹 3，算 3!!=3，压回	[1/2, 3]
9 / *	先弹右 3，再弹左 1/2；(1/2) * (3) = 3/2	[3/2]
10 / 4	压入数字 4	[3/2, 4]
11 / 5	压入数字 5	[3/2, 4, 5]
12 / +	先弹右 5，再弹左 4；(4) + (5) = 9	[3/2, 9]
13 / /	先弹右 9，再弹左 3/2；(3/2) / (9) = 1/6	[1/6]
14 / 6	压入数字 6	[1/6, 6]
15 / *	先弹右 6，再弹左 1/6；(1/6) * (6) = 1	[1]
16 / 7	压入数字 7	[1, 7]
17 / -	先弹右 7，再弹左 1；(1) - (7) = -6	[-6]
18 / 8	压入数字 8	[-6, 8]
19 / -	先弹右 8，再弹左-6；(-6) - (8) = -14	[-14]
20 / 9	压入数字 9	[-14, 9]
21 / *	先弹右 9，再弹左-14；(-14) * (9) = -126	[-126]

最终值为-126，数值栈峰值 3 项。

30.7 第 6 式

原式：((0*(1-2)+3+4)*5*6+(7+8))*9

先算 1−2=−1，再乘 0 得到 0；随后加 3、加 4 得 7。7×5×6=210，另一括号 7+8=15，相加 225 再乘 9。乘零会消去一项数值，但解析器仍应正确识别其完整子表达式。

```
0 1 2 - * 3 + 4 + 5 * 6 * 7 8 + + 9 *
```

步骤 / 读到 token	这一动作	动作后的完整数值栈
1 / 0	压入数字 0	[0]
2 / 1	压入数字 1	[0, 1]
3 / 2	压入数字 2	[0, 1, 2]
4 / -	先弹右 2，再弹左 1；(1) - (2) = -1	[0, -1]
5 / *	先弹右-1，再弹左 0；(0) * (-1) = 0	[0]
6 / 3	压入数字 3	[0, 3]
7 / +	先弹右 3，再弹左 0；(0) + (3) = 3	[3]
8 / 4	压入数字 4	[3, 4]
9 / +	先弹右 4，再弹左 3；(3) + (4) = 7	[7]
10 / 5	压入数字 5	[7, 5]
11 / *	先弹右 5，再弹左 7；(7) * (5) = 35	[35]
12 / 6	压入数字 6	[35, 6]
13 / *	先弹右 6，再弹左 35；(35) * (6) = 210	[210]
14 / 7	压入数字 7	[210, 7]
15 / 8	压入数字 8	[210, 7, 8]
16 / +	先弹右 8，再弹左 7；(7) + (8) = 15	[210, 15]
17 / +	先弹右 15，再弹左 210；(210) + (15) = 225	[225]
18 / 9	压入数字 9	[225, 9]
19 / *	先弹右 9，再弹左 225；(225) * (9) = 2025	[2025]

最终值为 2025，数值栈峰值 3 项。

30.8 第 7 式

原式：`(0!-1!+2)*3!/4!*5*6*(7+8)*9`

0!和 1!都等于 1，所以第一个括号 1−1+2=2，后续链与第 2 式相同。这样可以把两个表达式做变形核对：不同写法确实产生同一个中间值 2。

```
0 ! 1 ! - 2 + 3 ! * 4 ! / 5 * 6 * 7 8 + * 9 *
```

步骤 / 读到 token	这一动作	动作后的完整数值栈
1 / 0	压入数字 0	[0]
2 / !	弹 0，算 0!=1，压回	[1]
3 / 1	压入数字 1	[1, 1]
4 / !	弹 1，算 1!=1，压回	[1, 1]
5 / -	先弹右 1，再弹左 1；(1) - (1) = 0	[0]
6 / 2	压入数字 2	[0, 2]
7 / +	先弹右 2，再弹左 0；(0) + (2) = 2	[2]

步骤 / 读到 token	这一动作	动作后的完整数值栈
8 / 3	压入数字 3	[2, 3]
9 / !	弹 3，算 3!=6，压回	[2, 6]
10 / *	先弹右 6，再弹左 2；(2) * (6) = 12	[12]
11 / 4	压入数字 4	[12, 4]
12 / !	弹 4，算 4!=24，压回	[12, 24]
13 / /	先弹右 24，再弹左 12；(12) / (24) = 1/2	[1/2]
14 / 5	压入数字 5	[1/2, 5]
15 / *	先弹右 5，再弹左 1/2；(1/2) * (5) = 5/2	[5/2]
16 / 6	压入数字 6	[5/2, 6]
17 / *	先弹右 6，再弹左 5/2；(5/2) * (6) = 15	[15]
18 / 7	压入数字 7	[15, 7]
19 / 8	压入数字 8	[15, 7, 8]
20 / +	先弹右 8，再弹左 7；(7) + (8) = 15	[15, 15]
21 / *	先弹右 15，再弹左 15；(15) * (15) = 225	[225]
22 / 9	压入数字 9	[225, 9]
23 / *	先弹右 9，再弹左 225；(225) * (9) = 2025	[2025]

最终值为 2025，数值栈峰值 3 项。

30.9 第 8 式

原式：`((0-1)^2*(3!!/4-5+6*7+8))*9`

0−1=−1，其平方为 1。另一个括号中 3!!/4=3/4，6×7=42，合并成 3/4−5+42+8=183/4，最后乘 9，得到 1647/4。不要把 3/4 提前舍去。

```
0 1 - 2 ^ 3 !! 4 / 5 - 6 7 * + 8 + * 9 *
```

步骤 / 读到 token	这一动作	动作后的完整数值栈
1 / 0	压入数字 0	[0]
2 / 1	压入数字 1	[0, 1]
3 / -	先弹右 1，再弹左 0；(0) - (1) = -1	[-1]
4 / 2	压入数字 2	[-1, 2]
5 / ^	先弹右 2，再弹左-1；(-1) ^ (2) = 1	[1]
6 / 3	压入数字 3	[1, 3]
7 / !!	弹 3，算 3!!=3，压回	[1, 3]
8 / 4	压入数字 4	[1, 3, 4]
9 / /	先弹右 4，再弹左 3；(3) / (4) = 3/4	[1, 3/4]
10 / 5	压入数字 5	[1, 3/4, 5]

步骤 / 读到 token	这一动作	动作后的完整数值栈
11 / -	先弹右 5，再弹左 3/4；(3/4) - (5) = -17/4	[1, -17/4]
12 / 6	压入数字 6	[1, -17/4, 6]
13 / 7	压入数字 7	[1, -17/4, 6, 7]
14 / *	先弹右 7，再弹左 6；(6) * (7) = 42	[1, -17/4, 42]
15 / +	先弹右 42，再弹左-17/4；(-17/4) + (42) = 151/4	[1, 151/4]
16 / 8	压入数字 8	[1, 151/4, 8]
17 / +	先弹右 8，再弹左 151/4；(151/4) + (8) = 183/4	[1, 183/4]
18 / *	先弹右 183/4，再弹左 1；(1) * (183/4) = 183/4	[183/4]
19 / 9	压入数字 9	[183/4, 9]
20 / *	先弹右 9，再弹左 183/4；(183/4) * (9) = 1647/4	[1647/4]

最终值为 1647/4，数值栈峰值 4 项。

30.10 第 9 式

原式：`((0-1)^2*(3!!/4-5-6+7*8))*9`

左侧平方仍为 1。右括号是 3/4−5−6+7×8=3/4−11+56=183/4，与第 8 式巧合相等，但子表达式不同，分别算清才能确认。

```
0 1 - 2 ^ 3 !! 4 / 5 - 6 - 7 8 * + * 9 *
```

步骤 / 读到 token	这一动作	动作后的完整数值栈
1 / 0	压入数字 0	[0]
2 / 1	压入数字 1	[0, 1]
3 / -	先弹右 1，再弹左 0；(0) - (1) = -1	[-1]
4 / 2	压入数字 2	[-1, 2]
5 / ^	先弹右 2，再弹左-1；(-1) ^ (2) = 1	[1]
6 / 3	压入数字 3	[1, 3]
7 / !!	弹 3，算 3!!=3，压回	[1, 3]
8 / 4	压入数字 4	[1, 3, 4]
9 / /	先弹右 4，再弹左 3；(3) / (4) = 3/4	[1, 3/4]
10 / 5	压入数字 5	[1, 3/4, 5]
11 / -	先弹右 5，再弹左 3/4；(3/4) - (5) = -17/4	[1, -17/4]

步骤 / 读到 token	这一动作	动作后的完整数值栈
12 / 6	压入数字 6	[1, -17/4, 6]
13 / -	先弹右 6，再弹左-17/4；(-17/4) - (6) = -41/4	[1, -41/4]
14 / 7	压入数字 7	[1, -41/4, 7]
15 / 8	压入数字 8	[1, -41/4, 7, 8]
16 / *	先弹右 8，再弹左 7；(7) * (8) = 56	[1, -41/4, 56]
17 / +	先弹右 56，再弹左-41/4；(-41/4) + (56) = 183/4	[1, 183/4]
18 / *	先弹右 183/4，再弹左 1；(1) * (183/4) = 183/4	[183/4]
19 / 9	压入数字 9	[183/4, 9]
20 / *	先弹右 9，再弹左 183/4；(183/4) * (9) = 1647/4	[1647/4]

最终值为 1647/4，数值栈峰值 4 项。

30.11 第 10 式

原式：`(0!+1+2)!+(3!-4)^(5+6)-7*8+9`

0!+1+2=4，外面的阶乘得到 24。3!-4=2，5+6=11，乘方 2^11=2048。最后 24+2048-56+9=2025。阶乘作用于整个括号，而不是只作用于末项 2。

```
0 ! 1 + 2 + ! 3 ! 4 - 5 6 + ^ + 7 8 * - 9 +
```

步骤 / 读到 token	这一动作	动作后的完整数值栈
1 / 0	压入数字 0	[0]
2 / !	弹 0，算 0!=1，压回	[1]
3 / 1	压入数字 1	[1, 1]
4 / +	先弹右 1，再弹左 1；(1) + (1) = 2	[2]
5 / 2	压入数字 2	[2, 2]
6 / +	先弹右 2，再弹左 2；(2) + (2) = 4	[4]
7 / !	弹 4，算 4!=24，压回	[24]
8 / 3	压入数字 3	[24, 3]
9 / !	弹 3，算 3!=6，压回	[24, 6]
10 / 4	压入数字 4	[24, 6, 4]
11 / -	先弹右 4，再弹左 6；(6) - (4) = 2	[24, 2]
12 / 5	压入数字 5	[24, 2, 5]
13 / 6	压入数字 6	[24, 2, 5, 6]
14 / +	先弹右 6，再弹左 5；(5) + (6) = 11	[24, 2, 11]
15 / ^	先弹右 11，再弹左 2；(2) ^ (11) = 2048	[24, 2048]
16 / +	先弹右 2048，再弹左 24；(24) + (2048) = 2072	[2072]

步骤 / 读到 token	这一动作	动作后的完整数值栈
17 / 7	压入数字 7	[2072, 7]
18 / 8	压入数字 8	[2072, 7, 8]
19 / *	先弹右 8，再弹左 7；(7) * (8) = 56	[2072, 56]
20 / -	先弹右 56，再弹左 2072；(2072) - (56) = 2016	[2016]
21 / 9	压入数字 9	[2016, 9]
22 / +	先弹右 9，再弹左 2016；(2016) + (9) = 2025	[2025]

最终值为 2025，数值栈峰值 4 项。

30.12 第 11 式

原式：`(0+(1+2))^3/(4+5)*6!-(7+8)*9`

内括号 1+2=3，加 0 仍 3，立方 27。4+5=9，所以 27/9=3，再乘 6!=720 得到 2160；减去(7+8)×9=135，结果 2025。

```
0 1 2 + + 3 ^ 4 5 + / 6 ! * 7 8 + 9 * -
```

步骤 / 读到 token	这一动作	动作后的完整数值栈
1 / 0	压入数字 0	[0]
2 / 1	压入数字 1	[0, 1]
3 / 2	压入数字 2	[0, 1, 2]
4 / +	先弹右 2，再弹左 1；(1) + (2) = 3	[0, 3]
5 / +	先弹右 3，再弹左 0；(0) + (3) = 3	[3]
6 / 3	压入数字 3	[3, 3]
7 / ^	先弹右 3，再弹左 3；(3) ^ (3) = 27	[27]
8 / 4	压入数字 4	[27, 4]
9 / 5	压入数字 5	[27, 4, 5]
10 / +	先弹右 5，再弹左 4；(4) + (5) = 9	[27, 9]
11 / /	先弹右 9，再弹左 27；(27) / (9) = 3	[3]
12 / 6	压入数字 6	[3, 6]
13 / !	弹 6，算 6!=720，压回	[3, 720]
14 / *	先弹右 720，再弹左 3；(3) * (720) = 2160	[2160]
15 / 7	压入数字 7	[2160, 7]
16 / 8	压入数字 8	[2160, 7, 8]
17 / +	先弹右 8，再弹左 7；(7) + (8) = 15	[2160, 15]
18 / 9	压入数字 9	[2160, 15, 9]
19 / *	先弹右 9，再弹左 15；(15) * (9) = 135	[2160, 135]
20 / -	先弹右 135，再弹左 2160；(2160) - (135) = 2025	[2025]

最终值为 2025，数值栈峰值 3 项。

30.13 第 12 式

原式：`((0-1)^2!*(3!!/4-5+6*7+8))*9`

与第 8 式相比，指数由 2 变成 2!，二者恰好都为 2，因此数值仍 1647/4。RPN 必须保留“先算 2!再乘方”的过程，不能因为最后相同就抄成同一个记号串。

```
0 1 - 2 ! ^ 3 !! 4 / 5 - 6 7 * + 8 + * 9 *
```

步骤 / 读到 token	这一动作	动作后的完整数值栈
1 / `0`	压入数字 0	`[0]`
2 / `1`	压入数字 1	`[0, 1]`
3 / `-`	先弹右 1，再弹左 0；`(0) - (1) = -1`	`[-1]`
4 / `2`	压入数字 2	`[-1, 2]`
5 / `!`	弹 2，算 2!=2，压回	`[-1, 2]`
6 / `^`	先弹右 2，再弹左-1；`(-1) ^ (2) = 1`	`[1]`
7 / `3`	压入数字 3	`[1, 3]`
8 / `!!`	弹 3，算 3!!=3，压回	`[1, 3]`
9 / `4`	压入数字 4	`[1, 3, 4]`
10 / `/`	先弹右 4，再弹左 3；`(3) / (4) = 3/4`	`[1, 3/4]`
11 / `5`	压入数字 5	`[1, 3/4, 5]`
12 / `-`	先弹右 5，再弹左 3/4；`(3/4) - (5) = -17/4`	`[1, -17/4]`
13 / `6`	压入数字 6	`[1, -17/4, 6]`
14 / `7`	压入数字 7	`[1, -17/4, 6, 7]`
15 / `*`	先弹右 7，再弹左 6；`(6) * (7) = 42`	`[1, -17/4, 42]`
16 / `+`	先弹右 42，再弹左-17/4；`(-17/4) + (42) = 151/4`	`[1, 151/4]`
17 / `8`	压入数字 8	`[1, 151/4, 8]`
18 / `+`	先弹右 8，再弹左 151/4；`(151/4) + (8) = 183/4`	`[1, 183/4]`
19 / `*`	先弹右 183/4，再弹左 1；`(1) * (183/4) = 183/4`	`[183/4]`
20 / `9`	压入数字 9	`[183/4, 9]`
21 / `*`	先弹右 9，再弹左 183/4；`(183/4) * (9) = 1647/4`	`[1647/4]`

最终值为 1647/4，数值栈峰值 4 项。

30.14 第 13 式

原式：`((0-1)^2!*(3!!/4-5-6+7*8))*9`

与第 9 式同理，指数 2!先得到 2；括号 3/4−5−6+56=183/4，结果 1647/4。这里可用独立简化式核对栈结果。

```
0 1 - 2 ! ^ 3 !! 4 / 5 - 6 - 7 8 * + * 9 *
```

步骤 / 读到 token	这一动作	动作后的完整数值栈
1 / `0`	压入数字 0	`[0]`
2 / `1`	压入数字 1	`[0, 1]`
3 / `-`	先弹右 1，再弹左 0；`(0) - (1) = -1`	`[-1]`
4 / `2`	压入数字 2	`[-1, 2]`
5 / `!`	弹 2，算 2!=2，压回	`[-1, 2]`
6 / `^`	先弹右 2，再弹左-1；`(-1) ^ (2) = 1`	`[1]`
7 / `3`	压入数字 3	`[1, 3]`
8 / `!!`	弹 3，算 3!!=3，压回	`[1, 3]`
9 / `4`	压入数字 4	`[1, 3, 4]`
10 / `/`	先弹右 4，再弹左 3；`(3) / (4) = 3/4`	`[1, 3/4]`
11 / `5`	压入数字 5	`[1, 3/4, 5]`
12 / `-`	先弹右 5，再弹左 3/4；`(3/4) - (5) = -17/4`	`[1, -17/4]`
13 / `6`	压入数字 6	`[1, -17/4, 6]`
14 / `-`	先弹右 6，再弹左-17/4；`(-17/4) - (6) = -41/4`	`[1, -41/4]`
15 / `7`	压入数字 7	`[1, -41/4, 7]`
16 / `8`	压入数字 8	`[1, -41/4, 7, 8]`
17 / `*`	先弹右 8，再弹左 7；`(7) * (8) = 56`	`[1, -41/4, 56]`
18 / `+`	先弹右 56，再弹左-41/4；`(-41/4) + (56) = 183/4`	`[1, 183/4]`
19 / `*`	先弹右 183/4，再弹左 1；`(1) * (183/4) = 183/4`	`[183/4]`
20 / `9`	压入数字 9	`[183/4, 9]`
21 / `*`	先弹右 9，再弹左 183/4；`(183/4) * (9) = 1647/4`	`[1647/4]`

最终值为 1647/4，数值栈峰值 4 项。

30.15 第 14 式

原式：`(0!+1)*(2+3)*(4+5)/6*(7+8)*9`

三个括号分别为 2、5、9。乘除链先得 2×5×9/6=15，再乘 15 和 9，得到 2025。除法不是一个括号把后面所有乘项包起来，必须尊重左结合。

```
0 ! 1 + 2 3 + * 4 5 + * 6 / 7 8 + * 9 *
```

步骤 / 读到 token	这一动作	动作后的完整数值栈
1 / `0`	压入数字 0	`[0]`

步骤 / 读到 token	这一动作	动作后的完整数值栈
2 / !	弹 0，算 0!=1，压回	[1]
3 / 1	压入数字 1	[1, 1]
4 / +	先弹右 1，再弹左 1；(1) + (1) = 2	[2]
5 / 2	压入数字 2	[2, 2]
6 / 3	压入数字 3	[2, 2, 3]
7 / +	先弹右 3，再弹左 2；(2) + (3) = 5	[2, 5]
8 / *	先弹右 5，再弹左 2；(2) * (5) = 10	[10]
9 / 4	压入数字 4	[10, 4]
10 / 5	压入数字 5	[10, 4, 5]
11 / +	先弹右 5，再弹左 4；(4) + (5) = 9	[10, 9]
12 / *	先弹右 9，再弹左 10；(10) * (9) = 90	[90]
13 / 6	压入数字 6	[90, 6]
14 / /	先弹右 6，再弹左 90；(90) / (6) = 15	[15]
15 / 7	压入数字 7	[15, 7]
16 / 8	压入数字 8	[15, 7, 8]
17 / +	先弹右 8，再弹左 7；(7) + (8) = 15	[15, 15]
18 / *	先弹右 15，再弹左 15；(15) * (15) = 225	[225]
19 / 9	压入数字 9	[225, 9]
20 / *	先弹右 9，再弹左 225；(225) * (9) = 2025	[2025]

最终值为 2025，数值栈峰值 3 项。

30.16 第 15 式

原式：`(0!^1/2*3!!/(4+5)*6-(7+8))*9`

前半连续乘除仍得到 1；后面明确减去括号 7+8=15，所以括号值−14，乘 9 得−126。它与第 5 式的“先减 7 再减 8”在这个位置等价。

```
0 ! 1 ^ 2 / 3 !! * 4 5 + / 6 * 7 8 + - 9 *
```

步骤 / 读到 token	这一动作	动作后的完整数值栈
1 / 0	压入数字 0	[0]
2 / !	弹 0，算 0!=1，压回	[1]
3 / 1	压入数字 1	[1, 1]
4 / ^	先弹右 1，再弹左 1；(1) ^ (1) = 1	[1]
5 / 2	压入数字 2	[1, 2]
6 / /	先弹右 2，再弹左 1；(1) / (2) = 1/2	[1/2]
7 / 3	压入数字 3	[1/2, 3]
8 / !!	弹 3，算 3!!=3，压回	[1/2, 3]
9 / *	先弹右 3，再弹左 1/2；(1/2) * (3) = 3/2	[3/2]

步骤 / 读到 token	这一动作	动作后的完整数值栈
10 / 4	压入数字 4	[3/2, 4]
11 / 5	压入数字 5	[3/2, 4, 5]
12 / +	先弹右 5，再弹左 4；(4) + (5) = 9	[3/2, 9]
13 / /	先弹右 9，再弹左 3/2；(3/2) / (9) = 1/6	[1/6]
14 / 6	压入数字 6	[1/6, 6]
15 / *	先弹右 6，再弹左 1/6；(1/6) * (6) = 1	[1]
16 / 7	压入数字 7	[1, 7]
17 / 8	压入数字 8	[1, 7, 8]
18 / +	先弹右 8，再弹左 7；(7) + (8) = 15	[1, 15]
19 / -	先弹右 15，再弹左 1；(1) - (15) = -14	[-14]
20 / 9	压入数字 9	[-14, 9]
21 / *	先弹右 9，再弹左-14；(-14) * (9) = -126	[-126]

最终值为-126，数值栈峰值 3 项。

30.17 第 16 式

原式：`((0-1)^2*(3!!/4-5*(6-7-8)))*9`

0−1 的平方为 1。最内层 6−7−8 按左结合是−9，不是 6−(7−8)=7。于是 3/4−5×(−9)=183/4，最后乘 9，得 1647/4。

```
0 1 - 2 ^ 3 !! 4 / 5 6 7 - 8 - * - * 9 *
```

步骤 / 读到 token	这一动作	动作后的完整数值栈
1 / 0	压入数字 0	[0]
2 / 1	压入数字 1	[0, 1]
3 / -	先弹右 1，再弹左 0；(0) - (1) = -1	[-1]
4 / 2	压入数字 2	[-1, 2]
5 / ^	先弹右 2，再弹左-1；(-1) ^ (2) = 1	[1]
6 / 3	压入数字 3	[1, 3]
7 / !!	弹 3，算 3!!=3，压回	[1, 3]
8 / 4	压入数字 4	[1, 3, 4]
9 / /	先弹右 4，再弹左 3；(3) / (4) = 3/4	[1, 3/4]
10 / 5	压入数字 5	[1, 3/4, 5]
11 / 6	压入数字 6	[1, 3/4, 5, 6]
12 / 7	压入数字 7	[1, 3/4, 5, 6, 7]
13 / -	先弹右 7，再弹左 6；(6) - (7) = -1	[1, 3/4, 5, -1]
14 / 8	压入数字 8	[1, 3/4, 5, -1, 8]

步骤 / 读到 token	这一动作	动作后的完整数值栈
15 / -	先弹右 8，再弹左-1；(-1) - (8) = -9	[1, 3/4, 5, -9]
16 / *	先弹右-9，再弹左 5；(5) * (-9) = -45	[1, 3/4, -45]
17 / -	先弹右-45，再弹左 3/4；(3/4) - (-45) = 183/4	[1, 183/4]
18 / *	先弹右 183/4，再弹左 1；(1) * (183/4) = 183/4	[183/4]
19 / 9	压入数字 9	[183/4, 9]
20 / *	先弹右 9，再弹左 183/4；(183/4) * (9) = 1647/4	[1647/4]

最终值为 1647/4，数值栈峰值 5 项。

30.18 第 17 式

原式：`((0-1)^2!*(3!!/4-5*(6-7-8)))*9`

与第 16 式相比，指数为 2!，仍等于 2；其他过程相同。这种配对适合检查解析器是否把后缀阶乘正确附到指数上。

```
0 1 - 2 ! ^ 3 !! 4 / 5 6 7 - 8 - * - * 9 *
```

步骤 / 读到 token	这一动作	动作后的完整数值栈
1 / 0	压入数字 0	[0]
2 / 1	压入数字 1	[0, 1]
3 / -	先弹右 1，再弹左 0；(0) - (1) = -1	[-1]
4 / 2	压入数字 2	[-1, 2]
5 / !	弹 2，算 2!=2，压回	[-1, 2]
6 / ^	先弹右 2，再弹左-1；(-1) ^ (2) = 1	[1]
7 / 3	压入数字 3	[1, 3]
8 / !!	弹 3，算 3!!=3，压回	[1, 3]
9 / 4	压入数字 4	[1, 3, 4]
10 / /	先弹右 4，再弹左 3；(3) / (4) = 3/4	[1, 3/4]
11 / 5	压入数字 5	[1, 3/4, 5]
12 / 6	压入数字 6	[1, 3/4, 5, 6]
13 / 7	压入数字 7	[1, 3/4, 5, 6, 7]
14 / -	先弹右 7，再弹左 6；(6) - (7) = -1	[1, 3/4, 5, -1]
15 / 8	压入数字 8	[1, 3/4, 5, -1, 8]

步骤 / 读到 token	这一动作	动作后的完整数值栈
16 / -	先弹右 8，再弹左-1；(-1) - (8) = -9	[1, 3/4, 5, -9]
17 / *	先弹右-9，再弹左 5；(5) * (-9) = -45	[1, 3/4, -45]
18 / -	先弹右-45，再弹左 3/4；(3/4) - (-45) = 183/4	[1, 183/4]
19 / *	先弹右 183/4，再弹左 1；(1) * (183/4) = 183/4	[183/4]
20 / 9	压入数字 9	[183/4, 9]
21 / *	先弹右 9，再弹左 183/4；(183/4) * (9) = 1647/4	[1647/4]

最终值为 1647/4，数值栈峰值 5 项。

30.19 第 18 式

原式：`((0!+1!)^2+(3!-4)+5*6)*(7*8)+9`

0!+1!=2，平方为 4；3!−4=2；5×6=30，括号合计 36。7×8=56，36×56=2016，再加 9 得 2025。

```
0 ! 1 ! + 2 ^ 3 ! 4 - + 5 6 * + 7 8 * * 9 +
```

步骤 / 读到 token	这一动作	动作后的完整数值栈
1 / 0	压入数字 0	[0]
2 / !	弹 0，算 0!=1，压回	[1]
3 / 1	压入数字 1	[1, 1]
4 / !	弹 1，算 1!=1，压回	[1, 1]
5 / +	先弹右 1，再弹左 1；(1) + (1) = 2	[2]
6 / 2	压入数字 2	[2, 2]
7 / ^	先弹右 2，再弹左 2；(2) ^ (2) = 4	[4]
8 / 3	压入数字 3	[4, 3]
9 / !	弹 3，算 3!=6，压回	[4, 6]
10 / 4	压入数字 4	[4, 6, 4]
11 / -	先弹右 4，再弹左 6；(6) - (4) = 2	[4, 2]
12 / +	先弹右 2，再弹左 4；(4) + (2) = 6	[6]
13 / 5	压入数字 5	[6, 5]
14 / 6	压入数字 6	[6, 5, 6]
15 / *	先弹右 6，再弹左 5；(5) * (6) = 30	[6, 30]
16 / +	先弹右 30，再弹左 6；(6) + (30) = 36	[36]
17 / 7	压入数字 7	[36, 7]
18 / 8	压入数字 8	[36, 7, 8]
19 / *	先弹右 8，再弹左 7；(7) * (8) = 56	[36, 56]

步骤 / 读到 token	这一动作	动作后的完整数值栈
20 / *	先弹右 56，再弹左 36；(36) * (56) = 2016	[2016]
21 / 9	压入数字 9	[2016, 9]
22 / +	先弹右 9，再弹左 2016；(2016) + (9) = 2025	[2025]

最终值为 2025，数值栈峰值 3 项。

30.20 第 19 式

原式：((0!+1)*2)!+(3!-4)^(5+6)-7*8+9

0!+1=2，再乘 2 得 4，整个括号阶乘为 24。另一路 3!−4=2、5+6=11，得到 2048，最终 24+2048−56+9=2025。

```
0 ! 1 + 2 * ! 3 ! 4 - 5 6 + ^ + 7 8 * - 9 +
```

步骤 / 读到 token	这一动作	动作后的完整数值栈
1 / 0	压入数字 0	[0]
2 / !	弹 0，算 0!=1，压回	[1]
3 / 1	压入数字 1	[1, 1]
4 / +	先弹右 1，再弹左 1；(1) + (1) = 2	[2]
5 / 2	压入数字 2	[2, 2]
6 / *	先弹右 2，再弹左 2；(2) * (2) = 4	[4]
7 / !	弹 4，算 4!=24，压回	[24]
8 / 3	压入数字 3	[24, 3]
9 / !	弹 3，算 3!=6，压回	[24, 6]
10 / 4	压入数字 4	[24, 6, 4]
11 / -	先弹右 4，再弹左 6；(6) - (4) = 2	[24, 2]
12 / 5	压入数字 5	[24, 2, 5]
13 / 6	压入数字 6	[24, 2, 5, 6]
14 / +	先弹右 6，再弹左 5；(5) + (6) = 11	[24, 2, 11]
15 / ^	先弹右 11，再弹左 2；(2) ^ (11) = 2048	[24, 2048]
16 / +	先弹右 2048，再弹左 24；(24) + (2048) = 2072	[2072]
17 / 7	压入数字 7	[2072, 7]
18 / 8	压入数字 8	[2072, 7, 8]
19 / *	先弹右 8，再弹左 7；(7) * (8) = 56	[2072, 56]
20 / -	先弹右 56，再弹左 2072；(2072) - (56) = 2016	[2016]
21 / 9	压入数字 9	[2016, 9]
22 / +	先弹右 9，再弹左 2016；(2016) + (9) = 2025	[2025]

最终值为 2025，数值栈峰值 4 项。

30.21 第 20 式

原式：((0!^1+2)!!/(3!-4-5+6)-(7+8))*9

0!^1=1，加 2 为 3，双阶乘 3!!=3。分母 3!−4−5+6 按左结合为 6−4−5+6=3，所以商 1；减 15 得−14，再乘 9 得−126。

```
0 ! 1 ^ 2 + !! 3 ! 4 - 5 - 6 + / 7 8 + - 9 *
```

步骤 / 读到 token	这一动作	动作后的完整数值栈
1 / 0	压入数字 0	[0]
2 / !	弹 0，算 0!=1，压回	[1]
3 / 1	压入数字 1	[1, 1]
4 / ^	先弹右 1，再弹左 1；(1) ^ (1) = 1	[1]
5 / 2	压入数字 2	[1, 2]
6 / +	先弹右 2，再弹左 1；(1) + (2) = 3	[3]
7 / !!	弹 3，算 3!!=3，压回	[3]
8 / 3	压入数字 3	[3, 3]
9 / !	弹 3，算 3!=6，压回	[3, 6]
10 / 4	压入数字 4	[3, 6, 4]
11 / −	先弹右 4，再弹左 6；(6) − (4) = 2	[3, 2]
12 / 5	压入数字 5	[3, 2, 5]
13 / −	先弹右 5，再弹左 2；(2) − (5) = −3	[3, −3]
14 / 6	压入数字 6	[3, −3, 6]
15 / +	先弹右 6，再弹左-3；(−3) + (6) = 3	[3, 3]
16 / /	先弹右 3，再弹左 3；(3) / (3) = 1	[1]
17 / 7	压入数字 7	[1, 7]
18 / 8	压入数字 8	[1, 7, 8]
19 / +	先弹右 8，再弹左 7；(7) + (8) = 15	[1, 15]
20 / −	先弹右 15，再弹左 1；(1) − (15) = −14	[−14]
21 / 9	压入数字 9	[−14, 9]
22 / *	先弹右 9，再弹左-14；(−14) * (9) = −126	[−126]

最终值为-126，数值栈峰值 3 项。

30.22 第 21 式

原式：`((0!^1+2)!!/(3!-(4+(5-6)))-(7+8))*9`

分母的嵌套必须逐层算：5−6=−1，4+(−1)=3，3!−3=3。分子仍 3，故商 1，减 15 再乘 9 得−126。它与第 20 式分母数值相同，但 RPN 的减法次序不同。

```
0 ! 1 ^ 2 + !! 3 ! 4 5 6 - + - / 7 8 + - 9 *
```

步骤 / 读到 token	这一动作	动作后的完整数值栈
1 / 0	压入数字 0	[0]
2 / !	弹 0，算 0!=1，压回	[1]
3 / 1	压入数字 1	[1, 1]

步骤 / 读到 token	这一动作	动作后的完整数值栈
4 / ^	先弹右 1，再弹左 1；(1) ^ (1) = 1	[1]
5 / 2	压入数字 2	[1, 2]
6 / +	先弹右 2，再弹左 1；(1) + (2) = 3	[3]
7 / !!	弹 3，算 3!!=3，压回	[3]
8 / 3	压入数字 3	[3, 3]
9 / !	弹 3，算 3!=6，压回	[3, 6]
10 / 4	压入数字 4	[3, 6, 4]
11 / 5	压入数字 5	[3, 6, 4, 5]
12 / 6	压入数字 6	[3, 6, 4, 5, 6]
13 / −	先弹右 6，再弹左 5；(5) − (6) = −1	[3, 6, 4, −1]
14 / +	先弹右-1，再弹左 4；(4) + (−1) = 3	[3, 6, 3]
15 / −	先弹右 3，再弹左 6；(6) − (3) = 3	[3, 3]
16 / /	先弹右 3，再弹左 3；(3) / (3) = 1	[1]
17 / 7	压入数字 7	[1, 7]
18 / 8	压入数字 8	[1, 7, 8]
19 / +	先弹右 8，再弹左 7；(7) + (8) = 15	[1, 15]
20 / −	先弹右 15，再弹左 1；(1) − (15) = −14	[−14]
21 / 9	压入数字 9	[−14, 9]
22 / ∗	先弹右 9，再弹左-14；(−14) ∗ (9) = −126	[−126]

最终值为-126，数值栈峰值 5 项。

30.23 合上表格，试着自己找错

练习 1。 第 2 式如果使用 C++整数除法，中间 2∗6/24 会变成多少？最后还会是 2025 吗？

解答：2×6=12，整数 12/24 截为 0，后面无论乘多少仍为 0，最后结果 0。表达式字符串相同，数值类型与除法语义不同，计算问题就不同。本章有理数表适合精确核对数学值，不能直接冒充整数程序的输出。

练习 2。 第 16 式把 6−7−8 错读成 6−(7−8)，会怎样影响括号？

解答：正确为(6−7)−8=−9，错误为 6−(−1)=7。正确的 3/4−5×(−9)=183/4，错读后成 3/4−35=−137/4。RPN 中应是 6 7 − 8 −；若变成 6 7 8 − −，就是另一棵语法树。比较 RPN 可以在最终乘 9 之前就定位错误。

练习 3。 第 12 式与第 8 式结果相同，能否说明它们是同一个语法结构？

解答：不能。第 12 式需要先算指数 2!，第 8 式直接用 2；只是 2!=2 造成数值相同。换成指数 3 与 3!就会不同。测试解析器时需要既有结果不同的区分例，也有结果相同但结构不同的例子，避免把错误结构碰巧算对当成解析正确。

本章的数值栈峰值是求值 RPN 所需的空间，不是中缀转 RPN 时的运算符栈峰值。后者还取决于优先表、括号与底部哨兵计数；两种栈保存不同对象，不能拿一个峰值替另一个答案。

31 换掉数字以后，仍然能重做的六种推理

本章使用另设的输入讲方法，不公开转载本地受限样卷的原页、完整题面或个人信息。做这些题时，先把中间状态写出来，再用一个公式压缩它；若公式和表不一致，先查端点和计费对象，不急着记答案。

31.1 一、步长也在变，累计进度才决定轮数

考查下面的控制过程，i、j 都从 0 开始。每轮先把旧 j 加到 i，再让 j 加 1：

```
i=0, j=0
当i<n:
    i=i+j
    j=j+1
```

取 n=15，每步完整状态为：

已完成轮数 k	本轮加上的旧 j	轮末 i	轮末 j	下一轮还进吗
0	—	0	0	进
1	0	0	1	进
2	1	1	2	进
3	2	3	3	进
4	3	6	4	进
5	4	10	5	进
6	5	15	6	不进

第一轮 i 不变，但完整状态改变了：j 从 0 变 1。只看 i 会误判死循环。做完 k 轮，j=k，i 是 0 到 k−1 的和，因此

$$i = \frac{k(k-1)}{2}.$$

要停下来，必须且只需$i \ge n$。对 n>0，最小整数轮数是

$$k = \left\lceil \frac{1+\sqrt{1+8n}}{2} \right\rceil.$$

推导来自解二次不等式$k^2 - k - 2n \ge 0$，负根无关，取正根以后向上取整。本例正根 6，恰好 6 轮。n=0 时一开始不进，须单独返回 0，不能套正根式得 1。

只问渐近阶时，无需真的开方：当 k≥2，$k(k-1)/2$介于$k^2/4$与$k^2/2$之间，退出阈值就与$\sqrt{n}$相差固定倍数。所以轮数和常数主体时间均为$\Theta(\sqrt{n})$。

若把增量改成j^2，累计进度为$0^2 + 1^2 + \cdots + (k-1)^2 = \Theta(k^3)$，轮数变成$\Theta(n^{1/3})$；若改成$2^j$，累计为$2^k - 1$，轮数变成$\Theta(\log(n+1))$。每次先写“k 轮总前进多少”，再反解 n，适用于比背循环类型更广的情况。

需要精确机器结果时，浮点根式可能在很大的整数边界附近舍入。可用整数二分找最小 k，使$k(k-1)/2 \ge n$，或先估计再用整数比较修正；乘法也需防溢出。渐近推导和工程实现是两个相连但不同的任务。

31.2 二、容量每次只长平方根，扩容费怎样摊

规则改成：容量 C 满时，新容量为$C + \lceil \sqrt{C} \rceil$。初始容量 1，只做单项尾追加。下面按真规则列出到 16 项时发生过的扩容：

触发追加的序号	旧容量 C	增加格数$\lceil \sqrt{C} \rceil$	新容量	本次搬旧项
2	1	1	2	1
3	2	2	4	2

触发追加的序号	旧容量 C	增加格数$\lceil\sqrt{C}\rceil$	新容量	本次搬旧项
5	4	2	6	4
7	6	3	9	6
10	9	3	12	9
13	12	4	16	12

其余追加只写新项，不搬旧项。前 16 项共搬$1+2+4+6+9+12=34$项，另有 16 次新项写入，总 50。单个表还看不出一般阶，需要把扩容按容量大小分组。

看完整容量段$[M,2M]$。在这一段内，根号增长量至少约$\sqrt{M}$、至多约$\sqrt{2M}+1$，都与$\sqrt{M}$相差固定倍数。跨过长度约 M，需要$\Theta(\sqrt{M})$次扩容；每次旧容量在 M 与 2M 之间，搬$\Theta(M)$项，因此这一段总搬移是

$$\Theta(\sqrt{M})\cdot\Theta(M)=\Theta(M^{3/2}).$$

接着把容量段取 1 到 2、2 到 4、4 到 8……。这些段的搬移量按固定比$2^{3/2}$增长，所以总和不超过末段的固定倍数；在 n 下面选一个已完整走过、M 与 n 相差固定倍数的容量段，也给出$\Omega(n^{3/2})$下界。于是做 n 次追加的总复制为$\Theta(n^{3/2})$，加 n 次新写入后仍同阶。除以操作次数 n，前 n 次的摊还成本为$\Theta(\sqrt{n})$，不是常数。

还可以用积分检查同一结果：在容量 C 附近，每增加$\sqrt{C}$格付一次 C 的迁移费，因此每增加 1 格平均约付$C/\sqrt{C}=\sqrt{C}$。从小规模累计到 n，面积为

$$\int_1^n \sqrt{C}\,dC=\frac{2}{3}(n^{3/2}-1).$$

积分解释的是增长趋势，严格上下界来自前面的完整容量段夹逼；不能只把“次数大概除一下”当成离散精确等式。

若每次增加$\Theta(C^{\alpha})$，其中$0\le\alpha<1$固定，在$[M,2M]$中扩容次数为$\Theta(M^{1-\alpha})$，每次搬$\Theta(M)$，总量为$\Theta(M^{2-\alpha})$。全程也为$\Theta(n^{2-\alpha})$。α=0 就是固定增加常数、总平方；α=1 的固定比例增长可另用几何和得到总线性。若 α 随 C 变、存在额外大跳跃或允许删除，原操作序列与夹逼条件变化，要重新分析。

31.3 三、表达式树：先算左边，为什么会占着一格等右边

这里只统计 RPN 执行时的**数值栈**，不计运算符栈，也不计语言递归调用帧。叶子读入时压一个数，二元运算弹两个、压一个。固定先完成左子树、再右子树。

叶子所需峰值$S(\text{叶})=1$。对根 T，左子树为 L、右子树为 R：算左边时最多用$S(L)$格；左边完成后留下一个结果；算右边时这个左值仍在底下，要再加$S(R)$格。因此

$$S(T)=\max\big(\,S(L),1+S(R)\big)\,.$$

这个式子中的 1 是“已经算好的左结果”，不是根运算符自己占一个数值格。

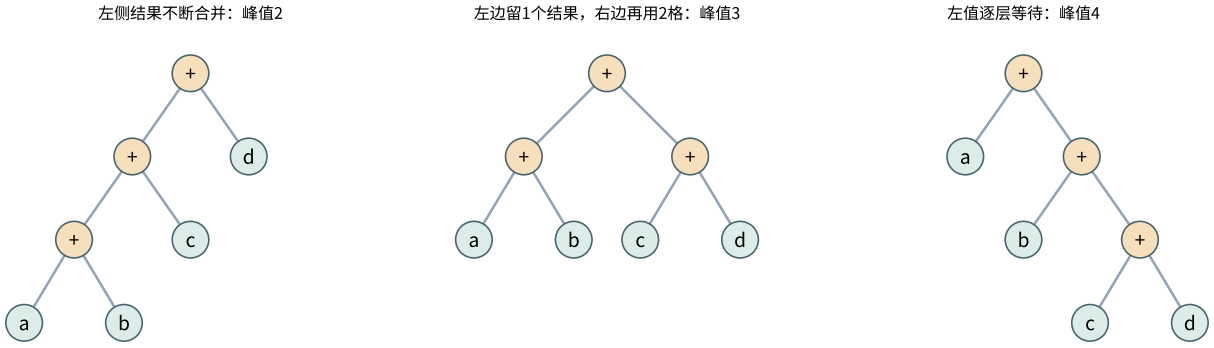


图 207　同样四个叶子的三棵表达式树。固定先算左边时，左倾树不断合并只需 2 格，平衡树需 3 格，右倾树的多个左值同时等待，需 4 格。

对`(a+b)+(c+d)`，用具体数 1、2、3、4 走一次：

```
读a=1: [1]
读b=2: [1,2]       峰2
读+:   [3]
读c=3: [3,3]
读d=4: [3,3,4]     峰3
读+:   [3,7]
读+:   [10]
```

左树需 2 格、右树需 2 格，根峰值 max(2,1+2)=3，完全吻合。换成左倾(((a+b)+c)+d)，每次刚压第二数就合并，峰值只有 2；换成右倾 a+(b+(c+d))，要先压 a 等，再压 b 等，再压 c、d，峰值 4。三棵树都有 4 个叶、3 个运算符，但求值顺序造成的等待不同。

如果允许先算右树，空间需求改为$\max(S(R), 1+S(L))$。选较小者可以省栈；但仍需保存每个结果的左右身份。计算减法时先算右边，不表示最后可以把“左减右”换成“右减左”；有副作用或规定执行次序的语言表达式也未必允许重排。

31.3.1 额外限制“左子树不少于右子树”时，峰值能有多大

设每个内部节点都满足左子树内部节点数不少于右子树。问达到峰值 s 至少需要多少内部节点，记为N_s。这里只考虑纯二元表达式，叶子不是内部节点。

$N_1 = 0$，一个叶就能达到峰值 1。选一棵达到峰值 s 且内部节点最少的树。如果峰值完全由左子树的$S(L) = s$产生，单独取这棵左子树会用更少节点也达到 s，与最小性冲突。所以在最小构造中，峰值必须由$1 + S(R)$产生，即右子树至少需要峰值 s−1、至少有N_{s-1}个内部节点；左侧按限制至少同样大，再加根，得到

$$N_s \geq 2N_{s-1} + 1.$$

取两棵相同的最小 s−1 构造作为左右孩子，就可以达到等号。因此不是只得到下界，而是

$$N_s = 2^{s-1} - 1.$$

前几个值为 0、1、3、7、15；对应峰值 1、2、3、4、5。若有 k 个内部节点，则$2^{s-1} - 1 \leq k$，取对数得

$$s \leq 1 + \lfloor \log_2(k+1) \rfloor.$$

15 个内部节点的完美树确实需 5 格。15 个内部节点的全左长树却只需 2 格；公式是可能达到的峰值上界，没有说同规模所有形状都同峰值。

31.4 四、全数组有很多逆序，指定区间里至少留下多少

逆序是下标 i<j 但值$a_i > a_j$的一对。长度 n 的数组共有$n(n-1)/2$对下标，指定长度 m 的连续区间内部有$m(m-1)/2$对；其余对都是“至少一端在区间外”。无论怎样摆值，区间外能承担的逆序最多是

$$B = \frac{n(n-1) - m(m-1)}{2}.$$

如果总逆序数是 I，内部至少要接住超过 B 的部分，所以

$$I_{\text{内}} \geq \max(0, I - B).$$

这里 I 是已知总量，B 是外部容量上界，不是已经保证能同时实现的外部实际逆序数。要声称最少“恰好”，还要给达到下界的构造。

取 n=6，指定前 4 项为区间，m=4，总 I=13。全部 15 对，内部最多 6 对，外部容量 B=9，所以内部至少 4。现在真的构造：

```
[5,6,3,4 | 2,1]
```

内部[5,6,3,4]的逆序是(5,3)、(5,4)、(6,3)、(6,4)，共 4；前 4 项每项都大于后面的 2、1，跨界有 8；尾部 2>1 又有 1，总 4+8+1=13。所有条件同时满足，所以在这个指定区间例子中最少确实为 4。

这比随意说“把剩下 9 个放外面”多了一步：构造确认三部分的数值大小关系能共存。若区间位置改变或另外要求某些位置固定，未必仍能原样达到。

若 I=5，同样的粗下界只能给 0。反例`[1,3,5,6 | 2,4]`前 4 项有序，内部 0 逆序；跨界逆序为(3,2)、(5,2)、(5,4)、(6,2)、(6,4)，恰 5。于是“总共有 5 个逆序，所以这里至少有一个”的直觉也被实际排列反驳了。

31.5 五、归约：新问题不能制造旧问题没有的答案

设旧问题给互异实数$x_1, \dots, x_n$，要判断有没有三个不同索引的数和为 0。把每个 x 变成点(x, x^3)，调用一个“找三点共线”的算法。为什么找共线能帮零和，必须证明两个方向。

任取三个互异数 a、b、c。横坐标不同，斜率分母都不为 0。a 与 b 对应点的斜率是

$$\frac{b^3 - a^3}{b - a} = a^2 + ab + b^2,$$

a 与 c 对应点的斜率是$a^2 + ac + c^2$。两斜率相减：

$$ab + b^2 - ac - c^2 = (b - c)(a + b + c).$$

b−c 不为 0，因此斜率相等当且仅当 a+b+c=0。正向说明真解不会丢；反向说明算法找到的共线三点不会是伪解。

a,b,c	和	AB 斜率	AC 斜率	共线吗
−2,−1,3	0	$4+2+1=7$	$4-6+9=7$	是
−2,−1,2	−1	7	$4-4+4=4$	否

生成 n 个点，在实数 RAM 且乘法单位成本的模型中每点只做常数次乘法，总$O(n)$；保留原索引，找到三点后也可常数时间映回原三数。若使用 L 位整数，立方结果约 3L 位，运算成本与输出长度要按位模型另算。若允许重复数值，点可能重合，必须先单独处理重复的零和组合或改变归约条件，不能直接复用互异证明。

把箭头写成“零和问题→共线问题”。这表示共线快算法可以解决零和问题；若已知零和在某个模型中有下界，才可转移成共线下界。仅仅说共线“看起来难”，不能反向证明零和难；本章也没有借这一归约宣称已经证明某个未给出的二次时间下界。

31.6 六、找下一个节点的费用，也属于遍历

先检查输入信息够不够唯一确定目标。普通先序 A,B 和后序 B,A，可以对应 A 左孩子 B，也可以对应 A 右孩子 B；没有空标记或真树限制，不能写一个算法“恢复唯一原树”。把不唯一问题硬编码成任选一种，得到的是一个可能解，承诺已经变了。

如果给 n 条 parent 记录，只能确定父子关系；若还要区分左右，必须另给孩子槽位或顺序。可以先扫描 n 条记录建立孩子表，预处理$O(n)$，再做常规 DFS。这个预处理不是免费，也不能从“父亲可直接读”推断“孩子也可以直接读”。

以树 A 左 B 右 C、B 右 D 为例，显式栈保存待办节点，先序如下：

```
初始[A]
弹A访问，先压C再压B：栈[C,B]
弹B访问，压D：栈[C,D]
弹D访问：栈[C]
弹C访问：栈[]
```

每个孩子只由唯一父亲压一次，每个节点也只弹一次，得到线性总工作。若改为“每次输出后，从根搜索下一个节点”，即使最终输出仍是 A、B、D、C，搜索过程中重复走过的边也可能很多。访问次数与定位费用必须分开。

同理，分治求和有对数层，并不只有对数时间。根处理左右两半，每层会出现许多调用；总时间把所有调用相加，调用栈空间则只看某一时刻同时活着的一条路径。第 32 章 G(G(n))又是另一种情况：先算内层返回值，再以它作为外层入口规模，不能把“两次调用”简单相加成两份同规模费用。

31.7 自己更换输入

1. 容量每次增加$\lceil C^{1/3} \rceil$，到 n 项的总复制是什么阶？说明“每次扩容多少次”的分母来自哪里。

2. `(a+b)*(c+(d+e))`固定左先右后，写 RPN 值栈峰值。随后尝试右先，是否减少？
3. 将 n=6、前 4 项区间、总 I=13 的构造换一个，使内部仍恰 4。不能只报公式。
4. 归约若只证明零和推出共线，会留下哪一种错误可能？

答案：

1. 在容量段$[M, 2M]$里，每次只增加$\Theta(M^{1/3})$格，要填满$\Theta(M)$格需要$\Theta(M^{2/3})$次，每次搬$\Theta(M)$，段总$\Theta(M^{5/3})$；全程同阶为$\Theta(n^{5/3})$。分母是每次实际增加的格数，不是每次复制量。
2. 左树需 2，右树 `c+(d+e)`需 3，先左后右为 max(2,1+3)=4。先右时右树内部仍按原顺序需 3，留 1 值后算左最多 1+2=3，所以根可降到 3；乘法允许该重排且无副作用时可以采用。若连右树内部也重排，还要单独求它的最小需求。
3. 例如`[6,4,3,5,2,1]`：前 4 项内逆序为(6,4)、(6,3)、(6,5)、(4,3)，共 4；跨界 8、尾部 1，总 13。数值不同、分段计数相同。
4. 可能有和不为 0 的三数也被映射成共线，导致新问题返回真而旧问题其实是假。只有双向等价加上合法索引映射，才能安全调用新问题的求解器。

32 习题 1–32：把十六段程序逐层数清楚

PS01 第 16 项让我们独立分析《数据结构习题解析》1-32。本章对应作者官方合体 PDF 第 446 到 455 页，亦即习题解析印刷页 24 到 33；第 456 页为空白。原程序可从作者教材页进入合体 PDF 阅读。下面保留题号顺序，用自己的推导解释每段执行成本，并把容易混淆的返回值、轮数与递归深度分别列出。
题目使用抽象机器，假设字长足够、移位是单位操作、递归不会因真实机器栈限制而失败。这里的 T 表示执行成本，V 表示返回值。把原程序直接交给固定字长 C++运行，可能先遇到溢出或非法移位；若用大整数，又要另算位成本。这些实现边界不改变题目指定模型下的推导，却决定实验能测到什么。

32.1 1：矩形循环区域，行数乘列数

控制代码（按原题端点）：

```
for (int i = 0; i < n; ++i)
    for (int j = 0; j < n; ++j) { /* 计一次 */ }
```

第一段的核心是 n 轮外循环，每轮 n 次内循环。若把一次主体执行画成坐标(i,j)，正好是 n×n 方格，每格访问一次，总 n^2次。
n=4 时，每行 4 次，共 4+4+4+4=16 次。每个循环还要增加变量、检查条件，内层条件每行约 n+1 次，总也为 $\Theta(n^2)$，不会改变阶。基础操作不是只有函数体里面那一条，但主项可由它代表。
把 n=3 的每一个内层状态列出来，就得到坐标格，不依赖“二重循环”这个外观：

```
外i=0：j=0 → 1 → 2，3次；j变3，条件失败。
外i=1：j重新置0，再走0 → 1 → 2，3次。
外i=2：j又重新置0，再走0 → 1 → 2，3次。
i变3，外条件失败；总9次主体。
```

每次重入内循环都会重新初始化 j，所以它不是“j 只从 0 走到 n 一次”。

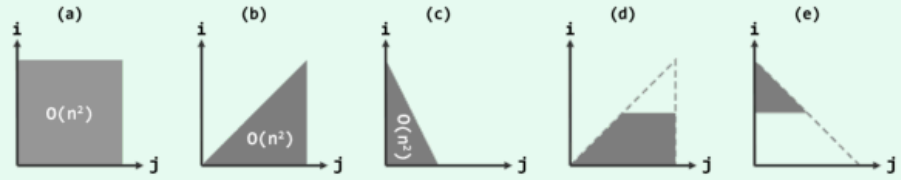
```
void F(int n) {
   for (int i = 0; i < n; i ++)
   for (int j = 0; j < n; j ++);
}
```

【解答】

这是最基本的二重循环模式，其特点是循环控制变量均按算术级数变化。内循环的每次迭代只需$O(1)$时间，故执行时间取决于总体的迭代次数。因外、内循环的范围分别是$i \in [0, n)$和$j \in [0, n)$，故累计迭代次数为：

$$\sum_{i=0}^{n-1}\sum_{j=0}^{n-1}1=\sum_{i=0}^{n-1}n=n\times n=O(n^2)$$

实际上在如图x1.9(a)所示的(i, j)坐标平面上，此二重循环的执行过程对应于匀速地自下而上、由左到右逐行扫过矩形$[0, n) \times [0, n)$。因此，这一过程所需的时间也就对应于该矩形的面积，就渐进意义而言即是$O(n^2)$。



图x1.9 二重循环执行时间的对应图形

相对于以上严格的计算，基于这一图形式理解的估算不仅更加简捷，而且可以更好地体现通过大O记号刻画渐进复杂度总体趋势的要义。而对于接下来更为复杂的情况，这种方法的上述优势则尤为突出。

```
void F(int n) {
   for (int i = 0; i < n; i ++)
   for (int j = i; j < n; j ++);
}
```

【解答】

这也是一种典型的二重循环模式，循环控制变量仍然按算术级数变化。与上例相比，外循环的范围依然是$i \in [0, n)$，但内循环的范围则变成$j \in [i, n)$，故累计迭代次数为：

$$\sum_{i=0}^{n-1}\sum_{j=i}^{n-1}1=\sum_{i=0}^{n-1}(n-i)=\sum_{(k=n-i)=1}^{n}k=n(n+1)/2=O(n^2)$$

也可套用以上基于图形的估算方法，在如图x1.9(b)所示的(i, j)坐标平面上，此二重循环的执行过程对应于匀速地自下而上、由左到右逐行扫过三角形$[0, n) \times [i, n)$。该三角形的面积大致为原矩形的一半——就渐进意义而言，时间复杂度依然是$O(n^2)$。

图 208　官方合体 PDF 第 447 页完整原图，含前两段程序与五种循环区域的几何图。阴影中的一个格代表一次主体动作；本章逐段重新核对边界，尤其第 5 段按实际 n-i 计数。

上界是常数倍 n^2，下界同样至少有 n^2 次主体，因此为 $\Theta(n^2)$。若只写“有两层 for 所以平方”，这份理由在第 6 与第 8 段就会失效；这里正确是因为每行确实有 n 项。

32.2 2：三角形区域，逐行长度不同

控制代码（按原题端点）：

```
for (int i = 0; i < n; ++i)
    for (int j = i; j < n; ++j) { /* 计一次 */ }
```

第二段内层从当前外层位置扫到边界。各行长度依次 n、n−1、…、1，因此

$$N(n)=\sum_{i=0}^{n-1}(n-i)=\frac{n(n+1)}{2}=\Theta(n^2).$$

n=4 时为 4+3+2+1=10。它大约是第一段的一半，但渐近阶相同。判断下界可只看前 n/2 行：每行至少 n/2 次，已足以贡献常数倍 n^2。

n=4 时每行实际 j 值是 `i=0:0,1,2,3`；`i=1:1,2,3`；`i=2:2,3`；`i=3:3`。将第 1 行与第 4 行相加是 4+1=5，第 2 行与第 3 行也是 3+2=5，共两对得到 10。一般把 1 到 n 与倒序 n 到 1 相加，每对为 n+1，共 n 对，再除 2，得到$n(n+1)/2$。求和公式不是额外假设，而是把同一批行长度配对。

如果循环端点改为不含当前位置，各行变 n−1、…、0，精确次数变 n(n−1)/2，仍平方级。端点差一会影响精确值，不能在考试问“执行几次”时直接忽略；问 Θ 时才可在证明后舍去低阶项。

32.3 3：内层步长 2013，常数再大也仍是常数

控制代码（按原题端点）：

```
for (int i = 0; i < n; ++i)
    for (int j = 0; j < i; j += 2013) { /* 计一次 */ }
```

内层每次增加固定 2013，第 i 轮约执行 i/2013 次。取整误差每轮至多常数，总误差 O(n)，主和为

$$\frac{1}{2013}\sum_{i=1}^{n} i=\Theta(n^2).$$

例如将 2013 暂时换成便于手算的 3，外层上限 i 从 1 到 9，相应循环次数可能按端点约定取 ceil(i/3)，依次 1、1、1、2、2、2、3、3、3，总 18；外层规模扩大，仍是二次增长。这个小数值示例解释步长作用，不替换原题中的固定常数。

按原常数取几个确切边界：i=0 时一次也不做；i=1 到 2013，j 只有 0，做 1 次；i=2014 到 4026，j 为 0、2013，做 2 次；i=4027 时 j 为 0、2013、4026，做 3 次。故每行恰为$\lceil i/2013\rceil$。n=4028 时外层最后 i=4027，总主体数为$2013\times1+2013\times2+1\times3=6042$。这个小表同时验证了严格小于边界和取整方向。

原常数很大，n 在几百时可能几乎看不到主体工作，测得曲线也可能像线性甚至常数。渐近结论讨论无界增长的 n，不由小规模测时曲线决定。若步长改成 n，就不再是固定常数，必须重新求和。

32.4 4：只做前半外循环，内层却仍然很长

控制代码（按原题端点）：

```
for (int i = 0; i < n / 2; ++i)
    for (int j = i; j < n; ++j) { /* 计一次 */ }
```

外层只有约 n/2 轮，每轮内循环长度在约 n/2 到 n 之间。上下夹逼为

$$c_1n^2\le N(n)\le c_2n^2$$

其中 c_1、c_2为与 n 无关的正常数，所以 $\Theta(n^2)$。n=8 时，若内层长度为 8、7、6、5，总 26，已经能看出不是只执行 4 次。

n=8 时外层 i 只取 0、1、2、3，因为整数 n/2 等于 4；内层分别取 `0..7`、`1..7`、`2..7`、`3..7`，行长 8、7、6、5。若 n=9，外层仍只有 4 行，行长 9、8、7、6；不能把 `i<n/2` 在整数代码里擅自当作$i<4.5$而多走 i=4。精确式可令$h=\lfloor n/2\rfloor$，总数$hn-h(h-1)/2$。

这里不必为了得到 Θ 而求一个很长的精确闭式。行数 Θ(n)、每行长度 Θ(n)，且界对所选所有行都成立，直接相乘即可。若某些行短某些行长，不能只取最小或最大后就宣称紧界，应检查足够多行都提供所需下界。

32.5 5：后半个三角形，仍有平方数量的格子

控制代码（按原题端点）：

```
for (int i = n / 2; i < n; ++i)
    for (int j = 0; j < n - i; ++j) { /* 计一次 */ }
```

外层走后半段，内层长度从约 n/2 逐步减到 1。总和约为 1+2+…+n/2，所以 Θ(n²)。n=8 的简化行长 4、3、2、1，总 10；n 翻倍时行数与最长行都翻倍，面积约变 4 倍。

n=8 时，外 i=4、5、6、7，对应内 j 分别为 0..3、0..2、0..1、0，长度 4、3、2、1。n=9 时从 i=4 开始，行长 5、4、3、2、1，共 15。一般令$s = \lceil n/2 \rceil$，主体数为$s(s+1)/2$。原页说明中内范围有一处写成[0,i)，但实际代码是 j<n-i；这里必须跟代码而不是跟那句说明。

旧解答旁的说明若写成另一个区间，应回到实际循环条件数格子，而不是同时抄两套互相冲突的文字。求和下标的范围是程序语义的一部分。

前 5 段的共同结论都是平方，但理由不全一样：矩形、完整三角形、固定步长三角形、梯形、半三角形。把每种区域画出来，后续遇到换边界的程序就能重新分析，而不必记住第几个答案。

32.6 6：外面 n 轮，里面翻倍到 n

控制代码（按原题端点）：

```
for (int i = 0; i < n; ++i)
    for (int j = 1; j < n; j <<= 1) { /* 计一次 */ }
```

内层状态 1、2、4、8……，在达到 n 以前执行约 $\log_2 n$ 次。外层每次都重新从 1 开始，所以每一轮都要重新支付这份成本，总 Θ(n log n)。

n=16 时，若条件为 j<n，内层取 1、2、4、8，共 4 次，外层 16 轮，总 64 次；若条件为 j≤n，则每轮多一次取 16，总 80 次。两者同阶，精确计数却不同。

精确地，第 t 次主体时$j = 2^t$，t 从 0 开始。合法条件$2^t < n$，因此 n>1 时每轮主体数为$\lceil \log_2 n \rceil$。n=9 时 j 是 1、2、4、8，做 4 次；n=8 时没有最后的 8，只做 3 次。外层 n 次重置 j，总主体数就是$n\lceil \log_2 n \rceil$，端点已经明确。

不要因为内层变量翻倍就说“所有轮合起来是一个几何和”。这里每个 j 对应一次常数操作，而不是做 j 次操作；内层总成本是项数 log n。若主体改成重复 j 次，才需相加 1+2+4+...得到 Θ(n)，再乘外层 n 会变平方级。

32.7 7：翻倍到固定上限，关于 n 就是常数

控制代码（按原题端点）：

```
for (int i = 0; i < n; ++i)
    for (int j = 1; j < 2013; j <<= 1) { /* 计一次 */ }
```

内层翻倍却只到 2013，与输入规模 n 无关。因为 2^{10}=1024、2^{11}=2048，不论 n 多大，内层都只有约 11 次检查范围内的主体动作。外层 n 轮，总 Θ(n)。

完整内层轨迹是 1,2,4,8,16,32,64,128,256,512,1024，共 11 次；下一值 2048 不小于 2013 而退出。因此主体精确执行 11n 次，内层退出判断另做 n 次。n 从 100 涨到百万，只改变重复这条固定轨迹的次数。

第 6 与第 7 的代码可能只差一个上限，渐近结论就从 n log n 变为 n。分析要问“哪个量会随输入增长”，不能只搜源代码里的乘法或移位符号。

若常数 2013 本身变成另一个输入参数 M，则应写 Θ(n log M)，再根据 n、M 之间是否有关联决定是否可化简成单变量形式。

32.8 8：步长随外层变化，出现调和和式

控制代码（按原题端点）：

```
for (int i = 1; i < n; ++i)
    for (int j = 0; j < n; j += i) { /* 计一次 */ }
```

第 i 轮内层跨步 i，遍历长度约 n，因此次数为 ceil(n/i)而非 i。外层 i 为 1 到 n−1 时，总数

$$\sum_{i=1}^{n-1}\lceil n/i\rceil = nH_{n-1} + O(n) = \Theta(n\log n).$$

n=8 时各轮 i=1 到 7 的次数 8、4、3、2、2、2、2，总 23。若外层也包含 i=8，再加 1。取整误差每项小于 1，n 项合计 O(n)，比主项 n log n 小。

n=8 各行真正访问的 j 是：

i	j 的值	次数
1	0,1,2,3,4,5,6,7	8
2	0,2,4,6	4
3	0,3,6	3
4	0,4	2
5	0,5	2
6	0,6	2
7	0,7	2

i=7 时并不是 1 次，因为 j=0 也执行一次。由$j = ti < n$得$t < n/i$，从 t=0 计数恰为$\lceil n/i\rceil$，所以这里不能误用向下取整。每项介于n/i与$n/i+1$之间，求和误差才小于 n。

H_n 为对数级，可把分母按 2 的幂分组：1 一组，2 到 3 一组，4 到 7 一组，每组总贡献在正常数范围内，共 Θ(log n)组。这个解释也说明为什么“大量后面很小的项”仍会累积出一个对数因子。

32.9 9：一个循环，累计步长是三角数

控制代码（按原题端点）：

```
for (int i = 0, j = 0; i < n; i += j, ++j) { /* 计一次 */ }
```

i、j 从 0 开始，每轮做 i+=j，再让 j 加 1。k 轮后 i=k(k−1)/2、j=k。n=10 时状态依次(0,1)、(1,2)、(3,3)、(6,4)、(10,5)，5 轮结束。

退出需要 k(k−1)/2≥n，故 k=Θ(√n)，每轮常数工作，总 Θ(√n)。第一轮 i 没变，但 j 变了；若只盯 i 就说“状态不动”，会漏掉下一轮增量来源。

这里的输入 n 以一个整数给出；若按 n 的二进制位数 b 计，√n 仍可约为$2^{b/2}$。模型中所谓“平方根时间”是关于数值 n，不自动等于关于输入编码长度的多项式时间。

32.10 10：i 的指数也在翻倍，得到双对数

控制代码（按原题端点）：

```
for (int i = 1, r = 1; i < n; i <<= r, r <<= 1) { /* 计一次 */ }
```

r 从 1 开始每轮加倍，i 从 1 开始每轮左移 r 位。左移 r 位是乘2^r，不是乘 r，也不是只乘 2。

逐轮列出：初始 i=1、r=1；第一轮 i=2、r=2；第二轮 i=8、r=4；第三轮 i=128、r=8；第四轮 i=32768、r=16。k 轮后 i 的二进制指数是 1+2+4+…+2^{k-1}=2^k−1，所以

$$i = 2^{2^k-1}.$$

两条逗号更新从左向右执行，i 使用的是本轮旧 r。若 n=1000：

已完成轮数 k	i	r	下一次 i<n
0	1	1	真
1	2	2	真
2	8	4	真
3	128	8	真
4	32768	16	假，停止

所以恰 4 轮。把“左移 r 位”误当“左移 1 位”会得到完全不同的轨迹；把更新顺序反过来也会变成 2、8 等幂次错位。

让 i 达到 n，需$2^k-1\ge\log_2 n$，故 k=Θ(log log n)。这是对数的对数，来源是指数本身在几何增长。真实 C++在移位位数达到类型位宽前就要特别处理，不能把这张抽象表无限照跑。

32.11 11：指数塔增长，反复取对数数回去

控制代码（按原题端点）：

```
for (int i = 1; i < n; i = 1 << i) { /* 计一次 */ }
```

i 每次变为2^i，状态 1、2、4、16、65536……。到达一个极大阈值 n，倒着问：对 n 反复取 $\log_2$，多少次降到不大于 1？这个次数记为 $\log_2^*$ n，读作迭代对数。

例如 n=65536，依次取 log 得到 16、4、2、1，共 4 次；正向从 1 到 65536 也是 4 次指数化。成本 Θ($\log^*$ n)，虽然现实规模下数值很小，作为无界函数仍会增长，不能严格写成 Θ(1)。

取 n=17，正向为 1→2→4→16→65536，4 次主体后结束；反向取对数是 17→约 4.087→约 2.031→约 1.022→约 0.032，也是 4 次降到不大于 1。这里 log 星数的是操作“反复取对数”的次数，不能把星号读成乘号。n≤1 时循环 0 次，讨论渐近阶时默认 n 足够大。

它也不是 log log n。若 n 是高度不断增长的指数塔，双对数仍随塔高后的庞大数值变化，log*只记录需要剥掉多少层指数。

32.12 12：G 返回平方，第二次调用吃的是返回值

控制代码（按原题端点）：

```
int F(int n) { return n > 0 ? G(G(n - 1)) : 0; }
int G(int n) { return n > 0 ? G(n - 1) + 2*n - 1 : 0; }
```

辅助 G(m)把前 m 个奇数相加：1+3+…+(2m−1)=m^2，所以返回值 V_G(m)=m^2，执行成本 T_G(m)=Θ(m)。主函数先算 G(n−1)，再将返回的$(n-1)^2$作为另一次 G 的参数。

取 n=5，第一次 G(4)加 4 个奇数得到 16；第二次 G(16)加 16 个奇数得到 256。总加法次数约 4+16=20，而返回 256。一般返回$(n-1)^4$，运行成本 Θ(n)+Θ($(n-1)^2$)=Θ(n^2)。

以 G(4)展开，调用依次是 G4→G3→G2→G1→G0；返回依次 0、1、4、9、16，新增的数为 1、3、5、7。每次从$k-1$项的平方加上$2k-1$，恰为k^2，因此返回值与成本都能归纳证明。第二次 G(16)则有 17 个调用、16 次加奇数；加上第一次 5 个调用，共 22 个 G 调用，而不是 256 个。若统计的是非基础递归步，则是 4+16=20，两个计数口径只差基例，均为平方阶。

把“返回多大”和“执行多久”分成两列，就不会把四次方返回值误当四次方时间。相反，一个函数也可以花很久最后只返回 0，输出大小本身并不决定所有计算成本。

32.13 13：昂贵的是循环条件里的开方子过程

控制代码（按原题端点）：

```
void F(int n) { for (int i = 1; i < n / G(i, 0); ++i) {} }
int G(int n, int k) {
    return n < 1 ? k : G(n - 2*k - 1, k + 1);
}
```

G 从输入 m 中依次减去 1、3、5……，第 k 次累计减去 k^2，首次使剩余非正时返回 k，所以 G(m,0)=ceil(√m)，成本 Θ(√m)。

例如 m=10，依次减 1 后余 9，减 3 余 6，减 5 余 1，减 7 余−6，共 4 步，ceil√10=4。m=9 时减 1、3、5 恰到 0，共 3 步，边界也吻合。

F 在第 i 次检查中调用 G(i,0)，条件大致 i<n/G(i,0)。以 G 约√i 代入，外层能走到 I 满足$I^{3/2}$约 n，所以 I=Θ($n^{2/3}$)。这只是条件被检查的轮数，不是总成本，因为每次检查本身花 Θ(√i)。

因此

$$T(n)=\Theta\left(\sum_{i=1}^{I}\sqrt{i}\right)=\Theta(I^{3/2})=\Theta(n).$$

平方根和为何是$I^{3/2}$？上界 I 项每项至多√I；下界后 I/2 项每项至少√(I/2)，两者相乘即可。若只写外循环 $n^{2/3}$，会漏算条件调用；若再无脑乘一个最大√I，则能给上界，但还需后半段提供匹配下界。

原解答一处前面得$n^{2/3}$，后面却写$n^{1/3}$的文字，应以前面阈值与程序重新推导为准。整数截断版本的条件 `i < floor(n/ceil√i)`可准确变成`(i+1)ceil√i <= n`，便于用纯整数检查。

把 n=30 的所有条件检查都列出来。G 返回的是向上取整平方根；整数除法先截断，因此条件$i<\lfloor 30/G(i,0)\rfloor$等价于$(i+1)G(i,0)\le 30$：

i	G(i,0)	整除 30/G	条件通过吗	本次 G 减奇数次数
1	1	30	是	1
2	2	15	是	2
3	2	15	是	2
4	2	15	是	2
5	3	10	是	3
6	3	10	是	3
7	3	10	是	3
8	3	10	是	3
9	3	10	是	3
10	4	7	否，退出	4

主体 9 轮，条件 10 次，条件中减奇数的总次数 26。最后一次虽然不进空主体，G(10,0)仍真的执行了，不能漏算。阈值接近$n^{2/3}$而非$n^{1/3}$；原页后续把阈值估成三次方根的优化文字同样不适用于原条件，不能只修前面的一个指数就继续引用后面的错误建议。

本书`示例/复杂度计数.py`把逐个减奇数与独立整数平方根核对，再将逐项条件阈值与单调二分核对。它验证有限范围算例，没有拿排版结论自己核对自己。

32.14 14：F 只有一条递归链，G 却会越来越长

控制代码（按原题端点）：

```
int F(int n) { return n > 0 ? G(2, F(n - 1)) : 1; }
int G(int n, int m) { return m > 0 ? n + G(n, m - 1) : 0; }
```

G(a,b)通过把 a 累加 b 次返回 ab，成本 Θ(b)。F 从 F(0)=1 起，每层通过 G 把前次返回值乘 2，故 F(n)=2^n。

n=4 时，F 各层返回 1、2、4、8、16，对应 G 的第二参数依次 1、2、4、8，工作量总 1+2+4+8=15，加 F 自身 4 层常数工作，仍 Θ(16)。一般

$$T(n)=\Theta\left(n+\sum_{k=1}^{n}2^{k-1}\right)=\Theta(2^n).$$

最高层的调用关系应分成“先返回一个数，再拿这个数当重复次数”：

```
F4 等F3返回8，再执行G(2,8)，8次加法，返回16。
 F3 等F2返回4，再执行G(2,4)，4次加法，返回8。
  F2 等F1返回2，再执行G(2,2)，2次加法，返回4。
   F1 等F0返回1，再执行G(2,1)，1次加法，返回2。
    F0 直接返回1。
```

F 的调用链最深只有 5 帧，但最后的 G(2,8)有 9 个含基例的 G 帧。一般在顶层 G 执行时，深度可达$2^{n-1}+1$，所以不止运行时间指数，朴素递归实现的最大额外调用栈也指数。不要只数 F 那一条短链就答空间 O(n)。这里指数时间来自每层 G 参数越来越大，不是 F 有两个递归孩子。G 自身的递归深度为 Θ(b)；代入最高层 b=2^{n-1}以后，最大深度才对 n 呈指数。不能把 G(a,b)直接说成深度2^b，那又多套了一层指数。

32.15 15：孩子规模不同，总量甚至可以低于线性

控制代码（按原题端点）：

```
int F(int n) { return n > 3 ? F(n >> 1) + F(n >> 2) : n; }
```

递推为$T(n)=T(n/2)+T(n/4)+\Theta(1)$，带常数基础情况。令 n=$2^k$，记 S_k=T($2^k$)，得到$S_k=S_{k-1}+S_{k-2}+\Theta(1)$，吸收常数后与 Fibonacci 同阶，所以 S_k=Θ(φ^k)，φ=(1+√5)/2。
换回 n，k=$\log_2$n，得

$$T(n)=\Theta(n^{\log_2\varphi})\approx\Theta(n^{0.69424}).$$

例如规模 64 的孩子为 32、16；32 再分 16、8；16 再分 8、4。虽然每个非叶节点有两个孩子，孩子总规模只有父的 3/4，不像两半分治保留全部规模。只数孩子个数会丢掉缩小比例。
若以“一次进入 F 计 1 个调用”作精确成本，可从基础情况逐项算：

n	两孩子	总调用数 C(n)	$(C(n)+1)/2$
1	基例	1	1
2	基例	1	1
4	2、1	3	2
8	4、2	5	3
16	8、4	9	5
32	16、8	15	8
64	32、16	25	13

令$n=2^k$，$S_k=C(2^k)$，则$S_k=S_{k-1}+S_{k-2}+1$。为什么能“吸收常数”？定义$U_k=(S_k+1)/2$，直接代入就有$U_k=U_{k-1}+U_{k-2}$，表的最后一列正是 1、1、2、3、5、8、13。用 Fibonacci 的上下界$U_k=\Theta(\varphi^k)$，再把$k=\log_2 n$代回即可。调用总数 25 不是 F(64)返回值；返回值由基例的数值决定，是另一个问题。
另一种解法令指数 p 满足$2^{-p}+4^{-p}=1$，置 x=2^{-p}，则 x+x^2=1，正根 x=1/φ，得 p=$\log_2\varphi$。整数向下取整与很小规模的基础分支，需用单调性和相邻二次幂夹逼处理，改变常数而非指数。

32.16 16：随机分支，把每轮期望加起来

控制代码（按原题端点）：

```
for (int i = n; i > 0; --i)
    if (!(rand() % i))
        for (int j = 0; j < n; ++j) { /* 计一次 */ }
```

外层 i 由 n 降到 1，每轮若随机数对 i 余 0，就执行 n 次内层。理想模型假设每个模数下余数均匀，触发概率 1/i。记该轮是否触发为 X_i，不管各轮是否独立，都有 E[X_i]=1/i。总成本含外循环 n 次及 nΣX_i，所以

$$E[T] = \Theta\!\left(n + n\sum_{i=1}^{n}\frac{1}{i}\right) = \Theta(n\log n).$$

n=4 时，触发概率依次 1/4、1/3、1/2、1，期望触发数 H_4=25/12，内层期望工作 $4H_4$=25/3。这个数可以不是整数，它是不同运行整数成本的平均。

对 n=4，拿一条确定的随机数输出记录 `[8,5,6,7]` 来单步看，分别用于 i=4、3、2、1：

i	rand 本次给的数	余数	内层执行次数
4	8	0	4
3	5	2	0
2	6	0	4
1	7	0	4

这一条运行内层 12 次，既不是平均25/3也不是最坏 16。期望是对所有可能随机记录加权平均，不能要求每一条记录都等于它。

最坏每轮触发，$\Theta(n^2)$；最好仍需 n 轮外循环，且 i=1 必触发一次 n 内层，故 $\Theta(n)$。同一程序给出三种不同答案，各自描述一个不同问题，不能把期望当最坏保证。

若真实 rand 在 0 到 R 均匀，余 0 的数为 0,i,2i,...,floor(R/i)i，共 floor(R/i)+1 个，因此准确概率

$$p_i = \frac{\lfloor R/i \rfloor + 1}{R+1}.$$

它一般不恰等于 1/i。如果 R+1≥n，每轮误差至多约 1/(R+1)，合计内层期望误差 $O(n^2/(R+1))=O(n)$，不改变 n log n 主阶。如果固定 R 而让 n 无限增长，i>R 时触发只对应随机数 0，概率固定 1/(R+1)，累积可能带来平方主项，不能继续引用同一理想渐近结论。

32.17 练习与答案

练习 1。 一个函数返回 n^8，能否仅由这个式子推出它至少需要 n^8步？

解答：不能。在固定字或抽象无限字模型中，连续常数次乘法即可计算 n^2、n^4、n^8。位模型下要考虑输出位数与乘法位成本，但输出数值 n^8本身不等于执行次数。第 12 段正是在防止把返回值套进时间结论。

练习 2。 第 13 段若预先用 O(1)表查得到每个 ceil√i，外循环时间还为 Θ(n)吗？

解答：单看已准备好的表且查询常数成本，循环轮数为 $\Theta(n^{2/3})$，每轮条件成本变常数，总循环成本也变成该阶。但预处理表与内存成本要另算，不能拿修改后的程序成本冒充原程序。题目中昂贵 G 调用的存在是结论的一部分。

练习 3。 第 16 段证明期望时，为什么没用各次随机分支独立？

解答：只求期望总和，线性期望永远成立，知道每轮触发概率就够。若求“所有分支都触发”的概率，或集中界，则可能需要独立性或其他相关条件。每份概率证明只使用它真正需要的假设，既不能少写，也不用额外乱加。

33 教材、习题解析、PS 与 PA，怎样对着解决同一个问题

遇到一段看不懂的证明，不必在同一页上反复硬读。先判断缺的是定义、具体执行、证明方法还是代码约定，再去对应材料补。教材给完整概念，课件常保留课堂推导的图与中间步骤，PS 提出容易混淆的条件，PA 则要求你把它们落实成在规模内可运行的程序。

课程名是数据结构。教材书名、旧版章号、课程课件节号与本书章号各有一套，不能拿某套号直接代替另一套。尤其 2026 教材目前可读部分重点覆盖算法概论到 AVL 的前六章，整套课件继续提供图、散列、堆、串等后续主题；旧版第三版与习题解析的章序又不同。

33.1 先知道自己翻的是哪一份书

本地 2026 教材为 352 页 PDF；作者官方合体 PDF 为 669 页，含第三版主教材与《习题解析》，习题解析从 PDF 第 413 页开始。官方入口是作者教材页，公开原件本地保存在`资料/教材与参考/数据结构（第 3 版）与习题解析（官方合体）.pdf`。

一份 PDF 中印刷页可能从前言重新编号，习题解析又从自己的页码开始。因此引用“第 24 页”不够；本书 32 章明确说“官方合体 PDF446 页、习题解析印刷 24 页”，两者定位到同一位置。搜索提取文字能帮助定位名词，涉及树形、箭头、求和上下限与分式时，仍应看对应原页。

下面按官方书签列出旧版习题解析的 292 道大题。许多题还有小问；它们与 PS01—05 的 184 个编号不是同一套，题量不能相加后就宣称全部完成。

旧版习题范围	大题数	官方 PDF 页	本书对应
1-1 至 1-32，绪论	32	424 至 456	01—03、22、28、31；32 专讲 1-32
2-1 至 2-41，向量	41	458 至 493	04—06、23
3-1 至 3-19，列表	19	496 至 508	06—07、24
4-1 至 4-26，栈与队列	26	510 至 524	08—09、25、30
5-1 至 5-30，二叉树	30	526 至 540	10—11、26
6-1 至 6-32，图	32	542 至 565	16—17
7-1 至 7-20，搜索树	20	568 至 578	12
8-1 至 8-20，高级搜索树	20	580 至 599	13—14
9-1 至 9-26，词典	26	602 至 615	15
10-1 至 10-22，优先级队列	22	618 至 632	11、18
11-1 至 11-10，串	10	634 至 640	19
12-1 至 12-14，排序	14	642 至 650	20

33.2 路线 1：复杂度题，先把“要数什么”固定

先读 01 章的计算模型，明确一条移位、大整数乘法和数组访问是否算常数。再读 02 章的三种和式与递归树，亲自写出每行次数。随后做 PS01：给程序一个小 n，列出循环状态，再写一般阈值。遇到 1-32，32 章按 16 段分别区分返回值、外层轮数和总调用成本。

例如第 13 段 G 算 ceil√i，主循环走到 i 约$n^{2/3}$。若只看外循环轮数，会答错；把每次条件调用的√i 成本求和，才得到线性。看答案之前要问：“这里哪一句是在数轮数，哪一句是在数工作？”

到 PA 时再将复杂度与规模核对。n=40 的二选一组合不能轻易做2^{40}枚举，可考虑两半；字符串长度很大但预算 k 小，可考虑带状 DP。理论阶是选择算法的约束，不是写完代码后用来装饰的结尾。

33.3 路线 2：向量与列表，同一个插入动作付不同的账

读向量课件原图时，追踪容量、size 与复制量，不要只记“扩容摊还常数”。04 章给纯追加序列，28 章再把势能总账写成等式。然后用 PS02 检查不同增长策略、成功失败查找长度和二分接口。

转到列表，先在纸上画前驱、当前节点、后继，写出四根链接。07 章删除例子专门保存旧 next，防止释放后使用。PS03 再检查稳定性与比较次数：列表节点摘接便宜，不等于在有序列表中寻找插入位置也便宜。

若程序输出排序值正确，仍应给相等项加 A、B 标签检查稳定性。选择“最后一个最大值”后远距离交换，可能让另一种相等键反序；06 章给了可手算的反例。用具体记录比“这个算法通常稳定吗”的印象判断可靠。

33.4 路线 3：栈队列，先写状态再谈峰值

08 章先手算括号和 RPN，再读老师中缀优先表。课程原表的乘方左结合与数学常用右结合应分开。PS04 有些题要求原程序，有些要求修改程序，不能把修改后的规则倒灌回原版。30 章对 21 式列出每次数字压入与每次运算后的完整数值栈，便于定位哪次减法或除法先算错。

09 章用窗口最大值和直方图说明删除候选的理由。之后读 PS04 里“单步空间”“整段峰值”“随机期望”的问题，再去 28 章看记录变量与并合界。固定一个窗口的平均候选数小，不代表无限长流的最大候选数同样小。

PA 中不一定直接出现栈，但同样需要问“哪些旧状态仍可能影响未来”。Filename 保留带内可达状态，Gift 保留有重复身份的半和，偏序 DP 保留每个 b 前缀的最佳值。这些压缩都应给出不会丢掉更优解的理由。

33.5 路线 4：树，把所有计数约定写在第一张纸上

先固定高度按边数还是节点数，空树高度取多少，二叉树是否区分左右，是否允许单孩子。10 章统一按边数，给增强先序解析；PS05 再检查普通遍历与带空标记遍历的唯一性。

扩充二叉树是在每个缺失孩子处补一个外部空叶。n 个原节点有 n+1 个空槽，所以补后外叶 n+1，所有原节点都成为双分支节点，共 n 个；不是只给原单孩子节点补一次。原叶的左右两个缺口也必须补。这个约定与增强先序中 n+1 个空孩子标记是同一份结构信息的两种写法。

BST、AVL 先用 12 章小树执行删除与旋转，再回到教材的通用节点重构；不要只看最终树形。Huffman 先用 11 章算合并权重总和，再读贪心交换证明和比较下界。最优编码的成本、构造算法的复杂度、输出所有码字的长度，是三个不同的量。

33.6 原书十二章，各完整走一类代表题

以下题号都来自旧版《习题解析》，定位均为官方 669 页合体 PDF 的物理页。它们让你看到题目与本书的具体接点，并不代替原书所有题。题意采用转述，算例另设。

33.6.1 绪论 1–1，PDF424 页：尺规算法也要证明输出关系

任务是在直线 l 外给定点 P，作过 P 的平行线。取 l 上一点 A，以 AP 为半径作圆，与 l 交于 B；再分别以 B、P 为圆心、同半径作圆，两圆除 A 外的另一交点为 C，连接 PC。

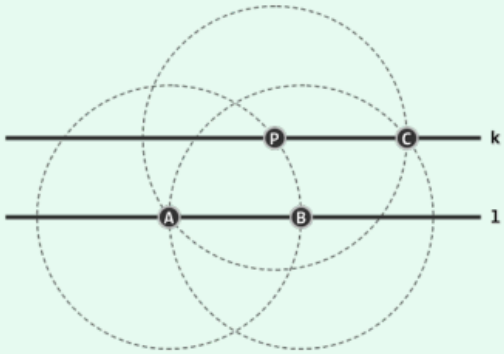
[1-1] 试借助基本的几何作图操作描述一个算法过程，实现“过直线外一点作其平行线”的功能。

【解答】

算法x1.1[①]给出了一个可行的算法，其原理及操作过程如图x1.1所示。

```
parallel(l, P)
输入：直线l及其外一点P
输出：经过P且平行于l的直线
1. 以l上任意一点A为中心、以|AP|为半径作圆
2. 在该圆与l的两个交点中，任取其一记作B
3. 分别以B和P为中心、以|AP|为半径各作一圆
4. 两圆相交于A及另一点，将后者记作C
5. 过P和C绘制一条直线k
```

算法x1.1 过直线外一点作其平行线



图x1.1 过直线外一点作其平行线

[1-2] 《海岛算经》讨论了如下遥测海岛高度的问题：

> **今有望海岛，立两表，齐高三丈，前后相去千步，令后表与前表参相直。从前表却行一百二十三步，人目著地取望岛峰，与表末参合。从后表却行一百二十七步，人目著地取望岛峰，亦与表末参合。问岛高及去表各几何？**

刘徽所给出的解法是：

> **以表高乘表间为实；相多为法，除之。所得加表高，即得岛高。**
> **求前表去岛远近者，以前表却行乘表间为实。相多为法。除之，得岛去表数。**

a） 该算法的原理是什么？

【解答】

刘徽通过设立两根立柱（表），利用光线直线传播的性质，根据海岛峰顶经过立柱顶端后到地面的投影位置，巧妙地计算出远方海岛的高度。

具体地如图x1.2所示，若将立柱高度（表高）记作h，后立柱、前立柱间距（表间）记作d，对应的投影距离分别记作d_1和d_2，则不难导出：

$$\text{海岛高度}H = d \times h / (d_1 - d_2) + h$$
$$= 1000 \times 3 / (127 - 123) + 3$$
$$= 753 \text{（丈）}$$
$$\text{海岛距离}D = d \times d_2 / (d_1 - d_2)$$

2

① 为与主教材相互区别，本习题解析中所有插图、表格、算法、代码等编号前，均增加“x”标识

图 209 官方合体 PDF 第 424 页完整原页。图中三个同半径的圆分别把 AP 的长度传到 AB、BC、PC，最后连 PC；原页下半另有海岛题，本节只解 1-1。

按图一步步核对，而不是看到最终平行线就结束：第一圆以 A 为圆心，P 在圆上；B 取这圆与 l 的交点，所以 AB=AP。第二、三圆分别以 B 和 P 为圆心、以同一个半径 AP 作图；A 在这两圆上，因此取两圆交点时要明确取另一个 C，而不是又取回 A。于是 BC=PC=AP。最后 P 与 C 不同，可以连成直线。

P 不在 l 上使 A、B、P 不共线，两个相交圆不是相切于 A 的一种退化情形，因此另一个交点存在。输入条件保证了操作本身可执行。

为什么 PC 平行 l？AB=AP，且 BC=PC=AP，所以 A、B、C、P 依次构成菱形，PC 与 AB 平行，AB 又在 l 上。P 不在 l 上保证相关三角形不退化。算法的基本操作是定半径作圆、取交点、连直线，并非必须先写成 C++才叫算法。01 章说明输入条件与基本操作模型，尺规题同样要交代这些前提。

33.6.2 向量 2–2，PDF458 页：重叠复制怎样选方向

源区间与目标可能重叠时，按地址判断。数组[A,B,C,D,E]将前三项复制到从下标 1 起的位置，目标在源右侧，必须倒着：先 C 到 3，再 B 到 2，再 A 到 1，结果[A,A,B,C,E]。若正向，A 先覆盖 B，后面会不断复制 A。

右移的每一格如下，尚未读出的源用原大写字母保持：

```
初始        [A,B,C,D,E]
先a[3]=a[2] [A,B,C,C,E]
再a[2]=a[1] [A,B,B,C,E]
后a[1]=a[0] [A,A,B,C,E]
```

若反过来先写 a[1]=a[0]，源 B 被覆盖；下次读取 a[1]得到 A，错误已不能靠最后一行补救。判断方向时画的是地址区间，不是比较元素值大小。

反过来将下标 1 到 3 的 B,C,D 复制到 0 到 2，目标在左，正向得到[B,C,D,D,E]。若倒着，先 D 覆盖下标 2 的 C，再向前读时已丢原 C。一般规则是右移重叠区从后向前，左移重叠区从前向后；不重叠时两向都可。04 章的插入和删除正是在使用这条覆盖依赖关系。

33.6.3 列表 3–3，PDF496 页：唯一数据节点删除以后是什么

双哨兵链表 H↔X↔T，X 既是首数据节点也是末数据节点。先保存 p=X.pred=H、q=X.succ=T；令 p.succ=q、q.pred=p，再释放 X，size 从 1 变 0。得到 H↔T，恰好是空表的不变式。

对两条真正的接线分别画中间状态：先写 H.succ=T，正向已经绕过 X，但 T.pred 暂时还指 X；再写 T.pred=H，双向关系恢复。到这时才销毁 X。循环或回调若可能在两行之间观察表，应按并发或重入协议处理；本题普通单线程删除把这几行作为一个局部修改完成。

```
删前：H <-> X <-> T，size=1
保存：left=H，right=T
接好：H <--------> T，size=0
释放：X不再拥有或连接后续节点
```

如果 X 析构会自动删除 succ，则本来独立的尾哨兵可能被误删；课程节点只按其约定释放自己的 data 与存储，容器统一拥有链，不能把两种所有权策略混用。

没有特殊分支仍能成立，是因为H、T虽然不属于用户数据，却提供合法邻居。不能将X删除后再读X.succ，也不能把 H 或 T 当作可删除数据。07 章的同一套 remove 对中间、首尾与单节点情形统一适用，前提是哨兵确实存在且不被当作数据。

33.6.4 栈与队列 4–1，PDF510 页：用列表实现栈，接口不止换个名字

令列表末端为栈顶：push(e)调用 insertAsLast(e)，pop 调用 remove(last())，top 返回 last.data；size 与 empty 沿用列表管理信息。双向列表有尾哨兵时，取末节点与在末端插删都是 O(1)，因此标准栈接口都可最坏常数。

空栈的 pop、top 必须拒绝或返回失败，不能把尾哨兵的数据当结果。若还继承了 find 接口，应核对失败返回：列表返回空位置，向量可能返回负秩，调用代码必须相应修改。原页代码用 last 与 insertAsLast，旁边个别“首元素”注释应按实际调用理解为末端；不能把互相冲突的注释当作两套同时成立的方向。

用链画四个接口的实际变化，H、T 为哨兵：

```
empty: H↔T, size=0, true
push A: H↔A↔T, size=1
push B: H↔A↔B↔T, size=2, top读T.pred即B
pop: 保存B.data, 令A.succ=T、T.pred=A, 释放B, size=1, 返回B
```

每次末端读 T.pred、插接或摘接都是固定条数，因而最坏常数。单链即使有 tail 指针，也没有 B 的前驱 A，尾删可能先扫描全链；原题基于有前后链接的 List，复杂度依据不能漏掉这一表示条件。

具体执行 push A、push B、pop 得到 B，剩 A，top 仍 A。若换成首端做栈顶也可，但遍历显示顺序会反过来，使用者若依赖“底到顶”的输出，需统一约定。08 章强调的是后进先出关系，而非底层一定用数组。

33.6.5 二叉树 5–1，PDF526 页：深度加高度为何不超过整树高

取任意节点 v，从根到 v 长 depth(v)，再取 v 子树里最深叶 x，v 到 x 长 height(v)。两段接起来是整树中一条根叶路径，所以 depth(v)+height(v)=depth(x)≤height(T)。若 v 子树包含至少一个整树最深叶，便取等；若不包含，子树内所有叶都更浅，不等号严格。

符号可以直接对应到一张树：

```
        R              depth(R)=0, height(R)=3
       / \
      A   Q            A深1高2, Q深1高0
     /
    B                  B深2高1
   /
  C                    C深3高0
```

取 v=A，子树里最深的是 C，根到 A 的 1 条边接上 A 到 C 的 2 条边，得到 3；取 v=Q 只能得到 1。一般式子里，max 只是“在 v 子树里挑最深后代”的操作，换成全树挑，候选更多，所以不会更小。

例如根 R 左边有三条边长链，右孩子 Q 是叶。整树高 3，Q 深 1、高 0，相加 1<3；左链上的节点均位于某条最深路径，相加 3。证明不要求树平衡，也不要求最深叶唯一。10 章统一的按边计高约定可直接用于这道题。

33.6.6 图 6–1，PDF542 页：关联矩阵乘自己转置，究竟保留了什么

无向链 A-B-C，两条边分别为列，关联矩阵为 `I=[[1,0],[1,1],[0,1]]`。两行点积共同计数两顶点关联的边，故 `I I^T=[[1,1,0],[1,2,1],[0,1,1]]`。对角是度 1,2,1，非对角为相邻边数；简单无环图下，非对角就是邻接矩阵。

先解释矩阵相乘怎样读取。I 的行是顶点，列是边 e1=AB、e2=BC：

$$I = \begin{pmatrix} 1 & 0 \\ 1 & 1 \\ 0 & 1 \end{pmatrix}.$$

乘II^T的第 A 行第 B 列，就是 I 的 A 行与 B 行的点积：$1\cdot 1 + 0\cdot 1 = 1$，共同碰到 1 条边。A 与 C 点积$1\cdot 0 + 0\cdot 1 = 0$，不相邻。B 与自己点积$1^2 + 1^2 = 2$，是度 2。每一格都来自这个规则，得到

$$II^T = \begin{pmatrix} 1 & 1 & 0 \\ 1 & 2 & 1 \\ 0 & 1 & 1 \end{pmatrix}.$$

有平行边时非对角可能大于 1，表示公共关联边的条数；本例没有自环。

有向链 A→B→C，把出发端记 −1、到达端记 +1，则 `I=[[-1,0],[1,-1],[0,1]]`，乘积为 `[[1,-1,0],[-1,2,-1],[0,-1,1]]`。它记录总度与忽略方向的连接数，却无法恢复边方向，因为将某列整体取反不改变乘积。别误把这个对称乘积叫作原有向邻接矩阵。

把 A⟶B 方向反过来，I 的第 1 列由`[-1,1,0]`改成`[1,-1,0]`，整列取负。任意两行在这一列相乘都同时多两个负号，贡献不变，所以乘积无法区分 A⟶B 与 B⟶A。对角度数与连接关系能保留，原方向却丢了；这个信息损失可以由一条边的实际变换直接证明。

应用例子是差分约束 x_v−x_u≤w，转成边 u⟶v 权 w。给各点一个超级源 0 权到达边，最短距离满足所有`d[v]≤d[u]+w`，可作可行赋值；若存在负环，将环上不等式相加会得 0≤负数，矛盾。17 章 Bellman–Ford 检测负环因此能处理这类可行性问题。另做一个完整可行性例子：

$$x_2 - x_1 \le 3, \qquad x_3 - x_2 \le -1, \qquad x_3 - x_1 \le 4.$$

转成 1⟶2 权 3、2⟶3 权 −1、1⟶3 权 4，并从超级源向三个点各连 0 边。初始 d1=d2=d3=0；扫 1⟶2 时 0≤0+3，不改；扫 2⟶3 时 0>0−1，把 d3 改 −1；扫 1⟶3 时 −1≤4，不改。下一轮全部不改，取赋值$(x_1, x_2, x_3) = (0, 0, -1)$，三项左侧依次 0、−1、−1，都满足对应上界。

若再加入$x_1 - x_3 \le -3$，沿 1⟶2⟶3⟶1 的环权和为 3−1−3=−1。把三个约束直接相加，变量全部抵消，得到$0 \le -1$，不可能。因此负环不是算法随意定的失败信号，它对应原约束之间的矛盾。

矩阵方向编码负责表达约束，乘积本身并没有保留全部有向信息。

33.6.7 搜索树 7–1，PDF568 页：局部孩子大小还不够

BST 要求左子树所有后代≤根，右子树所有后代≥根。中序按左、根、右列出，因此非降。反过来若中序非降，对任意根，所有左子树项都出现在根前且不大于根，所有右项都在根后且不小于根，故满足这种允许重复的 BST 定义。

仅检查孩子会漏错：

```
      10
     /
    5
     \
      12
```

根的左孩子 5<10，5 的右孩子 12>5，每条边局部都像 BST，但 12 属于 10 的整个左子树，违反≤10。中序实际输出 5、12、10，在 12⟶10 处下降。

检查时可以把祖先累积的范围传下去：根 10 允许任意实数；左孩子 5 继承“≤10”；5 的右孩子继承“≥5”还必须保留原来的“≤10”，所以允许范围为[5,10]，12 立即被拒绝。若递归时只传当前父亲限制，忘掉更高祖先的上界，正好会漏掉本例。12 章先用互异键得严格递增；原书这题允许非严格关系，重复键的策略需另加约定。

33.6.8 高级搜索树 8–1，PDF580 页：重复记录需要身份，整组查询需要边界

原题要伸展树允许多个相等数据项，并增加 searchAll 与 removeAll。可把内部比较键设成`(用户 key,唯一递增 id)`，按字典序严格比较；旋转只依赖这个总序，不会把相等用户键随机挤到不相干位置。对外查找仍只看用户 key，所有相等 key 在中序中连续。

依次插`(5,A,id1)`、`(3,B,id2)`、`(5,C,id3)`，每次把新项伸展到根。三次状态可以全部画出：

```
插A后:    5A        插B并右旋后:  3B        插C并两次左旋后:  5C
                                  \                          /
                                   5A                       5A
                                                           /
                                                          3B
```

内部次序始终`3B < 5A < 5C`。查询第一个用户 key 不小于 5，根 5C 是候选，向左 5A 也是候选，再向左 3B 太小、其右边空，故最左候选是 5A。把 5A 绕 5C 右旋到根：

```
      5A
     /  \
    3B   5C
```

一次 search(5)可返回 A，单项 remove(5)也只删除找到的一份，C 仍应存在。删除全部不能仅删根，也不能用一个布尔“5 存在”代表两条记录。

批量接口可以用**两次直接分裂**得到$O(\log n + k)$摊还界。分裂不仅要把等值范围找对，还要真正伸展走完搜索的节点；只旋转一个很早记录的候选，可能遗漏长搜索路径的成本。

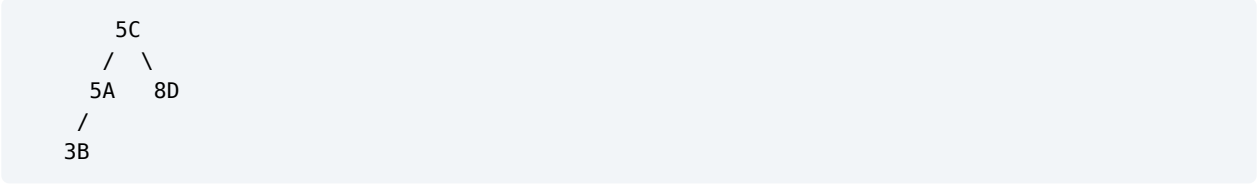
第一次分成 L（用户 key<x）与 R（用户 key≥x），按以下完整规则执行：

1. 从根搜索到空指针，不在命中等值时提前停。当前 key<x 就向右走，否则向左走；每走到真实节点，都把它记为 last。
2. 空树直接得到两个空树。非空树把**实际最后访问的 last** 伸展到根，正常修正每次旋转的 parent、孩子和 size。
3. 若新根 key<x，它就是小于 x 的最后一项。根及其左边归 L，根的右子树归 R：断 `root.right`。
4. 若新根 key≥x，它就是第一个不小于 x 的项。根及其右边归 R，根的左子树归 L：断 `root.left`。

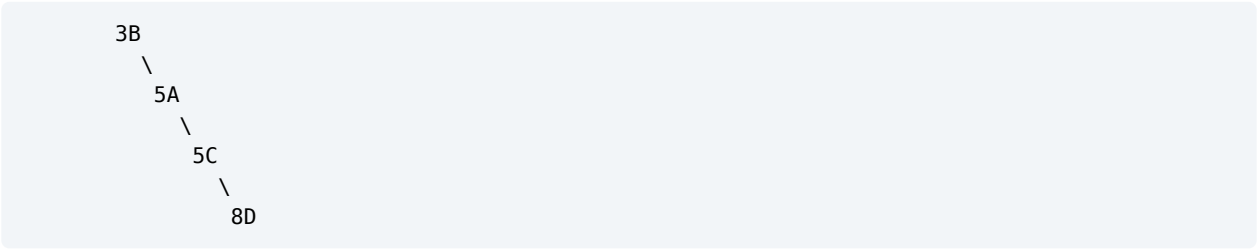
为什么 last 只能是边界两侧紧邻的项？搜索要么在“<x”的节点向右走到空，说明它与 x 之间已没有更大但仍<x 的节点；要么在“≥x”的节点向左走到空，说明没有更小但仍≥x 的节点。祖先给出的另一侧限制也一直保留。伸展保持中序，所以旋转后按根的一条边切开，仍恰好把整个全序切在 x 之前。

第二次把 R 分为 M（用户 key≤x）与 G（用户 key>x）。搜索规则相应改为：key≤x 向右，否则向左，仍一路到空、伸展 last。新根若≤x，断 `root.right`，左份为 M、右份为 G；新根若>x，断 `root.left`，左份为 M、右份为 G。R 原本全≥x，所以 M 中的项恰好全等于 x。每次断边后，把新分离的根 parent 置空，重算仍保留孩子的原根 size；若某份为空，直接保留为空。

把两边都非空的例子走全。查询 x=5，初态为一棵合法伸展树：

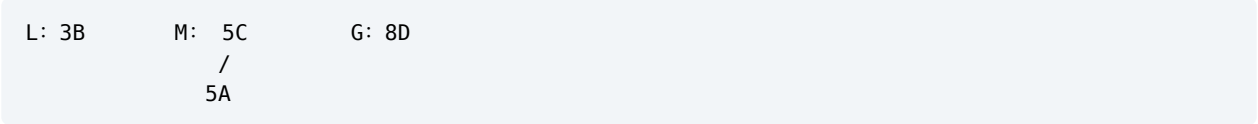


第一搜索依次访问 5C、5A、3B，再从 3B 的右边走到空，last 是 3B。3B 是左−左孙，先绕 5C 右旋得到 5A 为根、左 3B、右 5C（5C 右 8D）；再绕 5A 右旋，得到：



新根 3<5，断它的**右边**，得到 L 仅 3B，R 为 5A 右连 5C、5C 右连 8D。第一步并非找到 5A 就停，3B 的访问也被实际伸展覆盖了。

第二搜索在 R 中依次访问 5A、5C、8D，从 8D 左边到空，last 是 8D。它是右−右孙，连续两次左旋：先绕 5A，得到根 5C、左 5A、右 8D；再绕 5C，得到根 8D、左孩子 5C、5C 左孩子 5A。新根 8>5，断它的**左边**：



searchAll 中序遍历 M，得到 A、C 共 k=2 项，再合回 L、M、G；removeAll 销毁 M 的两个节点，再合 L 与 G，删除结果为 3B 右连 8D。若没有等值项，M 为空；若全表都小于 x，第一次新根是全表最大项、右边本来空，R 自然为空，不需要读取不存在的候选。

一个能检查复杂度口径的反例是：根 10，左边为 1→2→…→9 的右链，查询 x=9.5。第一个≥x 的候选一直是根 10，但真实搜索会走到 9。如果只伸展候选 10，树完全不变，反复查询每次都线性。这里会伸展实际 last=9，再按 9<9.5 断右边，整条访问路径都进入正常伸展分析。

合并两棵所有内部键满足左<右的伸展树，可把左树最大项伸展到左根，此时没有右孩子，再把右树接到其右边并修 parent、size。若其中一棵为空，直接返回另一棵。刚才 searchAll 后的合回也可执行：先将 M 接在 L 的 3B 右边，再把合树最大项 5C 伸展到根，此时左边是 3B、3B 右孩子为 5A，最后把 8D 接到 5C 右边。中序仍是 3B、5A、5C、8D；查询可以改变形状，不能改变记录内容与总序。每次分裂都搜索到 last 并将 last 伸展，搜索长度与这次伸展路径相配；合并也在找到左树最大项后将它伸展。各自按访问引理摊还$O(\log n)$，切掉一条边不会增加按子树大小计算的森林总势能；中间树遍历或销毁需$\Theta(k)$，所以总$O(\log n + k)$摊还时间、输出/删除量另计。k=0 时也要处理空 M。单次伸展可能很长，不能把摊还结论改成单次最坏对数。

仅仅说“找到首项后不断求后继”还不足以自动保证这个批量界：如果每次后继从根重查，可能多出 k 个对数；如果结束边界要走一条很深的非等值链，也需要分析那段代价。两次分裂明确圈出全部输出的子树，扫描成本才可以直接按 k 记账。第 13 章给伸展旋转与访问引理，本节把它们接到重复记录这个具体接口。

33.6.9 词典 9–2，PDF602 页：一座塔不要重复复制整个词条

跳转表某键高 4 层，若每层都复制一份长字符串和值对象，会浪费 4 份大载荷。可让底层数据节点唯一拥有 payload，高层索引只存指向同一 payload 或底层节点的引用及横纵链接。

例如某条记录 P 的 key 为 7、value 是一段长文本，塔高 3。把载荷只留一份：

```
第2层:  ... ↔ index7 ----payload----> P(key=7,value=长文本)
                 |                    ↑
第1层:  ... ↔ index7 ----payload---------+
                 |                    ↑
第0层:  ... ↔ data7  ----payload---------+
```

横向链接负责这一层前后跳，纵向链接负责上下，payload 引用只负责找到那一份记录。更新 value 把 P.value 改一次，不复制三份；key 通常不能直接原地修改，否则横向有序性会被破坏，应删后重插。

删除时先在每层把索引前后邻居接好，拆掉塔，再释放唯一载荷 P 一次；若先删 P，高层比较 key 时就可能读到悬空引用。若接口允许外部长期持有记录，则还要用共享所有权或稳定句柄规定什么时候可释放，塔高本身没有解决外部生命期。

查询在高层停止时，沿引用返回那一份数据；更新值只改一次，所有层看见同一记录。删除时先拆掉整座塔的索引链接，再释放唯一 payload，避免其他层留下悬空引用。若外部还持有引用，要由接口另行约定生命周期。链接数期望 O(n)的前提不变，但每层不再重复存可能很大的键和值；13 章的索引快车道因此与对象所有权接起来。

33.6.10 优先级队列 10–1，PDF618 页：四个基础实现，把三个接口都算一遍

无序向量尾部插入摊还 O(1)，找最大 O(n)，删最大可扫描后用末项填洞，总 O(n)。无序列表可常数插节点，找最大与删前寻找仍 O(n)。若缓存最大位置，getMax 可 O(1)，但删最大后重找下一最大仍需 O(n)，缓存没有免除这笔工作。

用插 4、1、7、6，再删最大，实际动作如下：

结构	插完前 3 项	插 6 时做什么	取得/删除 7 时做什么
无序向量	`[4,1,7]`	尾写 6，成`[4,1,7,6]`	扫到下标 2 的 7，用末项 6 填洞，成`[4,1,6]`
有序向量	`[1,4,7]`	把 7 右移，6 填进去，成`[1,4,6,7]`	直接读/删末项，成`[1,4,6]`
无序双链	`4↔1↔7`	尾接 6	扫出 7 的节点，常数摘链
有序双链	`1↔4↔7`	先走链接找到 4 与 7 之间，再常数接 6	尾指针直接读/摘 7

无序的代价落在寻找最大，有序的代价落在寻找并保持插入位置。链表“已知位置后插删常数”没有免除寻找位置。

有序升序向量在正确秩插入需寻找并搬移，O(n)，最大在尾，getMax 与尾部删除 O(1)或按收缩策略给摊还界。有序双向列表插入需 O(n)寻找位置，末端取最大与删末节点 O(1)。保持全序使删除极值便宜，却使插入较贵。18 章的完全二叉堆只维护父子序，getMax O(1)，插入与删最大 O(log n)，正是放松全序换来的平衡。

33.6.11 串 11–1，PDF634 页：前缀、后缀与子串的坐标

以 S=ABCDE 为例，长度为 5，长 2 前缀 AB，长 2 后缀 DE。按原题 Excel 式从 1 编号，mid(S,2,2)取 BC；换成本书从 0 起的区间，就是 S[1:3]。相等比较要求长度相同且每个对应字符相同，ABC 与 AbC 在区分大小写的 exact 语义下不同。

把同一个 S 从两种坐标写出来：

```
字符       A B C D E
Excel位置  1 2 3 4 5
本书下标   0 1 2 3 4
```

合法参数下，`mid(S,i,k)`的本书半开区间是`[i-1,i-1+k)`。例如 i=2、k=2，读下标 1、2 的 B、C，右端 3 不读；`left(S,2)`是`[0,2)`，`right(S,2)`是`[3,5)`。三个参数分别是起点、长度与排除的右边界，不能把 k 直接当结束下标。这里用 ASCII 与合法范围说明映射；不同工具的非法参数、Unicode 计数细节还需按其各自接口规则核对，不能把这张坐标表当作所有软件错误处理也完全相同。

这道看似简单的接口题在提醒你：课件的秩、语言切片与工具界面起点可能不同。字符长度也要看采用字节、码点还是其他单位；不能拿一个软件的显示长度直接当另一个 API 的字节偏移。19 章先固定字符模型，再定义 π 与匹配起点，正是为了防止这类坐标错位。

33.6.12 排序 12–1，PDF642 页：一个扫描指针也能完成轴点划分

原题版本 C 把轴值先放 lo，mi 指向已知小于轴的区域末尾，k 从 lo+1 向右扫。遇 a[k]<pivot，就先 mi 加 1，再交换 a[mi]与 a[k]；否则只前进 k。最后交换 a[lo]与 a[mi]，返回 mi。

习题[12-1] 第12章 排序

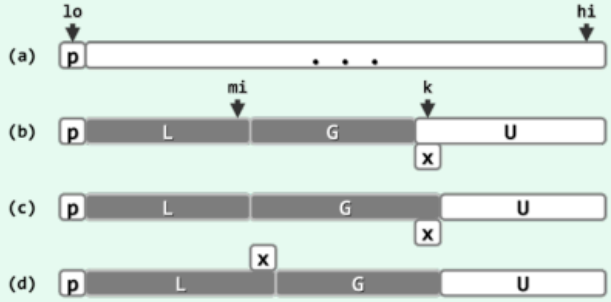
[12-1] 构造轴点的另一更为快捷的策略，思路如图 x12.1 所示：

始终将整个向量 V[lo, hi]划分为四个区间：

V[lo]，L = V(lo, mi]，G = V(mi, k)，U = V[k, hi]

其中 V[lo]为候选轴点，L/G 中的元素均不大/不小于 V[lo]，U 中元素的大小未知

初始时取 k - 1 = mi = lo，L 和 G 均为空；此后随着 k 不断递增，逐一检查元素 V[k]，并根据 V[k]相对于候选轴点的大小，相应地扩展区间 L（图(d)）或区间 G（图(c)），同时压缩区间 U。最终当 k - 1 = hi 时，U 不含任何元素，于是只需将候选轴点放至 V[mi]，即成为真正的轴点。



图x12.1 轴点构造算法（版本C）

a） 试依此思路，实现对应的划分算法 Vector::partition()；

【解答】

一种可行的实现方式，如代码x12.1所示。

```
template <typename T> //轴点构造算法：通过调整元素位置构造区间[lo, hi]的轴点，并返回其秩
Rank Vector<T>::partition ( Rank lo, Rank hi ) { //版本C
   swap ( _elem[lo], _elem[lo + rand() % ( hi - lo + 1 ) ] ); //任选一个元素与首元素交换
   T pivot = _elem[lo]; //以首元素为候选轴点——经以上交换，等效于随机选取
   int mi = lo;
   //++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++
   //   [ ---- < [lo] ----- ] [ ----- [lo] <= --- ] [ ----- unknown ----- ]
   // X x . . . . . . . . . x . . . . . . . . . . x . . . . . . . . . . x
   // |                     |                     |                     |
   // lo (pivot)            mi                    k                     hi
   //++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++
   for ( int k = lo + 1; k <= hi; k++ ) //自左向右扫描
      if ( _elem[k] < pivot ) //若当前元素_elem[k]小于pivot，则
         swap ( _elem[++mi], _elem[k] ); //将_elem[k]交换至原mi之后，使L子序列向右扩展
   //++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++
   //   [ --------- < [lo] ---------- ] [ ----------- [lo] <= ----------- ]
```


图 210　官方合体 PDF 第 642 页完整原图。四块依次是暂存轴值、小于轴区、不小于轴区、尚未知区；图下给出版本 C 代码。这里使用原题闭区间，最后下标 hi 参与扫描。

本段遵循原题闭区间[lo,hi]。若数组长度 n，整表调用应传 hi=n−1，不能传 n。为专看分区过程，下面固定首项作轴；原题随机选轴只是先交换一个位置到首项，不改变扫描不变量。

```
def partition_closed(a, lo, hi):
    pivot = a[lo]
    mi = lo
    for k in range(lo + 1, hi + 1):
        if a[k] < pivot:
            mi += 1
            a[mi], a[k] = a[k], a[mi]
    a[lo], a[mi] = a[mi], a[lo]
    return mi
```

前置条件是 0<=lo<=hi<len(a)；空区间由外层快排的基础分支直接跳过，不应调用本函数去读 a[lo]。
以[4,7,2,5,1]、lo=0、轴 4 为例：7 不动；2 小，mi=1，交换得[4,2,7,5,1]；5 不动；1 小，mi=2，得[4,2,1,5,7]；最后 4 与下标 2 交换，得[1,2,4,5,7]，轴点秩 2。扫描中[lo+1,mi]严格小于轴，[mi+1,k)不小于轴，未知段从 k 开始；每步都缩小未知段，扫描线性。
用竖线把每轮的四块画出来，轴一直单独放首格：

```
初始: 4 | 空 | 空       | 7,2,5,1       mi=0, k=1
看7: 4 | 空 | 7        | 2,5,1         7>=4, k前进
看2: 4 | 2  | 7        | 5,1           mi=1, 交换2与7
看5: 4 | 2  | 7,5      | 1             5>=4, 只扩G
看1: 4 | 2,1| 5,7      | 空            mi=2, 1与7交换
末换: 1,2 | 4 | 5,7                    返回mi=2
```

第三块内部不需要有序，7,5 在扫描中完全合法；划分只承诺相对轴的大小。每步要么把当前小项换到 L 末端，同时把原 G 首项转移到 G 末端；要么直接把当前项归 G。未知区每轮少 1，终止后轴与 mi 交换便让左侧全<轴、右侧全≥轴。后续递归是[lo,mi-1]和[mi+1,hi]，轴本人不再递归。
这里返回单个轴点，而 20 章的三路划分返回整个等值区间，递归边界不能混用。若所有值相等，版本 C 每次 mi 仍为 lo，递归侧大小 n−1，仍可能平方；随机选轴无法改变“所有值相同”这个事实，三路划分才会一次跳过全部等值项。

33.7 范围说明

这本书按现有章节讲解核心概念，按 22—26 章逐项处理本学期五张 PS，32 章专门重做原书 1-32。取得 669 页原件并建立 12 章索引，不等于 292 道原书大题及所有小问都已经逐题完整解答。查到一个题号以后，仍要看对应章节是否实际给出了该题的证明与计算。
课内样卷中带使用限制的原件留在本地，31 章用另设实例提炼方法。个人作业、转系资料和群聊照片也不因写教材而进入公开仓库。公开教材中的例子用于教学与检查，不替代课程发布的正式约束与独立提交。

资料覆盖、原图出处与使用说明

这是一份独立组织、重新讲解的教材式初稿。课程范围首先按教师资料；并非教师审核的官方教材，也不把补充证明和预习部分自动列作考试要求。

章节覆盖表

教学来源	本书对应
教材/整套讲义第 1 章算法概论	01—03、22、28、31
第 2 章向量	04—06、23
第 3 章列表	07、24
第 4 章栈与队列	08—09、25、30
第 5 章二叉树	10—11、26、29
第 6 章 BST 与 AVL	12
第 7 章搜索树应用	14
第 8 章高级搜索树	13
第 9 章词典/散列	15
第 10 章图基础	16
第 11 章图进阶	16—17
第 12 章优先级队列	18
第 13 章串	19
第 14 章排序	20
已发布 PS01—PS05	22—26 逐题导航，题数 32+43+37+35+37=184
PA1 讲解 PDF 与 PPTX	21：Filename、Interview、Gift、Graphics、二维偏序
编程/白盒指引与模板	27：规格、复杂度、边界与验证
两份本地样卷	31：只提炼通用方法，不转载课内受限原题

原生解析与视觉核对

PDF 提取原生文字与页码，PPT/PPTX 提取文字和内嵌图片，同时原生兼容导出完整幻灯片以保留箭头、布局、曲线与公式。机器已为可用课件生成完整页级联系表；写入笔记的关键原图逐幅视觉核对。未声称已对数千页的每一个细节完成逐字人工审校。PPT 导出中缺失中文字体替换为 Noto Sans CJK SC，原件不覆盖。

原生提取和视觉复核需要互相补足：抽出的字不能告诉你箭头指谁，图片也不总能准确给出公式符号。课件中的省略或不一致处，本书在相关段落说明，不把它们无条件当作定理。

原图逐项出处

- 老师原图-向量.png：数据结构/slides/分章课件/02 向量.pdf，PDF 第 29 页。直接渲染原始 PDF 页，不改图中结构。
- 老师原图-单调栈.png：数据结构/slides/分章课件/04 栈与列表.pdf，PDF 第 95 页。直接渲染原始 PDF 页，不改图中结构。
- 老师原图-Huffman.png：数据结构/slides/分章课件/05 二叉树.pdf，PDF 第 105 页。直接渲染原始 PDF 页，不改图中结构。

- 老师原图-增强遍历.png：数据结构/slides/分章课件/05 二叉树.pdf，PDF 第 87 页。直接渲染原始 PDF 页，不改图中结构。
- 老师原图-AVL.png：数据结构/slides/整套讲义/电子讲义(按章节).7z，DSA-2026F-Chapter/06.二叉搜索树.pdf，PDF 第 30 页。直接渲染原始 PDF 页，不改图中结构。
- 老师原图-KMP.png：数据结构/slides/整套讲义/电子讲义（按章节）.7z，DSA-2026F-Chapter/13.串.pdf，PDF 第 28 页。直接渲染原始 PDF 页，不改图中结构。

原创 SVG 图由工具脚本/教材/配图.py 生成，文字转为轮廓，可离线显示；它们是本书的补充示意，不伪装成教师原图。

连续步骤图演示-*.svg 由工具脚本/教材/逐步图.py 生成，含当前状态、执行动作和原因。PDF 显示每一步，网页在同一记录上提供前后步、播放、拖动时间轴和预测后揭示；关闭交互后仍可展开连续静态图。只有专门标记的自编 SVG 会启用播放器，教师原页保持完整图片。

本轮还实际对照 Sedgewick、Wayne 的归并讲解、平衡搜索树、最短路，以及 Erickson 动态规划章。参考其如何把过程显式展开，例子与中文讲解按本课程重新组织，不混用不同版本的算法约定。

版本和复核边界

2026-10-25，用新版 OneTHU 默认共享后台只读核验课程文件清单，三门课文件枚举均返回 complete=true。数据结构的综合资源接口对作业全量枚举仍给出未核实状态，因此本书不宣称覆盖了独立 OJ 未来/全部题目。随后已从作者官方入口取得 669 页主教材与《习题解析》合体 PDF；数据结构 32–33 章补充 1-32 的 16 种程序推导和全部 12 章交叉索引。取得全文不等于已经把其中 292 道大题所有小问逐一重做，具体覆盖按各章说明。

完整文字、公式与图在 Markdown 和 Typst 间共享；每次改正文后应运行构建器，Pandoc 公式转换警告将阻止成功构建。重要例题按推导、量纲、边界和独立小规模程序交叉检查，发现错误应同时修正文并重建两版本。

延伸阅读：MIT 算法课讲义。本书的章节编号与课程本地教材对应，不照搬另一门课的组织。

本版补充阅读的具体落点

33 章现在不仅列目录，还转述并独立讲解旧版习题 1-1、2-2、3-3、4-1、5-1、6-1、7-1、8-1、9-2、10-1、11-1、12-1，逐项给官方合体 PDF 物理页；相关页已结合原生文字与原页图形核对。13 章额外对照习题 8-2 的 PDF580 至 582 页，展开伸展访问引理的三种局部势能推导。32 章按 1-32 的 16 段分别给状态、阈值、求和与模型条件；这仍不表示 292 道原书大题及其全部小问已逐题解答。

30 章的 21 式从只有结果扩成每次实际运算与栈状态表，除法用精确有理数。教师原版乘方左结合与数学常见右结合现已分别说明；这 21 式没有无括号连续乘方链，数值不受该区别影响。代码中的乘方 token 使用自定义运算语义，不能直接按 C++异或解释。

原生提取用于准确检索定义和程序，视觉核对用于检查箭头、树形、上下限、缺失孩子和优先表；本版没有把生成联系表等同于已经逐页读完全部源材料。实际讲解范围以正文例题与题号为准。

读图、读程序和读公式时怎样查符号

本章用于遇到陌生写法时回查。正文第一次使用符号时仍会解释，读者不需要先背完这张表。一个字母在不同问题里可能表示不同的量，因此每道例题会重新说明它的含义。

数组格子和区间

数组 a=[8,3,6]是依次排列的三个元素。下标是位置的编号，本书从 0 开始：a[0]=8，a[1]=3，a[2]=6。长度记为n，本例$n=3$；合法下标为 0、1、2。下标 3 指向末尾之后，不能拿来读取元素。

a[lo,hi)表示从下标 lo 开始，一直取到 hi 之前；左方括号表示包含 lo，右圆括号表示不包含 hi。例如 a[1,3)取出 3、6，共两项。长度为何是$hi-lo$？这些位置依次是$lo, lo+1, \ldots, hi-1$，末项减首项再加 1，得到$(hi-1)-lo+1=hi-lo$。若$lo=hi$，没有位置可取，称为空区间。这里逗号分隔两个边界，不是两个独立数组元素。

代码里的 a[i:j]有时用来说明同一个半开区间；C++原生数组不提供这种切片语法，实际程序一般把两个边界作为参数传进去。a[i]则只指一个元素。

图中格子内是当前值，格子下方是下标。指针或游标写在格子上方。颜色只作辅助：橙色格表示本步正在检查或修改，具体改变仍会在图下写明。搬移中可能同时出现两个相同值，那是暂时复制；还在局部变量里保存的待插入值也必须一起看，不能只看数组就认定元素丢失。

赋值、比较和状态

x ← 5 读作“把 5 保存到 x”；C/C++代码写作 x = 5。它会改变状态。x == 5 是询问当前 x 是否等于 5，不进行保存。数学式$x=5$表达相等关系，需根据上下文与程序赋值区分。

程序写 x=x+1 时，先用右侧旧 x 算出结果，再覆盖左侧 x。例如旧值 4，右侧得到 5，最后 x 变为 5。这里并不是要求同一个数在数学上等于它自己加 1。为了在推导中消除歧义，可以写$x_{t+1}=x_t+1$，其中t是执行轮次，x_t是第t轮开始时的值。

状态是继续执行所必需的信息。例如插入排序进行到一半，状态至少包含数组 a、暂存值 x、正在检查的下标 j。若只画 a 而遗漏 x，就无法判断下一步要把什么填回去。不变量是某个指定时刻始终成立的陈述，例如“每轮外层循环开始时，a 的前 i 项已有序”；它不代表每条机器指令中间都保持同一句形式。

求和、乘积、取整与对数

符号$\sum_{i=1}^{4} i$读作“把 i 依次取 1、2、3、4 时得到的值加起来”，实际就是$1+2+3+4=10$。下面的 1 是起点，上面的 4 是终点，i是轮流替换的求和下标。若写$\sum_{i=1}^{4} 2i$，每一项先乘 2，得到$2+4+6+8=20$。下标不是额外相乘的一项。

$\prod_{i=1}^{4} i$使用大写希腊字母 Π 形状的乘积号，表示$1\times2\times3\times4=24$；它与求和号 Σ 不同。$n!$读作 n 的阶乘，等于$1\times2\times\cdots\times n$，约定$0!=1$。计算排列数时，“第一位 n 种、第二位 n−1 种……”相乘便得到$n!$。

$\lfloor x\rfloor$为不大于 x 的最大整数。例如$\lfloor 2.8\rfloor=2$、$\lfloor -2.8\rfloor=-3$。$\lceil x\rceil$为不小于 x 的最小整数，因此$\lceil 2.8\rceil=3$。负数向下取整与 C++整数除法朝 0 截断不同，程序中需要单独核对。

2^k表示 k 个 2 相乘；$\log_2 n$问“2 的几次方等于 n”。例如$2^3=8$，所以$\log_2 8=3$。当二分每次把候选数量减半，8 项依次变成 4、2、1，共减半 3 次，这就是对数出现的具体原因。$\lg n$在算法教材中常指$\log_2 n$，$\ln n$指以自然常数 e 为底的对数；本书在容易混淆处直接写底数。

复杂度记号比较的是什么

用$T(n)$表示输入大小为 n 时所计基本操作的数量。先确定操作模型，才知道一次比较、移动或大整数乘法各算多贵。O、Ω、Θ都是比较函数随 n 增长的速度；它们本身不指定最好、平均或最坏情况。

说$T(n)=O(n^2)$，意思是存在固定常数$c>0$和门槛n_0，使所有$n\ge n_0$都满足$T(n)\le cn^2$。例如$T(n)=3n^2+2n+7$，当$n\ge1$时，$2n\le2n^2$、$7\le7n^2$，于是$T(n)\le12n^2$。可取$c=12$、$n_0=1$。常数可以依赖具体算法，但不能随着 n 增长而增长。

说$T(n)=\Omega(n^2)$是相反方向的下界，要求充分大的 n 上$T(n)\geq cn^2$。同一个例子中$3n^2+2n+7\geq 3n^2$，所以也满足下界。上下两边都成立才写$T(n)=\Theta(n^2)$。仅给上界并没有说明它紧：这个函数也属于$O(n^3)$，但三次上界描述得较松。

最坏复杂度会先在长度 n 的所有合法输入里取最大操作数，再比较这个新函数的增长；平均复杂度则要先说输入遵循什么概率分布。均摊复杂度计算一串操作的总成本再除以操作数，不能把一个昂贵操作单独保证为常数，也不能无条件当成随机平均。

树、图与字符串中的局部记号

树的根是没有父节点的起点；孩子是从某节点向下一层直接连接的节点；后代还包括继续向下走多层的节点。叶节点没有孩子。空孩子位置表示没有节点，与键值为 0 的节点不同。树高按最长向下路径的边数计算：叶高 0、空树高−1。一个节点的子树含这个节点自身及全部后代，所以子树规模不是只数后代。

栈按后进先出取元素。本书横画时明确标“底→顶”，最右边才是可弹出的栈顶；竖画时会另标顶部。队列按先入先出，图中会标队首与队尾。哨兵是为了统一边界处理加上的辅助标记或节点，通常不算真实数据；统计栈峰值等题若把哨兵计入，则必须在题目约定里写出。

图的顶点是对象，边表示对象间的关系。有向边$u\to v$只保证从 u 到 v 可走，不能自动反向。V有时指顶点集合，有时在复杂度中简写顶点数；严谨写法$|V|$是集合中元素的个数。边集合 E 也同理。$w(u,v)$是边 u→v 的权重，不必等于两个顶点的编号差。$d[v]$在最短路算法中是当前找到的候选距离，在被证明确定前不等于最终答案。

字符串 T 是文本，P 是要寻找的模式。前缀从字符串开头开始，后缀在末尾结束；“真前缀”排除整个字符串自身。子串要求字符连续，子序列允许跳过中间字符但保持先后顺序。`abc` 的 `ac` 是子序列而非子串。KMP 保留的是已经匹配片段的前后缀关系；LCS 求的是两个串的最长公共子序列，两者回答不同问题。